Privacy-Preserving Machine Learning for Practitioners

Manuel V. Loureiro

Version 2026-10-15

Table of contents

Preface **1**
How this book is organized . . . 1

I. Prerequisites **2**
The common thread . . . 3
Why the chapters come in this order . . . 3
How to read a chapter . . . 4
What you will be able to do . . . 4

1. Probability distributions **5**
1.1. Why this matters . . . 5
1.2. Counts: binomial and Poisson . . . 5
1.3. Laplace and exponential . . . 7
1.4. Normal . . . 7
1.5. A density against its shifted copy . . . 8
1.6. Cheat sheet . . . 9
1.7. Further reading . . . 10

2. Concentration inequalities **11**
2.1. Why this matters . . . 11
2.2. Knowing only the variance: Markov and Chebyshev . . . 11
2.3. Normal and Laplace noise . . . 12
2.4. Counts: the Chernoff bound . . . 14
2.5. Averages of bounded quantities: Hoeffding and Bernstein . . . 15
2.6. Functions of many inputs: McDiarmid and Azuma . . . 16
2.7. Cheat sheet . . . 18
2.8. Further reading . . . 19

3. Divergences **21**
3.1. Why this matters . . . 21
3.2. KL divergence . . . 21
3.3. Rényi divergence and the worst case . . . 22
3.4. Total variation and the hockey-stick divergence . . . 23
3.5. Data processing and Pinsker's inequality . . . 23
3.6. Code . . . 25
3.7. Cheat sheet . . . 26
3.8. Further reading . . . 26

4. Hypothesis testing **27**
4.1. Why this matters . . . 27

4.2. Errors and the Neyman–Pearson lemma . . . 27
4.3. Trade-off curves . . . 28
4.4. One-number summaries . . . 29
4.5. Base rates: the Bayesian view . . . 30
4.6. Measuring rates from finite trials . . . 30
4.7. Code . . . 31
4.8. Cheat sheet . . . 32
4.9. Further reading . . . 33

5. Information theory **34**
5.1. Why this matters . . . 34
5.2. Entropy . . . 34
5.3. Mutual information and data processing . . . 36
5.4. Min-entropy and one-guess leakage . . . 36
5.5. Shannon against min-entropy leakage . . . 38
5.6. Fano's inequality . . . 39
5.7. Code . . . 39
5.8. Cheat sheet . . . 40
5.9. Further reading . . . 41

6. Optimization **42**
6.1. Why this matters . . . 42
6.2. Lipschitz, smooth and strongly convex losses . . . 42
6.3. SGD convergence . . . 43
6.4. Gradient noise and minibatches . . . 44
6.5. Sensitivity of regularized ERM . . . 44
6.6. Code . . . 45
6.7. Cheat sheet . . . 45
6.8. Further reading . . . 46

7. Algorithmic stability and generalization **47**
7.1. Why this matters . . . 47
7.2. The gap and uniform stability . . . 47
7.3. Regularized ERM is stable . . . 48
7.4. SGD is stable if you stop early . . . 49
7.5. Memorization and self-influence . . . 50
7.6. Code . . . 51
7.7. Cheat sheet . . . 51
7.8. Further reading . . . 51

8. Cryptography basics **52**
8.1. Why this matters . . . 52
8.2. Advantage and security levels . . . 52
8.3. Hybrid arguments . . . 53
8.4. The birthday bound . . . 53
8.5. Pseudorandomness and encryption . . . 54
8.6. Simulation and secret sharing . . . 55
8.7. Learning with errors . . . 56

8.8. Code . . . 56
8.9. Cheat sheet . . . 57
8.10. Further reading . . . 58

II. Threat models and attacks **59**
The common thread . . . 60
Why the chapters come in this order . . . 60
What you will be able to do . . . 61

9. Threat models and privacy games **62**
9.1. Why this matters . . . 62
9.2. Threat models . . . 62
9.3. Privacy games and baselines . . . 63
9.4. Advantage, total variation and TPR at a low FPR . . . 65
9.5. Average case and worst case . . . 65
9.6. Code . . . 67
9.7. Cheat sheet . . . 68
9.8. Further reading . . . 68

10. Membership inference **69**
10.1. Why this matters . . . 69
10.2. The loss attack and the generalization gap . . . 69
10.3. Shadow models . . . 71
10.4. LiRA: a likelihood-ratio test per record . . . 71
10.5. Evaluating attacks: TPR at a low FPR . . . 74
10.6. Why outliers are most exposed . . . 75
10.7. Running LiRA on a real model . . . 77
10.8. Code . . . 77
10.9. Cheat sheet . . . 78
10.10. Further reading . . . 79

11. Attribute inference and model inversion **80**
11.1. Why this matters . . . 80
11.2. The attribute inference game . . . 80
11.3. The imputation baseline . . . 81
11.4. Model inversion: the MAP attack . . . 83
11.5. Confidence masking . . . 85
11.6. Code . . . 85
11.7. Cheat sheet . . . 87
11.8. Further reading . . . 88

12. Property inference **89**
12.1. Why this matters . . . 89
12.2. The property game and its ceiling . . . 89
12.3. Why a property leaks when no record does . . . 90
12.4. Shadow models and meta-classifiers . . . 91
12.5. Neural networks and permutation invariance . . . 93

12.6. Code . . . 94
12.7. Cheat sheet . . . 95
12.8. Further reading . . . 95

13. Memorization and training-data extraction **97**
13.1. Why this matters . . . 97
13.2. Verbatim, approximate, discoverable, extractable . . . 97
13.3. Duplication and model size . . . 98
13.4. Canaries and the exposure metric . . . 99
13.5. Extraction without the prefix: generate, rank, verify . . . 101
13.6. Code . . . 101
13.7. Cheat sheet . . . 103
13.8. Further reading . . . 104

14. Gradient inversion **105**
14.1. Why this matters . . . 105
14.2. Exact recovery from a fully connected layer . . . 105
14.3. Label recovery from the last layer . . . 106
14.4. Deep Leakage from Gradients . . . 108
14.5. Batches and averaging . . . 108
14.6. Code . . . 111
14.7. Cheat sheet . . . 112
14.8. Further reading . . . 113

15. Linkage and re-identification **114**
15.1. Why this matters . . . 114
15.2. Quasi-identifiers and the linkage attack . . . 114
15.3. How many people are unique? . . . 115
15.4. Sparse, high-dimensional data: the Netflix attack . . . 117
15.5. Does releasing a sample protect? . . . 118
15.6. Code . . . 120
15.7. Cheat sheet . . . 121
15.8. Further reading . . . 122

16. Syntactic anonymity and how it breaks **123**
16.1. Why this matters . . . 123
16.2. k-anonymity . . . 123
16.3. Homogeneity, background knowledge and l-diversity . . . 124
16.4. Skewness, similarity and t-closeness . . . 126
16.5. The composition attack . . . 127
16.6. Code . . . 129
16.7. Cheat sheet . . . 130
16.8. Further reading . . . 130

17. Reconstruction attacks **132**
17.1. Why this matters . . . 132
17.2. How accurate is too accurate? . . . 132
17.3. Least squares under random and worst-case error . . . 133

17.4. The Fundamental Law: what stops reconstruction, and what doesn't 135
17.5. Reconstructing a census block 135
17.6. Code 136
17.7. Cheat sheet 138
17.8. Further reading 139

III. Differential privacy foundations **140**
Who this part is for 141
The common thread 141
How to read this part 141
What you will be able to do 142

18. The definition of differential privacy **143**
18.1. Why this matters 143
18.2. What "one person" means 143
18.3. The definition 144
18.4. Randomized response 145
18.5. What ε means for an attacker 146
18.6. How small δ must be 147
18.7. What differential privacy does not protect 147
18.8. Code 148
18.9. Cheat sheet 149
18.10. Further reading 149

19. Post-processing, group privacy and composition **150**
19.1. Why this matters 150
19.2. Post-processing 150
19.3. Group privacy 151
19.4. Sequential composition 152
19.5. Parallel composition 153
19.6. What composes and what doesn't 154
19.7. Code 154
19.8. Cheat sheet 155
19.9. Further reading 156

20. The Laplace and Gaussian mechanisms **157**
20.1. Why this matters 157
20.2. Sensitivity 157
20.3. The Laplace mechanism 158
20.4. The Gaussian mechanism 160
20.5. Laplace or Gaussian? 161
20.6. Code 162
20.7. Cheat sheet 163
20.8. Further reading 164

21. Advanced and optimal composition **165**
21.1. Why this matters 165

21.2. The privacy loss of a composition . . . 165
21.3. Advanced composition . . . 166
21.4. Optimal composition . . . 167
21.5. Adaptive composition . . . 169
21.6. Code . . . 171
21.7. Cheat sheet . . . 172
21.8. Further reading . . . 173

22. Selection: exponential mechanism, report-noisy-max, sparse vector 174
22.1. Why this matters . . . 174
22.2. The exponential mechanism . . . 174
22.3. Report-noisy-max and Gumbel noise . . . 176
22.4. The sparse vector technique . . . 177
22.5. When the sparse vector technique goes wrong . . . 178
22.6. Code . . . 179
22.7. Cheat sheet . . . 180
22.8. Further reading . . . 181

23. Beyond global sensitivity 182
23.1. Why this matters . . . 182
23.2. Local sensitivity, and why it leaks . . . 182
23.3. Smooth sensitivity . . . 183
23.4. Propose-test-release . . . 184
23.5. Subsample-and-aggregate . . . 185
23.6. Clipping . . . 186
23.7. Code . . . 188
23.8. Cheat sheet . . . 189
23.9. Further reading . . . 189

24. Discrete and floating-point-safe mechanisms 191
24.1. Why this matters . . . 191
24.2. Floating-point Laplace breaks differential privacy . . . 191
24.3. The snapping mechanism . . . 192
24.4. The discrete Laplace mechanism . . . 193
24.5. The discrete Gaussian mechanism . . . 193
24.6. Sampling exactly, and choosing a library . . . 194
24.7. Code . . . 196
24.8. Cheat sheet . . . 197
24.9. Further reading . . . 198

25. Local, shuffle and distributed trust models 199
25.1. Why this matters . . . 199
25.2. Central and local differential privacy . . . 199
25.3. Frequency estimation . . . 200
25.4. The shuffle model . . . 202
25.5. Distributed DP . . . 204
25.6. Comparing the four models . . . 204
25.7. Code . . . 207

25.8. Cheat sheet . . . 208
25.9. Further reading . . . 208

26. Lower bounds: what privacy costs **210**
26.1. Why this matters . . . 210
26.2. Pure DP pays for the number of candidates . . . 210
26.3. How many records privacy costs . . . 212
26.4. The lower bound is a membership inference attack . . . 214
26.5. Reconstruction: why answers must carry $\sqrt{n}$ noise . . . 214
26.6. Code . . . 216
26.7. Cheat sheet . . . 218
26.8. Further reading . . . 218

IV. Privacy accounting **220**
The common thread . . . 221
The running example . . . 222
How to read this part . . . 222
What you will be able to do . . . 222

27. Rényi differential privacy **223**
27.1. Why this matters . . . 223
27.2. The RDP curve . . . 223
27.3. RDP curves of the basic mechanisms . . . 224
27.4. Composition adds curves . . . 226
27.5. Post-processing and group privacy . . . 226
27.6. The moments accountant is RDP . . . 227
27.7. From RDP to (ε, δ) . . . 227
27.8. Code . . . 229
27.9. Cheat sheet . . . 229
27.10. Further reading . . . 230

28. From Rényi DP to (ε, δ) and to tests **231**
28.1. Why this matters . . . 231
28.2. Mironov's conversion and where it loses . . . 231
28.3. The improved conversion . . . 232
28.4. Choosing the order . . . 233
28.5. The slack no conversion removes . . . 233
28.6. RDP as a bound on attacks . . . 235
28.7. Code . . . 235
28.8. Cheat sheet . . . 236
28.9. Further reading . . . 237

29. Concentrated differential privacy **238**
29.1. Why this matters . . . 238
29.2. The definition of zCDP . . . 238
29.3. Pure DP steps in a zCDP budget . . . 239
29.4. From zCDP to (ε, δ) . . . 239

29.5. Group privacy . . . 241
29.6. Subsampling: why DP-SGD is not accounted in zCDP . . . 241
29.7. Approximate zCDP . . . 243
29.8. Code . . . 243
29.9. Cheat sheet . . . 244
29.10. Further reading . . . 245

30. f-DP and Gaussian differential privacy **246**
30.1. Why this matters . . . 246
30.2. f-DP . . . 246
30.3. Gaussian differential privacy . . . 248
30.4. Composition . . . 249
30.5. The central limit theorem and DP-SGD . . . 250
30.6. RDP as a lossy summary of the curve . . . 251
30.7. Code . . . 252
30.8. Cheat sheet . . . 253
30.9. Further reading . . . 254

31. Amplification by subsampling **255**
31.1. Why this matters . . . 255
31.2. Amplification for (ε, δ)-DP under Poisson sampling . . . 255
31.3. The sampling scheme must match the neighbouring relation . . . 256
31.4. The RDP accountant for DP-SGD . . . 259
31.5. Small q and the moments accountant . . . 261
31.6. Other mechanisms . . . 261
31.7. Code . . . 263
31.8. Cheat sheet . . . 264
31.9. Further reading . . . 265

32. Numerical accounting with privacy loss distributions **266**
32.1. Why this matters . . . 266
32.2. The privacy loss distribution . . . 266
32.3. Composition is convolution . . . 267
32.4. Pessimistic and optimistic discretization . . . 268
32.5. Composition by FFT . . . 269
32.6. The subsampled Gaussian . . . 269
32.7. The running example: DP-SGD . . . 270
32.8. Code . . . 272
32.9. Cheat sheet . . . 274
32.10. Further reading . . . 274

33. Amplification by shuffling and by iteration **275**
33.1. Why this matters . . . 275
33.2. Shuffled randomized response, exactly . . . 275
33.3. The clone bound for any local randomizer . . . 276
33.4. Amplification by iteration . . . 278
33.5. Hidden state: privacy that stops growing . . . 278
33.6. Code . . . 281

33.7. Cheat sheet . . . 281
33.8. Further reading . . . 282

34. Adaptive composition: filters, odometers and individual accounting 283
34.1. Why this matters . . . 283
34.2. Why plugging in the realized parameters fails . . . 283
34.3. The Rényi filter . . . 284
34.4. Filters for pure differential privacy . . . 285
34.5. Odometers . . . 286
34.6. Individual privacy accounting in DP-SGD . . . 287
34.7. Code . . . 289
34.8. Cheat sheet . . . 290
34.9. Further reading . . . 291

35. Choosing an accountant 292
35.1. Why this matters . . . 292
35.2. Every accountant on one run . . . 292
35.3. Calibrating the noise for a target ε . . . 293
35.4. Where each accountant is tight . . . 295
35.5. Matching the accountant to the code . . . 296
35.6. What to report . . . 297
35.7. Checking an accountant . . . 298
35.8. Code . . . 299
35.9. Cheat sheet . . . 301
35.10. Further reading . . . 301

V. Differential privacy in machine learning 303
The common thread . . . 304
The running examples . . . 305
How to read this part . . . 305
What you will be able to do . . . 305

36. Private empirical risk minimization 306
36.1. Why this matters . . . 306
36.2. Output perturbation . . . 306
36.3. Objective perturbation . . . 308
36.4. Gradient perturbation: noisy gradient descent . . . 309
36.5. Which to use . . . 311
36.6. When the model is not convex . . . 313
36.7. Code . . . 313
36.8. Cheat sheet . . . 314
36.9. Further reading . . . 315

37. DP-SGD 316
37.1. Why this matters . . . 316
37.2. The algorithm and what its guarantee assumes . . . 316
37.3. Clipping: bias, noise and the choice of C . . . 317

37.4. Batch size at a fixed budget . . . 318
37.5. Per-example gradient norms without per-example gradients . . . 320
37.6. Privacy against accuracy . . . 320
37.7. Code . . . 322
37.8. Cheat sheet . . . 323
37.9. Further reading . . . 324

38. Hyperparameter tuning and private selection **325**
38.1. Why this matters . . . 325
38.2. The best of k runs costs k runs . . . 325
38.3. Random stopping . . . 327
38.4. Rényi bounds for DP-SGD . . . 327
38.5. The validation score, and selecting among fixed candidates . . . 330
38.6. Tuning the running example . . . 331
38.7. What to do and what to report . . . 332
38.8. Code . . . 334
38.9. Cheat sheet . . . 335
38.10. Further reading . . . 336

39. DP-FTRL and matrix factorization **337**
39.1. Why this matters . . . 337
39.2. Training is a stream of prefix sums . . . 337
39.3. The matrix mechanism . . . 338
39.4. Tree aggregation and DP-FTRL . . . 339
39.5. The square-root factorization . . . 340
39.6. Several participations . . . 340
39.7. The running example: matrix mechanisms against DP-SGD . . . 343
39.8. Code . . . 347
39.9. Cheat sheet . . . 348
39.10. Further reading . . . 349

40. Private prediction: PATE and in-context learning **350**
40.1. Why this matters . . . 350
40.2. The noisy vote and its cost . . . 350
40.3. When a noisy vote is accurate . . . 351
40.4. Private answers against a private model . . . 352
40.5. PATE: from answers to a model . . . 353
40.6. Paying less when the teachers agree . . . 354
40.7. Private in-context learning . . . 356
40.8. Code . . . 356
40.9. Cheat sheet . . . 358
40.10. Further reading . . . 359

41. Public data and private fine-tuning **360**
41.1. Why this matters . . . 360
41.2. Why dimension costs, and when it does not . . . 360
41.3. The last layer: linear probing . . . 362
41.4. Low-rank adapters . . . 362

41.5. Gradients near a public subspace . . . 363
41.6. Pretraining and fine-tuning the running example . . . 363
41.7. What public data does not buy . . . 365
41.8. Code . . . 366
41.9. Cheat sheet . . . 367
41.10. Further reading . . . 367

42. User-level differential privacy **369**
42.1. Why this matters . . . 369
42.2. The privacy unit . . . 369
42.3. What a record-level model gives a heavy user . . . 370
42.4. User-level DP-SGD . . . 371
42.5. Contribution bounding . . . 372
42.6. Heavy users and the objective . . . 373
42.7. Choosing a strategy . . . 374
42.8. Code . . . 376
42.9. Cheat sheet . . . 376
42.10. Further reading . . . 377

43. Disparate impact **378**
43.1. Why this matters . . . 378
43.2. Disparate impact, measured . . . 378
43.3. Clipping reweights the groups . . . 379
43.4. The noise floor of a small group . . . 382
43.5. Measuring per-group accuracy . . . 383
43.6. Mitigations and what they cost . . . 384
43.7. Code . . . 386
43.8. Cheat sheet . . . 386
43.9. Further reading . . . 387

44. Choosing ε **388**
44.1. Why this matters . . . 388
44.2. What one (ε, δ) pair promises . . . 388
44.3. The whole curve of DP-SGD . . . 389
44.4. What an attacker can believe . . . 391
44.5. Reconstruction . . . 392
44.6. How small δ must be . . . 392
44.7. Deployed values and their units . . . 393
44.8. Justifying a choice . . . 393
44.9. Code . . . 395
44.10. Cheat sheet . . . 395
44.11. Further reading . . . 396

45. Auditing private training **397**
45.1. Why this matters . . . 397
45.2. From an attack to a lower bound . . . 397
45.3. Canaries and threat models . . . 398
45.4. Why even the best audit stops short . . . 399

45.5. One-run auditing 401
45.6. What a black-box attacker learns 402
45.7. Finding bugs 402
45.8. Code 404
45.9. Cheat sheet 406
45.10. Further reading 406

Appendices **408**

A. Glossary **408**

B. Index **411**

Preface

This is the practitioner edition of *Privacy-Preserving Machine Learning*. It is for people who build, review and audit machine-learning systems that handle personal data, and who need to know what each result says, when it applies, how to use it and what goes wrong in practice, without working through its proof.

It covers the same topics as the full edition in the full edition, in the same parts and chapters. Each topic is explained in four steps: the idea and why it works, the result or rule, a worked example with real numbers, and the pitfalls that catch people in practice. Each chapter ends with a cheat sheet and a link to the full edition's chapter, where every result is derived step by step.

How this book is organized

1. **Prerequisites** collects the mathematics the later parts use: distributions, tail bounds, ways to compare distributions, hypothesis testing, and the basics of information theory, optimization, stability and cryptography.
2. **Threat models and attacks** shows how models and data leak: membership inference, attribute and property inference, memorization and extraction, gradient inversion, re-identification, and reconstruction from aggregate statistics.
3. **Differential privacy foundations** defines the guarantee those attacks call for, builds its mechanisms, composes them, and says what privacy costs.
4. **Privacy accounting** tracks the guarantee through the many steps of private training, and ends with how to choose an accountant and what to report.
5. **Differential privacy in machine learning** trains models with the guarantee: where the noise goes, how the knobs of noisy, clipped gradient descent trade privacy for accuracy, what else spends the budget, who pays for the noise, which ε to choose, and how to audit the result.

The numbers in the text are computed by the scripts in the repository's `code/` directory, the same ones the full edition uses, and tests check that the scripts still produce them.

Part I.

Prerequisites

This part collects the mathematics the rest of the book stands on. It has no privacy in it: it is standard probability, statistics, optimization, information theory and cryptography, chosen because a later part needs it and taken only as far as that need goes. You need the basics of probability (expectation, variance, independence), enough machine learning to recognise a loss function and stochastic gradient descent (SGD), and enough Python to read a short script. If you know a topic, skim its cheat sheet; each later chapter restates in a sentence what it uses and links back here.

The common thread

Later parts study algorithms whose output is random, and they keep asking the same few questions about them. How far is a random output from its expected value? How different are the outputs on two inputs that differ in one example, and how well could anyone tell them apart? How much does a trained model depend on a single training example? How much does an observation reveal about something hidden, and what can an observer with limited computing power learn?

Two ideas answer most of them. The first is the **likelihood ratio** $p(x)/q(x)$: how many times more likely an observation is under one distribution than under another. It measures what one observation says about which distribution produced it, so telling two distributions apart always comes down to it: divergences summarize it, the best test thresholds it, and when it is bounded no single observation is strong evidence. The second idea is the **effect of one input**: how far one example can move an average, a minimizer or a loss. A quantity that no single input can move much has little room to vary, so small influence turns into narrow error bars, and in learning into a model that generalizes.

Why the chapters come in this order

The first four chapters build one argument, each on the last. Probability distributions in the full edition introduces the few families that model counts and noise, and ends by comparing a noise density with a copy of itself moved sideways: the normal's likelihood ratio grows without bound in the tails, the Laplace's is capped. That raises the question of how often the extreme ratios occur, and Concentration inequalities in the full edition answers it by turning what you know about a random quantity into an error bar that fails with a chosen small probability. Divergences in the full edition then compresses the difference between two distributions into one number, and shows that each candidate number answers a different question. Hypothesis testing in the full edition says what those numbers mean for anyone trying to tell the two distributions apart: the best test, the curve of its two errors, and why a detector should be judged where false alarms are rare.

Information theory in the full edition continues the same thread from a different angle, asking how much an observation reveals about a hidden value, on average and for a single best guess. Optimization in the full edition and Algorithmic stability and generalization in the full edition take up the second idea: how fast training converges, how far its result moves when one example changes, and why a model that depends little on each example generalizes. Cryptography basics in the full edition stands somewhat apart. It defines what an efficient observer must not learn, as two worlds an attacker must tell apart, and can wait until a later chapter needs it.

How to read a chapter

Each chapter opens with its practical question and the idea that answers it, then reasons each result from a problem, with a worked example and the consequences for practice, and closes with a cheat sheet. The derivations and the scripts that check every number are in the full edition, linked at the end.

What you will be able to do

By the end of this part you will be able to put an error bar on a noisy answer, a measured rate or the maximum of many noise draws, and choose the inequality that gives the narrowest valid one. You will be able to compute the Kullback–Leibler (KL), Rényi, total variation and hockey-stick divergences between two normal or two Laplace distributions, and explain why processing data cannot increase any of them. You will know the best test between two distributions, read its trade-off curve, report a detector by its true positive rate at a low false positive rate, and size a test set to measure that rate. You will also be able to say how fast gradient descent converges and how much a trained model moves when one example changes, compute what an observation leaks about a hidden value, and state a security property as a game between an attacker and two worlds.

1. Probability distributions

1.1. Why this matters

Later parts keep asking two questions about a random output: how wide is it, and how hard is it to tell apart from a slightly different one? Both answers depend on its distribution, and a handful of families cover almost every case: counts of events, such as errors on a test set, and additive noise, usually normal or Laplace.

These families are built from one another. A binomial is a sum of coin flips, a Poisson is a binomial with many unlikely trials, a Laplace is two exponential waiting times back to back, and a normal is what many small independent pieces add up to. Each family's spread and tail follow from how it is built, and so does what matters most later: how far a density drifts from a shifted copy of itself.

1.2. Counts: binomial and Poisson

Every measured rate (accuracy, error rate, false positive rate) is a count of yes/no outcomes divided by the number of trials. With n independent trials, each a yes with probability p, the count is Binomial(n, p). Its mean is np and its variance $np(1-p)$, because one trial has variance $p(1-p)$ and variances add over independent terms. Dividing the count by n, the measured rate $\hat{p}$ has standard deviation $\sqrt{p(1-p)/n}$: four times the data only halves the error.

The variance vanishes near $p = 0$, which sounds like good news for rare events but is not. What matters is the error relative to the rate, and the count's standard deviation, about $\sqrt{np}$, divided by its mean np is about $1/\sqrt{np}$. So relative precision is set by the expected count, not by the number of trials. A classifier that errs with probability 0.05 on 2,000 test examples makes about 100 errors, with standard deviation about 9.7, and the measured rate is typically within about half a point of 5%. At an error rate of 0.001 the same test set expects 2 errors, with standard deviation about 1.4, a relative error of about 70%, and one test set in seven shows no error at all.

When the trials are many and each is unlikely, the binomial depends only on the expected count $\mu = np$, and the limit is the **Poisson distribution** Poisson(μ). Its variance equals its mean, μ, because the factor $1 - p$ in the binomial variance tends to 1. A training loop that includes each of 60,000 examples independently with probability 0.01 gets batches whose size is close to Poisson(600): mean 600, standard deviation about 24.5 (Figure 1.1).

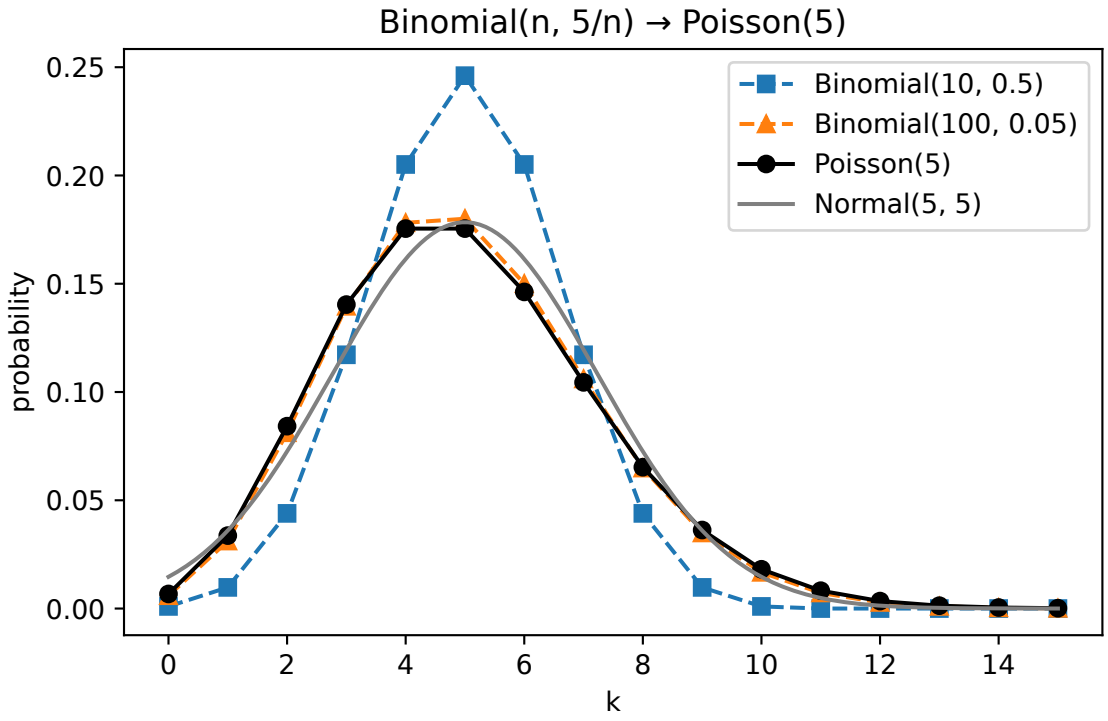


Figure 1.1.: Binomial$(n, 5/n)$ for $n = 10$ and 100 against Poisson(5) and its normal approximation. At $n = 100$ the binomial and the Poisson almost coincide.

1.3. Laplace and exponential

The waiting time for a rare event, with many independent chances each unlikely, follows the **exponential distribution** Exp(b): density $\frac{1}{b}e^{-y/b}$ on $y \geq 0$, mean b, tail $e^{-t/b}$. The **Laplace distribution** puts one exponential on each side of a centre. Lap(μ, b), with centre μ and scale b, has density

$$f(x) = \frac{1}{2b}e^{-|x-\mu|/b}. \tag{1.1}$$

So the size $|X - \mu|$ is exactly Exp(b): the average absolute error is b, the variance is $2b^2$ (so a standard deviation σ means $b = \sigma/\sqrt{2}$), and the two-sided tail is exact,

$$\Pr[|X - \mu| \geq t] = e^{-t/b}. \tag{1.2}$$

The Laplace has heavier tails than a normal of the same variance, and the log scale shows why (Figure 1.2). The logarithm of each density is its exponent: two straight lines $-|x|/b$ for the Laplace, a parabola $-x^2/(2\sigma^2)$ for the normal. Far enough out a parabola always falls below a line, so the normal's tail eventually becomes far lighter. With the same variance, the Laplace exceeds three times its scale with probability $e^{-3} \approx 0.050$, the normal only with probability about 0.034, and the gap widens. So a tail bound shaped for the normal fails for Laplace noise far enough out.

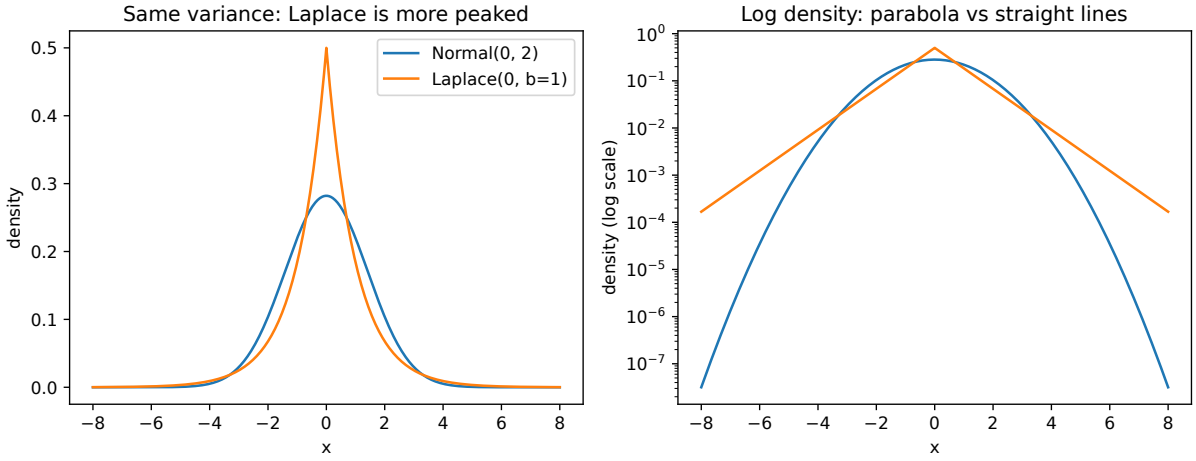


Figure 1.2.: Normal and Laplace densities with variance 2, on linear and log scales. The log of the normal density is a parabola; the log of the Laplace density is two straight lines.

Libraries differ in whether they take the scale b or the rate $1/b$ (NumPy's `rng.exponential(b)` takes the scale), so check before trusting a noise level.

1.4. Normal

$\mathcal{N}(\mu, \sigma^2)$ has density $\frac{1}{\sigma\sqrt{2\pi}}e^{-(x-\mu)^2/(2\sigma^2)}$, and every probability reduces to the standard normal cumulative distribution function (CDF) Φ, through $\Pr[X \leq x] = \Phi((x - \mu)/\sigma)$. It stays normal under sums: $\sum_i a_i X_i$ for independent normals is normal with mean $\sum_i a_i \mu_i$ and variance $\sum_i a_i^2 \sigma_i^2$.

That rule explains normal noise in many dimensions. A vector $X \sim \mathcal{N}(0, \sigma^2 I_d)$ has d independent coordinates, so its squared length is a sum of d terms of mean σ^2, and like any sum of many independent terms its relative spread shrinks: the length sits at $\sigma\sqrt{d}$, with relative spread about $1/\sqrt{2d}$ (Table 1.1). Yet the projection onto a fixed unit vector is a weighted sum whose squared weights add to 1, so it is just $\mathcal{N}(0, \sigma^2)$. Noise $\mathcal{N}(0, 0.01^2 I_d)$ on a model with $d = 10^6$ parameters has length about $0.01 \times 1000 = 10$, yet along any direction fixed in advance it is only $\mathcal{N}(0, 0.01^2)$: large in norm, small in every direction.

Table 1.1.: Mean and standard deviation of $\|X\|/\sqrt{d}$ over 2,000 draws of $X \sim \mathcal{N}(0, I_d)$.

d	1	10	100	1,000	10,000
mean of $\|X\|/\sqrt{d}$	0.802	0.977	0.998	1.000	1.000
std of $\|X\|/\sqrt{d}$	0.611	0.220	0.069	0.022	0.007

The **central limit theorem (CLT)** explains why the normal is everywhere: the standardized sum of n independent draws with finite variance approaches $\mathcal{N}(0, 1)$. But the skew of the summands averages out only like $1/\sqrt{n}$: for sums of exponentials the largest error in the CDF is still 0.0243 at $n = 30$. That is fine for the bulk and useless in the far tail, whose probabilities are much smaller than that error; tails need Concentration inequalities in the full edition. Small tails need care in code too: $1 - \Phi(x)$ subtracts two nearly equal numbers and loses every digit, so compute the tail directly as $\frac{1}{2}\operatorname{erfc}(x/\sqrt{2})$.

1.5. A density against its shifted copy

You observe $x = \mu + X$ with noise density f, and μ is either 0 or a known shift s. The two candidates give x the densities $f(x)$ and $f(x - s)$, the same curve moved sideways, and how much one observation favours the first is the **log ratio**

$$L(x) = \ln f(x) - \ln f(x - s). \tag{1.3}$$

If $|L(x)|$ is bounded, no single observation can settle the question; if not, some observations settle it almost for certain.

The log of each density is its exponent, so the answer comes from the shapes in Figure 1.2. Two copies of the normal's parabola have the same curvature, so subtracting them cancels the x^2 terms and leaves a straight line, unbounded:

$$L(x) = \frac{s^2 - 2sx}{2\sigma^2}. \tag{1.4}$$

The Laplace's log density is $-|x|/b$, a distance from the centre divided by b. Moving the centre by s changes the distance from any point by at most $|s|$, so

$$L(x) = \frac{|x - s| - |x|}{b}, \qquad |L(x)| \le \frac{|s|}{b}, \tag{1.5}$$

and the bound is reached on whole half-lines, every $x \le 0$ and every $x \ge s$, where both distances change by exactly s.

Take $s = 1$, $\sigma = 1$ and $b = 1$ (Table 1.2 and Figure 1.3). At $x = 10$ the normal log ratio is -9.5: the observation is about 13,360 times more likely under $\mu = 1$. The Laplace never says more than $e \approx 2.72$ times, however far out x lies.

Table 1.2.: The log ratio of a density and its copy shifted by $s = 1$, for $\mathcal{N}(0, 1)$ and Lap(1).

x	-2	0	0.5	1	3	10
Normal, $L(x) = (1 - 2x)/2$	2.5	0.5	0	-0.5	-2.5	-9.5
Laplace, $L(x) = \lvert x - 1\rvert - \lvert x\rvert$	1	1	0	-1	-1	-1

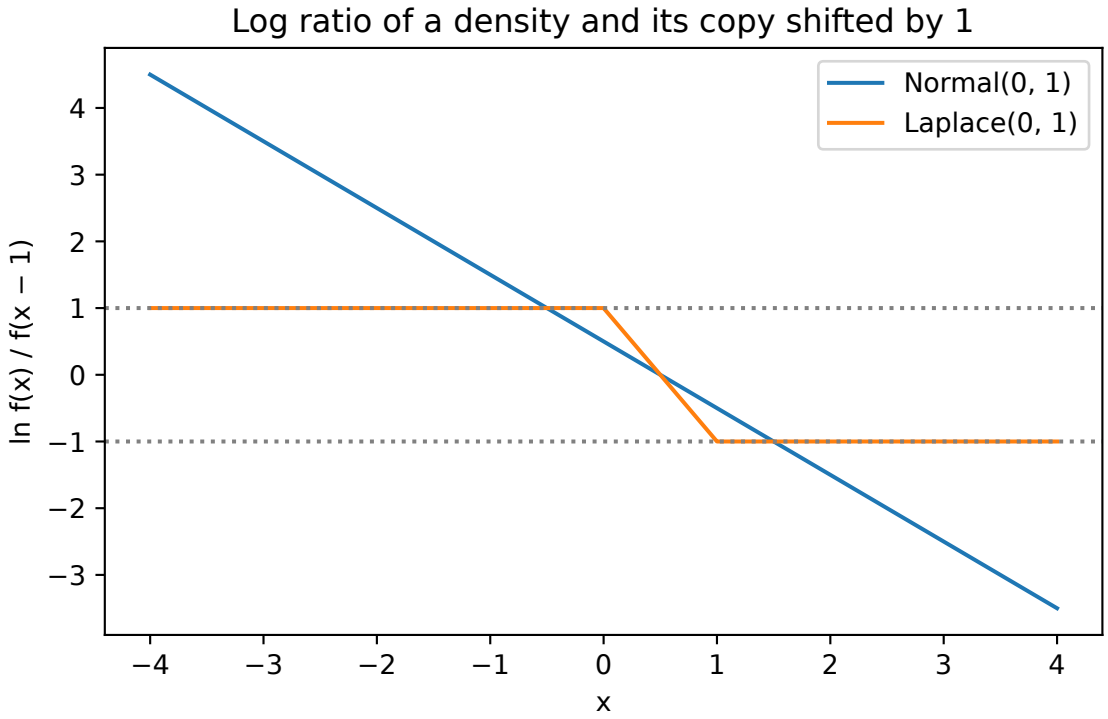


Figure 1.3.: The log ratio $L(x)$ of a density and its copy shifted by $s = 1$: a straight line for the standard normal, clipped at ± 1 for Lap(1).

But "unbounded" does not mean "often large": the normal's extreme ratios sit in tails with very little probability. How little, and what it buys an observer, is what Divergences in the full edition and Hypothesis testing in the full edition measure. And only shift over scale matters: doubling the noise is the same as halving the shift.

1.6. Cheat sheet

Table 1.3.: The five families: density or probability mass function, moments and tail.

Distribution	Density or probability mass function	Mean	Variance	Tail
Binomial(n, p)	$\binom{n}{k} p^k (1-p)^{n-k}$	np	$np(1-p)$	Chernoff, see Concentration inequalities in the full edition
Poisson(μ)	$e^{-\mu}\mu^k/k!$	μ	μ	$\approx (e\mu/k)^k$
Exp(b)	$\frac{1}{b}e^{-y/b}$, $y \geq 0$	b	b^2	$e^{-t/b}$
Lap(μ, b)	$\frac{1}{2b}e^{-\lvert x-\mu\rvert/b}$	μ	$2b^2$	$\Pr[\lvert X-\mu\rvert \geq t] = e^{-t/b}$
$\mathcal{N}(\mu, \sigma^2)$	$\frac{1}{\sigma\sqrt{2\pi}}e^{-(x-\mu)^2/(2\sigma^2)}$	μ	σ^2	$\Pr[X-\mu \geq t] \leq \frac{1}{2}e^{-t^2/(2\sigma^2)}$

Rules of thumb

- A measured rate has standard deviation $\sqrt{p(1-p)/n}$; its relative precision is set by the expected count np.
- Normal noise in d dimensions has norm $\approx \sigma\sqrt{d}$ and is $\mathcal{N}(0, \sigma^2)$ along every fixed direction.
- Laplace shifted by s: log ratio at most $|s|/b$. Normal: unbounded, but large only in the tails, which carry very little probability.

1.7. Further reading

- Blitzstein & Hwang, *Introduction to Probability* (2nd ed., 2019), ch. 3–5 and 7: every distribution here, with worked examples.
- Wasserman, *All of Statistics* (2004), ch. 2–3: a compact reference.
- Vershynin, *High-Dimensional Probability* (2018), §3.1: why the norm of a normal vector concentrates.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

2. Concentration inequalities

2.1. Why this matters

Almost every number in a private system is random. The noise added to a count is random by design. An error rate measured on a test set depends on which examples happened to be in it. The number of examples in a randomly sampled minibatch changes from one step to the next. Before you can use any of these numbers, you need to know how far it can stray from the value you care about.

A concentration inequality answers that question. It tells you how wide to make an interval around your estimate so that the true value falls outside it only rarely, say once in a thousand runs. This chapter calls that rare event a failure, and the chance of it the **failure probability**, written β. It calls the distance from the estimate to either end of the interval the **width**.

How wide the interval must be depends on how much you know about the random quantity. If you know only its variance, the width grows quickly as you ask for a smaller failure probability. If you also know the shape of its distribution, the width grows very slowly. The difference is small for a 95% interval. But later parts of this book ask for failure probabilities of one in a million or less, and there the shape can make an interval a hundred times narrower. This chapter shows how to use everything you know.

2.2. Knowing only the variance: Markov and Chebyshev

Start with the weakest thing you could know about a quantity: it is never negative, and you know its average. That alone limits how often it can be large. Suppose customers wait 2 minutes at a counter on average. Then at most a quarter of them can wait 8 minutes or more, because if more than a quarter did, those customers alone would push the average above 2 minutes. This is **Markov's inequality**. Write X for a quantity that is never negative, and $\mathbb{E}[X]$ for its average. For any threshold a, Markov's inequality says that $\Pr[X \geq a] \leq \mathbb{E}[X]/a$.

Markov's inequality on its own is rarely useful, because it ignores how spread out the quantity is. What makes it useful is that you can apply it to other quantities built from X. The first of these is the squared distance from the mean. Write μ for the mean of X. Its variance, written σ^2, is the average squared distance from μ. A squared distance is never negative, so Markov's inequality applies to it. Now pick a distance t. Being at least t away from the mean is the same as having a squared distance of at least t^2. Markov's inequality bounds the chance of that by σ^2/t^2. This is **Chebyshev's inequality**:

$$\Pr[|X - \mu| \geq t] \leq \frac{\sigma^2}{t^2}. \tag{2.1}$$

To turn the bound into an interval, set the right side equal to β and solve for t. The result is $t = \sigma/\sqrt{\beta}$. So with probability at least $1 - \beta$, the quantity lies within $\sigma/\sqrt{\beta}$ of its mean. The same works for an average of n independent samples. Averaging divides the variance by n, so the width becomes $\sigma/\sqrt{n\beta}$.

Chebyshev's inequality is weak because it must hold for every distribution with the given variance. Some of those distributions hide their spread in a rare spike far from the mean. To cover them, the width has to grow like $1/\sqrt{\beta}$ as the failure probability falls. A quantity whose tail falls off like a normal distribution's needs far less. Its width is $\sigma\sqrt{2\ln(1/\beta)}$, which grows only like the square root of a logarithm. Table 2.1 compares the two widths.

Table 2.1.: Width needed to fail with probability at most β, in standard deviations.

Failure probability β	Chebyshev, $1/\sqrt{\beta}$	Normal-shaped tail, $\sqrt{2\ln(1/\beta)}$
0.05	4.5	2.4
10^{-3}	31.6	3.7
10^{-6}	1,000	5.3

At a failure probability of 5%, the two widths are close: Chebyshev needs 4.5 standard deviations, and the normal-shaped tail needs 2.4. At one in a million, Chebyshev needs 1,000 standard deviations, while the normal-shaped tail needs 5.3. Chebyshev's inequality is fine for a quick 95% error bar. It is useless when failures must be very rare, as they must be in the later parts of this book.

2.3. Normal and Laplace noise

When you know the distribution of the noise, you don't need a general inequality. You can read the width directly off the distribution's tail, which is the chance of landing more than a given distance from the centre. The two noise distributions this book uses most are the Laplace and the normal distributions (Probability distributions in the full edition).

Laplace noise has a scale b, which sets its typical size, and we write it $\mathrm{Lap}(b)$. Its tail has an exact formula. A draw lands more than t from zero with probability $e^{-t/b}$. Setting this chance equal to β and solving for t gives the width. With probability $1 - \beta$, the noise stays within $b\ln(1/\beta)$ of zero.

Compare this with Chebyshev's inequality, which uses only the variance of Laplace noise. Take scale 1 and a 95% interval. Chebyshev's inequality gives a width of about 6.3, while the exact tail gives about 3.0. At a failure probability of one in a million, Chebyshev gives 1,414 and the exact tail gives 13.8. Ignoring the shape of the noise makes the interval a hundred times wider.

Normal noise with standard deviation σ has no simple formula for its tail, but a simple bound comes close. A draw lands more than t from zero with probability at most $e^{-t^2/(2\sigma^2)}$. Solving for t in the same way gives the width $\sigma\sqrt{2\ln(1/\beta)}$ from Table 2.1.

The shapes of the two tails explain why their widths grow at different rates. The Laplace tail falls like e^{-t}, so the width it needs grows like $\ln(1/\beta)$. The normal tail falls like e^{-t^2}, so it is the

square of the width that grows like $\ln(1/\beta)$. The width itself grows only like the square root of that, which is why the normal width grows so slowly.

Often you need many random quantities to be small at the same time. A system might release k noisy counts and need every one of them to be accurate. The **union bound** handles this. It says that the chance that at least one of several events happens is at most the sum of their separate chances. The sum can overcount, because an outcome in which two events happen is counted twice. But it never misses an outcome. So if you want all k counts to be accurate except with probability β, ask each one to fail with probability at most β/k.

This costs very little. Asking each quantity to fail with probability β/k instead of β adds only $\ln k$ inside each logarithm. All k Laplace draws then stay within $b \ln(k/\beta)$ of zero. All k normal draws stay within $\sigma\sqrt{2\ln(k/\beta)}$. Because the number of quantities enters only through a logarithm, controlling many of them at once is cheap.

The union bound looks crude, but for rare failures it is nearly exact. When each failure is rare, two failures at once are rarer still, so the double counting adds almost nothing. Take the largest of 100 Laplace draws with scale 1, and a failure probability of 0.05. The union bound gives the width $\ln(100/0.05) = 7.60$. In simulation, the largest draw exceeds 7.60 in 4.86% of runs. That is just under the 5% the bound allows (Figure 2.1).

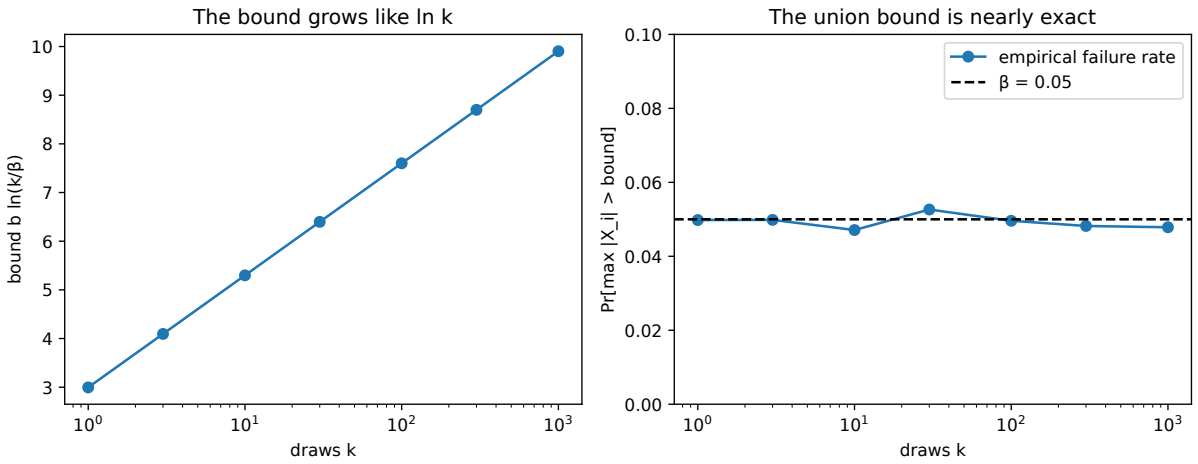


Figure 2.1.: Left: the width $b \ln(k/\beta)$ for the largest of k Laplace draws grows like $\ln k$. Right: the probability that the largest draw exceeds it stays at $\beta = 0.05$ for every k.

Sometimes you add noisy values together, for example when a total is computed from k noisy counts. Write S for the sum of k independent Laplace draws with scale b. Near its centre, the sum looks like a normal distribution, as the central limit theorem (CLT) predicts. Far from the centre it does not. There its tail keeps the slow decay of a single Laplace draw. A bound that is valid everywhere has to describe both regions:

$$\Pr[S \geq t] \leq \exp\Big(-\min\Big(\frac{t^2}{8kb^2}, \frac{t}{4b}\Big)\Big). \tag{2.2}$$

Read the formula in two parts. For moderate t, the first term inside the minimum is the smaller one, and the bound has the normal shape. For large t, the second term takes over, and the bound decays only like a single Laplace tail.

The difference matters whenever you need a guarantee. Take 10 draws with scale 1. The normal approximation says that their sum reaches 20 with probability 3.9×10^{-6}. Simulation gives 3.4×10^{-5}, about nine times more (Figure 2.2). A system that trusted the normal approximation would fail nine times as often as it promised. When the answer must be a guarantee, use a bound, not the CLT.

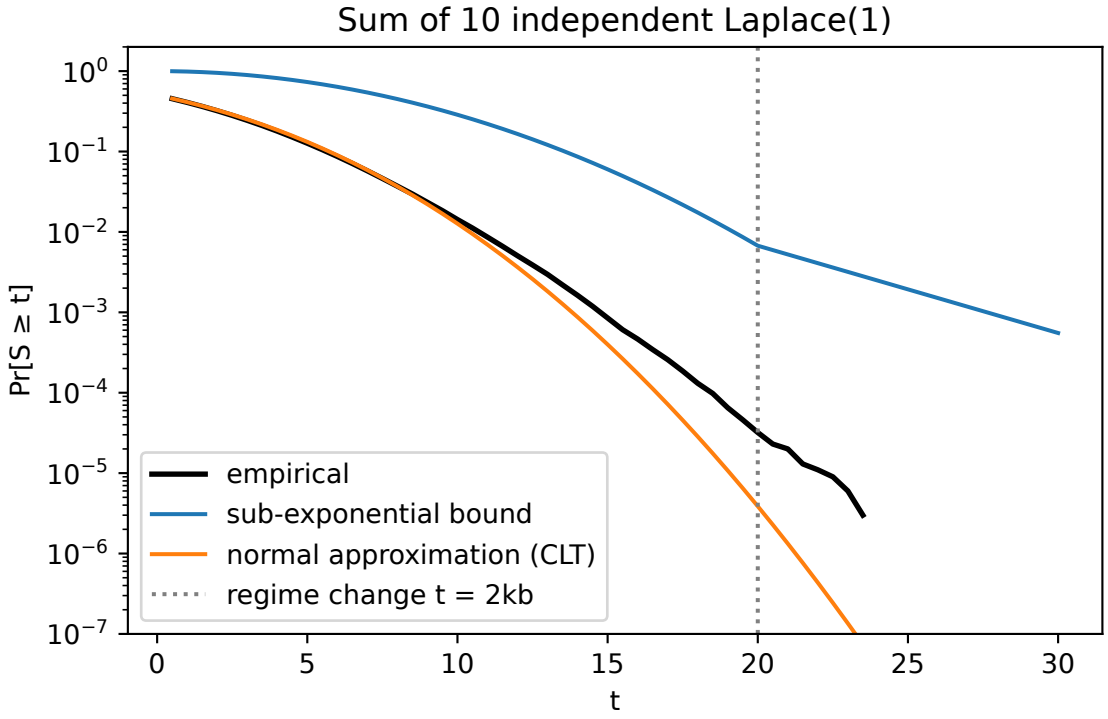


Figure 2.2.: Empirical tail of a sum of 10 Lap(1) draws, against the bound above and the normal approximation, which badly underestimates the far tail.

2.4. Counts: the Chernoff bound

Many quantities in machine learning are counts: errors on a test set, examples in a sampled minibatch, records that match a query. A count adds up many independent yes-or-no outcomes. Its tail has no simple formula, and Chebyshev's bound for it is far too loose. The **Chernoff bound** gives a tail that falls exponentially, using one more idea.

The idea is to apply Markov's inequality to $e^{\lambda X}$ instead of to X itself, where λ is a positive number you are free to choose. The exponential grows so fast that its average is dominated by the large values of X. So the average carries information about the whole tail, not just about the variance. The bound holds for every λ, and you pick the one that makes it smallest. For a count, the average of the exponential is also easy to compute. The count is a sum of independent outcomes, so the average is a product with one factor per outcome.

Carrying out this calculation gives a bound in terms of two numbers. The first is the expected count, written μ. The second is the relative deviation you ask about, written δ, so that $\delta = 0.2$

means 20% above or below μ. For a count S, the Chernoff bounds are

$$\Pr[S \geq (1+\delta)\mu] \leq \exp\Big(-\frac{\delta^2\mu}{2+\delta}\Big), \qquad \Pr[S \leq (1-\delta)\mu] \leq \exp\Big(-\frac{\delta^2\mu}{2}\Big). \tag{2.3}$$

The first bounds the chance that the count overshoots its mean by the fraction δ. The second bounds the chance that it undershoots, and holds for δ between 0 and 1.

The most important thing about these bounds is that the exponent contains the expected count μ, not the number of trials. Rare events therefore concentrate only once you expect many of them. A million trials that each succeed with probability one in a million expect a single success. The count could easily be 0 or 3, and no bound can say otherwise. The bound sees μ because it keeps each trial's probability. It therefore knows that a count of rare events has a variance of only about μ, instead of assuming the worst case.

The two tails also differ. For large δ, the exponent of the overshoot bound grows like $\delta\mu$ rather than $\delta^2\mu$, so it falls more slowly. Counts have long right tails, like the Poisson distribution. A count can overshoot its mean by a lot far more easily than it can undershoot it.

Here is how the bound is used in practice. A training loop forms each minibatch by including each of 60,000 examples independently with probability 0.01. The batch size is a count with expected value $\mu = 600$. How likely is a batch to be more than 20% too large? With $\delta = 0.2$, the overshoot bound gives about 1.8×10^{-5}. The undershoot bound gives about 6.1×10^{-6} for a batch more than 20% too small. Adding the two, a single batch leaves the range $600 \pm 20\%$ with probability about 2.4×10^{-5}. Over 1,000 training steps, the union bound multiplies this by 1,000. So the chance that any batch leaves that range is about 0.024.

2.5. Averages of bounded quantities: Hoeffding and Bernstein

Often you know neither the distribution nor the variance, only the range. A loss clipped to lie between 0 and 1 is an example, and so is a 0/1 indicator of a mistake. Knowing the range already tells you a lot. A quantity confined to an interval cannot have a heavy tail, because it cannot go past the ends at all. Its variance cannot be large either. The most spread-out quantity on an interval puts half its probability on each end, like a fair coin. Its variance is a quarter of the squared length of the interval.

Hoeffding's inequality turns this into a width for an average. Take n independent quantities, each between a and b. Write $\bar{X}_n$ for their average and μ for its expected value. Hoeffding's inequality says that the average concentrates at least as well as an average of fair coins on the two ends would. With probability at least $1 - \beta$,

$$|\bar{X}_n - \mu| < (b-a)\sqrt{\frac{\ln(2/\beta)}{2n}}. \tag{2.4}$$

The width shrinks like $1/\sqrt{n}$ as you add samples. As you lower the failure probability, it grows only like the square root of a logarithm, as for normal noise.

The most common use is to compare a model's error on its training data with its error on new data. For one fixed model, the training error is an average of n losses between 0 and 1, so Hoeffding's inequality applies directly. A learner, however, picks its model after seeing the data. So you need

the bound to hold for every model it might pick. Suppose the candidates form a finite set H, and write $|H|$ for their number. The union bound puts an extra $\ln|H|$ inside the logarithm. With probability at least $1-\beta$, every candidate's training error is then within $\sqrt{\ln(2|H|/\beta)/(2n)}$ of its true error. The number of candidates enters only through its logarithm, so even a huge set of them costs little.

Knowing only the range is Hoeffding's strength, since there is nothing to estimate. It is also its weakness. The inequality assumes the largest variance that the range allows. A quantity that is rarely 1 has a much smaller variance than that. The gap is widest exactly when you measure small rates, such as a false-positive rate.

Suppose a classifier makes 1 false positive on 1,000 negative examples, and you want a 95% upper bound on its true false-positive rate. One-sided Hoeffding adds about 0.039 to the measured rate of 0.001. That is about 40 times the rate you measured. The inequality assumes a variance of 0.25, the variance of a fair coin. The true variance of a rate near 0.001 is only about 0.001.

Bernstein's inequality fixes this by using the variance. Write σ^2 for the variance of each quantity, and M for the largest distance it can be from its mean. Then the width is about

$$\sqrt{\frac{2\sigma^2\ln(1/\beta)}{n}}+\frac{2M\ln(1/\beta)}{3n}. \tag{2.5}$$

The first term is the normal-shaped width, computed with the true variance. The second term pays for the range, but it shrinks like $1/n$ instead of $1/\sqrt{n}$. For the classifier, estimate the variance as 0.001. Bernstein's inequality then adds about 0.0045, against Hoeffding's 0.039. This is only a heuristic, because the variance was estimated from the same data.

The exact answer comes from the binomial distribution itself. The Clopper–Pearson upper bound on the rate is 0.0047, close to Bernstein's. Hypothesis testing in the full edition shows how to compute it.

Figure 2.3 shows the same effect on a second example. It averages 200 yes-or-no outcomes, each a yes with probability 0.1. The question is how likely the average is to reach 0.15. Hoeffding's inequality bounds that chance by 0.37. That is worse even than Chebyshev's inequality, which uses the variance and gets 0.18. The Chernoff bound, which keeps the probability of each outcome, gets 0.135. The true chance is about 9.6×10^{-3}.

2.6. Functions of many inputs: McDiarmid and Azuma

So far, every quantity was a sum of independent pieces. Two common situations break this. A trained model's error is a complicated function of its training data, not a sum. And in a sequential procedure, each step can depend on what happened before, so the pieces are not independent. Concentration still holds in both situations, provided that no single input or step can move the result by much.

Start with a function f of independent inputs $X_1,\dots,X_n$. Suppose that changing only the i-th input, with the others held fixed, moves the result by at most an amount c_i. **McDiarmid's**

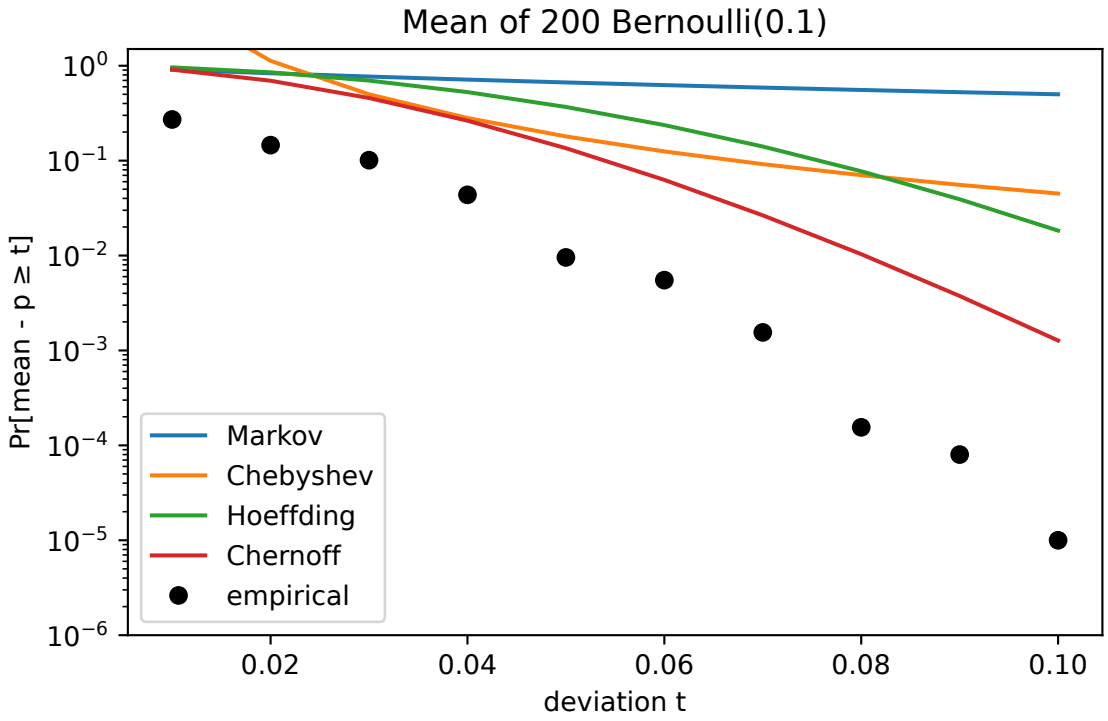


Figure 2.3.: Empirical tail of the mean of 200 Bernoulli(0.1) variables against the Markov, Chebyshev, Hoeffding and Chernoff bounds.

inequality says that the result then concentrates around its average as tightly as a sum of pieces with ranges c_i would:

$$\Pr[f(X) - \mathbb{E}f(X) \geq t] \leq \exp\Big(-\frac{2t^2}{\sum_i c_i^2}\Big). \tag{2.6}$$

The reason is easiest to see if you reveal the inputs one at a time. Before you see any input, your best guess of the result is its average. Each input you reveal updates the guess, and the i-th update lies in an interval of length at most c_i. After all inputs are revealed, the guess is the result itself. So the deviation from the average is a sum of n updates, each confined to a short interval. Such a sum concentrates the way Hoeffding's average does.

Take the training error of a fixed model, with losses between 0 and 1. Changing one example moves it by at most $1/n$. McDiarmid's inequality then gives back Hoeffding's bound. Its real use is for functions that are not averages, such as the error of a model retrained on the data. Algorithmic stability and generalization in the full edition uses it that way. Note that c_i must hold for every possible change to the i-th input, not just for typical ones, because the argument charges each input its largest possible move. This per-record influence returns in later parts under the name sensitivity.

The last step of the argument above did not need the updates to be independent. It needed only two things: each update averages zero given everything seen before, and each lies in a short interval. A running total with the first property is called a **martingale**. **Azuma's inequality** bounds how far it drifts. Suppose the k-th step always lies in an interval of length d_k. Then the total drifts by t or more with probability at most $\exp(-2t^2/\sum_k d_k^2)$. This is McDiarmid's bound, with step lengths in place of influences.

Azuma's inequality covers procedures that adapt as they go. Flip a coin 200 times. Before each flip, choose a coin that lands heads with probability 0.7 if the previous flip was heads, and one with probability 0.3 otherwise. The flips depend on each other, so Hoeffding's inequality does not apply. What you can bound is the drift: the number of heads, minus the number you would expect from the coins you chose. Given the past, each step of this drift averages zero and lies in an interval of length 1. Azuma's inequality bounds the chance of a drift of 20 or more by 1.83×10^{-2}. Simulation gives 1.12×10^{-3} (Figure 2.4).

The bound is loose for the same reason as Hoeffding's. It assumes that each step has the largest variance its interval allows, which is 0.25. The true variance of each step here is 0.21.

2.7. Cheat sheet

Table 2.2.: The widths of this chapter's bounds.

Bound	What you need to know	Width at failure probability β
Markov	$X \geq 0$, mean μ	μ/β
Chebyshev	Variance σ^2	$\sigma/\sqrt{\beta}$ (two-sided)
Normal noise $\mathcal{N}(0, \sigma^2)$	The distribution	$\sigma\sqrt{2\ln(1/\beta)}$; k draws: $\sigma\sqrt{2\ln(k/\beta)}$
Laplace noise $\mathrm{Lap}(b)$	The distribution	$b\ln(1/\beta)$; k draws: $b\ln(k/\beta)$

Bound	What you need to know	Width at failure probability β
Chernoff	A count of independent outcomes, mean μ	$\approx \sqrt{2\mu \ln(1/\beta)}$ for small deviations
Hoeffding	An average of n independent values in $[a, b]$	$(b-a)\sqrt{\ln(1/\beta)/(2n)}$ (one-sided)
McDiarmid, Azuma	Influences c_i or step lengths d_i	$\sqrt{\sum_i c_i^2 \ln(1/\beta)/2}$

Rules of thumb

- If you know only the variance, the width grows like $1/\sqrt{\beta}$. A normal-shaped tail grows like $\sqrt{\ln(1/\beta)}$, and an exponential tail like $\ln(1/\beta)$.
- To control k quantities at once, add $\ln k$ inside the logarithm.
- If the variance is far below what the range allows, use Chernoff, Bernstein or an exact interval, not Hoeffding.

2.8. Further reading

- Mitzenmacher & Upfal, *Probability and Computing*, ch. 4: the clearest Chernoff chapter, with computer-science examples.
- Vershynin, *High-Dimensional Probability* (2018), ch. 2: sub-Gaussian and sub-exponential variables in depth.
- Boucheron, Lugosi & Massart, *Concentration Inequalities* (2013), ch. 2 and 6: reference depth, including Bernstein's inequality.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

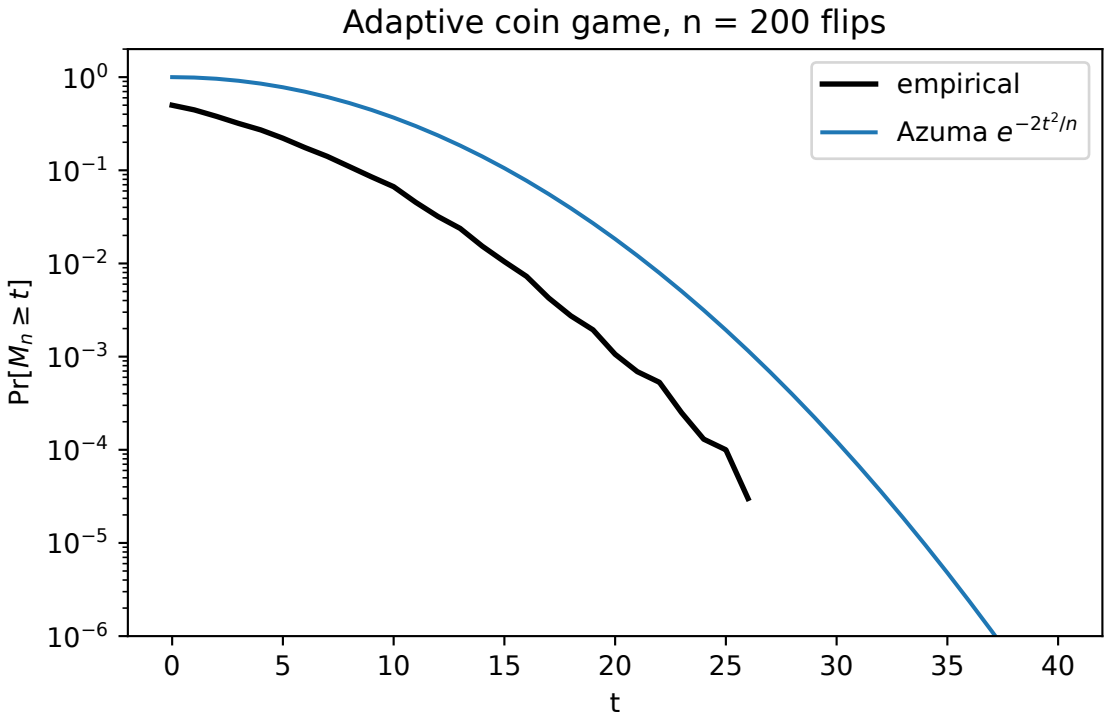


Figure 2.4.: Empirical tail of the drift in the adaptive coin game against the Azuma bound.

3. Divergences

3.1. Why this matters

How far is a fitted model from the truth? How many samples reveal a drift? Later parts ask the same about an algorithm's outputs on two different inputs. Each time you need one number for **how different two distributions P and Q are**, and there are several candidates because there are several questions.

All of them summarize the **likelihood ratio** $r(x) = p(x)/q(x)$, how many times more likely an outcome is under P than under Q; its logarithm $L(x) = \ln r(x)$ is the evidence for P, in nats. The running example is a die, fair (Q) or loaded so that a six comes up half the time (P). A six has $r = 0.5/(1/6) = 3$; any other roll has $r = 0.1/(1/6) = 0.6$.

3.2. KL divergence

Roll the loaded die many times and add up the evidence $\sum_i \ln r(x_i)$. Sixes are frequent, so the sum drifts upward. That rate, the average evidence per observation from P, is the **Kullback–Leibler (KL) divergence**,

$$D_{\mathrm{KL}}(P\|Q) = \mathbb{E}_P\left[\ln \frac{p(X)}{q(X)}\right] = \sum_x p(x) \ln \frac{p(x)}{q(x)}. \tag{3.1}$$

Independent observations multiply their likelihoods, so their evidence adds: n samples carry n times the KL of one. And since the drift grows like n while its fluctuation grows like $\sqrt{n}$, telling the two apart takes on the order of $1/\mathrm{KL}$ samples. For the die, $D_{\mathrm{KL}}(P\|Q) = 0.2939$ per roll.

KL is never negative and is zero only when $P = Q$, but it averages over its first argument, so it is asymmetric. The fair die proves its innocence more slowly, $D_{\mathrm{KL}}(Q\|P) = 0.2426$, because its evidence is the absence of sixes.

The direction also shapes fitted models. Maximum likelihood minimizes the forward KL, averaged over the data, so wherever the data go and the model has almost no mass, $\ln(p/q)$ explodes: the fit must cover everything. Variational inference minimizes the reverse KL, averaged over the model, which only punishes mass where the data aren't. Fitting one Gaussian to two separated bumps, the forward fit spreads over both and the reverse fit locks onto one; the second has forward KL 15.307, against 0.724 for the first. If Q gives zero probability to anything P can produce, that outcome is infinite evidence and KL is infinite.

For two normals with the same variance and means μ standard deviations apart, $D_{\mathrm{KL}} = \mu^2/2$; for $\mathrm{Lap}(0, b)$ against $\mathrm{Lap}(s, b)$, with $t = s/b$, $D_{\mathrm{KL}} = t + e^{-t} - 1$. The normal's is quadratic in the shift, which is why small drifts need many samples.

3.3. Rényi divergence and the worst case

KL is the average of $\ln r$, so $e^{D_{\mathrm{KL}}}$ is the geometric mean of r. Other means weight large ratios more: for the die under P, the geometric mean is 1.3416, the arithmetic mean 1.8000, the largest value 3.0000. The **Rényi divergence** of order α is the logarithm of the power mean of r with exponent $\alpha - 1$: for $\alpha > 0$, $\alpha \neq 1$,

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \mathbb{E}_Q[r^\alpha] = \frac{1}{\alpha - 1} \ln \sum_x p(x)^\alpha q(x)^{1-\alpha}. \tag{3.2}$$

The order is a dial: $\alpha \to 1$ gives KL, larger α weighs large ratios more, so D_α never decreases, and $\alpha \to \infty$ gives the **max-divergence** $D_\infty(P\|Q) = \ln \max_x r(x)$, the largest log ratio $\ln P(A)/Q(A)$ of any event. Every order adds over samples.

The normal pair's log ratio is a straight line, so higher orders keep finding larger ratios further out: $D_\alpha = \alpha\mu^2/2$ and $D_\infty = \infty$. The Laplace pair's is capped at t, so D_α levels off at $D_\infty = t$:

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \left(\frac{\alpha}{2\alpha - 1} e^{(\alpha-1)t} + \frac{\alpha - 1}{2\alpha - 1} e^{-\alpha t} \right) \qquad (\alpha \neq \tfrac{1}{2}, 1). \tag{3.3}$$

At equal variance, the Laplace pair is further apart at low orders, because its sharp peak moves visibly, and the normal pair above order 2.98, where its unbounded tail takes over (Table 3.1, Figure 3.1). "More different" must name the order.

Table 3.1.: Rényi divergences of a Laplace pair and a normal pair with the same shift and variance.

	KL	$D_{0.5}$	D_2	D_{10}	D_∞
Laplace, $t = 1$	0.3679	0.1891	0.6191	0.9287	1.0000
Normal, same variance	0.2500	0.1250	0.5000	2.5000	∞

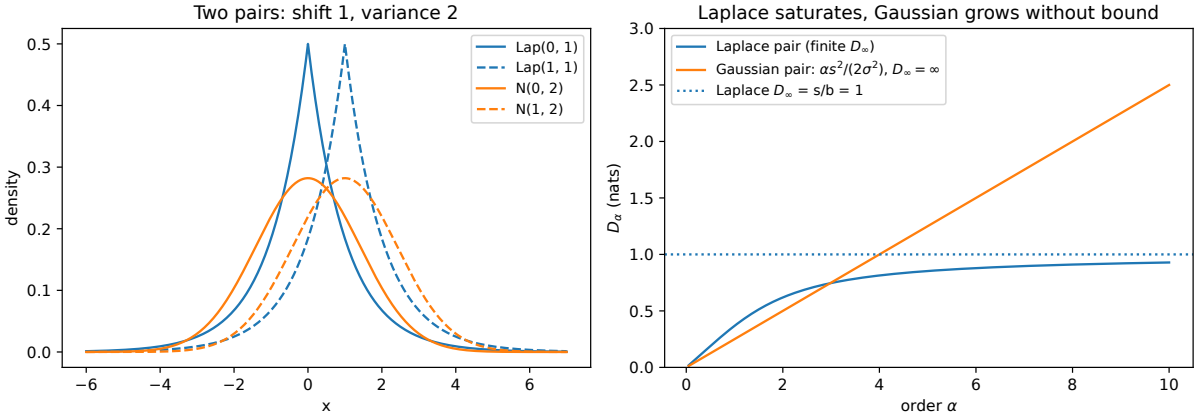


Figure 3.1.: Left: a Laplace pair and a normal pair, both with shift 1 and variance 2. Right: their Rényi divergences against the order α. The normal pair's grows linearly without bound; the Laplace pair's levels off at its max-divergence, 1.

Unbounded is not frequent, and Rényi divergences say how rare large evidence is, which an average cannot. Markov's inequality applied to r^α, whose Q-average is what D_α measures, gives for any $\alpha > 0$

$$Q[L \geq c] \leq \exp\big(-\alpha c + (\alpha - 1) D_\alpha(P\|Q)\big). \tag{3.4}$$

For the normal pair with $\mu = 1$, a draw from Q gives at least 2 nats of evidence for P with probability 0.0062. The bound gives 0.1353 at $\alpha = 1$, valid for any pair, and 0.0439 at $\alpha = 2.5$.

3.4. Total variation and the hockey-stick divergence

Draw p and q on the same axes. Where they overlap, an observation can't tell you which one produced it. **Total variation (TV)** is the mass that doesn't overlap:

$$\mathrm{TV}(P, Q) = \max_A |P(A) - Q(A)| = \frac{1}{2} \sum_x |p(x) - q(x)|. \tag{3.5}$$

The best single guess says "P" exactly where $p > q$, so with equal priors it is right with probability $\frac{1}{2} + \frac{1}{2}\mathrm{TV}$, and no classifier does better. The mean of any statistic with values in $[a, b]$ likewise moves by at most $(b - a)\,\mathrm{TV}$. For the normal pair, with Φ the standard normal cumulative distribution function (CDF), $\mathrm{TV} = 2\Phi(\mu/2) - 1$; at $\mu = 1$ it is 0.3829, so the best guess is right with probability 0.6915.

TV is capped at 1 and does not add over samples. A shift of a tenth of a standard deviation has TV 0.0399 in one sample, yet the best test on 400 samples is right with probability 0.8413. So compute KL or Rényi for n samples, where they add, and convert at the end (Section 3.5).

TV counts all excess mass, however weak its evidence. To keep only strong evidence, scale q up by $\gamma \geq 1$ and measure the mass of p still above γq, the **hockey-stick divergence**:

$$E_\gamma(P\|Q) = \max_A \big[P(A) - \gamma Q(A)\big] = \sum_x \big(p(x) - \gamma q(x)\big)_+, \tag{3.6}$$

with $(y)_+ = \max(y, 0)$. Only outcomes with ratio above γ count, so $E_1 = \mathrm{TV}$, E_γ falls as γ grows, and it reaches zero at $\gamma = e^{D_\infty}$. For the die only the six counts, and $E_\gamma = 0.5 - \gamma/6$ hits zero at $\gamma = 3$. For the Laplace pair, $E_\gamma = 1 - e^{-(t - \ln\gamma)/2}$ up to $\gamma = e^t$, zero beyond. The normal pair never reaches zero,

$$E_\gamma = \Phi\Big(\frac{\mu}{2} - \frac{\ln\gamma}{\mu}\Big) - \gamma\,\Phi\Big(-\frac{\mu}{2} - \frac{\ln\gamma}{\mu}\Big), \tag{3.7}$$

giving $E_2 = 0.1906$ and $E_4 = 0.0692$ at $\mu = 1$ (Figure 3.2): its large ratios exist but carry little mass.

3.5. Data processing and Pinsker's inequality

Binning, thresholds and added noise map several outcomes to one, and the output's likelihood ratio is then an average of theirs. Report the die's parity, and the six (ratio 3) joins the two and four (0.6 each) in an even result with ratio $(3 + 0.6 + 0.6)/3 = 1.4$. Averaging pulls strong evidence toward weak, and a divergence, a convex summary of the ratios, can only shrink. That is the **data processing inequality (DPI)**: if both inputs go through the same channel K, even

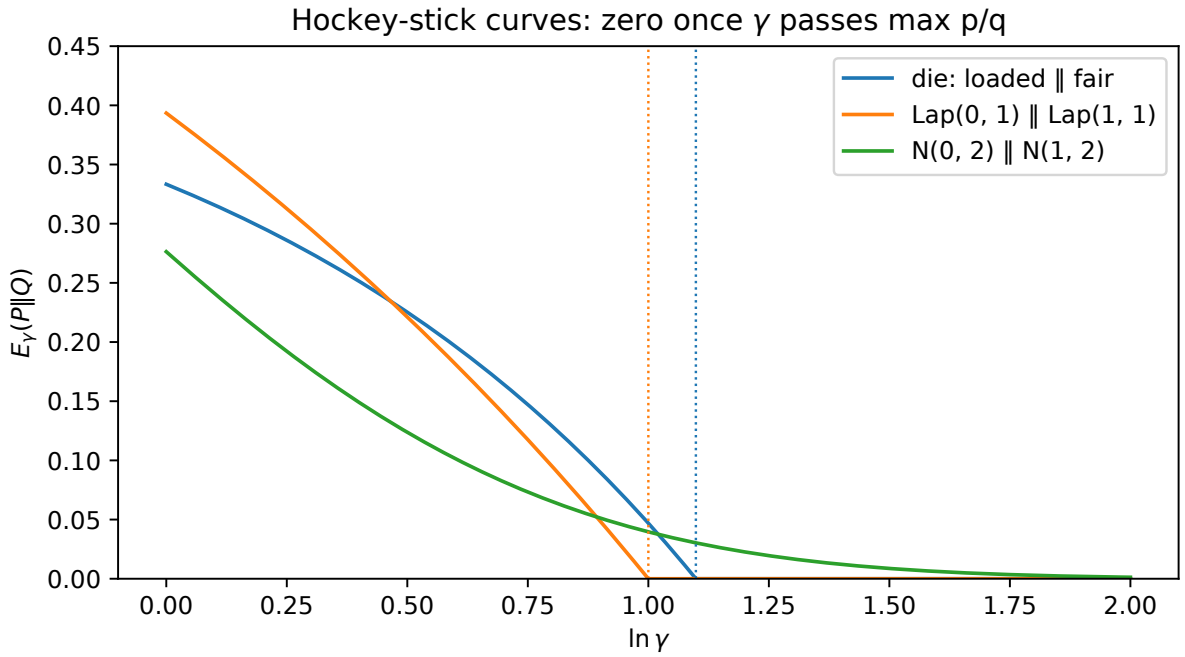


Figure 3.2.: The hockey-stick divergence $E_\gamma(P\|Q)$ against $\ln\gamma$ for the loaded die, a Laplace pair and a normal pair with the same variance. The die and Laplace curves reach zero at their max-divergences; the normal curve stays positive.

a randomized one, then $D(PK\|QK) \le D(P\|Q)$ for KL, TV, E_γ, every D_α and D_∞, and every **f-divergence** $\mathbb{E}_Q[f(r)]$ with f convex and $f(1) = 0$.

Nothing is lost when processing keeps the ratio: parity drops the die's KL from 0.2939 to 0.0823, while "is it a six?" keeps all of it. Thresholding draws from $\mathcal{N}(0,1)$ or $\mathcal{N}(1,1)$ at the midpoint lowers KL from 0.5 to 0.3090 but keeps all of the TV, since it is the best test. The DPI bounds the best test only: processing can help a bad classifier, never beat the best test on raw data.

Pinsker's inequality turns the KL you can compute into a bound on the TV you want to report, and Bretagnolle–Huber stays below 1:

$$\mathrm{TV} \le \sqrt{\tfrac{1}{2} D_{\mathrm{KL}}}, \qquad \mathrm{TV} \le \sqrt{1 - e^{-D_{\mathrm{KL}}}}. \tag{3.8}$$

Pinsker is tight for nearly equal distributions but passes 1 at KL = 2; Bretagnolle–Huber is better above 1.59 (Table 3.2). There is no converse: a tiny TV is compatible with infinite KL.

Table 3.2.: Exact total variation between n samples of $\mathcal{N}(0,1)$ and of $\mathcal{N}(0.1,1)$, and the two bounds computed from KL.

n	KL of n samples	Exact TV	Pinsker	Bretagnolle–Huber
100	0.50	0.3829	0.5000	0.6273
400	2.00	0.6827	1.0000	0.9299

3.6. Code

The closed forms for the two shifted pairs need only the standard library:

```
from math import erfc, exp, log, sqrt


def normal_cdf(x: float) -> float:
    return 0.5 * erfc(-x / sqrt(2))


def gauss_kl(mu: float) -> float:
    """KL between N(0, s^2) and N(mu * s, s^2): half the squared shift in standard deviation
    return mu**2 / 2


def gauss_tv(mu: float) -> float:
    return 2 * normal_cdf(mu / 2) - 1


def gauss_hockey(mu: float, gamma: float) -> float:
    """E_gamma between N(0, 1) and N(mu, 1)."""
    c = log(gamma)
    return normal_cdf(mu / 2 - c / mu) - gamma * normal_cdf(-mu / 2 - c / mu)


def laplace_kl(t: float) -> float:
    """KL between Lap(0, b) and Lap(t * b, b)."""
    return t + exp(-t) - 1


def laplace_renyi(alpha: float, t: float) -> float:
    """Renyi divergence of order alpha (not 1/2 or 1) between Lap(0, b) and Lap(t * b, b).""
    inner = alpha / (2 * alpha - 1) * exp((alpha - 1) * t) + (alpha - 1) / (2 * alpha - 1) *
    return log(inner) / (alpha - 1)


def laplace_tv(t: float) -> float:
    return 1 - exp(-t / 2)


print(f"{gauss_kl(1.0):.4f} {gauss_tv(1.0):.4f} {gauss_hockey(1.0, 2.0):.4f}")  # 0.5000 0.3
print(f"{laplace_kl(1.0):.4f} {laplace_renyi(2.0, 1.0):.4f} {laplace_tv(1.0):.4f}")  # 0.367
print(f"{laplace_renyi(10.0, 1.0):.4f}")  # 0.9287
```

The full edition's script, divergences.py, checks every closed form against simulation.

3.7. Cheat sheet

Table 3.3.: The divergences of this chapter, with closed forms for $\mathcal{N}(0,\sigma^2)$ against $\mathcal{N}(s,\sigma^2)$ ($\mu = s/\sigma$) and $\mathrm{Lap}(0,b)$ against $\mathrm{Lap}(s,b)$ ($t = s/b$).

Divergence	Definition	Normal pair, shift μ	Laplace pair, shift t	Use it for
KL	$\mathbb{E}_P[\ln r]$	$\mu^2/2$	$t + e^{-t} - 1$	Average evidence; sample sizes ($\approx 1/\mathrm{KL}$)
Rényi D_α	$\frac{1}{\alpha-1}\ln \mathbb{E}_Q[r^\alpha]$	$\alpha\mu^2/2$	Equation 3.3	From typical to worst-case evidence; tail of L
Max-divergence D_∞	$\ln \max r$	∞	t	Worst-case ratio over all events
TV	$\frac{1}{2}\sum\lvert p-q\rvert$	$2\Phi(\mu/2) - 1$	$1 - e^{-t/2}$	Best single guess: $\frac{1}{2} + \frac{1}{2}$TV
Hockey-stick E_γ	$\sum(p - \gamma q)_+$	Equation 3.7	$\left(1 - e^{-(t-\ln\gamma)/2}\right)_+$	Mass of large ratios; \$E_1 = \$ TV

Rules of thumb

- Compute in KL or Rényi, where independent samples add; convert to TV at the end.
- A bounded likelihood ratio (Laplace) gives a finite max-divergence; an unbounded one (normal) gives $D_\infty = \infty$, though for the normal pair every finite order stays finite.
- Processing never increases a divergence.

3.8. Further reading

- van Erven & Harremoës, "Rényi Divergence and Kullback–Leibler Divergence" (IEEE Transactions on Information Theory, 2014): the Rényi family and its properties.
- Cover & Thomas, *Elements of Information Theory* (2nd ed., 2006), ch. 2 and §11.6: KL and Pinsker's inequality.
- Polyanskiy & Wu, *Information Theory: From Coding to Learning* (2025), ch. 7: f-divergences and data processing at reference depth.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

4. Hypothesis testing

4.1. Why this matters

A spam filter, a fraud detector and a drift monitor all face one problem: **an observation came from one of two distributions, and you must decide which.** Later parts ask it about any test anyone could run, so you need the best test, how good it can be, and how to measure a given test from finite data.

The answer runs through the **likelihood ratio** $\Lambda(x) = q(x)/p(x)$, how many times more likely the observation is under the alternative Q than under the default P. The best test thresholds it, and sliding the threshold traces a curve of the best achievable errors that belongs to P and Q alone. Most of what goes wrong in practice comes from summarizing that curve by one number and losing the corner where false alarms are rare.

4.2. Errors and the Neyman–Pearson lemma

Test the null hypothesis $H_0 : X \sim P$ against the alternative $H_1 : X \sim Q$ with a rule $\phi(x) \in [0, 1]$, the probability of rejecting H_0 on seeing x. Its **false positive rate (FPR)**, or type I error, is $\alpha = \mathbb{E}_P[\phi]$; its **type II error** is $\beta = 1 - \mathbb{E}_Q[\phi]$, and $1 - \beta$ is its **power** or **true positive rate (TPR)**. The two pull against each other: rejecting one more outcome raises α and lowers β. A coin that rejects with probability α ignoring the data has $\beta = 1 - \alpha$; a useful test beats it.

For the most power on a fixed false-alarm budget, note that rejecting an outcome x costs $p(x)$ of budget and buys $q(x)$ of power, an exchange rate of $\Lambda(x)$. So spend the budget on the best rates first: reject where $\Lambda(x)$ is above a cut-off η. That is the **Neyman–Pearson lemma**: the test that rejects when $q(x) > \eta\, p(x)$, keeps H_0 when $q(x) < \eta\, p(x)$, and randomizes on ties so its FPR is exactly α, has the smallest β of any test with FPR at most α.

A spam filter counts suspicious words, capped at 3 (Table 4.1). The ratio increases with the count, so the best tests are thresholds on it, but on discrete data a threshold only reaches a few FPRs: rejecting counts of 3 spends 0.1, counts of 2 or 3 spend 0.3. To spend a budget of exactly 0.25, reject a count of 3 always and a count of 2 with probability 0.75, for power 0.625. No test does better: the best of 200,000 random tests with FPR at most 0.25 reaches 0.587.

Table 4.1.: Word-count distributions under ham (P) and spam (Q), and their likelihood ratio.

x	0	1	2	3
$p(x)$, ham	0.4	0.3	0.2	0.1
$q(x)$, spam	0.1	0.2	0.3	0.4
$\Lambda(x) = q(x)/p(x)$	0.250	0.667	1.500	4.000

Whenever Λ increases with a statistic, "ratio above η" is "statistic above some t", which is why thresholding a score works. A calibrated score $\Pr[H_1 \mid x]$ increases with $\Lambda(x)$, so thresholding it is optimal; a score that ranks outcomes differently from Λ can be beaten by re-ranking.

4.3. Trade-off curves

A single test is one point (α, β). To say how hard it is to tell P from Q, take the best β at every α: the **trade-off function** $T(\alpha) = \inf\{\beta_\phi : \alpha_\phi \le \alpha\}$. It lies below the coin flip's $1 - \alpha$, and the further below, the easier the problem. Its slope is minus the likelihood ratio at the threshold, since rejecting a little more of x trades $p(x)$ of FPR for $q(x)$ of power. The best test adds outcomes in decreasing order of Λ, so the curve flattens as it goes and is convex. Plotted as power against FPR, it is the optimal **receiver operating characteristic (ROC)** curve.

For $\mathcal{N}(0, 1)$ against $\mathcal{N}(\mu, 1)$, with Φ the standard normal cumulative distribution function (CDF), the best test thresholds x, and

$$T(\alpha) = \Phi(\Phi^{-1}(1 - \alpha) - \mu). \tag{4.1}$$

With n draws, the log ratios add up to a function of the sample mean, which shifts by δ with standard deviation $\sigma/\sqrt{n}$, so the same curve applies with $\mu = \delta\sqrt{n}/\sigma$: distinguishability grows like $\sqrt{n}$. A monitor that averages 100 readings to detect a drift of 0.2σ has $\mu = 2$, power 0.6388 at $\alpha = 0.05$, and only 0.1378 at 10^{-3}. Solving for n gives the sample size for power $1 - \beta'$ at level α,

$$n \ge \left(\frac{\Phi^{-1}(1 - \alpha) + \Phi^{-1}(1 - \beta')}{\delta/\sigma} \right)^2, \tag{4.2}$$

so power 0.9 at $\alpha = 0.01$ for $\delta/\sigma = 0.25$ needs 209 readings.

Near $\alpha = 0$ the normal curve is vertical, because the normal likelihood ratio is unbounded far out: the first sliver of false-alarm budget buys power at an unbounded rate. A Laplace shift is different. For $\mathrm{Lap}(0, b)$ against $\mathrm{Lap}(s, b)$, with $\kappa = s/b$, the ratio is clipped to $[e^{-\kappa}, e^{\kappa}]$, so the ends of the curve are straight lines with those slopes:

$$T(\alpha) = \begin{cases} 1 - e^{\kappa}\alpha, & \alpha < \frac{1}{2}e^{-\kappa}, \\ e^{-\kappa}/(4\alpha), & \frac{1}{2}e^{-\kappa} \le \alpha \le \frac{1}{2}, \\ e^{-\kappa}(1 - \alpha), & \alpha > \frac{1}{2}. \end{cases} \tag{4.3}$$

No test can buy power faster than e^{κ} per unit of FPR, so no test has power above $e^{\kappa}\alpha$. Table 4.2 compares a Laplace pair ($\kappa = 1$) with a normal pair ($\mu = 1.0301$) with the same best advantage, 0.3935 (Section 4.4). At $\alpha = 10^{-6}$ the normal pair is 36 times easier, because its unbounded ratio lets a test find rare, overwhelming evidence (Figure 4.1). Plan for the level you will use.

Table 4.2.: Best power at small false positive rates for a Laplace pair and a normal pair with the same best advantage.

α	Laplace power, $\kappa = 1$	Normal power, $\mu = 1.0301$
10^{-3}	2.718×10^{-3}	1.969×10^{-2}
10^{-6}	2.718×10^{-6}	9.829×10^{-5}

α	Laplace power, $\kappa = 1$	Normal power, $\mu = 1.0301$

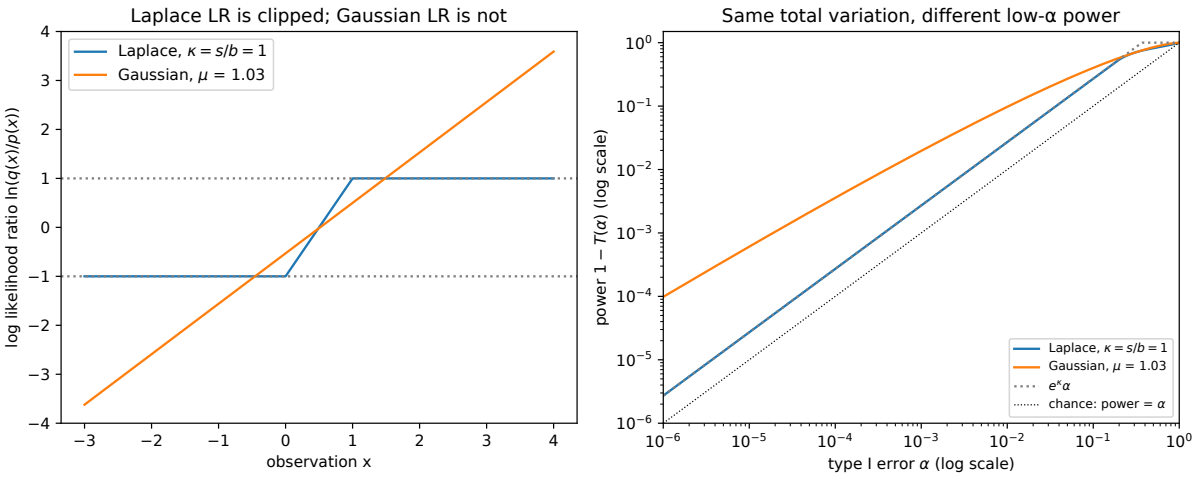


Figure 4.1.: Left: the log likelihood ratio of $\mathrm{Lap}(1,1)$ against $\mathrm{Lap}(0,1)$ is clipped at ± 1, while the normal one with the same best advantage is a straight line. Right: best power on log axes. The Laplace curve follows $e\alpha$ for small α; the normal curve rises far above it.

4.4. One-number summaries

The **advantage** of a test, $\mathrm{TPR} - \mathrm{FPR}$, is how much more often it fires under Q than under P. Its maximum over all tests is the **total variation (TV)** distance $\frac{1}{2}\sum_x |q(x) - p(x)|$, reached by rejecting where $q > p$: $2\Phi(\mu/2) - 1$ for the normal pair and $1 - e^{-\kappa/2}$ for the Laplace pair. But that is one point of the curve, at $\eta = 1$, and the two pairs above share it, 0.3935, despite their 36-fold gap at small α.

The **area under the curve (AUC)** of a ROC curve is the probability that a random positive scores above a random negative. It weights all FPRs equally, so the region below 10^{-3} barely moves it. If negatives score $\mathcal{N}(0,1)$ and positives $\mathcal{N}(m, \sigma^2)$, then $\mathrm{AUC} = \Phi(m/\sqrt{1+\sigma^2})$ and

$$\mathrm{TPR}(\mathrm{FPR}) = 1 - \Phi\Big(\frac{\Phi^{-1}(1 - \mathrm{FPR}) - m}{\sigma}\Big). \tag{4.4}$$

Detector A has positives scoring $\mathcal{N}(1,1)$, detector B $\mathcal{N}(1.5811, 2^2)$. Both have AUC 0.7602, but at an FPR of 10^{-3} their TPRs are 0.0183 and 0.2253: B catches 12 times as many positives (Figure 4.2). Report the TPR at a fixed low FPR chosen by the application, and plot ROC curves on log–log axes, where that corner is visible.

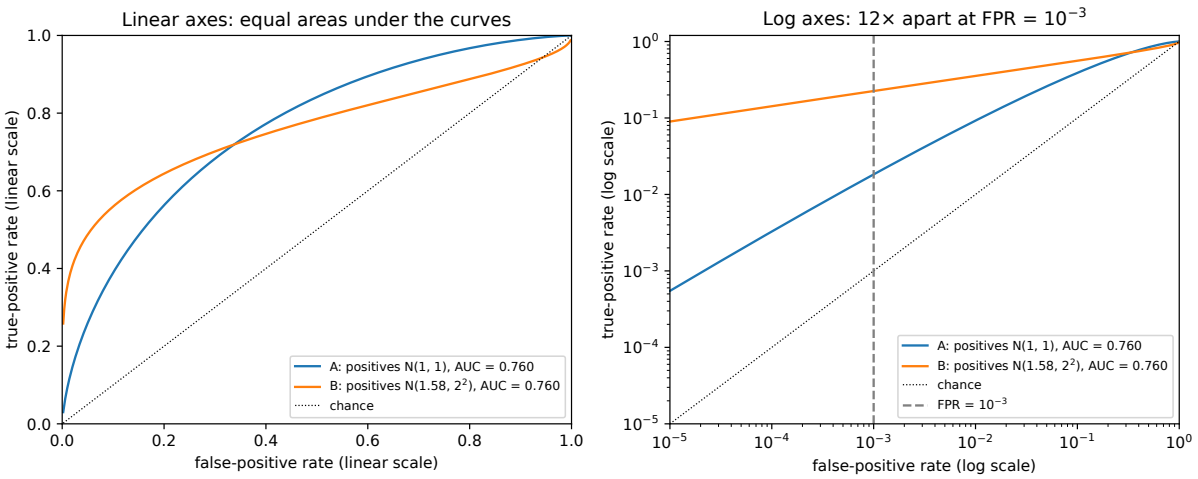


Figure 4.2.: ROC curves of two detectors with the same AUC of 0.760. Left, linear axes: the curves cross and look comparable. Right, log axes: at an FPR of 10^{-3} detector B has 12 times the TPR of detector A.

4.5. Base rates: the Bayesian view

With a known base rate, the useful question is how likely H_1 is given the observation, and the likelihood ratio answers it in one multiplication. With prior probability π for H_1,

$$\frac{\Pr[H_1 \mid x]}{\Pr[H_0 \mid x]} = \frac{\pi}{1-\pi} \times \Lambda(x). \tag{4.5}$$

If all you know is that the test fired, the likelihood ratio of that event is TPR/FPR. Use the $\mu = 2$ test where 1% of cases are positive. At an FPR of 0.05 an alarm has ratio $0.6388/0.05 = 12.8$, and the chance it is real is only 0.114. At 10^{-3} the ratio is 137.8 and the chance is 0.582. When positives are rare, the FPR, not the power, decides whether an alarm is worth acting on.

4.6. Measuring rates from finite trials

A measured FPR is k false positives among n negatives, a binomial count. A count of k fluctuates by about $\sqrt{k}$, so the relative uncertainty, about $1/\sqrt{k}$, is set by the number of events. The **Clopper–Pearson** interval at confidence $1-a$ is the range of rates $[l, u]$ that make the observed count unsurprising: $\Pr[\mathrm{Bin}(n, l) \geq k] = a/2$ and $\Pr[\mathrm{Bin}(n, u) \leq k] = a/2$. It covers the true rate with probability at least $1-a$. With no events the 95% upper bound is about $3/n$, the **rule of three**. Measuring an FPR of α to within a factor of about 1.2 takes some 100 expected events, so about $100/\alpha$ negatives.

Evaluate the $\mu = 2$ monitor at its 10^{-3} threshold on 10,000 negatives and 10,000 positives. One run gives 1,350 true positives, interval $[0.1284, 0.1419]$, but only 10 false positives, interval $[0.00048, 0.00184]$: the FPR is known only to within a factor of two. To certify an FPR below 10^{-3} at 95% confidence you need 2,995 negatives with no false positive at all.

The normal-approximation (Wald) interval is symmetric, too narrow for a skewed count, and collapses to $[0, 0]$ at $k = 0$. Hoeffding's interval uses only the range and ignores the tiny variance: its half-width here is 0.0136, over 13 times the rate. And a threshold tuned on the same data has a true FPR as uncertain as that interval, so choose it on one split and measure on another.

4.7. Code

The normal trade-off curve and the Clopper–Pearson interval need only the standard library; bisection on the upper tail keeps precision when α is tiny:

```
from collections.abc import Callable
from math import erfc, exp, lgamma, log, log1p, sqrt


def normal_cdf(x: float) -> float:
    return 0.5 * erfc(-x / sqrt(2))


def upper_quantile(a: float) -> float:
    """The point a standard normal exceeds with probability a, found by bisection on the tai
    lo, hi = -40.0, 40.0
    for _ in range(200):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if 0.5 * erfc(mid / sqrt(2)) > a else (lo, mid)
    return lo


def gaussian_power(alpha: float, mu: float) -> float:
    """Best power at false positive rate alpha between N(0, 1) and N(mu, 1)."""
    return 1 - normal_cdf(upper_quantile(alpha) - mu)


def binom_cdf(k: int, n: int, p: float) -> float:
    """Pr[Binomial(n, p) <= k], summed in log space."""
    if p <= 0.0:
        return 1.0
    if p >= 1.0:
        return 1.0 if k >= n else 0.0
    total = 0.0
    for i in range(k + 1):
        log_pmf = lgamma(n + 1) - lgamma(i + 1) - lgamma(n - i + 1) + i * log(p) + (n - i) *
        total += exp(log_pmf)
    return total


def clopper_pearson(k: int, n: int, confidence: float = 0.95) -> tuple[float, float]:
```

```
    """Exact two-sided interval for a rate after k successes in n trials."""
    a = (1 - confidence) / 2

    def solve(f: Callable[[float], float], target: float) -> float:
        """The rate p at which f, a binomial CDF that falls as p grows, equals target."""
        lo, hi = 0.0, 1.0
        for _ in range(100):
            mid = (lo + hi) / 2
            lo, hi = (mid, hi) if f(mid) > target else (lo, mid)
        return lo

    # Lower end: Pr[Bin(n, l) >= k] = a, that is Pr[Bin(n, l) <= k - 1] = 1 - a.
    lower = 0.0 if k == 0 else solve(lambda p: binom_cdf(k - 1, n, p), 1 - a)
    # Upper end: Pr[Bin(n, u) <= k] = a.
    upper = 1.0 if k == n else solve(lambda p: binom_cdf(k, n, p), a)
    return lower, upper


print(f"{gaussian_power(0.05, 2.0):.4f} {gaussian_power(1e-3, 2.0):.4f}")  # 0.6388 0.1378
low, high = clopper_pearson(10, 10_000)
print(f"[{low:.5f}, {high:.5f}]")  # [0.00048, 0.00184]
print(f"{1 - 0.05 ** (1 / 3000):.3e}")  # 9.981e-04
```

The last line is the exact 95% upper bound after no events in 3,000 trials, close to the rule of three's $3/n$.

4.8. Cheat sheet

Table 4.3.: The results of this chapter.

Result	Statement
Errors of a test ϕ	$\alpha = \mathbb{E}_P\phi$ (FPR), $\beta = 1 - \mathbb{E}_Q\phi$, power $= 1 - \beta$ (TPR)
Neyman–Pearson	Rejecting where $q/p > \eta$, randomizing at ties, is most powerful at its level
Normal shift μ	$T(\alpha) = \Phi(\Phi^{-1}(1-\alpha) - \mu)$; n averaged draws: $\mu = \delta\sqrt{n}/\sigma$
Laplace shift $\kappa = s/b$	Equation 4.3; power $\le e^{\kappa}\alpha$
Best advantage	$\max(\text{TPR} - \text{FPR}) = \text{TV}$; normal $2\Phi(\mu/2) - 1$, Laplace $1 - e^{-\kappa/2}$
AUC	$\Pr[S_+ > S_-]$; binormal $\Phi(m/\sqrt{1+\sigma^2})$
Bayes	Posterior odds = prior odds $\times\, \Lambda$; an alarm has $\Lambda = \text{TPR}/\text{FPR}$
Clopper–Pearson	$\Pr[\text{Bin}(n,l) \ge k] = \Pr[\text{Bin}(n,u) \le k] = a/2$; zero events: $u \approx 3/n$

Rules of thumb

- To find the best test, compute the likelihood ratio and threshold it.
- Compare detectors by their TPR at a fixed low FPR, on log axes; AUC and TV hide that regime.
- A bounded likelihood ratio caps power at small α: power $\leq \Lambda_{\max}\,\alpha$.
- To measure a rate near α, plan for about $100/\alpha$ trials and report a Clopper–Pearson interval.

4.9. Further reading

- Casella & Berger, *Statistical Inference* (2nd ed., 2002), §8.3: the Neyman–Pearson lemma and its one-sided extensions.
- Fawcett, "An introduction to ROC analysis" (Pattern Recognition Letters, 2006): ROC curves, AUC as a ranking probability, and convex hulls.
- Brown, Cai & DasGupta, "Interval Estimation for a Binomial Proportion" (Statistical Science, 2001): why the Wald interval fails and what to use instead.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

5. Information theory

5.1. Why this matters

How much does an observation tell you about something you cannot see, such as the word "offer" about whether an email is spam? There are two honest answers, because there are two questions. How many bits does the observation save you, on average, when you describe the hidden value? Shannon entropy and mutual information answer that. How much more likely is a single best guess to be right? Min-entropy and vulnerability answer that.

Every quantity here is the logarithm of a probability, and the two families differ only in where the logarithm sits: Shannon's quantities average the log over all outcomes, the one-guess quantities take the log of the best probability. Usually they agree. When a rare outcome reveals a lot they disagree by orders of magnitude, and knowing which to trust is the skill this chapter builds.

Logarithms are base 2, so quantities are in **bits**; one nat, the unit of natural logarithms, is $1/\ln 2 \approx 1.443$ bits.

5.2. Entropy

A measure of uncertainty should give a fair coin one unit and two independent fair coins two. Probabilities of independent events multiply, so to make uncertainties add you take a logarithm: an outcome of probability $p(x)$ has **surprisal** $\log(1/p(x))$, one bit for a fair coin's heads, nothing for a certainty. **Entropy** is the average surprisal,

$$H(X) = -\sum_x p(x) \log p(x), \qquad 0 \le H(X) \le \log|\mathcal{X}|, \tag{5.1}$$

zero for a constant and largest when all values are equally likely, since then no outcome is less surprising than another. For a coin with heads probability p it is the **binary entropy** $h(p)$. The **conditional entropy** $H(X \mid Y) = \sum_y p(y)\, H(X \mid Y = y)$ is the uncertainty left after seeing Y, averaged over what you might see.

Take a spam filter. One email in eight is spam; every spam contains "offer", and so does one ham email in seven. Before reading, $H(X) = h(1/8) = 0.5436$ bits. If the word is absent the email is certainly ham: 0 bits. If it is present, which happens a quarter of the time, spam and ham are equally likely: a full bit, more than before. On average the word still helps, $H(X \mid Y) = 0.25$ bits, because the common observation settles the question and the rare one only muddies it (Figure 5.1). So observing never increases uncertainty on average, $H(X \mid Y) \le H(X)$, but a particular observation can.

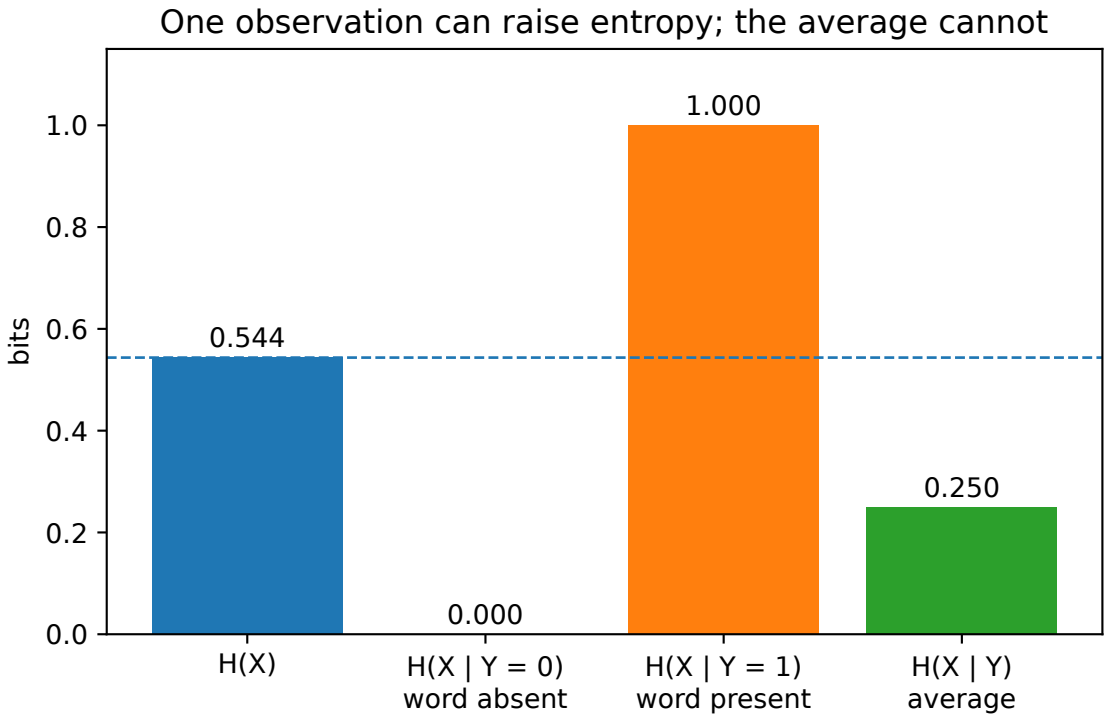


Figure 5.1.: Entropy of the spam label before observing the keyword, after each of the two possible observations, and on average. The rare observation raises the entropy above $H(X)$ (dashed line); the average falls below it.

5.3. Mutual information and data processing

What Y tells you about X is the drop in uncertainty it causes, the **mutual information**

$$I(X;Y) = H(X) - H(X \mid Y) = D(P_{XY} \,\|\, P_X P_Y). \tag{5.2}$$

The second form, a Kullback–Leibler (KL) divergence (Divergences in the full edition), says why it behaves as it does: Y helps exactly to the extent that the joint distribution differs from what independence would give. So it is non-negative, zero only under independence, and symmetric. For the keyword, $I(X;Y) = 0.5436 - 0.25 = 0.2936$ bits.

The **data processing inequality** says that if Z is computed from Y alone, possibly with fresh randomness, then $I(X;Z) \le I(X;Y)$. Z has no access to X except through Y, so whatever it knows about X it learned from Y, and the computation can only lose some on the way. Detect the keyword with a tokenizer that errs 10% of the time, and the feature keeps 0.1668 bits, 57% of the clean one. Denoising, feature extraction and training are all processing; they can make information easier to use, never larger. So a bound on the raw observation covers everything anyone computes from it.

But mutual information is an average over x and y, and a small average can hide a few outcomes on which Y gives X away completely (Section 5.5).

5.4. Min-entropy and one-guess leakage

Someone with one guess does not care how many bits describe a value on average, only how likely their best guess is: the probability of the most likely value. That is the **vulnerability**, and its log the **min-entropy**,

$$V(X) = \max_x p(x), \qquad H_\infty(X) = -\log V(X) \le H(X). \tag{5.3}$$

A uniform 4-digit PIN has 13.2877 bits of both. If 10% of people choose 1234, the Shannon entropy barely moves, to 12.4278 bits, because the other 90% is still spread evenly and dominates the average. The min-entropy falls to 3.3219 bits: one guess in ten succeeds.

If the guesser sees Y first, the best strategy is the **maximum a posteriori (MAP)** guess, the value most likely given y. On observation y it wins with probability $\max_x p(x \mid y)$, and weighting by $p(y)$ gives the **posterior vulnerability**, the sum of the largest entry in each column of the joint table:

$$V(X \mid Y) = \sum_y \max_x p(x,y). \tag{5.4}$$

The **min-entropy leakage** $\mathcal{L} = \log\big(V(X \mid Y)/V(X)\big)$ is the factor by which the observation multiplies the chance of a correct first guess, in bits: 1 bit doubles it. For four secret values with prior $(0.4, 0.3, 0.2, 0.1)$, observed through the channel of Section 5.7, a blind guess wins with probability 0.4 and the MAP guesser with 0.66 (Figure 5.2), a leakage of 0.7225 bits.

Because the formula is a success probability, two shortcuts are wrong. The guesser compares joint probabilities, not channel entries $C[x,y] = p(y \mid x)$: on y_3 it picks x_3 over x_4, which produces y_3 more often, because x_3 is twice as likely to begin with. And the log comes after averaging: averaging per-observation min-entropies gives 0.6548 bits instead of 0.5995, a number that is

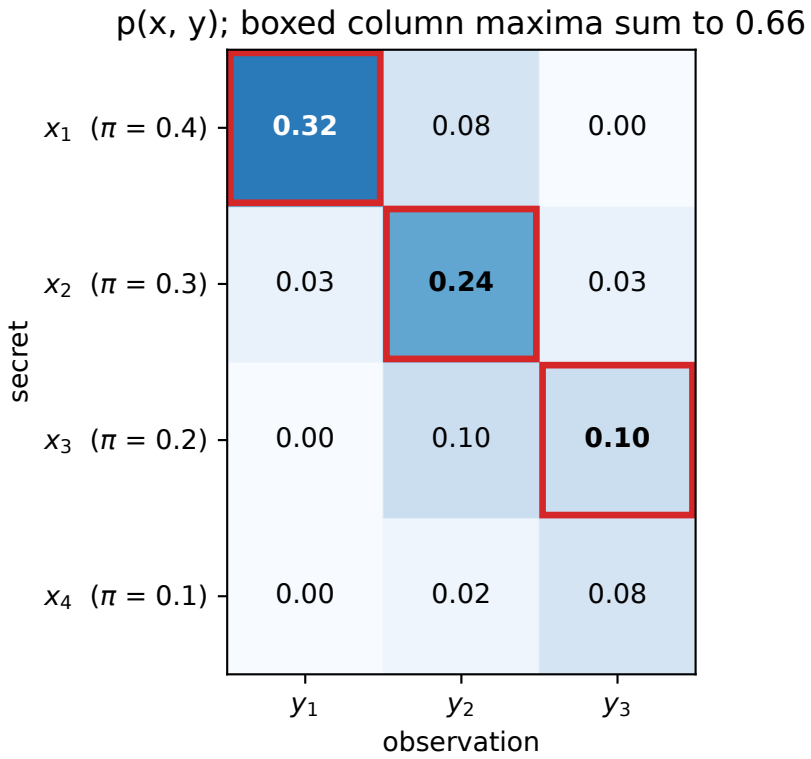


Figure 5.2.: The joint distribution $p(x, y) = \pi(x)C[x, y]$ of four secret values and three observations. The MAP guesser picks the boxed cell in each column, and its success probability is their sum $V(X \mid Y) = 0.66$.

no guesser's success rate. When you do not know the prior, use the worst case over priors, the **min-capacity** $\log \sum_y \max_x C[x, y]$, which needs only the channel: 1.2630 bits here.

5.5. Shannon against min-entropy leakage

Shannon leakage, the mutual information, and min-entropy leakage agree when every outcome reveals the same amount: reveal the lowest m bits of a uniform n-bit secret and both say m. They disagree when a channel reveals everything on a few outcomes and nothing on the rest. Suppose it reveals the whole secret when its lowest j bits are zero, with probability 2^{-j}. Shannon averages the log, so that outcome's n bits count with weight 2^{-j}: about $2^{-j}n$ in all. The guesser, though, now wins whenever the reveal happens, so the success rate jumps from 2^{-n} to more than 2^{-j}, a min-entropy leakage close to $n - j$ (Figure 5.3).

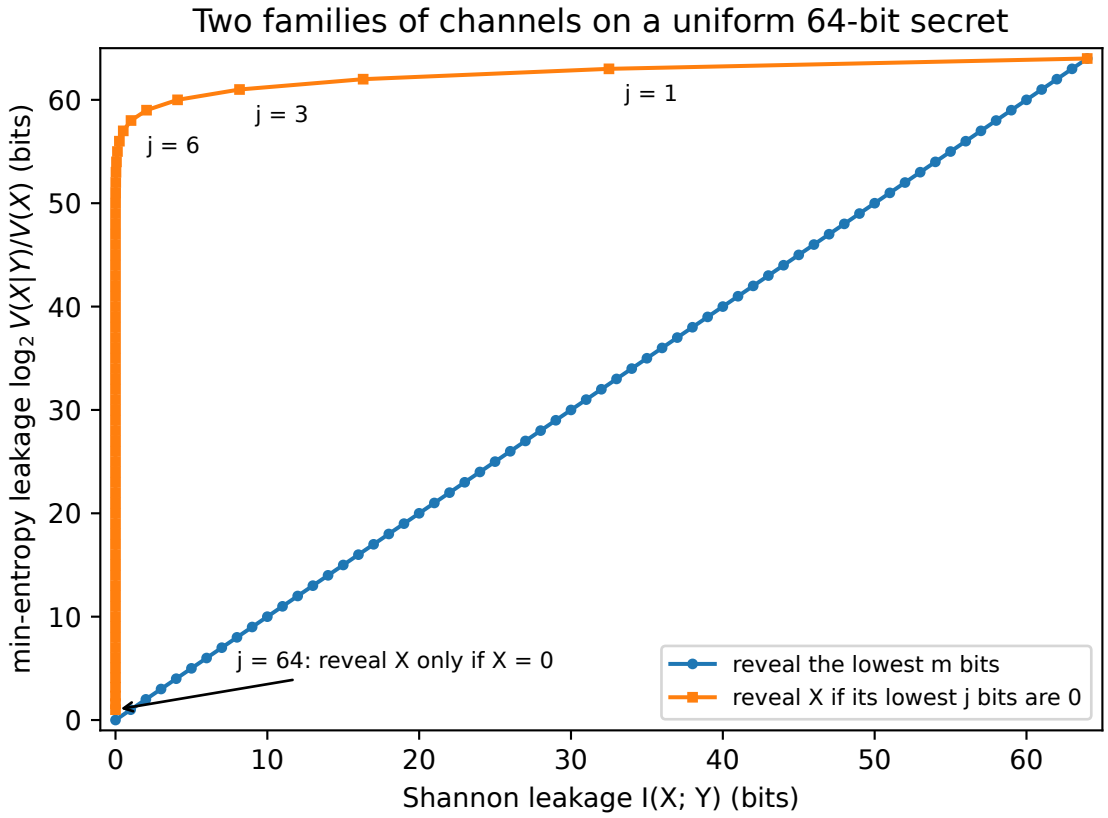


Figure 5.3.: Shannon leakage against min-entropy leakage for two families of deterministic channels on a uniform 64-bit secret. Revealing the low m bits gives equal leakages (diagonal). Revealing the whole secret with probability 2^{-j} gives a small Shannon leakage but a min-entropy leakage close to $64 - j$.

In Table 5.1, Shannon calls the first channel less leaky, yet a one-shot guesser does better against it by a factor of more than 10^{15}. When the adversary gets one try, use min-entropy.

Table 5.1.: Shannon and min-entropy leakage of two deterministic channels on a uniform 64-bit secret.

Channel	Shannon leakage	Min-entropy leakage	One-guess success
Reveal all if the low 3 bits are zero	8.1686 bits	61.0000 bits	0.1250
Reveal the low 9 bits	9.0000 bits	9.0000 bits	2.78×10^{-17}

Both measures still average over the secret, so neither says how exposed one value is. Revealing a 32-bit secret only when it is all zeros leaks 7.786×10^{-9} Shannon bits and 1 min-entropy bit, yet exposes that one value completely. Later parts build a worst-case measure.

5.6. Fano's inequality

Mutual information can also show that no estimator beats some error rate. Suppose a guess of X from Y is wrong with probability P_e. The uncertainty left about X is then at most the uncertainty about whether the guess is right, $h(P_e)$, plus, when it is wrong, which of the other $M - 1$ values is true:

$$H(X \mid Y) \le h(P_e) + P_e \log(M-1), \qquad \text{so} \qquad P_e \ge \frac{H(X \mid Y) - 1}{\log M}. \tag{5.5}$$

Solve the left form numerically; the right form bounds $h(P_e)$ by 1 and is weaker. A label uniform over 8 classes (3 bits), with features carrying at most 1 bit, keeps at least 2 bits, so every classifier errs at least 0.3730 of the time (the weak form gives 0.3333).

For estimation, take M candidate parameters whose sample distributions are within KL divergence κ bits of each other: n samples carry at most $n\kappa$ bits about which is true, so $P_e \ge 1 - (n\kappa + 1)/\log M$. With 1024 candidates and $\kappa = 0.01$, 100 samples leave $P_e \ge 0.8000$ and 400 leave $P_e \ge 0.5000$.

The bound is tight only when a wrong guess leaves the truth spread evenly over the other values. If errors go to one neighbour, little uncertainty remains after a wrong guess and Fano underestimates: shifting each of 8 symbols to its neighbour with probability 0.2 gives a bound of 0.0954 against a true MAP error of 0.2000.

5.7. Code

Everything in Section 5.4 comes from the joint table. This computes the guessing example:

```
from math import log2

prior = [0.4, 0.3, 0.2, 0.1]
channel = [  # channel[x][y] = probability of observation y given secret x
    [0.8, 0.2, 0.0],
    [0.1, 0.8, 0.1],
```

```
    [0.0, 0.5, 0.5],
    [0.0, 0.2, 0.8],
]


def joint(prior: list[float], channel: list[list[float]]) -> list[list[float]]:
    return [[px * c for c in row] for px, row in zip(prior, channel)]


def mutual_information(p: list[list[float]]) -> float:
    """Shannon leakage I(X; Y) in bits, from the joint table p[x][y]."""
    px = [sum(row) for row in p]
    py = [sum(col) for col in zip(*p)]
    return sum(
        pxy * log2(pxy / (px[x] * py[y]))
        for x, row in enumerate(p)
        for y, pxy in enumerate(row)
        if pxy > 0
    )


def posterior_vulnerability(p: list[list[float]]) -> float:
    """Success probability of the best one-shot guess after seeing y: sum of column maxima."
    return sum(max(col) for col in zip(*p))


p = joint(prior, channel)
v_prior = max(prior)
v_post = posterior_vulnerability(p)
print(f"{v_prior:.4f} {v_post:.4f}")  # 0.4000 0.6600
print(f"{log2(v_post / v_prior):.4f}")  # 0.7225
print(f"{mutual_information(p):.4f}")  # 0.6865
print(f"{log2(sum(max(col) for col in zip(*channel))):.4f}")  # 1.2630
```

It prints the two vulnerabilities, the min-entropy leakage, the Shannon leakage and the min-capacity. The full edition's script, information_theory.py, computes every number in this chapter:

```
uv run python code/part01-prerequisites/information_theory.py
```

5.8. Cheat sheet

Table 5.2.: The quantities of this chapter.

Quantity	Formula	Notes
Entropy	$H(X) = -\sum_x p(x) \log p(x)$	$0 \le H \le \log\lvert\mathcal{X}\rvert$
Conditional entropy	$H(X \mid Y) = \sum_y p(y) H(X \mid Y = y)$	$\le H(X)$ on average only
Mutual information	$I(X;Y) = H(X) - H(X \mid Y) = D(P_{XY} \,\|\, P_X P_Y)$	Symmetric; zero iff independent
Data processing	Z computed from Y: $I(X;Z) \le I(X;Y)$	Bound the raw observation
Vulnerability, min-entropy	$V(X) = \max_x p(x)$, $H_\infty = -\log V$	$H_\infty \le H$
Posterior vulnerability	$V(X \mid Y) = \sum_y \max_x p(x,y)$	Success of the MAP guesser
Min-entropy leakage	$\log V(X \mid Y)/V(X)$	1 bit doubles one-guess success
Min-capacity	$\log \sum_y \max_x C[x,y]$	Worst case over priors
Fano	$H(X \mid Y) \le h(P_e) + P_e \log(M-1)$	Lower bound on any estimator's error

Rules of thumb

- Shannon quantities average logs; min-entropy quantities take the log of a best-case probability. Use the one that matches how many tries the adversary gets.
- Both are averages over the secret; neither says how exposed a single value is.
- Bound the raw observation: processing never adds information.

5.9. Further reading

- Cover & Thomas, *Elements of Information Theory* (2nd ed., 2006), ch. 2: the standard reference for every Shannon result here.
- MacKay, *Information Theory, Inference, and Learning Algorithms* (2003): the same with a machine-learning slant, free online.
- Smith, "On the Foundations of Quantitative Information Flow" (FoSSaCS 2009): vulnerability, and why Shannon leakage is the wrong measure for one-try guessing.
- Alvim et al., *The Science of Quantitative Information Flow* (2020): the full theory of leakage and capacity.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

6. Optimization

6.1. Why this matters

Training a model minimizes the average loss over the training set, $F(w) = \frac{1}{n}\sum_i \ell(w, z_i)$, over parameters w, almost always with stochastic gradient descent (SGD). Two questions come up every time. How fast does SGD get there, and how should you set its step size when its gradients are noisy? And how much does the answer depend on any single training example: if you replace one and retrain, does the minimizer barely move, or can it jump?

Both answers come from a few constants of the loss: how steep it can be, how fast its slope can change, how strongly it curves upward, and how much the random gradients scatter. Every result here is a statement about them, so know which ones a result assumes, and whether your loss has them.

6.2. Lipschitz, smooth and strongly convex losses

Four properties of the loss matter. A loss is L-**Lipschitz** when its gradient norm never exceeds L, so moving the parameters by d changes the loss by at most Ld. It is β-**smooth** when its gradient changes by at most β per unit of distance; then the tangent is a trustworthy guide for a step of about $1/\beta$, and that is the largest safe step size. It is convex when its curvature is never negative, and λ-**strongly convex** when the curvature is at least λ in every direction; adding $\frac{\lambda}{2}\|w\|^2$ to a convex loss makes it so.

The logistic loss $\ln(1 + e^{-y\langle w,x\rangle})$, with features scaled to $\|x\| \le 1$, is the standard example. Its slope is a probability times x, so it is 1-Lipschitz; its curvature is $p(1-p) \le \frac{1}{4}$, so it is $\frac{1}{4}$-smooth. It is convex but not strongly convex, because far from the data it flattens out. A ridge term makes it λ-strongly convex and $(\frac{1}{4} + \lambda)$-smooth.

Lipschitz and smooth bound different derivatives, so neither implies the other: $|w|$ is 1-Lipschitz but its slope jumps at zero, and $w^2/2$ is 1-smooth but its slope is unbounded. The parabola is the general case: a strongly convex function curves upward everywhere, so its slope grows at least in proportion to the distance from the minimizer, and it cannot be Lipschitz on all of $\mathbb{R}^d$. The ridge-regularized logistic loss is therefore Lipschitz only on a ball $\|w\| \le R$, with gradient at most $1 + \lambda R$ there. Algorithms keep the iterates in the ball by **projection**, $\Pi(u) = u \min(1, R/\|u\|)$, which never moves a point farther from anything inside the ball.

6.3. SGD convergence

SGD steps against the gradient of one random example, or of a small random batch: cheap, right on average, but noisy. Even at the minimizer the individual gradients are not zero, so with a fixed step the iterates keep jumping around it. Averaging the iterates cancels the jumps; shrinking the step makes them smaller.

For a convex loss, the error of the averaged iterate after T steps of size η, starting within R of the minimizer, with random gradients of mean squared size at most G^2, is

$$\mathbb{E}[F(\bar{w}_T)] - F^* \leq \frac{R^2}{2\eta T} + \frac{\eta G^2}{2}. \tag{6.1}$$

The two terms are a trade-off. The first is the cost of travelling the distance R with steps of size η: small steps take long. The second is the jitter each step adds: large steps overshoot. Balancing them gives the best step $\eta = R/(G\sqrt{T})$ and an error of $RG/\sqrt{T}$, with $G = L$ for an L-Lipschitz loss. The bound holds for the average of the iterates, not the last one, so average.

Table 6.1 tests the bound on logistic regression with $n = 1{,}000$, $d = 5$, $R = 5$ and $L = 1$.

Table 6.1.: Projected SGD on logistic regression against the convex Lipschitz bound, averaged over 20 runs.

Steps T	Step η	Observed $F(\bar{w}_T) - F^*$	Bound $RL/\sqrt{T}$
100	0.5000	4.44×10^{-2}	0.500
1,000	0.1581	6.46×10^{-3}	0.158
10,000	0.0500	7.71×10^{-4}	0.050

The bound holds, loose by a factor of 11 at $T = 100$ and 65 at $T = 10{,}000$. The gap grows because the observed error falls much faster than $1/\sqrt{T}$, closer to $1/T$: the data curve the loss near the minimizer, and the convex bound does not know it. Every constant in a bound is a worst case the run does not reach, so trust how a bound scales, not its size.

Curvature is what speeds SGD up. On a λ-strongly convex loss, the farther the iterate is from the minimizer, the harder the gradient pulls it back, and with steps $\eta_t = 2/(\lambda(t+1))$ and an average that weights later iterates more, the error falls to $2G^2/(\lambda(T+1))$. The price is the $1/\lambda$: a weak regularizer gives a fast rate only after many steps.

On a non-convex loss, such as a neural network's, no gradient method can promise the global minimum. What smoothness guarantees is a point where the gradient is small: for a β-smooth loss that can fall at most Δ from the start, with gradient variance σ^2 and $\eta \leq 1/\beta$,

$$\frac{1}{T}\sum_{t=1}^{T} \mathbb{E}\|\nabla F(w_t)\|^2 \leq \frac{2\Delta}{\eta T} + \beta\eta\sigma^2. \tag{6.2}$$

It is the same trade-off, progress against noise, and a small gradient is all you get: the point may be a saddle or a poor local minimum.

6.4. Gradient noise and minibatches

Every bound above pays for the mean squared size of the random gradients, which is the squared true gradient plus the variance of the estimate. Averaging a batch of B independent gradients divides the variance by B, from σ_1^2 to σ_1^2/B, a little less when drawn without replacement. Noise of mean zero added to each gradient adds its mean squared size σ^2: independent of the gradient and zero on average, it neither reinforces nor cancels it. So the convex bound holds with $G^2 = L^2 + \sigma^2$:

$$\eta^* = \frac{R}{\sqrt{(L^2+\sigma^2)T}}, \qquad \frac{R\sqrt{L^2+\sigma^2}}{\sqrt{T}}. \tag{6.3}$$

To reach a fixed error, T must grow in proportion to $L^2 + \sigma^2$, so noise multiplies the steps needed by $1 + \sigma^2/L^2$. Noise of size $\sigma = 3L$ makes that ten times as many.

Table 6.2.: Projected SGD with added gradient noise, $T = 10{,}000$: error of the averaged iterate at the bound's step and at the best step on the grid, 10 runs per setting.

Noise σ	η^* from the bound	Observed at η^*	Best η on the grid	Observed there
0	0.0500	4.99×10^{-4}	0.178	1.85×10^{-4}
1	0.0354	1.58×10^{-3}	0.056	1.38×10^{-3}
3	0.0158	9.58×10^{-3}	0.056	5.10×10^{-3}

Table 6.2 shows both effects on the logistic problem. Noise pushed the best step down from 0.178 to 0.056, so retune the step size whenever the gradient noise changes. And the bound's η^* is conservative, by a factor of 3.6 without noise, for the same reason the bound is loose: the real gradients and starting distance are smaller than L and R. Larger batches are not free either: a batch of B costs B gradient evaluations and divides the variance by B, so per evaluation the gain cancels, and batches pay mainly through parallelism.

6.5. Sensitivity of regularized ERM

The last question is about the problem, not the optimizer. Regularized **empirical risk minimization (ERM)** returns the minimizer of $J(w; D) = \frac{1}{n}\sum_i \ell(w, z_i) + \frac{\lambda}{2}\|w\|^2$. Replace one example: how far can the minimizer move?

The argument has two steps. Each example carries weight $1/n$, and its gradient is at most L long, so swapping it changes the gradient of J at any point by at most $2L/n$, the old one's L/n out and the new one's L/n in. At the old minimizer the new objective's gradient is therefore at most $2L/n$. Strong convexity says the gradient grows by at least λ per unit of distance from the minimizer, so a point where it is that small lies within $(2L/n)/\lambda$ of the new minimizer:

$$\|w^*(D) - w^*(D')\| \le \frac{2L}{\lambda n}. \tag{6.4}$$

The loss at any point then changes by at most L times that, $2L^2/(\lambda n)$. Later parts call this movement a **sensitivity**.

With $n = 1{,}000$ and $\lambda = 0.1$ the bound is $2/(0.1 \times 1{,}000) = 0.02$, and over 20 replacements in regularized logistic regression the largest observed move is 6.66×10^{-3}: smaller, because real data add curvature of their own and two gradients rarely point in opposite directions at full length. Because one example weighs $1/n$, ten times the data gives a tenth of the movement (Figure 6.1).

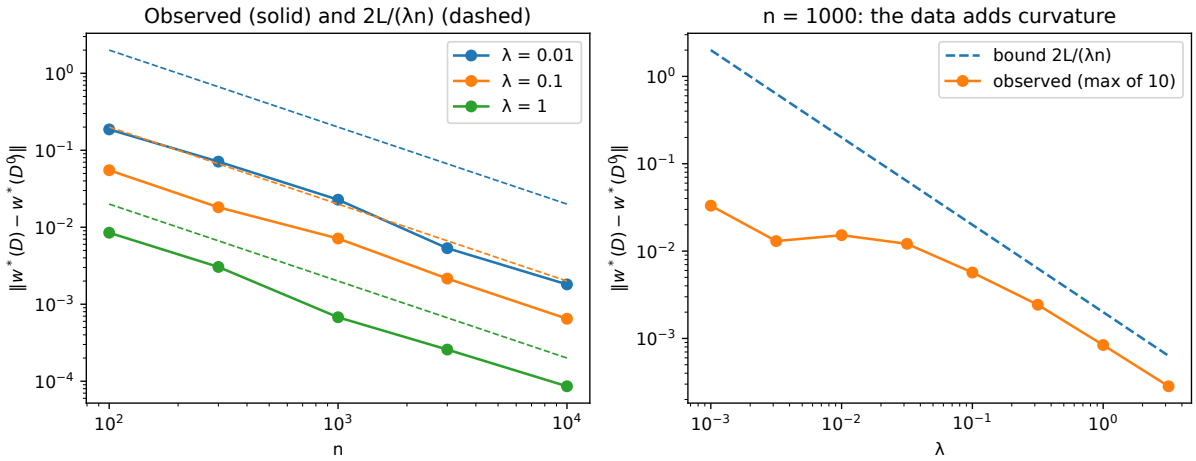


Figure 6.1.: Left: largest observed movement of the regularized logistic minimizer when one of n examples is replaced (solid), against $2L/(\lambda n)$ (dashed), for three values of λ. Right: the same at $n = 1000$ as a function of λ: the bound blows up as $\lambda \to 0$, while the observed movement levels off.

The argument shows what the bound needs. It is about the exact minimizer, so an optimizer stopped early adds its own error. And it needs the curvature: without a regularizer, the absolute loss gives the median, and on data made of 0s and 1s one replaced value can move it by 1 whatever n is. On a non-convex loss with two valleys, one label can flip which valley is deeper, and the minimizer of a double-well loss jumps by 2.0854 at $n = 11$ and still by 2.0010 at $n = 1{,}001$. For non-convex models, bound one example's influence on the *algorithm* instead, such as a fixed number of SGD steps (Algorithmic stability and generalization in the full edition).

6.6. Code

The formulas are one line each, so there is no snippet. The full edition's script, optimization.py, runs every experiment above:

```
uv run python code/part01-prerequisites/optimization.py
```

6.7. Cheat sheet

Table 6.3.: The results of this chapter.

Result	Assumptions	Bound
Logistic loss	$\|x\| \le 1$	1-Lipschitz, $\frac{1}{4}$-smooth, convex, not strongly convex
Ridge term $\frac{\lambda}{2}\|w\|^2$	Added to a convex loss	λ-strongly convex; Lipschitz only on a ball
SGD, convex	$\mathbb{E}\|g\|^2 \le G^2$, start within R, $\eta = R/(G\sqrt{T})$	$RG/\sqrt{T}$, averaged iterate
SGD, strongly convex	$\eta_t = 2/(\lambda(t+1))$, t-weighted average	$2G^2/(\lambda(T+1))$
SGD, non-convex	β-smooth, variance σ^2, $\eta \le 1/\beta$	$\frac{1}{T}\sum_t \mathbb{E}\|\nabla F(w_t)\|^2 \le \frac{2\Delta}{\eta T} + \beta\eta\sigma^2$
Minibatch variance	B of n	σ_1^2/B; times $\frac{n-B}{n-1}$ without replacement
Added gradient noise	Mean zero, $\mathbb{E}\|\xi\|^2 = \sigma^2$	$G^2 = L^2 + \sigma^2$; $\eta^* = R/\sqrt{(L^2+\sigma^2)T}$
Regularized ERM	Convex L-Lipschitz loss	Minimizer moves $\le 2L/(\lambda n)$; loss $\le 2L^2/(\lambda n)$

Rules of thumb

- Without curvature SGD's error falls like $1/\sqrt{T}$, with it like $1/(\lambda T)$. Average the iterates either way.
- Noise multiplies the steps needed by $1 + \sigma^2/L^2$; lower the step size when gradients get noisier.
- One example moves a λ-regularized minimizer by at most $2L/(\lambda n)$. Without strong convexity there is no such bound.

6.8. Further reading

- Shalev-Shwartz & Ben-David, *Understanding Machine Learning* (2014), ch. 12–14: convexity, regularization and stability, and SGD, at the level of this chapter.
- Bottou, Curtis & Nocedal, "Optimization Methods for Large-Scale Machine Learning" (SIAM Review 2018): noise, step sizes and batch sizes in practice.
- Bubeck, "Convex Optimization: Algorithms and Complexity" (2015): the rates for each combination of assumptions, with lower bounds.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

7. Algorithmic stability and generalization

7.1. Why this matters

Your model's training loss is low. Will its loss on new data be low too, and which training examples does it depend on? For a model fixed before you saw the data, the training loss is an unbiased estimate of the test loss. For a model fitted to the data it is biased downward, because training picked the model that looks good on exactly these examples. That bias, test loss minus training loss, is the **generalization gap**.

Instead of counting how rich the model class is, this chapter looks at the training algorithm, and its one idea is that **a model generalizes when it does not depend much on any single training example.** The gap is exactly the average effect of swapping one example out, and two knobs you already turn control that effect: regularization strength, and how long you train with stochastic gradient descent (SGD). Later parts reuse these quantities.

Notation. The training set is $S = (z_1, \dots, z_n)$, drawn independently from a distribution $\mathcal{D}$, and $S^{(i)}$ is S with example i replaced by a fresh draw z_i'. The algorithm A returns the model $A(S)$, with test loss $R(A(S))$ and training loss $\hat{R}_S(A(S))$.

7.2. The gap and uniform stability

The test loss of $A(S)$ can be measured on the fresh point z_i', which the model never saw. Swap the names of z_i and z_i'. They have the same distribution, so nothing changes on average, but the same number now reads as the loss on z_i of a model trained on $S^{(i)}$, which did not see z_i. The training loss is the loss on z_i of the model that did. So for any algorithm,

$$\mathbb{E}[\text{gap}] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}\Big[\ell(A(S^{(i)}), z_i) - \ell(A(S), z_i)\Big] : \tag{7.1}$$

the expected gap is how much worse each example does when it is left out, averaged over examples.

To bound the gap, bound each bracket. A is β**-uniformly stable** if replacing any one example moves the loss of the trained model by at most β at every test point, for every training set and replacement (in expectation over A's own randomness, for SGD). Then every bracket is at most β, and so is the expected gap. No model class appears, so the bound holds for large models too.

You train once, though, so you want the gap of your model, not its average over training sets. The gap is a function of n independent examples, none of which can move it by much, and that

is the situation of McDiarmid's inequality (Concentration inequalities in the full edition). With losses in $[0, M]$, with probability at least $1 - \delta$,

$$R(A(S)) \le \hat{R}_S(A(S)) + \beta + (2n\beta + M)\sqrt{\frac{\ln(1/\delta)}{2n}}. \tag{7.2}$$

The M part is the usual $1/\sqrt{n}$ fluctuation you would pay even for a fixed model. The $2n\beta$ part is the price of learning, and it vanishes only if β falls faster than $1/\sqrt{n}$. It is loose, because it charges every example the full 2β although their effects largely cancel; sharper bounds (Feldman & Vondrák 2018) are meaningful whenever $\beta \to 0$, at any rate.

The running example is logistic regression on 10-dimensional features uniform on the unit sphere, regularized with $\lambda = 0.01$. With $n = 50$ the training loss is 0.4603 and the test loss 0.5781, a gap of about 0.119; with $n = 1000$ the gap is about 0.006. At $n = 10{,}000$ and $\delta = 0.05$, the high-probability bound adds 0.065 to the training loss at $\lambda = 1$ but 0.614 at $\lambda = 0.1$, mostly through the $2n\beta$ term.

7.3. Regularized ERM is stable

Regularized **empirical risk minimization (ERM)** minimizes the training loss plus $\frac{\lambda}{2}\|w\|^2$. The penalty makes the objective a bowl with curvature at least λ, and one example, carrying weight $1/n$, can only tilt such a bowl so far. Optimization in the full edition turns that into a number: with a convex, L-Lipschitz loss, replacing one example moves the minimizer by at most $2L/(\lambda n)$, so the loss at any point moves by at most L times that,

$$\beta = \frac{2L^2}{\lambda n}. \tag{7.3}$$

Doubling λ halves β, and β falls like $1/n$, faster than $1/\sqrt{n}$, so both bounds above go to zero as the data grow.

Table 7.1 flips one label in logistic regression with unit-norm features ($L = 1$), keeping λn fixed so the bound stays at 0.02.

Table 7.1.: Replace-one loss changes of regularized logistic regression against the stability bound.

n	λ	Bound $2L^2/(\lambda n)$	Largest loss change	Ratio
100	1	0.02	0.00505	0.25
1,000	0.1	0.02	0.00499	0.25
10,000	0.01	0.02	0.00285	0.14

The slack comes from the worst cases in the argument: two examples' gradients rarely point in opposite directions at full length, and the data add curvature of their own, which the bound ignores. That second slack grows when the regularizer is weak, which is why the ratio drops to 0.14 at $\lambda = 0.01$. The measured expected gap is smaller still: about 3% of the bound at $\lambda = 0.01$, 11% to 13% at $\lambda = 1$.

The argument needs a Lipschitz loss, so squared loss qualifies only on a bounded set of weights and labels. And stability is not accuracy: a large λ closes the gap by pulling the weights towards zero and underfitting, so the test loss is U-shaped in λ.

7.4. SGD is stable if you stop early

Large models are trained by SGD for a number of steps, not to a minimizer. Run SGD on S and on $S^{(i)}$ with the same random order. On steps that pick a shared example, both runs take a gradient step on the same convex, ρ-smooth loss, and such a step does not push two points apart as long as $\eta \leq 2/\rho$. Only the steps that pick example i can separate them, by at most $2L\eta_t$, and those come about once per epoch. Adding up the jumps gives (Hardt, Recht & Singer 2016)

$$\beta \leq \frac{2L^2}{n} \sum_{t=1}^{T} \eta_t. \tag{7.4}$$

With a constant step η for E epochs this is $2L^2\eta E$. Stability degrades with training time, and the n cancels: for a fixed number of epochs, more data does not help the bound.

So early stopping is a trade-off, not a trick: training longer lowers the optimization error but raises β. With unregularized logistic regression, $n = 100$ and $\eta = 0.25$, the measured gap grows from 0.036 after 1 epoch to 0.121 after 40, and the test loss is lowest after about 5 epochs while the training loss keeps falling (Figure 7.1).

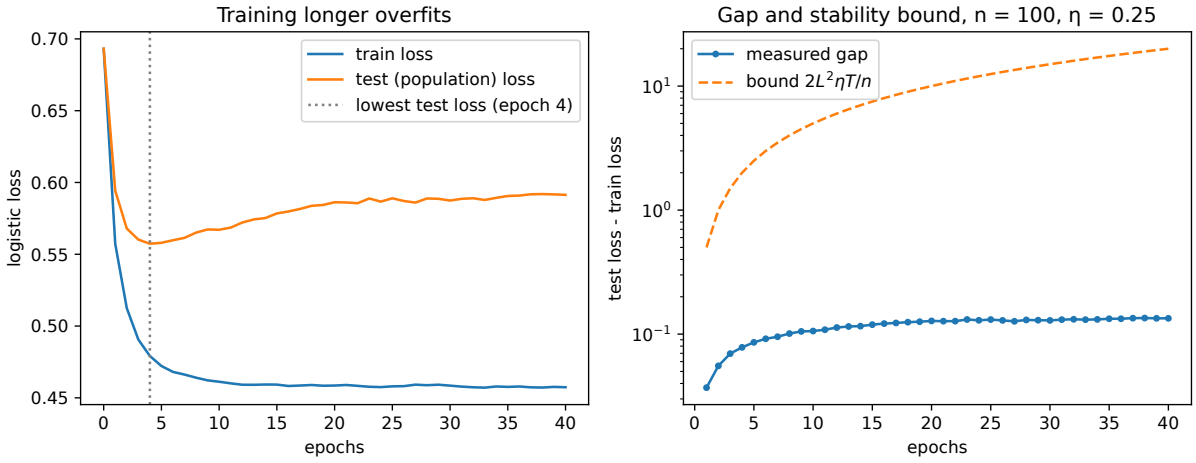


Figure 7.1.: Left: train and test loss of SGD on logistic regression ($n = 100$, $\eta = 0.25$) against the number of epochs. Right: the measured gap against the stability bound $2L^2\eta T/n$, on a log scale. Both grow with training time; the bound grows faster.

The bound, $0.5E$ here, is loose by a factor of 14 after one epoch and 165 after 40, because the argument only uses that a step does not expand distances; on a curved loss it usually contracts them, so the runs drift apart much more slowly. On a non-convex loss a gradient step can push points apart, and the bound grows much faster with T.

7.5. Memorization and self-influence

The bracket in the gap identity can be measured example by example, by retraining without each one: the **self-influence**, or **memorization score**, of example i,

$$\mathrm{mem}_i(S) = \ell(A(S^{(i)}), z_i) - \ell(A(S), z_i), \tag{7.5}$$

and by the identity its average is the expected gap. With $n = 40$ and $\lambda = 0.01$, the measured gap is 0.1482 ± 0.0073 and the average self-influence 0.1422 ± 0.0025, equal within error.

The scores are uneven, and the reason is redundancy. A typical example is fitted whether or not it is present, because its neighbours carry the same information. An atypical or mislabelled one is fitted only if it is there itself. So examples whose label goes against the trend have average self-influence 0.284, against 0.097 for the rest (Figure 7.2).

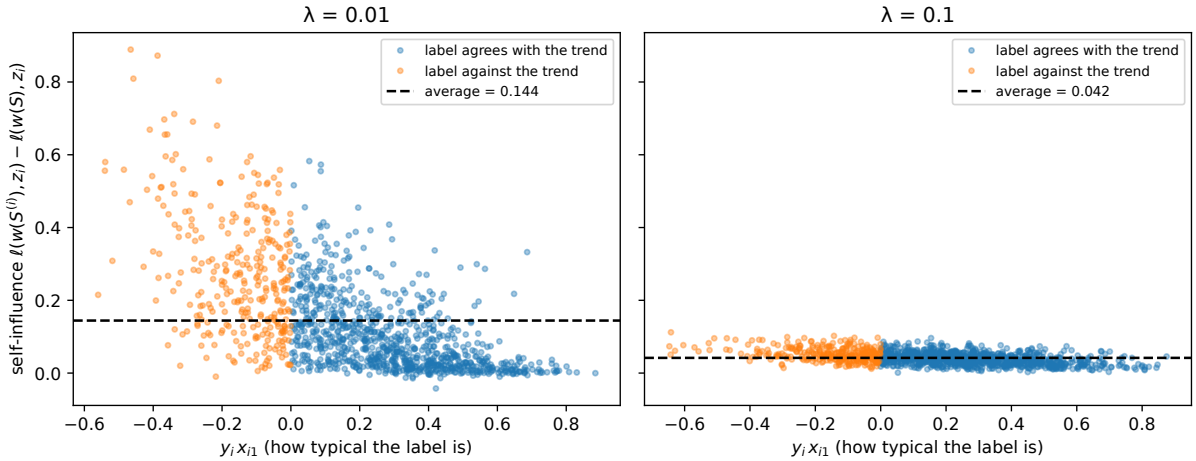


Figure 7.2.: Self-influence of each training example against $y_i x_{i1}$, which measures how typical its label is, for $n = 40$ and two values of λ. Examples whose label goes against the trend have the largest self-influence; regularization flattens the profile. The dashed line is the average, which equals the expected gap.

The extreme case is the 1-nearest-neighbour classifier, which predicts the label of the closest training point. It fits every training example using that example itself, so its training error is 0 and its β is 1, the largest possible for the 0-1 loss. Its test error falls only to 0.352 at $n = 4{,}000$, towards a limit of 0.338, while the stable logistic regression reaches 0.259 on the same data. Zero training error is a warning, not a result.

A small average gap still allows a few heavily memorized examples; only uniform stability caps each. And memorization is not always a defect: Feldman (2020) shows that on data with a long tail of rare subpopulations, fitting the singletons is needed for accuracy. So expect the rarest examples to be the most memorized, and measure them rather than the average.

7.6. Code

There is no snippet: every number here comes from retraining models. The full edition's script, stability.py, runs all the experiments:

```
uv run python code/part01-prerequisites/stability.py
```

7.7. Cheat sheet

Table 7.2.: The results of this chapter.

Result	Assumptions	Statement
Gap identity	None	$\mathbb{E}[\text{gap}] = \frac{1}{n}\sum_i \mathbb{E}[\text{mem}_i]$
Expected gap	β-uniformly stable	$\lvert\mathbb{E}[\text{gap}]\rvert \le \beta$
High probability	Also $0 \le \ell \le M$	gap $\le$ $\beta + (2n\beta + M)\sqrt{\ln(1/\delta)/(2n)}$
Regularized ERM	Convex, L-Lipschitz loss, $\frac{\lambda}{2}\lVert w\rVert^2$	$\beta = 2L^2/(\lambda n)$
SGD	Convex, ρ-smooth, L-Lipschitz, $\eta_t \le 2/\rho$	$\beta \le (2L^2/n)\sum_t \eta_t$
Lookup table, 1-nearest neighbour	0-1 loss	$\beta = 1$; training error 0

Rules of thumb

- Regularization strength and training time are the two stability knobs: $\beta \propto 1/\lambda$ for ERM, $\beta \propto \eta T$ for SGD.
- A useful high-probability bound needs β to fall faster than $1/\sqrt{n}$.
- The examples a model memorizes most are the atypical ones.

7.8. Further reading

- Bousquet & Elisseeff, "Stability and Generalization" (JMLR 2002): uniform stability and the high-probability bound.
- Shalev-Shwartz & Ben-David, *Understanding Machine Learning* (2014), ch. 13: the gap identity and the stability of regularized ERM.
- Hardt, Recht & Singer, "Train Faster, Generalize Better: Stability of Stochastic Gradient Descent" (ICML 2016): SGD stability and early stopping.
- Feldman & Zhang, "What Neural Networks Memorize and Why" (NeurIPS 2020): memorization scores measured on image benchmarks.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

8. Cryptography basics

8.1. Why this matters

Some computations must hide a key, a message, or a value split among several holders. What does it mean that an observer learns nothing it should not? Against unlimited time nothing can be hidden, since an attacker could try every key, so cryptography asks that every attacker with realistic computing power learns almost nothing.

The precise version is a game. A challenger picks one of two worlds by a fair coin, such as the encryptions of two messages, and shows the result to an adversary, who guesses the world. The adversary's **advantage** is how much better than the coin it does, and a scheme is secure when every efficient adversary's advantage is tiny.

8.2. Advantage and security levels

A **distinguisher** sees one sample and outputs a bit. Its skill at telling X from Y is how much more often it says 1 on one than on the other,

$$\mathrm{Adv}_A(X,Y) = \big|\Pr[A(X)=1] - \Pr[A(Y)=1]\big|, \tag{8.1}$$

and in the coin game it wins with probability $\frac{1}{2} + \frac{1}{2}\mathrm{Adv}$. A classifier of real against generated images is one, with advantage its true positive rate minus its false positive rate.

The best distinguisher says 1 exactly on the outcomes more likely under X than under Y. Its advantage is the probability X puts on those outcomes in excess of Y, the **total variation (TV) distance**, and no distinguisher, however much time it has, can beat it. TV distance 0 is **perfect** security, since there is nothing to find; a tiny one is **statistical** security. **Computational** security allows a large TV distance, as long as no efficient adversary can find the outcomes that differ; this is what makes short keys possible.

"Tiny" needs a definition. The asymptotic one is **negligible**: eventually below one over every polynomial in the key length. But that is about the limit, and says nothing at a fixed key length: $2^{-\sqrt{\lambda}}$ is negligible, yet still 3.9×10^{-4} at $\lambda = 128$. Real systems fix the key length, so ask for concrete numbers; the usual one is **κ-bit security**: an attacker running for time t gets advantage at most about $t/2^{\kappa}$, which is what brute force over 2^{κ} keys achieves. A year at 10^{18} keys per second tries $2^{84.7}$ keys and finds a 128-bit key with probability 9.3×10^{-14}.

8.3. Hybrid arguments

Proofs rarely compare the two worlds directly. They walk from one to the other through intermediate **hybrids**, changing one thing at a time. Any gap between the ends must show up somewhere along the chain, so the advantages of the steps add up:

$$\mathrm{Adv}(H_0, H_m) \le \sum_{i=1}^{m} \mathrm{Adv}(H_{i-1}, H_i). \tag{8.2}$$

Swapping samples one at a time, m independent samples give at most m times the one-sample advantage.

That factor is safe but pessimistic, because it reflects how the proof is organized, not how the best attacker works. Take m flips of a coin with heads probability 0.51 against m fair flips (Table 8.1). The hybrid bound $m\delta$, with $\delta = 0.01$, is right for one flip and useless by 100. The best attacker pools the flips; its advantage grows like $\delta\sqrt{m}$, because the bias in the count grows like $m\delta$ while its random spread grows like $\sqrt{m}$.

Table 8.1.: Advantage in telling m flips of a 0.51-coin from m fair flips; the majority vote is simulated over 200,000 games per entry, so at $m = 100$ it exceeds the best possible by sampling noise.

Flips m	Majority vote (simulation)	Best possible (TV)	Hybrid bound $\min(1, m\delta)$
1	0.008	0.010	0.010
100	0.082	0.080	1.000
10,000	0.481	0.683	1.000

Majority vote, a natural test, stalls near 0.48: its threshold $m/2$ sits at the centre of the fair coin's count, so the fair count lands above it half the time however large m is. A reasonable-looking test can be far from the best, which is why security is measured against every adversary.

8.4. The birthday bound

Among q random values there are about $q^2/2$ pairs, and each pair collides with probability $1/N$, so collisions become likely once $q^2/2$ approaches N, that is after about $\sqrt{N}$ draws, not N. Precisely, for q independent uniform draws from N values,

$$1 - e^{-q(q-1)/(2N)} \le \Pr[\text{collision}] \le \frac{q(q-1)}{2N}, \tag{8.3}$$

with the 50% point at $q \approx 1.18\sqrt{N}$. The lower bound is the accurate one (Table 8.2).

Table 8.2.: Collision probability, exact against the two bounds.

Case	q	N	Exact	Union bound	$1 - e^{-q(q-1)/(2N)}$
Shared birthday in a class	23	365	0.507	0.693	0.500
Hashing trick: 1,000 features into 2^{20} buckets	1,000	2^{20}	0.379	0.476	0.379
Random 32-bit identifiers for 100,000 records	100,000	2^{32}	0.688	1.164	0.688

So anything random of κ bits starts colliding after about $2^{\kappa/2}$ draws: random 32-bit identifiers for 100,000 records are more likely than not to clash. The same counting limits how much data one key may protect. A block cipher on n-bit blocks never maps two inputs to the same output, while a random function sometimes does, so after q blocks an observer can tell them apart only through a collision, with advantage at most $q^2/2^{n+1}$. For 128-bit blocks, 2^{32} blocks (64 GiB) cost at most 2^{-65}, but at 2^{64} blocks the bound reaches $\frac{1}{2}$. A q-query bound is a data limit: change keys long before it.

8.5. Pseudorandomness and encryption

A short random key can stand in for a lot of randomness. A **pseudorandom generator (PRG)** stretches a short seed into a long string that no efficient distinguisher tells from uniform. A **pseudorandom function (PRF)** F_k turns a key into a huge table of random-looking values: no efficient adversary, choosing its own queries, can tell it from a truly random function. In proofs, both let you replace "pseudorandom" by "truly random" at the cost of a small advantage.

The **one-time pad** $m \oplus K$, with a uniform key as long as the message, has **perfect secrecy**: XOR with a uniform key gives a uniform ciphertext whatever the message. But the key is as long as the data, and it must never be reused, since $(m_1 \oplus K) \oplus (m_2 \oplus K) = m_1 \oplus m_2$ cancels the key.

Practical schemes aim for indistinguishability under **chosen-plaintext attack (CPA)**: an adversary who can get any message encrypted picks two, receives the encryption of one, and must not tell which. This forces encryption to be randomized. If encrypting m_0 always gives the same ciphertext, the adversary asks for the encryption of m_0 and compares, winning with advantage 1.000: deterministic encryption of a column of labels reveals how often each label occurs. The scheme $(r, m \oplus F_k(r))$, with a fresh random r from a set $\mathcal{R}$, works because each message gets a fresh pad as long as r never repeats. Its advantage after q encryptions is at most twice the PRF advantage plus $q/|\mathcal{R}|$, and the second term is real: with 8-bit nonces and 16 encryptions, the

attack that waits for a repeated r wins with advantage 0.0607, against the bound 0.0625. Size nonces with the birthday bound.

8.6. Simulation and secret sharing

Many procedures must reveal something, such as the final result, and nothing else. "Nothing else" is hard to state directly, since the participant may combine what it sees cleverly. **Simulation** sidesteps this: if the participant could have faked everything it sees from what it is allowed to learn, then what it sees contains nothing more. Formally, a procedure reveals nothing beyond $L(x)$ if an efficient **simulator**, given only $L(x)$, produces output distributed like the participant's real **view**, perfectly, statistically or computationally.

Additive secret sharing is the simplest case. To split a secret x in $\mathbb{Z}_p$, the integers mod p, among h holders, draw $h-1$ uniform random shares and set the last to x minus their sum, mod p. Shares of two secrets add to shares of their sum. Any $h-1$ shares are uniform whatever x is, because subtracting a uniform number mod p wraps around and hits every value equally often. The simulator outputs uniform numbers: perfect security.

The wrap-around is essential. Over the integers, with r uniform below 16 and the share $x-r$ unreduced, the share's range moves with the secret: values possible for one secret are impossible for the other, and the TV distance between $x=0$ and $x=7$ is $7/16 = 0.4375$ (Figure 8.1). In general it is $|x-x'|/R$ for a mask range R.

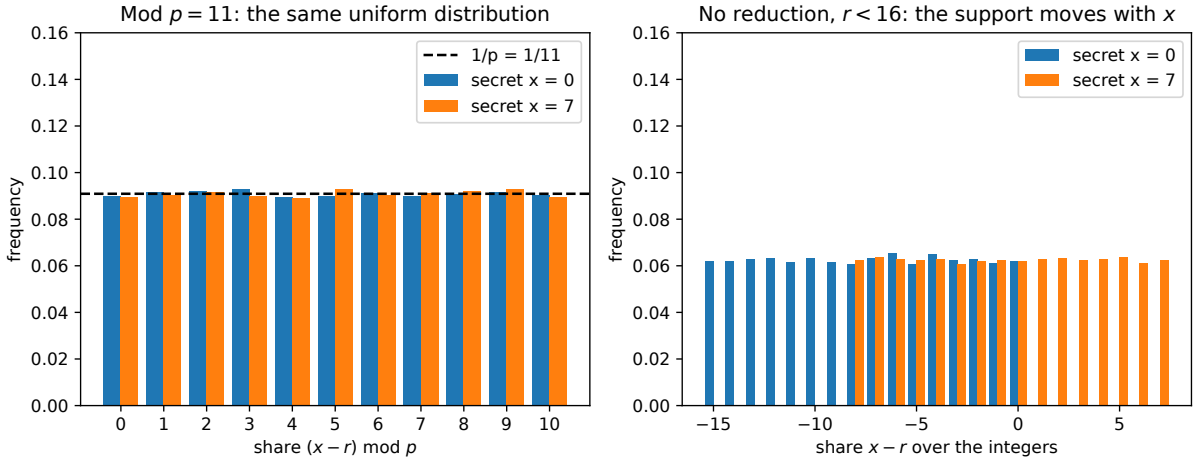


Figure 8.1.: Histograms of the second share for secrets $x=0$ and $x=7$. Left: reduced mod $p=11$, both are uniform. Right: over the integers with $r<16$, the support shifts with the secret.

Simulation proves only that nothing beyond $L(x)$ leaks; whether $L(x)$ itself is acceptable to reveal is a separate question.

8.7. Learning with errors

Learning with errors (LWE) hides a secret vector s behind linear equations whose right-hand sides are slightly wrong: samples $(a, \langle a, s\rangle + e \bmod p)$ with a uniform and a small error e. Without the errors, Gaussian elimination recovers s. With them, elimination solves the wrong system exactly and multiplies the small errors by large numbers mod p: on a 16-dimensional secret it recovers all 16 coordinates without noise, and 0 of 16 with errors in $\{-1, 0, 1\}$. LWE is as hard as worst-case lattice problems, with no known efficient quantum attack.

Regev encryption of a bit m adds $m \cdot \frac{p}{2}$ to a sample. The key holder subtracts $\langle a, s\rangle$ and is left with $m \cdot \frac{p}{2} + e$, which rounds to the right bit as long as $|e| < p/4$. Adding k ciphertexts adds their bits (that is, their XOR) and also their errors, so with errors bounded by B, decryption is guaranteed when

$$k < \frac{p}{4B}. \tag{8.4}$$

With $p = 4096$ and $B = 32$ that is $k < 32$. But independent errors partly cancel, so the summed error typically grows like $\sigma\sqrt{k}$, not kB, and Table 8.3 shows that failures stay rare well beyond the guarantee.

Table 8.3.: Decryption failure rate of a sum of k Regev ciphertexts, $p = 4096$, errors uniform on $\{-32, \dots, 32\}$.

Ciphertexts added k	Empirical failure rate	Normal approximation
31	0.0000	0.0000
256	0.0007	0.0006
1,024	0.0868	0.0881
4,096	0.3814	0.3833

Noise is a budget: every operation spends some, and once it nears $p/4$ decryption becomes a coin flip. Going beyond the worst-case budget trades a guarantee for a failure probability.

8.8. Code

The birthday bound needs one product:

```python
from math import exp, prod


def collision_probability(q: int, n: int) -> float:
    """Exact probability that q uniform draws from n values contain a repeat."""
    return 1 - prod(1 - i / n for i in range(q))


def union_bound(q: int, n: int) -> float:
    return q * (q - 1) / (2 * n)
```

```
def lower_bound(q: int, n: int) -> float:
    return 1 - exp(-union_bound(q, n))


print(f"{collision_probability(23, 365):.3f} {union_bound(23, 365):.3f}")  # 0.507 0.693
print(f"{collision_probability(1_000, 2**20):.3f} {lower_bound(1_000, 2**20):.3f}")  # 0.379
print(f"{collision_probability(100_000, 2**32):.3f} {union_bound(100_000, 2**32):.3f}")  # 0
```

The lines are the three rows of Table 8.2.

The full edition's script, crypto_basics.py, computes every number in this chapter. Its schemes are toys: never use them to protect anything.

```
uv run python code/part01-prerequisites/crypto_basics.py
```

8.9. Cheat sheet

Table 8.4.: The results of this chapter.

Result	Statement
Advantage	$\lvert\Pr[A(X)=1]-\Pr[A(Y)=1]\rvert = 2\lvert\Pr[\text{win}]-\frac{1}{2}\rvert \le \mathrm{TV}(X,Y)$
κ-bit security	Time t buys advantage at most about $t/2^{\kappa}$
Hybrids	Advantages add along a chain; m samples cost at most a factor m
Birthday bound	$\Pr[\text{collision}] \le q(q-1)/(2N)$; 50% at $q \approx 1.18\sqrt{N}$
Encryption	One-time pad: perfect, never reuse. CPA security needs randomness: $(r, m \oplus F_k(r))$
Simulation	Secure if a simulator fakes the view from the allowed leakage
Additive sharing mod p	Any $h-1$ shares uniform: perfect security
Regev sums	Correct while $kB < p/4$; typical noise grows like $\sigma\sqrt{k}$

Rules of thumb

- Random values of κ bits collide after about $2^{\kappa/2}$ draws: size identifiers, nonces and key lifetimes accordingly.
- In LWE schemes, noise is a budget that each operation spends.

8.10. Further reading

- Katz & Lindell, *Introduction to Modern Cryptography* (3rd ed., 2020), ch. 2–3: perfect secrecy, pseudorandomness and CPA security.
- Lindell, "How to Simulate It — A Tutorial on the Simulation Proof Technique" (2017): the simulation paradigm step by step.
- Peikert, "A Decade of Lattice Cryptography" (2016): LWE, its variants and their parameters.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

Part II.

Threat models and attacks

This part teaches you to break models before the rest of the book teaches you to protect them, and the order is deliberate. A privacy defence is a promise that some adversary cannot learn some secret, and you cannot judge the promise until you know who the adversary is, what it sees and how well the best one could do. An evaluation that calls a model private is likewise only as honest as the attack it ran.

Read this part if you have to decide whether a trained model, a released dataset or a set of published statistics leaks information about the people behind it, whether you build the system, audit it or review a report about it. You need the true positive rate (TPR) and false positive rate (FPR) of a test from the Prerequisites; each chapter restates what it uses. Differential privacy (DP), the defence most of the book is about, comes in the next part, and it is built to defeat exactly the games set up here.

The common thread

Every attack in this part is a **game**. A challenger trains a model or releases statistics that depend on a hidden **secret**, the adversary gets the view its **threat model** allows, and it guesses the secret. Only the secret changes from chapter to chapter: whether a record was used, a sensitive field, a memorized string, a name, the whole dataset. Writing attacks this way forces two questions that reports often leave open: what was the adversary assumed to see and know, and how well would a guess have done without the view?

When the secret is one bit, the game is a hypothesis test. The adversary sees one output and must decide which world produced it, the one with the record or the one without. The best it can do is ask how much more likely that output is in one world than in the other, and accuse where that ratio is largest. This best test depends only on the two distributions of what the adversary sees, so its success caps every attack, including those not yet invented. The strongest practical membership attack, the likelihood ratio attack (LiRA), approximates it, which is why it is the one to run in an audit.

The second idea is that the worst case is not the average case. A model leaks most about the records it cannot predict without having seen them: outliers, duplicated strings, unique combinations of attributes. An average over all records dilutes those few into a harmless-looking number. Yet a verdict like "this person was in the data" only harms someone when it is confident, which requires that the attack almost never accuses the wrong person. So attacks are scored by their TPR at a low FPR, and a defence has to hold for every record.

A third idea closes the part: answer enough statistics accurately enough and the answers pin the data down, however they were computed. Some noise is not optional, and the next part asks how little is enough.

Why the chapters come in this order

Threat models and privacy games in the full edition comes first because it fixes the vocabulary the others use: view, game, advantage, baseline, and the ceiling on any attack. Membership inference in the full edition follows because its secret is the smallest, one bit, and the attacks on models after it build on it; it is also the attack you will run most often. The next four chapters

attack models for larger secrets, in any order: a sensitive field in Attribute inference and model inversion in the full edition, a fact about the training population in Property inference in the full edition, training text in Memorization and training-data extraction in the full edition, and the input behind a gradient in Gradient inversion in the full edition.

The last three chapters attack released data, from the weakest defence to the strongest. Linkage and re-identification in the full edition shows that removing names fails when a few attributes single a person out. Syntactic anonymity and how it breaks in the full edition hides each record in a crowd of look-alikes and shows the attacks that get past it. Reconstruction attacks in the full edition releases only aggregates and shows that enough accurate ones give the data back. Read these in order: each releases less than the one before, and each still leaks.

Each chapter opens with the practical question and the idea that answers it, explains each result with its reason, and ends with a cheat sheet. The examples are toys whose numbers come from the full edition's scripts, which also hold the derivations.

What you will be able to do

You will be able to state any attack as a game with a threat model and a baseline, and refuse to read its number until you know both. You will run a membership audit with shadow models and LiRA, report it as a TPR at a low FPR with the number of non-members behind it, and explain why the exposed records are the outliers. You will tell a leak about individuals from inference about the population, and know why rounding confidences does not close it. And you will know why removing names, hiding records in crowds and publishing only aggregates each fail, which is the case for the guarantee of the next part.

9. Threat models and privacy games

9.1. Why this matters

A hospital trains a model on patient records and offers it to doctors through a web service. Is the model private? The question has no answer until you say **against whom**. An attacker who reads only predicted labels sees far less than one who downloads the weights, and one who knows every other patient in the training set can isolate a single record far better than one who knows only the population. A **threat model** fixes these assumptions. Without one, "the model leaks" and "the model is safe" are both untestable, and two attack numbers cannot be compared.

Once the threat model is fixed, an attack becomes a **game**. A **challenger** runs the pipeline with a hidden **secret**, such as whether one patient's record was used, hands the **adversary** what the threat model lets it see, its **view**, and the adversary guesses the secret. The one idea of this chapter: an attack is worth only its margin over a guess made without the view. When the secret is one bit, the game is a hypothesis test: the best attack can be worked out and caps every real one, and what matters is how it does when it almost never accuses the wrong person.

9.2. Threat models

A threat model answers five questions.

1. **Goal**: the secret. Membership of a record, a sensitive field of a known record (attribute), the records themselves (reconstruction), or a statistic of the whole dataset (property).
2. **Access**: from least to most, the model's decisions on chosen inputs (**label-only**), its scores such as confidences or losses (**black-box**), its parameters (**white-box**), or everything training exposed, such as per-step gradients (the **transcript**).
3. **Behaviour**: an **honest-but-curious** adversary only analyses what it legitimately sees; a **malicious** one may query adaptively, insert crafted records into the training data (**poisoning**) or tamper with the protocol.
4. **Trust**: a trusted curator holds the data (**central** model), or nobody is trusted (**local** model).
5. **Side information**: the population, a public dataset, some or all of the other records, the target's non-sensitive fields.

Access is the one axis with a natural order, because each level lets the adversary compute the level below: from the weights it can compute any score, from a score the label, and the final weights are the last entry of the transcript. Whatever an attack does with less access, an adversary with more access can do too, by first throwing the extra away. So for the best attack at each level,

$$\text{best advantage from labels} \leq \text{from scores} \leq \text{from weights} \leq \text{from the transcript}, \tag{9.1}$$

and likewise for the best true positive rate (TPR) at every false positive rate (FPR), defined in Section 9.3. More side information or permission to cheat never weakens the best adversary either, since it can ignore what it knows and choose to behave. The same reasoning tells you what output tweaks do: noising or rounding returned scores, or returning labels only, is processing the adversary could have done itself, so it can lower the best attack but never raise it.

A service returning the top class gives label-only access, one returning confidences black-box access, published weights white-box access, and a federated-training server sees the transcript; a model exposed in several ways is as risky as the most exposed. Side information is the axis you control least, so a guarantee should survive the strongest assumption: an adversary who knows every record except the target's, which a malicious one approaches by planting records.

The chain compares the *best* attacks, not the ones people run. A security team fits the simplest anomaly detector: the mean θ of $n = 100$ records with $d = 50$ normal features, flagging a point x when its score $\|x - \theta\|^2$ exceeds $\tau = d$. Training on a record pulls θ slightly towards it, so a member sits a little closer to θ than it otherwise would. But some targets are far from the mean whether or not they were used, so the raw score mixes "unusual record" with "not a member". The **calibrated score** $\|z\|^2 - \|z - \theta\|^2$ subtracts the target's own distance from the origin and leaves how far θ leans towards z. That is almost exactly the white-box likelihood ratio: it drops only a term that shifts the threshold from one target to the next, and in this model the score carries everything about θ the decision needs.

Table 9.1.: Membership advantage against the centroid detector with $n = 100$ and $d = 50$, from 100,000 games per world.

View	Attack	Best advantage TPR − FPR
White-box	likelihood ratio of θ	0.2789
Black-box	calibrated score $\|z\|^2 - \|z - \theta\|^2$	0.2789
Black-box	raw loss $-\|z - \theta\|^2$	0.0407
Label-only	not flagged, $\tau = d$	0.0400

Table 9.1 respects the chain, but the large gap is inside the black-box level: calibrating each target against its own typicality multiplies the advantage by about seven, after which the weights add nothing. So "the attack we tried failed" proves little. The leak also grows with the number of directions one record can move the model in, to 0.7407 at $d = 500$ (Figure 9.1).

9.3. Privacy games and baselines

A game pins down how the secret and data are drawn, what the adversary sees, and how its guess is scored against a **baseline**, the best guess from the side information alone. The baseline is part of the game: a model that lets an attacker guess a rare diagnosis "with 90% accuracy" reveals nothing if 90% of people do not have it.

In the membership game a hidden bit b decides the world: $b = 1$ means the target's record is in the training set (it is a **member**), $b = 0$ that it is a fresh draw from the population. The attack's **true positive rate (TPR)** is the fraction of members it flags, its **false positive rate (FPR)** the fraction of non-members it wrongly flags. With prior π_1 that $b = 1$ and $\pi_0 = 1 - \pi_1$, it wins

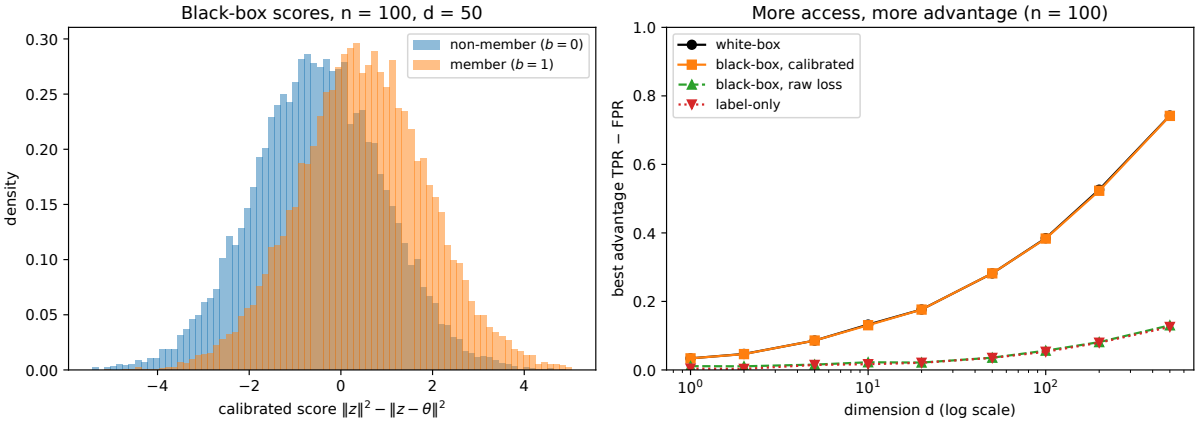


Figure 9.1.: Left: the calibrated black-box score for members and non-members of the centroid detector, $n = 100$, $d = 50$. Right: the best advantage at each access level as the dimension d grows. White-box and calibrated black-box coincide; raw loss and label-only are far below.

with probability $\pi_1 \,\mathrm{TPR} + \pi_0(1 - \mathrm{FPR})$, since the prior decides how many games are played in each world. That is the trap. An attack with TPR 0.6 and FPR 0.2 wins 0.7 of games at a fair prior and 0.78 at $\pi_1 = 0.1$, which looks better but is worse: always saying "not a member" wins 0.9 there.

The fix is to score the attack inside each world. The **advantage**

$$\mathrm{Adv} = \mathrm{TPR} - \mathrm{FPR} \tag{9.2}$$

is how much more often the attack says "member" to members than to non-members. An attack that ignores its view says "member" equally often in both worlds and scores 0 at any prior; a perfect attack scores 1; and at a fair prior the winning probability is $\frac{1}{2} + \frac{1}{2}\mathrm{Adv}$. The attack above has advantage 0.4 whatever the prior.

When the bit is rare, the best attack keeps the default guess "0" and overturns it on a view only when that gains more than it costs. Saying "1" on a view wins the games where $b = 1$, in proportion to π_1 times the view's probability in that world, and loses those where $b = 0$, in proportion to π_0 times its probability there. So it says "1" only on views more than π_0/π_1 times as likely when the bit is 1: 9 times at $\pi_1 = 0.1$. Take a count of people with a sensitive condition, published with unit Gaussian noise, and an adversary who knows every other record. At a fair prior the best guess of the target's bit is right with probability 0.6915. At prior 0.1 the baseline is 0.9 and the best guess reaches only 0.9013: one record shifts the count by one standard deviation of noise, which rarely yields 9-to-1 evidence.

The other games change the secret and the baseline. In the **attribute** game the secret is a sensitive field of a known member, and comparing with a guess from the population asks only "does the model help?". Comparing the attack's success on members with its success on non-members asks "does it help *because this person was in the data*?", and only that is a breach of the person's privacy (Attribute inference and model inversion in the full edition). The **reconstruction** and **property** games target the records and a statistic of the population; like membership, they draw data from the population and so measure an average case. The **distinguishing** game lets the adversary choose two datasets that differ in one record, a worst case (Section 9.5).

9.4. Advantage, total variation and TPR at a low FPR

A two-world game is a test of "not a member" against "member", so the results of Hypothesis testing become ceilings on every attack. Write P and Q for the distributions of the adversary's view without and with the target. Flagging a set of views costs its probability under P in false positives and earns its probability under Q in true positives, so with a fixed budget of false positives the best attack flags the views with the largest likelihood ratio q/p first. That gives two ceilings:

$$\mathrm{Adv} \le \mathrm{TV}(P, Q), \qquad \mathrm{TPR} \le 1 - T(P, Q)(\mathrm{FPR}). \tag{9.3}$$

The **total variation (TV)** distance is the advantage of flagging exactly the views more likely under Q. The **trade-off function** $T(P, Q)(\alpha)$ is the smallest fraction of members any attack with FPR α must miss. The first ceiling caps one point of the **receiver operating characteristic (ROC)** curve, the plot of TPR against FPR as the threshold moves; the second caps the whole curve. Both depend only on P and Q, so they bound attacks nobody has invented yet, which makes them the right target for a defence.

But the advantage is the wrong point of the curve to report. It is where TPR − FPR is largest, while privacy breaks at the end where the attack almost never accuses a non-member. An attack that flags 1% of members with no false alarms has advantage about 0.01, yet identifies them with certainty; one right on 55% of members and wrong on 45% of non-members has advantage 0.1 and identifies nobody with confidence. What a flagged person cares about is the **precision**, the share of "member" verdicts that are right: a fraction π_1 of targets are members and the attack flags TPR of them, a fraction π_0 are not and it flags FPR of those, so

$$\Pr[\text{member} \mid \text{flagged}] = \frac{\pi_1 \,\mathrm{TPR}}{\pi_1 \,\mathrm{TPR} + \pi_0 \,\mathrm{FPR}}. \tag{9.4}$$

Only a small FPR keeps this high, and the rarer members are, the smaller it must be.

The noisy count with the other 100 records known shows the trade. The best attack says "member" where the two bell curves cross, with advantage TV = 0.3829 (0.3809 over 200,000 simulated games per world). Hold the FPR at 10^{-3} instead: the threshold rises until one non-member in a thousand passes, and the TPR falls to 0.0183. The attack catches fewer than 2% of members, but when it fires it is right with precision 0.948 at a fair prior, or 0.156 if one target in a hundred is a member. So report the TPR at a low FPR such as 10^{-3}, draw the ROC curve on log axes where that end is visible (Figure 9.2), and state the prior with any precision.

9.5. Average case and worst case

An **average-case** game draws the data and the target from the population, and the adversary knows only the population. A **worst-case** game lets the adversary know, or choose, the other records and the target. The noisy count shows why they differ. An adversary who does not know the other 100 records sees an average over all the crowds they might form, and the target's single unit of shift is buried under their uncertainty, which acts like extra noise of variance 25 on top of the released noise of variance 1. Knowing the crowd removes that blur, and choosing it picks the crowd where the target stands out most. So the best advantage only grows from not knowing, to knowing, to choosing (Table 9.2).

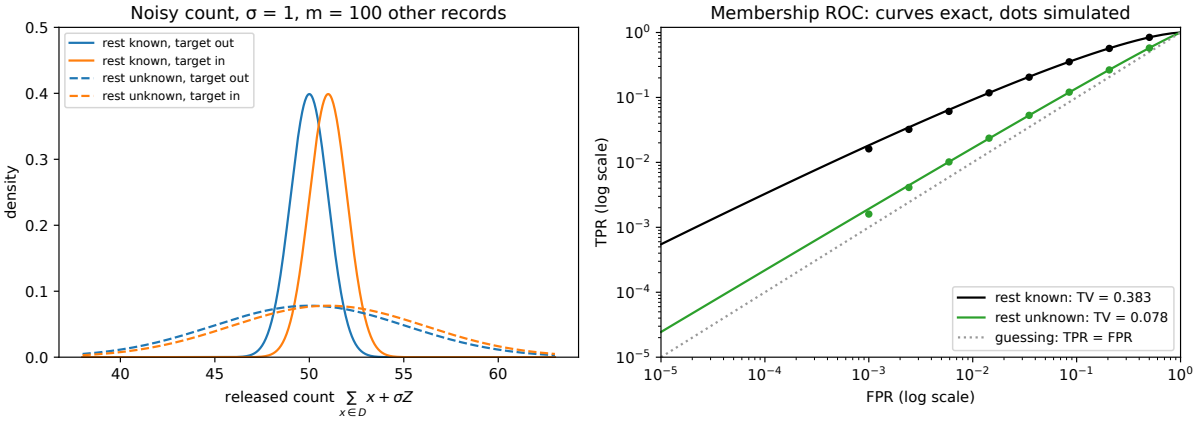
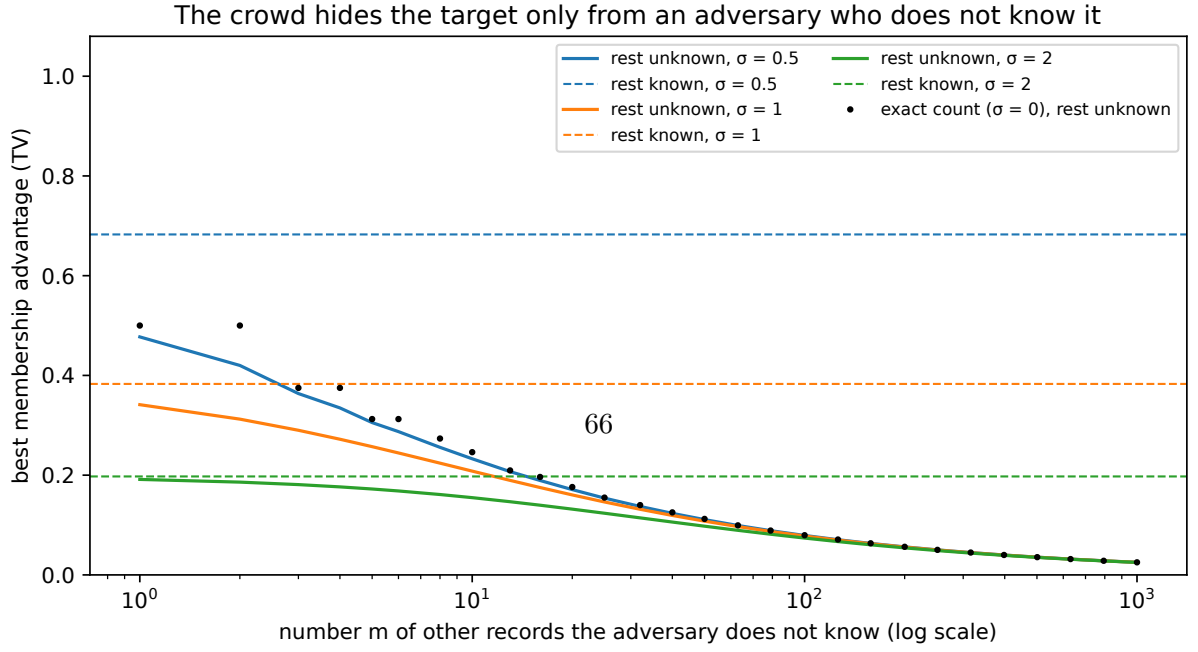


Figure 9.2.: Left: the released count in the two worlds when the other 100 records are known (solid) and unknown (dashed). Right: the membership ROC curves of the best attacks on log axes. The dotted diagonal is random guessing.

Table 9.2.: Membership advantage on a noisy count with 100 other records, for adversaries with different knowledge.

Adversary	Best advantage (exact)	Simulated (200,000 games per world)
Knows the other records ($\sigma = 1$)	0.3829	0.3809
Does not know them, target's bit 1	0.0779	0.0751
Does not know them, target's bit random	0.0390	0.0398
Knows the other records, exact count ($\sigma = 0$)	1	—
Does not know them, exact count ($\sigma = 0$)	0.0796	—

Not knowing the crowd cuts the advantage by a factor of five, and turns an exact count from a complete giveaway into a small leak; as the crowd grows the average case falls further while the worst case does not move (Figure 9.3). A defence that relies on that blur fails against an adversary who knows the other records or planted them.



chosen at random among $n = 1000$ has an average advantage of $1/n = 0.001$ and looks almost perfectly safe, yet every run exposes one person completely. Privacy is a promise to each person, so what matters is the advantage against the worst-off one.

Average-case numbers, which are what an audit can estimate, are therefore lower bounds on the worst case, never upper bounds. Audits narrow the gap by playing the adversary who chooses the target: they plant **canaries**, records built to be as exposed as possible, and attack those (Memorization and training-data extraction in the full edition). The guarantee of The definition of differential privacy goes all the way: it bounds the worst case for every pair of datasets that differ in one record, and so every membership-style game at once.

9.6. Code

When the attack score is a unit normal for non-members and shifted for members, the best TPR at any FPR and the precision take a few lines:

```
from statistics import NormalDist

Z = NormalDist()


def tpr_at_fpr(fpr: float, shift: float) -> float:
    """Best TPR at a given FPR when the attack score is N(0, 1) for non-members, N(shift, 1)
    return 1 - Z.cdf(Z.inv_cdf(1 - fpr) - shift)


def precision(tpr: float, fpr: float, prior: float) -> float:
    """Fraction of 'member' verdicts that are right when a fraction `prior` of targets are m
    return prior * tpr / (prior * tpr + (1 - prior) * fpr)


advantage = 2 * Z.cdf(0.5) - 1  # best TPR - FPR for a shift of 1: the total variation dista
tpr = tpr_at_fpr(1e-3, 1.0)
print(f"{advantage:.4f}")  # 0.3829
print(f"{tpr:.4f}")  # 0.0183
print(f"{precision(tpr, 1e-3, 0.5):.3f}")  # 0.948
print(f"{precision(tpr, 1e-3, 0.01):.3f}")  # 0.156
```

The full edition's script, threat_models.py, computes every number in this chapter and checks it by simulation:

```
uv run python code/part02-attacks/threat_models.py
```

9.7. Cheat sheet

Table 9.3.: The results of this chapter.

Result	Statement	Notes
Winning probability	$\pi_1 \mathrm{TPR} + \pi_0(1 - \mathrm{FPR})$; fair prior: $\frac{1}{2} + \frac{1}{2}\mathrm{Adv}$	Compare with the baseline $\max(\pi_0, \pi_1)$
Advantage	$\mathrm{Adv} = \mathrm{TPR} - \mathrm{FPR}$	Independent of the prior; 0 for an attack that ignores its view
Access chain	labels $\leq$ scores $\leq$ weights $\leq$ transcript, for the best attack	A better attack on the same view can matter more than more access
Ceilings	$\mathrm{Adv} \leq \mathrm{TV}(P, Q)$; $\mathrm{TPR} \leq 1 - T(P, Q)(\mathrm{FPR})$	Bound every attack, including future ones
Precision	$\pi_1 \mathrm{TPR}/(\pi_1 \mathrm{TPR} + \pi_0 \mathrm{FPR})$	Falls fast when members are rare
Average and worst case	not knowing the others $\leq$ knowing them $\leq$ choosing them	An average-case number is a lower bound

Rules of thumb

- Read an attack's threat model before its number.
- Compare every attack with its baseline. Accuracy near the prior is no attack.
- Report the TPR at a low FPR, with the ROC curve on log axes.
- Treat an average-case audit as a lower bound on the worst case.
- Calibrate scores per target before asking for more access.

9.8. Further reading

- Salem et al., "SoK: Let the Privacy Games Begin! A Unified Treatment of Data Inference Privacy in Machine Learning" (IEEE S&P 2023): every inference attack written as a game, with the relations between them.
- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022): why attacks should be reported by TPR at a low FPR on log-scaled ROC curves.
- Jayaraman, Wang, Knipmeyer, Gu & Evans, "Revisiting Membership Inference Under Realistic Assumptions" (PETS 2021): rare members, priors, precision and threshold choice.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

10. Membership inference

10.1. Why this matters

A hospital trains a model to predict a disease and publishes it. You hold one patient's record. **Was that record in the training set?** If it was, you have learned that the patient was treated there for that disease. This is **membership inference**, the standard way to measure leakage: if an attacker cannot even tell whether a record was used, it can do little else with it. An attack is scored by its **true positive rate (TPR)**, the fraction of members it flags, and its **false positive rate (FPR)**, the fraction of non-members it wrongly flags; its **advantage** is TPR minus FPR (Threat models and privacy games in the full edition).

The one idea: **membership inference is a test about one record.** Train a model many times on data containing a target record and look at its output on that record; train it without the record and look again. Training is random, so each case gives a distribution, the record's **IN** and **OUT** distributions, and the attacker must say which one an output came from. A typical record is predicted well whether or not the model saw it, so its two distributions nearly coincide; an atypical record is predicted badly unless the model saw it, so they are far apart. So one threshold for every record confuses "easy record" with "member", strong attacks compare each record with itself, and the exposed records are the atypical ones.

The numbers come from one experiment, the setup of Carlini et al. (2022) scaled down to a linear model: a pool of 950 typical points and 50 outliers whose label is a coin flip, so a model gets an outlier right only by fitting it; regularized logistic regression trained on random halves of the pool, 128 target and 64 shadow models; 64,000 member and 64,000 non-member pairs.

10.2. The loss attack and the generalization gap

A model fits its training data better than other data, so the simplest attack, due to Yeom et al. (2018), says "member" when the loss is low. Its advantage is a number you already know. Let the loss ℓ lie in $[0, B]$ and say "member" with probability $1 - \ell/B$. Then the TPR is one minus the average member loss over B, the FPR the same with the non-member loss, and their difference is the gap between the two average losses, which are the training and test loss:

$$\text{Adv} = \frac{\text{expected test loss} - \text{expected training loss}}{B}. \tag{10.1}$$

With the 0-1 loss the attack is "member if classified correctly", with advantage test error minus training error.

In the running example the training error is 0.1495 and the test error 0.2663, and "member if correctly classified" has TPR 0.8505, FPR 0.7337 and advantage 0.1168, exactly the gap. Yet it identifies nobody with confidence, flagging almost three non-members in four (Figure 10.1).

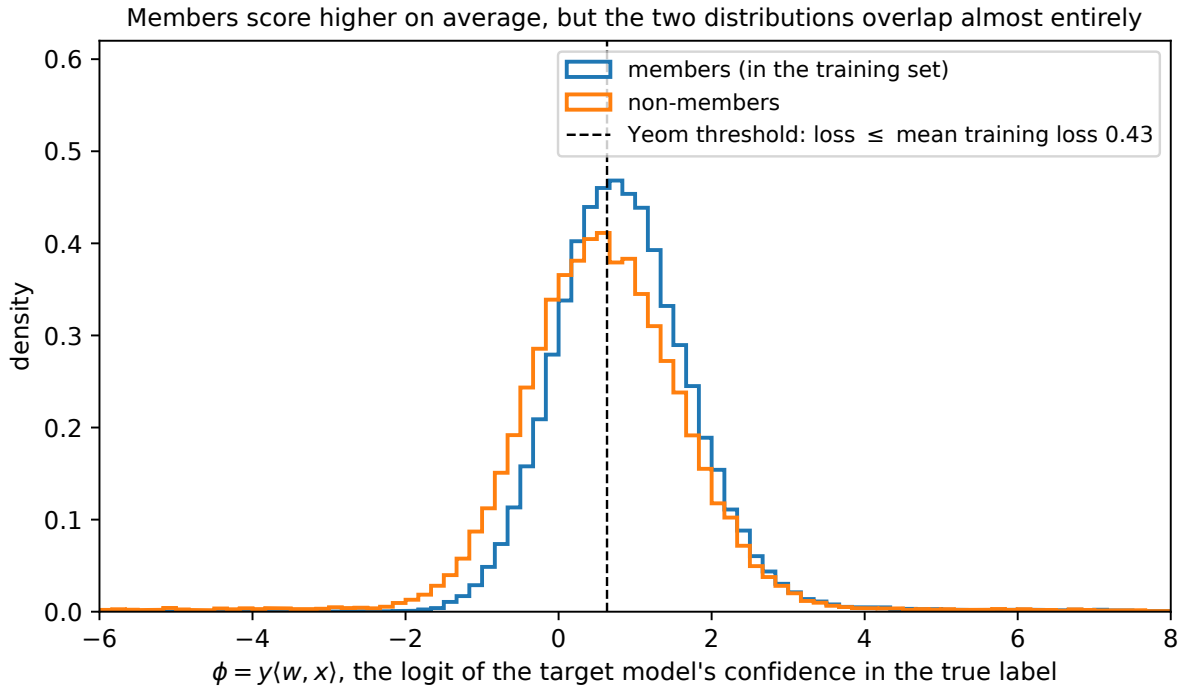


Figure 10.1.: Distribution of ϕ, the logit of the target model's confidence in the true label, over member and non-member pairs. The dashed line is the mean-training-loss threshold: the attack says "member" to its right.

So **overfitting implies leakage**, and you can read a lower bound off train and test error without running anything. The converse fails because of what the gap averages: it equals the average **self-influence** of the training records, how much each record's own loss rises when it is left out. An average hides who is memorized, and an attack that ignores which record it is looking at cannot find them. **A small gap does not mean a private model.**

10.3. Shadow models

A better attack needs to know what members' outputs look like, without seeing the target's training set. Shokri et al. (2017) had the attacker train **shadow models** of its own, with the same recipe on data from the same population, so that it knows who was a member of each. It labels their outputs "in" and "out" and trains an **attack model** to classify them. With enough shadow data the attack model learns the probability of membership given the output, which is the best test that output allows. But the output is pooled over records, so this is the best attack *among those that treat all records alike*: it cannot know that a given confidence is high for one record and low for another. Here it amounts to a per-class loss threshold.

What shadow models really buy is **calibration**: the attacker sets its threshold where a chosen fraction of its own shadow non-members pass, and trusts the real non-members to behave the same. Set for an FPR of 1% on the shadows, the threshold gives 0.0095 on the target models, but its TPR there is 0.0098, chance. Imitate the target's recipe, since it sets how strongly each record is fitted: shadows trained differently estimate the wrong IN and OUT distributions. A crude imitation, even shadow data from another distribution, often still works, because "lower loss means member" transfers across tasks (Salem et al. 2019).

10.4. LiRA: a likelihood-ratio test per record

The right comparison is with the *same record's* output under models that did and did not train on it. The **likelihood ratio attack (LiRA)** of Carlini et al. (2022) collects each record's outputs on IN and OUT shadow models, summarizes each set by a Gaussian, and asks which better explains the target model's output. The output is $\phi = \ln \frac{p_y}{1-p_y}$, the log-odds of the confidence p_y in the true label. Any threshold on confidence, loss or ϕ is the same rule, but for fitting Gaussians ϕ is the right scale: confidences pile up against 1 while their log-odds keep spreading out, and over retrainings ϕ is close to Gaussian.

Let $\mu_{\text{in}}, \sigma_{\text{in}}$ be the mean and standard deviation of ϕ_z over the shadow models that trained on record z, and $\mu_{\text{out}}, \sigma_{\text{out}}$ over those that did not. **Online LiRA** scores the observed ϕ_z by the log of how much more likely it is under the IN fit than under the OUT fit,

$$\ln \Lambda_z = \ln \mathcal{N}(\phi_z;\, \mu_{\text{in}}, \sigma_{\text{in}}^2) - \ln \mathcal{N}(\phi_z;\, \mu_{\text{out}}, \sigma_{\text{out}}^2), \tag{10.2}$$

where $\mathcal{N}(\phi; \mu, \sigma^2)$ is the normal density at ϕ, and says "member" above a threshold **the same for every record**. That is optimal for a simple reason. The attacker sees a record and an output, and the record is drawn the same way whether or not it ends up in training, so it carries no evidence and cancels from the likelihood ratio, leaving the record's own ratio. Thresholding that is the Neyman–Pearson test, so when the fits are right no attack has a higher TPR at any

FPR. Online LiRA needs shadows trained *with* each target; **offline LiRA** uses OUT models only, trained once and reused, and scores $(\phi_z - \mu_{\text{out}})/\sigma_{\text{out}}$, how many OUT standard deviations the output lies above its usual value.

Table 10.1.: Two kinds of record. A typical record is predicted confidently whether or not it was trained on; an atypical one is predicted wrongly unless it was.

Kind	Share	OUT: $\phi \sim$	IN: $\phi \sim$	Separation s	Best TPR at FPR 10^{-3}, this kind alone
Typical	95%	$\mathcal{N}(3,1)$	$\mathcal{N}(3.5,1)$	0.5	0.0048
Atypical	5%	$\mathcal{N}(-2,1)$	$\mathcal{N}(2,1)$	4	0.8185

Two kinds of record show what each step buys (Table 10.1), with every attack set to a pooled FPR of 10^{-3}. One threshold on ϕ must sit high, because typical non-members score high and are 95% of non-members, so atypical members almost never reach it and the TPR is 0.0048. Offline LiRA gives every record FPR 10^{-3}, so each kind gets its own best TPR: $0.95 \times 0.0048 + 0.05 \times 0.8185 = 0.0455$. Online LiRA reaches 0.0503, the best possible, by moving false-positive budget from typical records, where it buys almost nothing because the curves overlap, to atypical ones, where it buys a lot (Figure 10.2).

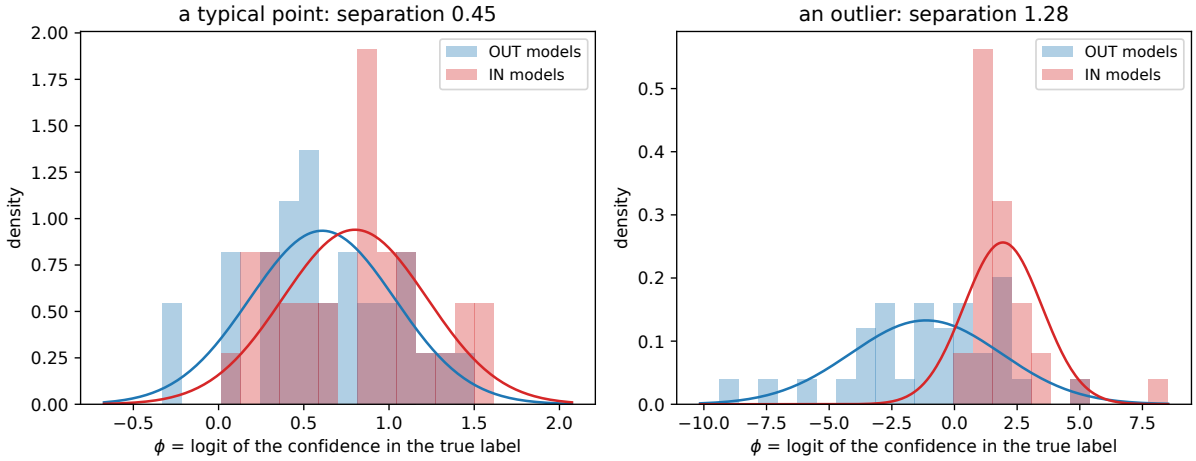


Figure 10.2.: The logit ϕ of one point over the 64 shadow models, split into the 32 that trained on it (IN) and the 32 that did not (OUT), with the fitted Gaussians. Left: a typical point, whose two distributions overlap. Right: an outlier, predicted wrongly unless the model trained on it.

In practice LiRA fails through its fits: an underestimated per-record spread makes an ordinary output look extreme, a false positive exactly at the strict thresholds that matter. With few shadow models, use one **global variance**, estimated from all records at once (Figure 10.3). And compute ϕ from the logits f as $f_y - \ln \sum_{y' \neq y} e^{f_{y'}}$, never from $1 - p_y$, which rounds to 0 for a confident model.

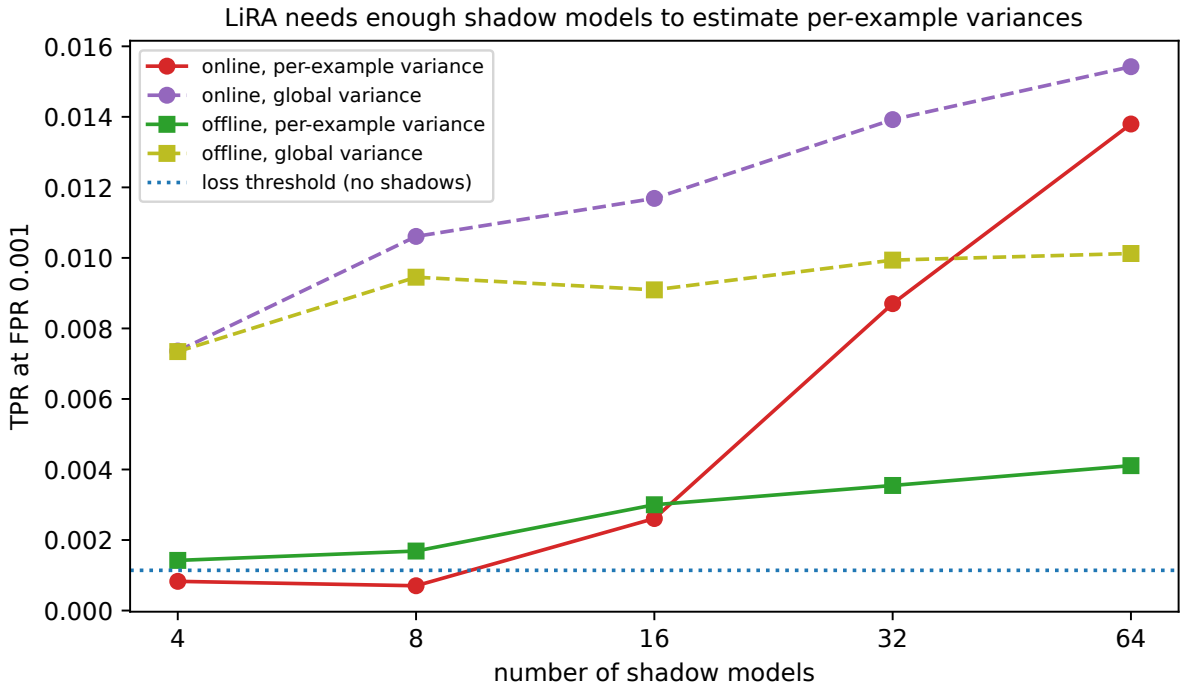


Figure 10.3.: TPR at FPR 10^{-3} of four LiRA variants as the attacker uses 4 to 64 shadow models. With a global variance the attack is strong even with few shadow models; with per-record variances it needs many. The dotted line is the loss attack.

10.5. Evaluating attacks: TPR at a low FPR

Accuracy and the **area under the curve (AUC)**, the probability that a random member outscores a random non-member, average over everyone. A breach is not an average: it is a confident verdict about one person, which requires a low FPR. Attack A recognizes 5% of members with certainty and nobody else; attack B shifts every member's score slightly. Since the AUC weighs every member–non-member pair equally, a tiny edge on everyone buys as much of it as certainty about a few, and the two have the same AUC (Table 10.2). At FPR 10^{-3} they differ by a factor of 38: A is a severe breach for one person in twenty, B a negligible one for all.

Table 10.2.: Two attacks with the same AUC. Only the TPR at a low FPR tells them apart.

	AUC	Best advantage	TPR at FPR 10^{-3}
A: 5% of members exposed	0.5250	0.0500	0.05095
B: every member shifted by $\mu = 0.0887$	0.5250	0.0354	0.00134

So report the TPR at a low FPR such as 10^{-3}, and the receiver operating characteristic (ROC) curve on log axes, where chance is the diagonal at every scale. Measuring that end takes data: an FPR is a count of false positives, a count of k has a relative error of about $1/\sqrt{k}$, and seeing k of them at FPR α takes about k/α non-members. Give each rate a confidence interval, such as the exact **Clopper–Pearson interval**. With 5,000 non-members and no false positives you can only bound the FPR by about $3/5{,}000 = 6 \times 10^{-4}$.

Table 10.3.: Every attack on the running example: 128 target models, 64 shadow models, 64,000 member and 64,000 non-member pairs.

Attack	AUC	Best advantage	TPR at FPR 10^{-3}	TPR at FPR 10^{-2}
Loss threshold (Yeom)	0.577	0.122	0.0009	0.0103
Shadow attack model (Shokri)	0.577	0.121	0.0008	0.0104
LiRA offline	0.632	0.189	0.0030	0.0273
LiRA online	0.629	0.175	0.0149	0.0386
LiRA online, global variance	0.623	0.169	0.0177	0.0396

By AUC the attacks in Table 10.3 are close, and by best advantage offline LiRA looks strongest. At FPR 10^{-3} the loss and shadow attacks are at chance, while online LiRA finds 952 of the 64,000 members, 17.0 times the loss attack, with TPR interval $[0.0140, 0.0158]$. It uses the same statistic as the loss attack; comparing each record with itself is the whole gain. Its FPR rests on 64 false

positives, interval $[0.00077, 0.00128]$. (As in most papers, the threshold was set on the data that measure the rates; a real attacker sets it on its shadows.)

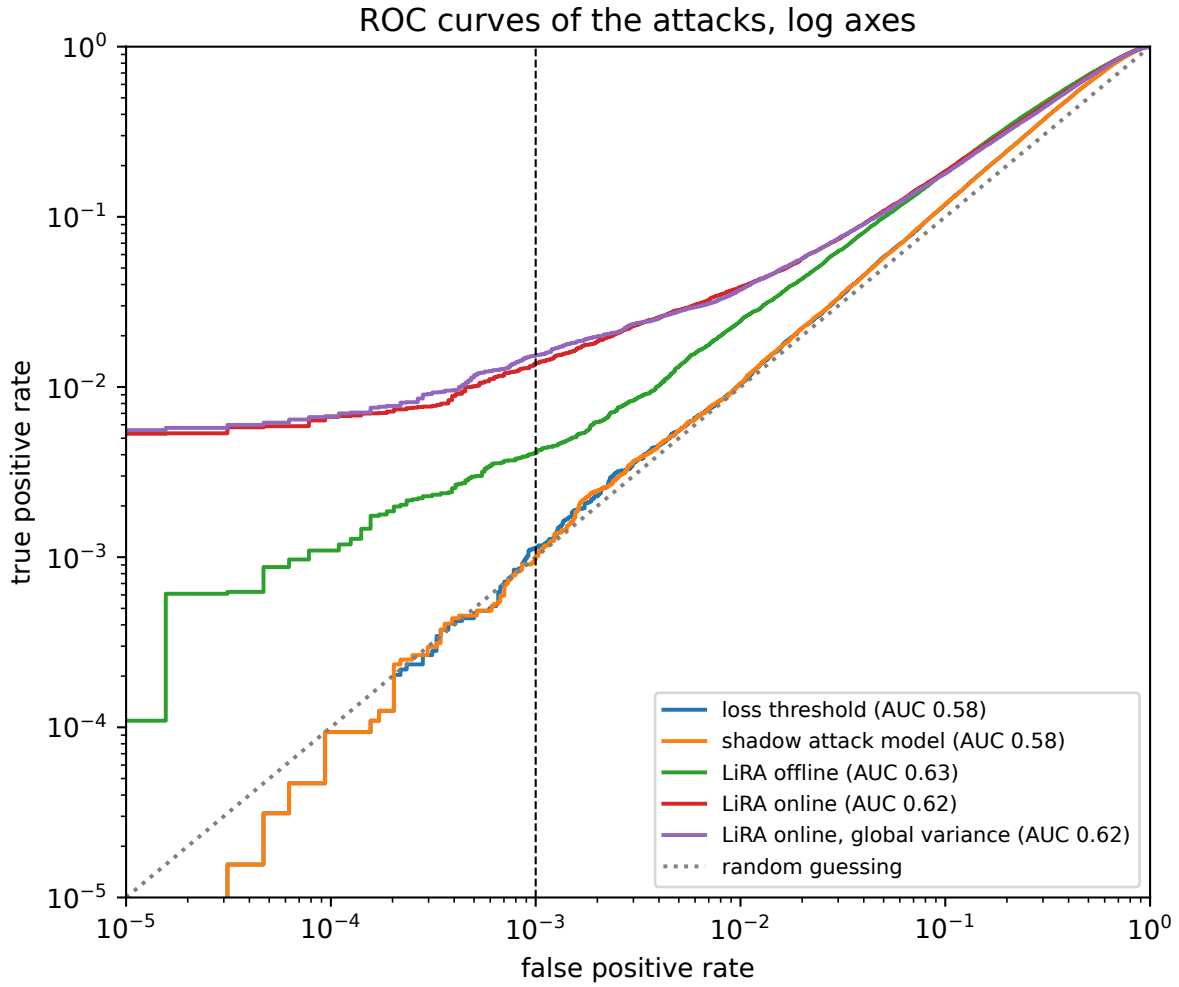


Figure 10.4.: ROC curves of the five attacks on log axes. The loss threshold and the shadow attack model follow the diagonal below FPR 0.01; online LiRA stays far above it. The dashed line marks FPR 10^{-3}.

10.6. Why outliers are most exposed

Only records whose IN and OUT distributions are far apart pass a strict threshold, so they make up the low-FPR end of the curve. Standardize a record's output against its OUT distribution: if both are Gaussians of common width σ_z, a non-member's output becomes a standard normal and a member's the same curve shifted by the **separation** s_z. The best test on that record then has

$$\mathrm{TPR}_z(\alpha) = 1 - \Phi(\Phi^{-1}(1-\alpha) - s_z), \qquad s_z = \frac{\mu_{\text{in}} - \mu_{\text{out}}}{\sigma_z}, \tag{10.3}$$

where Φ is the standard normal cumulative distribution function (CDF), so $\Phi^{-1}(1-\alpha)$ is the cut-off a non-member exceeds with probability α, about three standard deviations at 10^{-3}. A

separation of 0.5 leaves a member's mean far below that cut-off, for a TPR of 0.0048; a separation of 4 puts it beyond, for 0.8185. The numerator is the record's self-influence, how much training on z raises the model's output on z, and it is large when the rest of the data predict the record wrongly.

Table 10.4.: The running example, split into typical points and outliers.

	Typical points (950)	Outliers (50)
Mean separation s_z	0.461	1.539
Online LiRA at pooled FPR 10^{-3}: TPR	0.0020	0.2597
Online LiRA at pooled FPR 10^{-3}: FPR	0.00086	0.00375

At FPR 10^{-3}, online LiRA flags 26% of outlier members and 0.2% of typical ones (Table 10.4), so 87% of the members it finds are outliers, from 5% of the pool. Outliers whose coin-flip label happens to agree with the trend are predicted correctly unseen and are not exposed (Figure 10.5). So a low loss is normal for an easy record and strong evidence for a hard one.

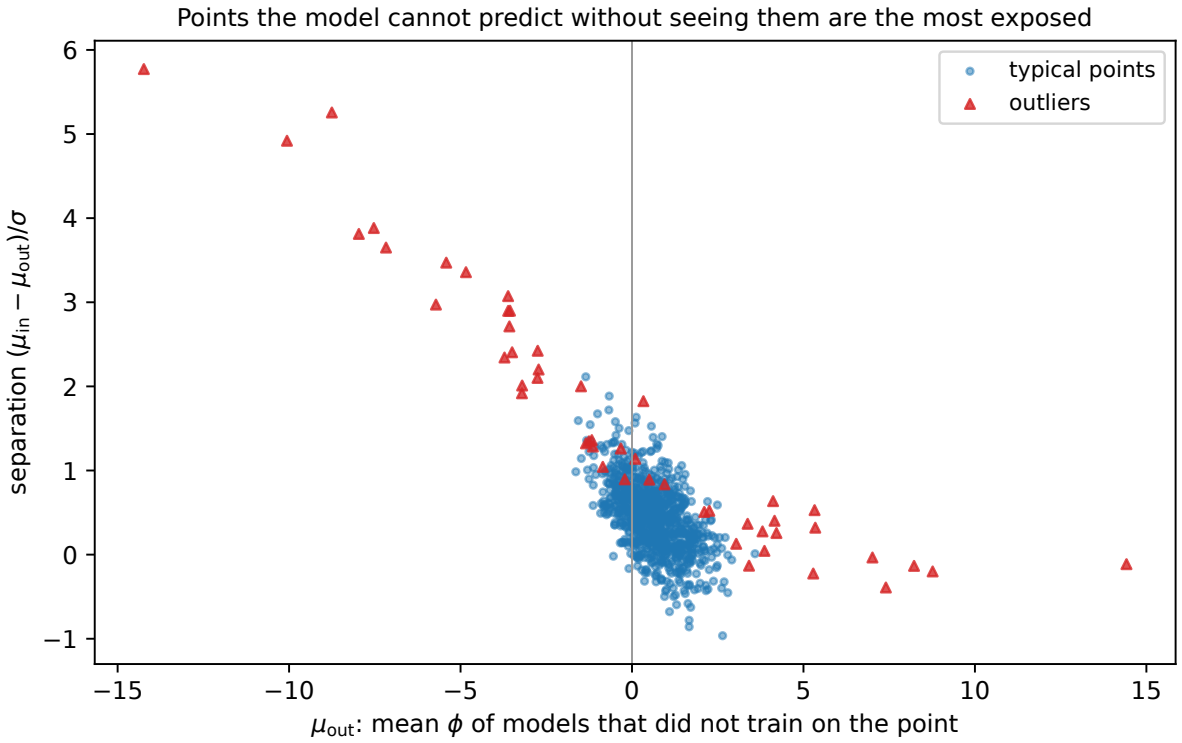


Figure 10.5.: Each pool point's separation against μ_{out}, the mean ϕ of the shadow models that did not train on it. Outliers (triangles) whose label the rest of the data contradict have the largest separations.

The loss attack measures the average self-influence, but the low-FPR end is governed by the largest, so a model with a small gap can still expose a few records completely. **Removing the outliers does not fix it.** A record's OUT distribution depends on what the other records predict about it, so exposure is relative: Carlini et al. (2022b) removed the most exposed records,

retrained, and found the next layer exposed. A defence has to bound the influence of the worst record, which is what differential privacy does.

10.7. Running LiRA on a real model

On an image classifier trained on CIFAR-10, the protocol is the running example's, with hours of training in place of seconds; Carlini et al. report online LiRA about ten times more powerful than earlier attacks there at low FPR.

1. **Split.** For each of $N + 1$ models choose a random half of the pool. One model plays the target; rotating it gives many (model, record) pairs.
2. **Train** every model with the target's exact recipe, including augmentation.
3. **Statistic.** Compute ϕ from the logits, on each record and a few augmented copies.
4. **Fit** IN and OUT Gaussians per record over the shadow models.
5. **Score** each (target model, record) pair with Equation 10.2, or the offline score.
6. **Evaluate.** Pool all pairs, sweep one threshold, plot the ROC curve on log axes, and report the TPR at FPR 10^{-3} with the number of non-member pairs behind it.

With 16 shadow models each record is IN for about 8, too few for per-record variances: on the running example 16 shadows give a TPR at FPR 10^{-3} of 0.0023 per-record and 0.0151 with a global variance. One target with 25,000 non-member images allows only 25 false positives at that FPR, so rotate over targets. To audit the worst case, plant **canaries**, records built to be as exposed as possible, as members.

10.8. Code

This reproduces the two-kinds example; the last two lines score a typical record's output 3.244 OUT standard deviations above its OUT mean, exactly on online LiRA's threshold.

```
from math import log, pi, sqrt
from statistics import NormalDist

Z = NormalDist()


def log_normal_pdf(x: float, mu: float, sd: float) -> float:
    return -0.5 * ((x - mu) / sd) ** 2 - log(sd * sqrt(2 * pi))


def lira_online(phi: float, mu_in: float, sd_in: float, mu_out: float, sd_out: float) -> flo
    """Log of how much more likely the target's phi is under the IN fit than under the OUT f
    return log_normal_pdf(phi, mu_in, sd_in) - log_normal_pdf(phi, mu_out, sd_out)


def lira_offline(phi: float, mu_out: float, sd_out: float) -> float:
```

```
    """How many OUT standard deviations the target's phi lies above its OUT mean."""
    return (phi - mu_out) / sd_out


def tpr_at_fpr(fpr: float, separation: float) -> float:
    """Best TPR at a given FPR for a record whose IN and OUT scores are unit normals `separa
    return 1 - Z.cdf(Z.inv_cdf(1 - fpr) - separation)


typical, atypical = tpr_at_fpr(1e-3, 0.5), tpr_at_fpr(1e-3, 4.0)
print(f"{typical:.4f}")  # 0.0048
print(f"{atypical:.4f}")  # 0.8185
print(f"{0.95 * typical + 0.05 * atypical:.4f}")  # 0.0455
print(f"{lira_offline(6.244, 3.0, 1.0):.3f}")  # 3.244
print(f"{lira_online(6.244, 3.5, 1.0, 3.0, 1.0):.3f}")  # 1.497
```

The full edition's script, membership_inference.py, runs every attack and prints every number in this chapter:

```
uv run python code/part02-attacks/membership_inference.py
```

10.9. Cheat sheet

Table 10.5.: The results of this chapter.

Result	Formula	Notes
Loss attack	Adv = expected gap $/B$; 0-1 loss: test error − train error	Average-case; a lower bound on leakage
Shadow attack model	Learns the membership probability of the output	Best record-blind attack; its real use is calibrating thresholds
Online LiRA	$\ln\mathcal{N}(\phi;\mu_{\text{in}},\sigma^2_{\text{in}}) - \ln\mathcal{N}(\phi;\mu_{\text{out}},\sigma^2_{\text{out}})$, one threshold for all records	Optimal when the fits are right; needs IN models per target
Offline LiRA	$(\phi-\mu_{\text{out}})/\sigma_{\text{out}}$	OUT models only, reusable
Per-record best TPR	$1-\Phi(\Phi^{-1}(1-\alpha)-s_z)$, $s_z=(\mu_{\text{in}}-\mu_{\text{out}})/\sigma_z$	Large separation: outliers
Measuring FPR α	About k/α non-members for k false positives	Relative error about $1/\sqrt{k}$

Rules of thumb

- Audit with LiRA, not with a global loss threshold, which is near chance at low FPR.

- Report the TPR at FPR 10^{-3} or lower, on log-scaled ROC curves, with the number of non-members behind it. Never accuracy or AUC alone.
- With fewer than about 64 shadow models, use a global variance.
- Expect the exposed records to be outliers, rare subgroups and mislabelled points; removing them exposes the next ones.
- Overfitting implies leakage; no overfitting does not imply its absence.

10.10. Further reading

- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022): LiRA online and offline, the global variance, and the case for TPR at low FPR. The protocol to follow in an audit.
- Shokri, Stronati, Song & Shmatikov, "Membership Inference Attacks against Machine Learning Models" (IEEE S&P 2017): shadow models and attack models.
- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (IEEE CSF 2018): the loss attack and its link to the generalization gap.
- Carlini, Jagielski, Zhang, Papernot, Terzis & Tramèr, "The Privacy Onion Effect: Memorization is Relative" (NeurIPS 2022): why removing the most exposed records does not help.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

11. Attribute inference and model inversion

11.1. Why this matters

A hospital trains a model that predicts a lung disease from a patient's age band and whether they smoke, and puts it behind an interface anyone can query. An insurer knows your age and, from a claim, that you have the disease. Can it use the model to find out whether you smoke? It can guess better than without the model. But that is not yet a privacy failure: if smokers get the disease more often, a published study saying so would allow the same guess about anyone with the disease. The question that matters for *you* is **whether the model reveals more about your attribute because your record was in its training data.**

Attribute inference asks an adversary to recover one sensitive field of a record, such as smoking status, a genotype or an income band, from the other fields and access to a model. **Model inversion** is the classic way to do it: query the model once for each candidate value of the sensitive field, and keep the value that best explains the label you already know.

The one idea: **compare with the guesser who never sees the model.** Picture a second insurer with no model but perfect knowledge of the population; its best guess from the visible fields is the **imputation baseline**. A model trained without your record was built from other people's data only, so it can teach the adversary about the population but nothing about you, and the baseline already assumes the population is known. So for a non-member no attack beats the baseline, while for a member it can. Success up to the baseline is population inference; success on members beyond what non-members get is the leak about individuals. Measuring it means comparing members with non-members, which makes attribute inference a membership question in disguise.

Throughout, a record is (s, x, y): a sensitive attribute s with a few values, public features x and a label y. The model's **confidence** $f_y(x, s)$ is the probability it gives label y at input (x, s).

11.2. The attribute inference game

"The attack recovered 78% of smoking statuses" sounds alarming, but it means nothing until you know how often the attribute could be guessed without the model, and how often the attack succeeds on people who were not in the training set. Yeom et al. (2018) built both comparisons into a game. A challenger trains a model, then flips a fair coin to pick the target either from the training set (a **member**) or fresh from the population (a **non-member**). The adversary sees the target's x and y and the model, and guesses s. The score is the difference,

$$\mathrm{Adv}^{\mathrm{attr}} = \Pr[\text{guess right} \mid \text{member}] - \Pr[\text{guess right} \mid \text{non-member}]. \tag{11.1}$$

Whatever the attack learned about the population helps members and non-members equally and cancels. What is left is what the model knows about its own training records.

The difference also ties attribute inference to membership. Hand an adversary the whole record and let it say "member" exactly when the attribute attack's guess is right. Its true positive rate (TPR) is the success on members, its false positive rate (FPR) the success on non-members, and its advantage TPR − FPR is the attribute advantage. So the attribute advantage obeys the membership ceiling, the total variation distance between what the adversary sees for members and for non-members, and a model that leaks nothing about membership leaks nothing about attributes either.

In this chapter's toy experiment, a logistic regression with a binary sensitive attribute (prior 0.3), ten public features and a binary label is trained on $n = 100$ records, 200 times. The inversion attack of Section 11.4 is right on 0.7832 of members and 0.7262 of non-members, an advantage of 0.0569 with a standard error of 0.0045. Read as a membership test, its likelihood ratio when it fires is only 1.08 and its balanced accuracy 0.5285. Most of its 78% success would be there if the person had never been in the data. This is also why an attribute attack is a weak membership test at a low FPR: it outputs one bit and is right on most people either way, so a "right" verdict is barely evidence. To measure membership, use the attacks of Membership inference in the full edition; to report an attribute attack, give its success on members, on non-members and the imputation baseline, never one rate.

11.3. The imputation baseline

Population inference is worth measuring directly, since any model of the population, or any published statistic, gives it away. Filling in a missing field from the others is **imputation**, and the best possible imputation, by a guesser who knows the population exactly, picks for each visible (x, y) the value of s most probable in the population: the **maximum a posteriori (MAP)** guess. It is the best because, for each visible (x, y), any guess is right exactly on the probability of the value it names, so naming the most probable value wins the most. Many published attribute attacks do no better than imputation from similar data (Jayaraman & Evans 2022).

For a non-member, the model was trained on other people only, so as far as this record goes it is an observation independent of it, and an independent observation cannot change which value of s is most likely. Every attack therefore satisfies the **non-member ceiling**

$$\Pr[\text{guess right} \mid \text{non-member}] \le V(S \mid X, Y) = \sum_{x,y} \max_{s} p(s, x, y), \tag{11.2}$$

where $p(s, x, y)$ is the population probability of a record, and V, the **vulnerability**, is the success of the MAP guess: for each visible (x, y), the probability of its most likely s, summed. An attack that ignores the model cannot tell the worlds apart at all, because a member, ignoring the model, is just a record drawn from the population. So model-free attacks have zero advantage, and Bayes imputation reaches the ceiling for members and non-members alike.

A worked population makes this concrete. Smoking has prior 0.3; a fraction 0.4 of non-smokers and 0.6 of smokers are old; the disease has probability 0.1, 0.3, 0.4 and 0.7 for young non-smokers, old non-smokers, young smokers and old smokers. The probability of a record is the chance of

its smoking status, times the chance of its age band given that, times the chance of its disease outcome given both, which gives Table 11.1.

Table 11.1.: The smoking table: the joint distribution of smoking status, age band and disease, with the column maxima in bold and the posterior probability of smoking in each column.

$p(s, x, y)$	young, no disease	young, disease	old, no disease	old, disease
non-smoker ($s = 0$)	**0.378**	0.042	**0.196**	0.084
smoker ($s = 1$)	0.072	**0.048**	0.054	**0.126**
Pr[smoker $\mid x, y$]	0.1600	0.5333	0.2160	0.6000

With no observation the best guess is "non-smoker", right 0.7 of the time. Age alone changes the probability of smoking but never pushes it past one half, so the guess stays "non-smoker" and the success stays 0.7. Adding the disease pushes the posterior past one half in both disease columns (Figure 11.1), the guess becomes "smoker" there, and the success is the sum of the bold entries, $V(S \mid X, Y) = 0.748$, a min-entropy leakage of 0.0957 bits. A good model trained on 10,000 patients from this population lets an insurer reach 0.748 on *anyone*, and no non-member can be attacked better.

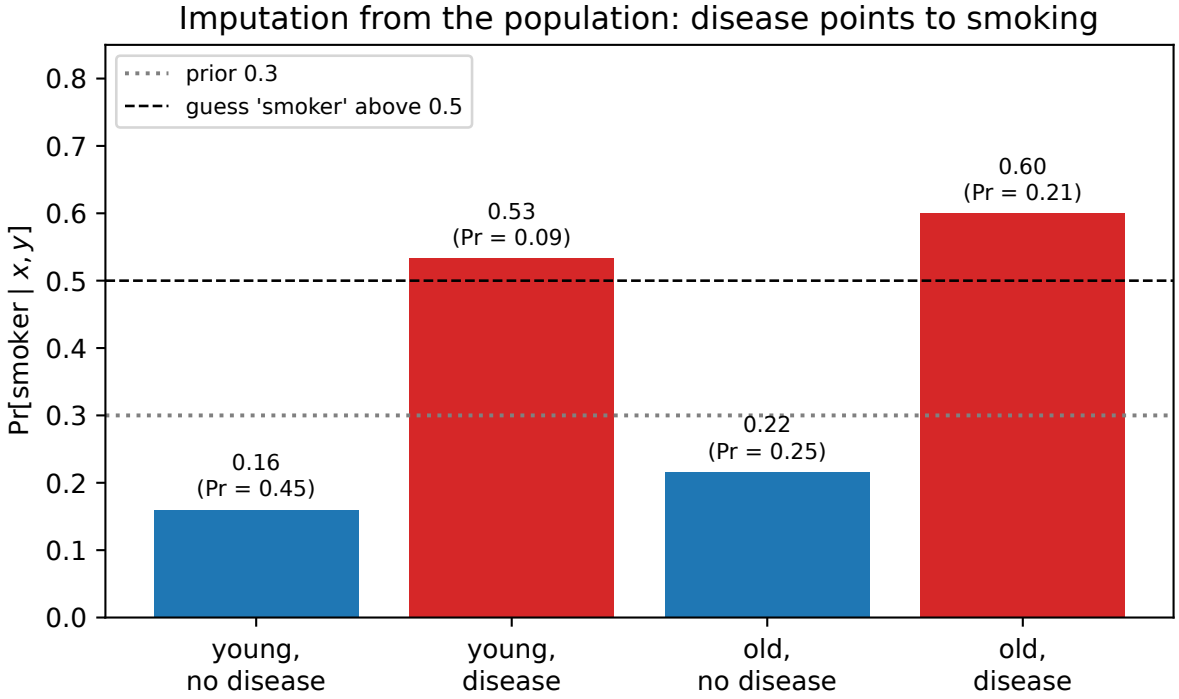


Figure 11.1.: The posterior probability of smoking in each cell of the smoking table, with each cell's probability in brackets. Red bars cross $\frac{1}{2}$, where imputation guesses "smoker".

The ceiling is a property of the population, not of the model, and it can be high: if x and y nearly determine s, every model and every statistic "reveals" s. That is the "smoking causes cancer" case

of Dwork & Roth (2014): the study disclosed a fact about the world, not about its participants. It can still harm people, but no mechanism that limits what one record contributes can prevent it, since the harm would remain with your record removed; it is a question for policy and fairness. In an audit, estimate the baseline with the best imputation model you can fit on data from the same population, and run the attack on held-out non-members too.

11.4. Model inversion: the MAP attack

Imputation needs the link $p(y \mid x, s)$ between label and attribute, and the adversary usually does not know it. But a model that predicts y from x and s is an estimate of exactly that link. Fredrikson et al. (2014) used this on a warfarin dosing model to infer patients' genetic markers from their demographics and dose, and called it **model inversion**; confidences make it stronger (Fredrikson, Jha & Ristenpart 2015). The attack is Bayes' rule: the value of s that best explains what you see maximizes "how plausible is s for someone with these features" times "how likely is the known label if the attribute were s". The adversary supplies the first factor from what it knows about the population and gets the second by querying the model at each candidate:

$$\hat{s}(x, y) = \arg\max_{s} \; p(s \mid x)\, f_y(x, s). \tag{11.3}$$

Use $p(s \mid x)$, not the marginal $p(s)$ that Fredrikson et al. used, or the attack throws away what x says about s.

Everything the attack does comes from how the model's confidences differ from the population's probabilities. If the model is **calibrated**, its confidences equal those probabilities, and inversion *is* imputation, with success $V(S \mid X, Y)$ on members and non-members alike. Take an old patient with the disease in the smoking table. The adversary knows Pr[smoker | old] = 0.3913 and queries a calibrated model, which returns 0.7 for a smoker and 0.3 for a non-smoker. The scores are 0.2739 for "smoker" and 0.1826 for "non-smoker", so it guesses "smoker", and normalized they give exactly the posterior 0.6000 of Table 11.1.

A trained model differs from the population in two ways, one per world. On members it is too sure of the truth: training raises the confidence in each record's label at its *true* s, while the other value was never paired with that record, so the confidence there stays closer to honest. The ratio that decides the attack is tilted towards the truth, which is overfitting, the generalization gap of Membership inference in the full edition seen through the sensitive attribute. On non-members the model's confidences carry the noise of a small training set rather than the population's exact probabilities, so the attack does *worse* than imputation. On the toy model, Bayes imputation is right 0.7517 of the time. MAP inversion with the trained model is right on 0.7832 of members, 0.0315 *above* that ceiling, which no knowledge of the population could produce, and on 0.7262 of non-members, 0.0254 below it. So weak results on fresh records say nothing about members.

More data closes both gaps, because the model approaches the population: the advantage is 0.1610 at $n = 25$, 0.0111 at $n = 400$ and 0.0052 at $n = 1600$ (Figure 11.2).

The same idea can invert a whole input: on a face recognition model whose classes are people, searching for the image that maximizes one class's confidence recovers a blurred, recognizable face. That is population inference about the class, and a privacy problem because the class is one person.

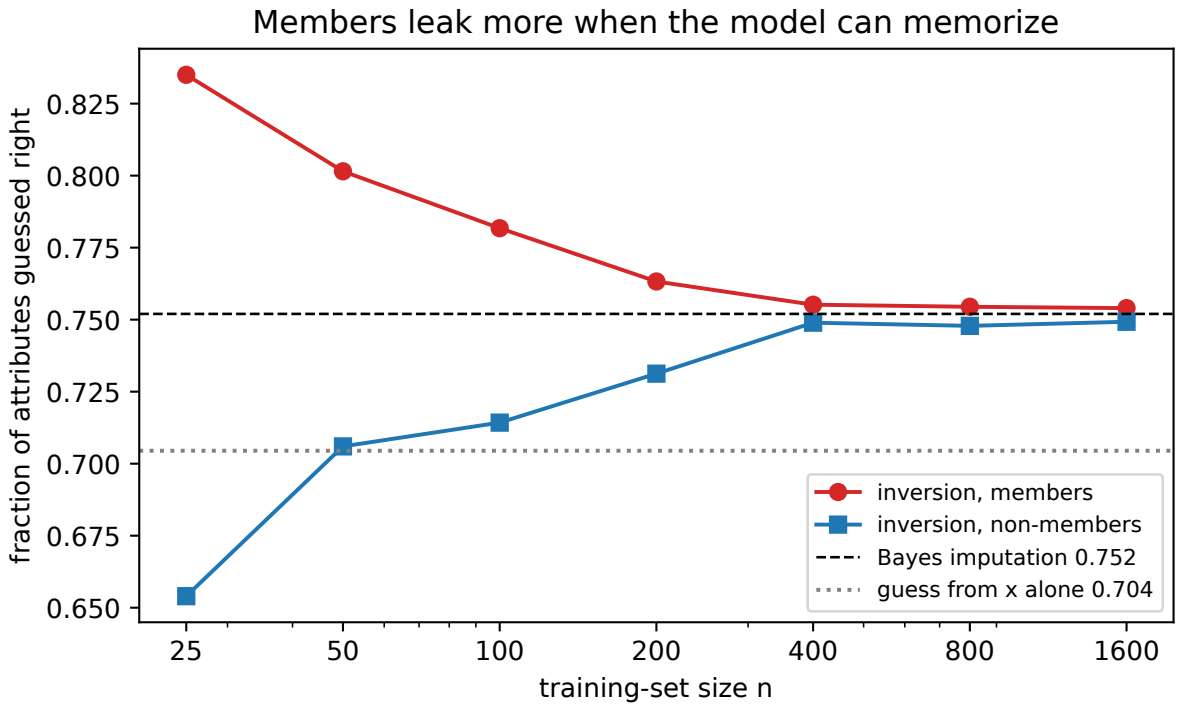


Figure 11.2.: Success of MAP inversion on members and non-members against the training-set size n. Members start above the Bayes imputation ceiling and non-members below it; both converge to the ceiling as the model approaches the population.

11.5. Confidence masking

If confidences make inversion stronger, why release them? Rounding them, or returning only the top label, is cheap and leaves the prediction unchanged. Such **confidence masking** only coarsens what the adversary sees, and whoever sees the full answers could apply the mask themselves, so it can never help the best attacker. For any masking function g applied to the answers F the adversary gets by querying every candidate s,

$$V(S \mid X, Y, g(F)) \leq V(S \mid X, Y, F), \tag{11.4}$$

on members and on non-members separately. But the inequality says nothing about the *difference* between the two worlds, which is the leak, and masking lowers each by amounts that need not match.

Labels alone still leak, for the same reason confidences do. On a member, the model usually predicts the training label at the true s, because it fit that record, and an honest label at the other value, so "the label matches at only one candidate" points to the truth. On a non-member the two predicted labels usually agree, and the attack falls back on the guess from x alone. So masking hurts population inference more than the individual leak. Table 11.2 attacks the same 200 models through three interfaces. Rounding to one decimal changes almost nothing, since the comparison between candidates rarely changes sign. Returning labels only drops non-member success to 0.7092, close to the 0.7045 of guessing from x alone, while member success stays 0.0224 above the imputation ceiling, so the advantage *grows* from 0.0569 to 0.0650.

Table 11.2.: MAP inversion success through three interfaces, averaged over 200 models trained on 100 records each. Guessing from x alone gives 0.7045 and Bayes imputation 0.7517.

Interface	Members	Non-members	Advantage
Full confidences	0.7832	0.7262	0.0569
Confidences rounded to one decimal	0.7836	0.7253	0.0583
Top label only	0.7741	0.7092	0.0650

Masking is obfuscation, not a guarantee: it removes one attack's input without bounding what any attack learns about a member. So don't count a drop in overall attack success as a privacy gain; check the member/non-member gap. What shrinks the gap is less overfitting, for example more training data, and what bounds it for every attacker and every output format is differential privacy.

11.6. Code

The imputation baseline and the inversion scores of the smoking example take a few lines with the standard library:

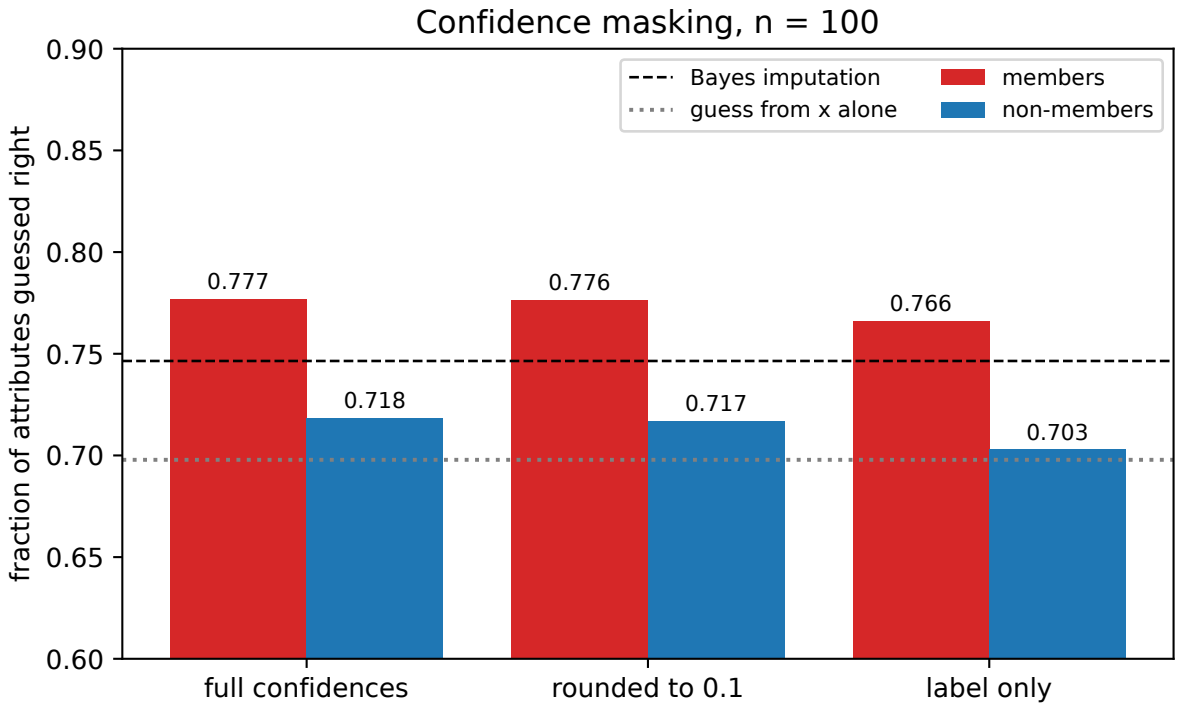


Figure 11.3.: MAP inversion success on members and non-members through three interfaces. The dashed line is Bayes imputation, the dotted line the guess from the public features alone.

```
from itertools import product
from math import log2

P_SMOKER = 0.3
P_OLD = {0: 0.4, 1: 0.6}  # Pr[old | s]
P_DISEASE = {(0, 0): 0.1, (0, 1): 0.3, (1, 0): 0.4, (1, 1): 0.7}  # Pr[disease | s, old]


def joint(s: int, old: int, disease: int) -> float:
    """Probability of the record (smoker s, old, disease) in the population."""
    p_s = P_SMOKER if s else 1 - P_SMOKER
    p_x = P_OLD[s] if old else 1 - P_OLD[s]
    p_y = P_DISEASE[(s, old)] if disease else 1 - P_DISEASE[(s, old)]
    return p_s * p_x * p_y


def best_guess_success(sees_age: bool, sees_disease: bool) -> float:
    """Success of the MAP guess of s for a guesser who knows the population and sees the cho
    cells: dict[tuple[int, int], list[float]] = {}
    for s, old, disease in product((0, 1), repeat=3):
        key = (old if sees_age else -1, disease if sees_disease else -1)
        cells.setdefault(key, [0.0, 0.0])[s] += joint(s, old, disease)
    return sum(max(cell) for cell in cells.values())


def inversion_scores(old: int, confidence: dict[int, float]) -> dict[int, float]:
    """MAP inversion: prior of s given the age band, times the model's confidence in the kno
    prior = {s: (P_SMOKER if s else 1 - P_SMOKER) * (P_OLD[s] if old else 1 - P_OLD[s]) for
    total = sum(prior.values())
    return {s: prior[s] / total * confidence[s] for s in (0, 1)}


v0, vx, vxy = best_guess_success(False, False), best_guess_success(True, False), best_guess_
print(f"{v0:.4f} {vx:.4f} {vxy:.4f}")  # 0.7000 0.7000 0.7480
print(f"{log2(vxy / v0):.4f}")  # 0.0957
scores = inversion_scores(old=1, confidence={0: 0.3, 1: 0.7})  # a calibrated model, queried
print(f"{scores[1]:.4f} {scores[0]:.4f}")  # 0.2739 0.1826
print(f"{scores[1] / (scores[0] + scores[1]):.4f}")  # 0.6000
```

The full edition's script, attribute_inference.py, plays the game on the toy model and prints every number in this chapter:

```
uv run python code/part02-attacks/attribute_inference.py
```

11.7. Cheat sheet

Table 11.3.: The results of this chapter.

Result	Statement	Notes
Attribute advantage	Success on members − success on non-members	Population inference cancels
Reduction to membership	"Member if the guess is right": TPR − FPR = attribute advantage	Bounded by the membership ceiling; weak at low FPR
Non-member ceiling	Success on non-members $\leq V(S \mid X, Y) = \sum_{x,y} \max_s p(s, x, y)$	Imputation reaches it; model-free attacks have zero advantage
MAP inversion	$\hat{s} = \arg\max_s p(s \mid x)\, f_y(x, s)$	Calibrated model: inversion is imputation
Masking	$V(S \mid X, Y, g(F)) \leq V(S \mid X, Y, F)$	The member/non-member gap can grow

Rules of thumb

- Report success on members, on non-members, and the imputation baseline. One success rate says nothing about privacy.
- Success up to the imputation baseline is population inference; any model or statistic of the population gives it.
- Success on members beyond non-members is the individual leak. It comes from overfitting and shrinks as the training set grows.
- Rounding confidences or returning labels only changes the attack, not what the model memorized.

11.8. Further reading

- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (IEEE CSF 2018): the attribute game with members and non-members, and its link to membership and overfitting.
- Fredrikson, Jha & Ristenpart, "Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures" (CCS 2015): inversion from confidences, face reconstruction, and rounding as a countermeasure.
- Jayaraman & Evans, "Are Attribute Inference Attacks Just Imputation?" (CCS 2022): the imputation baseline, and what it takes to beat it.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

12. Property inference

12.1. Why this matters

A hospital trains a readmission model on its 1,000 patients and shares it with a research partner. You have checked that no attack can tell, much better than a coin flip, whether any one patient was in the training set. The partner can still learn whether 45% or 55% of the hospital's patients are over 65, and be right almost every time. That secret is about no patient in particular. It is a **property** of the training set as a whole, a statistic such as the fraction t of records in some subgroup, and **property inference** is the attack that recovers it from a trained model (Ateniese et al. 2015; Ganju et al. 2018).

How can a model hide every record and still give away their mix? Think of it as an average over its n records, plus noise. The luck of which records were sampled moves an average by about $1/\sqrt{n}$; changing one record moves it by about $1/n$, which is lost in that noise, so membership inference gets harder as n grows. Changing the subgroup's share from t_0 to t_1 alters about $n|t_1 - t_0|$ records at once, and together they move the model by an amount proportional to $|t_1 - t_0|$ that does not shrink with n. Against noise of size $1/\sqrt{n}$, the signal is about $\sqrt{n}\,|t_1 - t_0|$: property inference gets *easier* with more data. That comparison of scales explains why record-level defences miss properties, why deleting a column does not help, and how to predict an attack's accuracy before running it.

12.2. The property game and its ceiling

As for every attack in Threat models and privacy games in the full edition, write the attack as a game. A challenger flips a fair bit b, draws n records from a population in which the subgroup has share t_b, trains a model, and hands the attacker the weights (white-box) or answers to queries (black-box). The attacker knows the two shares, has data from both groups, and can train models the way the victim does. It guesses b. Suri & Evans (2022) call this **distribution inference**.

There is no target record, and that changes the score. A membership attacker faces many targets, and a few exposed with high confidence matter, so you report the true positive rate (TPR) at a low false positive rate (FPR). Here each dataset carries one secret, so the natural score is plain **accuracy**, the probability of guessing b, against a baseline of $\frac{1}{2}$. With a fair bit it equals $\frac{1}{2}(1 + \text{advantage})$, where the **advantage** is $\text{TPR} - \text{FPR}$.

What is the most any attacker can win? Its only evidence is that some views are more likely under one share than the other, so the best it can do is name the share that makes its view more likely, and that attacker's advantage is the total variation (TV) distance between the two distributions of the model. The model, being computed from the data, cannot separate the two worlds better than the data do: anyone holding the data could train it and run the same test.

When each record shows its group, the only thing in the data that bears on t is the binomial count of subgroup records. So for every model trained on the data,

$$\text{advantage} \le \mathrm{TV}(\mathrm{Bin}(n, t_0), \mathrm{Bin}(n, t_1)). \tag{12.1}$$

Releasing the count attains this ceiling, because it keeps everything the data say about t. A model that never sees the group leaks less, and how much less is something you measure.

For the hospital, the count has mean 450 in one world and 550 in the other, with a standard deviation of about 15.7. The means are more than six standard deviations apart, and Equation 12.1 gives 0.9985: an attacker who can read the count off the model is right more than 99.9% of the time. At $n = 100$ the means are 45 and 55, the standard deviation is about 5, the distributions overlap much more, and the ceiling falls to 0.6827; 200,000 simulated games reach 0.6811.

Whether this is a harm depends on whom the property describes. It arises when the property describes the *data owner*: one hospital's patient mix, one company's customers, one lab's machines. If the dataset is a small sample of a public population, its property is public anyway.

12.3. Why a property leaks when no record does

On the released count both attackers can be analysed exactly. The membership attacker asks whether its target, a subgroup member, is in the data. The target adds exactly 1 to the count, where the record in its place would add 1 only a fraction t of the time, so the attacker is looking for one extra unit in a count that already wanders by about $\sqrt{n}$. The property attacker asks whether the share was t_0 or t_1; the count's fraction centres on one or the other, with a spread of about $\sqrt{\bar{t}(1-\bar{t})/n}$ around the midpoint share $\bar{t} = \frac{t_0+t_1}{2}$, and what matters is how many spreads separate the centres. With Φ the standard normal distribution function, the best advantages are approximately

$$\mathrm{Adv}_{\mathrm{mem}} \approx \frac{1-t}{\sqrt{2\pi(n-1)t(1-t)}}, \qquad \mathrm{Adv}_{\mathrm{prop}} \approx 2\Phi\Big(\frac{\sqrt{n}\,|t_1 - t_0|}{2\sqrt{\bar{t}(1-\bar{t})}}\Big) - 1. \tag{12.2}$$

The first is one unit over the spread of the count, so it falls like $1/\sqrt{n}$. The second depends on n and the gap only through $\sqrt{n}\,|t_1 - t_0|$, so doubling the gap has the same effect as multiplying n by 4.

Table 12.1 puts numbers on both; Section 12.6 computes them. At $n = 10{,}000$ the membership attacker is right with probability 0.502, and the property attacker is never wrong to four decimals. The data, the release and the attacker's knowledge are the same; only the question differs (Figure 12.1).

Table 12.1.: Best advantages from a released count of subgroup records. Property: $t_0 = 0.45$ against $t_1 = 0.55$. Membership: one subgroup record, population share $t = 0.5$.

n	Property, exact	Property, formula	Membership, exact	Membership, formula
100	0.6827	0.6827	0.0398	0.0401
1,000	0.9985	0.9984	0.0126	0.0126

n	Property, exact	Property, formula	Membership, exact	Membership, formula
10,000	1.0000	1.0000	0.0040	0.0040

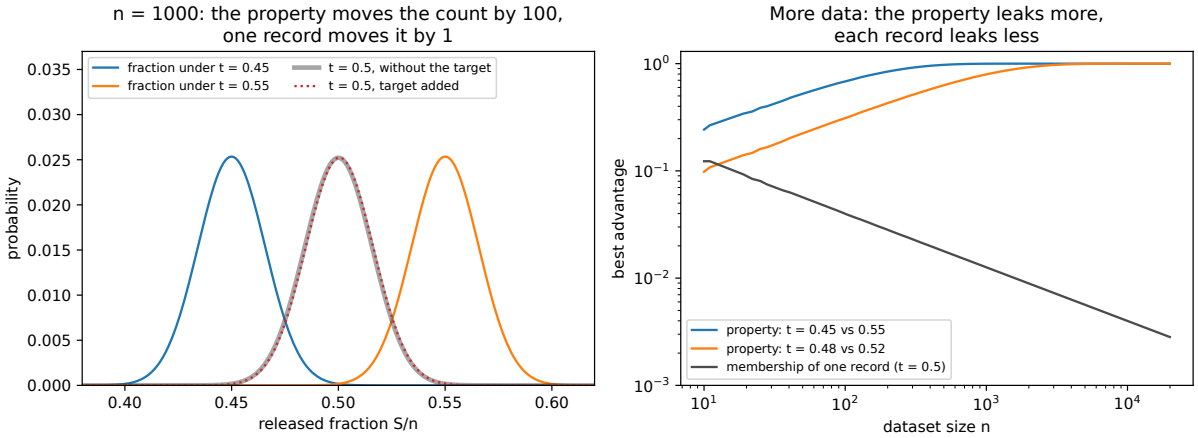


Figure 12.1.: Left: the released fraction of subgroup records at $n = 1{,}000$ under shares 0.45 and 0.55, which barely overlap, and under share 0.5 with and without one target record, which are indistinguishable. Right: the best property advantage for two gaps and the best membership advantage of one record, against n. More data protects the record and exposes the property.

So a property is a group of records, and a protection measured one record at a time says little about it. For the hospital, moving from 45% to 55% changes 100 of its 1,000 patients, and differential privacy covers groups only through its group-privacy property, whose guarantee weakens with the size of the group (Properties of differential privacy). To hide the property, the release must be noisy on the scale of $n|t_1 - t_0|$ records, and noise that large also destroys what the release was for: hiding the property means hiding the population. Deleting the group column does not escape this, because whatever correlates with the group still changes what the model fits, as the next section shows. And when outsiders contribute training data, audit with poisoning in scope: a few injected records can make the property far easier to infer (Mahloujifar et al. 2022).

12.4. Shadow models and meta-classifiers

For a real model nobody can write down the distribution of the trained weights, so the attacker samples it. It draws many datasets with share t_0 and many with t_1 from auxiliary data, trains a **shadow model** on each exactly as the victim trains, labels each shadow model's parameters with its share, and trains a classifier on those parameter vectors, the **meta-classifier**, which it then applies to the victim's model. This is the shadow-model recipe of Membership inference in the full edition, with whole models as the data points.

Which meta-classifier, and how accurate? A model fitted by minimizing an average loss really does behave like an average: its parameters scatter approximately as a Gaussian around the **population fit** $\theta(t)$, the model infinitely many records would give, with a covariance Σ_1/n that

shrinks like $1/n$. Between two Gaussians with the same covariance, the quadratic parts of the log-densities cancel and the log-likelihood ratio is linear, so the best test is a hyperplane between the two clouds: train a **linear discriminant**. Its accuracy depends only on how far apart the centres are in units of the noise. For a small gap they differ by the slope θ' of the population fit times the gap, which gives

$$\text{accuracy} \approx \Phi(\kappa\sqrt{n}\,|t_1 - t_0|/2), \qquad \kappa = \sqrt{\theta'^{\top}\Sigma_1^{-1}\theta'}. \tag{12.3}$$

κ is how fast the model moves with the share, in units of its per-record noise. If the property does not change what the model fits, $\theta' = 0$ and nothing leaks to first order, however large n is. If it does, the leak is a side effect of fitting the data well, and no care in the training code removes it. κ also compares any model with the count, whose $\kappa = 1/\sqrt{\bar{t}(1-\bar{t})}$ is 2 at $\bar{t} = \frac{1}{2}$: a model with a smaller κ leaks the property as well as the count would on a fraction $(\kappa\sqrt{\bar{t}(1-\bar{t})})^2$ of its records, the unit in which Suri & Evans (2022) report leakage.

Here is a model that never sees the group and leaks anyway. Records have two features and a label; in the majority risk rises with the second feature, in the subgroup it falls. A logistic regression sees features and labels only. At share $\frac{1}{2}$ the two rules cancel in the second weight, and as the subgroup grows its rule wins: the population fit's second weight moves from 0.442 at share 0.25 to -0.442 at share 0.75. Against that weight's per-record noise this gives $\kappa = 0.647$, against 2 for the count, so the model leaks the property as well as $(0.647/2)^2 = 0.105$ of its records would, about one in ten.

With 150 shadow models per hypothesis and 150 fresh target models, the meta-classifier's accuracy matches Equation 12.3 in most cells of Table 12.2 to within two standard errors, each at most 0.029. Plotted against $\sqrt{n}\,|t_1 - t_0|$, the four gaps fall on one curve (Figure 12.2): a gap of 0.05 at $n = 1{,}000$ is about as easy as a gap of 0.1 at $n = 200$, since $\sqrt{1000} \times 0.05 = 1.58$ against $\sqrt{200} \times 0.1 = 1.41$.

Table 12.2.: Accuracy of the linear meta-classifier against logistic regressions that never see the group, by dataset size n and gap $|t_1 - t_0|$, with the prediction of Equation 12.3 in parentheses.

n	gap 0.05	gap 0.1	gap 0.2	gap 0.4
50	0.523 (0.546)	0.520 (0.590)	0.653 (0.676)	0.830 (0.818)
100	0.523 (0.564)	0.670 (0.627)	0.733 (0.741)	0.863 (0.901)
200	0.503 (0.590)	0.670 (0.676)	0.803 (0.819)	0.957 (0.965)
500	0.633 (0.641)	0.787 (0.765)	0.910 (0.926)	0.997 (0.998)
1,000	0.713 (0.695)	0.867 (0.847)	0.980 (0.979)	1.000 (1.000)

The misses are where $n \leq 200$ and the gap is at most 0.1. There the true effect is small, the noise in the shadow models swamps it, and the meta-classifier finds the separating direction badly: the attack is weaker than the best test, not the model safer. So an auditor should report the accuracy predicted from κ next to the measured one. Shadow models also need data from both groups, the victim's algorithm and its dataset size, or they sample the wrong distribution.

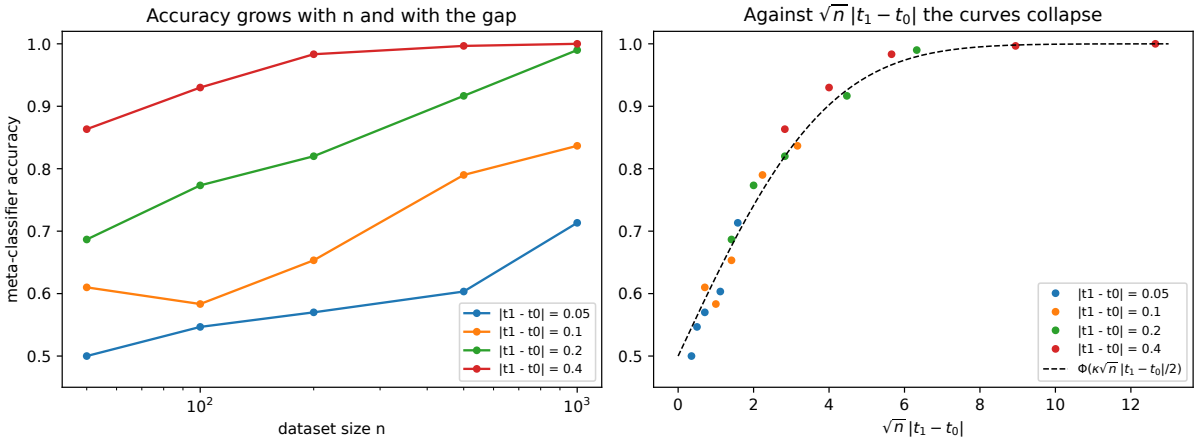


Figure 12.2.: Left: meta-classifier accuracy against n for four gaps between the two shares. Right: the same accuracies against $\sqrt{n}\,|t_1 - t_0|$, where they collapse onto the predicted curve with $\kappa = 0.647$.

12.5. Neural networks and permutation invariance

Swap two hidden units of a network, with their incoming and outgoing weights, and it computes the same function. For tanh units, flipping the signs of a unit's weights in and out also changes nothing, because tanh is odd. Training from a symmetric random start keeps these symmetries, so each weight slot holds a random one of the network's units with a random sign. The property shows in *which* units the network has, not in which slot holds which, and averaging slot by slot washes it out: for tanh networks every raw weight has mean zero under both hypotheses, and a linear meta-classifier on raw weights is at chance (Ganju et al. 2018).

The fix is a representation that ignores order and sign. A **canonical form** flips each unit so its outgoing weight is positive and sorts the units by it; ReLU networks have no sign symmetry, so sorting alone suffices. It is fragile when two units have nearly equal outgoing weights, which swap places from one network to the next. A **set representation** sums a function of each unit's parameter vector u_j, and a sum ignores order; $\sum_j u_j u_j^\top$ ignores signs too, and a learned DeepSets network (Zaheer et al. 2017) is the general form. **Probe outputs**, the model's answers on fixed inputs, need no fix: they measure the function, which is what the property changes, and a function does not depend on how it is parameterized. Table 12.3 compares them on tanh networks with 4 hidden units, trained on 200 records with share 0.2 or 0.8, 150 shadow and 150 target networks per side.

Table 12.3.: The same linear meta-classifier against the same tanh networks, by representation of the network; v_j is unit j's outgoing weight.

Representation	Features	Accuracy
Raw parameters	17	0.497
Canonical order (flip so $v_j > 0$, sort by v_j)	17	0.807
Unit second moments $\sum_j u_j u_j^\top$	11	0.877
Outputs on 16 probe inputs	16	1.000

The ranking follows the argument, and it means that a meta-classifier at chance on raw weights says the representation is wrong, not that the model is safe, and deeper networks need an invariant representation at every layer. The table also corrects an intuition about access. White-box access is never weaker for the *best* attack, since whoever holds the weights can compute any query answer, but for a fixed simple attack the representation matters more than the access, and here black-box probes beat raw weights. A model served only through an API is not safe from property inference (Zhang, Tople & Ohrimenko 2021).

12.6. Code

The ceiling of Equation 12.1 and the two advantages of Section 12.3 take a few lines:

```
from math import erf, exp, lgamma, log, sqrt


def binom_pmf(n: int, t: float) -> list[float]:
    """Pr[Bin(n, t) = k] for k = 0..n, in log space so that large n does not overflow."""
    return [exp(lgamma(n + 1) - lgamma(k + 1) - lgamma(n - k + 1) + k * log(t) + (n - k) * l


def property_ceiling(n: int, t0: float, t1: float) -> float:
    """Best advantage of any property attack when the group is visible: TV(Bin(n, t0), Bin(n
    return 0.5 * sum(abs(p - q) for p, q in zip(binom_pmf(n, t0), binom_pmf(n, t1)))


def property_normal(n: int, t0: float, t1: float) -> float:
    """The normal approximation 2 Phi(sqrt(n) |t1 - t0| / (2 sqrt(tbar (1 - tbar)))) - 1."""
    tbar = (t0 + t1) / 2
    z = sqrt(n) * abs(t1 - t0) / (2 * sqrt(tbar * (1 - tbar)))
    return erf(z / sqrt(2))  # equals 2 Phi(z) - 1


def membership_best(n: int, t: float) -> float:
    """Best advantage for one subgroup record from the same count: (1 - t) max_k Pr[Bin(n -
    return (1 - t) * max(binom_pmf(n - 1, t))


print(f"{property_ceiling(100, 0.45, 0.55):.4f} {property_normal(100, 0.45, 0.55):.4f}")  #
print(f"{property_ceiling(1000, 0.45, 0.55):.4f} {property_normal(1000, 0.45, 0.55):.4f}")
print(f"{membership_best(100, 0.5):.4f} {membership_best(10000, 0.5):.4f}")  # 0.0398 0.0040
```

The full edition's script runs the shadow-model attacks and reproduces every number in this chapter:

```
uv run python code/part02-attacks/property_inference.py
```

12.7. Cheat sheet

Table 12.4.: The results of this chapter.

Result	Formula	Notes
Score	Accuracy $= \frac{1}{2}(1 + \mathrm{TPR} - \mathrm{FPR})$	One secret per dataset; baseline $\frac{1}{2}$
Ceiling, group visible	$\mathrm{TV}(\mathrm{Bin}(n, t_0), \mathrm{Bin}(n, t_1))$	Attained by releasing the count
Property, released count	$2\Phi(\sqrt{n}\,\lvert t_1 - t_0\rvert/(2\sqrt{\bar{t}(1-\bar{t})})) - 1$	$\to 1$ as n grows
Membership, released count	$\approx (1-t)/\sqrt{2\pi(n-1)t(1-t)}$	$\to 0$ like $1/\sqrt{n}$
Meta-classifier accuracy	$\Phi(\kappa\sqrt{n}\,\lvert t_1 - t_0\rvert/2)$	Linear discriminant on parameters; count has $\kappa = 1/\sqrt{\bar{t}(1-\bar{t})}$
Neural networks	Raw weights at chance (tanh)	Canonical order, unit second moments, or probe outputs

Rules of thumb

- Property inference gets easier with more data, membership inference harder. What matters is $\sqrt{n}\,\lvert t_1 - t_0\rvert$.
- A property changes about $n\lvert t_1 - t_0\rvert$ records at once. Defences sized for one record do not cover it.
- Audit with shadow models trained like the victim and a representation that respects the network's symmetries; report accuracy next to the prediction from κ.

12.8. Further reading

- Ganju, Wang, Yang, Gunter & Borisov, "Property Inference Attacks on Fully Connected Neural Networks using Permutation Invariant Representations" (CCS 2018): the practical attack on neural networks, neuron sorting and the DeepSets meta-classifier.
- Suri & Evans, "Formalizing and Estimating Distribution Inference Risks" (PoPETs 2022): the property game as a test between two training distributions, and leakage reported in records.
- Zhang, Tople & Ohrimenko, "Leakage of Dataset Properties in Multi-Party Machine Learning" (USENIX Security 2021): black-box property inference, including properties of attributes removed from the training data.
- Mahloujifar, Ghosh & Chase, "Property Inference from Poisoning" (IEEE S&P 2022): how a few injected records make a property far easier to infer.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

13. Memorization and training-data extraction

13.1. Why this matters

A language model trained on scraped text sees phone numbers, addresses, keys and private messages. Ask it to continue "Anna Smith's phone number is" and it may finish with the real number. Nothing was breached: the number is stored in the weights and comes out through the normal interface. That is **training-data extraction**. Membership inference in the full edition asks whether a record you already hold was in the training set; extraction hands the attacker the record itself, often one it did not know existed.

The idea that organizes the chapter is the **rank** of a secret under the model. A language model gives every string a probability. Fix the context in which a secret appears, say "my pin is", and list every possible filler in order of the model's probability, most likely first. The secret's rank is its position in that list, and it is also the number of guesses an attacker needs if it tries the most likely fillers first. Memorization is the model putting a training string far above the strings that could have stood in its place. Every measure below reads that rank in its own way.

The examples use one toy: a contact list of 5,000 background lines like `toneka 14524` (a made-up name and a five-digit pin), plus 600 secret lines, 100 each repeated 1, 2, 4, 8, 16 and 32 times. The model is a character n-gram model, which predicts each character from the previous $n-1$ by counting. Its order n plays the role of model size.

13.2. Verbatim, approximate, discoverable, extractable

"The model memorized its training data" can mean several things, and the numbers differ by orders of magnitude between them, so a report must say which it measures. The definitions differ in what the attacker is assumed to know. A string is **extractable** (Carlini et al. 2021) if *some* prompt makes greedy decoding, which appends the most likely next symbol at each step, emit it. That is the real attacker, who has only the model, but measuring it means searching over all prompts. A suffix is **discoverable** (Carlini et al. 2023) if prompting with its own training prefix makes greedy decoding emit it. That assumes an attacker who holds the prefix, which is strong, but it is easy to measure: prompt with each training prefix and compare. Discoverable implies extractable, since the prefix is one prompt among all. A string is **k-eidetic** if it is extractable and appears in at most k training documents; small k is the worrying case, a secret seen once and given back.

The definitions also differ in how close the output must be. The ones above are **verbatim**; a string is **approximately** memorized (Ippolito et al. 2023) if the output is within some distance of it. Verbatim matching misses near copies, and a phone number with one digit changed has still leaked to anyone who reads it. Ippolito et al. show that prompts which change the style slip

approximate copies past a filter that blocks verbatim training text. A verbatim rate is therefore a lower bound on the leak, not a measure of it, and a verbatim filter is not a defence.

Greedy decoding is a convenience for measurement; deployed models sample. A string greedy decoding never emits can still come out. If one sample reproduces it with probability p, then m independent samples all miss it with probability $(1-p)^m$, which is at most e^{-pm}, so

$$m \geq \frac{\ln(1/\beta)}{p} \tag{13.1}$$

samples extract it with probability at least $1-\beta$. The budget is inversely proportional to p: a string with probability 10^{-3} after its prefix comes out with 99% confidence ($\beta = 0.01$) within about 4,600 samples. In the toy, an order-5 model reproduces a secret's pin in one sample with probability 1.4×10^{-3} on average at 4 copies and 0.142 at 32 copies, which means 3,252 and 31 samples for a 99% chance. Duplication made the search trivial.

13.3. Duplication and model size

Which strings get memorized? In the n-gram toy the answer can be read off the counts. Greedy decoding follows the largest count at each step, so memorization is a vote. Each copy of a training line adds one vote for the line's own next symbol in every context the line passes through. The line comes out when, at every step, its votes beat the strongest rival from the rest of the corpus, so the number of copies it needs, k^*, is its largest gap to a rival plus one, set by its most contested step. The fraction of lines memorized at k copies is the fraction with $k^* \leq k$; this matched greedy decoding exactly in all 36 cells of Table 13.1, because it is a restatement of what greedy decoding does with counts.

Model size works by splitting the vote. A longer context is a finer partition of the training positions: fewer positions share a given context of length 7 than of length 2. So rival counts shrink as the order grows, while the line's own k votes stay with it in every context it passes through. In the limit every line has contexts of its own and is memorized from a single copy. That is the toy version of "larger models memorize more": a larger model can afford to give rare strings parameters of their own.

Table 13.1 shows both effects. Along a row, at order 5, the verbatim rate climbs from 0 at one copy to 0.70 at 32. Down a column, a line seen once is safe up to order 5, memorized 8% of the time at order 6, 70% at order 7 and 98% at order 8; those singletons are 1-eidetic.

Table 13.1.: Fraction of 100 secret lines per cell whose pin greedy decoding reproduces from the true prefix, verbatim / with at most one digit wrong. Rows are model orders, columns copies in the training set.

Order n	$k=1$	$k=2$	$k=4$	$k=8$	$k=16$	$k=32$
3	0.00 / 0.00	0.00 / 0.03	0.00 / 0.00	0.00 / 0.00	0.00 / 0.02	0.00 / 0.05
4	0.00 / 0.00	0.00 / 0.00	0.00 / 0.01	0.00 / 0.00	0.00 / 0.00	0.13 / 0.17
5	0.00 / 0.00	0.00 / 0.00	0.05 / 0.06	0.11 / 0.15	0.42 / 0.42	0.70 / 0.70
6	0.08 / 0.13	0.65 / 0.69	0.76 / 0.78	0.80 / 0.85	0.90 / 0.91	0.96 / 0.97
7	0.70 / 0.78	0.95 / 0.96	0.98 / 0.98	0.98 / 0.99	0.99 / 0.99	0.97 / 0.98

Order n	$k = 1$	$k = 2$	$k = 4$	$k = 8$	$k = 16$	$k = 32$
8	0.98 / 0.98	1.00 / 1.00	1.00 / 1.00	0.99 / 0.99	1.00 / 1.00	1.00 / 1.00

For real models, Carlini et al. (2023) found that discoverable memorization grows log-linearly with duplication, model size and prompt length, and that the largest model they measured, GPT-J, reproduces at least 1% of its training data from 50-token prefixes.

The vote picture also says who is exposed first. A line whose name shares little with other names reaches a context of its own sooner, so rare and atypical strings are memorized first; this is the high **self-influence** of atypical examples from Algorithmic stability and generalization, which can only be fitted by memorizing them. Deduplication lowers k for every repeated string, so it is the cheapest defence: Lee et al. (2022) found that models trained on deduplicated data emit memorized text about ten times less often without losing quality. But it cannot touch a singleton in a large model, where $k = 1$ already suffices; deduplicating the order-8 model's data would barely change its 98%. And because larger models memorize from fewer copies, an audit passed at one size must be redone after scaling up.

13.4. Canaries and the exposure metric

Discoverable memorization is all or nothing; an auditor wants a graded answer with a known false alarm rate. Carlini et al. (2019) plant **canaries**, random secrets of a fixed format such as "my pin is" followed by five random digits, in the training data, then read off the trained model's rank of the canary among all values of that format. Because you chose the canary at random, you know the answer and the size of the space $\mathcal{R}$. The **exposure** reads the rank on a log scale,

$$\text{exposure}(\hat{r}) = \log_2 |\mathcal{R}| - \log_2 \text{rank}(\hat{r}). \tag{13.2}$$

Before seeing the model, an attacker guessing the canary faces $|\mathcal{R}|$ equally likely values; guessing in the model's order, it needs rank tries. Exposure is the difference in bits, the number of halvings of the search the model has done for the attacker.

What makes exposure an audit tool is that its behaviour without memorization is known exactly. If the model does not depend on the canary, for instance because it was never inserted, the canary is just a uniformly random value, so its rank is uniform over the list. The chance of landing in the top fraction 2^{-e} of the list is then 2^{-e}:

$$\Pr[\text{exposure} \geq e] \leq 2^{-e}. \tag{13.3}$$

So exposure is $-\log_2$ of a p-value, and the test "inserted if exposure $\geq e$" has false positive rate (FPR) at most 2^{-e}. "Was this canary inserted?" is a membership question, but where membership inference calibrates its test with shadow models, here the canary's own randomness does the calibration. In the toy, uninserted pins show mean exposure 1.443 bits, the $\log_2 e$ the uniform rank predicts, and a fraction $1/32 = 2^{-5}$ with exposure at least 5.

For "my pin is 58745", $|\mathcal{R}| = 10^5$ and the maximum exposure is 16.61 bits. Table 13.2 gives the exact exposure, from scoring all 10^5 pins. The order-4 model reaches the maximum at 16 copies. The order-2 model barely moves even at 64, because its one-character contexts are shared with every line, the vote-splitting argument run backwards.

Table 13.2.: Exposure in bits of the canary “my pin is 58745” against the number of copies k in the training set. The maximum is $\log_2 10^5 = 16.61$.

Order n	$k=0$	$k=1$	$k=2$	$k=4$	$k=8$	$k=16$	$k=32$	$k=64$
2	0.40	0.41	0.43	0.47	0.55	0.73	1.21	2.54
3	0.89	4.05	4.44	4.71	5.24	6.59	9.69	15.61
4	3.76	9.98	12.15	15.02	15.61	16.61	16.61	16.61

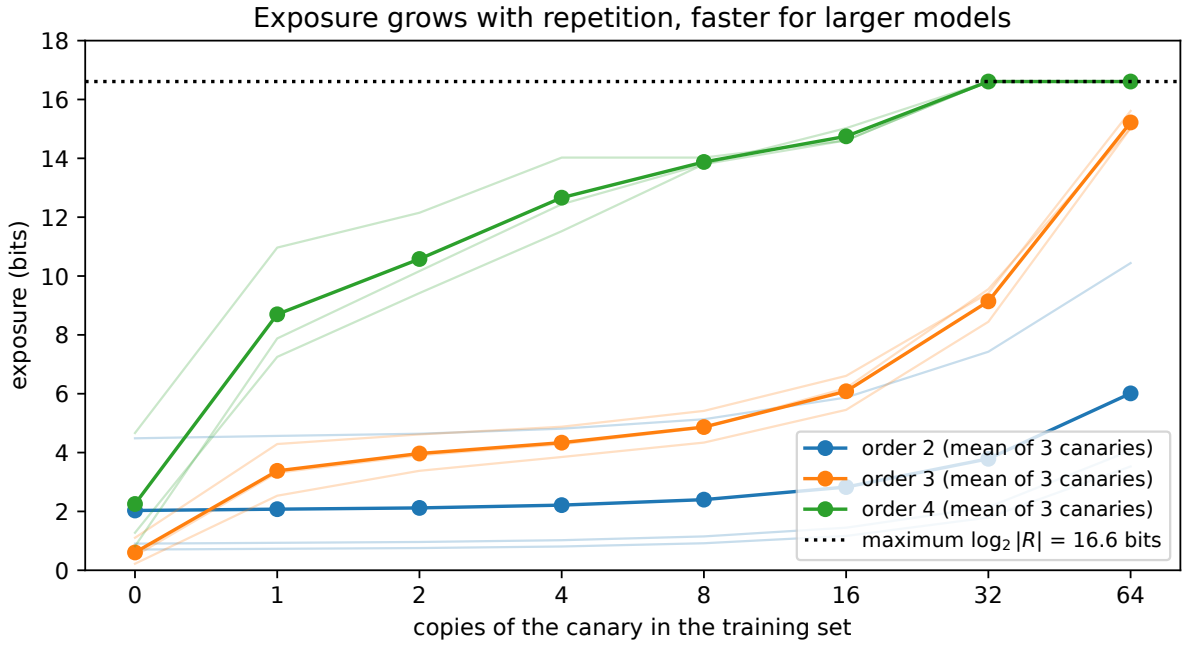


Figure 13.1.: Exposure against copies of the canary for n-gram models of order 2, 3 and 4; thin lines are individual canaries and markers their mean, and the dotted line is the maximum. Larger models reach it with fewer copies.

For a large space the rank must be estimated. Counting how many of m random candidates score above the canary is reliable, but a canary above all of them only says its rank fraction is below about $1/m$, so it cannot resolve exposures above $\log_2 m$. Fitting a distribution to the scores reaches further, but only as far as its tail is right, and the tail is the part you cannot sample. At order 3 with 8 copies, counting 10,000 random pins gives 5.35 bits against an exact 5.24, while a normal fit gives 8.28, because the scores have a heavier right tail than a normal. Ties are a further trap: a model that has never seen a context gives every candidate the same score, so every rank is the last and every exposure 0, which hides rather than measures.

Because you control what was planted, use canaries to compare training runs that differ in one thing: with and without deduplication, before and after an unlearning method, or with private training (Part 3). Carlini et al. (2019) found that differentially private training kept their canaries at the exposure of canaries never inserted.

13.5. Extraction without the prefix: generate, rank, verify

A real attacker has only the model. Carlini et al. (2021) extracted training data from GPT-2 in three steps: generate many samples, rank them by how likely they are to be memorized, and verify the top of the ranking against a search or public sources. The ranking is the hard step, and it is a hypothesis test. A model can find a string likely because text like it is common, or because it memorized it, and only the second is a leak. Ranking by the target's likelihood alone surfaces the common, such as licences and boilerplate, which reveal nothing. A **reference model**, trained on similar data that excludes the target's training set, stands in for "common text", and the right score is how much more likely the target finds x than the reference does:

$$\Lambda(x) = \ln p_\theta(x) - \ln p_{\text{ref}}(x). \tag{13.4}$$

Text both models know cancels. By the Neyman–Pearson lemma of Hypothesis testing, this likelihood ratio is the most powerful ranking at every FPR; it is the per-example calibration of membership inference again. The reference can be crude: a smaller model of the same family, the zlib-compressed length of the text, or the target's own likelihood on the lowercased text.

In the toy, the attacker draws 20,000 lines from the order-6 model, whose training set also contains 50 public lines repeated 100 times each. Among the 14,852 distinct samples are 197 private training lines. Of the top 100 by likelihood only 45 are private, since the public lines are the most likely strings the model knows. Ranked by Equation 13.4, with a reference trained on an independent contact list that also contains the public lines, 85 of the top 100 are private (Figure 13.2).

Without the prefix the attacker gets much less than is discoverable: none of the secrets inserted 1, 2 or 4 times appear in the samples, though 76% of the 4-copy secrets are discoverable at order 6. A sample must first produce the right name before it can produce the pin, and only heavily duplicated lines make their names likely enough.

"Less than discoverable" can still be a great deal. Nasr et al. (2023) attacked an aligned chat model, which rarely emits training data in normal use because alignment taught it to answer as an assistant, not to continue documents. Asked to repeat a single word forever, it diverged into text that looked like pretraining documents; about 200 US dollars of queries extracted over 10,000 unique memorized examples, including personal contact details. Alignment changed how the model usually responds, not what it had memorized, so it is not a defence, and a red team should test prompts that escape the chat behaviour. Verify outputs against the training set, or against a large public corpus as a lower bound. Report precision at the top of the ranking, which is the low-FPR operating point an attacker checking candidates by hand cares about, not recall over all memorized strings.

13.6. Code

Exposure and sample budgets are what you compute in an audit, and they need only logarithms:

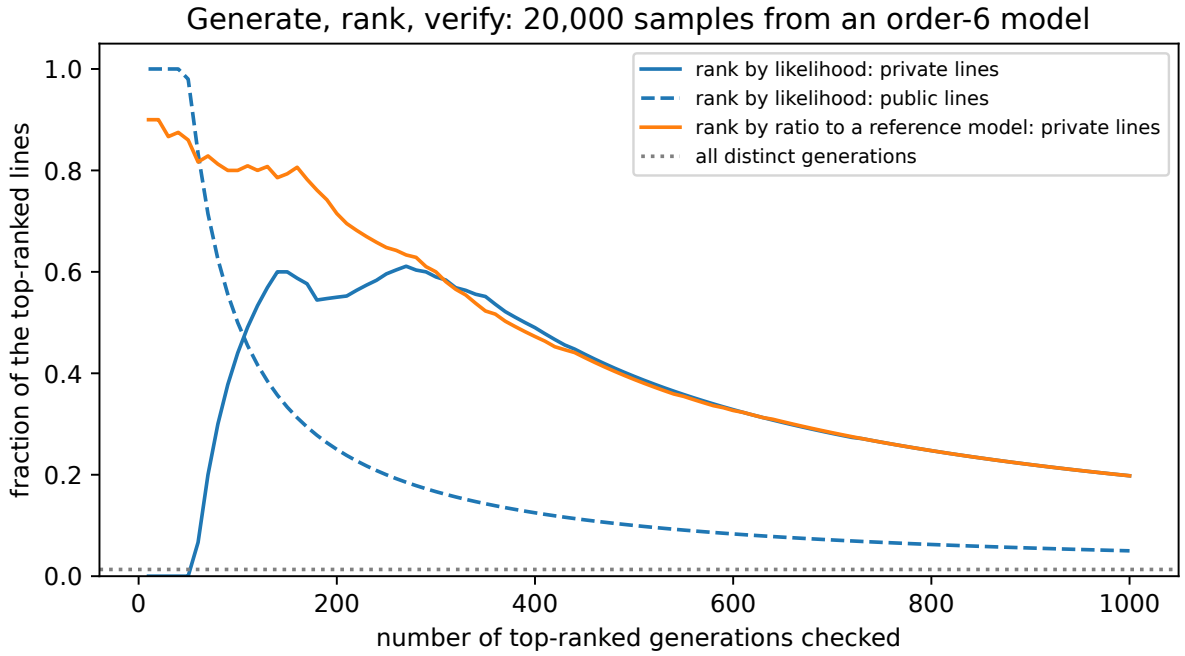


Figure 13.2.: Fraction of the top-ranked distinct generations that are private training lines (solid) or public lines (dashed), from 20,000 samples of an order-6 model. Ranking by likelihood puts the public lines first; ranking by the likelihood ratio to a reference model puts the private lines first.

```python
from math import ceil, log, log2


def exposure(rank: int, space: int) -> float:
    """Bits of guessing the model saves: log2 |R| - log2 rank."""
    return log2(space) - log2(rank)


def null_p_value(exposure_bits: float) -> float:
    """Upper bound on the chance that a canary never inserted scores this high."""
    return 2.0**-exposure_bits


def samples_needed(p: float, beta: float) -> int:
    """Smallest m with (1 - p)^m <= beta: samples for a 1 - beta chance of drawing a string
    return ceil(log(beta) / log(1 - p))


def samples_bound(p: float, beta: float) -> int:
    """The simpler sufficient budget ln(1 / beta) / p."""
    return ceil(log(1 / beta) / p)


print(f"{exposure(1, 10**5):.2f}")  # 16.61
print(f"{exposure(6018, 10**5):.2f}")  # 4.05
print(f"{null_p_value(10):.5f}")  # 0.00098
print(samples_needed(0.142, 0.01))  # 31
print(samples_bound(1e-3, 0.01))  # 4606
```

Rank 6,018 of 10^5 is the order-3 canary at one copy. An exposure of 10 bits has a p-value below 0.001, whatever the size of the space. The full edition's script runs every experiment in this chapter:

```
uv run python code/part02-attacks/memorization.py
```

13.7. Cheat sheet

Table 13.3.: The results of this chapter.

Result	Formula	Notes
Discoverable	Greedy decoding from the true prefix emits the suffix	Easy to measure; implies extractable
Extractable, k-eidetic	Some prompt emits it; in at most k documents	The real attacker's notion

Result	Formula	Notes
Approximate	Output within distance τ	Verbatim rates understate the leak
Samples needed	$m \geq \ln(1/\beta)/p$	p the probability of the string after the prompt
Exposure	$\log_2 \lvert\mathcal{R}\rvert - \log_2 \text{rank}$	Bits of guessing saved
Exposure null	$\Pr[\text{exposure} \geq e] \leq 2^{-e}$; mean $\to 1.443$ bits	$-\log_2$ of a p-value
Ranking generations	$\ln p_\theta(x) - \ln p_{\text{ref}}(x)$	Cancels text that is likely under any model

Rules of thumb

- Memorization is a rank: the secret above the strings it could have been.
- Duplication and model size drive memorization. Deduplicate, but expect singletons in large models to be memorized anyway.
- Audit with planted canaries and report exposure, which is graded and has a known null.
- Rank generations against a reference model, never by raw likelihood.

13.8. Further reading

- Carlini, Liu, Erlingsson, Kos & Song, "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries, exposure and how to estimate it.
- Carlini et al., "Extracting Training Data from Large Language Models" (USENIX Security 2021): the generate-rank-verify attack and its reference-based rankings.
- Carlini, Ippolito, Jagielski, Lee, Tramèr & Zhang, "Quantifying Memorization Across Neural Language Models" (ICLR 2023): how memorization scales with duplication, model size and prompt length.
- Nasr et al., "Scalable Extraction of Training Data from (Production) Language Models" (2023): discoverable against extractable memorization, and the divergence attack on an aligned chat model.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

14. Gradient inversion

14.1. Why this matters

A phone keyboard learns from what its users type, and the raw text never leaves the phones. The server sends the current model to each phone, the phone computes the gradient of the loss on its own few examples, and only that gradient goes back. This is **federated learning (FL)**, often described as private because "the data stay on the device". If you design, review or red-team such a system, the question is: **what does one client's gradient reveal about the client's data?**

Often, everything, because many entries of a gradient are simple functions of the input. Take a fully connected layer, which computes $a = Wx + b$. The weight W_{jk} multiplies the input x_k, so nudging W_{jk} changes a_j by x_k times the nudge, and the loss responds to a_j at some rate $\delta_j = \partial \ell / \partial a_j$ set by everything above the layer. The chain rule multiplies the two rates, and the bias, which is added to a_j directly, gets δ_j alone:

$$\nabla_W \ell = \delta \, x^\top, \qquad \nabla_b \ell = \delta. \tag{14.1}$$

The weight gradient is an **outer product**: row j is the input x scaled by the single number δ_j, so the layer's input sits in every row. The rest of the network, the loss and the label only set the scales. With one example the scale can be divided out; with a batch the gradient is a sum of one outer product per example, and the attacker's job is to pull the sum apart.

The attacker here is an **honest-but-curious** server in the sense of Threat models and privacy games in the full edition: it follows the protocol, sends the real model and only analyses what it receives. It knows the architecture and every parameter, because it chose them, and it sees each client's update $g = \frac{1}{B} \sum_i \nabla_\theta \ell(\theta; x_i, y_i)$, the average gradient over the client's B examples. Reconstructions are scored by the **relative error** $\|\hat{x} - x\| / \|x\|$ against the closest true example. The experiments use 8×8 images of four shapes and a network with one hidden layer of 32 units at its random initialization: 2,212 gradient entries against 64 unknown pixels per example.

14.2. Exact recovery from a fully connected layer

For a single example, Equation 14.1 already gives the attack (Phong et al. 2018). Row j of the first layer's weight gradient is $\delta_{1,j} x^\top$, and entry j of its bias gradient is the same number $\delta_{1,j}$, so dividing one by the other cancels the only unknown scale. Every unit j with a nonzero bias gradient gives

$$x = \frac{(\nabla_{W_1} \ell)_{j,:}}{(\nabla_{b_1} \ell)_j}. \tag{14.2}$$

No search, no prior on the data and no label are needed, and the attack checks itself: all usable units must give the same answer, so if their ratios agree, the attacker knows it has the input.

Take a two-unit ReLU network with $W_1 = \begin{pmatrix} 0.4 & -0.2 & 0.1 \\ 0.3 & 0.5 & -0.4 \end{pmatrix}$, $b_1 = (0.1, -0.1)$, $W_2 = \begin{pmatrix} 1 & -1 \\ 0.5 & 0.5 \end{pmatrix}$ and no output bias, and an input $x = (0.5, 1, 0.2)$ with the first class as label. The backward signal at the first layer is $\delta_1 = (-0.3279, 0.9838)$, so the two rows of the weight gradient are $(-0.1640, -0.3279, -0.0656)$ and $(0.4919, 0.9838, 0.1968)$, visibly the same vector at two scales. Dividing each by its bias entry gives $(0.5, 1, 0.2)$ twice; Section 14.6 runs it. On an 8×8 image and the 32-unit network, 13 units are switched on, and each of the 13 ratios equals the image to a relative error of at most 6.7×10^{-17}, rounding error (Figure 14.1).

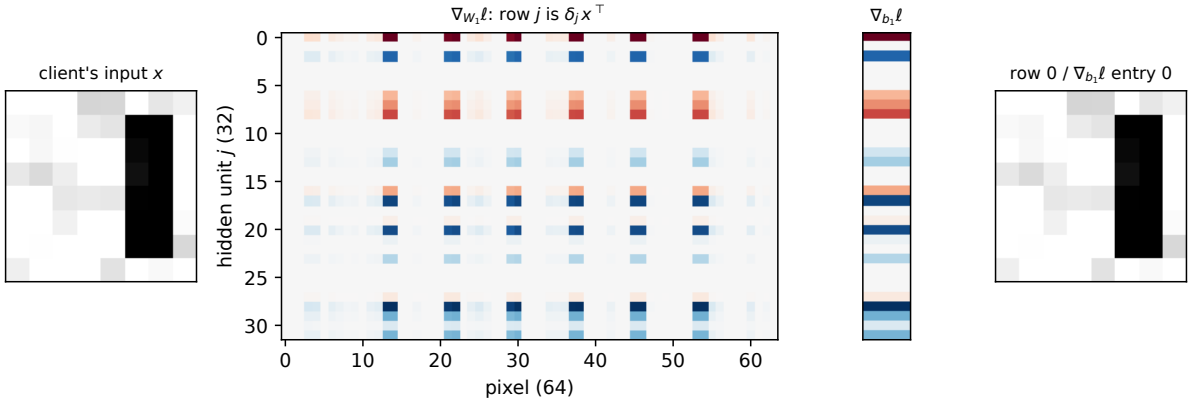


Figure 14.1.: Exact recovery from the first-layer gradient of a ReLU network with 32 hidden units. Left: the client's input. Middle: the weight gradient, one row per hidden unit, next to the bias gradient; every nonzero row is the image times its bias entry, and switched-off units give zero rows. Right: one row divided by its bias entry, which is the input exactly.

Because the division cancels δ_1, whatever produced δ_1 is irrelevant: twenty layers above the first, or a custom loss, change nothing. A ReLU unit switched off on x has $\delta_{1,j} = 0$ and gives nothing, but one active unit is enough. Without a bias there is no denominator, yet every row still points along x, so x is known up to one number that a one-dimensional search finds. The same division at a deeper fully connected layer yields that layer's input, an intermediate feature vector. What really breaks the closed form is a convolutional first layer: a convolution applies one small filter at every position of the image, so its gradient is a sum of outer products over all the image patches, and no single division separates them. A batch of more than one example is a sum too, and breaks it only partly (Section 14.5).

14.3. Label recovery from the last layer

The label is often the most sensitive part of an example (the diagnosis, the next word typed), and the last layer gives it away by a sign (Zhao, Mopuri & Bilen 2020). With softmax and cross-entropy, the backward signal at the logits is the predicted probabilities minus the one-hot label, $p - e_y$. Softmax probabilities lie strictly between 0 and 1, so every entry is positive except

the one at the true label, $p_y - 1$. By Equation 14.1 the last-layer bias gradient is exactly this signal, so for one example the label is the position of its only negative entry:

$$y = \arg\min_k (\nabla_{b_2} \ell)_k. \tag{14.3}$$

Without a bias the weight gradient tells the same story: its row k is the signal's entry k times the hidden activations, and if those are never negative (ReLU or sigmoid) the multiplication keeps the sign, so the label is the only row with a negative sum. Both rules were right in 1,000 of 1,000 trials. Nothing in the argument depends on how good the model is, so they are exact at any point of training; only a probability that rounds to 1 in floating point hides the sign.

For a batch of B, the bias gradient is the average of $p_i - e_{y_i}$. The one-hot parts add up to the label histogram, and the probability parts to the model's total predicted probability for each class, so the number n_k of examples of class k is

$$n_k = \sum_{i=1}^{B} p_{ik} - B\,(\nabla_{b_2} \ell)_k. \tag{14.4}$$

The equation is exact; the only unknown is the total prediction $\sum_i p_{ik}$. The attacker knows the model, so it estimates that total as B times the model's average prediction on data of its own, and rounds. Since true counts are integers, rounding is exact whenever the estimate is off by less than one half for every class.

A batch of 16 has true counts $(4, 2, 4, 6)$. With the average prediction taken over 200 of the attacker's own images, Equation 14.4 gives $(3.88, 2.00, 4.19, 5.92)$, which rounds to the truth. Assuming the model predicts every class equally gives $(2.12, 3.01, 4.27, 6.60)$, which does not, because even an untrained network is not uniform. Over 200 random batches, the averaged estimate gets all counts exact for 1.000 of batches of 4, 0.975 of batches of 16 and 0.525 of batches of 64 (Figure 14.2): the error of the estimate grows with B, and once it passes one half, rounding starts to miss, though the counts stay close.

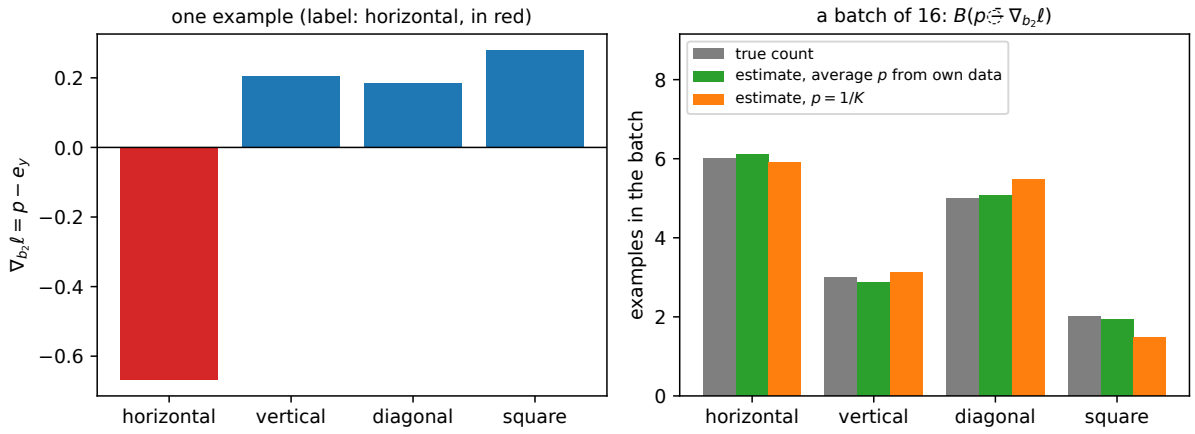


Figure 14.2.: Labels from the last-layer bias gradient. Left: for one example it has a single negative entry, at the true label. Right: for a batch of 16, the counts estimated with the model's average prediction on the attacker's own images are close to the true counts; with a uniform guess they are off by up to about half an example.

So a batch leaks its label histogram even when no input is recovered. And knowing the label helps the next attack: it removes a discrete unknown, so the search of Section 14.4 runs over the input only.

14.4. Deep Leakage from Gradients

Convolutions, normalization layers or missing biases leave no closed form. Deep Leakage from Gradients (DLG) (Zhu, Liu & Han 2019) needs none: guess an input, compute the gradient it would produce, and move the guess until that gradient matches the one observed. If only the true input produces the observed gradient, a matching guess is the input. With the labels fixed as above, DLG minimizes the **matching loss**, the squared distance between the gradient of the guessed batch and g:

$$\hat{x} = \arg\min_{x'} R(x'), \qquad R(x') = \Big\| \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell(\theta; x'_i, y_i) - g \Big\|^2. \tag{14.5}$$

It starts from random noise and runs a gradient method on R, which automatic differentiation provides by differentiating through the backward pass.

Two consequences follow from that construction. Since R is built from gradients, its own gradient is made of second derivatives of the network. ReLU's slope is constant on each side of zero, so its second derivative is zero almost everywhere and the search gets no signal through it; that is why Zhu et al. replaced ReLU by the sigmoid, as the experiments here do. And since the true input has $R = 0$, the smallest possible value, the final R reports the outcome: at rounding error the match is exact, while a loss that stopped falling well above zero means the search is stuck, because R is not convex.

On one image, the sigmoid network with 32 hidden units, and 600 steps of the Adam optimizer from uniform noise, the relative error is 7.5×10^{-2} at step 50, already visually correct, and 5.8×10^{-14} at step 600, with matching loss 1.3×10^{-27} (Figure 14.3).

A failed run is therefore an optimization failure, not evidence of privacy, and the attacker sees it and retries with restarts, better optimizers or image priors. Geiping et al. (2020), comparing gradients by cosine distance and penalizing differences between neighbouring pixels, recover recognizable images from trained networks and from batches.

14.5. Batches and averaging

Clients usually send a batch average, and averaging is often said to protect individuals: the gradient describes the batch, not anyone in it. Averaging does break Equation 14.2 and makes DLG harder, but the outer-product structure survives it.

The first-layer gradient is now a sum of B outer products, one per example. Every row is a combination of the B inputs, and the matrix has rank at most B, so the server learns the subspace that contains the batch, which for B much smaller than the input dimension already rules out almost every input. A unit's ratio from Equation 14.2 is now a weighted average of the inputs, each weighted by its share of the unit's backward signal, and it equals one input exactly when the

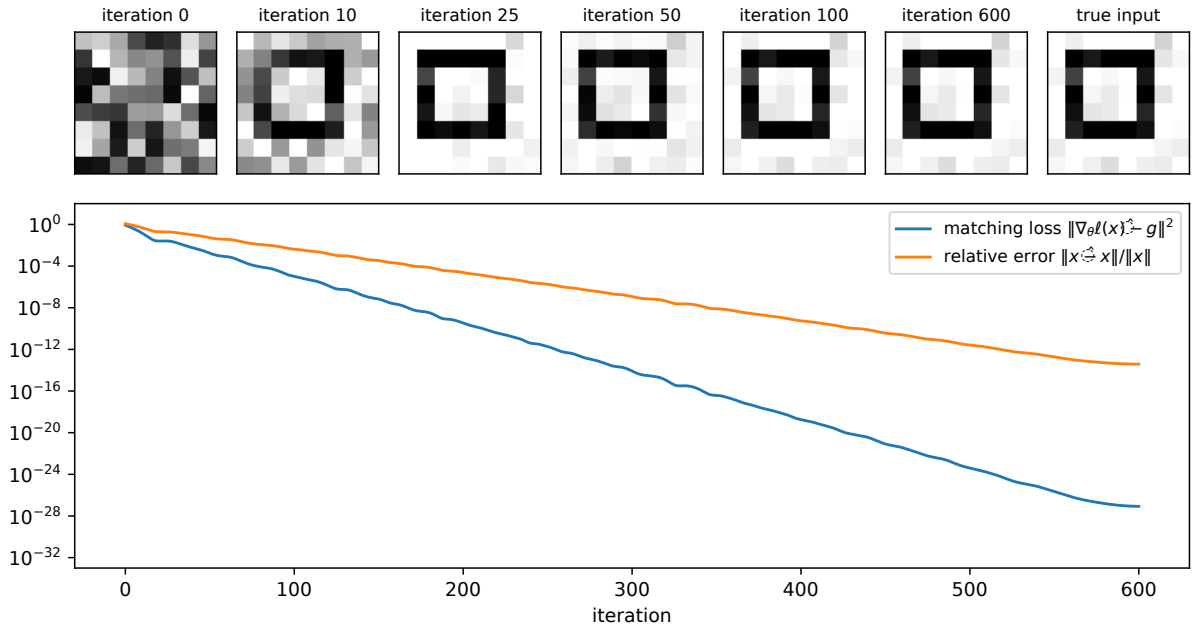


Figure 14.3.: Deep Leakage from Gradients on one image, sigmoid network with 32 hidden units. Top: the reconstruction at selected iterations, starting from noise, and the true input. Bottom: the matching loss and the relative error on a log scale; both fall until they reach rounding error.

unit is active for that example only, when it **isolates** it. If each of m ReLU units fires on each example like an independent fair coin, a unit isolates a given example with probability 2^{-B}, and the example escapes only if all m units miss it:

$$P_{\text{iso}}(B, m) = 1 - (1 - 2^{-B})^m. \tag{14.6}$$

Each extra example halves the chance per unit, and each extra unit adds a chance. An even chance at $B = 10$ needs 710 units, and real first layers have thousands.

A batch of 4 through the 32-unit ReLU network gives a rank-4 gradient, and the unit ratios recover all 4 images exactly, where the formula predicts 0.873 per example. Table 14.1 tests the formula more widely. For Gaussian inputs, whose units really do fire like coins, it matches. For images it is far too optimistic about privacy at large batches: at $B = 16$ in the 1024-unit network it predicts 1.6% exposed, and 69.1% is. Images are correlated, so a unit tends to fire on many of them or on few, and units that fire on exactly one are much more common than coin flips allow (Figure 14.4).

Table 14.1.: Fraction of batch examples recovered exactly from first-layer unit ratios of a random ReLU network, against Equation 14.6.

m	B	Equation 14.6	Gaussian inputs	Images
64	4	0.984	0.975	0.838
64	8	0.222	0.188	0.550

m	B	Equation 14.6	Gaussian inputs	Images
64	16	0.001	0.000	0.144
256	8	0.633	0.562	0.769
256	12	0.061	0.046	0.542
256	16	0.004	0.006	0.375
1024	12	0.221	0.196	0.850
1024	16	0.016	0.009	0.691

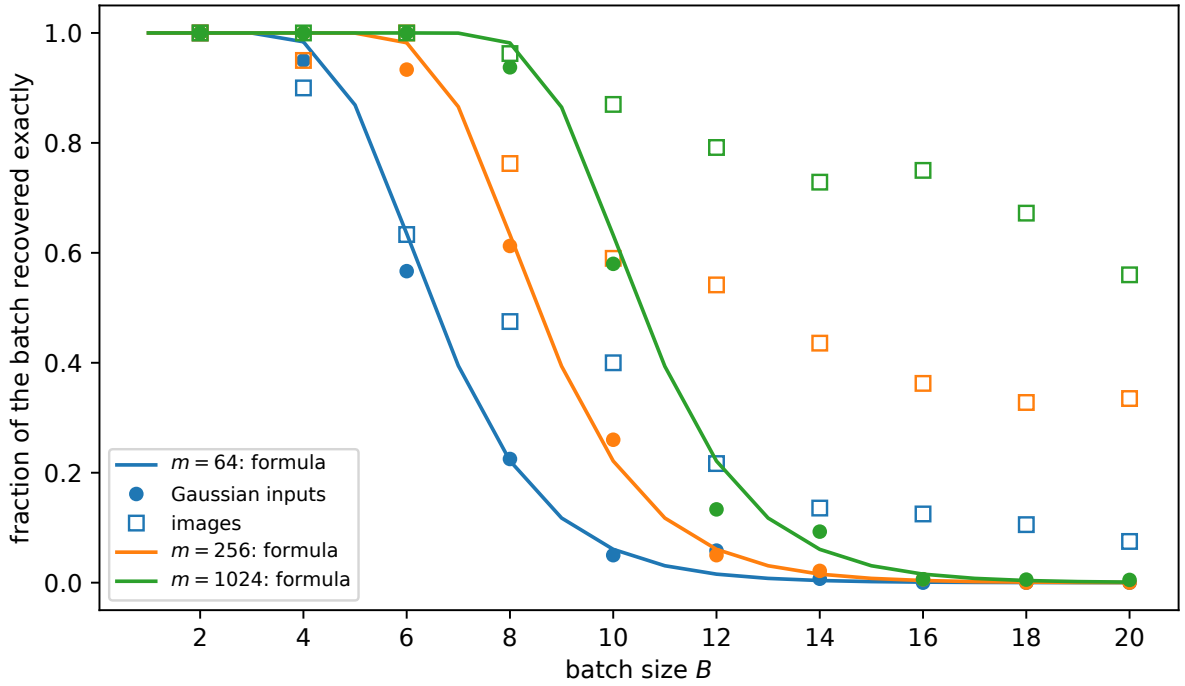


Figure 14.4.: Fraction of a batch recovered exactly by first-layer unit ratios, for ReLU networks with 64, 256 and 1024 hidden units. Lines: the independence formula. Filled circles: Gaussian inputs, which follow it. Open squares: the images, which are correlated, so isolation decays much more slowly.

DLG has to untangle B inputs from one sum. On the sigmoid network it recovers all single images, 0.67 of examples in batches of 2, 0.29 at $B = 4$ and none at $B = 8$, where the matching loss stalls near 10^{-5} and the reconstructions are blends. That is a stuck search, not missing information: 2,212 equations remain.

So batching is not a defence. The label histogram, the span and every isolated example come out exactly. Sending the parameter change after several local steps does not help either: every step's first-layer gradient is built from outer products with the same inputs, so for one example the change is still a multiple of $x^\top$ in every row, and the division still recovers x. And all of this is what an honest server can do with random weights. A malicious server chooses the weights, and can plant units that fire on rare inputs only (Boenisch et al. 2023) or that sort inputs into bins by a feature (Fowl et al. 2022), so that most of a large batch is isolated by design. Defences have to change what the server sees: differential privacy (Part 3) bounds what any update reveals, and

secure aggregation, which reveals only the sum over many clients, is introduced in Local, shuffle and distributed trust models.

14.6. Code

The hand example of Section 14.2, with the label rule and the isolation formula, in plain Python. The script indexes classes from 0, so label 0 is the first class.

```
from math import ceil, exp, log

Vector = list[float]
Matrix = list[Vector]


def client_gradient(w1: Matrix, b1: Vector, w2: Matrix, x: Vector, y: int) -> tuple[Matrix,
    """What a client sends for one example: the gradients of W1, b1 and b2 (one ReLU layer,
    a = [sum(wk * xk for wk, xk in zip(row, x)) + bj for row, bj in zip(w1, b1)]
    h = [max(aj, 0.0) for aj in a]
    z = [sum(wk * hk for wk, hk in zip(row, h)) for row in w2]
    total = sum(exp(zk) for zk in z)
    delta2 = [exp(zk) / total - (1.0 if k == y else 0.0) for k, zk in enumerate(z)]  # p - e
    delta1 = [sum(delta2[k] * w2[k][j] for k in range(len(w2))) * (1.0 if a[j] > 0 else 0.0)
    grad_w1 = [[dj * xk for xk in x] for dj in delta1]  # outer product delta1 x^T
    return grad_w1, delta1, delta2  # the bias gradients equal the backward signals


def recover_input(grad_w1: Matrix, grad_b1: Vector) -> Vector:
    """Exact recovery: a row of the weight gradient divided by its bias entry."""
    j = next(j for j, g in enumerate(grad_b1) if g != 0.0)
    return [g / grad_b1[j] for g in grad_w1[j]]


def recover_label(grad_b2: Vector) -> int:
    """The only negative entry of the last-layer bias gradient is the label."""
    return min(range(len(grad_b2)), key=lambda k: grad_b2[k])


def isolation(batch: int, units: int) -> float:
    """Chance that some ReLU unit fires for one given example of the batch only, under the c
    return 1 - (1 - 2.0**-batch) ** units


def width_for_even_odds(batch: int) -> int:
    """Smallest number of units that isolates each example with probability at least 1/2."""
    return ceil(log(2) / -log(1 - 2.0**-batch))
```

```
w1 = [[0.4, -0.2, 0.1], [0.3, 0.5, -0.4]]
b1 = [0.1, -0.1]
w2 = [[1.0, -1.0], [0.5, 0.5]]
gw1, gb1, gb2 = client_gradient(w1, b1, w2, x=[0.5, 1.0, 0.2], y=0)
print([round(v, 4) for v in recover_input(gw1, gb1)])  # [0.5, 1.0, 0.2]
print([round(v, 4) for v in gb2], recover_label(gb2))  # [-0.6559, 0.6559] 0
print(f"{isolation(4, 32):.3f}")  # 0.873
print(width_for_even_odds(10))  # 710
```

The full edition's script runs every attack in this chapter on the image network, including DLG with a hand-written double backpropagation, in about half a minute:

```
uv run python code/part02-attacks/gradient_inversion.py
```

14.7. Cheat sheet

Table 14.2.: The results of this chapter.

Result	Formula	Needs	Fails when
Outer product	$\nabla_W \ell = \delta x^\top$, $\nabla_b \ell = \delta$	A fully connected layer	—
Exact recovery, one example	$x = (\nabla_{W_1}\ell)_{j,:} / (\nabla_{b_1}\ell)_j$	First layer fully connected with bias	Convolutional first layer; batches (partly)
Label, one example	$y = \arg\min_k (\nabla_{b_2}\ell)_k$	Softmax and cross-entropy	p_y rounds to 1
Label counts, batch	$n_k = \sum_i p_{ik} - B(\nabla_{b_2}\ell)_k$	Last-layer bias; a guess of $\sum_i p_{ik}$	Large B with a poor guess
DLG	$\min_{x'} \lVert G(x') - g \rVert^2$, G the gradient of the guessed batch	Twice-differentiable network; labels	Search stuck (final $R \gg 0$), larger batches
Span of a batch	Rank $\leq B$, rows in span$\{x_i\}$	Fully connected first layer	—
Isolation	$1 - (1 - 2^{-B})^m$ under independence	ReLU units, random weights	Real data: much slower decay

Rules of thumb

- One example through a fully connected layer with a bias: the input costs one division, the label one sign.
- Batching helps against search but not against structure. Label counts, the span and every isolated example leak, and with real data that is most of a batch of 10 or more on a wide layer.

- An honest server can already do all of this; a malicious one can do it for large batches by design. Protect updates with differential privacy and secure aggregation, not with batching or local steps.

14.8. Further reading

- Zhu, Liu & Han, "Deep Leakage from Gradients" (NeurIPS 2019): the gradient-matching attack on images and text.
- Geiping, Bauermeister, Dröge & Moeller, "Inverting Gradients — How easy is it to break privacy in federated learning?" (NeurIPS 2020): cosine matching with an image prior, and attacks on trained networks, local steps and batches.
- Huang, Gupta, Song, Li & Arora, "Evaluating Gradient Inversion Attacks and Defenses in Federated Learning" (NeurIPS 2021): how attack strength depends on assumptions such as known labels and untrained models; read it before trusting a published attack number.
- Boenisch, Dziedzic, Schuster, Shamsabadi, Shumailov & Papernot, "When the Curious Abandon Honesty: Federated Learning Is Not Private" (IEEE EuroS&P 2023): trap weights and exact recovery from large batches by a malicious server.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

15. Linkage and re-identification

15.1. Why this matters

A hospital shares its discharge records with researchers after deleting names and addresses, keeping postcode, date of birth, sex and diagnosis. But most people are the only person with their postcode, birth date and sex, so anyone holding a table with names and those three columns, such as a voter roll, can join the two and put a name back on most rows. This is a **linkage attack**, and naming a "de-identified" record is **re-identification**. It needs only the release and some auxiliary data, so run it before you sign off on sharing a dataset.

What makes it work is that identifying information is measured in bits, and bits add up. Picking one person out of n takes $\log_2 n$ bits, because each bit halves the candidates: 27.9 bits for the 248.7 million people in the United States in 1990, 32.9 for 8 billion. An attribute with v equally likely values carries at most $\log_2 v$ bits, and for independent attributes the bits add, because probabilities multiply and the logarithm turns the product into a sum. Once the bits an adversary knows exceed $\log_2 n$ by a few, almost everyone is unique.

15.2. Quasi-identifiers and the linkage attack

Direct identifiers, such as a name, name a person on their own. **Sensitive attributes**, such as a diagnosis, are what the release is about. **Quasi-identifiers** sit in between: they name nobody alone, but in combination they single people out, and they also appear in other datasets next to names. ZIP code, birth date and sex are the classic example. Deleting the first kind and keeping the third is the most common form of "de-identification", and linkage is aimed at exactly that gap.

Group the records of a release into **equivalence classes**, sets of records with identical quasi-identifiers. An adversary who knows the target is in the release, and knows the target's quasi-identifiers, finds the target's class and picks a record from it: right for sure if the class has one record, one time in k if it has k. Each class therefore contributes exactly one record's worth of wins however large it is, and the best possible success is

$$\Pr[\text{win}] = \frac{J}{n}, \tag{15.1}$$

the number J of distinct quasi-identifier combinations divided by the number n of records.

Table 15.1.: A de-identified hospital release: no names, three quasi-identifiers, one sensitive attribute.

ZIP code	Birth date	Sex	Diagnosis
02138	1961-03-14	F	asthma
02138	1972-11-02	M	diabetes
02138	1972-11-02	M	hypertension
02139	1958-06-21	F	depression
02139	1990-01-01	F	migraine
02139	1990-01-01	F	asthma
02139	1990-01-01	F	fracture
02141	1983-09-30	M	pneumonia

Table 15.2.: Part of a voter roll: names with the same three quasi-identifiers.

Name	ZIP code	Birth date	Sex
Ana Silva	02138	1961-03-14	F
Ben Cole	02138	1972-11-02	M
Carla Diaz	02139	1958-06-21	F
Dan Eto	02141	1983-09-30	M

Join Table 15.1 with Table 15.2. Ana, Carla and Dan are each alone in their class, so their diagnoses come out with certainty; Ben shares his class with one other record, so the adversary is right about him half the time. The classes have sizes 1, 1, 1, 2 and 3, so $J = 5$ for $n = 8$ and the success is $5/8 = 0.625$. Sweeney re-identified the governor of Massachusetts this way in the late 1990s: on the Cambridge voter roll, six people shared his birth date, three were men, and one lived in his ZIP code.

Every unique record is a certain win, so the fraction of unique records is a floor on the success, and predicting uniqueness is the core of a risk assessment. Conversely, if every class has at least k records then $J \le n/k$ and the success is at most $1/k$. That is the promise of k-anonymity, and it holds against this adversary only: had all three women born on 1990-01-01 had asthma, the adversary would learn it without picking a record, the gap studied in Syntactic anonymity and how it breaks in the full edition. And which columns are quasi-identifiers is not a property of the table but a guess about what auxiliary data exist; in sparse data, every column is one (Section 15.4).

15.3. How many people are unique?

To predict uniqueness for a release not yet made, use a model of the population. Call a person's combination of attribute values their **cell**, and suppose each person falls in cell c with probability p_c, independently. A person in cell c is unique when all $n-1$ others land elsewhere, which happens

with probability $(1-p_c)^{n-1}$; averaging over the person's own cell gives the expected fraction unique,

$$\frac{\mathbb{E}U}{n} = \sum_c p_c(1-p_c)^{n-1}. \tag{15.2}$$

With M equally likely cells this becomes the bits picture. Each person stays alone with probability about $e^{-n/M}$, where n/M is the average occupancy of a cell. With $b = \log_2 M$ bits the occupancy is $2^{\log_2 n - b}$, so the fraction unique is about $\exp(-2^{\log_2 n - b})$: it depends only on the bits beyond the $\log_2 n$ needed to name one person, the same curve for a town and for the planet (Figure 15.1, right). Every extra bit halves the occupancy, so the curve is steep: half the people are unique at 0.53 bits beyond $\log_2 n$, 99% at 6.64 bits beyond.

So "33 bits identify anyone" is misleading: 33 bits for 8 billion people put about one person in the average cell, where only a fraction 0.394 is unique, because random labels collide often. 99% uniqueness takes 39.5 bits.

Sweeney's attributes show the accounting. About 40,000 ZIP codes are worth 15.3 bits, a birth date over 80 years 14.8, and sex 1.0: at most 31.1 bits against the 27.9 that single out one American, so about 0.106 people per cell and a uniqueness of $e^{-0.106} = 0.899$ if cells were equally likely. From 1990 census tables she estimated 87%. The gap is skew: people crowd into popular cells, which hold nobody unique, while rare cells stay empty. Correlation pulls the same way, because correlated attributes (ZIP code and income) say partly the same thing, so their joint bits are less than the sum. A bits sum is a quick screen, not an estimate. Table 15.3 reveals the attributes of a synthetic census of 10,000 people one at a time ($\log_2 n = 13.29$ bits).

Table 15.3.: Fraction of 10,000 people who are unique as attributes are revealed: the exact expectation, the collision floor, and the uniform-cells value for the same number of cells. H is the entropy of the attributes known so far.

Attributes known	Bits H	Exact	Collision bound	Uniform cells
sex, age band, region	9.31	0.0158	0.0000	0.0004
+ birth month	12.90	0.3328	0.0834	0.5215
+ marital status	14.60	0.5789	0.4237	0.8498
+ children	16.48	0.7939	0.7443	0.9732
+ occupation	20.39	0.9747	0.9736	0.9986
+ blood group	22.57	0.9925	0.9924	0.9998

With 12.90 bits, just under $\log_2 n$, a third of people are unique; about 7 bits beyond it, 97.5% are. The uniform model overstates uniqueness, by up to 0.27, because the attributes are skewed: a few large regions hold many people, who are not unique. A floor that never overstates comes from the **collision probability** $\sum_c p_c^2$, the chance that two random people share a cell: $\mathbb{E}U/n \geq (1-\sum_c p_c^2)^{n-1}$. For independent attributes it is a product over columns, since two people share a cell exactly when they agree on every attribute, so the floor needs only each column's frequencies. It is loose when a few crowded cells hold most people and tight once most cells are nearly empty. For a real audit, compute Equation 15.2 from a model of the joint distribution, such as the Gaussian copula of Rocher, Hendrickx and de Montjoye (2019).

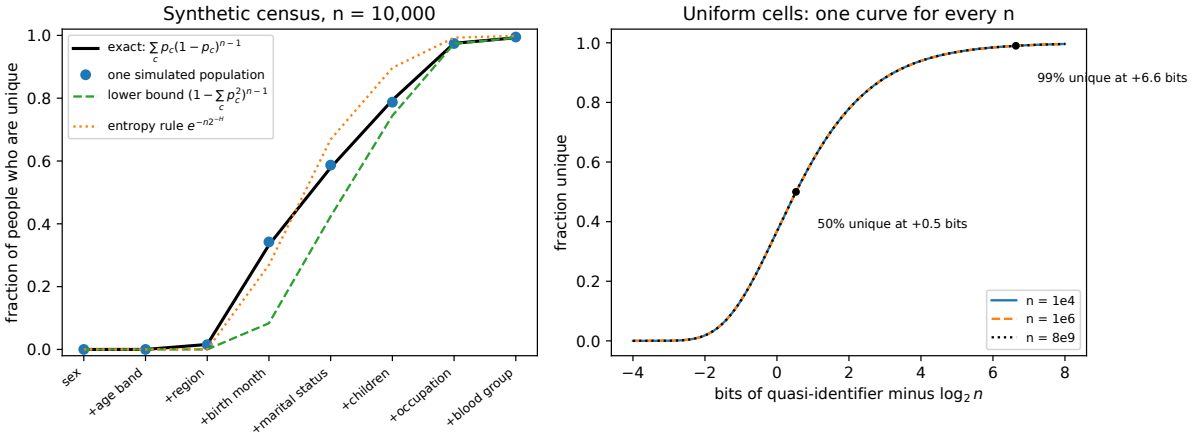


Figure 15.1.: Left: the fraction of people who are unique in the synthetic census as attributes are added, exact against one simulated population, with the collision floor and an entropy-based rule of thumb. Right: with uniform cells, the fraction unique depends only on the bits beyond $\log_2 n$; the curves for $n = 10^4$, 10^6 and 8×10^9 coincide.

15.4. Sparse, high-dimensional data: the Netflix attack

In 2006 Netflix released about 100 million ratings by 480,189 subscribers, names replaced by random numbers. There are no columns to join on, and what an adversary knows ("she watched these films around these dates") is partial and noisy, so the release looks safe. Narayanan and Shmatikov (2008) showed that it is not, by the same accounting. Most films are rated by few people, so each film in a record carries many bits: one rated by 1% of subscribers is worth $\log_2 100 = 6.64$ bits, and three such films carry 19.9 bits, more than the 18.9 that single out a subscriber.

Since no record matches exactly, the attack scores every record by how well it agrees with the auxiliary information aux, item by item:

$$\mathrm{Score}(\mathrm{aux}, r) = \sum_{i \in \mathrm{aux}} w_i \, \mathrm{Sim}(\mathrm{aux}_i, r_i), \tag{15.3}$$

where Sim is 1 if record r agrees on item i (say, within one star) and 0 otherwise. Rarer items get larger weights, because agreeing on a film half the subscribers rated says little and agreeing on an obscure one says a lot; Narayanan and Shmatikov used $w_i = 1/\ln|\mathrm{supp}(i)|$, with $|\mathrm{supp}(i)|$ the number of records that rated item i. The likelihood ratio of "target" against "unrelated" gives the principled version, $w_i \approx \ln(1/s_i) + \ln\frac{1-\varepsilon}{\varepsilon}$, with s_i the chance an unrelated record agrees on item i and ε the chance the adversary is wrong about it. Because the score adds up evidence, noise costs little: a wrong item costs the target one term, and unrelated records rarely gain one. On Netflix, 8 ratings of which 2 may be wrong, with dates known within 14 days, identify 99% of records.

Table 15.4.: Re-identification rate in a synthetic ratings matrix with 5,000 users: the fraction of 400 targets whose record gets the top score, for three weightings of Equation 15.3.

Auxiliary information	Equal weights	$1/\ln\lvert\mathrm{supp}(i)\rvert$	$\ln(n/\lvert\mathrm{supp}(i)\rvert)$
2 items	0.428	0.440	0.402
4 items	0.936	0.941	0.924
8 items	1.000	1.000	1.000
8 items, 2 of them wrong	0.943	0.989	0.978

A synthetic matrix of 5,000 users with Netflix-like density shows the bits deciding (Table 15.4). An item carries a median of 5.83 bits against $\log_2 5000 = 12.29$ needed, so two items fall a little short and re-identify about 43% of targets, while four clear the bar and re-identify 93%. The weighting barely matters except when some items are wrong: then equal weights drop to 94%, because a wrong popular item lets many unrelated records score. For a defender, perturbing a few values does not protect sparse data; the score needs a margin, not a perfect match.

If the target might not be in the release, the adversary must also decide when to trust the top score. The **eccentricity test** declares a match only when the best score s_1 stands out from the second best s_2, in standard deviations σ of all scores: $(s_1 - s_2)/\sigma \geq \varphi$. How often an outsider's top score stands out by chance depends on the score distribution, so no threshold is safe in the abstract. With 8 items of which 2 are wrong, the popular $\varphi = 1.5$ matches 94.8% of insiders but also "matches" 26.3% of outsiders. Holding false matches to 1% of outsiders takes $\varphi = 4.83$, which still re-identifies 73.0% of insiders. So calibrate the threshold on records known not to be the target, and read the attack at a small false positive rate (Figure 15.2).

15.5. Does releasing a sample protect?

Agencies often release a 1% sample, arguing plausible deniability: a record unique in the sample need not be unique in the population. Let each person be in the release with probability π, the **sampling fraction**, and let λ be the expected number of other people in the target's cell.

Suppose the adversary knows a person's cell and finds exactly one record there. Either the target was sampled and nobody else in the cell was, or the target was left out and one other person was. Each other person in the cell supplies a wrong story as likely as the right one, and since the release shows no second record, those others are all unsampled, about $\lambda(1-\pi)$ of them. The match is right with probability right over right plus wrong:

$$\kappa \approx \frac{1}{1 + \lambda(1-\pi)}. \tag{15.4}$$

Going the other way, a record alone in the sample is named by a population register, such as a voter roll, when nobody unsampled shares its cell, which has probability

$$\xi \approx e^{-\lambda(1-\pi)}. \tag{15.5}$$

Sampling enters both only through $1-\pi$, which is already close to 1 for any small sample. So what protects a person is a crowded cell, not the sampling: shrinking the sample can never push

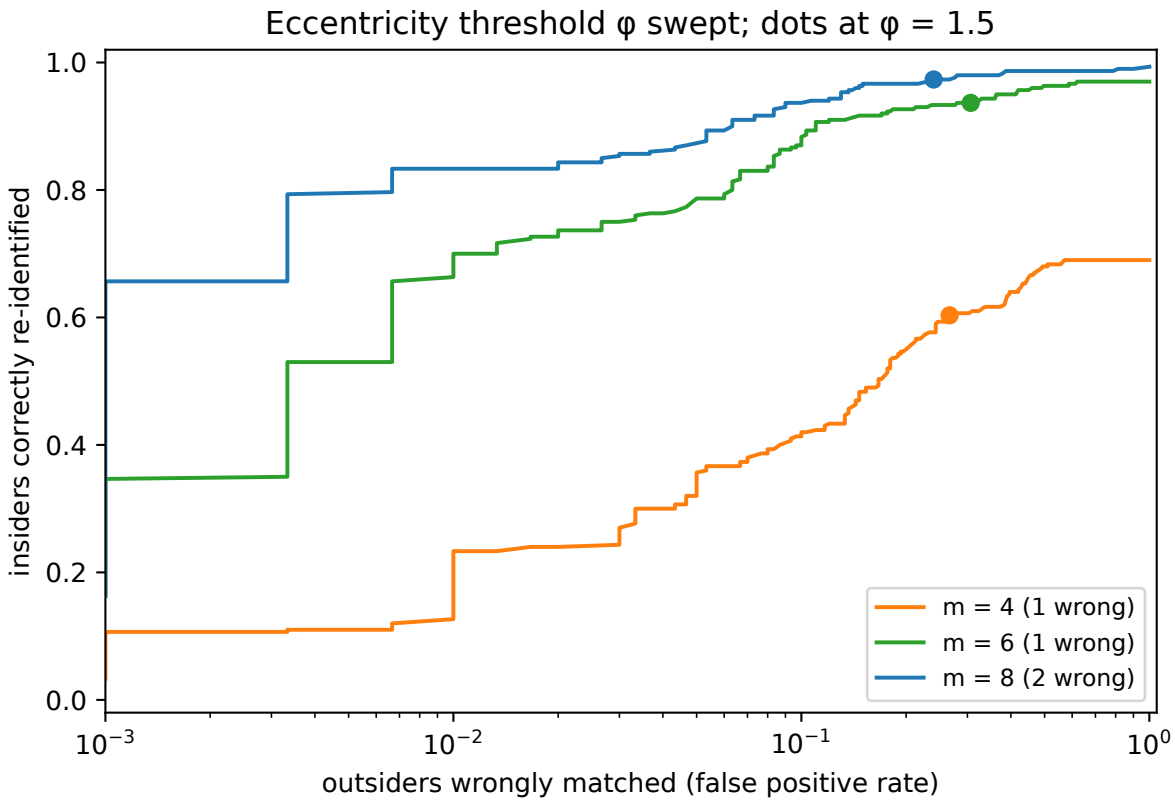


Figure 15.2.: True against false positive rates of the eccentricity test as the threshold φ is swept, for 4, 6 and 8 auxiliary items with 1, 1 and 2 of them wrong. The dots mark $\varphi = 1.5$. A false positive rate of 0 is drawn at the left edge.

κ below $1/(1+\lambda)$ or ξ below $e^{-\lambda}$, and for a rare cell both stay near 1 (Table 15.5). Going from a 50% to a 1% sample lowers κ by only 0.04 for a rare cell, and by 0.16 for a crowded one.

Table 15.5.: Probability that a match in a sample is correct: from a person to the release (κ) and from a sample-unique record to the population (ξ).

λ (expected others in the cell)	κ, $\pi = 0.01$	κ, $\pi = 0.5$	ξ, $\pi = 0.01$	ξ, $\pi = 0.5$
0.1	0.910	0.952	0.906	0.951
0.5	0.669	0.800	0.610	0.779
2	0.336	0.500	0.138	0.368

Over a whole release, a small sample grants deniability only to people who were not unique in the population anyway. In a synthetic population of 100,000 with seven attributes, 82.8% of people are unique, and in a 1% sample 0.8301 of the sample-unique records are population-unique. Nor does "any match might be wrong" help, because a sample is a fair picture of the population: the adversary can estimate each cell's λ from the sample itself and compute ξ record by record. Doing that from one 1% sample predicts 0.817 on average against an actual 0.824, and the records it rates above 0.95 are population-unique 99.3% of the time.

Removing names, perturbing values and sampling all failed the same way: they left enough bits in each record and bet on what the adversary didn't know. A defence has to hold whatever auxiliary data exist. The classic attempts, k-anonymity and its successors, are tested against that requirement in Syntactic anonymity and how it breaks in the full edition, and The definition of differential privacy gives the guarantee that meets it.

15.6. Code

Three numbers summarise the linkage risk of a release you hold: J/n, the fraction of unique records and the smallest class size. For a planned release, the uniform-cells formula and Equation 15.4 give quick estimates.

```
from collections import Counter
from math import exp, log2


def linkage_report(rows: list[tuple[str, ...]]) -> tuple[float, float, int]:
    """Linkage success J/n, fraction of unique records, and smallest class size of a release
    sizes = Counter(rows).values()  # one count per distinct quasi-identifier combination
    n = len(rows)
    return len(sizes) / n, sum(1 for k in sizes if k == 1) / n, min(sizes)


def uniform_unique(n: float, cells: float) -> float:
    """Expected fraction of n people who are unique when they fall in `cells` equally likely
    return exp(-n / cells)
```

```
def match_correct(lam: float, pi: float) -> tuple[float, float]:
    """In a sample of fraction pi, with lam expected others in the cell: (kappa, xi)."""
    return 1 / (1 + lam * (1 - pi)), exp(-lam * (1 - pi))


release = [
    ("02138", "1961-03-14", "F"), ("02138", "1972-11-02", "M"), ("02138", "1972-11-02", "M"),
    ("02139", "1958-06-21", "F"), ("02139", "1990-01-01", "F"), ("02139", "1990-01-01", "F"),
    ("02139", "1990-01-01", "F"), ("02141", "1983-09-30", "M"),
]
success, unique, k = linkage_report(release)
print(f"{success:.3f} {unique:.3f} {k}")  # 0.625 0.375 1
cells = 40_000 * 365 * 80 * 2  # ZIP code x birth date over 80 years x sex
print(f"{log2(cells):.1f} {uniform_unique(248.7e6, cells):.3f}")  # 31.1 0.899
kappa, xi = match_correct(0.1, 0.01)
print(f"{kappa:.3f} {xi:.3f}")  # 0.910 0.906
```

The full edition's script, linkage.py, computes every number in this chapter, including the synthetic Netflix experiment and the sampling simulations:

```
uv run python code/part02-attacks/linkage.py
```

15.7. Cheat sheet

Table 15.6.: The results of this chapter.

Result	Formula	Notes
Bits to name one person	$\log_2 n$	33 for the world, 28 for the US, 13 for 10,000 people
Linkage success	J/n	Target known to be in the release; at most $1/k$ if every class has $\geq k$ records
Expected fraction unique	$\sum_c p_c(1-p_c)^{n-1}$	Needs a model of the joint distribution
Uniform cells, b bits	$\exp(-2^{\log_2 n - b})$	50% unique at $\log_2 n + 0.53$ bits, 99% at $\log_2 n + 6.64$
Collision floor	$(1-\sum_c p_c^2)^{n-1}$	Needs only the marginals for independent attributes
Sparse-data score	$\sum_i w_i \operatorname{Sim}(\text{aux}_i, r_i)$, $w_i \approx \ln(1/s_i) + \ln\frac{1-\varepsilon}{\varepsilon}$	Each agreeing item is worth $\log_2(1/s_i)$ bits
Eccentricity	$(s_1 - s_2)/\sigma \geq \varphi$	Calibrate φ on outsiders at a small false positive rate

Result	Formula	Notes
Match in a sample	$\kappa \approx 1/(1+\lambda(1-\pi))$, $\xi \approx e^{-\lambda(1-\pi)}$	At least $1/(1+\lambda)$ and $e^{-\lambda}$ for every π

Rules of thumb

- Add up the bits of every attribute an adversary might know; above $\log_2 n$, assume most people are unique.
- Score a release you hold by J/n and its unique records, not by whether names were removed.
- Removing names, releasing a sample or perturbing a few values does not anonymize a dataset.

15.8. Further reading

- Sweeney, "Simple Demographics Often Identify People Uniquely" (2000): the 87% estimate and its variants with coarser geography.
- Narayanan & Shmatikov, "Robust De-anonymization of Large Sparse Datasets" (IEEE Symposium on Security and Privacy, 2008): the scoring algorithm, the eccentricity test and the Netflix experiments.
- Rocher, Hendrickx & de Montjoye, "Estimating the success of re-identifications in incomplete datasets using generative models" (Nature Communications, 2019): per-record re-identification risk from a copula model, and why sampling does not protect.
- Ohm, "Broken Promises of Privacy: Responding to the Surprising Failure of Anonymization" (UCLA Law Review, 2010): the legal side, with the Massachusetts, search-log and Netflix cases.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

16. Syntactic anonymity and how it breaks

16.1. Why this matters

Linkage and re-identification in the full edition showed that deleting names does not anonymize a table: the remaining **quasi-identifiers**, such as age, ZIP code and sex, single out most people. Between 1998 and 2008 a family of definitions answered by constraining the released table itself, each one proposed to stop an attack on the one before. **k-anonymity** hides every record in a crowd of at least k look-alikes. **l-diversity** asks each crowd to hold several different sensitive values. **t-closeness** asks each crowd to look like the whole table. They are **syntactic**: each is a property of the published table that you can check by reading it, with no reference to how it was produced or what else the adversary knows. Statistics offices, regulators' guidance and tools such as ARX still use them.

What they share is also what breaks them: each fixes the adversary's knowledge in advance. k-anonymity assumes the adversary knows the quasi-identifiers and nothing else, l-diversity allows a few facts about the target, t-closeness the table's overall distribution. Each attack in this chapter hands the adversary one piece of knowledge the definition did not anticipate, and the guarantee then says nothing. A second release, the most ordinary auxiliary information there is, breaks all three. Those failures are the specification for differential privacy.

One fact drives every attack below. Records are grouped into **equivalence classes**, sets of records with the same (generalized) quasi-identifiers, and each record has one **sensitive attribute** S, such as a diagnosis. An adversary who knows the target's quasi-identifiers can find the target's class E but not its record, so the best it can believe, its **posterior**, is that the target looks like a random member of the class:

$$\Pr[S_{\text{target}} = v \mid \text{release}] = \frac{n_E(v)}{|E|} = P_E(v), \tag{16.1}$$

where $n_E(v)$ counts the records of E with value v. If the adversary also knows the target's value is not u, it crosses out the records with u and renormalizes over the rest.

16.2. k-anonymity

Sweeney and Samarati's answer to linkage was to make sure no combination of quasi-identifiers singles out fewer than k people. A release is **k-anonymous** if every equivalence class has at least k records. A custodian gets there by **generalization** (age 37 becomes 35–39, ZIP code 02139 becomes 021**) and by **suppression**, deleting the few records that still stand out. What this buys follows from the linkage game: an adversary who finds the target's class must guess among

its records, so each class yields one record's worth of wins, and the success is the number of classes J over the number of records n. Classes of at least k records cannot number more than n/k, so

$$\Pr[\text{win}] = \frac{J}{n} \leq \frac{1}{k}. \tag{16.2}$$

The price is precision, and it is larger than it looks. Cells of the generalized quasi-identifiers vary in size around their average, and the unlucky small ones violate the definition, so the average must be well above k. For 10,000 people, keeping 99% of records in classes of at least 2 needs 4.61 people per cell on average, more than twice k, which allows the quasi-identifiers 11.08 bits, 2.20 below $\log_2 n$. Since 99% of people are unique 6.64 bits above it, the window between "almost everyone is unique" and "almost everyone is hidden" is only about 9 bits wide, and every column you publish spends some of it.

The **Mondrian** algorithm (LeFevre, DeWitt & Ramakrishnan, 2006) starts from the whole table as one box and repeatedly splits a box at the median of its widest quasi-identifier, as long as both halves keep k records. Because it recodes locally, classes stay close to size k. Table 16.1 applies it to a synthetic hospital table of 10,000 patients, with age, ZIP code and sex as quasi-identifiers and one of eight diagnoses that depend on age. With exact quasi-identifiers the linkage success is 0.933.

Table 16.1.: Mondrian k-anonymization of a synthetic hospital table of 10,000 patients. Mean width is the class's range as a fraction of each quasi-identifier's full range. Certain disclosure is the fraction of records in a class with a single diagnosis. Guess accuracy is the fraction of records whose diagnosis is the most common one in their class.

k	Classes	Sizes	Mean width	Linkage J/n	Certain disclosure	Guess accuracy
2	4,091	2–5	0.005	0.4091	0.2797	0.675
5	1,542	5–11	0.016	0.1542	0.0232	0.593
10	748	10–20	0.028	0.0748	0.0000	0.562
20	366	20–41	0.043	0.0366	0.0000	0.540
50	129	50–99	0.075	0.0129	0.0000	0.506

Linkage stays below $1/k$ at little cost in precision: at $k = 5$ a class's range covers 1.6% of each quasi-identifier's span on average. But the guarantee bounds the chance of picking out the target's record, not of learning the diagnosis, which is the point of the release. At $k = 5$, 2.3% of patients sit in a class with a single diagnosis, and the release states their diagnosis outright, no linkage needed.

16.3. Homogeneity, background knowledge and l-diversity

Machanavajjhala and colleagues (2006) made the gap concrete. Table 16.2 is 3-anonymous.

Table 16.2.: A 3-anonymous release with three classes of three records.

ZIP code	Age	Diagnosis
130**	< 30	heart disease
130**	< 30	heart disease
130**	< 30	heart disease
148**	4*	cancer
148**	4*	flu
148**	4*	heart disease
130**	3*	cancer
130**	3*	cancer
130**	3*	flu

Alice, 26, is in the first class, where every record says heart disease, so by Equation 16.1 her posterior is 1: the **homogeneity attack**, full disclosure without re-identification. Bob, 35, is in the third class, with posterior $\frac{2}{3}$ for cancer. His neighbour knows he did not have flu, crosses out the flu record, and cancer has posterior 1: the **background knowledge attack**. Both work because one value dominates the class, so l-diversity asks each class for enough different values.

Its three versions differ in what they bound. A class is **distinct ℓ-diverse** if it holds at least ℓ different values, and **entropy ℓ-diverse** if its distribution is as spread out as ℓ equally common values (entropy at least $\log_2 \ell$). Neither limits the share of the most common value, so neither caps the posterior: an entropy 3-diverse class over 8 values can have one value in 73% of its records, because the other 27%, spread over seven values, add enough entropy. The third version targets the background knowledge attack directly. Sort a class's value counts as $r_1 \geq r_2 \geq \ldots \geq r_m$; the class is **recursive (c, ℓ)-diverse** if

$$r_1 < c\,(r_\ell + \cdots + r_m). \tag{16.3}$$

The right side is the tail left after setting aside the $\ell - 1$ most common values. An adversary who rules out $\ell - 2$ values cannot make the top count larger, and whatever it removes, the values that remain besides the top one add up to at least that tail. So the top value still holds less than c parts out of $c + 1$. Every remaining value therefore keeps a posterior below $c/(1 + c)$: 0.500, 0.667 and 0.750 for $c = 1, 2, 3$. Even that cap lets the adversary be fairly sure. l-diversity protects against certainty, not likelihood, and only against an adversary who rules out at most $\ell - 2$ values; one who knows something softer ("probably not flu") is not covered.

To judge what a release teaches, compare with what the adversary would believe without it. On the hospital table (Table 16.3), the most common diagnosis overall is right for 18.5% of patients, but **imputation**, guessing the most likely diagnosis for each patient's age from the population, is right for 53.6%. So most of the 59.3% that a 5-anonymous release achieves is the age–diagnosis pattern any study would report, a fact about the population rather than a secret about anyone. Measure attribute disclosure against imputation, or you will count the study's findings as leaks.

Table 16.3.: Mondrian with l-diversity on the same hospital table: each split is allowed only if both halves satisfy the criterion.

Criterion	Classes	Sizes	Mean width	Linkage J/n	Certain disclosure	Guess accuracy
$k = 5$	1,542	5–11	0.016	0.1542	0.0232	0.593
$k = 5$, distinct $\ell = 3$	1,132	5–33	0.022	0.1132	0.0000	0.574
$k = 5$, recursive $(c = 2, \ell = 3)$	389	5–641	0.113	0.0389	0.0000	0.417

Against that baseline, distinct 3-diversity removes certain disclosure at almost no cost. Recursive $(2, 3)$-diversity forces some classes to hold hundreds of patients and pushes accuracy below imputation: it has started destroying the pattern the release exists to show.

16.4. Skewness, similarity and t-closeness

Li, Li and Venkatasubramanian (2007) found two attacks that diversity does not see. In the **skewness attack**, 1% of the table has HIV, and a class that is half HIV is 2-diverse by every definition, yet it raises the belief that a member has HIV from 1% to 50% (Figure 16.1). In the **similarity attack**, a class holding gastric ulcer, gastritis and stomach cancer is 3-diverse and still says "stomach disease". So counting values is the wrong yardstick: what matters is how far a class moves the adversary from what it would believe anyway, the whole table's distribution P.

A release is **t-close** if every class's distribution is within distance t of the table's, $d(P_E, P) \le t$. With the **total variation (TV)** distance, $\mathrm{TV}(P, Q) = \frac{1}{2}\sum_v |P(v) - Q(v)|$, the guarantee has a direct reading. TV is the largest gap between the two distributions' probabilities of any set of values, so every question the adversary can ask about the target's value gets an answer within t of the table's:

$$|P_E(A) - P(A)| \le t \quad \text{for every set of values } A. \tag{16.4}$$

In particular, guessing the class's top value is right with probability at most $\max_v P(v) + t$, and on the hospital table (Table 16.4) the accuracy stays under that bound.

Table 16.4.: Mondrian with t-closeness (total variation) on the hospital table. The table's most common diagnosis has frequency 0.185.

Criterion	Classes	Sizes	Mean width	Largest TV	Guess accuracy	Bound $\max_v P(v)+t$
$k = 5$	1,542	5–11	0.016	0.937	0.593	—
$k = 5$, $t = 0.3$	495	6–60	0.291	0.300	0.275	0.485

Criterion	Classes	Sizes	Mean width	Largest TV	Guess accuracy	Bound $\max_v P(v)+t$
$k = 5$, $t = 0.15$	136	14–304	0.324	0.149	0.222	0.335

The bound is additive, and that is its weakness. A difference of 0.15 is small next to a value held by 40% of patients and enormous next to one held by 1%: at $t = 0.15$ a 1% value may rise to 16%, a sixteen-fold increase for exactly the people who most need protection. The half-HIV class sits at TV 0.49; a class with 10% HIV sits at 0.09, passes $t = 0.1$, and still makes HIV ten times as likely. A guarantee for rare values has to bound the ratio of beliefs, not their difference.

t-closeness also fights the purpose of the release. If age predicts diagnosis, a class of similar ages has a different diagnosis distribution from the whole table, so making every class close to the table means mixing ages until the pattern is gone. In Table 16.4 the classes span about 30% of each quasi-identifier's range, and guess accuracy falls to 0.222–0.275, far below the 0.536 of imputation. The release now hides a population fact that was never a secret.

The distance also decides what counts as similar, and TV, which treats all values as equally different, misses the similarity attack. Li et al. use the **earth mover's distance (EMD)**, the least work to reshape one distribution into the other when moving mass between two values costs their distance. With a ground distance that puts gastritis near gastric ulcer, a class full of stomach diseases is far from the table and fails. For m ordered values (salary bands, age groups) at spacing $1/(m-1)$, the work is the net mass that has to cross each gap between neighbouring values:

$$\mathrm{EMD}(P_E, P) = \frac{1}{m-1} \sum_{i=1}^{m-1} \Big| \sum_{j \le i} \big(P_E(j) - P(j)\big) \Big|. \tag{16.5}$$

16.5. The composition attack

Every definition so far concerns one table. Ganta, Kasiviswanathan and Smith (2008) asked about two. Two hospitals in one city publish their tables, each properly k-anonymous or ℓ-diverse, without coordinating. A patient treated at both sits in a class of each, and the true diagnosis is in both classes, because the patient's own record carries it. So the adversary intersects the two sets of values, $\{v : n_{E_A}(v) > 0\} \cap \{v : n_{E_B}(v) > 0\}$, and the diagnosis is **pinned down** when one candidate is left. Neither hospital did anything wrong.

How often this works depends on class size, for a clear reason: every other member of a class brings a wrong value, and each wrong value is another chance for the two classes to share one. In a model where the other members are drawn from the population, the probability of pinning is 0.8734 for classes of 2 and 0.1615 for classes of 5, falling to 0.0015 for classes of 10. On data, take a city of 15,000 people in which two hospitals each treat a random 10,000, so 6,649 are treated at both, and each publishes a 5-anonymous Mondrian release (Table 16.5).

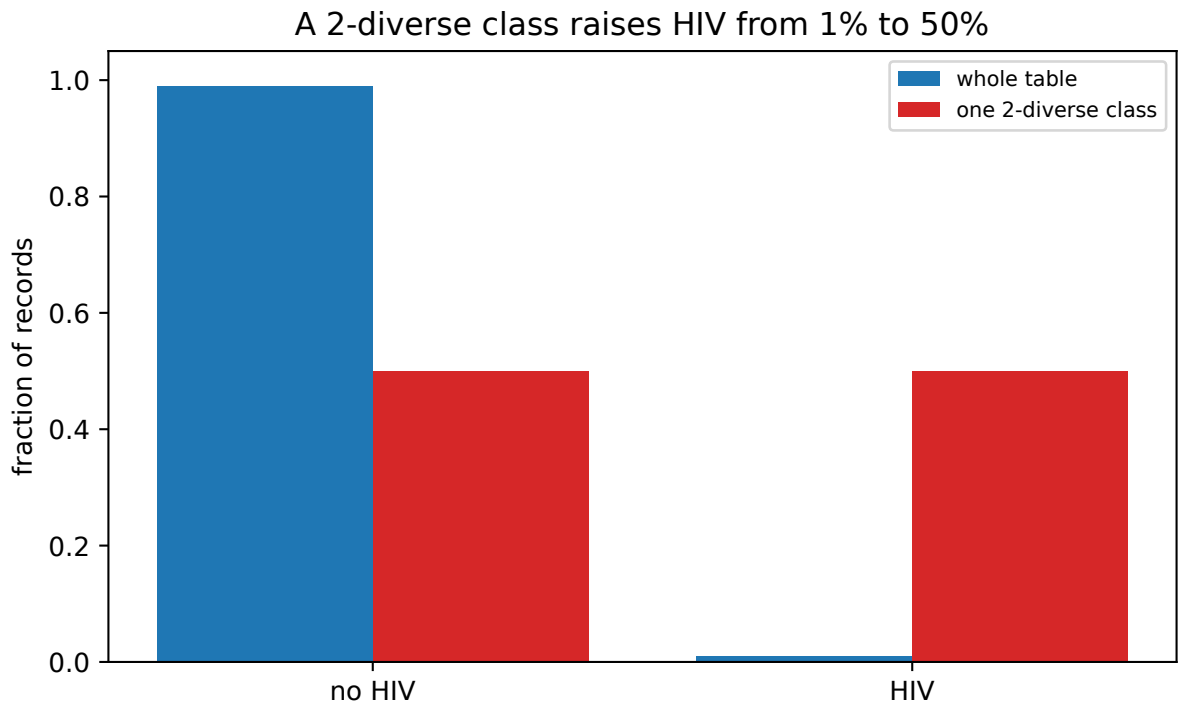


Figure 16.1.: The skewness attack: a table in which 1% of patients have HIV, and a class of it that is half HIV. The class is 2-diverse but at total variation 0.49 from the table.

Table 16.5.: The composition attack on two independent hospital releases, for the 6,649 people treated at both. "Pinned" is the fraction left with a single possible diagnosis.

Releases	Pinned by A alone	Pinned by B alone	Pinned by A and B	Mean candidates, A alone	Mean candidates, A and B
$k = 5$	0.0289	0.0262	0.0985	2.79	2.38
$k = 5$, distinct $\ell = 3$	0.0000	0.0000	0.0027	3.22	2.92

Together, two 5-anonymous releases pin down the diagnosis of 9.9% of shared patients, more than three times either alone. With distinct 3-diversity neither release pins anyone, and together they pin 18 patients. The model understates the risk: a patient's class here holds 6.8 records on average, and the model predicts only 2.9% pinned for classes of 7, because real classes group people of similar age, who share diagnoses more often than random draws.

No custodian can prevent this by checking its own table harder, since it cannot know every other release its patients appear in. A better definition must therefore do three things. It has to hold whatever the adversary knows, since another release is just auxiliary information. It has to say how leakage adds up when releases are combined. And it has to bound the ratio of beliefs with and without a person's data, so that rare values are protected. The definition of differential privacy starts from these three requirements.

16.6. Code

An auditor can check every definition of this chapter by grouping the rows. The snippet audits Table 16.2: smallest class, fewest distinct values, entropy-equivalent ℓ, recursive $(2, 2)$-diversity and TV distance from the table.

```
from collections import Counter, defaultdict
from math import log2


def classes(rows: list[tuple[str, str, str]]) -> list[Counter[str]]:
    """Group (ZIP, age, diagnosis) rows by their quasi-identifiers; count the diagnoses in e
    groups: defaultdict[tuple[str, str], Counter[str]] = defaultdict(Counter)
    for zip_code, age, diagnosis in rows:
        groups[(zip_code, age)][diagnosis] += 1
    return list(groups.values())


def entropy_l(counts: Counter[str]) -> float:
    """2 ** entropy: the number of equally common values with the same spread."""
    total = sum(counts.values())
    return 2 ** -sum(c / total * log2(c / total) for c in counts.values())


def recursive_diverse(counts: Counter[str], c: float, l: int) -> bool:
    """Recursive (c, l)-diversity: the top count is below c times the counts from the l-th l
    r = sorted(counts.values(), reverse=True)
    return r[0] < c * sum(r[l - 1 :])


def tv_to_table(counts: Counter[str], table: Counter[str]) -> float:
    """Total variation between a class's distribution and the whole table's."""
    nc, nt = sum(counts.values()), sum(table.values())
    return sum(abs(counts[v] / nc - table[v] / nt) for v in table) / 2


rows = [
    ("130**", "<30", "heart disease"), ("130**", "<30", "heart disease"), ("130**", "<30", "
    ("148**", "4*", "cancer"), ("148**", "4*", "flu"), ("148**", "4*", "heart disease"),
    ("130**", "3*", "cancer"), ("130**", "3*", "cancer"), ("130**", "3*", "flu"),
]
table = Counter(diagnosis for _, _, diagnosis in rows)
found = classes(rows)
print(min(sum(e.values()) for e in found))  # 3
print(min(len(e) for e in found))  # 1
print(" ".join(f"{entropy_l(e):.2f}" for e in found))  # 1.00 3.00 1.89
print(all(recursive_diverse(e, 2, 2) for e in found))  # False
```

```
print(" ".join(f"{tv_to_table(e, table):.3f}" for e in found))  # 0.556 0.111 0.444
```

The full edition's script, syntactic_anonymity.py, implements Mondrian with each criterion and runs every attack of this chapter:

```
uv run python code/part02-attacks/syntactic_anonymity.py
```

16.7. Cheat sheet

Table 16.6.: The syntactic definitions and the attacks on them.

Definition or attack	Condition	What it gives, and where it ends
Posterior in a class	$P_E(v) = n_E(v)/\|E\|$	Ruling out values renormalizes over the rest
k-anonymity	Every $\|E\| \geq k$	Linkage $J/n \leq 1/k$; nothing about the sensitive attribute
Distinct, entropy l-diversity	$\geq \ell$ values; entropy $\geq \log_2 \ell$	No cap on the top value's share
Recursive (c, ℓ)-diversity	$r_1 < c(r_\ell + \cdots + r_m)$	Posterior $< c/(1+c)$ after ruling out $\ell - 2$ values
t-closeness (TV)	$\mathrm{TV}(P_E, P) \leq t$	Every belief moves by $\leq t$; a 1% value may rise to $1\% + t$
t-closeness (EMD)	$\mathrm{EMD}(P_E, P) \leq t$	Similar values are close; catches the similarity attack
Composition	Intersect the values of the target's classes	Breaks all three; no single-table check prevents it

Rules of thumb

- Of a "k-anonymous" release, ask what each class's sensitive values reveal, not just how big it is.
- Measure attribute disclosure against imputation from the population.
- Rare values need a bound on the ratio of beliefs, not their difference.
- Assume another release exists.

16.8. Further reading

- Sweeney, "k-Anonymity: A Model for Protecting Privacy" (2002): the definition, generalization and suppression.

- Machanavajjhala, Kifer, Gehrke & Venkitasubramaniam, “l-Diversity: Privacy Beyond k-Anonymity” (2007) and Li, Li & Venkatasubramanian, “t-Closeness: Privacy Beyond k-Anonymity and l-Diversity” (ICDE, 2007): the attacks on the sensitive attribute and the definitions that answer them.
- Ganta, Kasiviswanathan & Smith, “Composition Attacks and Auxiliary Information in Data Privacy” (KDD, 2008): the intersection attack and why a definition must survive auxiliary information.
- Article 29 Working Party, “Opinion 05/2014 on Anonymisation Techniques”: how European regulators judge these techniques against singling out, linkability and inference.

Derivations. Every result in this chapter is derived step by step in the full edition’s chapter in the full edition.

17. Reconstruction attacks

17.1. Why this matters

A hospital keeps one sensitive bit per patient, whether they tested positive, and never releases it. It only answers counting questions, "how many of patients 3, 17, 42, … tested positive?", and adds a little noise to every answer. Is that safe? Dinur and Nissim (2003) showed that it is not, unless the noise is large. Each answer is a linear equation whose unknowns are the patients' bits, so a table of aggregates is a system of equations, and enough equations with small enough errors determine the unknowns.

The error of the answers is therefore the only thing standing between the attacker and the solution, and it fails in two ways. It can be too small: to hide a wrong guess, the error must be as large as the amount by which that guess changes the answers, which for a random query is about $\sqrt{n}$, where n is the number of people. Or there can be too many answers: errors that are independent from answer to answer are averaged away by asking more. Dwork and Roth call this the **Fundamental Law of Information Recovery**: overly accurate answers to too many questions destroy privacy. If you run or audit a system that answers aggregate queries, check both.

Throughout, the secret is a bit vector $d = (d_1, \dots, d_n)$, one bit per person. A **subset-sum query** asks how many people in a set S have bit 1. The **curator**, the party holding d, answers m queries, each with an error, and "error at most E" means every answer is within E of the truth.

17.2. How accurate is too accurate?

Start with the strongest attacker, who may ask all 2^n subset sums, and throws away every candidate database that contradicts an answer by more than the curator's error allows. Any candidate c that survives is close to d, for a simple reason. The bits where c is wrong form two sets: the ones it missed and the ones it invented. The attacker asked about both. On the set of missed ones, d sums to the size of the set and c to zero, yet both are within E of the same published answer, so the set has at most $2E$ people; the same holds for the invented ones. A surviving candidate therefore has at most $4E$ wrong bits, and the bound is tight. With $n = 10^6$ people and every answer off by up to 1000, the attacker gets at most 4,000 bits wrong, 0.4% of them. Error well below n is useless against unlimited queries; a curator whose error lets the attacker recover all but a vanishing fraction of the bits is called **blatantly non-private**.

Dinur and Nissim's main result replaces the 2^n questions with random subsets, each including every person with probability $\frac{1}{2}$. The price is a lower threshold, and the reason is that a random query cannot aim. Suppose a candidate is wrong on k bits. A query aimed at exactly those bits would move the answer by k, but a random query picks up each wrong bit with probability $\frac{1}{2}$, so

its sum over them behaves like k coin flips and moves by only about $\sqrt{k}$. An error below that scale cannot hide a candidate wrong on a constant fraction of the bits. So with error well below $\sqrt{n}$ on a number of random queries slightly above n, a **linear-program decoder** recovers all but a small fraction of the bits in polynomial time: it finds any vector with entries between 0 and 1 whose answers are within E of every published answer, and rounds it.

The proved constants are poor; at $n = 1000$ the proof needs about $33n$ queries. Don't read them as a safety margin, because the scaling is what matters and real attacks beat the constants easily: least squares with $m = 4n$ already recovers 96.55% of the bits under Gaussian noise of $\sigma = 0.25\sqrt{n}$ (Section 17.3). Random queries aren't needed either. Cohen and Nissim (2020) ran a linear-program reconstruction against a commercial query system that added noise to its answers and recovered most of a secret column.

The same logic says which distortions don't count as noise. A curator that rounds every answer to a multiple of 4 has error at most 2, but the rounding is a known function of the true sum, so an attacker who applies it to each candidate's answers recovers d exactly. And a few wildly wrong answers don't help the curator: a variant of the decoder that minimises the total absolute error, instead of requiring every answer to fit, tolerates a constant fraction of them (Dwork, McSherry and Talwar, 2007).

17.3. Least squares under random and worst-case error

The attack you would actually run is simpler. **Least squares** picks the vector whose answers come closest to the published ones in squared distance, then clips each coordinate to $[0, 1]$ and rounds it. Because its output is a linear function of the answers, its error is a linear function of the curator's, and you can predict its success.

With independent Gaussian noise of standard deviation σ on each of m random subset sums, each decoded coordinate is off by a normal error with standard deviation about $2\sigma/\sqrt{m-n}$. The reasoning is the familiar one for averages: n equations are needed to pin down n unknowns, and the $m - n$ beyond those average the noise down by $\sqrt{m-n}$; the factor 2 appears because a random query includes each person only half the time. A bit comes out wrong when its error crosses $\frac{1}{2}$, so the expected fraction recovered is

$$\Phi\left(\frac{\sqrt{m-n}}{4\sigma}\right), \tag{17.1}$$

where Φ is the standard normal cumulative distribution function. Two consequences follow. At a fixed ratio m/n, only $\sigma/\sqrt{n}$ matters, so $\sqrt{n}$ is again the scale of the threat. And at a fixed σ the fraction tends to 1 as m grows, so to hold the attack at a fixed accuracy the noise must grow like $\sqrt{m}$.

Table 17.1 checks the formula on four random databases of $n = 500$ bits with $m = 2000$ queries.

Table 17.1.: Least-squares reconstruction of $n = 500$ bits from $m = 2000$ random subset sums with Gaussian noise.

$\sigma/\sqrt{n}$	0.1	0.25	0.5	1	2
σ	2.24	5.59	11.18	22.36	44.72
Recovered, simulated	1.0000	0.9655	0.8130	0.6550	0.5875
Recovered, Equation 17.1	1.0000	0.9585	0.8069	0.6676	0.5858

Noise of order $\sqrt{n}$ is not automatically safe: at $\sigma = 0.25\sqrt{n}$, 96.55% of the bits come back, and recovery degrades smoothly instead of switching off at a threshold. The column at $\sigma = 2\sqrt{n}$ shows the second failure. That curator looks safe at $m = 4n$, where only 0.5875 of the bits come back, but if it answers any number of queries at the same noise level, an attacker who asks $m = 106.1\,n$ recovers 90% of them (Figure 17.1). A bound on the error of each answer protects nothing when the number of answers is unbounded; the error budget has to cover all of them together.

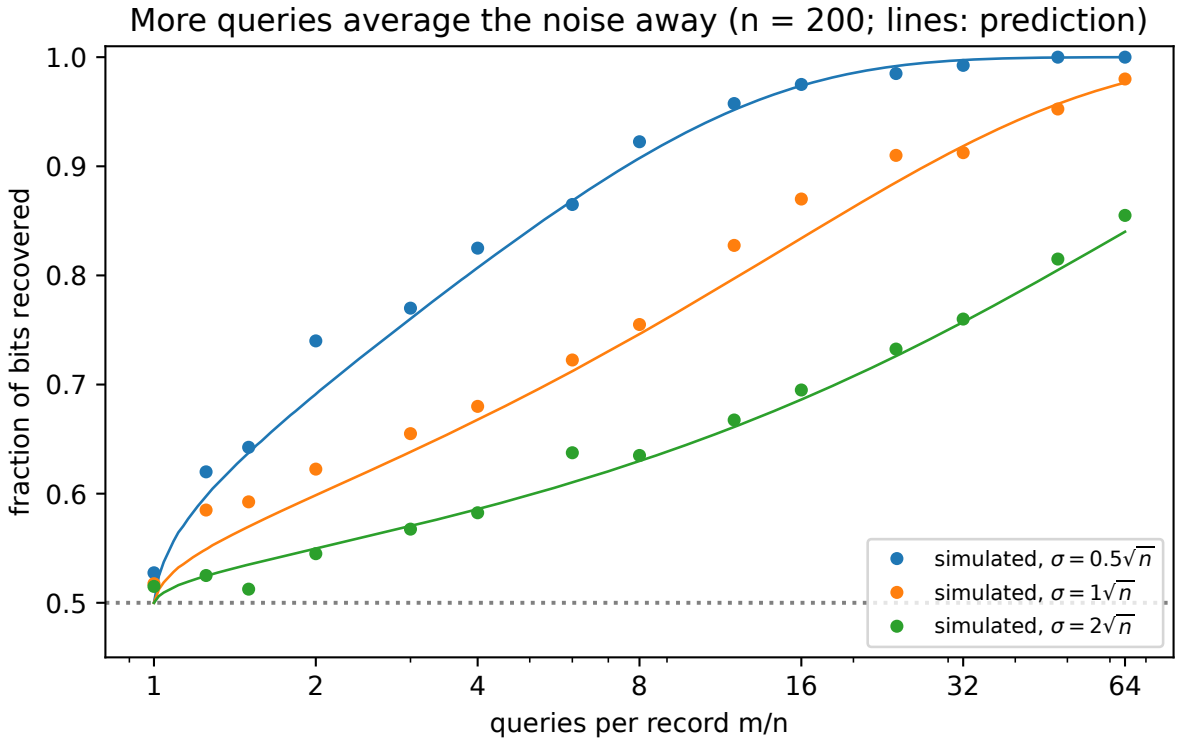


Figure 17.1.: Fraction of bits recovered by least squares at $n = 200$ as the number of queries grows, for three noise levels. Lines are Equation 17.1; dots are simulations. Near $m = n$ the system is barely determined and the noise is amplified; beyond that, more queries average the noise away, and even at $\sigma = 2\sqrt{n}$ the attack recovers most bits with $64n = 12\,800$ queries.

Gaussian noise is also the friendly case. Least squares amplifies errors most along one direction, that of the smallest singular value $s_{\min}(A)$ of the query matrix, and independent noise points in

all directions equally. A curator that pushes every error along that weakest direction does far more damage: at $E = 0.25\sqrt{n} = 5.59$ it causes 143 wrong bits, against about 17 for Gaussian noise of the same scale. The general guarantee, at most $4mE^2/s_{\min}(A)^2$ wrong bits, covers every curator. So when a system's errors are not independent random noise, test it with errors chosen against the decoder, not only with Gaussian noise of the same size.

17.4. The Fundamental Law: what stops reconstruction, and what doesn't

Is error of order $\sqrt{n}$ enough to stop reconstruction? A **sampling curator** shows it can be. It keeps each person with probability p in a secret set, answers every query from that set, and scales up by $1/p$ so that answers are right on average. Each answer's error is then a sum of independent zero-mean terms, one per person, so it is of order $\sqrt{n}$, and the largest of all m errors is only a logarithmic factor above that, of order $\sqrt{n \log m}$. But the people left out never affect any answer, so nothing reveals their bits, and no attacker recovers more than a fraction $p + \frac{1-p}{2}$ of a uniformly random d on average: the sampled bits, plus a coin flip for the rest. At $n = 500$, $m = 2000$ and $p = \frac{1}{2}$, the largest error is 28.00, or $1.25\sqrt{n}$, and least squares recovers 0.7560 of the bits, right at the ceiling of three quarters.

That is not privacy, though. The same curator answers exactly for the people in its sample, and an attacker with enough queries learns all of their bits: half the people are fully exposed so that the other half can be fully hidden. Stopping blatant reconstruction is a much weaker goal than protecting everyone. A guarantee has to hold for every individual, which is what The definition of differential privacy provides, and it has to control both halves of the law, the error scale and the number of answers. That is why differential privacy tracks a budget spent across queries.

How few queries can any attack use? Count bits. The secret carries n bits; an answer that takes at most V values carries at most $\log_2 V$; and a guess that may be wrong on a fraction β of the bits still leaves $h(\beta)$ bits of uncertainty per person, where $h(\beta) = -\beta \log_2 \beta - (1-\beta)\log_2(1-\beta)$ is the **binary entropy**. So any attack needs at least

$$m \geq \frac{n\,(1 - h(\beta))}{\log_2 V} \tag{17.2}$$

queries. At $n = 1000$ and 5% wrong bits, exact counts need at least 71.6 queries, and rounding every count to a multiple of 32 raises the floor only to 141.5. Coarser answers carry fewer bits each, but this is a floor on what an attack needs, not a defence: a curator that answers unlimited queries has only roughly doubled the minimum the attacker must ask, and the attacker can simply ask more.

17.5. Reconstructing a census block

Statistical offices publish tables, not query answers, but every cell is a query, and many tables come from the same confidential records. Garfinkel, Abowd and Martindale (2019) showed how far this goes with a toy block of seven people, each with an age, a sex, a race and a marital status, where nobody under 18 is married. Table 17.2 is everything published about such a block. Means to two decimals give exact sums, since the sum is a whole number: $34.29 \times 7 = 240.03$, so the ages total 240.

Table 17.2.: The published statistics for a fictional census block of seven people. A median over an even count is the average of the two middle ages.

Group	Count	Median age	Mean age
Total	7	33	34.29
Female	4	31	30.25
Male	3	33	39.67
Black	4	33.5	36.25
White	3	33	31.67
Married	4	43.5	47.75

A constraint solver searches for every block consistent with these eighteen numbers, and each table cuts the candidates down: of 14,790 age vectors that fit the sex statistics, 387 admit a race split, 33 a marital split, and one all three. It is the true block (Table 17.3).

Table 17.3.: The block reconstructed from Table 17.2, which is the true block.

Person	1	2	3	4	5	6	7
Age	7	15	27	33	35	52	71
Sex	F	M	F	M	F	F	M
Race	B	B	W	W	W	B	B
Married	no	no	no	yes	yes	yes	yes

Withholding a number rarely helps, because the remaining numbers usually imply it: eight of the twelve medians and means leave the block unique when withheld (Figure 17.2). The worst case for the attacker, withholding the married mean, the only statistic about the single people's ages, still leaves only 45 blocks, and 51% of them have the true sex, race and marital pattern of all seven people.

Real publications are more exposed than the toy, not less: they cross-classify many variables at many geographic levels, which is the "too many questions" half of the law. And the usual protections leave most constraints exact. Swapping a small fraction of households between blocks, the main protection of the 2010 Census, changes some answers; coarsening ages and suppressing small cells remove some constraints; the rest still bind. The Census Bureau's own reconstruction of the 2010 Census, described by Abowd (2021), rebuilt a record for each of the 308,745,538 people counted. About 46% matched a real person exactly on block, sex, age, race and ethnicity, and linking to commercial data confirmed the identity of about 17% of the population. The Bureau's response was to move the 2020 Census to a method based on differential privacy.

17.6. Code

To audit a query system or plan a release, three numbers come first: what fraction least squares would recover at your noise level, how many queries per person push it to a target, and the fewest queries any attack needs. The snippet computes all three, using the finite-m variance $4\sigma^2 n/((n+1)(m-n-1))$ that the full edition derives, which is about $4\sigma^2/(m-n)$.

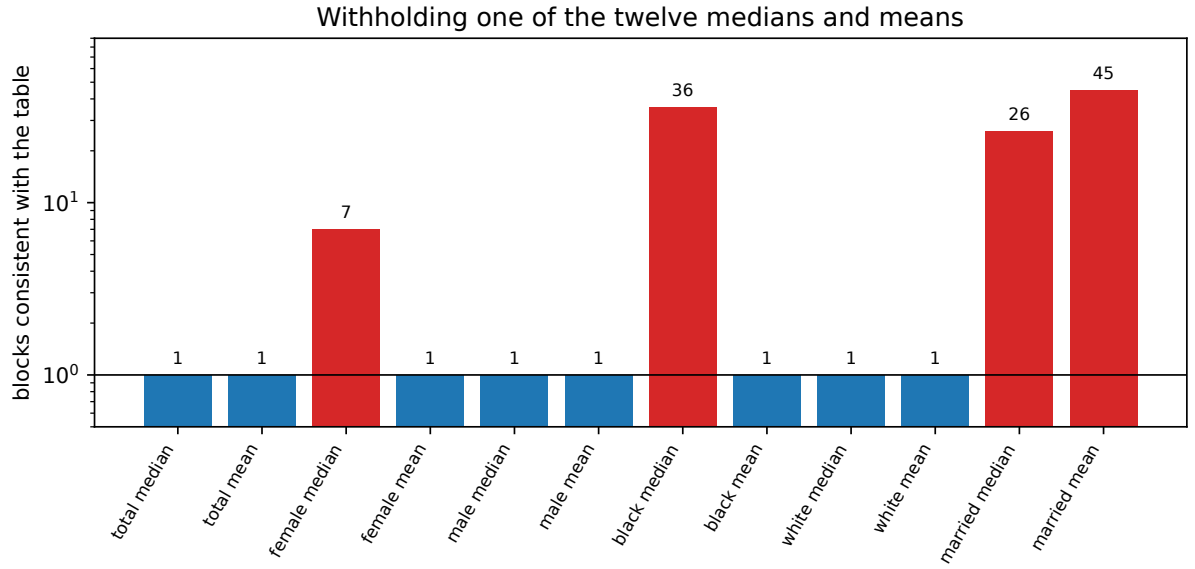


Figure 17.2.: Number of seven-person blocks consistent with Table 17.2 when one median or mean is withheld (log scale). Blue bars: the block is still unique. Red bars: a few candidates remain, still far fewer than the possible blocks.

```python
from math import log2, sqrt
from statistics import NormalDist

PHI = NormalDist()


def ls_recovered(n: int, m: int, sigma: float) -> float:
    """Predicted fraction of n secret bits that least squares recovers from m noisy random s
    sd = 2 * sigma * sqrt(n / ((n + 1) * (m - n - 1)))  # spread of each decoded bit's error
    return PHI.cdf(0.5 / sd)  # a bit comes out wrong when its error crosses 1/2


def queries_per_person(target: float, sigma_over_sqrt_n: float) -> float:
    """m/n at which least squares recovers a fraction `target` of the bits, for noise sigma
    return 1 + (4 * sigma_over_sqrt_n * PHI.inv_cdf(target)) ** 2


def binary_entropy(p: float) -> float:
    return -p * log2(p) - (1 - p) * log2(1 - p)


def min_queries(n: int, wrong: float, values: int) -> float:
    """Fewest answers, each taking at most `values` values, from which any attack gets all b
    return n * (1 - binary_entropy(wrong)) / log2(values)
```

```
n, m = 500, 2000
print(" ".join(f"{ls_recovered(n, m, c * sqrt(n)):.4f}" for c in (0.1, 0.25, 0.5, 1, 2)))  #
print(f"{queries_per_person(0.9, 2.0):.1f}")  # 106.1
print(f"{min_queries(1000, 0.05, 1001):.1f}")  # 71.6
print(f"{min_queries(1000, 0.05, 1000 // 10 + 2):.1f}")  # 106.9
print(f"{min_queries(1000, 0.05, 1000 // 32 + 2):.1f}")  # 141.5
```

The first line is the prediction row of Table 17.1. The full edition's script, reconstruction.py, runs the attacks themselves: the exponential attack, least squares under random and worst-case error, the sampling curator and the census-block search.

```
uv run python code/part02-attacks/reconstruction.py
```

17.7. Cheat sheet

Table 17.4.: The regimes of reconstruction and the rules for each.

Setting	Result	Notes
All 2^n subset sums, error $\leq E$	Any consistent candidate has $\leq 4E$ wrong bits	Tight; error $o(n)$ is blatantly non-private
Random subset sums, error $\leq E$	Error well below $\sqrt{n}$ on about $n \log n$ queries: a linear program recovers almost all bits	Proved constants are pessimistic
Least squares, Gaussian noise σ	Fraction recovered $\approx \Phi(\sqrt{m-n}/(4\sigma))$	Noise must grow like $\sqrt{m}$ to hold it fixed
Least squares, worst-case error	Wrong bits $\leq 4mE^2/s_{\min}(A)^2$	Adversarial errors hurt more than random ones
Sampling curator	Error $O(\sqrt{n \log m})$; recovery $\leq p + \frac{1-p}{2}$	Stops reconstruction, exposes the sampled people
Information bound	$m \geq n(1 - h(\beta))/\log_2 V$	Exact counts: of order $n/\log n$ queries
Published tables	Each cell is a constraint	Swapping, coarsening and suppression leave most constraints exact

Rules of thumb

- The threat scale of a random subset-sum query is $\sqrt{n}$, of the worst query n. Errors well below that reveal the database.
- Budget the error over all answers, not per answer.
- Rounding, swapping and suppression are not noise: an attacker can simulate them or route around them.
- Stopping reconstruction is not privacy; ask what the system guarantees for each individual.

17.8. Further reading

- Dinur & Nissim, “Revealing Information while Preserving Privacy” (PODS 2003): the original attacks and the notion of blatant non-privacy.
- Dwork, Smith, Steinke & Ullman, “Exposed! A Survey of Attacks on Private Data” (Annual Review of Statistics and Its Application, 2017): a readable survey of reconstruction and tracing attacks.
- Garfinkel, Abowd & Martindale, “Understanding Database Reconstruction Attacks on Public Data” (CACM 2019): the seven-person block reconstructed with a solver.
- Cohen & Nissim, “Linear Program Reconstruction in Practice” (Journal of Privacy and Confidentiality, 2020): the attack against a deployed query system.

Derivations. Every result in this chapter is derived step by step in the full edition’s chapter in the full edition.

Part III.

Differential privacy foundations

The previous part showed that models and statistics leak: membership, attributes, training text, whole datasets. Each attack found a difference between what a system releases with your record and without it. This part introduces the defence the rest of the book is built on, **differential privacy (DP)**, which makes that difference provably small, and takes it far enough for you to build a private release, check one somebody else built, and say what it costs. The finer tools for thousands of training steps come in the next part.

Who this part is for

Read it if you build, review or audit a system that publishes statistics or trains models on personal data and claims a privacy guarantee. A claim such as "trained with $\varepsilon = 8$, $\delta = 10^{-5}$" is only useful if you can say what it promises and to whom, check the noise behind it, and spot the mistakes that silently void it. The part assumes the attack games of Threat models and attacks and a little probability from the Prerequisites: the Laplace and normal distributions, and the error rates of a test.

The common thread

A randomized algorithm is differentially private when changing one person's record changes the probability of any output by at most a factor e^{ε}, up to a small slack δ. The whole part reads that sentence in three ways, each answering a practical question.

Read as a **bound on every attack**, it tells you what the guarantee is worth. An attacker asking whether you are in the data ends by saying yes or no, and the definition caps how much more often they can say yes when it is true than when it is false. So a membership attack's true positive rate is at most e^{ε} times its false positive rate, plus δ, whatever the attacker knows and however much they compute. That is how you turn ε into something a risk owner understands, and, run backwards, how an attack can prove a claimed ε wrong.

Read as a **recipe for mechanisms**, it tells you how to build a release. If one record can move an answer by at most some amount, its **sensitivity**, then noise that is wide compared with that amount makes the two worlds look alike. The size of the noise follows from the sensitivity and ε, so most of the work is bounding how far one record can move the answer and choosing the cheapest noise that hides it.

Read as a **budget to spend**, it tells you how guarantees combine. Anything computed from a private output is free, because computing cannot add evidence the output didn't contain. But every new look at the data adds its own evidence, so privacy losses add up over releases, and over the many records one person may own.

How to read this part

The chapters follow the order in which those questions come up when you build something. The definition in the full edition comes first, because every later claim is checked against it. Post-processing, group privacy and composition in the full edition comes next: they let you prove

each building block once and assemble a system from blocks. Then the Laplace and Gaussian mechanisms in the full edition give the two building blocks almost everything else uses. Advanced and optimal composition in the full edition then revisits the budget, because adding up losses is safe but far too pessimistic for many releases; its exact accounting is what the next part builds on.

The next two chapters cover what noise on a number does not: selection in the full edition, where the output is a choice among options, and data whose worst case is far from typical in the full edition, where noise sized for the worst case would drown the answer. Discrete and floating-point-safe mechanisms in the full edition is short and practical: a mechanism private on paper can leak through how a computer draws random numbers, so read it before implementing one. Local, shuffle and distributed trust models in the full edition asks who sees the raw data. Lower bounds in the full edition comes last because it closes the loop with the previous part's attacks: it says how much accuracy any private release must lose, and so when a privacy target and an accuracy target cannot both be met.

Each chapter starts from the practical question, explains every result with the reason it holds and a worked example, and ends with a cheat sheet. The derivations are in the full edition, linked at the end of each chapter.

What you will be able to do

By the end of this part you will be able to state a DP guarantee with its neighbouring relation and unit of privacy, and translate a claimed (ε, δ) into a cap on what an attacker can achieve. You will be able to compute the sensitivity of a query, calibrate Laplace and Gaussian noise exactly, add up the cost of a pipeline, and choose a δ that doesn't hide a catastrophe. You will know which mechanism fits a selection problem or data with a heavy worst case, why naive floating-point noise is not private, what the local and shuffle models buy, and what accuracy no private mechanism can beat.

18. The definition of differential privacy

18.1. Why this matters

Every attack in Threat models and attacks worked the same way. The attacker looked at what a system released and asked which of two worlds produced it: the one where a given person's record was in the data, or the one where it was not. Membership inference asks this about a trained model, reconstruction asks it about a table of statistics, and each succeeded because the output looked different enough in the two worlds for a test to tell them apart.

So the defence has to make the two worlds look alike, not just to today's attacks but to every test anyone could run. **Differential privacy (DP)** is that requirement written down. It is a property of the algorithm that produces the release, so you can check it before any data are touched, and it holds whatever the attacker knows or computes.

18.2. What "one person" means

The guarantee compares two datasets that differ in one person, and the **neighbouring relation** says what that means. There are two standard choices. Under **add/remove**, the second dataset is the first with one record added or taken away. The attacker's question is then "is this person in the data at all?", which is exactly the membership question, and the dataset's size is private too. Under **replace**, both datasets have the same size and one record's value is swapped for another. Now the attacker is assumed to know who is in the data, and the guarantee only hides what their record says.

Replacing Alice's record by Dan's is removing Alice and then adding Dan: two add/remove steps, each allowed to shift the output by the guaranteed amount. So ε under add/remove becomes 2ε under replace. The reverse fails, because a replace guarantee never compares datasets of different sizes: publishing the number of records is perfectly private under replace and not private at all under add/remove.

You can see the factor of 2 in a histogram of smokers and non-smokers, noised for $\varepsilon = 1$ per record. Removing Carol changes one bar by one, and the largest privacy loss (defined in the next section) is 1.0000. Changing Alice's answer from smoker to non-smoker moves two bars at once, one down and one up, and the loss doubles to 2.0000.

The second choice is what a record is. If each training example is a record, the guarantee protects each example. A person who wrote 200 messages is then 200 records, and hiding the person means hiding all 200 at once, which the guarantee covers only at a much higher price (Post-processing, group privacy and composition in the full edition). So always ask of a claimed ε: per what? The answer should be the unit you care about, usually the person.

Neighbours range over every record the data format allows, including a person aged 900. A mechanism must hide even that one, which is why private algorithms clip their inputs first.

18.3. The definition

Run a randomized algorithm M, called a **mechanism**, on neighbouring datasets D and D', and look at an output y. The natural measure of what y reveals is how much more likely it is in one world than in the other. Its logarithm is the **privacy loss**,

$$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}, \tag{18.1}$$

where $p_{M(D)}(y)$ is the probability (or density) of output y on dataset D. A loss of 0 tells the attacker nothing; a loss of 3 means the output is $e^3 \approx 20$ times more likely with the record, which is strong evidence.

Why a ratio and not a difference? A change from 0.50 to 0.51 and one from 0 to 0.01 are the same difference, but the second creates an output possible only when the person is present, which identifies them with certainty. Only a ratio, infinite here, catches it.

Differential privacy bounds this ratio. M is (ε, δ)**-differentially private** if for every pair of neighbours and every set S of outputs,

$$\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{18.2}$$

The probabilities are over the mechanism's randomness, with the data fixed. With $\delta = 0$ this is **pure DP**: no event becomes more than e^{ε} times more likely, which is the same as saying the privacy loss of every single output is at most ε. At $\varepsilon = 0.1$ an event of probability 0.5 in one world has probability at most $0.5e^{0.1} = 0.553$ in the other, so the release barely moves anyone's odds.

Pure DP is too strict for some useful mechanisms. With Gaussian noise, far-out outputs are much more likely in one world than the other, because the normal density falls off faster than any fixed ratio can follow. Those outputs are astronomically rare, and δ is the allowance for them: the ratio bound may fail, but only on outputs whose total excess probability is at most δ. This is **approximate DP**.

The way δ is counted matters in practice. The definition allows each output's probability p in one world to be up to e^{ε} times its probability q in the other, and only the excess, $p - e^{\varepsilon}q$, has to be paid for by δ. An output with loss L has $p = e^{L}q$, so the excess is a fraction $1 - e^{\varepsilon - L}$ of its probability: close to zero when L is just above ε, close to all of it when L is huge. The smallest valid δ, computed this way, is the **hockey-stick divergence** between the two output distributions. So δ is not "the probability that privacy fails": that probability is larger, because it counts outputs just over the line in full. For a count released with Gaussian noise of standard deviation 2 at $\varepsilon = 1$, that probability is 4.006×10^{-2} while the true δ is 6.830×10^{-3}, nearly six times smaller (Figure 18.1). If you calibrate with the tail probability, you add noise you didn't need.

Ask for a larger ε and fewer outputs exceed it, so the required δ shrinks: for the Gaussian count above, δ falls from 6.830×10^{-3} at $\varepsilon = 1$ to 3.401×10^{-10} at $\varepsilon = 3$. A mechanism therefore has a whole **privacy curve** $\delta(\varepsilon)$, and any single pair you quote is one point on it. Report the

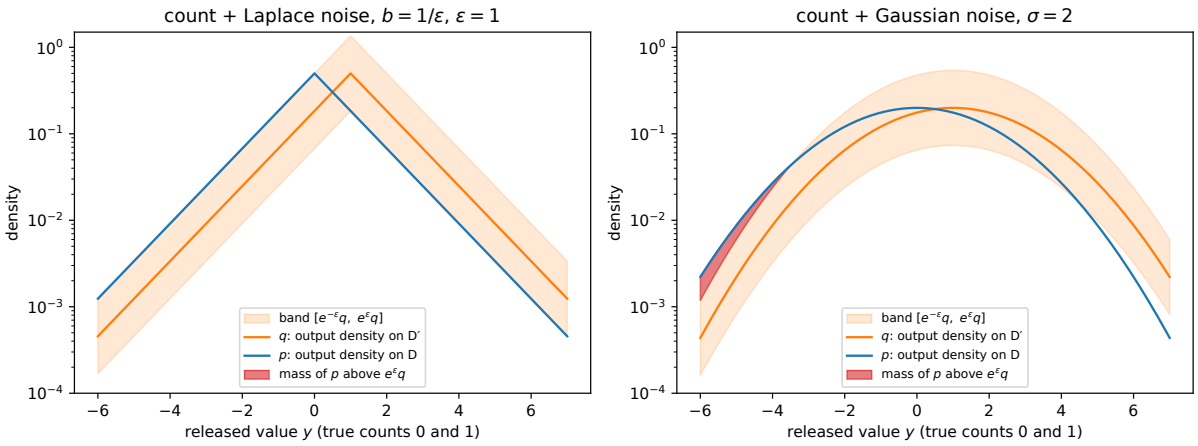


Figure 18.1.: The definition as a picture, for a count released with noise at $\varepsilon = 1$. The shaded band runs from $e^{-\varepsilon}$ to e^{ε} times the output density on D' (orange). Left: with Laplace noise of scale $1/\varepsilon$, the density on D (blue) stays inside the band, so the release is pure DP. Right: with Gaussian noise of standard deviation 2, it leaves the band in the tail, and the red mass is what δ must cover.

mechanism's parameters along with the pair, because the accountants of Part 4 combine curves, not points.

Never check approximate DP output by output: the slack is shared across a whole event. Two datasets whose outputs are spread over 1,000 disjoint values pass that check with $\delta = 0.0010$, yet one look at any output reveals the dataset, and the true δ is 1.0000.

18.4. Randomized response

The oldest mechanism shows how privacy and useful statistics coexist. To learn what fraction of people did something embarrassing, ask each person to flip a coin in private: on heads, answer truthfully; on tails, flip again and answer "yes" on heads, "no" on tails. Any single "yes" might be a coin, so it proves nothing about the person. But the coins behave predictably over many people, so the fraction can be recovered.

This scheme reports the truth with probability $t = 0.75$. A "yes" is then $0.75/0.25 = 3$ times more likely from someone whose true answer is yes than from someone whose answer is no, and no report has a larger ratio, so this is pure DP with $\varepsilon = \ln 3 = 1.0986$. In general, **randomized response** at level ε tells the truth with probability $t = e^{\varepsilon}/(1 + e^{\varepsilon})$, chosen so that the ratio is exactly e^{ε}.

The lying pulls the share $\bar{y}$ of "yes" reports towards one half by a known amount, and undoing that pull gives an unbiased estimate of the true fraction π,

$$\hat{\pi} = \frac{\bar{y} - (1 - t)}{2t - 1}, \qquad \mathrm{sd}(\hat{\pi}) \approx \frac{1}{\varepsilon\sqrt{n}} \ \text{ for small } \varepsilon. \tag{18.3}$$

The cost of privacy is visible in the denominator. When ε is small, t is close to one half, the reports are nearly pure coin flips, and $2t - 1$ is small; dividing by it blows up the noise. Halving ε

roughly doubles the error, and since error falls like $1/\sqrt{n}$, you need four times as many people to win it back.

In a survey of 1,000 people with true fraction 0.3, the estimate's standard deviation is 0.0274 at $\varepsilon = \ln 3$, about twice the 0.0145 of an ordinary survey of that size. At $\varepsilon = 0.5$ it is 0.0626, and at $\varepsilon = 0.1$ it is 0.316, larger than the fraction being measured (Figure 18.2).

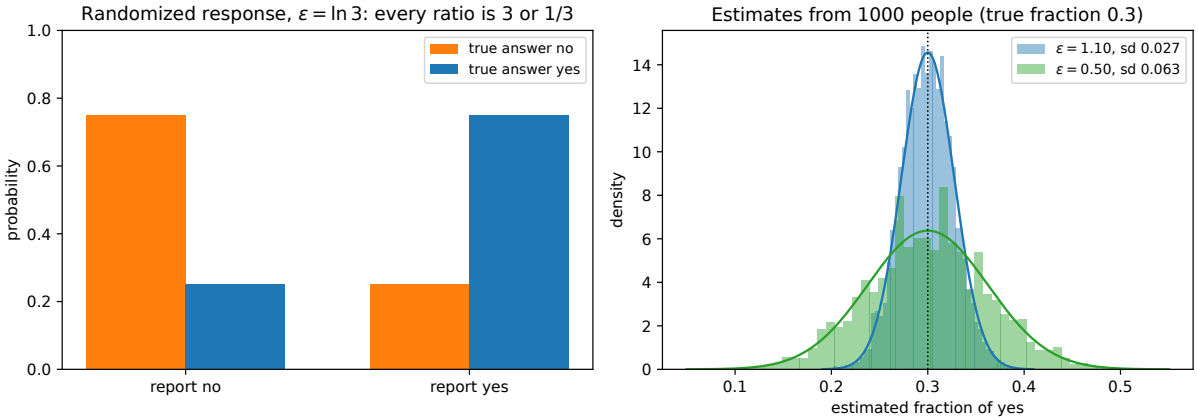


Figure 18.2.: Left: randomized response at $\varepsilon = \ln 3$ as output probabilities; each report is 3 times more likely under one true answer than under the other. Right: estimates of the fraction of "yes" from 4,000 simulated surveys of 1,000 people with true fraction 0.3, at $\varepsilon = \ln 3$ and $\varepsilon = 0.5$, with the normal curves the standard-deviation formula predicts.

Because each person randomizes before their answer leaves them, nobody needs to be trusted. That is the **local** model, and the large error is its price; a trusted curator who noises the aggregate once does far better, and Local, shuffle and distributed trust models in the full edition compares the options.

18.5. What ε means for an attacker

Whatever an attacker does, they end by deciding "in" or "out", and the set of outputs on which they say "in" is an event. So the definition bounds how much more often they can say "in" when it is true than when it is false.

Write the attacker's **true positive rate (TPR)** for how often they say "in" when the person is in, and their **false positive rate (FPR)** for how often they say "in" when the person is out. Applying Equation 18.2 to their "in" event gives, for every possible attack,

$$\text{TPR} \le e^{\varepsilon}\,\text{FPR} + \delta. \tag{18.4}$$

This is the form to remember: an attacker who must rarely accuse wrongly keeps the FPR small, and the bound then caps the TPR at a small multiple of it. At $\varepsilon = 1$ and $\delta = 10^{-5}$, no attack reaches a TPR above 0.002728 at an FPR of 10^{-3}.

Read backwards, the same inequality turns an attack into an audit. If someone claims a model is $(2, 10^{-5})$-DP and you run an attack that reaches a TPR of 0.05 at an FPR of 0.001, the claim

cannot be right: the cap at that FPR is 0.0074. Solving for ε shows the attack proves $\varepsilon \geq 3.91$ (Section 18.8). Audits use **canaries**, records planted to be as easy to detect as possible, because an ordinary record would give a much weaker bound.

Two other readings follow from the same bound. Averaged over a fair coin for "in" or "out", an attacker's **advantage**, TPR − FPR, is at most $\tanh(\varepsilon/2)$ under pure DP, about $\varepsilon/2$ when ε is small. And because one output can change the likelihood ratio by at most e^{ε}, it can multiply anyone's odds that the person is in by at most e^{ε}, whatever they believed before (Table 18.1).

Table 18.1.: The largest membership advantage, and the largest posterior probability after one output of an ε-DP mechanism for three prior beliefs.

ε	Advantage bound	Prior 0.01	Prior 0.1	Prior 0.5
0.1	0.0500	0.0110	0.1094	0.5250
1	0.4621	0.0267	0.2320	0.7311
$\ln 3$	0.5000	0.0294	0.2500	0.7500

These readings also show where the guarantee weakens. At $\varepsilon = 5$ the advantage bound is 0.9866, an attacker right 99.3% of the time: a model trained at $\varepsilon = 8$ may resist real attacks, but the guarantee no longer promises it. And the release bounds the change in a belief, not the belief: an adversary already 99% sure from other sources stays about that sure.

18.6. How small δ must be

A deliberately terrible mechanism shows how small δ must be. Publish each record, in full, with probability δ. Any one person is exposed only with probability δ, and otherwise the output ignores them, so this is $(0, \delta)$-DP.

But it publishes $n\delta$ records on average, and at least one with probability about $1 - e^{-n\delta}$. With $n = 10{,}000$ and $\delta = 1/n$, someone's record is published verbatim with probability 0.6321. So δ must be much smaller than $1/n$, where n counts people, not records. The common choice of 10^{-5} for 60,000 training examples is only just below $1/n$, and 10 million users need δ well below 10^{-7}.

This usually costs little: reasonable mechanisms, like the Gaussian count, spread δ over many outputs of moderate loss, so their privacy curve falls steeply and a slightly larger ε buys a much smaller δ.

18.7. What differential privacy does not protect

The guarantee compares the world with your record to the world without it, so anything learned in both worlds is outside it. If a study shows that smokers often get cancer, beliefs about every smoker change, including those never in the data; that is the study doing its job.

It also treats records as separate. If five people in a household always answer alike, an attacker who learns one answer learns all five, and hiding the household means hiding five records at once.

For a count released with Laplace noise at $\varepsilon = 1$ per record, the household's privacy loss can reach 5.0000, and an attacker asking whether the household took part can be right with probability up to 0.959. Match the unit of privacy to the unit of correlation: protect households or users, not rows.

Finally, a DP proof covers the algorithm, not the code: floating-point noise samplers can leak through their low-order bits (Discrete and floating-point-safe mechanisms in the full edition), and timing can leak. The implementation needs its own review.

18.8. Code

This snippet turns a claimed (ε, δ) into what an attacker can achieve, and a measured attack into a lower bound on ε:

```
from math import exp, log, sqrt


def advantage_bound(eps: float, delta: float = 0.0) -> float:
    """The largest membership advantage, TPR - FPR, against any (eps, delta)-DP mechanism."""
    return (exp(eps) - 1 + 2 * delta) / (exp(eps) + 1)


def tpr_cap(eps: float, delta: float, fpr: float) -> float:
    """The largest true positive rate any attack can reach at a given false positive rate."""
    return min(1.0, exp(eps) * fpr + delta)


def epsilon_lower_bound(tpr: float, fpr: float, delta: float) -> float:
    """The smallest eps consistent with an attack measured at (fpr, tpr)."""
    return log((tpr - delta) / fpr)


def posterior_bound(eps: float, prior: float) -> float:
    """The largest posterior probability after one output of an eps-DP mechanism."""
    return exp(eps) * prior / (exp(eps) * prior + 1 - prior)


def rr_sd(eps: float, n: int) -> float:
    """Standard deviation of the randomized-response estimate of a fraction from n people."""
    return exp(eps / 2) / ((exp(eps) - 1) * sqrt(n))


def exposure(delta: float, n: int) -> float:
    """Probability that 'publish each record with probability delta' exposes someone."""
    return 1 - (1 - delta) ** n
```

```
print(f"{advantage_bound(1.0):.4f}")  # 0.4621
print(f"{tpr_cap(1.0, 1e-5, 1e-3):.6f}")  # 0.002728
print(f"{tpr_cap(2.0, 1e-5, 1e-3):.4f}")  # 0.0074
print(f"{epsilon_lower_bound(0.05, 1e-3, 1e-5):.2f}")  # 3.91
print(f"{posterior_bound(1.0, 0.01):.4f}")  # 0.0267
print(f"{rr_sd(log(3), 1000):.4f}")  # 0.0274
print(f"{exposure(1e-4, 10_000):.4f}")  # 0.6321
```

A real audit must also account for the uncertainty in measured rates (Membership inference). The full edition's script checks every number in the chapter:

```
uv run python code/part03-dp-foundations/definition.py
```

18.9. Cheat sheet

Table 18.2.: The results of this chapter.

Result	Formula	Notes
(ε, δ)-DP	$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta$ for all neighbours, all S	State the neighbouring relation and unit of privacy
Add/remove vs replace	ε under add/remove gives 2ε under replace	Only add/remove hides participation
Randomized response	Truth with probability $e^{\varepsilon}/(1+e^{\varepsilon})$; error $\approx 1/(\varepsilon\sqrt{n})$	Local: no trusted collector
Attack cap	$\mathrm{TPR} \leq e^{\varepsilon}\,\mathrm{FPR} + \delta$	Audit: $\varepsilon \geq \ln((\mathrm{TPR} - \delta)/\mathrm{FPR})$
Posterior	Odds multiplied by at most e^{ε}	Bounds the change in belief, not the belief
Choosing δ	$\delta \ll 1/n$, n people	$\delta = 1/n$ can expose someone with probability 0.63

18.10. Further reading

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 2: the definition, randomized response, what DP does and does not promise.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015): the hypothesis-testing reading behind the attack cap and auditing.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

19. Post-processing, group privacy and composition

19.1. Why this matters

A private mechanism is never the whole system. You round its output and train models on it, your users each contributed fifty records, and an analyst asks a hundred queries or a training run takes thousands of noisy steps. Which of these cost privacy, and how do the costs add up? Three properties of differential privacy (DP) answer that without a new proof for every pipeline: you prove each building block once, then add up the costs.

All three follow from one quantity, the **privacy loss** of The definition of differential privacy in the full edition: the log of how much more likely an output is when the data are D than when they are a neighbouring dataset D', one record different. Pure ε-DP keeps it between $-\varepsilon$ and ε for every output. The one idea of this chapter is that **privacy losses add along a path of datasets or across a sequence of mechanisms, and only average under processing.** Each section applies it to one situation.

19.2. Post-processing

Suppose you transform a private output: round it, threshold it, fit a model to it. Several raw outputs can lead to the same processed one, and the probability of the processed output is the sum of theirs. So its likelihood ratio between the two datasets is a weighted average of their ratios. If every raw ratio lies between $e^{-\varepsilon}$ and e^{ε}, an average of them does too. Processing pools evidence, and pooling can only dilute it. This **data processing inequality** gives the rule: if M is (ε, δ)-DP and f is any function that does not read the data, deterministic or with its own coins, then publishing $f(M(D))$ is (ε, δ)-DP too. Its whole privacy curve $\delta(\varepsilon)$ lies on or below that of M.

Collect a randomized-response answer at $\varepsilon_0 = \ln 3$, which tells the truth with probability $\frac{3}{4}$, so each report is 3 times more likely under one true answer than under the other. Before publishing, flip it with probability $\frac{1}{4}$ using a fresh coin. A published 1 can come from a kept 1, whose ratio is 3, or from a flipped 0, whose ratio is $\frac{1}{3}$, and the two paths carry equal weight. The published 1 therefore has ratio $\frac{1}{2}(3 + \frac{1}{3}) = \frac{5}{3}$: it has probability 0.625 when the true bit is 1 and 0.375 when it is 0. The release is $\ln \frac{5}{3} = 0.5108$-DP, well below the original $\ln 3 = 1.0986$, because the coin mixed outputs that pointed in opposite directions.

Processing you do for utility rarely mixes like that. Rounding a count with Laplace noise of scale 1 to an integer leaves its pure ε at 1.0000: a rounding cell that lies entirely below both possible counts holds outputs that all have the same loss, and an average of equal ratios is that ratio. Keeping only whether the noisy count exceeds $\frac{1}{2}$ merges outputs with losses of both signs, and

lowers ε to 0.8318 (Figure 19.1). So report the guarantee of M and treat any gain from processing as a bonus you don't count on.

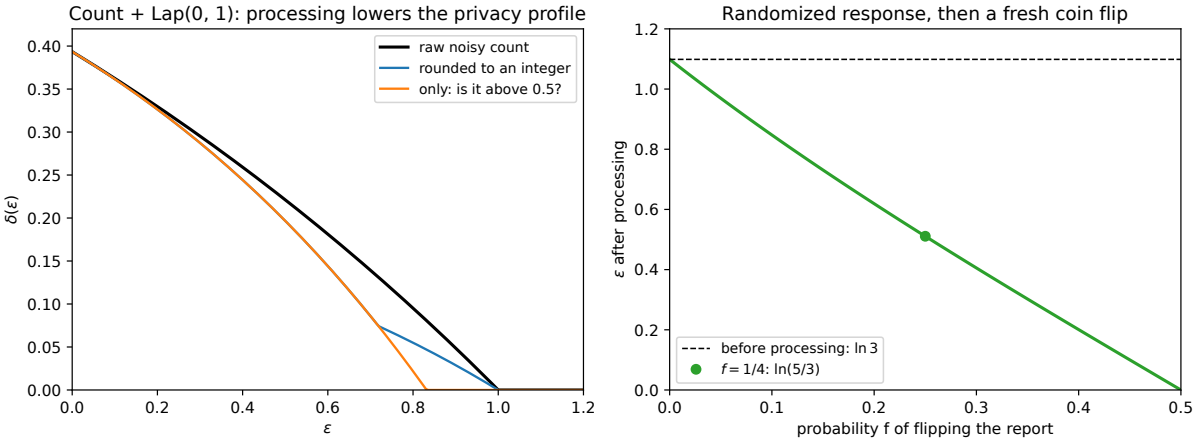


Figure 19.1.: Left: the privacy curve $\delta(\varepsilon)$ of a count with Laplace noise of scale 1 (black), after rounding (blue), and after keeping only whether it exceeds $\frac{1}{2}$ (orange); processing only lowers the curve. Right: randomized response at $\varepsilon_0 = \ln 3$ followed by a fresh coin that flips the answer with probability f; the dot is $f = \frac{1}{4}$, where $\varepsilon = \ln \frac{5}{3}$.

The argument has one condition. The averaging used the same processing under D and under D', and that holds only if f never looks at the data. "Take the noisy top category, then report its exact count" fails it: the second step reads the data, so it is a new mechanism, and a non-private one. Within it, the rule is powerful: a model trained on DP statistics, synthetic data from a DP model and a dashboard are all post-processing, and so is every attack of Threat models and attacks, since an attacker's guess is a function of the output, so the definition's bounds hold even against attacks invented later. That is why a DP system privatizes at one narrow point, such as noisy counts or noisy gradients, and treats everything after it as public.

19.3. Group privacy

The definition protects one record, but a user may own fifty messages, and a household shares an address and an income. To cover datasets that differ in k records, walk from D to the other dataset one record at a time, through k neighbouring pairs. Each step can multiply an event's probability by at most e^{ε}, so k steps multiply it by at most $e^{k\varepsilon}$: pure ε-DP becomes $k\varepsilon$-DP for the group.

The slack grows faster. Each step adds its own δ, and every later step multiplies everything accumulated so far, including the earlier δs, by e^{ε} again. So the δs pile up in a geometric sum. An (ε, δ)-DP mechanism is, for datasets k records apart,

$$\Big(k\varepsilon,\ \delta_k\Big)\text{-DP}, \qquad \delta_k = \delta \sum_{j=0}^{k-1} e^{j\varepsilon} = \delta\, \frac{e^{k\varepsilon} - 1}{e^{\varepsilon} - 1}. \tag{19.1}$$

Some mechanisms attain this bound exactly, so no better rule holds for every (ε, δ)-DP mechanism.

The price is steep. A per-record guarantee of $(0.5, 10^{-5})$ gives a user with 10 records $(5, 2.272 \times 10^{-3})$. A particular mechanism can do better than the general bound: for a count with Gaussian noise, ten records just shift the count by ten instead of one, and its exact guarantee for that user is $(5, 7.592 \times 10^{-4})$. That is still weak. At $\varepsilon = 1$, a group of 10 multiplies δ by 12818.3, and δ had to be far below one over the number of people to begin with (The definition of differential privacy in the full edition), so the group guarantee is often vacuous. Pure DP fares no better: 0.1-DP per record is 10-DP for a user with 100 records.

So if you care about users, don't patch a per-record guarantee with group privacy: define neighbours as datasets that differ in one user's records, bound each user's contribution, and prove DP at that level directly. Group privacy is the fallback when you cannot.

19.4. Sequential composition

Now run several mechanisms on the same data, each with fresh noise, and let the adversary see the whole **transcript** of outputs. Because the noise draws are independent, the probability of a transcript is the product of the probabilities of its parts, so its privacy loss, a logarithm, is the sum of theirs. So if each M_i is $(\varepsilon_i, \delta_i)$-DP, releasing all their outputs is

$$\Big(\sum_{i=1}^{k} \varepsilon_i,\ \sum_{i=1}^{k} \delta_i\Big)\text{-DP}. \tag{19.2}$$

This is **basic composition**. Unlike group privacy, the δs add plainly, because every mechanism compares the same pair of datasets: there is no chain of steps to multiply them through.

The argument only needed each mechanism to be private whatever the earlier outputs were, so the rule also covers **adaptive** composition, where an analyst picks each query after reading the last answer, as long as each query's budget $(\varepsilon_i, \delta_i)$ is fixed in advance. If the budgets themselves depend on the answers ("keep going until the estimate looks good"), Equation 19.2 no longer applies in general; that needs privacy filters, covered in Advanced and optimal composition in the full edition. Pure DP is the exception: each step's privacy loss is at most its ε_i at every output, so the losses still add up to at most $\sum_i \varepsilon_i$ whatever order the budgets were chosen in.

Basic composition is the true worst case, but that case is rare. Ask one person the same yes/no question k times, each with fresh randomized response at $\varepsilon_0 = \ln 3$. The transcript in which every answer is truthful has loss exactly $k \ln 3$, so no smaller pure ε holds. But that transcript has probability $(\frac{3}{4})^k$. Typical transcripts mix truthful answers, which add $\ln 3$ to the loss, with lies, which subtract it, and the two partly cancel: the mean loss is half the bound (Figure 19.2). A sum of k independent bounded terms spreads only like $\sqrt{k}$ around its mean, so if you allow a small δ to write off the rare extreme transcripts, the ε you need is the mean plus a margin that grows like $\sqrt{k}$. The mean per answer is $\varepsilon_0 \tanh(\varepsilon_0/2)$, about $\varepsilon_0^2/2$ when ε_0 is small, so for small per-step budgets the margin dominates and ε grows roughly like $\sqrt{k}$ instead of k. At $\varepsilon_0 = 0.1$ and $k = 1000$ answers, basic composition gives 100, while the exact ε at $\delta = 10^{-6}$ is 19.345. Ten counts with Gaussian noise, each $(0.5, 10^{-5})$-DP, cost $(5, 10^{-4})$ by basic composition, while their exact ε at that δ is 1.504.

So basic composition suits a handful of releases. It already overstates the ten Gaussian counts by a factor of 3.3, and for thousands of training steps it is useless: you need the accountants

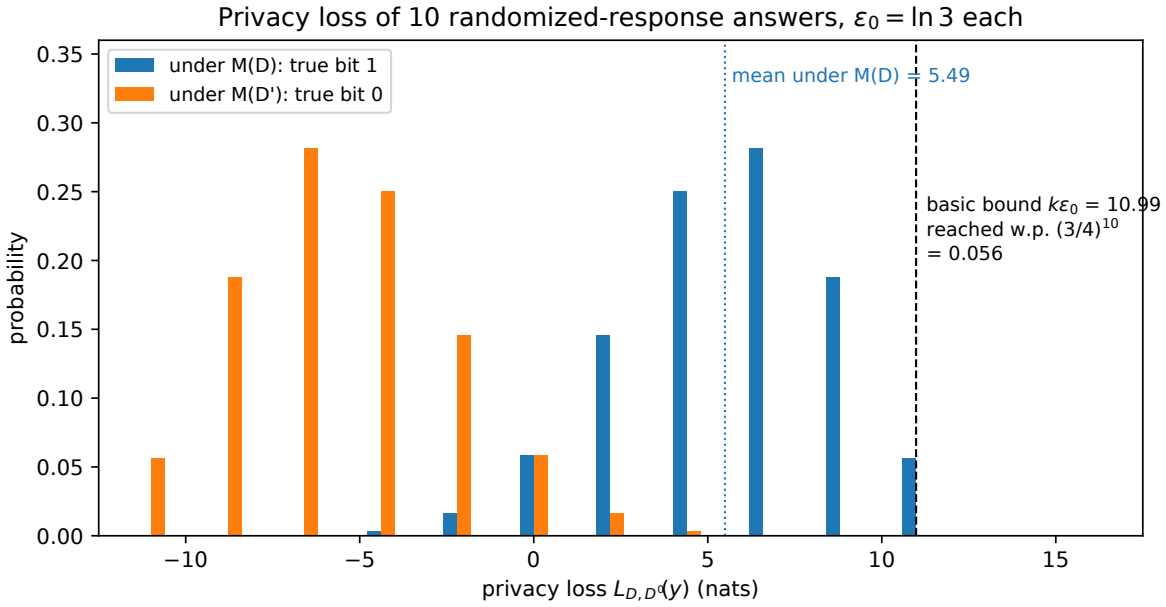


Figure 19.2.: Distribution of the privacy loss of 10 randomized-response answers at $\varepsilon_0 = \ln 3$ each, when the true bit is 1 (blue) or 0 (orange). The basic bound $10 \ln 3$ is reached, but only with probability $(3/4)^{10} = 0.056$; the mean under the first dataset is half of it.

of Advanced and optimal composition in the full edition and Part 4, which exploit exactly this cancellation.

19.5. Parallel composition

If each mechanism sees a different, disjoint part of the data, one person's record lands in only one part. The other parts get identical inputs in both worlds, so their outputs contribute zero privacy loss, and only the part that changed costs anything. Split the universe of possible records into disjoint parts by a rule fixed in advance, run M_i on the records in part i with independent noise, and suppose each M_i is $(\varepsilon_i, \delta_i)$-DP under add/remove, where neighbours differ by one record added or removed. Then the whole release is

$$\left(\max_i \varepsilon_i,\ \max_i \delta_i\right)\text{-DP under add/remove.} \tag{19.3}$$

This is why a histogram costs one ε, not one per bin. Laplace noise of scale $1/\varepsilon$ on each bin is ε-DP however many bins there are; for two bins the full edition's script finds exactly 1.0000 at $\varepsilon = 1$. The same reasoning is why local randomized response stays $\ln 3$-DP however many people answer: each report depends only on its owner's answer, so changing one person's answer changes one report. That is a replace guarantee; under add/remove, the number of reports itself reveals who took part.

The neighbouring relation matters here. Under replace, where one record's value is changed, a record can move from one bin to another, and then two parts see a difference: the two-bin histogram becomes 2.0000-DP. That is why a histogram's sensitivity is quoted as 2 under replace

(The Laplace and Gaussian mechanisms in the full edition). When each part is a fixed slot, as each person is in local randomized response, a replacement changes one slot only and the maximum applies again.

The argument also needs the partition to be fixed before the data are seen. "The top 10% of earners and the rest" moves its boundary when one high earner is added, which can shift other records across it and change both parts, so more than one mechanism sees a difference. Use fixed bins, or treat a data-dependent split as a mechanism of its own.

19.6. What composes and what doesn't

Each property rests on an assumption that its argument used: post-processing needs the processing to ignore the data, composition needs independent noise and the same neighbouring relation at every step, and both need budgets fixed in advance. Most real failures break one of them, and badly.

Shared noise is the clearest case. Release the number of smokers and the number of smokers aged 40 or over, adding the same Laplace draw Z to both. Each release alone is 1-DP. But subtracting one from the other cancels Z and leaves the exact number of smokers under 40, which changes when Alice, a smoker aged 30, is removed. Composition assumed the transcript's probability is a product of independent parts; with one shared draw it is not, and one release can undo the other's noise. In 200,000 simulated runs, an attacker who subtracts the two releases is right about Alice every time, an accuracy of 1.0000. With fresh noise for each count the same attack reaches 0.6206, and no attack can beat 0.6967.

In code, the bug hides in two mechanisms seeded with the same random seed, a cached noise value re-added to a related query, or noise reused across the shards of a distributed system. Returning the same noisy answer to a repeated identical query is fine, because that is post-processing of one release. An exact statistic published elsewhere is the same mistake in another form: it can be differenced against your DP release just as the two counts were, and the guarantee covers only what your mechanism released. Finally, a guarantee proved under add/remove costs up to twice as much under replace, because a replacement is a removal plus an addition, so mixing the two relations inside one accountant is a factor-2 bug.

19.7. Code

Group privacy is where hand calculations go wrong most often, because δ does not simply scale by k. This snippet computes the three budget rules of this chapter:

```
from math import exp


def group_privacy(eps: float, delta: float, k: int) -> tuple[float, float]:
    """The guarantee for a group of k records, from an (eps, delta) guarantee per record."""
    delta_k = delta * sum(exp(j * eps) for j in range(k))
    return k * eps, delta_k
```

```python
def basic_composition(budgets: list[tuple[float, float]]) -> tuple[float, float]:
    """The guarantee of several mechanisms run on the same data with fresh noise."""
    return sum(e for e, _ in budgets), sum(d for _, d in budgets)


def parallel_composition(budgets: list[tuple[float, float]]) -> tuple[float, float]:
    """The guarantee of mechanisms run on disjoint parts of the data, under add/remove."""
    return max(e for e, _ in budgets), max(d for _, d in budgets)


eps_k, delta_k = group_privacy(0.5, 1e-5, 10)
print(f"{eps_k:.1f} {delta_k:.3e}")  # 5.0 2.272e-03
print(f"{group_privacy(0.5, 1e-5, 2)[1]:.3e}")  # 2.649e-05
print(f"{group_privacy(1.0, 1.0, 10)[1]:.1f}")  # 12818.3
print(basic_composition([(0.5, 1e-5)] * 10))  # (5.0, 0.0001)
print(parallel_composition([(0.5, 0.0)] * 50))  # (0.5, 0.0)
```

The third line is the factor by which δ grows for a group of 10 at $\varepsilon = 1$. Basic composition here is a valid upper bound only; for the exact cost of many Gaussian releases, use an accountant. The full edition's script checks every number in this chapter by exact computation and simulation:

```
uv run python code/part03-dp-foundations/properties.py
```

19.8. Cheat sheet

Table 19.1.: The results of this chapter.

Property	Rule	Notes
Post-processing	$f \circ M$ is (ε, δ)-DP	Only if f never reads the data; can lower ε, never raise it
Group privacy	$(k\varepsilon,\ \delta\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1})$ for k records	Tight in general; prefer user-level neighbours
Basic composition	$(\sum \varepsilon_i, \sum \delta_i)$	Also adaptive, with budgets fixed in advance; loose for many releases
Parallel composition	$(\max \varepsilon_i, \max \delta_i)$ under add/remove	Up to twice that under replace; partition fixed in advance
Shared noise	No guarantee at all	Every mechanism needs fresh noise

Rules of thumb

- Privatize once, as early as possible; everything after it is free.

- For users with many records, define neighbours at the user level instead of relying on group privacy.
- Use basic composition for a handful of releases; for many, use an accountant.

19.9. Further reading

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014): post-processing, group privacy and basic composition (Proposition 2.1, Theorems 2.2 and 3.16), the standard reference.
- McSherry, "Privacy Integrated Queries" (SIGMOD 2009): parallel composition, and a query system built on post-processing and composition as its only rules.
- Near & Abuah, *Programming Differential Privacy*, the chapters on the properties of DP: sequential and parallel composition and post-processing, with code.
- Rogers, Roth, Ullman & Vadhan, "Privacy Odometers and Filters: Pay-as-you-Go Composition" (NeurIPS 2016): what changes when the budgets themselves are chosen adaptively.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

20. The Laplace and Gaussian mechanisms

20.1. Why this matters

You want to publish a statistic with a differential privacy (DP) guarantee: a count, a mean, a histogram, or a gradient during training. The definition of differential privacy in the full edition says what the release must satisfy: on two **neighbouring** datasets D and D', which differ in one person's record, every set S of outputs must have $\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta$. Almost every DP system builds such a release the same way: compute the exact answer $f(D)$, add fresh random noise Z, and publish

$$M(D) = f(D) + Z. \tag{20.1}$$

The one idea of this chapter is what this does to the **privacy loss**, the log of how much more likely an output is under D than under D'. On the two datasets the mechanism places the same noise around two different centres, $f(D)$ and $f(D')$. So the privacy loss is the log-ratio of the noise density and a copy of itself shifted by the change one record makes, and it depends on two things: how big that shift can be, and how the log of the noise density falls off. The Laplace density's log falls off in a straight line, which bounds the loss everywhere and gives pure DP. The normal density's log falls off like a parabola, which leaves the loss unbounded but large only on rare outputs, and gives (ε, δ)-DP. The sections follow the decisions in order: sensitivity, which noise, and how much.

20.2. Sensitivity

To hide one record, the noise must cover the largest change one record can make to the answer, its **sensitivity**. Because the guarantee must hold for every pair of neighbours, it is a worst case over every dataset the domain allows, not a property of your data. So you can compute it before looking at the data, but it can be infinite: a sum of unbounded numbers changes by as much as one record likes, so you must clip every value to a fixed range first.

For a vector of answers, the two noises measure a change differently. The ℓ_1 norm adds up the absolute changes over all coordinates; the ℓ_2 norm is their straight-line length, the square root of the sum of squares. The ℓ_p sensitivity is the largest change between neighbours,

$$\Delta_p f = \max_{D \simeq D'} \left\| f(D) - f(D') \right\|_p, \qquad p \in \{1, 2\}. \tag{20.2}$$

The two agree when one record moves a single coordinate, and they are furthest apart when it moves all d coordinates by the same amount: a change of 1 in each of d coordinates has ℓ_1 norm d but ℓ_2 norm only $\sqrt{d}$. That gap decides, in Section 20.5, which noise to use.

The answer also depends on what "differ in one record" means. Under **add/remove** one dataset has an extra record, so its size is private. Under **replace** one record is swapped for another, so the size n is fixed and public. Each entry of Table 20.1 is the effect of the single worst record. A histogram has sensitivity 1 under add/remove, because the new record raises one bin by 1, but 2 under replace, because the swapped record leaves one bin and joins another. The count of all records has sensitivity 0 under replace, because n never changes; if the size of your dataset is itself sensitive, use add/remove. And a record at the top of the range raises every one of d threshold counts, which makes that query dense.

Table 20.1.: Sensitivities of the standard queries under the two neighbouring relations.

Query	Add/remove Δ_1	Add/remove Δ_2	Replace Δ_1	Replace Δ_2
Count $\lvert D\rvert$	1	1	0	0
Sum, records in $[0, B]$	B	B	B	B
Mean, records in $[0, B]$, n public	—	—	B/n	B/n
Histogram with k bins	1	1	2	$\sqrt{2}$
d threshold counts $\#\{x \geq j\}$, $j = 1..d$	d	$\sqrt{d}$	d	$\sqrt{d}$

For example, a study releases the mean age of $n = 1000$ patients, with ages clipped to $[0, 100]$, under replace neighbours. One patient's age can move the sum by at most 100, and the mean by $100/1000 = 0.1$: a tenth of a year, however old they are. Since the noise is proportional to the sensitivity, the same noise on a histogram gives an ε twice as large under replace, so always state the relation with ε. Computing the sensitivity from your data, say from the largest value present, leaks information (Beyond global sensitivity in the full edition).

20.3. The Laplace mechanism

The **Laplace distribution** $\mathrm{Lap}(b)$ with scale b has density $\frac{1}{2b}e^{-|z|/b}$, a two-sided exponential. Its log falls off in a straight line with slope $1/b$ on each side, so sliding the density over by a distance Δ changes its log at any point by at most Δ/b. Hence no output can be more than $e^{\Delta/b}$ times as likely under one dataset as under its neighbour. With independent noise on d coordinates the log-ratios of the coordinates add, each bounded by its own shift divided by b, so the total is bounded by the ℓ_1 norm of the shift. Setting the scale so that the worst shift gives exactly ε, adding independent $\mathrm{Lap}(b)$ noise to each coordinate with

$$b = \frac{\Delta_1 f}{\varepsilon} \tag{20.3}$$

is ε-DP. No smaller scale works, and the worst case is not rare: for a count, every output below both true answers has privacy loss exactly ε, and half the outputs land there (Figure 20.1, left).

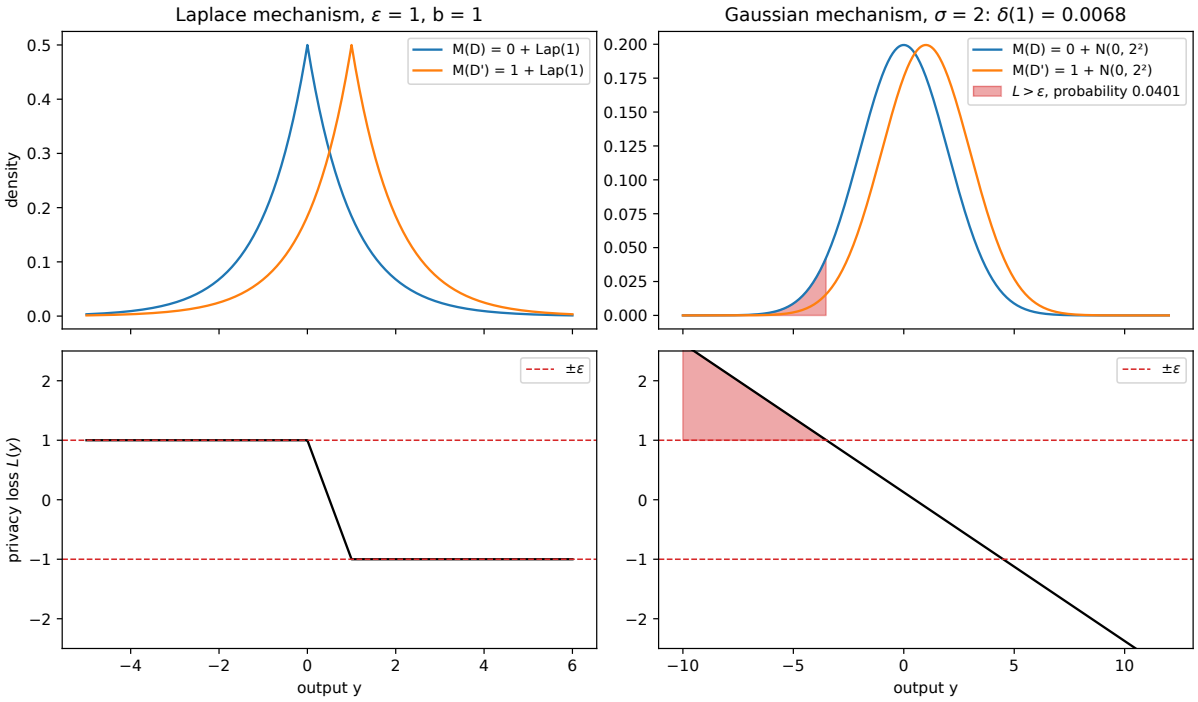


Figure 20.1.: Left column: a count released with Lap(1) noise on neighbouring datasets with true counts 0 and 1, and the privacy loss $L(y)$, the log of how much more likely the output y is under one dataset than the other. It equals $+\varepsilon$ for every $y \leq 0$ and $-\varepsilon$ for every $y \geq 1$, never more. Right column: the same count with $\mathcal{N}(0, 2^2)$ noise. The privacy loss is a straight line, and exceeds $\varepsilon = 1$ on the shaded tail, which has probability 0.0401 under $M(D)$. The exact $\delta(1) = 0.0068$ is smaller, because the bound subtracts e^{ε} times the mass of $M(D')$ on the same tail.

The error is the noise itself: b on average in each coordinate. Its exponential tail puts one coordinate within $b\ln(1/\beta)$ with probability $1-\beta$; giving each of d coordinates a failure probability of β/d puts all of them within $b\ln(d/\beta)$. Say 1,234 of a hospital's 10,000 patients have diabetes. Released with $\varepsilon = 0.1$ under add/remove, the sensitivity is 1, so the noise is Lap(10): the error is 10 on average and below $10\ln 20 = 29.96$ with probability 0.95, a relative error of 2.4%. A histogram of diagnoses with 20 bins at $\varepsilon = 1$ gets Lap(1) in each bin, and every bin is within $\ln(20/0.05) = 5.99$ of its true count with probability 0.95.

Notice what the scale does not contain: the size of the data. One record moves a count by 1 whether the database holds a hundred records or a hundred million, so the noise is the same, and the relative error of a count of n is about $1/(n\varepsilon)$. Large populations are cheap to protect and small subgroups are expensive. The flip side is that the sensitivity must count every number one record can move: ten counts that each get Lap($1/\varepsilon$), and that one record can all change, have $\Delta_1 = 10$ and form a 10ε-DP release. Drawing the noise is another matter: textbook floating-point Laplace samplers break the guarantee through their low-order bits, as Discrete and floating-point-safe mechanisms in the full edition shows, so use a vetted DP library rather than `numpy.random.laplace`.

20.4. The Gaussian mechanism

Gaussian noise, the other natural choice, is calibrated to the ℓ_2 sensitivity. Independent normal coordinates with equal variance look the same in every direction, so you can rotate the axes until the shift points along one of them; the other coordinates then carry no evidence, and only the shift's straight-line length matters. The price is the parabola. The log of a normal density falls off like $-z^2$, so the log-ratio of two shifted copies is a straight line in the output, and grows without limit far in the tail (Figure 20.1, right). Gaussian noise therefore cannot give pure DP. It gives (ε, δ)-DP, where δ pays for the rare outputs whose loss exceeds ε.

As the definition in the full edition showed, δ is not the probability of those outputs: only their excess over e^ε times their probability on the neighbour has to be paid for. For two normal distributions both terms are normal tail probabilities, which gives the exact answer in closed form (Balle & Wang 2018). Only the ratio $\mu = \Delta_2 f/\sigma$ enters, the number of noise standard deviations between the two answers, so doubling both the sensitivity and the noise changes nothing. Adding $\mathcal{N}(0, \sigma^2)$ noise to each coordinate is (ε, δ)-DP exactly when $\delta \geq \delta_\mu(\varepsilon)$, where

$$\delta_\mu(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^\varepsilon\, \Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) \tag{20.4}$$

and Φ is the standard normal cumulative distribution function. The first term is the probability, on one dataset, of the outputs whose loss exceeds ε; the second subtracts e^ε times their probability on the neighbour. Less noise means a larger μ and a larger $\delta_\mu(\varepsilon)$, so exactly one noise level hits a target δ, and you find it by bisection (Section 20.6). That is the **analytic calibration**, the smallest σ that meets a target (ε, δ).

Many papers and libraries still use the **classic calibration** of Dwork & Roth,

$$\sigma = \frac{\Delta_2 f \sqrt{2\ln(1.25/\delta)}}{\varepsilon}, \tag{20.5}$$

which is proved only for $\varepsilon < 1$. Its proof bounds δ by the probability of the bad outputs, dropping the subtracted term, then loosens that bound again into a simple formula, so at $\varepsilon = 1$ and $\delta = 10^{-5}$ the classic σ delivers a δ about 240 times smaller than asked for. Below $\varepsilon = 1$ the classic formula uses 1.3 to 1.7 times the necessary noise (Table 20.2). Above it, nothing in the proof applies: at $\delta = 10^{-5}$ the formula happens to stay safe up to $\varepsilon = 8.42$ and uses too little noise beyond, because the classic σ shrinks like $1/\varepsilon$ while the exact one shrinks more slowly.

Table 20.2.: The noise the classic formula and the analytic calibration ask for at $\delta = 10^{-5}$, in units of the ℓ_2 sensitivity.

ε	Classic $\sigma/\Delta_2 f$	Analytic $\sigma/\Delta_2 f$	Ratio
0.1	48.4481	30.7496	1.576
0.5	9.6896	7.0318	1.378
1	4.8448	3.7306	1.299
2	2.4224	1.9938	1.215
4	1.2112	1.0812	1.120
8	0.6056	0.6002	1.009

Back to the diabetes count, now at $\varepsilon = 0.1$ and $\delta = 10^{-5}$. The analytic calibration gives $\sigma = 30.75$, and a normal variable stays within 1.96 standard deviations with probability 0.95, so the 95% error is 60.27, against 29.96 for Laplace noise with $\delta = 0$. For a single count, the Laplace mechanism wins on both error and guarantee.

A Gaussian release has a whole curve of guarantees, one δ for every ε. At $\sigma = \Delta_2 f$ the curve passes through $\delta = 0.1269$ at $\varepsilon = 1$ and reaches $\delta = 10^{-5}$ at $\varepsilon = 4.3772$ (Figure 20.2). So when you report a Gaussian release, report $\sigma/\Delta_2 f$ (or μ) as well as the pair you chose: the accountants of Part 4 compose the curve, not the pair. And choose δ much smaller than one over the number of people, for the reason given in The definition of differential privacy in the full edition: a mechanism that publishes each record verbatim with probability δ satisfies the definition with that δ.

20.5. Laplace or Gaussian?

The choice comes down to how many coordinates one record moves, because that is what separates $\Delta_1 f$ from $\Delta_2 f$. Compare the typical error per released number at the same ε: it is $\sqrt{2}\,\Delta_1 f/\varepsilon$ for Laplace noise, its standard deviation, and $\Delta_2 f\,\sigma_1$ for Gaussian noise, where σ_1 is the analytic σ for unit sensitivity at your (ε, δ): 3.7306 at $\varepsilon = 1$, $\delta = 10^{-5}$.

For a **sparse** query, where a record moves one coordinate, as in a histogram, the two sensitivities are equal, and Laplace noise wins at every d: an error of 1.41 against 3.73 at those parameters, with a stronger guarantee besides. For a **dense** query, where a record moves all d coordinates, the Laplace error grows like d and the Gaussian error like $\sqrt{d}$, so the Gaussian wins once d exceeds $(\varepsilon\sigma_1)^2/2$, which is 6.96 at those parameters (Figure 20.3). Threshold counts of ages are such a query, since one person's age moves every count at or below it. At 100 thresholds Gaussian noise is 3.79 times more accurate, and at 1,000 thresholds 11.99 times (Table 20.3).

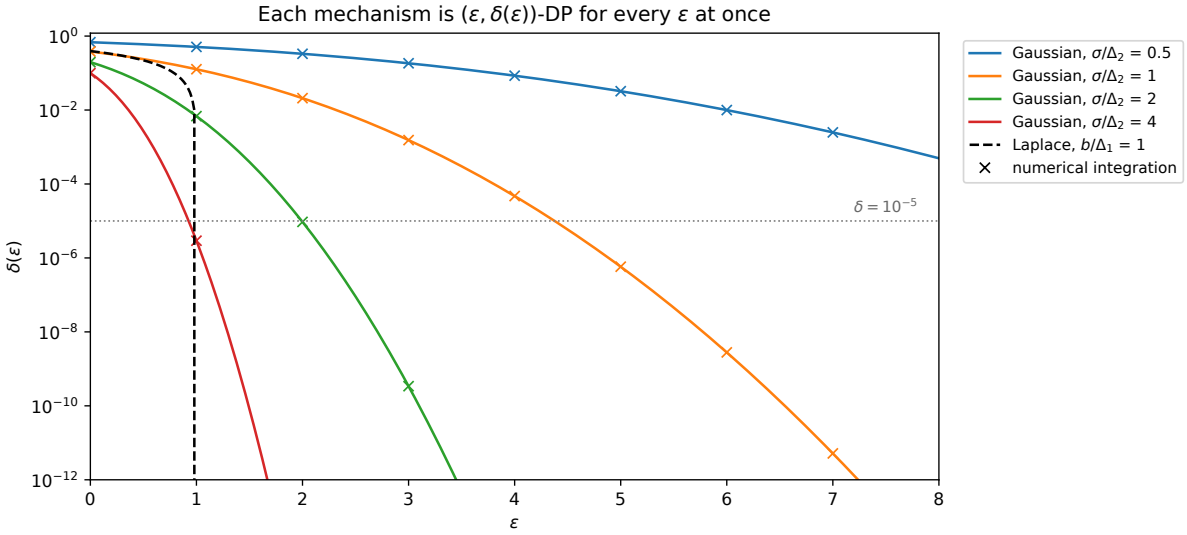


Figure 20.2.: The privacy curve $\delta(\varepsilon)$ of the Gaussian mechanism for four noise levels $\sigma/\Delta_2 f$, on a log scale, and of the Laplace mechanism with $b = \Delta_1 f$ (dashed). Crosses are the same divergence computed by numerical integration. The Laplace curve drops to zero at $\varepsilon = 1$, its pure-DP level; every Gaussian curve stays positive but falls faster than exponentially. The dotted line marks $\delta = 10^{-5}$.

Table 20.3.: Root-mean-square error per count for d threshold counts at $\varepsilon = 1$, $\delta = 10^{-5}$.

d	Laplace	Gaussian	Laplace / Gaussian
1	1.41	3.73	0.38
10	14.14	11.80	1.20
100	141.42	37.31	3.79
1000	1414.21	117.97	11.99

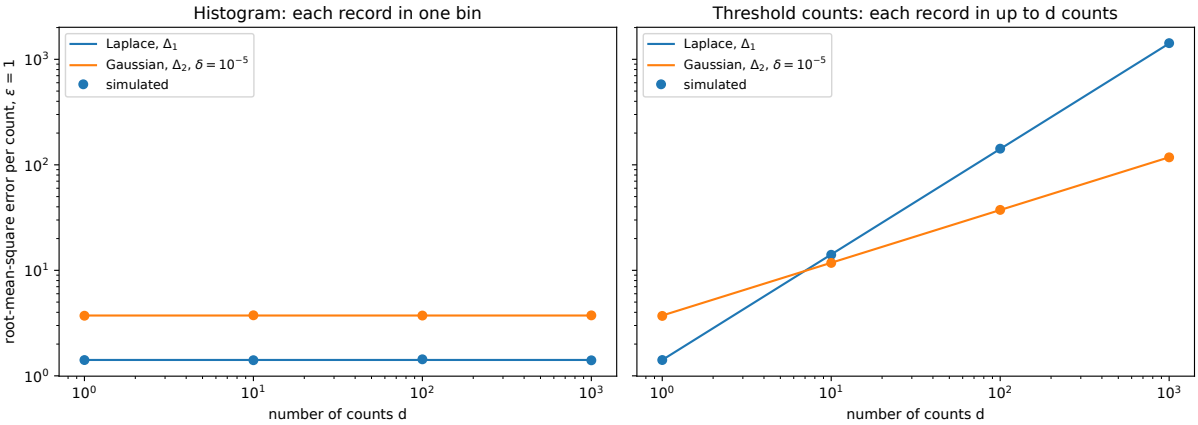


Figure 20.3.: Root-mean-square error per count at $\varepsilon = 1$ for the Laplace mechanism (calibrated to Δ_1) and the Gaussian mechanism (analytic calibration to Δ_2, $\delta = 10^{-5}$), as the number of released counts d grows; dots are simulations. Left: a histogram, where each record falls in one bin, so neither error depends on d and Laplace noise is smaller. Right: threshold counts, where a record can move all d counts; the Laplace error grows like d, the Gaussian like $\sqrt{d}$, and they cross near $d = 7$.

This is why private training of neural networks uses Gaussian noise. One example's gradient moves every coordinate of a model with millions of parameters, the densest query there is, and clipping each gradient bounds its ℓ_2 norm, which is exactly what the Gaussian mechanism is calibrated to. The comparison is at equal ε only: the Gaussian release also carries a δ.

```
def gaussian_delta(eps: float, mu: float) -> float:
    """The exact delta of the Gaussian mechanism at eps, where mu = sensitivity / sigma."""
    return normal_cdf(mu / 2 - eps / mu) - exp(eps) * normal_cdf(-mu / 2 - eps / mu)


def analytic_sigma(eps: float, delta: float, sensitivity: float = 1.0) -> float:
    """The smallest sigma for which the Gaussian mechanism is (eps, delta)-DP."""
    lo, hi = 1e-6, 100.0  # gaussian_delta grows with mu: bisect for gaussian_delta(eps, mu)
    for _ in range(200):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if gaussian_delta(eps, mid) < delta else (lo, mid)
    return sensitivity / lo


print(f"{analytic_sigma(1.0, 1e-5):.4f}")  # 3.7306
print(f"{analytic_sigma(0.1, 1e-5):.4f}")  # 30.7496
```

It returns $\sigma_1 = 3.7306$ at $\varepsilon = 1$, $\delta = 10^{-5}$, and 30.7496 at $\varepsilon = 0.1$, the analytic column of Table 20.2. For the Laplace mechanism there is nothing to solve: the scale is $\Delta_1 f/\varepsilon$. In production, take the noise itself from a DP library that samples it safely, and use this calculation to check the scale it reports.

The full edition's script, laplace_gaussian.py, computes every number in this chapter and checks each mechanism by simulation:

```
uv run python code/part03-dp-foundations/laplace_gaussian.py
```

20.7. Cheat sheet

Table 20.4.: The results of this chapter.

Result	Formula	Notes
Sensitivity	$\Delta_p f = \max_{D \simeq D'} \lVert f(D) - f(D') \rVert_p$	$\Delta_2 \le \Delta_1 \le \sqrt{d}\,\Delta_2$; depends on the neighbouring relation
Count, sum in $[0, B]$, mean with public n	1; B; B/n	Add/remove; either; replace
Laplace mechanism	$\mathrm{Lap}(\Delta_1 f/\varepsilon)$ per coordinate: ε-DP	All d errors below $b \ln(d/\beta)$ with probability $1-\beta$
Gaussian mechanism, exact	$\delta_\mu(\varepsilon) = \Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu}) - e^\varepsilon \Phi(-\frac{\mu}{2} - \frac{\varepsilon}{\mu})$, $\mu = \Delta_2 f/\sigma$	Solve for the smallest σ by bisection
Gaussian mechanism, classic	$\sigma = \Delta_2 f \sqrt{2 \ln(1.25/\delta)}/\varepsilon$	Only for $\varepsilon < 1$; 1.3–1.7 times too much noise

Result	Formula	Notes
Error per coordinate	Laplace $\sqrt{2}\,\Delta_1 f/\varepsilon$; Gaussian $\Delta_2 f\,\sigma_1(\varepsilon,\delta)$	Dense queries: Gaussian wins once $d > (\varepsilon\sigma_1)^2/2$

Rules of thumb

- Compute the sensitivity from the domain, not the data, and state the neighbouring relation with every ε.
- Report $\sigma/\Delta_2 f$ for a Gaussian release, not just one (ε, δ) pair.
- Sample the noise with a vetted DP library, never with a plain floating-point sampler.

20.8. Further reading

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3 and Appendix A: the Laplace mechanism and the classic Gaussian calibration.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising" (ICML 2018): the exact privacy curve and the analytic calibration.
- Near & Abuah, *Programming Differential Privacy*, the chapters on sensitivity and on the Laplace and Gaussian mechanisms: the same material with code.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

21. Advanced and optimal composition

21.1. Why this matters

A differential privacy (DP) system rarely answers one query. A dashboard refreshes a hundred counts; a training run takes thousands of noisy gradient steps. Each step is (ε, δ)-DP: for any two **neighbouring** datasets D and D', which differ in one person's record, the probability of any set of outputs changes by at most a factor e^{ε}, plus a small slack δ. You need one guarantee for the whole run, as small as the truth allows, because it sets how much noise each step can afford.

Post-processing, group privacy and composition in the full edition gave the first answer, **basic composition**: k steps that are each ε-DP are together $k\varepsilon$-DP. It is correct, and for large k far too pessimistic. A hundred counts at $\varepsilon = 0.1$ give $k\varepsilon = 10$, which lets the outputs multiply an attacker's odds by up to $e^{10} \approx 22{,}000$: nothing worth stating. Yet at $\delta = 10^{-5}$ the same hundred counts are $(4.31, 10^{-5})$-DP, and no analysis that knows only each step's ε can say less.

Basic composition assumes that every step produces its most revealing output at once. But the evidence a run leaks about a person is a sum of the evidence each step leaks, and a sum of many small random terms almost never reaches its maximum. Every result in this chapter charges for the tail of that sum instead of its maximum, and pays for the rare outputs beyond it with δ.

21.2. The privacy loss of a composition

Run a mechanism on D and look at its output y. The log of how much more likely y was under D than under D' is the **privacy loss** of that output: positive when y favours D, negative when it favours D', zero when it says nothing. Its distribution over random outputs is the **privacy loss distribution**. Pure ε-DP says that its absolute value never exceeds ε.

Now run k steps. The probability of the whole output sequence is the probability of the first output, times that of the second given the first, and so on. The ratio of two such products is the product of the step-by-step ratios, and its log is a sum: the total loss is the sum of the per-step losses. This needs no independence: each term is the loss of whatever step was run after the outputs seen so far.

The total loss tells you δ. The definition lets each output be up to e^{ε} times likelier under D than under D', with δ covering the excess. An output whose loss L is at most ε costs nothing. An output with a larger loss costs the part of its probability above the allowed ratio, a fraction $1 - e^{\varepsilon - L}$ of it: almost nothing just above ε, all of it when the loss is infinite. Averaging over outputs, the smallest δ that works at a given ε is

$$\delta(\varepsilon) = \mathbb{E}\big[(1 - e^{\varepsilon - L})_+\big] \le \Pr[L > \varepsilon], \tag{21.1}$$

where $(x)_+ = \max\{x, 0\}$. So a composition theorem only needs the right tail of the total loss.

Each step's loss can reach ε, but its average is much smaller. When ε is small, an output that favours D is only slightly more likely than one that favours D', so positive and negative losses nearly cancel, and what remains is of order ε^2: at most $\varepsilon \tanh(\varepsilon/2)$, about $\varepsilon^2/2$. It is never negative, because each step is a little more likely to point to the truth than away from it, so these averages add up to a steady drift.

Randomized response makes this concrete, and turns out to be the worst case. It reports a person's bit truthfully with probability $t = e^\varepsilon/(1 + e^\varepsilon)$, so each report has loss exactly $+\varepsilon$ (truthful) or $-\varepsilon$ (a lie). Take the dashboard's 100 counts at $\varepsilon = 0.1$, as randomized response: $t = 0.5250$. The total loss has mean 0.4996 and standard deviation 0.9988. Basic composition's $k\varepsilon = 10$ needs all 100 reports to be truthful, which happens with probability $t^{100} = 1.0 \times 10^{-28}$, 9.5 standard deviations above the mean. A sum of many small independent terms is nearly normal, and the optimal ε' at $\delta' = 10^{-5}$, 4.31, sits 3.8 standard deviations above the mean, where only a sliver of the distribution remains (Figure 21.1).

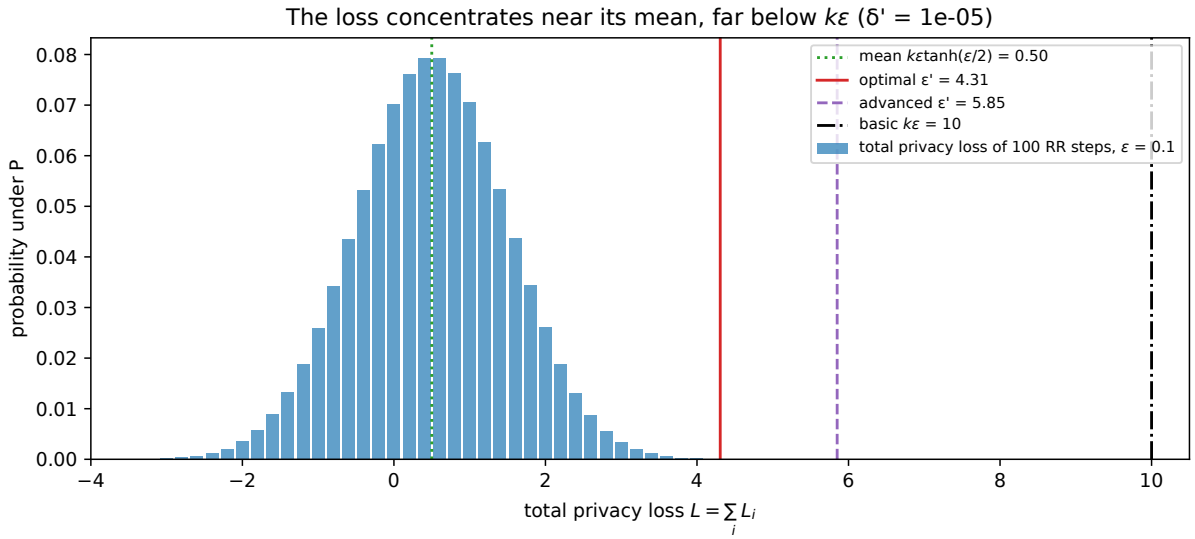


Figure 21.1.: The distribution of the total privacy loss of 100 rounds of randomized response at $\varepsilon = 0.1$. The vertical lines mark its mean, the ε' of optimal composition at $\delta' = 10^{-5}$, the ε' of advanced composition, and basic composition's $k\varepsilon = 10$, which the loss reaches with probability 10^{-28}.

Two consequences follow. The spread grows like $\sqrt{k}\,\varepsilon$ but the drift like $k\varepsilon^2/2$, so once $\varepsilon\sqrt{k}$ is of order 1 or more the drift is comparable to the spread, and the savings over $k\varepsilon$ shrink. And the trick needs $\delta > 0$: with $\delta = 0$ you must cover even the all-truthful output, which has positive probability, so you are stuck with $k\varepsilon$.

21.3. Advanced composition

Dwork, Rothblum & Vadhan (2010) turned this picture into a theorem. Split the total loss into the drift, the sum of the per-step averages, and a fluctuation around it. The fluctuation is a

sum of k terms, each confined to a range of width 2ε whatever happened before, and Azuma's inequality, a version of Hoeffding's inequality for terms that may depend on earlier terms, says it exceeds $\varepsilon\sqrt{2k\ln(1/\delta')}$ with probability at most δ'. Adding the drift gives the bound: k steps that are each (ε, δ)-DP, chosen adaptively, are together $(\varepsilon', k\delta + \delta')$-DP with

$$\varepsilon' = \varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon\tanh(\varepsilon/2). \tag{21.2}$$

The original theorem uses a cruder drift, $k\varepsilon(e^{\varepsilon} - 1)$, about twice as large for small ε; both are valid. The extra $k\delta$ is the chance that some step uses its own slack.

The price of the $\sqrt{k}$ is the factor $\sqrt{2\ln(1/\delta')}$, which is 4.80 at $\delta' = 10^{-5}$, the cost of cutting far into the tail. So the bound beats $k\varepsilon$ only once $\sqrt{k}$ has outgrown that factor. At $\varepsilon = 0.1$ with the original drift, that is from $k = 29$ on; at $k = 10$ it gives 1.623 (1.567 with the tanh drift), worse than basic composition's 1, so always take the smaller of the two. Inverting the leading term gives a budget: to be $(\varepsilon_{\text{total}}, \delta')$-DP after k steps, give each step roughly $\varepsilon = \varepsilon_{\text{total}}/\sqrt{2k\ln(1/\delta')}$, much more than the $\varepsilon_{\text{total}}/k$ basic composition allows.

For the 100 counts at $\delta' = 10^{-5}$, the fluctuation term is 4.7985 and the drift $100 \times 0.1\tanh(0.05) = 0.4996$, so $\varepsilon' = 5.2981$; with the original drift, 5.8502. Table 21.1 and Figure 21.2 follow the bounds as k grows. At $k = 10{,}000$, where $\varepsilon\sqrt{k} = 10$, the drift has taken over: the original drift costs 105 of the 153, which is why the tanh form is worth using.

Table 21.1.: Total ε' at $\delta' = 10^{-5}$ for k mechanisms that are each 0.1-DP.

k	Basic $k\varepsilon$	Advanced, $e^{\varepsilon} - 1$ drift	Advanced, Equation 21.2	Optimal
10	1.000	1.623	1.567	0.994
100	10.000	5.850	5.298	4.307
1,000	100.000	25.691	20.170	17.787
10,000	1,000.000	153.156	97.944	91.691

The bound is conservative in three places. Azuma's inequality assumes each step's loss has the largest variance its range allows. The original drift is about twice the true average loss. And the tail bound in Equation 21.1 counts every output above ε' in full, ignoring the discount for outputs just above it. So the guarantee is far stronger than it claims: at $k = 100$ the exact δ at the advanced $\varepsilon' = 5.85$ is 1.96×10^{-9}, not 10^{-5}. Use it for quick estimates, not for the number you report.

21.4. Optimal composition

The best bound comes from asking which (ε, δ)-DP mechanism is worst. Kairouz, Oh & Viswanath (2015) answered with an argument built on **post-processing**: further randomized steps that do not look at the data can only make the two worlds harder to tell apart, so they never increase $\delta(\varepsilon')$. They showed that every (ε, δ)-DP mechanism can be obtained by post-processing one particular mechanism, a four-output randomized response that spends both parameters to the full: with probability δ it announces which dataset it ran on, and otherwise it runs randomized

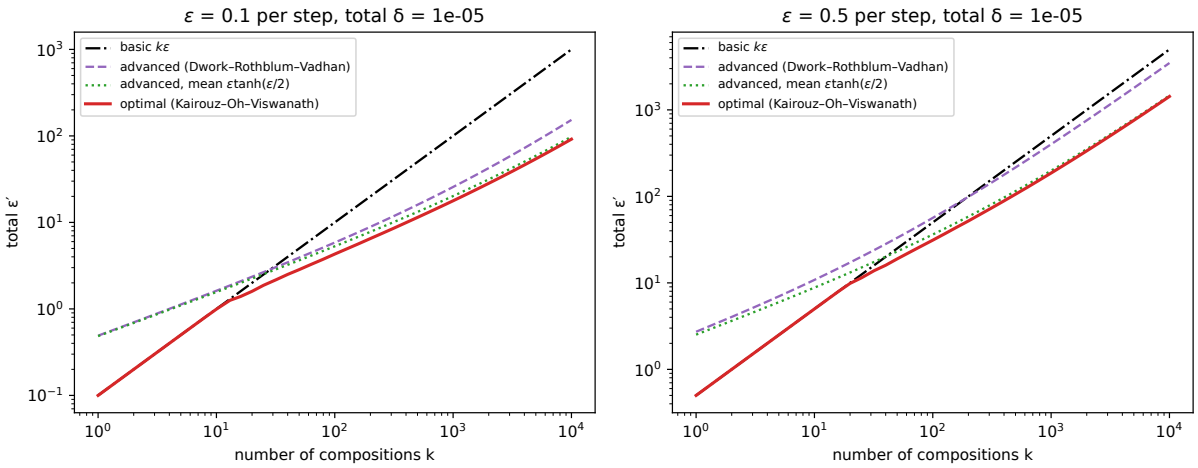


Figure 21.2.: Total ε' at $\delta = 10^{-5}$ after k compositions of ε-DP mechanisms, for $\varepsilon = 0.1$ (left) and $\varepsilon = 0.5$ (right), on log axes. Basic composition grows like k; advanced and optimal composition grow like $\sqrt{k}$ until the drift term takes over. The optimal curve is never above any other.

response at level ε. So any composition of k steps, even an adaptive one, is a post-processing of k copies of it, and can be no worse than those copies, whose loss you already know.

Their δ has two parts. Either some step announces the dataset, which happens with probability $1-(1-\delta)^k$, and the output is impossible under D', so it counts in full. Or none does, and the run is k rounds of randomized response: exactly j of the reports are lies with binomial probability, and the loss is then $(k-2j)\varepsilon$. Putting that loss into Equation 21.1 gives, for every composition of k steps that are each (ε,δ)-DP, adaptive or not,

$$\delta(\varepsilon') \le 1-(1-\delta)^k(1-\delta_{\mathrm{RR}}(\varepsilon')), \qquad \delta_{\mathrm{RR}}(\varepsilon') = \sum_{j=0}^{k}\binom{k}{j}t^{k-j}(1-t)^j\left(1-e^{\varepsilon'-(k-2j)\varepsilon}\right)_+, \quad (21.3)$$

with $t = e^{\varepsilon}/(1+e^{\varepsilon})$. The k copies are themselves a valid composition, so the bound is attained: no better one holds for the whole class. To get the total ε' for a target δ, bisect on ε' (Section 21.6).

For the 100 counts, the optimal ε' at $\delta' = 10^{-5}$ is 4.3068: 19% below the tanh advanced bound and 57% below basic composition. If each step also has $\delta = 5\times10^{-8}$, the announcements use up 5.0×10^{-6} of a total 10^{-5}, and the optimal ε' is 4.4469. The one trap is numerical. Kairouz, Oh & Viswanath's closed form divides numbers that overflow: at $\varepsilon = 1$ and $k = 1{,}000$ the denominator $(1+e)^{1000}$ is about 10^{570}, beyond any double. Summed in log space nothing overflows; the snippet gives $\delta(550) = 6.094\times10^{-4}$ there.

"Optimal" means optimal for mechanisms you know only by their (ε,δ). A real mechanism usually does better, because its loss is not always at $\pm\varepsilon$. Laplace noise on a count sometimes lands between the two true counts, where the loss is strictly between $-\varepsilon$ and ε; those outputs carry less evidence, so its mean loss at $\varepsilon = 1$ is 0.3679 against randomized response's 0.4621. With different ε_i per step, computing the exact bound is #P-complete, the class of hard counting problems (Murtagh & Vadhan 2016), so use an accountant.

The bigger waste is summarizing a mechanism by one point at all. A Gaussian mechanism has a whole curve $\delta(\varepsilon)$, and its privacy loss is itself normal, so k Gaussian steps with noise σ

on a sensitivity-1 query compose exactly into a single Gaussian step with shift $\sqrt{k}/\sigma$. With $\sigma = 20$, 1,000 steps are exactly $(7.51, 10^{-5})$-DP. Converting each step to its best (ε, δ) point and composing optimally gives 62.7, about 8 times worse (Figure 21.3): the point forgets how the curve continues. When you know the mechanisms, compose their curves with the accountants of Privacy accounting.

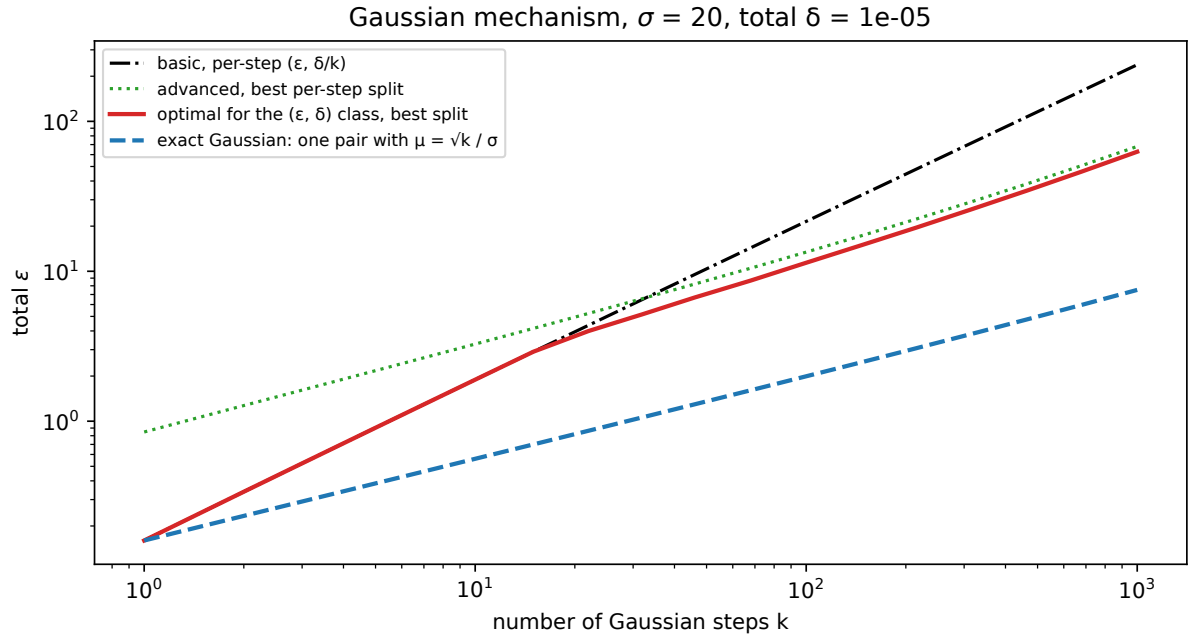


Figure 21.3.: Total ε at $\delta = 10^{-5}$ after k Gaussian mechanisms with $\sigma = 20$ on a query of sensitivity 1. The first three curves convert each step to one (ε, δ) point, with the best split of δ, then compose with basic, advanced or optimal composition. The dashed curve composes the exact Gaussian curves, a single Gaussian with shift $\sqrt{k}/\sigma$, and lies far below.

21.5. Adaptive composition

In practice the next query depends on the last answer: an analyst zooms into the largest bin of a noisy histogram, a training loop takes its next gradient at parameters set by earlier noisy gradients. This costs nothing extra, because every argument above looked at one step given the outputs so far, and once those are fixed, so is the choice of step. In the **adaptive composition experiment** of Dwork, Rothblum & Vadhan, an adversary picks each (ε, δ)-DP mechanism as any function of the earlier outputs, and basic, advanced and optimal composition all hold for what it sees.

An analyst who runs 100 steps at $\varepsilon = 0.1$, choosing a Laplace query after any output that pointed to D and a randomized-response query otherwise, shows it: in 100,000 simulated runs its δ at $\varepsilon' = 2$ is 0.0196, below the optimal bound of 0.0201, as it must be (Figure 21.4).

The same reasoning shows where adaptivity breaks. The theorems fix k, ε and δ in advance. If each ε_i is picked after seeing earlier outputs, the width of each term's range depends on the

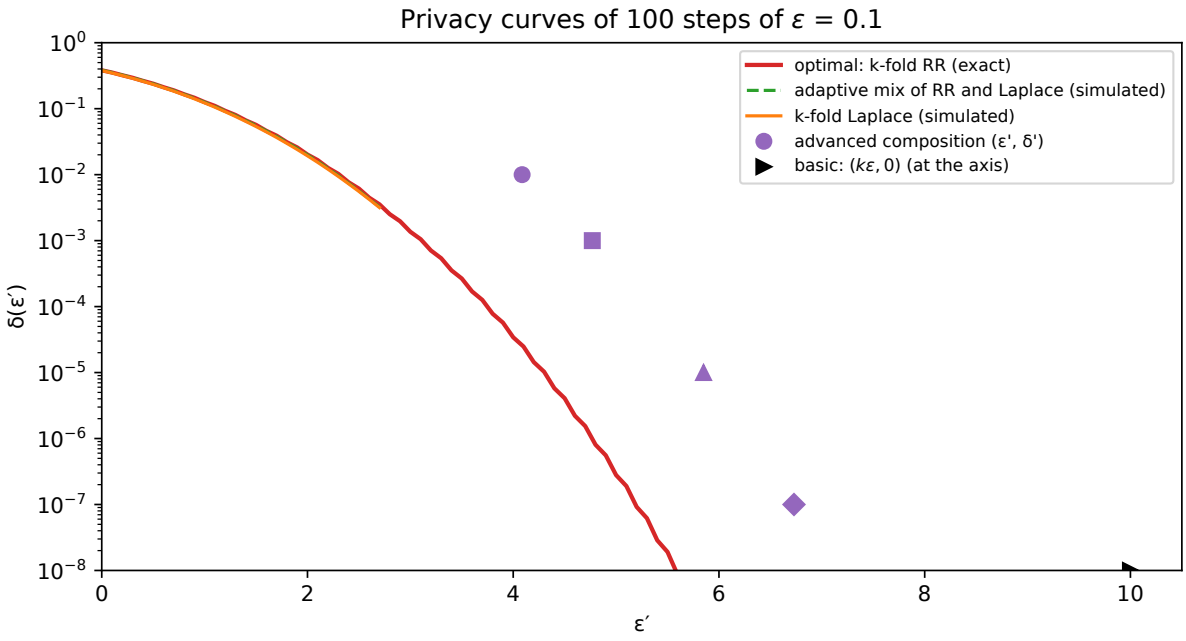


Figure 21.4.: The privacy curve $\delta(\varepsilon')$ of 100 steps at $\varepsilon = 0.1$: the exact curve of composed randomized response (optimal composition), simulated curves for composed Laplace mechanisms and for the adaptive analyst, the advanced-composition pairs (ε', δ') for several δ', and basic composition at $(10, 0)$. The three compositions nearly coincide; advanced composition lies far to the right.

history, and Azuma's inequality, which needs those widths fixed, no longer applies. A rule such as "keep querying until the budget is spent" therefore needs its own theorem, a **privacy filter**, and reporting the spend so far at every moment needs a **privacy odometer**, which costs more (Adaptive composition). Pure DP under basic composition is the exception: each loss is bounded by its ε_i on every output, so $\sum_i \varepsilon_i$ bounds the total whatever order the budgets came in.

Choices must also be made from the outputs, never from the raw data. A stopping rule that looks at the noisy answers is post-processing and free; one that looks at the raw data leaks: a system that answers private counts until the *true* count of some group crosses a threshold reveals that count through the number of answers it gives, and the run is not DP for any finite ε, even though every answer is.

21.6. Code

The optimal bound needs only a binomial sum, evaluated in log space so that nothing overflows, and a bisection. This is the whole of it, with advanced composition alongside:

```
from math import exp, expm1, lgamma, log, log1p, sqrt, tanh


def advanced_epsilon(eps: float, k: int, delta_prime: float) -> float:
    """Advanced composition with the sharp drift term k * eps * tanh(eps / 2)."""
    return eps * sqrt(2 * k * log(1 / delta_prime)) + k * eps * tanh(eps / 2)


def optimal_delta(eps_total: float, eps: float, k: int, delta: float = 0.0) -> float:
    """Kairouz-Oh-Viswanath: the exact delta(eps_total) of k (eps, delta)-DP steps, in log s
    log_t, log_1mt = -log1p(exp(-eps)), -log1p(exp(eps))  # log of truthful / lying probabil
    logs: list[float] = []
    for j in range(k + 1):  # j of the k reports lie
        loss = (k - 2 * j) * eps
        if loss <= eps_total:
            break
        log_binom = lgamma(k + 1) - lgamma(j + 1) - lgamma(k - j + 1)
        logs.append(log_binom + (k - j) * log_t + j * log_1mt + log(-expm1(eps_total - loss)
    top = max(logs, default=0.0)
    delta_rr = exp(top) * sum(exp(x - top) for x in logs) if logs else 0.0
    return 1 - (1 - delta) ** k * (1 - delta_rr)


def optimal_epsilon(eps: float, k: int, target_delta: float, delta: float = 0.0) -> float:
    """The smallest total epsilon whose optimal delta is at most target_delta, by bisection.
    lo, hi = 0.0, k * eps
    for _ in range(100):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if optimal_delta(mid, eps, k, delta) > target_delta else (lo, mid
    return hi
```

```
print(f"{advanced_epsilon(0.1, 100, 1e-5):.4f}")  # 5.2981
print(f"{optimal_epsilon(0.1, 100, 1e-5):.4f}")  # 4.3068
print(f"{optimal_epsilon(0.1, 100, 1e-5, delta=5e-8):.4f}")  # 4.4469
print(f"{optimal_epsilon(0.1, 1000, 1e-5):.3f}")  # 17.787
print(f"{optimal_delta(550, 1.0, 1000):.3e}")  # 6.094e-04
```

It reproduces the numbers of Section 21.3 and Section 21.4, including the case where the closed form overflows. The full edition's script, composition.py, checks the bound against brute-force enumeration and simulates the Laplace and adaptive compositions:

```
uv run python code/part03-dp-foundations/composition.py
```

21.7. Cheat sheet

Table 21.2.: The composition results of this chapter, for k steps that are each (ε, δ)-DP.

Result	Formula	Notes
Privacy loss of a composition	$L = L_1 + \cdots + L_k$	Adaptive or not
δ from the loss	$\delta(\varepsilon) = \mathbb{E}[(1 - e^{\varepsilon - L})_+] \leq \Pr[L > \varepsilon]$	Only the tail of L matters
Mean loss of an ε-DP step	$\leq \varepsilon \tanh(\varepsilon/2) \approx \varepsilon^2/2$	Equality for randomized response
Basic composition	$(k\varepsilon, k\delta)$	Adaptive; parameters may adapt for pure DP
Advanced composition	$(\varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon\tanh(\varepsilon/2),\ k\delta + \delta')$	Quick budget: $\varepsilon \approx \varepsilon_{\text{total}}/\sqrt{2k\ln(1/\delta')}$ per step; take the smaller of this and basic
Optimal composition	$1 - (1-\delta)^k(1-\delta_{\text{RR}}(\varepsilon'))$, Equation 21.3	Tight for mechanisms known only by (ε, δ); compute in log space
Known mechanism, such as Gaussian	Compose the whole curve	Point summaries can be off by a factor of 8
Adaptivity	Queries may adapt to outputs	Privacy parameters and data-dependent stopping may not

21.8. Further reading

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.5: basic and advanced composition, the version most libraries cite.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015): the hypothesis-testing view, the worst-case mechanism and the optimal bound.
- Rogers, Roth, Ullman & Vadhan, "Privacy Odometers and Filters: Pay-as-you-Go Composition" (NeurIPS 2016): why adaptively chosen privacy parameters need new theorems.
- Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS Series B, 2022): composing whole privacy curves, and why the total loss looks normal.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

22. Selection: exponential mechanism, report-noisy-max, sparse vector

22.1. Why this matters

Many questions ask you to **pick** something rather than compute a number. Which of 50 hyperparameter settings scores best on the validation set? Which of a thousand monitored statistics first crosses an alarm level? The answer is a name from a list, and "model 7 plus noise" is not a model.

The Laplace and Gaussian mechanisms in the full edition of differential privacy (DP) add noise to numbers. You could release a noisy score for every candidate, but that is k releases, and if one record can move all k scores, their costs add up and the noise must grow with k, which is often in the thousands. Yet you only wanted the winner, and you never have to pay for the losers.

The one idea of this chapter is to **release the position, not the values**. Each mechanism adds noise to scores it never publishes and releases only which candidate won. A candidate wins when its own noise clears a bar set by the best of the others, and a maximum of numbers that each move by at most Δ moves by at most Δ, however many numbers there are. So the number of candidates never enters the privacy cost. It enters only the accuracy, and only as $\ln k$.

22.2. The exponential mechanism

Each of k candidates has a **score** $u(D, r)$, how good candidate r is on dataset D, and its **sensitivity** Δ is the largest change in any one score between two **neighbouring** datasets, which differ in one person's record. A deterministic argmax can jump to another candidate when one record moves a score across a tie, so a private selection must be random, tilted toward high scores. The question is how much tilt you can afford.

McSherry and Talwar (2007) answered by giving each candidate a weight that grows exponentially with its score, and picking it with probability equal to its share of the total weight. The **exponential mechanism** outputs candidate r with probability

$$\Pr[M_E(D) = r] = \frac{\exp\left(\varepsilon\, u(D, r)/(2\Delta)\right)}{\sum_s \exp\left(\varepsilon\, u(D, s)/(2\Delta)\right)}, \tag{22.1}$$

which is a softmax of the scores at temperature $2\Delta/\varepsilon$. Why exponential, and why the 2? Changing one record moves each score by at most Δ, and an exponential turns that additive change into a multiplicative one: each weight changes by at most a factor $e^{\varepsilon/2}$, whatever the score. The probability of r is its weight divided by the total, and both can move against you at once, one

record raising r's score while lowering its competitors'. Each costs at most $e^{\varepsilon/2}$, so the ratio is at most e^{ε}: the mechanism is ε-DP, and k appears nowhere in that argument.

The 2 can be dropped when the scores are **monotone**, meaning one record always pushes all scores the same way. Counts under add/remove neighbours are monotone; a histogram under replace neighbours is not, because moving a record from one bin to another raises one count and lowers another. With monotone scores, r's weight and the total cannot both move against you, so weights $e^{\varepsilon u/\Delta}$ are also ε-DP. Check this under your neighbouring relation before you drop the 2.

How much accuracy does the randomness cost? Call the **gap** the distance between the best score and the chosen one. A candidate whose score is at least c below the best has at most $e^{-\varepsilon c/(2\Delta)}$ times the best candidate's weight, so even if all k candidates were that bad, together they would be picked with probability at most $k\,e^{-\varepsilon c/(2\Delta)}$. Setting that to e^{-t} gives, with probability at least $1 - e^{-t}$,

$$\text{gap} \le \frac{2\Delta}{\varepsilon}(\ln k + t), \tag{22.2}$$

and an expected gap of at most $\frac{2\Delta}{\varepsilon}(\ln k + 1)$. The $\ln k$ comes from the factor k in front of an exponential: doubling the number of candidates adds only $\frac{2\Delta}{\varepsilon}\ln 2$, and a lower bound in the full edition shows that no ε-DP mechanism can avoid it.

Take 50 models scored on a private validation set of 1,000 examples: model r gets $900 - r$ right, the score is the count of correct predictions, and $\Delta = 1$. At $\varepsilon = 1$ the mechanism picks the best model only 39% of the time, yet loses just 1.54 correct predictions on average, 0.15% accuracy: its mistakes are near-ties, which cost almost nothing (Table 22.1, Figure 22.1). Because the counts are monotone, the weights $e^{\varepsilon u}$ cut the expected gap from 1.54 to 0.58.

Table 22.1.: The exponential mechanism selecting among 50 models with scores $900 - r$. The gap is in correct predictions out of 1,000.

ε	Pr[best]	$\mathbb{E}$[gap]	Bound $2(\ln 50 + 1)/\varepsilon$	$\mathbb{E}$[gap], monotone weights
0.1	0.0531	15.03	98.24	9.17
1	0.3935	1.54	9.82	0.58
5	0.9179	0.09	1.96	0.01

The bound is loose here, 9.82 against 1.54, because it treats every candidate as if it sat just c below the best, while most of these are far below and nearly weightless. Use it to choose ε, not to predict the result.

Two consequences follow from the proof. The candidate list must be fixed before you look at the data, because the argument compares the same list on both datasets; a candidate read off the data leaks through its mere presence. And never exponentiate scores in the thousands directly. Adding a constant to every score multiplies every weight by the same factor, which cancels, so subtract the largest score first, or use the Gumbel form of the next section.

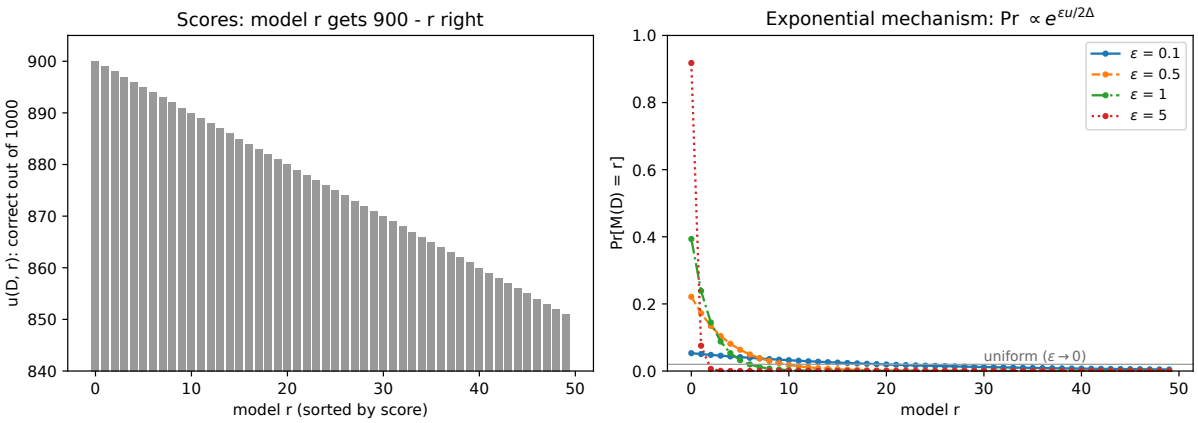


Figure 22.1.: Left: the scores of 50 models, model r getting $900 - r$ of 1,000 validation examples right. Right: the probability that the exponential mechanism selects each model, for four values of ε. Small ε spreads the probability toward uniform; large ε concentrates it on the best model.

22.3. Report-noisy-max and Gumbel noise

The other natural recipe adds noise to every score and reports the index of the largest, and *only the index*. **Report-noisy-max** outputs $\arg\max_r \left(u(D, r) + Z_r\right)$ with independent $Z_r \sim \text{Lap}(b)$, and it is ε-DP with $b = 2\Delta/\varepsilon$.

To see why the losers cost nothing, fix the noise of every other candidate. Their noisy scores are now fixed numbers, and candidate i wins exactly when its own noise clears a bar: the best of the others' noisy scores minus i's own score. One record moves the maximum by at most Δ, however many candidates it ranges over, and moves i's score by at most Δ, so the bar moves by at most 2Δ. Laplace noise of scale b pays for a shift s with a factor $e^{s/b}$, so $b = 2\Delta/\varepsilon$ makes the cost e^{ε}. With monotone scores the maximum and i's score move the same way, the bar moves by at most Δ, and $b = \Delta/\varepsilon$ suffices: the same factor of 2, for the same reason, as in the exponential mechanism.

The proof also shows why you must release only the index. It holds the other noises fixed and averages over candidate i's noise; publishing the winning noisy score publishes that very noise, and the guarantee is gone. If you need the winner's score, spend extra budget on a fresh Laplace answer for that one query.

Accuracy follows from the same picture. If no noise exceeds $b \ln(k/\beta)$, which happens with probability at least $1 - \beta$, the noise cannot reorder candidates more than twice that apart, so the gap is at most $2b \ln(k/\beta)$. On the 50 models at $\varepsilon = 1$, Laplace report-noisy-max with $b = 2$ picks the best model with probability 0.4241, against 0.3935 for the exponential mechanism, and has expected gap 1.40 against 1.54. Neither dominates in general.

They become the same mechanism with a different noise. If you add **Gumbel** noise of scale $2\Delta/\varepsilon$ instead of Laplace noise, the argmax picks each candidate with exactly the probability of Equation 22.1. This Gumbel-max trick gives the stable sampler promised above: it works in score units, with no normalizer to overflow. On five candidates, a million Gumbel draws land within 0.0008 of the exponential mechanism's exact probabilities, about the sampling error, while

Laplace noise of the same scale, with a different tail, is off by a total variation distance of 0.0216. Taking the c largest Gumbel-noised scores gives the same distribution as running the exponential mechanism c times and removing each winner (Durfee & Rogers 2019), at a cost of $c\varepsilon$ by basic composition.

22.4. The sparse vector technique

Now the candidates arrive one at a time and you want the first that is good enough, such as the first statistic to cross an alarm level. Answering each query with the Laplace mechanism would cost ε per query, mostly to learn "below". The **sparse vector technique** (Dwork, Naor, Reingold, Rothblum & Vadhan 2009) compares every query with one noisy threshold and pays only for the one above it.

Its basic form, **AboveThreshold** (Dwork & Roth 2014, Algorithm 1), takes queries of sensitivity Δ, possibly chosen adaptively, a threshold T, and budgets $\varepsilon_1+\varepsilon_2=\varepsilon$. It draws the threshold noise $\rho\sim\mathrm{Lap}(\Delta/\varepsilon_1)$ once. For each query it draws fresh noise $\nu_i\sim\mathrm{Lap}(2\Delta/\varepsilon_2)$; if $f_i(D)+\nu_i\geq T+\rho$ it outputs $\top$ ("above") and **halts**, and otherwise outputs $\perp$ ("below") and continues. It is ε-DP for any number of $\perp$ answers.

The same bar argument explains why. The output "stop at query k" says two things: the noisy threshold sits above every earlier noisy answer, and at or below the k-th. The first is a statement about one number, the largest earlier noisy answer, which moves by at most Δ however many queries came before; sliding the threshold noise by that much keeps all the $\perp$ answers in place, at a cost of e^{ε_1}. The single $\top$ is paid for by sliding its own fresh noise by at most 2Δ, at a cost of e^{ε_2}. With monotone queries that slide is at most Δ, and $\nu_i\sim\mathrm{Lap}(\Delta/\varepsilon_2)$ suffices (Lyu, Su & Li 2017).

The accuracy, unlike the privacy, depends on k, because each of the k query noises could happen to be large. With the equal split and probability at least $1-\beta$, every $\perp$ query has $f_i(D)<T+\alpha$ and the $\top$ query has $f_k(D)\geq T-\alpha$, where

$$\alpha=\frac{8\Delta(\ln k+\ln(2/\beta))}{\varepsilon}. \tag{22.3}$$

Only queries within α of T can go either way. Say you monitor $k=1000$ counting queries ($\Delta=1$) with $T=500$ at $\varepsilon=1$. With $\beta=0.05$, $\alpha=84.77$, so with probability at least 0.95 no query below 415.23 fires and no query above 584.77 is skipped. If every count is 400 except query 700, which is 600, the output is query 700; in 10,000 simulated runs it always was (Figure 22.2 shows a smaller run). Answering all 1,000 queries with the Laplace mechanism instead would add noise of scale 1,000 to counts of a few hundred.

AboveThreshold answers a yes/no question; to release the value of the query that fired, spend extra budget on a fresh Laplace answer. To find the first c queries above the threshold, restart it with fresh threshold noise after each $\top$, at ε/c per run. Any split of ε is private; Lyu, Su & Li give a more accurate one than the equal split.

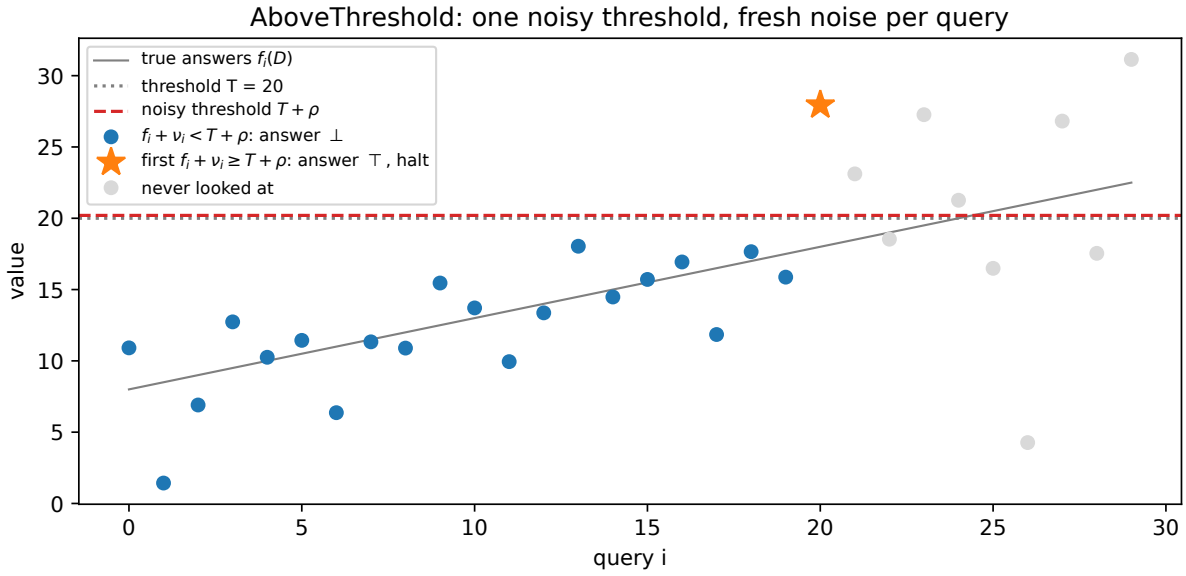


Figure 22.2.: One run of AboveThreshold with $\varepsilon = 1$ on 30 queries whose true answers rise from 8 to 22.5, with threshold $T = 20$. Each query's noisy answer is compared with the same noisy threshold. The first one above it is reported and the run halts.

22.5. When the sparse vector technique goes wrong

The proof has three ingredients, and each protects one step: the fresh noise ν_i pays for the $\top$, the halt limits the run to the one $\top$ that was paid for, and the noisy values stay secret because the proof averages over them. Lyu, Su & Li (2017) surveyed published variants, each a seemingly harmless optimization that weakens one ingredient; several are not DP at all (Table 22.2).

Table 22.2.: Variants of the sparse vector technique and their actual privacy, from Lyu, Su & Li (2017).

Variant (as numbered by Lyu, Su & Li)	What it changes	Privacy shown by Lyu, Su & Li
Algorithm 1 (theirs); Algorithm 2 (Dwork & Roth)	Correct, with c answers above	ε
Algorithm 3 (Roth's 2011 lecture notes)	Outputs the noisy value $f_i + \nu_i$ for each $\top$	∞
Algorithm 4 (Lee & Clifton 2014)	Query noise does not grow with c	$\frac{1+6c}{4}\varepsilon$
Algorithm 5 (Stoddard et al. 2014)	No noise on the queries	∞
Algorithm 6 (Chen et al. 2015)	Never halts	∞

The **no-halt variant** is the instructive one, because it uses exactly AboveThreshold's noise and only skips the halt. Each further $\top$ would need another slide of its own noise, another factor e^{ε_2},

and nothing pays for it. Take neighbours where the first m queries are 0 and the last m are 1 on D, and the reverse on D'. The output "m below, then m above" is a little more likely under D at every one of its $2m$ answers, and without a halt these small factors multiply: the privacy loss is at least $m\varepsilon/4$, which grows without bound. At $\varepsilon = 1$, AboveThreshold's largest loss levels off at exactly ε, so its proof is tight, while the no-halt variant's grows linearly in m (Figure 22.3, left).

A reviewer who checks only noise scales will miss this bug. An empirical audit catches it. If a mechanism is ε-DP, any event is at most e^{ε} times as likely under D as under D', so the log-ratio of a test's firing rates is a lower bound on the true ε. The test "more $\top$ answers in the second half than in the first" gives a log-ratio of 1.40 at $m = 40$, which already proves the no-halt variant is not 1-DP (Figure 22.3, right). When you review an implementation, check the ingredients directly: fresh noise for every query (reusing one draw is Algorithm 5's bug), one threshold noise per run, a halt after the allowed number of $\top$ answers, and no published noisy values.

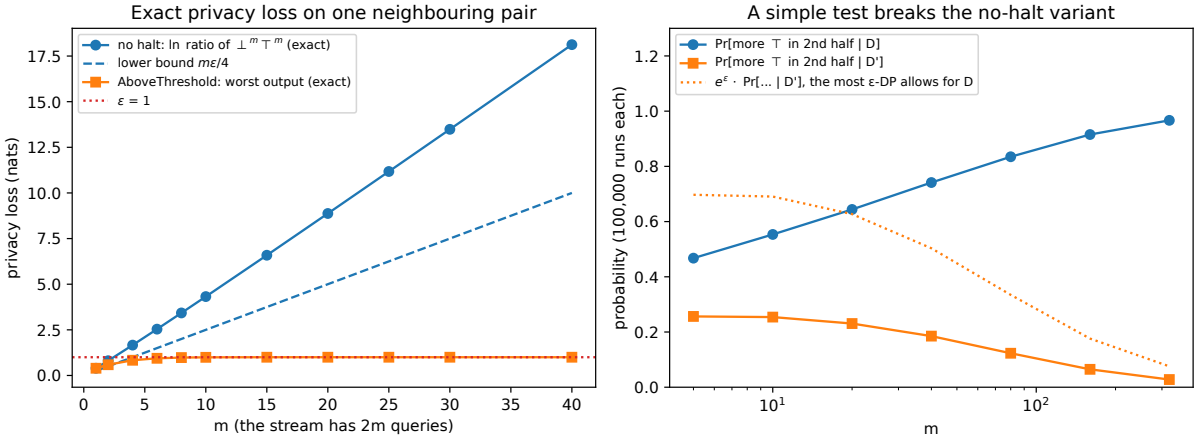


Figure 22.3.: Left: privacy loss on the counterexample with $\varepsilon = 1$, computed exactly. For the no-halt variant the loss of $\perp^m\top^m$ grows linearly in m, above the lower bound $m\varepsilon/4$; for AboveThreshold the largest loss levels off at ε. Right: the no-halt variant under the test "more $\top$ answers in the second half"; once the probability under D exceeds e^{ε} times that under D' (dotted), no ε-DP mechanism could produce it.

22.6. Code

Each mechanism fits in a few lines. Note where AboveThreshold draws its two noises:

```
import random
from math import exp, log
from typing import Sequence


def exponential_mechanism_probs(scores: Sequence[float], eps: float, sensitivity: float) ->
    """Selection probabilities, proportional to exp(eps * u / (2 * sensitivity)), without ov
    top = max(scores)
    weights = [exp(eps * (u - top) / (2 * sensitivity)) for u in scores]
```

```
    total = sum(weights)
    return [w / total for w in weights]


def gumbel_select(scores: Sequence[float], eps: float, sensitivity: float, rng: random.Rando
    """One draw of the exponential mechanism: add Gumbel noise of scale 2 * sensitivity / ep
    beta = 2 * sensitivity / eps
    noisy = [u - beta * log(-log(1 - rng.random())) for u in scores]
    return max(range(len(scores)), key=noisy.__getitem__)


def laplace(scale: float, rng: random.Random) -> float:
    return rng.expovariate(1 / scale) - rng.expovariate(1 / scale)


def above_threshold(
    answers: Sequence[float], threshold: float, eps: float, sensitivity: float, rng: random.
) -> int | None:
    """Index of the first answer whose noisy value reaches the noisy threshold; equal budget
    noisy_threshold = threshold + laplace(2 * sensitivity / eps, rng)  # drawn once per run
    for i, answer in enumerate(answers):
        if answer + laplace(4 * sensitivity / eps, rng) >= noisy_threshold:  # fresh noise e
            return i  # halt, and release only the index
    return None


def svt_margin(k: int, eps: float, sensitivity: float, beta: float) -> float:
    return 8 * sensitivity * (log(k) + log(2 / beta)) / eps


print([round(p, 4) for p in exponential_mechanism_probs([3, 2, 1, 0], 1.0, 1.0)])  # [0.4551
print(f"{svt_margin(1000, 1.0, 1.0, 0.05):.2f}")  # 84.77
counts = [600.0 if i == 699 else 400.0 for i in range(1000)]  # query 700 is the only high o
print(above_threshold(counts, 500.0, 1.0, 1.0, random.Random(0)))  # 699
```

The first line is a vote with counts $(3, 2, 1, 0)$ at $\varepsilon = 1$; the last finds query 700 of the alarm stream. Floating-point samplers are fine for checking a calculation but not for production (Discrete and floating-point-safe mechanisms in the full edition): use a vetted DP library. The full edition's script, selection.py, computes every number in this chapter and checks each mechanism by exact integration and simulation:

```
uv run python code/part03-dp-foundations/selection.py
```

22.7. Cheat sheet

Table 22.3.: The selection mechanisms of this chapter.

Mechanism	Output	Privacy	Accuracy
Exponential mechanism	r with probability $\propto e^{\varepsilon u(D,r)/(2\Delta)}$	ε; $e^{\varepsilon u/\Delta}$ for monotone scores	Gap $\leq \frac{2\Delta}{\varepsilon}(\ln k + t)$ w.p. $1 - e^{-t}$
Report-noisy-max, Laplace	$\arg\max_r(u + \mathrm{Lap}(2\Delta/\varepsilon))$, index only	ε; $\mathrm{Lap}(\Delta/\varepsilon)$ for monotone scores	Gap $\leq 2b\ln(k/\beta)$ w.p. $1-\beta$
Report-noisy-max, Gumbel	$\arg\max_r(u + \mathrm{Gumbel}(2\Delta/\varepsilon))$	Identical to the exponential mechanism	Same
AboveThreshold	First $f_i + \mathrm{Lap}(2\Delta/\varepsilon_2) \geq T + \mathrm{Lap}(\Delta/\varepsilon_1)$, then halt	$\varepsilon_1 + \varepsilon_2$, any number of $\perp$	$\alpha = 8\Delta(\ln k + \ln\frac{2}{\beta})/\varepsilon$, equal split
c answers above	AboveThreshold restarted c times at ε/c	ε by basic composition	α with ε/c in place of ε

22.8. Further reading

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3, §3.4 and §3.6: report-noisy-max, the exponential mechanism and AboveThreshold, with proofs.
- Lyu, Su & Li, "Understanding the Sparse Vector Technique for Differential Privacy" (Proceedings of the VLDB Endowment, 2017): the general algorithm, the budget split and the survey of broken variants. The best single source on the technique.
- Durfee & Rogers, "Practical Differentially Private Top-k Selection with Pay-what-you-get Composition" (NeurIPS 2019): one-shot Gumbel top-c selection.
- Near & Abuah, *Programming Differential Privacy*, the chapters on the exponential mechanism and the sparse vector technique: short implementations.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

23. Beyond global sensitivity

23.1. Why this matters

The Laplace and Gaussian mechanisms in the full edition set their noise by the **global sensitivity**: the most one record can move the answer, over every pair of neighbouring datasets that could exist, not just yours. For a count that is the right scale. For many statistics it is not. The median of ages in $[0, 100]$ has global sensitivity 100, because in a dataset of half zeros and half hundreds one person swings it from end to end. The mean of incomes has infinite sensitivity, and for an estimator fitted by an optimizer nobody knows it. Yet on a realistic sample of 1,001 ages, one person moves the median by about a tenth of a year, because the ages near the middle are packed tightly together.

Can you add noise on the scale of the data you hold and keep a differential privacy (DP) guarantee? Yes, if you respect one idea: **the noise scale is itself a function of the data, so it must be private too.** Each technique below makes the scale safe to reveal in a different way. Throughout, datasets have a fixed size n and neighbours differ by one **replaced** record.

23.2. Local sensitivity, and why it leaks

The obvious fix is to take the maximum over your own dataset's neighbours only, the **local sensitivity**:

$$\mathrm{LS}_f(D) = \max_{D' :\ d(D,D')=1} |f(D) - f(D')|, \tag{23.1}$$

where the **Hamming distance** $d(D, D')$ counts the records in which two datasets differ. For the median of n sorted values (with n odd), one change can push the median at most one place up or down the sorted list, so the local sensitivity is the larger of the two gaps next to the median.

But noise of scale $\mathrm{LS}_f(D)/\varepsilon$ publishes the local sensitivity along with the answer, and one record can change it enormously. With ages in $[0, 100]$, the dataset $(0, 0, 0, 0, 100)$ has median 0 with gaps of 0 on both sides, so its "noisy" median is exactly 0. Its neighbour $(0, 0, 0, 100, 100)$ has the same median, but a gap of 100 above it, so its release is spread out and never exactly 0. An output of 0 identifies the first dataset with certainty, whatever ε you claim.

Nor does it need a degenerate dataset. The neighbours of Table 23.1 both have median 30, but at $\varepsilon = 1$ their noise scales are 10 and 60. The narrow Laplace density falls off six times faster than the wide one, so the further an output lands from 30, the more it favours D': the privacy loss grows in a straight line with the distance and has no bound. It passes 1 beyond a distance of 33.50, which happens with probability 0.572 under D' and only 0.0351 under D, and covering that would take $\delta = 0.477$ (Figure 23.1).

Table 23.1.: Two neighbouring datasets with the same median and very different local sensitivities.

Dataset	Values	Median	LS
D	10, 20, 30, 31, 90	30	10
D' (31 replaced by 90)	10, 20, 30, 90, 90	30	60

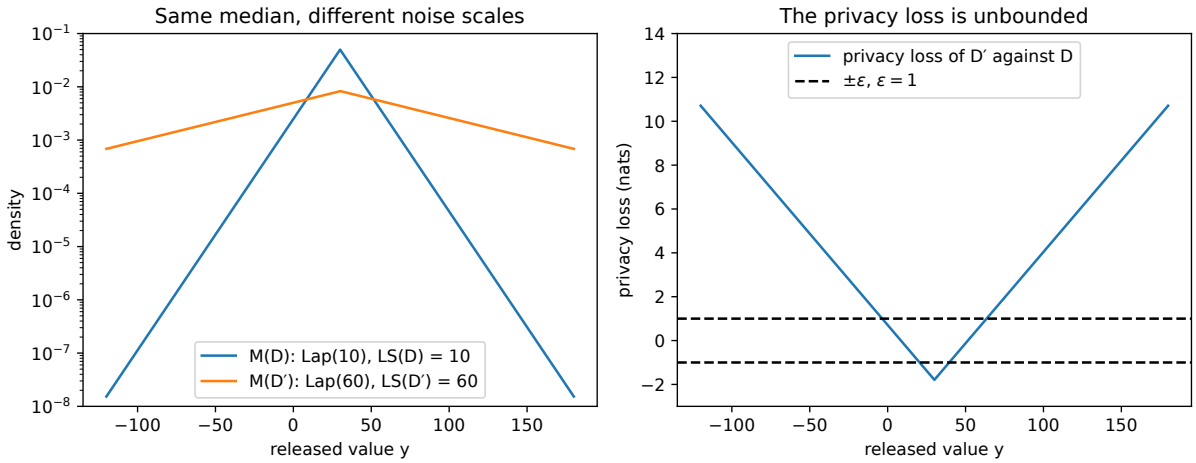


Figure 23.1.: Left: the median of D and of D' released with Laplace noise scaled to their own local sensitivities, 10 and 60. Right: the privacy loss grows linearly with the distance from 30 and crosses $\varepsilon = 1$ at distance 33.5, so no ε bounds it.

The same leak hides in any scale computed from the private data, such as a clip bound set to the observed maximum. Each such number needs one of the repairs below or its own share of the budget.

23.3. Smooth sensitivity

The first repair, due to Nissim, Raskhodnikova and Smith (2007), uses a scale that cannot jump: an upper bound on the local sensitivity that changes by at most a factor e^{β} between neighbours. A bounded *factor* is the right kind of control because a privacy loss is a log-ratio, and stretching the noise by e^{β} shifts log-densities by about β.

What is the smallest such bound? If some dataset k changes away has a large local sensitivity, the bound at yours can be smaller only by the factor $e^{-\beta}$ per step on the way. So it must be at least that dataset's local sensitivity discounted by $e^{-k\beta}$, and the β**-smooth sensitivity** is exactly the largest of these discounted values:

$$S^*_\beta(D) = \max_{D''} \ \mathrm{LS}_f(D'')\, e^{-\beta\, d(D,D'')}. \tag{23.2}$$

For the median, k changes can at most empty the stretch between two sorted values $k+1$ ranks apart next to the median; calling the widest such gap $A^{(k)}$, S^*_β is the largest $e^{-k\beta}A^{(k)}$, an exact $O(n^2)$ computation (Section 23.7).

The release stretches noise of a fixed shape Z by this bound, $f(D) + (S^*_\beta(D)/\alpha)\,Z$, where α sets how many noise scales one record's shift is worth. Which shapes work depends on their tails, because between neighbours the loss contains the log-ratio of a density and a copy stretched by up to e^β. For Laplace noise the log-density is linear in $|z|$, so the stretch changes it by an amount proportional to $|z|$: unbounded, exactly the leak of Table 23.1 in miniature. A power-law tail has a log-density that grows only like $\ln|z|$, so a stretch changes it by a bounded amount everywhere. That is why **pure DP needs heavy tails**: with standard Cauchy noise and $\alpha = \beta = \varepsilon/6$, releasing $f(D) + \frac{6}{\varepsilon} S^*_{\varepsilon/6}(D) \cdot Z$ is ε-DP. Laplace noise can still be used, if you pay for its rare large draws with a δ: the release is (ε, δ)-DP when

$$\beta + e^\beta \alpha + (e^\beta - 1)\ln(1/\delta) \le \varepsilon. \tag{23.3}$$

The last term charges the change of scale once for each unit of $\ln(1/\delta)$, which forces β down to roughly $\varepsilon/(2\ln(1/\delta))$, much smaller than the Cauchy's $\varepsilon/6$.

Release the median of 1,001 synthetic ages at $\varepsilon = 1$. With global sensitivity the median absolute error is 69.31 years. The local sensitivity is 0.1386, and the Cauchy release's smooth sensitivity, 0.1632, is barely above it, because with this many people even seven changes cannot open a wide gap near the middle; its error is 0.979. Laplace noise at $\delta = 10^{-6}$, with $\alpha = 1/2$, allows only $\beta = 0.0322$, so more distant datasets count and $S^* = 0.5144$, but the lighter tails make up for it: error 0.713. Both are 70 to 100 times better than the global release, and the gain grows with n, because the smooth bound follows the spacing of the data near the median (Figure 23.2). The same reasoning shows where it fails: for $n \lesssim 20$, a few changes can open a gap across nearly the whole range, and smooth sensitivity loses to global sensitivity.

The heavy tail has its own cost: about one Cauchy release in 16 is off by more than ten times the typical error, and the Cauchy has no mean, so don't average such values. And a bound you substitute for S^*, for a statistic without an exact formula, works only if it is itself β-smooth.

23.4. Propose-test-release

Without a formula like the median's, Dwork and Lei (2009) take a cruder route. Propose a bound b on the local sensitivity before looking at the data, then ask how many records would have to change before the local sensitivity could exceed b; call that distance $\mathrm{dist}_b(D)$. This count is safe to release with noise: a path from a neighbour to a bad dataset can always go through your dataset first, so one record moves the distance by at most 1. **Propose-test-release** adds $\mathrm{Lap}(1/\varepsilon)$ to the distance, outputs $\perp$ ("no answer") if the result is at most $\ln(1/\delta)/\varepsilon$, and otherwise releases $f(D) + \mathrm{Lap}(b/\varepsilon)$.

Why is that private? If the proposal is right for a pair of neighbours, so that their answers differ by at most b, the release is an ordinary Laplace mechanism, and test plus release cost ε each: 2ε in all. If it is wrong, both datasets have local sensitivity above b, so both sit at distance 0, and the test passes only if the noise alone clears the threshold, which happens with probability $\delta/2$. That lucky pass is what δ pays for, so the mechanism is $(2\varepsilon, \delta)$-DP. The noise scale is now your proposal, which reveals nothing, and $\perp$ says only that the data were probably near an unstable point.

The threshold is demanding: at $\varepsilon = 1$ and $\delta = 10^{-6}$ it is $\ln(10^6) = 13.82$, so the data must be stable against more than a dozen changes. On the 1,001 ages with $b = 1$ year, the distance is

21, the test passes with probability 0.999621, and the release has median error 0.693 years at a total cost of 2. On 101 ages the distance is 1 and the test almost never passes: one year is far too optimistic there (Figure 23.2, right).

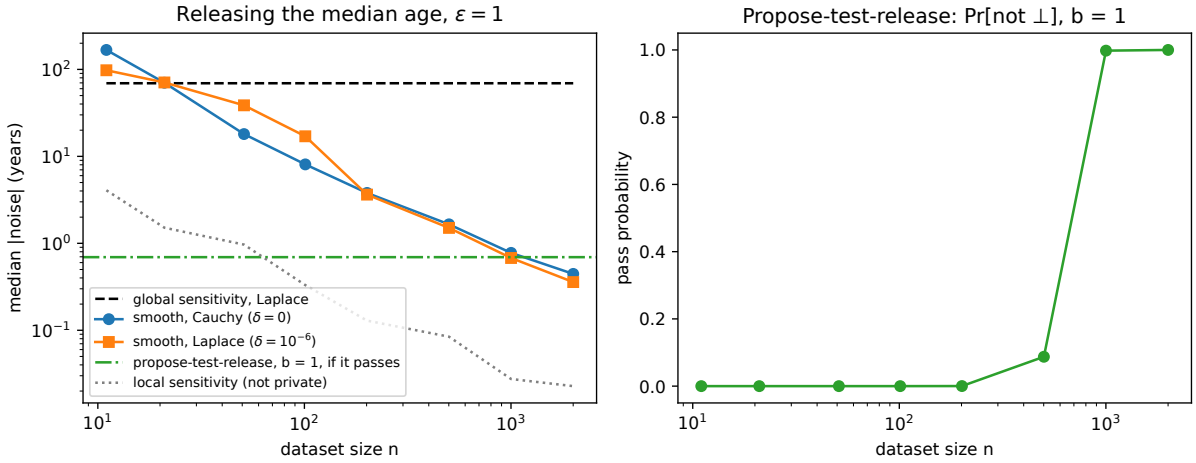


Figure 23.2.: Left: median absolute noise when releasing the median age at $\varepsilon = 1$, by global sensitivity, smooth sensitivity with Cauchy noise and with Laplace noise ($\delta = 10^{-6}$), and propose-test-release with $b = 1$ when it passes, as the number of people grows. The dotted line is the non-private local sensitivity. Right: the probability that propose-test-release with $b = 1$ answers.

A proposal taken from the data, such as twice the observed local sensitivity, brings back the leak of Section 23.2. Plan for $\perp$: every retry with a larger b spends budget again.

23.5. Subsample-and-aggregate

When you cannot analyse f at all, such as a trained model, make the sensitivity small by construction. Split the records into B disjoint blocks by a partition fixed in advance (say, by a hash of a record identifier), run f on each block, clip each block answer z_j to $[0, R]$, and release

$$M(D) = \frac{1}{B}\sum_{j=1}^{B} \mathrm{clip}_{[0,R]}(z_j) + \mathrm{Lap}\Big(\frac{R}{B\varepsilon}\Big). \tag{23.4}$$

One record lives in one block, so it changes one clipped answer by at most R and the average by at most R/B. That is the whole proof, and it never looks inside f, so the release is ε-DP for any f. It is also why the partition must not depend on the values: otherwise one record could reshuffle many blocks.

The number of blocks sets a trade-off. The noise falls like $1/B$, because one record's influence is diluted over more blocks. But each block sees only n/B records, and many estimators are biased on small samples, so the bias grows with B. The sampling error hardly moves: each block estimate is noisier by a factor B in variance, and averaging B of them undoes it. Take the rate of $n = 10{,}000$ exponential inter-arrival times, estimated by $1/\bar{x}$, at $\varepsilon = 1$ with $R = 5$. Its global sensitivity is infinite. With 10 blocks the noise standard deviation is 0.7071 and swamps

everything; with 1,000 blocks of 10 records the bias is 0.1111. At 250 blocks the two are of similar size and the error is 0.0396, about four times the non-private 0.0100, for an estimator whose sensitivity you never had to bound (Figure 23.3). Choose B from an error model, not by trying values on the data.

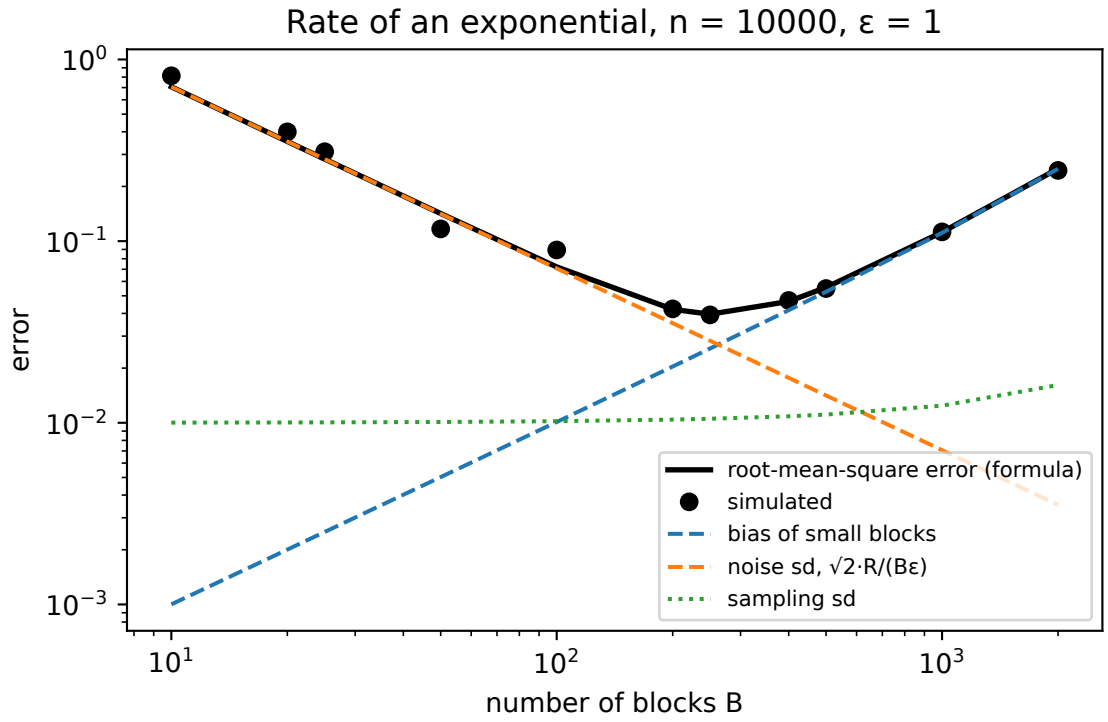


Figure 23.3.: Root-mean-square error of subsample-and-aggregate for the exponential rate against the number of blocks B, with its three components: the bias of the estimator on small blocks, the Laplace noise, and the sampling error.

The range R is still a global bound, now on the output of f, and a loose R means large noise. With a noisy vote as the combiner, this is private aggregation of teacher ensembles (PATE), where models trained on disjoint shards vote.

23.6. Clipping

For the mean of unbounded data, **clipping** forces a bound: replace each value by $\min(x, C)$, and one record moves the mean by at most C/n, so $\frac{1}{n}\sum_i \min(x_i, C) + \mathrm{Lap}(C/(n\varepsilon))$ is ε-DP for non-negative data. Every "assume the data lie in $[0, \Lambda]$" of earlier chapters is a clip in disguise, and the same move on per-example gradients is the core of private model training.

The price is bias: clipping removes the part of each value above C, so the estimate is low by the average excess $\mathbb{E}[(X - C)_+]$. Adding the sampling variance of the clipped values and the noise variance gives the mean squared error,

$$\mathrm{err}(C)^2 = \mathbb{E}[(X - C)_+]^2 + \frac{\mathrm{Var}(\min(X, C))}{n} + \frac{2C^2}{n^2\varepsilon^2}. \tag{23.5}$$

Raising C shrinks the first term and grows the last. The noise shrinks with n but the bias does not, so with more data you can clip further into the tail. Given a model of your data, minimize Equation 23.5 numerically before seeing any records.

Log-normal incomes, in thousands, with median 40 and mean 55.085, show the trade-off at $n = 1000$ and $\varepsilon = 1$ (Table 23.2). Clipping at the median cuts half the population back and leaves a bias of 23. The optimum, 331.34, touches only 0.4% of people and has error 1.666, just 1% above the 1.649 of the unclipped, non-private mean. And the optimum moves out as predicted: 177.5 at $n = 100$, 557.3 at $n = 10{,}000$ (Figure 23.4).

Table 23.2.: The clipped private mean of log-normal incomes, $n = 1000$, $\varepsilon = 1$, for four clip bounds.

Clip bound C	$\Pr[X > C]$	Bias	Noise sd	Root-mean-square error
40 (the median)	0.5000	23.415	0.057	23.417
257.23 (99th percentile)	0.0100	0.924	0.364	1.767
331.34 (optimal)	0.0041	0.438	0.469	1.666
1000	0.0000	0.006	1.414	2.169

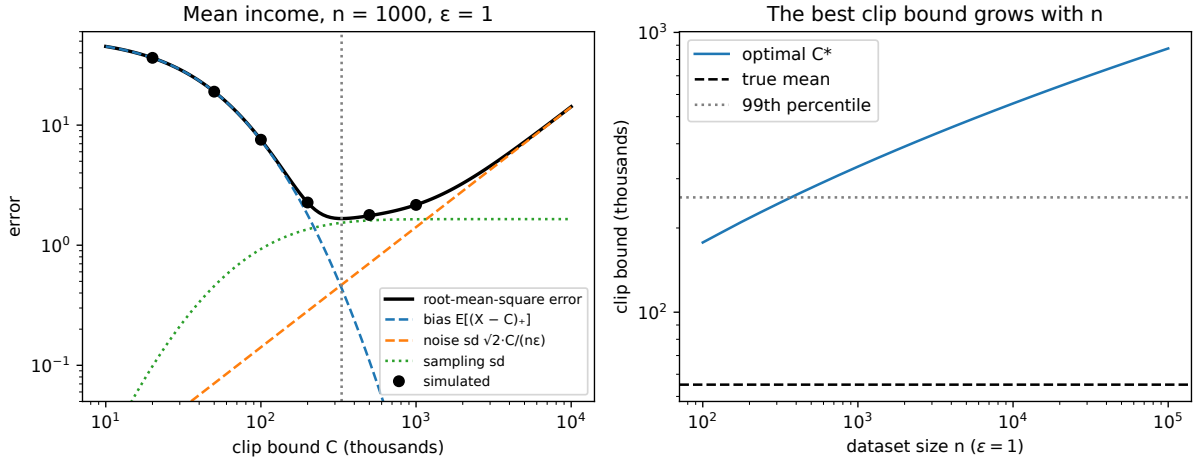


Figure 23.4.: Left: root-mean-square error of the clipped private mean of log-normal incomes against the clip bound C, with its bias, noise and sampling parts; the dotted line marks the optimum near 331. Right: the optimal clip bound grows with the dataset size.

Three consequences follow. Choosing C from the data, say the empirical 99th percentile, makes the noise scale a statistic again and reopens the leak of Section 23.2; take it from domain knowledge, or spend budget on a private quantile with the exponential mechanism in the full edition or the methods above. Clipping from above only ever lowers values, so it always underestimates, and clipping to $[-C, C]$ is unbiased only when the cuts on both sides cancel, as they do for distributions symmetric about 0. And under add/remove neighbours n itself is private, so release

a noisy clipped sum and a noisy count, divide, and charge both to the budget (Post-processing, group privacy and composition in the full edition).

23.7. Code

The smooth sensitivity of the median is easy to get subtly wrong, mostly in the padding of the order statistics. The snippet computes it for $D = (1, 3, 4, 6, 9)$ in $[0, 10]$ with $\beta = 0.5$: the local sensitivity is 2, but one change opens a gap of 5, and $e^{-0.5} \times 5 = 3.03$ wins. It also solves for the β of the age example.

```
from math import exp, log


def gap_within(xs: list[float], upper: float, k: int) -> float:
    """A(k): the widest gap next to the median that changing k records can open."""
    n, m = len(xs), (len(xs) + 1) // 2  # n odd; m is the rank of the median

    def x(i: int) -> float:  # order statistic i, padded with 0 below and `upper` above
        return 0.0 if i < 1 else upper if i > n else xs[i - 1]

    return max(x(m + t) - x(m + t - k - 1) for t in range(k + 2))


def smooth_sensitivity_median(values: list[float], upper: float, beta: float) -> float:
    """The beta-smooth sensitivity of the median of values in [0, upper]."""
    xs = sorted(values)
    return max(exp(-k * beta) * gap_within(xs, upper, k) for k in range(len(xs) + 1))


def laplace_beta(eps: float, delta: float, alpha: float) -> float:
    """The largest beta with beta + e^beta * alpha + (e^beta - 1) * ln(1/delta) <= eps."""
    lo, hi = 0.0, eps
    for _ in range(100):
        mid = (lo + hi) / 2
        ok = mid + exp(mid) * alpha + (exp(mid) - 1) * log(1 / delta) <= eps
        lo, hi = (mid, hi) if ok else (lo, mid)
    return lo


data = [1.0, 3.0, 4.0, 6.0, 9.0]
print(f"{gap_within(sorted(data), 10.0, 0):.2f}")  # 2.00
print(f"{smooth_sensitivity_median(data, 10.0, 0.5):.2f}")  # 3.03
print(f"{laplace_beta(1.0, 1e-6, 0.5):.4f}")  # 0.0322
```

The distance for propose-test-release is the first `k` with `gap_within(xs, upper, k) > b`. The full edition's script computes every number in this chapter and checks the formula for $A^{(k)}$ by

brute force:

```
uv run python code/part03-dp-foundations/sensitivity.py
```

23.8. Cheat sheet

Table 23.3.: The techniques of this chapter, with neighbours that replace one record.

Technique	Release	Privacy	Use when
Local sensitivity	$f(D) + \text{Lap}(\text{LS}_f(D)/\varepsilon)$	**Not private**	Never
Smooth sensitivity, Cauchy	$f(D) + \frac{6}{\varepsilon} S^*_{\varepsilon/6}(D)\, Z$	ε	Median, quantiles; n in the hundreds or more
Smooth sensitivity, Laplace	$f(D) + \frac{S^*_\beta(D)}{\alpha}\text{Lap}(1)$, $\beta + e^\beta\alpha + (e^\beta - 1)\ln\frac{1}{\delta} \le \varepsilon$	(ε, δ)	Same, when you want light tails
Propose-test-release	$\perp$ if $\text{dist}_b + \text{Lap}(\frac{1}{\varepsilon}) \le \frac{\ln(1/\delta)}{\varepsilon}$, else $f(D) + \text{Lap}(\frac{b}{\varepsilon})$	$(2\varepsilon, \delta)$	Stable statistics without a smooth-sensitivity formula
Subsample-and-aggregate	Mean of B clipped block outputs $+\text{Lap}(\frac{R}{B\varepsilon})$	ε	Black-box estimators and models
Clipped mean	$\frac{1}{n}\sum \min(x_i, C) + \text{Lap}(\frac{C}{n\varepsilon})$; bias $\mathbb{E}[(X - C)_+]$	ε	Means and sums of unbounded data

Rules of thumb

- Pure DP with a data-dependent scale needs heavy-tailed noise; Laplace noise needs a δ and a β near $\varepsilon/(2\ln(1/\delta))$.
- Fix clip bounds, block counts and proposals before looking at the data, from an error formula or with a separate slice of the budget.

23.9. Further reading

- Nissim, Raskhodnikova & Smith, "Smooth Sensitivity and Sampling in Private Data Analysis" (STOC 2007): smooth sensitivity, the admissible noise distributions, the median formula and the original sample-and-aggregate framework.
- Dwork & Lei, "Differential Privacy and Robust Statistics" (STOC 2009): propose-test-release, and robust private estimators built on it.

- Amin, Kulesza, Muñoz Medina & Vassilvitskii, "Bounding User Contributions: A Bias-Variance Trade-off in Differential Privacy" (ICML 2019): choosing clip bounds when users contribute many records.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

24. Discrete and floating-point-safe mechanisms

24.1. Why this matters

The proof of the Laplace mechanism in the full edition compares two densities on the real line. Your computer has no real numbers. It has floating-point numbers, a finite set with uneven gaps, and a source of random bits, and the function that "samples Lap(b)" produces some distribution on that finite set, about which the proof says nothing. Mironov (2012) showed that the textbook sampler, in double precision, leaks the true answer through the low-order bits of its output.

The one idea of this chapter is that **differential privacy (DP) is a property of the output distribution the code actually produces**, so the code must produce exactly the distribution the proof is about. There are two ways to get there. Keep floating point and repair the output, by rounding it to a coarse grid so that every output is reachable from every input. Or leave floating point behind: make the query integer-valued and sample integer noise exactly, with integer and rational arithmetic only. The second gives the discrete Laplace and discrete Gaussian mechanisms, whose samplers deliver exactly what their proofs assume.

24.2. Floating-point Laplace breaks differential privacy

Suppose some output can be produced from a dataset D but never from its neighbour D'. Seeing that output tells an observer the data were D, with certainty. The DP inequality $\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta$, applied to the set S of such outputs, has a right-hand side of $e^{\varepsilon} \cdot 0 + \delta$, and the factor e^{ε} is useless however large it is. So the whole probability of those outputs must be paid for by δ:

$$\delta \;\ge\; \Pr[M(D) \in S] \tag{24.1}$$

for every ε. Any output reachable from one input only rules out pure DP, and this support check is the quickest audit of an implementation.

The textbook sampler fails it. It draws U uniform on $(0, 1)$ and a random sign ξ, and outputs $f(D) + \xi \cdot b \ln(1/U)$, rounding the logarithm and the sum to the nearest float. Floats are dense near 0 and sparse far from it: between 2^e and 2^{e+1} they are evenly spaced, and the spacing doubles with each power of 2. With $f(D) = 0$, an output near 0 is a noise value as it is, on the fine grid of floats near 0. With $f(D') = 1$, the same output must come from 1 minus a noise value near 1, which lies on the coarser grid there. So most of the fine-grained outputs of $M(D)$ near 0 cannot come from $M(D')$, and away from the inputs the two sets of outputs only partly coincide.

Suppose a hospital releases a count with this sampler at $b = 1$, which would be 1-DP in exact arithmetic, and an analyst who knows the count is 0 or 1 checks whether the released float can be produced from 1. In a toy model with 12-bit floats, small enough to enumerate exactly,

$\Pr[M(D) \in S] = 0.4791$: the check identifies the count 0 with certainty 47.91% of the time, where the proof promised odds moved by at most a factor e (Figure 24.1).

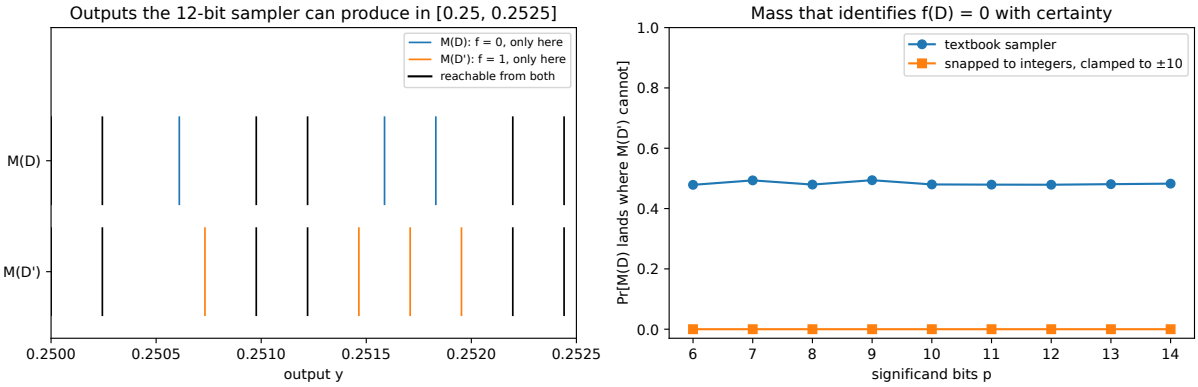


Figure 24.1.: The toy 12-bit floating-point Laplace sampler with $b = 1$. Left: the outputs that $f(D) = 0$ (top) and $f(D') = 1$ (bottom) can produce in a narrow window, coloured by whether the other input can produce them too. Right: the probability that $M(D)$ produces an output $M(D')$ cannot, against the number of significand bits, for the textbook sampler (flat, near 48%) and the snapped one (zero).

More precision does not help, because extra bits refine both grids together and the mismatch stays: from 6 to 14 bits, about 48% of the mass sits on outputs the other input can't produce. Mironov found the same effect at the 53 bits of a double in several DP libraries, and Jin et al. (2022) found such leaks in later ones. The tails leak as well: a 53-bit uniform is never below 2^{-53}, so the noise never exceeds about $36.74\,b$, and the outputs just beyond that bound from one input are unreachable from the other. So `numpy.random.laplace`, or any `f + scale * log(u)`, is not a DP mechanism, whatever ε the code claims.

24.3. The snapping mechanism

The leak lives in which float the output is, not in where it lands on the real line, and that is the idea of Mironov's **snapping mechanism**. Round the noisy answer to a grid at least as coarse as the noise scale, and the bits that depend on the input are discarded; every input now reaches the same finite set of grid points, and the support argument has nothing to bite on.

Why does rounding cost no privacy? Each grid point collects all the noise values that round to it, an interval of the real line. In exact arithmetic its probability under D and under D' is the probability that a Laplace variable lands in one of two intervals shifted by at most one unit, and the Laplace density changes by at most a factor $e^{1/\lambda}$ under such a shift. Rounding exact noise is just post-processing. What remains is the floating-point error, which moves the interval endpoints slightly. That error is a fraction $\eta = 2^{-53}$ of the numbers involved, so the mechanism clamps the input and the output to $[-B, B]$ to keep those numbers, and the error, bounded.

Concretely, for a query with sensitivity 1 and noise scale λ, let Λ be the smallest power of 2 at or above λ. Clamp $f(D)$ to $[-B, B]$, add floating-point Laplace noise of scale λ, round to the

nearest multiple of Λ, and clamp again. For $\lambda < B < 2^{46}\lambda$ the result is ε-DP with

$$\varepsilon = \frac{1 + 12B\eta}{\lambda} + \frac{2^{-49}B}{\lambda}, \qquad \eta = 2^{-53} : \tag{24.2}$$

the exact mechanism's $1/\lambda$ plus two terms that pay for the floating-point error and grow with the clamping range. For $\lambda = 1$ and $B = 10$ the excess is about 3.1×10^{-14}. In the toy, snapping to integers and clamping to $[-10, 10]$ makes both inputs reach the same 21 outputs at every precision, and the largest privacy loss is 1.0010 at 12 bits and 1.0002 at 14, against 1 for the exact mechanism.

The costs follow from the construction. Rounding adds up to $\Lambda/2 \le \lambda$ of error, and clamping biases answers near $\pm B$, so choose B from the domain, knowing that ε grows with it. And because snapping post-processes a mechanism that is not DP, its proof rests on Mironov's bookkeeping and on an ideal uniform and a correct logarithm: use a vetted implementation, not your own variant.

24.4. The discrete Laplace mechanism

Most DP queries are counts and histograms, integer-valued with sensitivity 1, and for them the natural noise is integer-valued too: the output has no low-order bits to leak, and the noise can be sampled exactly. Take the Laplace density, keep its values at the integers and renormalize: that is the **discrete Laplace** distribution $\mathrm{Lap}_{\mathbb{Z}}(b)$, and adding it is the **geometric mechanism** of Ghosh, Roughgarden and Sundararajan (2009). Each step away from 0 multiplies the probability by $r = e^{-1/b}$, as the continuous density drops by $e^{-1/b}$ per unit, so its probability mass function (PMF) is

$$\Pr[Z = z] = \frac{1 - r}{1 + r}\, r^{|z|}, \qquad z \in \mathbb{Z}, \qquad r = e^{-1/b}. \tag{24.3}$$

The privacy guarantee is the continuous one. Shifting the input by one unit changes each output's probability by at most the factor $e^{1/b}$, and shifting by an integer only relabels the terms of the normalizing sum, so it doesn't change the normalizer. For an integer-valued query with ℓ_1 sensitivity $\Delta_1 f$, adding independent $\mathrm{Lap}_{\mathbb{Z}}(b)$ noise to each coordinate is ε-DP with $\varepsilon = \Delta_1 f / b$, the same calibration as before.

A count at $\varepsilon = 1$ has $b = 1$ and $r = e^{-1} = 0.3679$. The exact count comes out with probability 0.4621, and the noise variance is 1.8413, slightly below the 2 of continuous Lap(1): same guarantee, a little less noise. For a single count nothing does better: every user, whatever their loss function and prior, can post-process the geometric mechanism's output to do as well as with any other ε-DP mechanism.

The query must be integer-valued, since a non-integer shift moves the noise off the lattice and brings back the support problem. Rounding $f(D) + \mathrm{Lap}(b)$ to an integer is no substitute: it would be private in exact arithmetic, but it needs the floating-point sampler of Section 24.2.

24.5. The discrete Gaussian mechanism

The Gaussian mechanism is the workhorse for dense vectors, because its noise follows the ℓ_2 sensitivity (The Laplace and Gaussian mechanisms in the full edition). Canonne, Kamath and

Steinke (2020) define its integer analogue the same way: the **discrete Gaussian** $\mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$ gives each integer x probability proportional to $e^{-x^2/(2\sigma^2)}$. The US Census Bureau used it for the 2020 Census, and it is the noise of choice before secure aggregation, which works with integers.

The discrete Laplace argument carries over. Integer shifts leave the normalizer unchanged, so the log-ratio of two shifted PMFs is the log-ratio of the two bell curves: the privacy loss is the continuous Gaussian's, linear in the output, evaluated at integers. The outputs whose loss exceeds ε therefore form a tail, and for a query whose value differs by Δ between neighbours the smallest δ at each ε has the same form as the continuous Gaussian's analytic curve, with discrete tails in place of continuous ones:

$$\delta(\varepsilon) = \Pr\left[X > \frac{\varepsilon\sigma^2}{\Delta} - \frac{\Delta}{2}\right] - e^{\varepsilon} \Pr\left[X > \frac{\varepsilon\sigma^2}{\Delta} + \frac{\Delta}{2}\right], \qquad X \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma^2). \tag{24.4}$$

The first term is the probability of the outputs that favour D by more than e^{ε}; the second subtracts the part of it that the factor e^{ε} already covers.

Same form does not mean same value. The privacy loss now takes only lattice values, one per integer output, so as ε crosses one of them a whole output enters or leaves the sum, and the discrete curve has a kink there and oscillates around the continuous one (Figure 24.2). For sensitivity 1 at $\varepsilon = 1$, it gives $\delta = 0.6032$ against 0.5099 for the continuous Gaussian at $\sigma = 0.5$, 0.1414 against 0.1269 at $\sigma = 1$, and 0.0072 against 0.0068 at $\sigma = 2$; at other ε, such as just past a kink, the discrete curve is below, so neither dominates. So don't reuse the continuous formula to report the discrete Gaussian's δ: it can understate it, and more so for small σ. Compute Equation 24.4 directly, as Section 24.7 does. In Rényi divergence, which Rényi differential privacy uses for composition, the discrete Gaussian is never worse than the continuous one, so Gaussian accountants apply unchanged.

Restricting the bell curve to integers also pulls a little mass towards 0, so the variance is below σ^2, by a relative 2.11×10^{-7} at $\sigma = 1$ but down to $0.8601\,\sigma^2$ at $\sigma = 0.5$.

Two consequences matter in machine learning. A real-valued query, such as a clipped gradient, must be scaled by $1/\gamma$ for a small step γ and rounded to integers first. Randomized rounding (up with probability equal to the fractional part) keeps it unbiased, but the rounded query can have a slightly larger sensitivity, and that is the one to calibrate to. And unlike continuous Gaussians, a sum of discrete Gaussians is not exactly a discrete Gaussian; for $\sigma \geq 1$ it is very close, which schemes where each client adds a share of the noise rely on.

24.6. Sampling exactly, and choosing a library

Inverting a cumulative distribution function (CDF) needs the discrete Gaussian's normalizer, which is irrational, and a floating-point uniform, which brings back every problem of Section 24.2. Canonne, Kamath and Steinke (2020) avoid both with samplers that use only integer and rational arithmetic and uniform random integers.

The building block is a coin that comes up 1 with probability $e^{-\gamma}$, made from coins with rational probabilities. Flip coins with probabilities $\gamma, \gamma/2, \gamma/3, \ldots$ until one fails: the chance of surviving k flips is $\gamma^k/k!$, a term of the power series of $e^{-\gamma}$, and reporting whether the first failure came at an odd step adds those terms with alternating signs, giving exactly $e^{-\gamma}$. Such coins yield an exact discrete Laplace sampler. The discrete Gaussian then comes by rejection: the discrete Laplace has

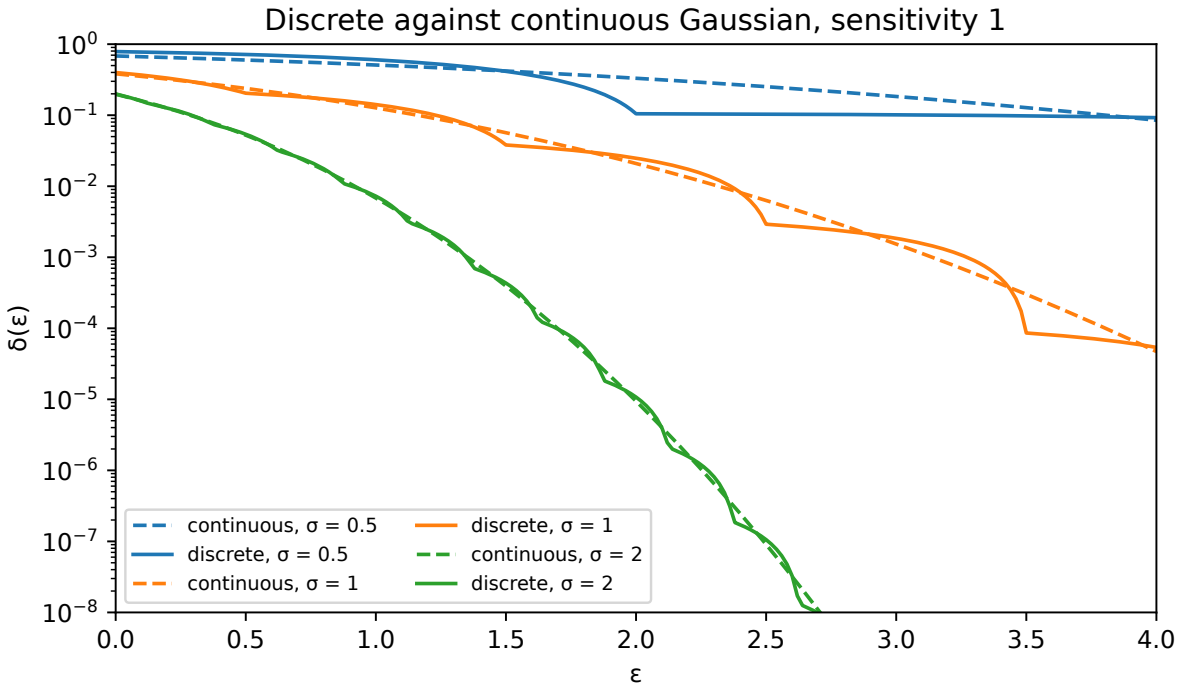


Figure 24.2.: Discrete against continuous Gaussian for sensitivity 1: $\delta(\varepsilon)$ of the discrete Gaussian (solid) and of the continuous Gaussian (dashed), for $\sigma = 0.5$, 1 and 2. The discrete curve has a kink at each value of its lattice-valued privacy loss, and approaches the continuous one as σ grows.

heavier tails, so it can serve as a proposal, and each draw is kept with a probability, decided by one exact coin, chosen so that proposal times acceptance is proportional to the Gaussian weights. Rejection never needs the normalizer.

They are cheap. A discrete Laplace draw takes about 10 to 12 uniform integers. A discrete Gaussian draw is accepted with probability 0.5185 at $\sigma^2 = \frac{1}{4}$ and 0.7532 at $\sigma^2 = 100$, so it takes fewer than two proposals on average. A histogram of 1,000 bins with $\mathcal{N}_{\mathbb{Z}}(0, 100)$ noise per bin uses about 15.6 uniform integers per bin, about 15,600 in all, and no floating-point operation.

Exactness is not timing safety. The number of rejection rounds depends on the output, so the running time leaks information about the noise, and Jin et al. (2022) show such leaks are exploitable. So a review of a library or pipeline asks four questions. Is the noise drawn by an exact integer sampler or a snapping-style repair, with no floating-point `exp`, `log` or `random()` inside the sampler? Do the random bits come from a cryptographic source (in Python, `secrets.SystemRandom()`), not a seeded generator? Is the running time padded or fixed? And does the reported calibration match the mechanism used: the discrete Gaussian's own $\delta(\varepsilon)$, the rounded query's sensitivity, the clamping range?

24.7. Code

Take the noise itself from a vetted library. What you can check yourself is the calibration: that the δ reported for a discrete Gaussian matches Equation 24.4. The snippet sums the PMF over the integers and compares it with the continuous Gaussian's exact curve.

```
from math import ceil, erfc, exp, sqrt


def discrete_gaussian_delta(eps: float, sigma: float, sens: int = 1) -> float:
    """Exact delta(eps) of the discrete Gaussian mechanism, summed over the integers."""
    reach = ceil(40 * sigma) + sens  # the mass beyond 40 sigma is far below double precisio
    weight = {y: exp(-y * y / (2 * sigma**2)) for y in range(-reach - sens, reach + 1)}
    total = sum(weight[y] for y in range(-reach, reach + 1))
    delta = 0.0
    for y in range(-reach, reach + 1):  # outputs of f(D) = 0 and f(D') = sens
        p, q = weight[y] / total, weight[y - sens] / total
        delta += max(p - exp(eps) * q, 0.0)
    return delta


def normal_cdf(x: float) -> float:
    return 0.5 * erfc(-x / sqrt(2))


def gaussian_delta(eps: float, sigma: float, sens: float = 1.0) -> float:
    """Exact delta(eps) of the continuous Gaussian mechanism, for comparison."""
    mu = sens / sigma
    return normal_cdf(mu / 2 - eps / mu) - exp(eps) * normal_cdf(-mu / 2 - eps / mu)
```

```
print(f"{discrete_gaussian_delta(1.0, 0.5):.4f} {gaussian_delta(1.0, 0.5):.4f}")  # 0.6032 0
print(f"{discrete_gaussian_delta(1.0, 1.0):.4f} {gaussian_delta(1.0, 1.0):.4f}")  # 0.1414 0
print(f"{discrete_gaussian_delta(1.0, 2.0):.4f} {gaussian_delta(1.0, 2.0):.4f}")  # 0.0072 0
```

Floating point is fine here: this computes a guarantee, not a release. The full edition's script implements the exact samplers with `fractions.Fraction` and integers, checks each against its exact distribution, and enumerates the toy floating-point sampler:

```
uv run python code/part03-dp-foundations/discrete_mechanisms.py
```

24.8. Cheat sheet

Table 24.1.: The results of this chapter.

Result	Formula	Notes
Support argument	$\delta \geq \Pr[M(D) \in S]$, $S =$ outputs reachable from D only	Any such output rules out pure DP; the quickest audit
Floating-point Laplace	$f(D) + \xi\, b \ln(1/U)$, rounded	Not DP: about 48% of the mass identifies the input in the toy, at every precision
Snapping	Clamp, add noise, round to $\Lambda \geq \lambda$, clamp	$\varepsilon = \frac{1+12B\eta}{\lambda} + \frac{2^{-49}B}{\lambda}$ for sensitivity 1
Discrete Laplace	$\frac{1-r}{1+r} r^{\lvert z\rvert}$, $r = e^{-1/b}$	$\varepsilon = \Delta_1 f / b$; variance $\frac{2r}{(1-r)^2}$, just below $2b^2$
Discrete Gaussian	$\propto e^{-x^2/(2\sigma^2)}$ on the integers	$\delta(\varepsilon)$ by Equation 24.4; same Rényi bound as the continuous Gaussian
Exact samplers	Integers, rationals, uniform integers	Fewer than two proposals per discrete Gaussian draw; pad the running time

Rules of thumb

- Never ship `f(D) + laplace(b)` in floating point. Snap and clamp, or make the query integer-valued and use an exact integer sampler.
- Always compute the discrete Gaussian's $\delta(\varepsilon)$ with Equation 24.4; the continuous formula can understate it.
- An exact distribution is not enough: running time that depends on the output is a separate side channel.

24.9. Further reading

- Mironov, "On Significance of the Least Significant Bits for Differential Privacy" (CCS 2012): the floating-point attack and the snapping mechanism.
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020): the discrete Gaussian's guarantees and the exact samplers; the reference implementation to compare a library against.
- Jin, McMurtry, Rubinstein & Ohrimenko, "Are We There Yet? Timing and Floating-Point Attacks on Differential Privacy Systems" (IEEE S&P 2022): what still goes wrong in modern libraries, including timing.
- Kairouz, Liu & Steinke, "The Distributed Discrete Gaussian Mechanism for Federated Learning with Secure Aggregation" (ICML 2021): the discrete Gaussian when each client adds its own noise.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

25. Local, shuffle and distributed trust models

25.1. Why this matters

Every mechanism so far in this part assumed a **trusted curator** that holds the raw data and releases only a mechanism's output. The guarantee of differential privacy (DP) covers that output, not the curator's database, its engineers or a court order. Often the curator is exactly the party users don't want to see their data: a browser vendor collecting usage statistics, a keyboard app learning new words. So what can you trust instead, and what does it cost in accuracy?

The answer turns on *where the noise enters*. The four models of Figure 25.1 all release a noisy statistic; they differ in who sees the data before the noise is added. The further upstream it enters, the fewer parties you trust and the more noise reaches the answer. A report randomized on a person's device must hide its owner from everyone, so it carries about as much noise as a curator adds once to the whole answer. Sum n such reports and their variances add: the noise grows like $\sqrt{n}$ while the count grows like n. The **shuffle** and **distributed** models move the noise back downstream without handing the data to anyone: anonymity hides which report is whose, and cryptography hides everything but the sum.

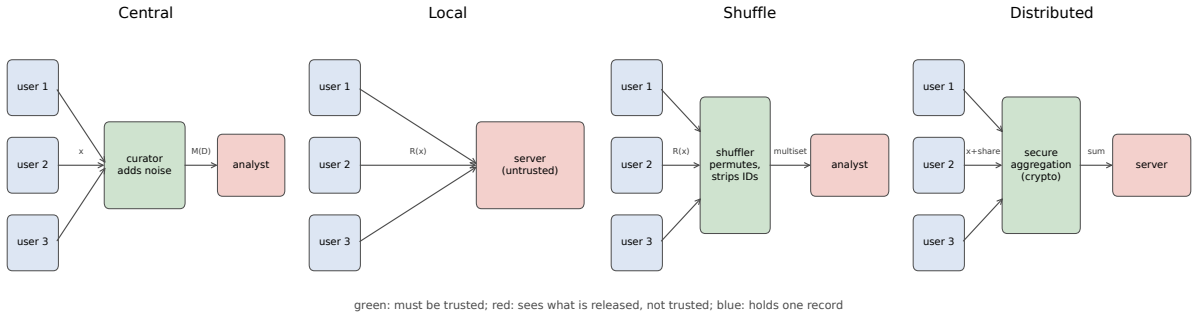


Figure 25.1.: The four trust models. Blue boxes hold one record each, green boxes must be trusted, and red boxes see what is released and are not trusted. Only the central model gives raw records to a single party.

25.2. Central and local differential privacy

In the **local model** each person runs a **local randomizer** R on their own record and sends only the result. The receiver may know whose report it is, so the report must look about the same whatever the record: for any two records x, x' and any set S of reports,

$$\Pr[R(x) \in S] \leq e^{\varepsilon_0} \Pr[R(x') \in S]. \tag{25.1}$$

Changing one record changes one report, so the collection is ε_0-DP against everyone, the collector included: a breach of its database reveals only reports, which are already private. The neighbours are **replace** neighbours, because the number of reports already shows who took part.

The basic local randomizer is **randomized response**: report your true bit with probability $t = e^{\varepsilon_0}/(1+e^{\varepsilon_0})$, the flipped bit otherwise (The definition of differential privacy in the full edition). The lying pulls the mean report towards one half by a known amount, and undoing the pull estimates the fraction of ones without bias, with standard deviation

$$\mathrm{sd}(\hat{\pi}) = \frac{e^{\varepsilon_0/2}}{(e^{\varepsilon_0}-1)\sqrt{n}}. \tag{25.2}$$

Of $n = 10{,}000$ people, say 3,000 hold a 1, and each answers at $\varepsilon_0 = \ln 3$. The estimate's standard deviation is 0.00866, or 86.6 people on the count. A trusted curator adding Laplace noise of scale $1/\varepsilon$ to the count has standard deviation 1.287: same people, same ε, and the local survey is 67.3 times noisier.

That is the model's fault, not randomized response's. Squeezing every report into a probability band e^{ε_0} wide shrinks what it says about the population by a factor of order $(e^{\varepsilon_0}-1)^2$, about ε_0^2, so you need that many times more people to learn the same thing. No local protocol beats error of order $1/((e^{\varepsilon_0}-1)\sqrt{n})$ on a fraction (Duchi, Jordan & Wainwright 2018), and randomized response is within a constant of it. A curator's error is of order $1/(n\varepsilon)$, so the ratio grows like $\sqrt{n}$ (Figure 25.2). For a standard deviation of 0.001 at $\varepsilon = 1$, randomized response needs 920,674 people and the central Laplace mechanism 1,415.

So local DP suits frequent items in very large populations, as in Google's RAPPOR in Chrome and Apple's telemetry. Since the guarantee is about a report's content, a network address stored beside it still links it to the person. And ε_0 is per report: a user who reports daily releases a new output about the same record each day, and by composition the losses add (Post-processing, group privacy and composition in the full edition).

25.3. Frequency estimation

The workhorse of deployed local DP is **frequency estimation**: each person holds one of k values, and the analyst wants the fraction f_v holding each value v. In every scheme in use, a **frequency oracle**, a report "supports" v with probability p if the person holds v and a smaller q otherwise. The share of supporting reports is then $q + f_v(p-q)$ on average, so with C_v supporting reports, subtracting q and dividing by $p-q$ gives an unbiased estimate, with exact variance

$$\hat{f}_v = \frac{C_v/n - q}{p-q}, \qquad \mathrm{Var}\,\hat{f}_v = \frac{f_v p(1-p) + (1-f_v)q(1-q)}{n(p-q)^2}. \tag{25.3}$$

Design comes down to making the gap $p-q$ large at a given ε_0, since its square divides the variance.

k-ary randomized response reports the true value with probability $p = e^{\varepsilon_0}/(e^{\varepsilon_0}+k-1)$ and each other value with $q = 1/(e^{\varepsilon_0}+k-1)$. The ratio p/q is the allowed e^{ε_0}, but both shrink as k grows: with many values the report is nearly uniform, and a rare value's variance grows linearly in k. **Optimized unary encoding** (Wang et al. 2017) sends a one-hot vector of k bits, each randomized. Changing a person's value changes only two bits, whatever k is, so the variance

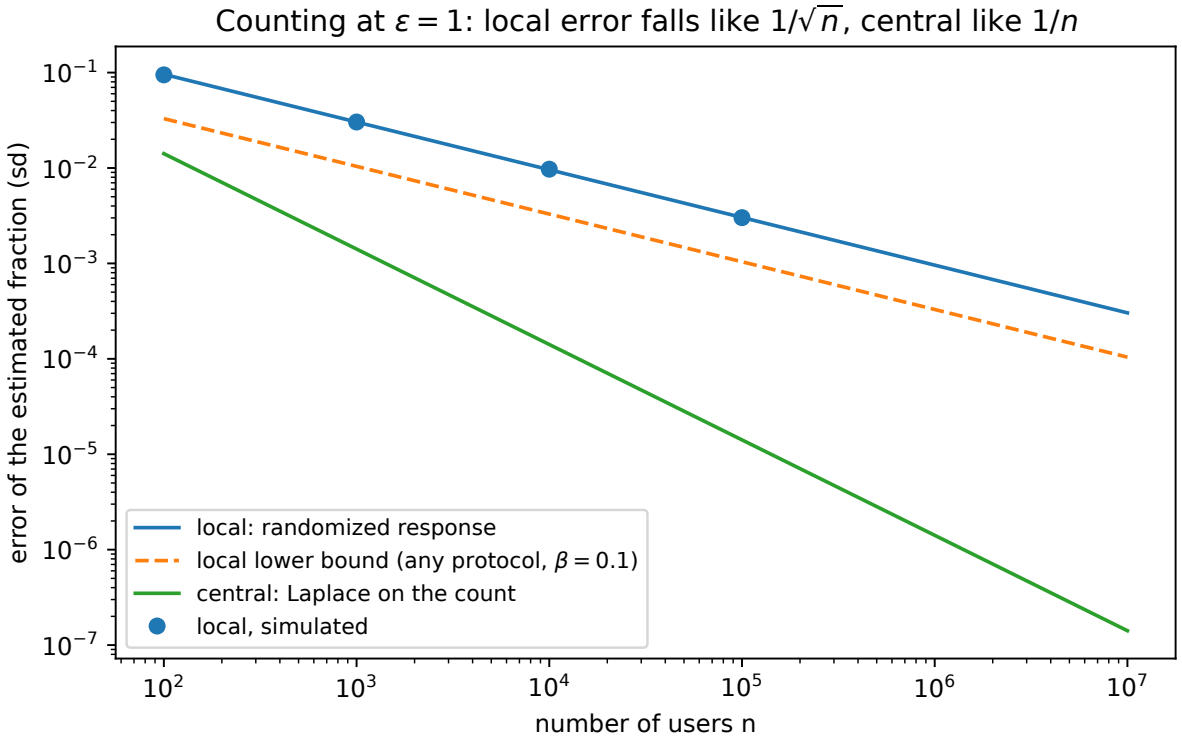


Figure 25.2.: Standard deviation of an estimated fraction at $\varepsilon = 1$, against the number of people. Randomized response (line and simulated dots) falls like $1/\sqrt{n}$ and stays a constant factor above the lower bound for any local protocol; the central Laplace mechanism falls like $1/n$.

doesn't depend on k; the price is a k-bit report. It keeps a true 1 with probability $p = \frac{1}{2}$ but invents a 1 with only $q = 1/(e^{\varepsilon_0} + 1)$, because almost all bits are 0 and invented 1s are the noise that hurts most. Setting $f_v \to 0$ in Equation 25.3 gives each scheme's noise floor,

$$k\text{-ary: } \frac{e^{\varepsilon_0} + k - 2}{n(e^{\varepsilon_0} - 1)^2}, \qquad \text{optimized unary: } \frac{4e^{\varepsilon_0}}{n(e^{\varepsilon_0} - 1)^2}, \tag{25.4}$$

and k-ary randomized response wins exactly when $k < 3e^{\varepsilon_0} + 2$ (Figure 25.3).

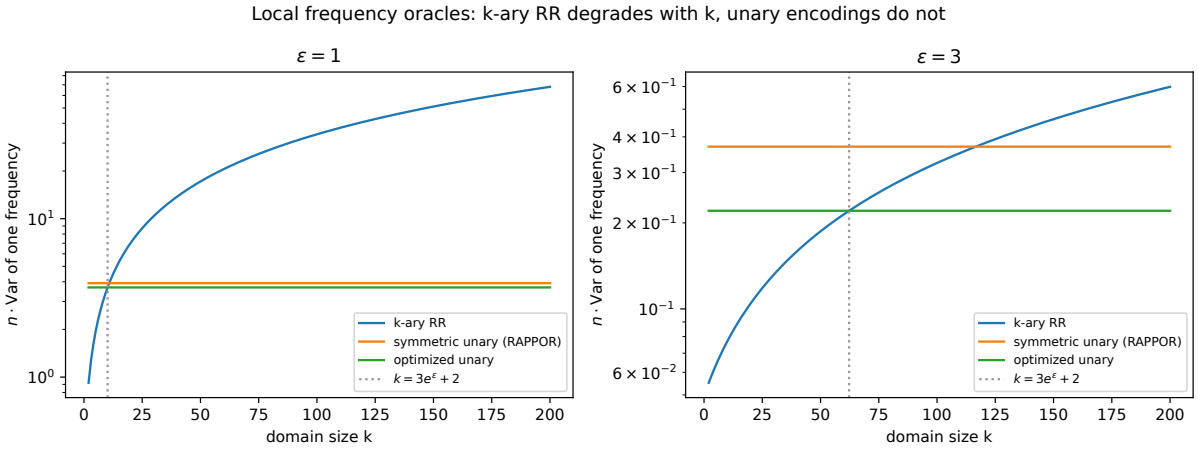


Figure 25.3.: n times the variance of a rare value's estimated frequency, against the domain size k, at $\varepsilon_0 = 1$ and $\varepsilon_0 = 3$. k-ary randomized response grows linearly in k; the unary encodings are flat; the dotted line is the crossover $k = 3e^{\varepsilon_0} + 2$.

A keyboard app wants the frequency of $k = 128$ emoji at $\varepsilon_0 = 1$ from 10^6 users. A rare emoji's standard deviation is 0.0066 under k-ary randomized response and 0.0019 under optimized unary encoding. An estimate must stand about three standard deviations above zero to be seen, so emoji used by fewer than about 0.6% of users are lost either way: these oracles are optimal local protocols, and the $\sqrt{n}$ gap remains. For large k, hashing schemes (local hashing, Hadamard response) reach the unary variance with reports of about $\log k$ bits. And because of composition, RAPPOR (Erlingsson, Pihur & Korolova 2014) memoizes one permanent randomized response per value and adds fresh noise around it in each report, so averaging a user's daily reports recovers at most the permanent randomized value, not the value itself.

25.4. The shuffle model

The local model loses $\sqrt{n}$ because the analyst knows whose report is whose. In the **shuffle model** a **shuffler** strips identifiers and timing and forwards the reports in random order, so the analyst sees what was sent but not by whom. It is trusted only to unlink: it sees reports, which are already ε_0-DP.

Anonymity helps because of what Equation 25.1 says about everyone else. Another person's report is at most e^{ε_0} times less likely than if they held the target's value, so with probability about $e^{-\varepsilon_0}$ each of them sends a report that looks exactly like one the target might have sent. The target hides among about $ne^{-\varepsilon_0}$ such clones, and more people means more clones (Amplification by

shuffling and by iteration). Theorem 3.1 of Feldman, McMillan & Talwar (2021) makes this a bound: shuffling n reports from ε_0-DP local randomizers is (ε, δ)-DP under replace neighbours with

$$\varepsilon \le \ln\left(1 + \frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1}\left(\frac{8\sqrt{e^{\varepsilon_0}\ln(4/\delta)}}{\sqrt{n}} + \frac{8e^{\varepsilon_0}}{n}\right)\right), \tag{25.5}$$

provided $\varepsilon_0 \le \ln\left(n/(16\ln(2/\delta))\right)$. The bracket falls like $1/\sqrt{n}$, the clones at work. The proviso, rewritten as $ne^{-\varepsilon_0} \ge 16\ln(2/\delta)$, demands enough expected clones; below it the theorem says nothing.

The payoff is that each report can be randomized weakly. For central $\varepsilon = 1$ at $\delta = 10^{-6}$, bisection on Equation 25.5 (Section 25.7) allows $\varepsilon_0 = 3.504$ at $n = 10^4$, 5.701 at 10^5 and 7.992 at 10^6. At 7.992 a single report is almost the truth, yet the shuffled collection is $(1, 10^{-6})$-DP. Because the allowed e^{ε_0} grows in proportion to n, the error returns to the central rate up to a factor $\sqrt{\ln(1/\delta)}$. At $n = 10^3$ there are too few clones: the proviso caps ε_0 at 1.460.

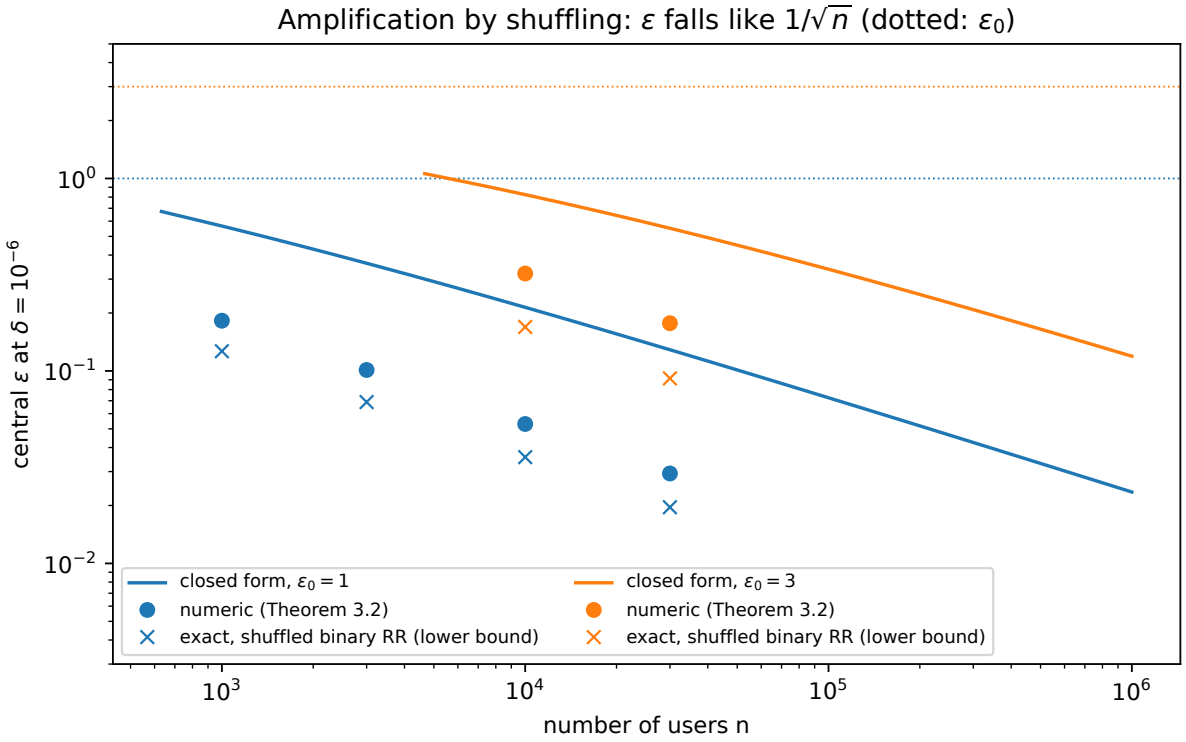


Figure 25.4.: Central ε at $\delta = 10^{-6}$ after shuffling, against n, for $\varepsilon_0 = 1$ and $\varepsilon_0 = 3$: the closed form of Feldman, McMillan & Talwar (lines, drawn where its condition holds), their numeric bound (dots) and the exact ε of shuffled binary randomized response (crosses). The dotted lines mark ε_0, the guarantee without shuffling.

The closed form is loose (Figure 25.4). The same paper computes the clone comparison numerically, and that bound is still valid for every ε_0-DP randomizer: at $n = 10^4$ and $\varepsilon_0 = 1$ it gives 0.0530 where the closed form gives 0.2140. Reason with the closed form, deploy with a numerical accountant.

The amplification lasts only while the shuffler does its one job. If it colludes with the analyst,

the guarantee falls back to the local ε_0, which you just made large, so keep it reasonable. If the adversary controls m senders, it can remove their known reports, leaving $n - m$ people to provide clones. And the theorem covers only the set of reports, so timing, message sizes and counts must not leak, which is why real shufflers batch and pad.

25.5. Distributed DP

The **distributed model** adds the central amount of noise once, in pieces. Independent Gaussians add up to a Gaussian whose variance is the sum of theirs, so if each of n clients adds $\mathcal{N}(0, \sigma^2/n)$, the sum carries exactly the $\mathcal{N}(0, \sigma^2)$ of the central Gaussian mechanism (The Laplace and Gaussian mechanisms in the full edition). A **secure aggregation** protocol (Bonawitz et al. 2017), built on additive secret sharing (Cryptography basics), shows the server nothing it couldn't compute from the sum, so its view is as private as the central mechanism's output.

Each client can't add the full σ^2 to be safe: the sum would then carry $n\sigma^2$, the local model again. The small shares make the accuracy central, and they are also the weakness, since no share is enough noise alone. If the adversary knows the noise of t clients, because they collude or dropped out, it subtracts their shares, leaving a Gaussian mechanism with noise

$$\sigma\sqrt{\frac{n-t}{n}}. \tag{25.6}$$

So over-provision: to tolerate a fraction ρ of such clients, each adds variance $\sigma^2/(n(1-\rho))$, and the honest ones alone still supply σ^2.

A server aggregates a histogram through 100 clients at $(1, 10^{-6})$-DP. Under replace a changed value moves two counts by 1, so the ℓ_2 sensitivity is $\sqrt{2}$, and the exact calibration gives total noise $\sigma = 5.975$, or 0.597 per client. If 20% of clients collude with the server, the delivered guarantee is $\varepsilon = 1.128$; at 50%, 1.455; with only 10% honest, 3.525 (Figure 25.5). Nobody outside can see how many are honest, so decide the fraction up front.

Secret shares can be uniformly random only in a finite set, so secure aggregation works on integers modulo some q, chosen large enough that the true sum never wraps around. Clients round their values and add integer noise whose sum still has a known distribution. **Skellam** noise, a difference of two Poisson variables, sums exactly to Skellam noise because Poissons add, and its privacy is close to a Gaussian's (Agarwal, Kairouz & Liu 2021); discrete Gaussian shares sum approximately to a discrete Gaussian (Kairouz, Liu & Steinke 2021). Laplace noise doesn't split: n shares of $\mathrm{Lap}(b/n)$ sum to variance $2b^2/n$, not $2b^2$. And secure aggregation hides inputs without checking them: a malicious client can send a vector with no noise, and bounding that needs zero-knowledge proofs or verifiable aggregation.

25.6. Comparing the four models

To choose, fix the central guarantee and compare errors. The full edition's script estimates an 8-value histogram at $(1, 10^{-6})$ in all four models, each with its best mechanism from this chapter (Table 25.1, Figure 25.6).

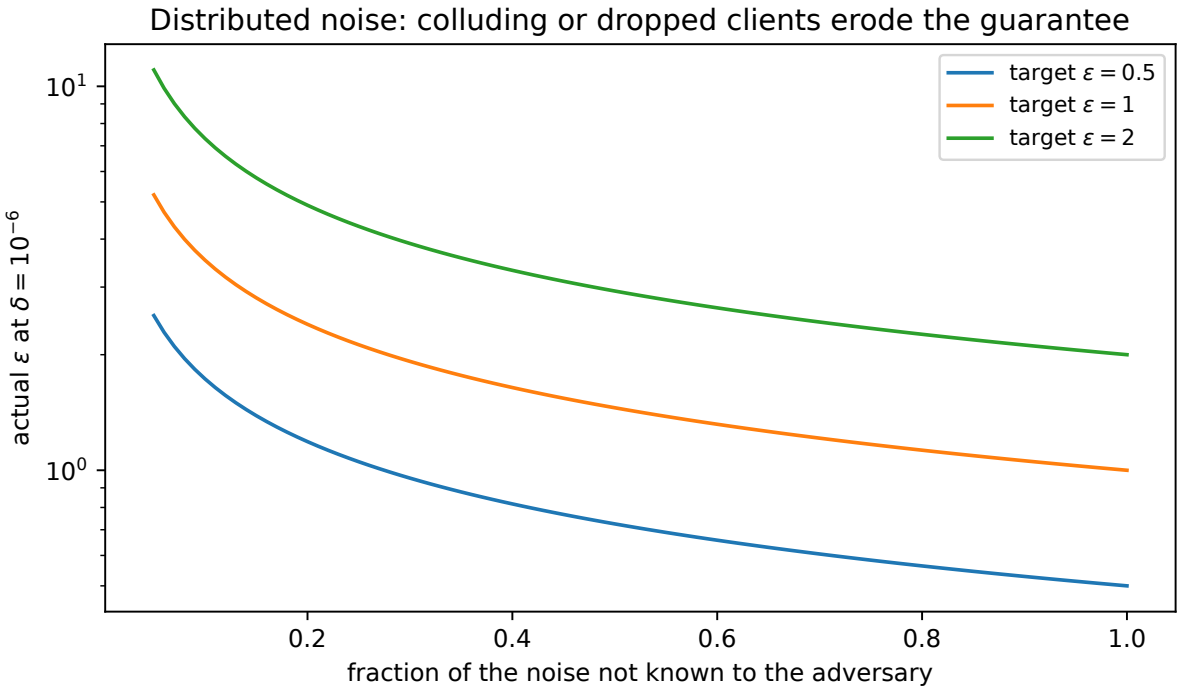


Figure 25.5.: The actual ε at $\delta = 10^{-6}$ of distributed Gaussian noise calibrated for a target ε, against the fraction of the total noise that the adversary does not know. At fraction 1 every client is honest and the target is met; below it, colluding or dropped clients leave less noise than planned.

Table 25.1.: Expected squared ℓ_2 error of an 8-value histogram of fractions at the same central $(\varepsilon, \delta) = (1, 10^{-6})$ in all four trust models.

n	Central	Distributed	Shuffle	Local
10^3	6.40×10^{-5}	2.86×10^{-4}	9.35×10^{-3}	2.71×10^{-2}
10^4	6.40×10^{-7}	2.86×10^{-6}	4.88×10^{-5}	2.71×10^{-3}
10^5	6.40×10^{-9}	2.86×10^{-8}	4.76×10^{-7}	2.71×10^{-4}

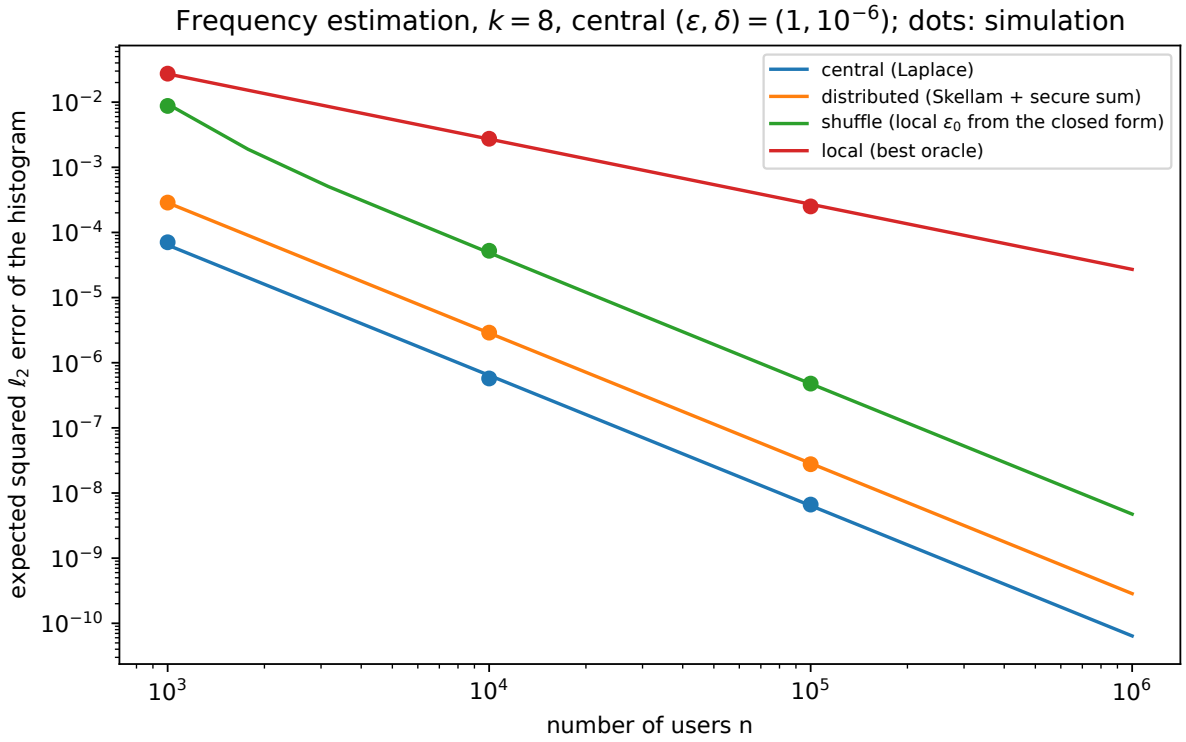


Figure 25.6.: Expected squared ℓ_2 error of an 8-value histogram against the number of people, at the same central $(\varepsilon, \delta) = (1, 10^{-6})$ in all four trust models. Lines are exact; dots are simulations. Central and distributed fall like $1/n^2$, local like $1/n$; shuffle bends from the local to the central slope as n grows.

Central and distributed squared errors fall like $1/n^2$, because their noise is fixed while counts grow; local error falls like $1/n$, so at $n = 10^5$ it is about 42,000 times the central one. Shuffle moves towards central as the allowed ε_0 grows with n: at $n = 10^4$ it is about 56 times better than local. Distributed is about 4.5 times the central Laplace error, because a Gaussian pays for its δ with a wider spread; it gains only in high dimension, where its ℓ_2 calibration beats Laplace's ℓ_1.

Every column has the same (ε, δ) against a different adversary: only the local model protects against the collector. A report of "$\varepsilon = 1$" without its trust model can't be compared with anything.

25.7. Code

Two calculations recur in local and shuffled deployments: which frequency oracle to use, and how large a local ε_0 the shuffler allows.

```
from math import exp, log, log1p, sqrt


def rare_variance_krr(eps0: float, k: int) -> float:
    """n times the variance of a rare value's frequency under k-ary randomized response."""
    return (exp(eps0) + k - 2) / (exp(eps0) - 1) ** 2


def rare_variance_oue(eps0: float) -> float:
    """n times the variance of a rare value's frequency under optimized unary encoding."""
    return 4 * exp(eps0) / (exp(eps0) - 1) ** 2


def shuffle_cap(n: int, delta: float) -> float:
    """The largest eps0 the shuffling theorem covers: ln(n / (16 ln(2/delta)))."""
    return log(n / (16 * log(2 / delta)))


def shuffle_epsilon(eps0: float, n: int, delta: float) -> float:
    """Central epsilon after shuffling n eps0-DP reports (Feldman, McMillan & Talwar 2021)."
    if eps0 > shuffle_cap(n, delta):
        raise ValueError("eps0 is above the theorem's cap")
    e0 = exp(eps0)
    bracket = 8 * sqrt(e0 * log(4 / delta)) / sqrt(n) + 8 * e0 / n
    return log1p((e0 - 1) / (e0 + 1) * bracket)


def max_local_epsilon(eps: float, n: int, delta: float) -> float:
    """The largest eps0 whose shuffled reports are (eps, delta)-DP by the closed form."""
    lo, hi = 0.0, shuffle_cap(n, delta)
    if shuffle_epsilon(hi, n, delta) <= eps:
        return hi
    for _ in range(100):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if shuffle_epsilon(mid, n, delta) <= eps else (lo, mid)
    return lo


print(f"{rare_variance_krr(1.0, 128):.3f} {rare_variance_oue(1.0):.3f}")  # 43.596 3.683
print(f"{shuffle_epsilon(1.0, 10_000, 1e-6):.4f}")  # 0.2140
print(" ".join(f"{max_local_epsilon(1.0, n, 1e-6):.3f}" for n in (10**3, 10**4, 10**5, 10**6
```

The full edition's script, trust_models.py, computes every number in this chapter, including the numerical shuffle bound:

```
uv run python code/part03-dp-foundations/trust_models.py
```

25.8. Cheat sheet

Table 25.2.: The results of this chapter.

Result	Formula	Notes
Local randomizer	$\Pr[R(x) \in S] \le e^{\varepsilon_0} \Pr[R(x') \in S]$ for all x, x'	Private against everyone, the collector included; replace neighbours
Randomized response	$\mathrm{sd} = e^{\varepsilon_0/2}/((e^{\varepsilon_0}-1)\sqrt{n})$	Optimal among local protocols up to a constant
Local versus central	Local error of order $1/((e^{\varepsilon_0}-1)\sqrt{n})$; central $\sqrt{2}/(n\varepsilon)$	The gap grows like $\sqrt{n}$
Frequency oracle	$\hat{f}_v = (C_v/n - q)/(p-q)$, variance Equation 25.3	Unbiased, exact variance
k-ary randomized response	Rare variance $(e^{\varepsilon_0}+k-2)/(n(e^{\varepsilon_0}-1)^2)$	Best for $k < 3e^{\varepsilon_0}+2$
Optimized unary encoding	Rare variance $4e^{\varepsilon_0}/(n(e^{\varepsilon_0}-1)^2)$	Independent of k; k-bit reports
Shuffling	Equation 25.5, for $\varepsilon_0 \le \ln(n/(16\ln(2/\delta)))$	Loose; deploy with a numerical bound
Distributed Gaussian	n shares $\mathcal{N}(0, \sigma^2/n)$ sum to $\mathcal{N}(0, \sigma^2)$	t colluders leave $\sigma\sqrt{(n-t)/n}$

Rules of thumb

- Local DP costs a factor of about $\sqrt{n}$ in error over central DP; no local protocol avoids it.
- A shuffler lets each report use a large ε_0; compute the central ε numerically before deploying.
- State the trust model next to (ε, δ).

25.9. Further reading

- Wang, Blocki, Li & Jha, "Locally Differentially Private Protocols for Frequency Estimation" (USENIX Security 2017): frequency oracles, optimized unary encoding and local hashing, compared.

- Feldman, McMillan & Talwar, "Hiding Among the Clones" (FOCS 2021): the closed form used here and the numerical bound to deploy with.
- Kairouz, Liu & Steinke, "The Distributed Discrete Gaussian Mechanism for Federated Learning with Secure Aggregation" (ICML 2021): distributed noise end to end, with integer noise and modular arithmetic.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

26. Lower bounds: what privacy costs

26.1. Why this matters

Someone hands you a design for a private release: a histogram over a million locations with a maximum error of five records at $\varepsilon = 1$, say. Before reading their code, you can check whether that accuracy is possible at all. The earlier chapters gave **upper bounds**, what one mechanism achieves. A **lower bound** says that *no* mechanism satisfying differential privacy (DP) can do better. When it matches a known mechanism, that mechanism is optimal, and more accuracy can only come from more data or less privacy. When a claim beats a lower bound, the claim is wrong.

How can anything be said about every mechanism, when you can't try them all? By using only the two things you know about it: it is accurate, and it is private. **Accuracy is an attack**: an accurate output reveals something about the data, which of many datasets was used, whether a person is in it, what their secret bits are. DP caps every attack. If an accurate mechanism would hand an attacker more than DP allows, it can't be both, and solving that clash for the error or for the number of records n gives the bound. Throughout, the error is at most α with probability at least $1 - \beta$, and the examples use $\varepsilon = 1$, $\beta = 0.05$ and, for approximate DP, $\delta = 10^{-6}$.

26.2. Pure DP pays for the number of candidates

Suppose the output must point to one of m candidates, such as the most common of a million ZIP codes. For each candidate there is a dataset on which an accurate answer points to it with probability at least $1 - \beta$. Now run the mechanism on one reference dataset instead, say the empty one. Its output probabilities add up to 1, so most candidates get only a small share. But pure DP extended to datasets k records apart (**group privacy**, Post-processing, group privacy and composition in the full edition) says no event becomes more than $e^{k\varepsilon}$ times less likely, so every candidate keeps a share of at least about $e^{-k\varepsilon}$ on the reference. A budget of 1 can't cover m such demands unless k is large. This counting argument is **packing**: if each candidate's dataset is within k records of the reference, then

$$k \geq \frac{\ln\left(m(1-\beta)\right)}{\varepsilon}. \tag{26.1}$$

Two consequences follow under add/remove neighbours. To report the **mode** of N elements reliably, the top element must lead by at least $\ln(N(1-\beta))/\varepsilon$ records. And a histogram accurate to α in every bin can find a spike of height more than 2α, so any ε-DP histogram over N bins has

$$\alpha \geq \frac{1}{2}\left(\frac{\ln\left(N(1-\beta)\right)}{\varepsilon} - 1\right). \tag{26.2}$$

The Laplace histogram has error at most $\ln(N/\beta)/\varepsilon$, since each of N noises exceeds α with probability $e^{-\alpha\varepsilon}$, and report-noisy-max (Selection in the full edition) finds the mode at a lead of the same order. Both grow like $\ln N/\varepsilon$, so both are optimal within a factor of about 2 (Figure 26.1). The factor is mostly slack in the argument: the argument counts only the reference output's mass inside candidates' regions, but for Laplace that output is pure noise around zero and almost none of it lands in any region. That wasted mass is the factor of 2.

For $N = 10^6$ locations, the most common one must lead by at least 13.76 records before any ε-DP mechanism can report it reliably; report-noisy-max needs 18.44. Every ε-DP histogram has, at the 95% level, an error of at least 6.38 in some bin; Laplace has 16.79.

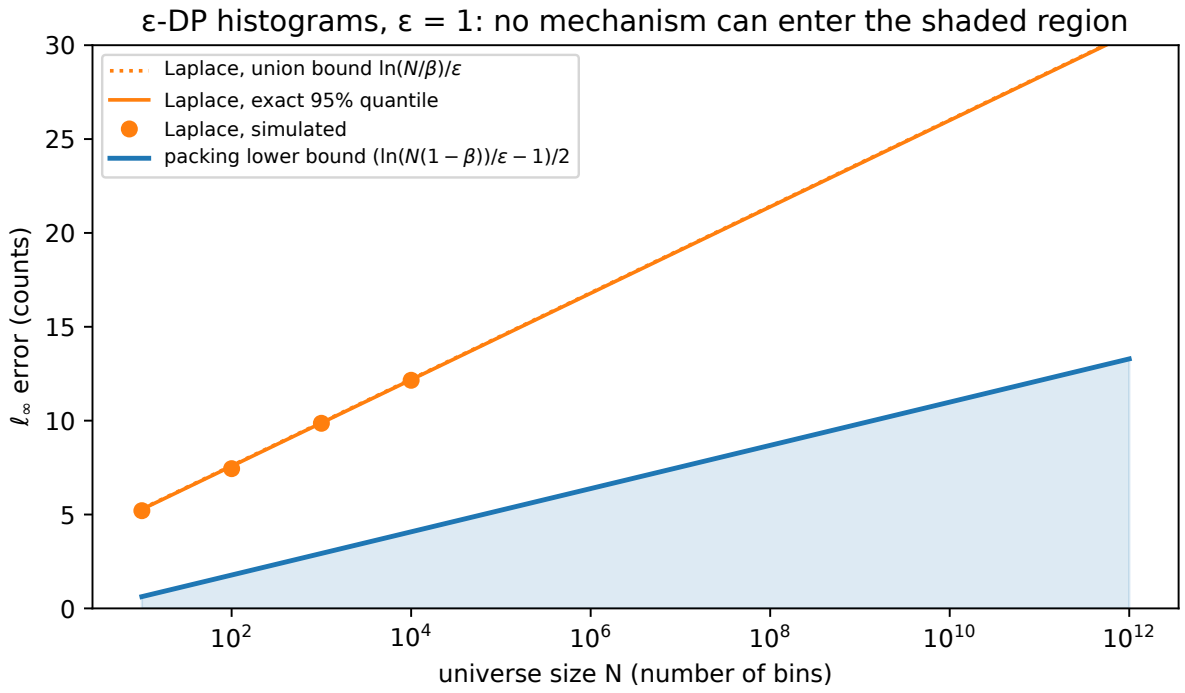


Figure 26.1.: ℓ_∞ error of ε-DP histograms over N bins at $\varepsilon = 1$, failure probability 0.05. Orange: the Laplace mechanism, exact (line), union bound (dotted, indistinguishable) and simulated (dots). Blue: the packing lower bound; no ε-DP mechanism has an error in the shaded region. Both grow like $\ln N$.

Packing needs $\delta = 0$. With $\delta > 0$, group privacy over k records carries an additive slack that grows with k, while the share $e^{-k\varepsilon}$ shrinks; past about $\ln(1/\delta)/\varepsilon$ records (13.82 at $\delta = 10^{-6}$) the slack swamps it and the argument demands nothing. That is not a weak proof but a real escape. The stability-based histogram (Vadhan 2017) adds noise only to non-empty bins and releases one only if its noisy count clears a threshold of order $\ln(1/\delta)/\varepsilon$. Empty bins are never released, so nothing pays for the N bins, and its error doesn't grow with N. For a huge, mostly empty universe, approximate DP is how you escape the $\ln N$.

26.3. How many records privacy costs

An error of 6 records is large in a dataset of 50 and negligible in a million, so dividing by n turns a bound on error into a bound on records. That lets you compare privacy's cost with one you pay anyway, sampling error. For a histogram, an error of α in every frequency needs

$$n \geq \frac{\ln\left(N(1-\beta)\right) - \varepsilon}{2\alpha\varepsilon} \tag{26.3}$$

records under ε-DP, while sampling alone needs up to $\ln(2N/\beta)/(2\alpha^2)$. The private term grows like $1/\alpha$ and the statistical one like $1/\alpha^2$, so privacy adds a term that dominates only for coarse answers at strong privacy, $\alpha \gtrsim \varepsilon$. Over 10^6 cells to within $\alpha = 0.1$, privacy needs at least 64 records and sampling up to 875; at $\alpha = 0.01$, 638 and 87,522.

With many attributes, pure and approximate DP part ways. To release the means of d yes/no attributes, coded ± 1, each within α, a pure-DP mechanism faces 2^d candidates, one per corner of the cube, and packing charges $\ln 2^d$: about $d \ln 2/(\alpha\varepsilon)$ records, linear in d. The Gaussian mechanism needs only $2\sqrt{d}\, z/(\mu\alpha)$, where z is the normal quantile with tail $\beta/(2d)$ and μ is the largest ratio of sensitivity to noise the exact calibration allows (0.2367 at $\varepsilon = 1$, $\delta = 10^{-6}$; The Laplace and Gaussian mechanisms in the full edition). The $\sqrt{d}$ comes from calibrating to the ℓ_2 sensitivity, $2\sqrt{d}/n$, instead of the ℓ_1 sensitivity, $2d/n$, which is possible only with $\delta > 0$. There is no contradiction with packing, which stops working at $\delta > 0$.

Is $\sqrt{d}$ itself unavoidable, or could a cleverer mechanism do better? Packing can't answer, so **fingerprinting** (Bun, Ullman & Vadhan 2014) looks only at neighbours, where δ stays small. Draw a random dataset and release its means accurately. To follow the true means the release must follow the data, so it correlates with the records, by a total amount of order d shared among n records: each member's correlation with the release is of order d/n. A stranger's correlation is random, with spread of order $\sqrt{d}$. DP lets a member stand out from a stranger by only a constant multiple of that spread, so $d/n \lesssim \sqrt{d}$, which is $n \gtrsim \sqrt{d}$. The tight version is n of order $\sqrt{d\ln(1/\delta)}/(\alpha\varepsilon)$ (Steinke & Ullman 2016): the Gaussian mechanism is optimal up to a logarithm.

Table 26.1.: Records needed to release d attribute means with ℓ_∞ error 0.1 (on the scale $[-1, 1]$) with probability 0.95, at $\varepsilon = 1$ and, for the Gaussian, $\delta = 10^{-6}$.

	Pure DP, packing (lower)	Pure DP, Laplace (upper)	Approximate DP, Gaussian (upper)
$d = 100$	683	15,202	2,941
$d = 1000$	6,921	198,070	10,836
$d = 10{,}000$	69,304	2,441,215	38,570
$d = 100{,}000$	693,137	29,017,315	134,299

In Table 26.1 the packing column grows tenfold with each tenfold increase in d, the Gaussian column only about $\sqrt{10}$-fold, so it must cross below: from $d = 2731$ attributes on, the Gaussian mechanism needs fewer records than *any* pure-DP mechanism (Figure 26.2). A model with a few thousand parameters is already past that point, which is one reason private machine learning uses the Gaussian mechanism.

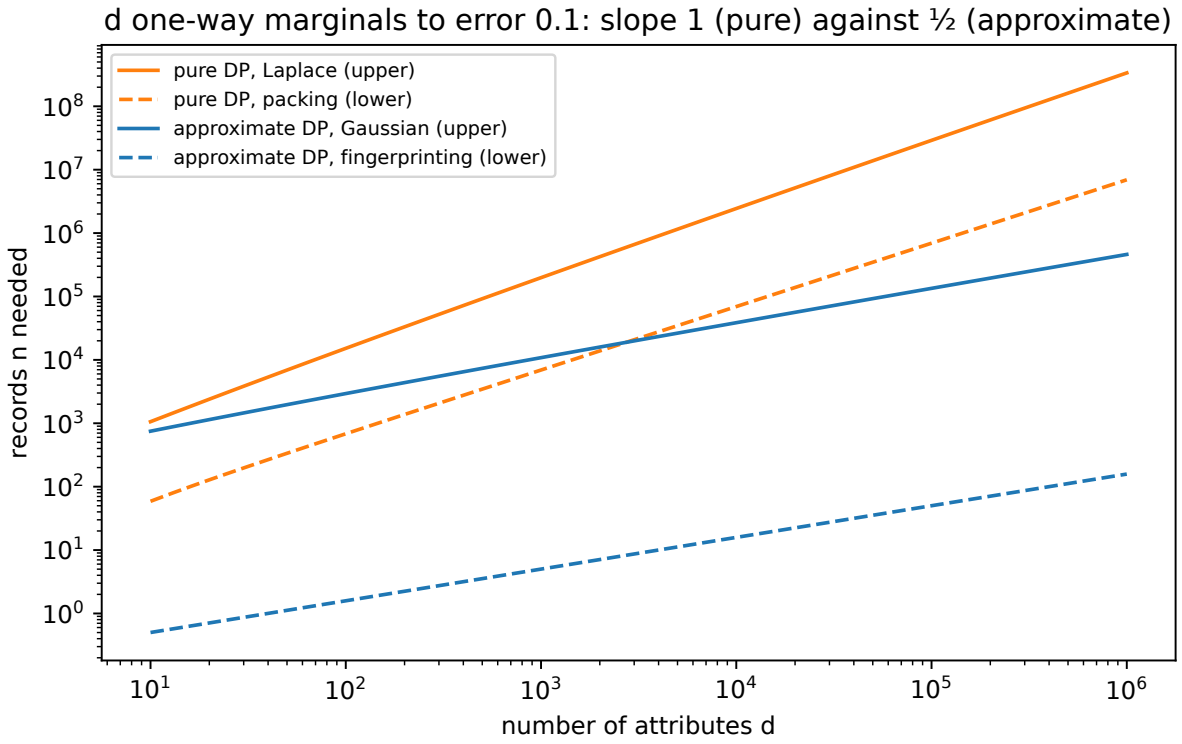


Figure 26.2.: Records needed to release d attribute means to ℓ_∞ error 0.1 with probability 0.95, at $\varepsilon = 1$ and $\delta = 10^{-6}$, on log–log axes. Orange: pure DP (Laplace upper bound, packing lower bound), both of slope 1. Blue: approximate DP (Gaussian upper bound, fingerprinting lower bound), both of slope ½. The constants of the fingerprinting bound are loose; the slopes are the point.

Use the fingerprinting bound for its rate, not its numbers: in the simple form the full edition proves, it says $n \geq 5.0$ at $d = 1000$, against 10,836 for the Gaussian mechanism. Its δ term is harmless only for small δ, and that is no artefact: publishing each record with probability δ is $(0, \delta)$-DP, so with δ near $1/n$ no bound of this kind could hold. And "privacy is only an additive term" fails for small subgroups, where n is the subgroup's size.

26.4. The lower bound is a membership inference attack

The fingerprinting argument is an attack you can run. In 2008 Homer et al. showed that from the published allele frequencies of a genome-wide association study you could tell whether a given person's DNA was in it, and their test is, up to details, the fingerprinting score. Given the released means $\hat{q}$ of d attributes, with Gaussian noise σ per coordinate, the population means p and a target record y, the attacker computes $Z(y) = \langle \hat{q}, y - p \rangle$ and says "member" when it is large. Each member pulls the release towards its own record by $1/n$ of its deviation from the population, so members' scores are shifted up; a stranger's record is independent of the release, so its score is centred at zero. With attribute means drawn uniformly, the shift and the spread of the scores are

$$\text{gap} = \frac{2d}{3n}, \qquad \text{variance} = d\left(\frac{2}{15} + \frac{8}{15n} + \frac{2\sigma^2}{3}\right). \tag{26.4}$$

The ratio of gap to spread decides the attack. Without noise it is of order $\sqrt{d}/n$, so tracing works once d is large next to n^2. Noise adds $2\sigma^2/3$ to the variance per attribute, and the ratio becomes of order $\sqrt{d}/(n\sigma)$, so the attack fails once $\sigma \gg \sqrt{d}/n$. That is the order of noise the Gaussian mechanism adds, as it must be for a mechanism both private and as accurate as fingerprinting allows.

The full edition releases the means of $n = 50$ records with $d = 2000$ attributes. Without noise the gap is 26.67 and the spread 16.97, and a quarter of the members (a **true positive rate** (TPR) of 0.242) are identified at a **false positive rate** (FPR) of 1%. With the noise of a $(1, 10^{-5})$-DP Gaussian mechanism, $\sigma = 6.67$, the TPR at 1% FPR drops to 0.014. DP guarantees no more: any test fires on members at most e^{ε} times as often as on strangers, plus δ, so $\text{TPR} \leq e^{\varepsilon}\,\text{FPR} + \delta = 0.0272$. The attack fades around $\sigma = \sqrt{d}/n = 0.89$ (Figure 26.3, Figure 26.4).

So aggregates are not safe because they are aggregates: what protects members is the ratio of n to $\sqrt{d}$, or noise. This is the attack of Membership inference on statistics instead of models, and it is reported the same way, as TPR at a low FPR, because identifying a few people with confidence is already a failure. When auditing with this score, take p from a separate reference sample: estimating it from the release itself subtracts part of each member's own pull and weakens the signal.

26.5. Reconstruction: why answers must carry $\sqrt{n}$ noise

Reconstruction attacks recover almost all of n secret bits from about n random subset counts whose error grows more slowly than $\sqrt{n}$ (Dinur & Nissim 2003): any set of bits that fits such accurate answers must agree with the truth almost everywhere. Read as a lower bound, this says such a release can't be private. Guessing one person's bit is a test between two neighbouring

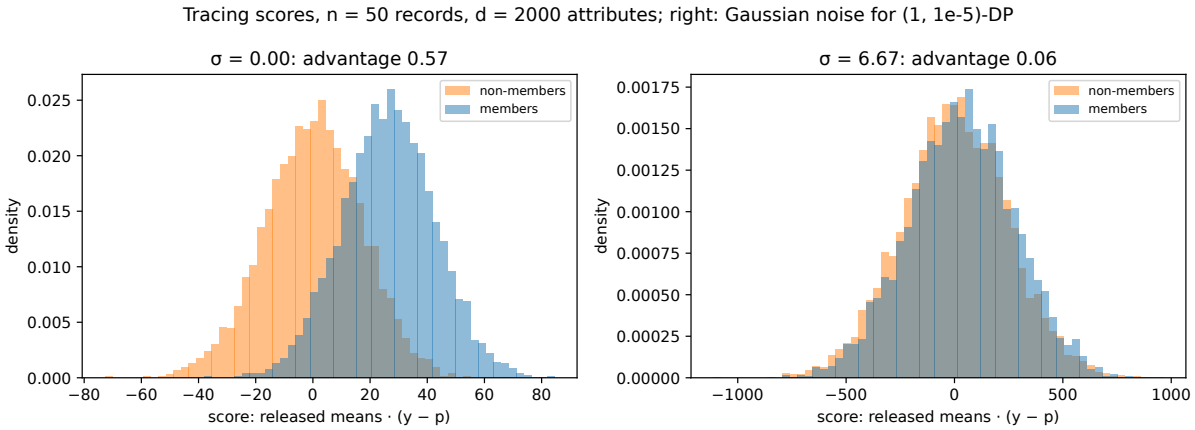


Figure 26.3.: Tracing scores of members (blue) and non-members (orange) for $n = 50$ records and $d = 2000$ attributes. Left: exact means; the members' scores are shifted by $2d/(3n) = 26.7$. Right: means released with the Gaussian noise of a $(1, 10^{-5})$-DP mechanism; the two distributions nearly coincide.

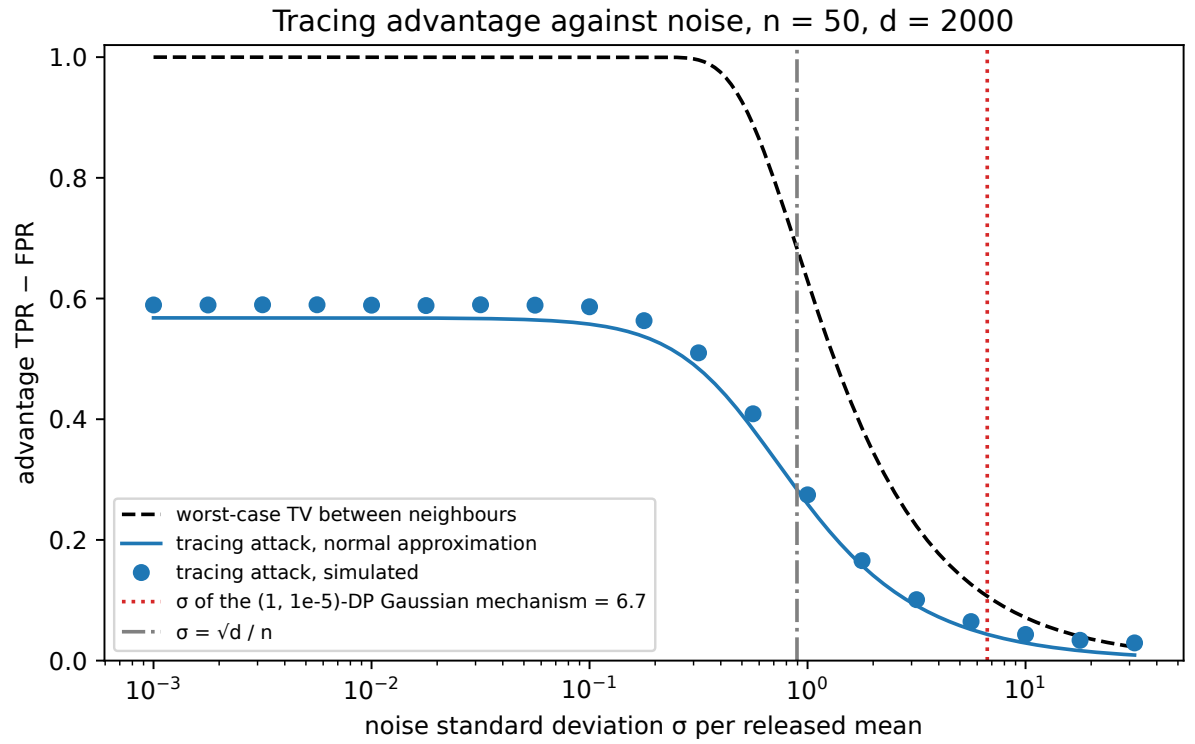


Figure 26.4.: Advantage of the tracing attack against the noise standard deviation, $n = 50$, $d = 2000$. Dots: simulation; solid line: the normal approximation; dashed: the worst-case bound on every attack. The attack fails once σ is well above $\sqrt{d}/n$; the DP Gaussian mechanism adds $\sigma = 6.7$.

datasets, so DP caps how often any guess is right, and the fraction recovered, an average of such guesses, obeys the same cap: at most $(e^\varepsilon + \delta)/(e^\varepsilon + 1)$, 73.11% at $\varepsilon = 1$. Turned around, recovering a fraction r proves

$$\varepsilon \geq \ln \frac{r - \delta}{1 - r}. \tag{26.5}$$

So answers to that many counting queries need noise of order $\sqrt{n}$, and the Gaussian mechanism matches: changing one bit moves each of k answers by at most 1, an ℓ_2 distance of at most $\sqrt{k}$, so it adds noise of order $\sqrt{k}$ per answer. Laplace, calibrated to the ℓ_1 distance k, pays of order k.

The full edition answers 800 random subset sums of 200 bits with Gaussian noise and decodes by least squares. Recovering 0.9607 of the bits proves $\varepsilon \geq 3.20$. The attack recovers 0.8058 at noise $0.5\sqrt{n}$, above the 73.11% an $\varepsilon = 1$ mechanism allows, and 0.6710 at $\sqrt{n}$, below it, so an $\varepsilon = 1$ mechanism needs noise between the two (Figure 26.5).

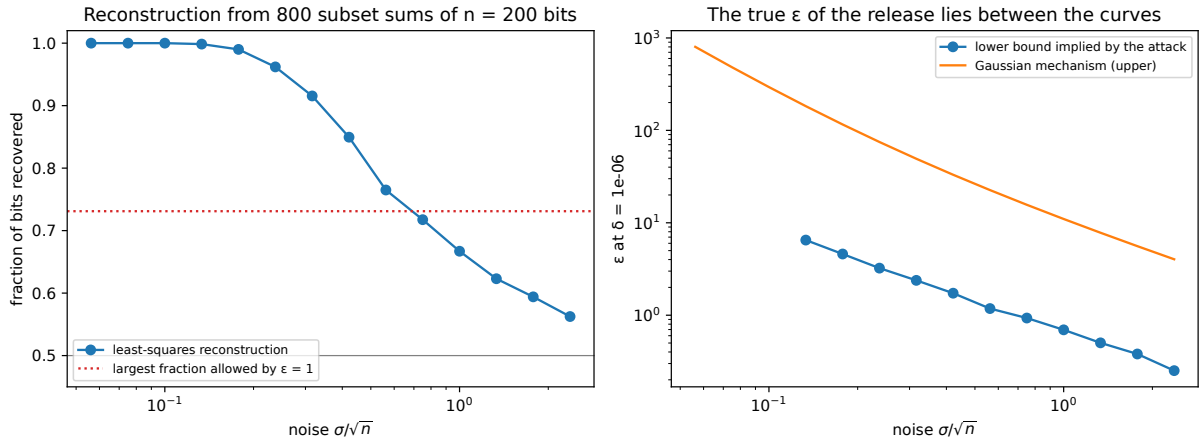


Figure 26.5.: Left: fraction of 200 secret bits recovered by least squares from 800 subset sums with Gaussian noise of standard deviation σ, against $\sigma/\sqrt{n}$. The dotted line is the most that any $\varepsilon = 1$ mechanism allows. Right: the ε that the reconstruction proves (blue) and the ε that the Gaussian mechanism guarantees (orange), at $\delta = 10^{-6}$; the true privacy loss of the release lies between them.

An attack proves only a lower bound on ε. Recovering 67% of the bits at $\sigma = \sqrt{n}$ proves $\varepsilon \geq 0.71$, but least squares is not the best attack, so it doesn't make the release $(0.71, 10^{-6})$-DP; the mechanism's analysis guarantees only $\varepsilon \leq 11.00$. The true loss lies between, and stronger attacks narrow the gap, which is what privacy auditing does.

26.6. Code

Checking a claimed accuracy is a few lines of arithmetic: the pure-DP histogram floor and the Laplace error, the records needed for d means under pure DP and with the Gaussian mechanism, and the ε a reconstruction proves.

```
from math import exp, log, sqrt
from statistics import NormalDist
```

```
PHI = NormalDist().cdf


def histogram_error_floor(bins: int, eps: float, beta: float) -> float:
    """Smallest max-bin error any eps-DP histogram can have with probability 1 - beta (packi
    return max(0.0, (log(bins * (1 - beta)) / eps - 1) / 2)


def laplace_histogram_error(bins: int, eps: float, beta: float) -> float:
    """Max-bin error the Laplace histogram reaches with probability 1 - beta (exact)."""
    return -log(1 - (1 - beta) ** (1 / bins)) / eps


def means_records_floor_pure(d: int, eps: float, alpha: float, beta: float) -> float:
    """Records any eps-DP release of d means in [-1, 1] needs for max error alpha (packing).
    return ((d * log(2) + log(1 - 2.0**-d) + log(1 - beta)) / eps - 1) / alpha


def gaussian_mu(eps: float, delta: float) -> float:
    """Largest sensitivity / sigma for which the Gaussian mechanism is (eps, delta)-DP."""
    lo, hi = 1e-6, 100.0
    for _ in range(200):
        mu = (lo + hi) / 2
        d = PHI(mu / 2 - eps / mu) - exp(eps) * PHI(-mu / 2 - eps / mu)
        lo, hi = (mu, hi) if d < delta else (lo, mu)
    return lo


def means_records_gaussian(d: int, eps: float, delta: float, alpha: float, beta: float) -> f
    """Records the Gaussian mechanism needs for the same release (an upper bound)."""
    z = NormalDist().inv_cdf(1 - beta / (2 * d))
    return 2 * sqrt(d) * z / (gaussian_mu(eps, delta) * alpha)


def epsilon_proved(recovered: float, delta: float) -> float:
    """Lower bound on epsilon from an attack that recovers this fraction of uniform secret b
    return log((recovered - delta) / (1 - recovered))


print(f"{histogram_error_floor(10**6, 1.0, 0.05):.2f} {laplace_histogram_error(10**6, 1.0, 0
print(f"{means_records_floor_pure(1000, 1.0, 0.1, 0.05):.0f} {means_records_gaussian(1000, 1
print(f"{epsilon_proved(0.9607, 1e-6):.2f}")  # 3.20
```

Back to the opening claim: a pure $\varepsilon = 1$ histogram over 10^6 locations with maximum error 5 at 95% confidence is impossible, since no mechanism goes below 6.38. Unless the design uses $\delta > 0$ and exploits sparsity, ask what it is really doing. The full edition's script, lower_bounds.py, computes every number in this chapter and runs the attacks:

```
uv run python code/part03-dp-foundations/lower_bounds.py
```

26.7. Cheat sheet

Table 26.2.: The lower bounds of this chapter, with the mechanism that matches each.

Result	Statement	Matching mechanism
Packing (pure DP)	m candidates within k records of one dataset: $k \geq \ln(m(1-\beta))/\varepsilon$	—
Mode over N elements (pure)	Lead $k \geq \ln(N(1-\beta))/\varepsilon$	Report-noisy-max
Histogram, ℓ_∞ (pure)	$\alpha \geq \frac{1}{2}(\ln(N(1-\beta))/\varepsilon - 1)$	Laplace: $\alpha \leq \ln(N/\beta)/\varepsilon$
Packing under $\delta > 0$	Forces at most $k \approx \ln(1/\delta)/\varepsilon$	Stability-based histogram
Histogram, records	$n \geq (\ln(N(1-\beta)) - \varepsilon)/(2\alpha\varepsilon)$; sampling needs $\approx \ln(2N/\beta)/(2\alpha^2)$	Laplace
d means (pure)	$n \gtrsim d \ln 2/(\alpha\varepsilon)$	Laplace: $n = 2d\ln(d/\beta)/(\alpha\varepsilon)$
d means (approximate)	n of order $\sqrt{d\ln(1/\delta)}/(\alpha\varepsilon)$ (fingerprinting)	Gaussian: $n = 2\sqrt{d}\,z/(\mu\alpha)$
Tracing score	Gap $2d/(3n)$; variance $d(\frac{2}{15} + \frac{8}{15n} + \frac{2\sigma^2}{3})$	Defeated by $\sigma \gg \sqrt{d}/n$
Any membership test	$\mathrm{TPR} - \mathrm{FPR} \leq \frac{e^\varepsilon - 1 + 2\delta}{e^\varepsilon + 1}$; $\mathrm{TPR} \leq e^\varepsilon \mathrm{FPR} + \delta$	—
Reconstruction	Recovering a fraction r proves $\varepsilon \geq \ln \frac{r-\delta}{1-r}$	Gaussian with $\sigma \propto \sqrt{k}$

Rules of thumb

- Pure DP pays ln(number of candidates)/ε in error, linear in d for d-dimensional outputs; approximate DP pays about $\sqrt{d}$.
- Released aggregates over many attributes leak membership; noise of order $\sqrt{d}/n$ stops it.
- An attack bounds ε from below; only the mechanism's analysis bounds it from above.

26.8. Further reading

- Vadhan, "The Complexity of Differential Privacy" (2017), §5: packing, fingerprinting and reconstruction lower bounds side by side, and the stability-based histogram.
- Dwork, Smith, Steinke & Ullman, "Exposed! A Survey of Attacks on Private Data" (Annual Review of Statistics and Its Application, 2017): reconstruction and tracing attacks for a statistics audience.
- Steinke & Ullman, "Between Pure and Approximate Differential Privacy" (Journal of Privacy and Confidentiality, 2016): the tight $\sqrt{d\ln(1/\delta)}$ bound for attribute means.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

Part IV.

Privacy accounting

The previous part defined differential privacy (DP), built the basic mechanisms and showed how (ε, δ) guarantees add up when mechanisms run one after another. That is enough for a handful of releases, not for training a model, which runs thousands of noisy steps, each on a random subset of the data, and must then state one ε. No single theorem gives that number. It comes from an **accountant**, a procedure that tracks a summary of each step's privacy loss, combines the summaries, and converts the result into the (ε, δ) you report. Two accountants given the same training run can report values of ε that differ by a large factor, and both can be correct upper bounds; only the tighter one tells you what the noise bought.

Read this part if you train models with DP, or review or audit a model that claims an ε. It assumes Differential privacy foundations: the definition, the privacy loss of an output, the Gaussian mechanism and its exact curve $\delta(\varepsilon)$, and composition. Each chapter restates in a sentence what it uses.

The common thread

The privacy loss of an output is the log of how much more likely that output is with your record than without it. Over a run, the per-step losses add, so for the loss itself composition is easy. For (ε, δ) it is hard, because a pair of numbers is one point of a whole curve and forgets almost everything about how the loss is distributed. Composing points pays for that lost information at every step. Every accountant in this part makes the same move instead: **track a richer summary of the privacy loss, one that composes simply, and convert to** (ε, δ) **only once, at the end.**

The chapters differ in the summary they keep, and their order follows from it. The first two introduce Rényi differential privacy (RDP), the summary most libraries use: a curve of exponential moments of the loss that adds up step by step, and the conversion that turns the final curve into the reported pair. Every later chapter starts from that curve, so they come first. Concentrated DP then squeezes the curve into one number ρ, which describes Gaussian noise exactly and many other mechanisms only loosely, and f-DP goes the other way, keeping the whole trade-off between an attacker's false positives and missed detections, which reads directly as attack power.

None of these summaries yet explains why training is private at all. The differentially private version of stochastic gradient descent (SGD), DP-SGD, clips each example's gradient and adds Gaussian noise, and if every step used the whole dataset its guarantee would be useless. It is private because each step touches a random fraction of the records, so most steps don't see a given record at all. Amplification by subsampling turns that into a much smaller per-step loss. It is a separate question from the summary, since it decides which pair of output distributions you account for, and it must match how the training code really samples its data. Numerical accounting then keeps the whole distribution of the loss and composes it with a discretization error it bounds, which makes it the tool that checks every other accountant. These two chapters are the practical core, in that order, because the numerical accountant takes the subsampled step as its input.

The last chapters handle departures from that setting. Shuffling instead of sampling, or releasing only the final model, can give extra privacy, but only under assumptions the code has to meet. Budgets chosen while training runs, or privacy spent differently per record, break the assumption

that each step's bound is fixed in advance, and need filters and odometers. The final chapter puts the pieces together.

The running example

Every chapter that can compute it reports the same number: the ε at $\delta = 10^{-5}$ of DP-SGD with Poisson sampling rate $q = 0.01$, noise multiplier $\sigma = 1$ and up to $k = 10{,}000$ steps. That is about one hundred epochs at a typical setting. A single Gaussian mechanism with $\sigma = 1$ serves as the small example. Watching the same number move from chapter to chapter shows what each tool gains, and Choosing an accountant in the full edition puts all the values side by side.

How to read this part

Read the RDP in the full edition and conversion in the full edition chapters first, then subsampling in the full edition and numerical accounting in the full edition. The chapters on concentrated DP in the full edition and f-DP in the full edition explain the other summaries you will meet in papers and libraries. Read shuffling and iteration in the full edition if your code shuffles instead of sampling or releases only the final model, adaptive composition in the full edition if your budgets are chosen during training, and the last chapter last. The derivations are in the full edition.

What you will be able to do

By the end of this part you will be able to compute the RDP curve of a Gaussian training step, compose a run of them and convert the result with the right conversion. You will know what each summary keeps and loses, how much subsampling buys and which sampling scheme it assumes, and how to check any accountant against a numerical one. And for a DP-SGD run you will be able to pick an accountant, state its assumptions, and report ε in a way someone else can check.

27. Rényi differential privacy

27.1. Why this matters

You train a model with 1000 steps of full-batch noisy gradient descent. Each step computes a quantity whose **sensitivity** is 1, meaning one person's record moves it by at most 1, and adds Gaussian noise whose standard deviation is $\sigma = 20$ times that sensitivity; σ is the **noise multiplier**. What is the total differential privacy (DP) guarantee at $\delta = 10^{-5}$?

Advanced composition turns each step into one $(\varepsilon_0, \delta_0)$ pair and composes the pairs; with the best split of δ it gives $\varepsilon = 68.01$. The true answer is $\varepsilon = 7.51$, nine times smaller, and the damage is done in the first move. A Gaussian step has no single $(\varepsilon_0, \delta_0)$: for every ε_0 there is a smallest δ_0 that works, so it has a whole curve of valid pairs. Picking one point throws away the rest of the curve, which composition needs.

So keep a curve that is easy to compose. **Rényi differential privacy (RDP)** records, for each **order** $\alpha > 1$, an exponential moment of the privacy loss. These curves add up over a run, with no δ spent along the way, and you convert the total to (ε, δ) once, at the end. For the 1000 Gaussian steps this gives $\varepsilon = 8.84$, 18% above the exact 7.51.

27.2. The RDP curve

Run a mechanism M on neighbouring datasets D and D' to get output distributions $P = M(D)$ and $Q = M(D')$. The **privacy loss** of an output y is $L(y) = \ln \frac{p(y)}{q(y)}$, the log of how much more likely y is under D than under D'. Over several steps the losses add, because the likelihood ratios multiply.

Which summary of a loss survives being summed? Not a tail probability, which is what (ε, δ) records. Exponential moments do: for independent losses, the average of $e^{\lambda(L_1+L_2)}$ is the product of the averages of $e^{\lambda L_1}$ and $e^{\lambda L_2}$, so their logarithms add. The **Rényi divergence** of order α is the log of one such moment, scaled:

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \mathbb{E}_P\big[e^{(\alpha-1)L}\big] = \frac{1}{\alpha - 1} \ln \int p^\alpha q^{1-\alpha}\, dy. \tag{27.1}$$

A mechanism is (α, ε)**-RDP** if $D_\alpha(M(D)\|M(D')) \le \varepsilon$ for every pair of neighbours, and its **RDP curve** $\varepsilon(\alpha)$ is the smallest such ε at each order, the divergence of the worst pair.

The order sets how much large losses count, since an exponential moment is dominated by them, the more so the larger α. As $\alpha \to 1$ the curve tends to the average loss, the Kullback–Leibler (KL) divergence; as $\alpha \to \infty$ it tends to the largest loss of any output, so a mechanism is ε-DP exactly when $\varepsilon(\infty) \le \varepsilon$. In between it never decreases. A loss that is usually small but sometimes huge is caught only at large orders.

Randomized response shows both ends. It reports one person's bit truthfully with probability $p = 0.75$ and flips it otherwise, so the loss is $\ln 3$ on an output that agrees with the bit and $-\ln 3$ on one that doesn't. The average loss is $(0.75 - 0.25)\ln 3 = 0.5493$, the curve's value as $\alpha \to 1$; it rises through 0.8473 at order 2 and 1.0666 at order 10 to the largest loss, $\ln 3 = 1.0986$. A single point such as "$(2, 0.8473)$-RDP" says nothing about higher orders, where the curve may keep rising, so accountants store the whole curve on a grid of orders.

27.3. RDP curves of the basic mechanisms

A mechanism that adds noise to a query's answer produces the same noise centred at two different answers. Its curve is the divergence between the noise and a shifted copy, and since a larger shift makes the copies easier to tell apart, the worst pair is the one with the largest shift, the sensitivity.

For the **Gaussian mechanism** the log of the ratio of two normal densities is a difference of two squares whose quadratic terms cancel, so the loss is linear in the output and therefore itself normal. The log of a normal variable's exponential moment is quadratic in λ, and dividing by $\alpha - 1$ leaves a straight line. With sensitivity 1,

$$\varepsilon(\alpha) = \frac{\alpha}{2\sigma^2}. \tag{27.2}$$

For a vector query only the Euclidean distance between the two answers matters, because the noise looks the same in every direction, so σ is the noise standard deviation in units of the ℓ_2 sensitivity. In the differentially private version of stochastic gradient descent (SGD), DP-SGD, which clips each example's gradient and adds Gaussian noise, that sensitivity is the clipping norm, so the σ you plug in is the noise standard deviation divided by the clipping norm.

A linear loss has no maximum, so the curve grows without limit: the Gaussian mechanism is not ε-DP for any ε. It is RDP at every finite order, though, and that is all composition and conversion need. The **Laplace mechanism**, which adds noise of density $\frac{1}{2b}e^{-|y|/b}$, behaves differently. Its loss at y is $(|y-1| - |y|)/b$, which the triangle inequality keeps between $-1/b$ and $1/b$, so its curve levels off at its pure-DP level $1/b$; randomized response levels off at $\ln \frac{p}{1-p}$ for the same reason. All three curves are exact (Mironov 2017); the closed forms are in Table 27.3, and Figure 27.1 and Table 27.1 compare them.

Table 27.1.: RDP curves of the three basic mechanisms with sensitivity 1.

Mechanism	$\varepsilon(1.5)$	$\varepsilon(2)$	$\varepsilon(10)$	$\varepsilon(64)$	$\varepsilon(\infty)$
Gaussian, $\sigma = 1$	0.7500	1.0000	5.0000	32.0000	∞
Laplace, $b = 1$	0.5129	0.6191	0.9287	0.9891	1
Randomized response, $p = 0.75$	0.7340	0.8473	1.0666	1.0940	1.0986

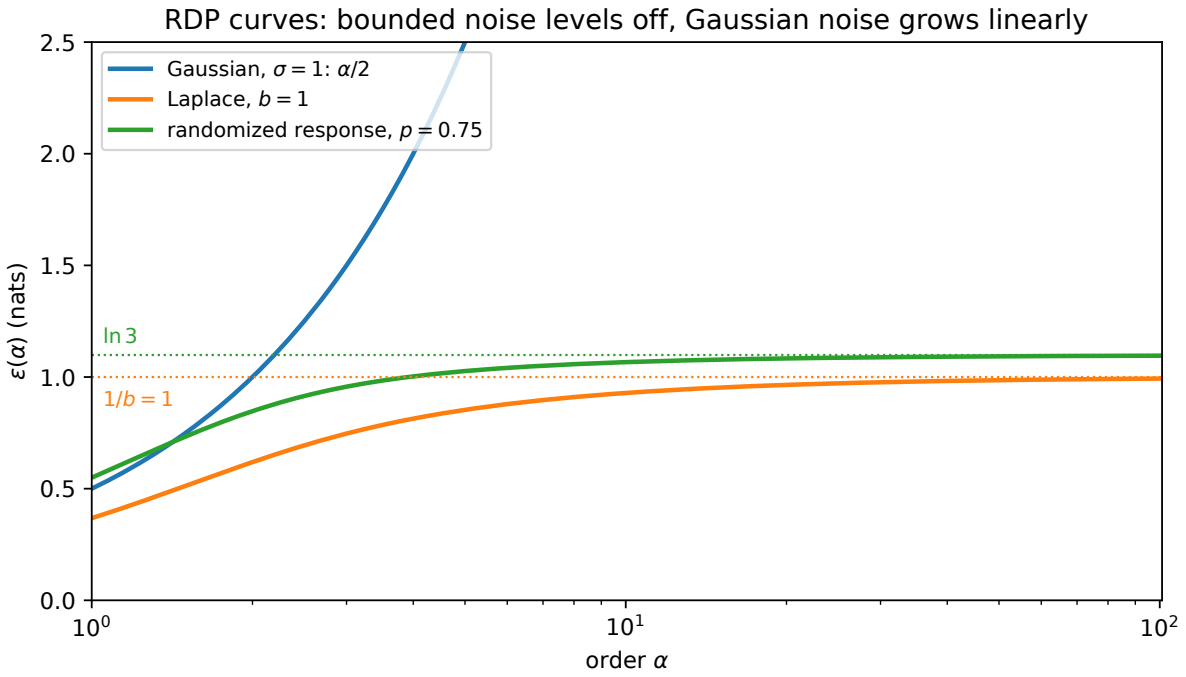


Figure 27.1.: RDP curves of the Gaussian mechanism ($\sigma = 1$), the Laplace mechanism ($b = 1$) and randomized response ($p = 0.75$), all with sensitivity 1, on a log scale in α (so the straight line $\alpha/2$ looks curved). The Laplace and randomized response curves level off at their pure DP levels $1/b$ and $\ln 3$ (dotted); the Gaussian curve keeps growing.

These are curves of a step that uses the whole dataset. A step on a random subset has a smaller curve, derived in Amplification by subsampling in the full edition.

27.4. Composition adds curves

Training steps are not independent, since each gradient depends on the model so far, so the product argument of Section 27.2 needs one more idea. Fix the first output and look at the second step alone. Whatever the first output was, the second step is some mechanism with curve at most ε_2, so its moment is at most $e^{(\alpha-1)\varepsilon_2(\alpha)}$; averaging over the first output leaves the first step's moment times that constant. Taking the log, the curves add:

$$\varepsilon_M(\alpha) \le \sum_{i=1}^{k} \varepsilon_i(\alpha) \qquad \text{for every } \alpha > 1, \tag{27.3}$$

with equality for independent steps (Mironov 2017). The argument also shows the limit of adaptivity: each step's bound must hold whatever the earlier outputs were, so it has to be fixed in advance. A noise level or step count chosen on the fly from the data needs the filters of Adaptive composition in the full edition.

The smallest δ for a given ε has no such product structure. It is the probability by which one output distribution exceeds e^{ε} times the other, a positive part of a difference, which does not factor over steps. So never convert each step first and compose afterwards: that is where the factor of nine in the opening example comes from.

For Gaussian steps the sum has a picture. Each of k steps shifts its output by at most 1, and the k shifts together form a vector of length $\sqrt{k}$; a shift of $\sqrt{k}$ under noise σ is the same as a shift of 1 under noise $\sigma/\sqrt{k}$. So k steps with noise multiplier σ have the curve $k\alpha/(2\sigma^2)$, the curve of one Gaussian step with noise multiplier $\sigma/\sqrt{k}$. With $\sigma = 20$ and 1000 steps that is one Gaussian with $\sigma = 0.6325$.

The same picture shows why DP-SGD needs more than noise. If every step of the part's running example, 10,000 steps with $\sigma = 1$, used the whole dataset, the run would be one Gaussian with noise multiplier $1/\sqrt{10{,}000} = 0.01$, and even the exact guarantee at $\delta = 10^{-5}$ is $\varepsilon = 5425.5$: no guarantee at all. DP-SGD is private only because each step touches a random 1% of the data, which Amplification by subsampling in the full edition accounts for.

27.5. Post-processing and group privacy

Anything computed from a private output without looking at the data again, such as a released model's predictions, cannot raise the curve: $\varepsilon_{g\circ M}(\alpha) \le \varepsilon_M(\alpha)$ for any map g that ignores the data. A processed output's likelihood ratio is an average of the ratios of the outputs that could have produced it, and averaging can only lower an exponential moment.

Groups are harder. For pure DP a group of k records costs $k\varepsilon$, because likelihood ratios along a chain of neighbours multiply. For RDP, bounding the average of that product needs higher moments of each factor, so a bound at order α for one record buys one at only about order $\alpha/2$ for two. Mironov's (2017) Proposition 2: if M is (α, ε)-RDP with $\alpha \ge 2^{c+1}$, then on datasets that

differ in 2^c records it satisfies the bound of order $\alpha/2^c$ with $3^c\varepsilon$ in place of ε. This bound is loose, so use the mechanism when you can: a group of 2^c records shifts a Gaussian answer by 2^c, so the exact group curve is Equation 27.2 with sensitivity 2^c, and at $\sigma = 1$ the generic bound is 1.5 times too large for two records and 3.375 times for eight. A mechanism that is already ε-DP keeps the pure-DP bound $k\varepsilon$ at every order, which can win: for Laplace with $b = 1$, two records apart at order 2, the exact value is 1.5958, the generic bound 2.4411 and the pure-DP bound 2.

27.6. The moments accountant is RDP

Abadi et al. (2016) trained neural networks with DP by tracking, for the worst pair of neighbours, the log of the moment generating function (MGF) of the privacy loss, $K(\lambda) = \ln \mathbb{E}_P[e^{\lambda L}]$: the **moments accountant**. The definition of Section 27.2 says that $(\alpha-1)D_\alpha$ is exactly this log-MGF at $\lambda = \alpha - 1$, so the two are the same curve in different units:

$$K(\lambda) = \lambda\,\varepsilon(\lambda + 1), \qquad \varepsilon(\alpha) = \frac{K(\alpha - 1)}{\alpha - 1}. \tag{27.4}$$

For randomized response, $K(1) = \ln(0.75 \times 3 + 0.25 \times \frac{1}{3}) = 0.8473 = 1 \cdot \varepsilon(2)$. What was new in 2016 was not the bookkeeping but a bound on the curve of the subsampled step. So when a "moments accountant" and an "RDP accountant" disagree, look at that curve, the order grid or the conversion.

27.7. From RDP to (ε, δ)

After composing you hold a curve, but you report a pair. An output whose loss is at most some threshold c already satisfies the e^c bound of DP, so only outputs with loss above c can break it, and you can charge their total probability to δ. That probability is a tail of the loss, and Markov's inequality bounds a tail by an exponential moment, which is exactly what each point of the curve provides. Setting the bound equal to δ and solving for c gives **Mironov's conversion**: an $(\alpha, \varepsilon(\alpha))$-RDP mechanism is $(\varepsilon_\delta, \delta)$-DP for every $0 < \delta < 1$, with

$$\varepsilon_\delta = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}. \tag{27.5}$$

Every order gives a valid pair. The curve grows with the order and the tail term shrinks, so you minimize over the grid after composing. For a Gaussian curve $\alpha/(2\sigma^2)$ the minimum has a closed form, where the two order-dependent terms are equal:

$$\alpha^* = 1 + \sigma\sqrt{2\ln(1/\delta)}, \qquad \varepsilon_\delta = \frac{1}{2\sigma^2} + \frac{\sqrt{2\ln(1/\delta)}}{\sigma}. \tag{27.6}$$

For k steps, put $\sigma/\sqrt{k}$ in place of σ.

For one Gaussian step with $\sigma = 1$ at $\delta = 10^{-5}$, where $\ln(1/\delta) = 11.5129$, order 2 gives $1+11.5129 = 12.5129$, dominated by the tail term, and the best grid order, 5.8, gives 5.2985. The exact ε is 4.3772, so the conversion is 21% too large, and at $\varepsilon = 5.2985$ the true δ is 1.300×10^{-7}, 76.9 times smaller than promised. Table 27.2 and Figure 27.2 return to the opening example.

Table 27.2.: Total ε at $\delta = 10^{-5}$ for k Gaussian steps with $\sigma = 20$: basic and advanced composition of per-step $(\varepsilon_0, \delta_0)$ points, RDP with Mironov's conversion, and the exact value.

k	Basic	Advanced	RDP + Mironov (best order)	Exact
1	0.160	0.847	0.251 (128)	0.160
10	1.892	3.270	0.771 (31)	0.561
100	21.510	13.423	2.524 (10.6)	1.993
1000	238.578	68.008	8.838 (4)	7.511

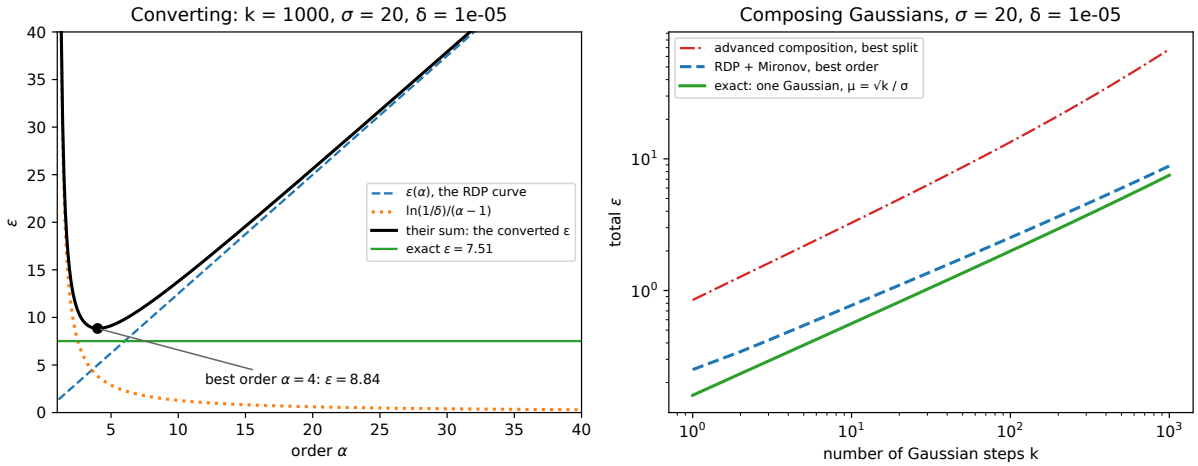


Figure 27.2.: Left: converting the curve of 1000 Gaussian steps with $\sigma = 20$ at $\delta = 10^{-5}$. The converted ε (black) is the RDP curve (dashed) plus the tail term $\ln(1/\delta)/(\alpha - 1)$ (dotted); its minimum over the grid, 8.84 at $\alpha = 4$, sits above the exact 7.51 (green). Right: total ε against the number of steps, for advanced composition, RDP with Mironov's conversion, and the exact value.

Advanced composition is 5 to 9 times the exact value, worse as k grows, because every step was collapsed to a point; RDP is 18% to 57% above, better as k grows. The best order falls as the run grows, because a steeper composed curve makes large orders expensive: one step with $\sigma = 20$ wants $\alpha \approx 97$, a thousand steps want $\alpha \approx 4$, and the full-batch run of Section 27.4 wants 1.048, below the grid's smallest order 1.1. So a grid must run from just above 1 to a few hundred; a best order at its end means it is too short.

The remaining gap is slack: the conversion charges the whole probability of every output above the threshold to δ, although an output just over the line breaks the bound only slightly, and Markov's inequality then overestimates that tail. The slack depends on the composed $\mu = \sqrt{k}/\sigma$: 1.07 times exact at $\mu = 10$, 1.21 at $\mu = 1$ and 1.76 at $\mu = 0.01$. Weak mechanisms pay most, because as μ shrinks the exact ε falls faster than the tail bound. So don't report this conversion; use the improved one of From Rényi DP to (ε, δ) and to tests in the full edition, or a numerical accountant.

27.8. Code

RDP accounting for full-batch Gaussian steps is a curve on a grid of orders, a sum over steps and a minimum over orders. This snippet uses the part's grid of 154 orders.

```
from math import log

ORDERS = [1 + x / 10 for x in range(1, 100)] + list(range(12, 64)) + [128, 256, 512]


def gaussian_rdp(alpha: float, sigma: float, steps: int = 1) -> float:
    """RDP curve of `steps` Gaussian steps with noise multiplier sigma (sensitivity 1)."""
    return steps * alpha / (2 * sigma**2)


def mironov_eps(curve: list[float], delta: float) -> tuple[float, float]:
    """Best (epsilon, order) from Mironov's conversion over the grid."""
    return min((eps + log(1 / delta) / (a - 1), a) for a, eps in zip(ORDERS, curve))


one = [gaussian_rdp(a, sigma=1.0) for a in ORDERS]
run = [gaussian_rdp(a, sigma=20.0, steps=1000) for a in ORDERS]
full_batch = [gaussian_rdp(a, sigma=1.0, steps=10_000) for a in ORDERS]
print("%.4f at order %g" % mironov_eps(one, 1e-5))  # 5.2985 at order 5.8
print("%.3f at order %g" % mironov_eps(run, 1e-5))  # 8.838 at order 4
print("%.1f at order %g" % mironov_eps(full_batch, 1e-5))  # 5615.1 at order 1.1
```

The last line shows the best order stuck at the bottom of the grid. The full edition's script, renyi_dp.py, computes every number in this chapter, including the exact Gaussian values:

```
uv run python code/part04-privacy-accounting/renyi_dp.py
```

27.9. Cheat sheet

Table 27.3.: The results of this chapter.

Result	Formula	Notes
(α, ε)-RDP	$D_\alpha(M(D)\|M(D')) \le \varepsilon$ for all neighbours	Non-decreasing in α; $\varepsilon(\infty)$ is pure DP
Gaussian, noise multiplier σ	$\varepsilon(\alpha) = \alpha/(2\sigma^2)$	Exact; σ in units of the ℓ_2 sensitivity
Laplace, scale b	$\frac{1}{\alpha-1}\ln\left(\frac{\alpha}{2\alpha-1}e^{(\alpha-1)/b} + \frac{\alpha-1}{2\alpha-1}e^{-\alpha/b}\right)$	Exact; levels off at $1/b$

Result	Formula	Notes
Randomized response, keep p	$\frac{1}{\alpha-1}\ln\left(p^\alpha(1-p)^{1-\alpha}+(1-p)^\alpha p^{1-\alpha}\right)$	Exact; levels off at $\ln\frac{p}{1-p}$
Composition	$\varepsilon_M(\alpha)\le\sum_i\varepsilon_i(\alpha)$	Adaptive steps; equality for independent ones
k Gaussian steps	One Gaussian with $\sigma/\sqrt{k}$	Full-batch only
Group of 2^c records	Order $\alpha/2^c$, $3^c\varepsilon(\alpha)$, if $\alpha\ge 2^{c+1}$	Loose; for the Gaussian use sensitivity 2^c
Moments accountant	$K(\lambda)=\lambda\,\varepsilon(\lambda+1)$	Same numbers as RDP
Mironov's conversion	$\varepsilon(\alpha)+\ln(1/\delta)/(\alpha-1)$, best α	1.07 to 1.76 times exact for a composed Gaussian
Gaussian optimum	$\alpha^*=1+\sigma\sqrt{2\ln(1/\delta)}$	σ is the composed noise multiplier

Rules of thumb

- Compose in RDP and convert once at the end. Never convert each step to (ε_0,δ_0) first.
- Keep an order grid from just above 1 to a few hundred, and check that the best order is not at its edge.
- Report the noise multiplier, the number of steps and the sampling scheme along with ε, so others can recompute it.

27.10. Further reading

- Mironov, "Rényi Differential Privacy" (IEEE CSF 2017): the definition, composition, the curves of the basic mechanisms, group privacy and the conversion. Short and readable.
- Abadi et al., "Deep Learning with Differential Privacy" (CCS 2016): DP-SGD and the moments accountant, with the experiments that made private deep learning practical.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (arXiv 2019): the curves of subsampled steps, and the reference behind most RDP accountants.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

28. From Rényi DP to (ε, δ) and to tests

28.1. Why this matters

A Rényi differential privacy (RDP) accountant adds up the curves $\varepsilon(\alpha)$ of a run's steps, one value per **order** $\alpha > 1$ (Rényi differential privacy in the full edition). Nobody reports that curve. A paper, a model card or a regulator asks for an (ε, δ) differential privacy (DP) guarantee, or for a statement about attacks, so every RDP analysis ends with a conversion, and the conversion decides the number you publish.

Ask what a conversion is allowed to know. The **privacy loss** of an output is the log of how much more likely it is under one neighbouring dataset than under the other, and the RDP curve records exponential moments of that loss and nothing else. A conversion must therefore give a δ that is safe for every mechanism with that curve, not just for yours. Mironov's conversion, the original one, uses the moments crudely. The **improved conversion** uses them almost as well as anything can, at no cost. What remains, 5 to 10% of ε for the Gaussian mechanism, belongs to the summary itself.

The running example is the Gaussian mechanism. A step adds noise with standard deviation σ times the sensitivity, and k such steps compose to one Gaussian whose two output distributions are $\mu = \sqrt{k}/\sigma$ noise standard deviations apart. Its RDP curve is $\varepsilon(\alpha) = \alpha\mu^2/2$, so every number below depends only on μ. All use $\delta = 10^{-5}$ and the part's grid of 154 orders, from 1.1 to 512.

28.2. Mironov's conversion and where it loses

Start from what δ is. The smallest valid δ at a given ε is an average over outputs: an output whose loss L exceeds ε breaks the e^{ε} bound only by its excess probability, which is the fraction $1 - e^{\varepsilon - L}$ of its probability. An output just over the line contributes almost nothing, and one far over it contributes almost all of its probability.

Mironov's conversion bounds that average in two steps. It first charges the whole probability of every output with $L > \varepsilon$ to δ, which replaces δ by the tail probability of the loss. It then bounds the tail with Markov's inequality applied to $e^{(\alpha-1)L}$, whose average is what the RDP curve provides. Setting the result equal to δ gives

$$\varepsilon' = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}, \tag{28.1}$$

minimized over the orders of the grid.

Both steps lose on the Gaussian. Its loss is normal, and ε sits several standard deviations above the mean, where the density falls steeply; most of the tail lies just above ε, exactly where each output's true contribution is near zero, so the first step overcounts. The second step loses more.

Markov's inequality is tight only for a variable that puts all its mass at 0 and at the threshold, and a normal loss is nothing like that.

For a mean released with the Gaussian mechanism at $\sigma = 1$, so $\mu = 1$, the exact ε is 4.377. Charging the whole tail alone raises it to 4.765, and the Markov step to 5.299, at order 5.8. At $\varepsilon = 5.299$ the true δ is 1.300×10^{-7}: you pay for $\delta = 10^{-5}$ and get far less. The conversion is valid, just loose: safe but overstated. Older code and papers, including the original moments accountant, use it, so check which conversion each side used before comparing your ε with a published one.

28.3. The improved conversion

Taken together, the two steps replace each output's contribution $(1 - e^{\varepsilon - L})_+$ by an exponential in L that lies above it, and an exponential is used because it is the only function whose average the RDP curve tells you. Mironov's exponential equals 1 at $L = \varepsilon$, where the true contribution is still 0. A lower exponential still stays above the contribution, touching it just to the right of ε. Using the lowest one is the whole improvement (Balle, Barthe, Gaboardi, Hsu & Sato 2020; Canonne, Kamath & Steinke 2020). An $(\alpha, \varepsilon(\alpha))$-RDP mechanism is then (ε', δ)-DP with

$$\varepsilon' = \varepsilon(\alpha) + \ln \frac{\alpha - 1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha - 1}, \tag{28.2}$$

again minimized over the grid, and reported as 0 if it comes out negative.

Compare the two formulas at the same order and both the curve and the $\ln(1/\delta)$ term cancel. What is left, the saving, depends only on the order, not on the mechanism or δ:

$$\Delta(\alpha) = \frac{\ln \alpha}{\alpha - 1} + \ln \frac{\alpha}{\alpha - 1}. \tag{28.3}$$

It is large at small orders and fades at large ones (Table 28.1). That gives you a diagnostic. If two accountants report values about 0.4 apart from the same curve at a best order near 10, one of them is using Mironov's conversion. Opacus and Google's `dp_accounting` both use the improved one, and if you implement a conversion yourself, use Equation 28.2.

Table 28.1.: What the improved conversion gains over Mironov's at the same order: the factor by which its δ is smaller, and the saving in ε.

α	δ factor	ε saving $\Delta(\alpha)$
2	0.2500	1.3863
10	0.0387	0.3612
100	0.0037	0.0566

For the mean released at $\sigma = 1$, the improved conversion gives $(4.729, 10^{-5})$-DP at order 5.4, against Mironov's 5.299 and the exact 4.377. It removes 0.570 of the 0.921 excess, from the same curve. It even beats the 4.765 of charging the whole tail, because its exponential is allowed to dip below 1 just above ε, where the true contributions are small. At $\mu = 10$, which is 100 steps at $\sigma = 1$, it gives 96.116 against Mironov's 98.026 and the exact 91.817.

28.4. Choosing the order

The converted ε' is the curve, which grows with the order, plus a term in $\ln(1/\delta)/(\alpha-1)$, which shrinks with it, and the best order balances the two. For a composed Gaussian with curve $\rho\alpha$, where $\rho = \mu^2/2 = k/(2\sigma^2)$, setting the derivative to zero gives $\rho(\alpha-1)^2 = \ln\frac{1}{\alpha\delta}$, so roughly

$$\alpha^* - 1 \approx \sqrt{\ln(1/\delta)/\rho} = \sigma\sqrt{2\ln(1/\delta)/k}. \tag{28.4}$$

The formula says how the best order moves. More steps or less noise make the curve steeper, so high orders get expensive and the best order moves toward 1. A smaller δ pushes it up, but only through the square root of a logarithm, so δ hardly matters: at $\mu = 1$, going from $\delta = 10^{-5}$ to 10^{-10} moves the best order only from 5.432 to 7.483.

Table 28.2 shows the range a grid must cover. One release at $\sigma = 10$ wants order 40.525, a long run at $\sigma = 1$ wants 1.048. At $k = 10{,}000$ that order is below the grid's smallest, 1.1, and the grid's answer is 135.987 larger than the continuous optimum.

Table 28.2.: Best orders of the improved conversion for the composed Gaussian at $\delta = 10^{-5}$.

σ	k	Best order (continuous / grid)	ε (continuous / grid)
10	1	40.525 / 41	0.375 / 0.375
1	1	5.432 / 5.4	4.728 / 4.729
1	100	1.472 / 1.5	96.035 / 96.116
1	10,000	1.048 / 1.1	5475.791 / 5611.778

So if an accountant reports its best order at either end of its grid, the true optimum lies outside and the reported ε is larger than necessary: extend the grid in that direction. The other end matters too. The differentially private version of stochastic gradient descent (SGD), DP-SGD, runs subsampled steps (Amplification by subsampling in the full edition), whose curves are tiny at small orders and therefore want large ones, which is why the grid ends at 128, 256 and 512.

28.5. The slack no conversion removes

The improved conversion still stays above the exact ε. That is the summary's fault, not the formula's, and a counterexample shows why. Take a mechanism with only two outputs: almost all its probability sits on an output with loss close to 0, and a sliver on an output with loss 4.5956. This mechanism satisfies the Gaussian's RDP curve at $\mu = 1$ at every order, yet at $\varepsilon = 4.377$, the Gaussian's exact value, its δ is 4.4698×10^{-5}, not 10^{-5}. A certificate computed from the curve alone cannot tell the two mechanisms apart, so it must cover both, and it cannot certify 4.377. The best any method can certify from the curve is 4.7284 to four decimals, which is what the improved conversion gives at its best real order.

Across run lengths (Table 28.3, Figure 28.1), the best conversion from the curve and the improved conversion agree to three decimals, and both exceed the exact ε by 5 to 10%.

Table 28.3.: ε at $\delta = 10^{-5}$ for the Gaussian with $\sigma = 10$ composed k times: exact, the best any conversion from the RDP grid can give, the improved conversion and Mironov's.

k	μ	Exact	Optimal from the curve	Improved	Mironov
1	0.100	0.341	0.375	0.375	0.485
10	0.316	1.199	1.308	1.308	1.568
100	1.000	4.377	4.729	4.729	5.299
1000	3.162	17.857	19.054	19.054	20.175
10,000	10.000	91.817	96.116	96.116	98.026

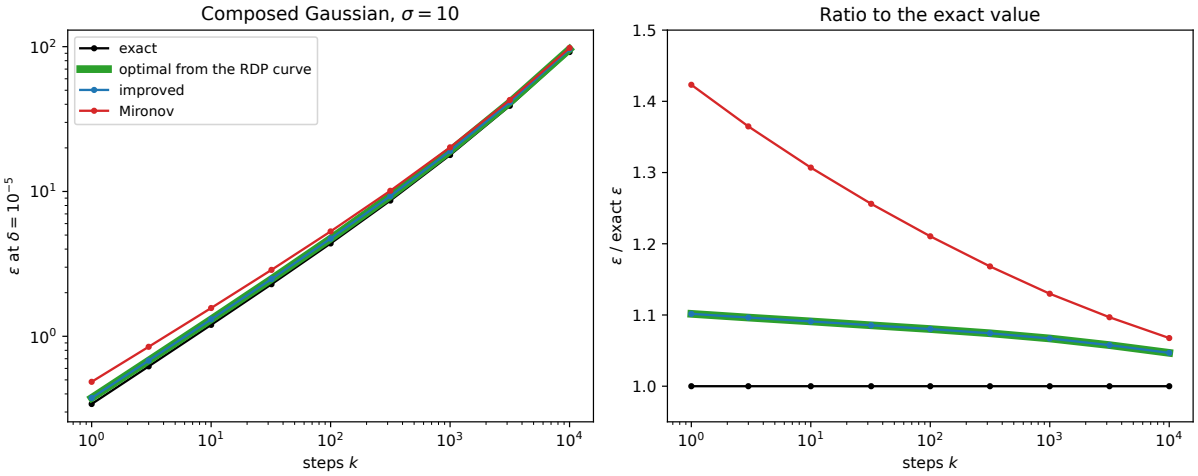


Figure 28.1.: Left: ε at $\delta = 10^{-5}$ for the Gaussian with $\sigma = 10$ composed k times, exact (black), optimal from the RDP curve (wide green), improved (blue, on top of the green) and Mironov (red). Right: each divided by the exact value. The improved conversion is as good as any conversion from the curve; the remaining 5 to 10% belongs to the RDP summary itself.

So if a reviewer asks whether a better conversion would lower your $(4.729, 10^{-5})$ for a Gaussian with $\mu = 1$, the answer is no. When the slack matters, change the summary, not the formula: track the trade-off function (f-DP and Gaussian differential privacy in the full edition) or the whole privacy loss distribution (Numerical accounting with privacy loss distributions in the full edition). The 5 to 10% is for the plain Gaussian. DP-SGD's subsampled steps have a curve that is not a line, and the gap can be much larger; for the part's running example ($q = 0.01$, $\sigma = 1$, $k = 10{,}000$), RDP gives 7.43 with Mironov's conversion and 6.71 with the improved one, and Choosing an accountant in the full edition measures the gap.

The same reasoning governs mixing RDP with other guarantees. An (ε, δ)-DP guarantee with $\delta > 0$ implies no RDP bound at any order. A mechanism that publishes your record with probability δ is $(0, \delta)$-DP, but it has an output that is impossible without you, an infinite likelihood ratio, and so an infinite exponential moment. Never feed an (ε, δ) component into an RDP accountant as if it had a curve; you need to know how its δ arises, for example from Gaussian noise. A pure ε-DP guarantee has no such outputs, and it implies $(\alpha, \min(\varepsilon, \alpha\varepsilon^2/2))$-RDP at every order (Bun & Steinke 2016).

28.6. RDP as a bound on attacks

A membership test looks at the output and decides "your record was used" or "it was not". Whatever it computes, its verdict is a coin: it says "used" with probability a, its false-positive rate, when the record was absent, and with probability $1 - b$, its power, when it was present. The verdict is computed from the output, so it cannot be further apart in the two worlds than the output itself, and the Rényi divergence between the two coins is at most $\varepsilon(\alpha)$ at every order (Balle et al. 2020). Each order thus constrains every test's (a, b).

The constraints of different orders bend differently. Large orders weigh the extreme outputs heavily, so they guard small false-positive rates; small orders guard the bulk. An (ε, δ) pair, by contrast, constrains every test by one straight line, power $\leq e^{\varepsilon} a + \delta$, which can touch the curved region the orders allow at only one point and lies far above it elsewhere.

The gap matters for audits. An auditor attacks the mean released at $\sigma = 1$ with a false-positive rate of $a = 10^{-3}$. The RDP curve guarantees that no test has power above 2.4910×10^{-2}; the best test against the Gaussian actually reaches 1.8298×10^{-2}. The $(4.729, 10^{-5})$ guarantee from the improved conversion only says power $\leq e^{4.729} \times 10^{-3} + 10^{-5} = 0.1131$, more than four times the curve's bound. So when you translate an RDP result into attack risk, don't go through one (ε, δ) pair: at low false-positive rates, the ones that matter for membership attacks, it throws away most of what the curve says. Use the curve, or better, the trade-off function of f-DP and Gaussian differential privacy in the full edition.

28.7. Code

Both conversions fit in a few lines. This snippet converts the composed Gaussian's curve on the part's grid of orders, for the single release ($\mu = 1$) and for 100 steps at $\sigma = 1$ ($\mu = 10$).

```
from math import log

ORDERS = [1 + x / 10 for x in range(1, 100)] + list(range(12, 64)) + [128, 256, 512]


def mironov(rdp: float, alpha: float, delta: float) -> float:
    return rdp + log(1 / delta) / (alpha - 1)


def improved(rdp: float, alpha: float, delta: float) -> float:
    """Balle et al. (2020); Canonne, Kamath & Steinke (2020). Clipped at 0."""
    return max(0.0, rdp + log((alpha - 1) / alpha) - (log(delta) + log(alpha)) / (alpha - 1)


def report(mu: float, delta: float = 1e-5) -> str:
    """Both conversions for a composed Gaussian with mu = sqrt(steps) / sigma."""
    curve = [a * mu**2 / 2 for a in ORDERS]
    mir = min((mironov(e, a, delta), a) for a, e in zip(ORDERS, curve))
    imp = min((improved(e, a, delta), a) for a, e in zip(ORDERS, curve))
```

```
    return "Mironov %.3f at order %g, improved %.3f at order %g" % (*mir, *imp)


print(report(1.0))  # Mironov 5.299 at order 5.8, improved 4.729 at order 5.4
print(report(10.0))  # Mironov 98.026 at order 1.5, improved 96.116 at order 1.5
```

To convert any other mechanism, replace the curve by its RDP values on the same grid. The full edition's script, rdp_conversions.py, also computes the exact Gaussian values, the optimal conversion and the bound on tests:

```
uv run python code/part04-privacy-accounting/rdp_conversions.py
```

28.8. Cheat sheet

Table 28.4.: The results of this chapter.

Result	Formula	Notes
Mironov's conversion	$\varepsilon' = \varepsilon(\alpha) + \ln(1/\delta)/(\alpha - 1)$	Valid but loose; 1.07–1.42 times exact on the Gaussian at $\sigma = 10$
Improved conversion	$\varepsilon' = \varepsilon(\alpha) + \ln \frac{\alpha-1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha-1}$	Use this; clip at 0
Saving at a fixed order	$\Delta(\alpha) = \frac{\ln \alpha}{\alpha-1} + \ln \frac{\alpha}{\alpha-1}$	1.39 at order 2, 0.36 at 10, 0.06 at 100
Best order, composed Gaussian	$\rho(\alpha-1)^2 = \ln \frac{1}{\alpha\delta}$, $\rho = k/(2\sigma^2)$	Moves toward 1 as k grows
Unavoidable slack	5 to 10% of ε for the Gaussian	Larger for subsampled DP-SGD
(ε, δ) to RDP	Impossible for $\delta > 0$	Need the mechanism, not the pair
ε-DP to RDP	$\varepsilon(\alpha) \le \min(\varepsilon, \alpha\varepsilon^2/2)$	Bun & Steinke 2016

Rules of thumb

- Always use the improved conversion. It is free and, for the Gaussian, matches the best any method can do from the curve.
- Check the best order. If it sits at either end of the grid, extend the grid.
- Expect an RDP accountant to overstate the plain Gaussian's ε by 5 to 10%, and subsampled DP-SGD's by more. If that matters, use a privacy loss distribution accountant.
- Read an RDP result as a bound on attacks through the whole curve, not through one (ε, δ) pair.

28.9. Further reading

- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020): the improved conversion as a tangent line, and the constraint an RDP curve puts on tests.
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020): the short proof of the improved conversion, Proposition 12.
- Steinke, "Composition of Differential Privacy & Privacy Amplification by Subsampling" (2022 survey): privacy loss, RDP and conversions in one place, with their tightness.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

29. Concentrated differential privacy

29.1. Why this matters

You run a pipeline of Gaussian releases, such as noisy counts, answers in a query system or noisy marginals for synthetic data, and you want to keep the differential privacy (DP) budget as one number that you add to as you go. Rényi DP in the full edition describes each mechanism by a curve: its **Rényi differential privacy (RDP) curve** $\varepsilon(\alpha)$ gives, for each order $\alpha > 1$, the largest Rényi divergence between the outputs on **neighbouring** datasets, which differ in one person's record. Composition adds the curves. For the Gaussian mechanism with noise multiplier σ, the noise standard deviation in units of the sensitivity, the curve is a straight line through the origin, $\varepsilon(\alpha) = \alpha/(2\sigma^2)$. A line through the origin is fixed by its slope, so for Gaussian noise one number already carries everything.

Zero-concentrated differential privacy (zCDP), the most widely used form of concentrated differential privacy (CDP), turns that observation into a definition: describe any mechanism by the slope of a line through the origin that lies above its RDP curve. Lines add to a line, so composition is addition of slopes, and converting a line to (ε, δ) has a closed form. The rest of the chapter asks one question: how well does a line fit the curve? For the Gaussian, exactly. For pure DP mechanisms with large ε, badly. For the subsampled steps of differentially private stochastic gradient descent (SGD), DP-SGD for short, it fits so badly that zCDP is the wrong tool.

29.2. The definition of zCDP

A mechanism M is **ρ-zCDP** if its RDP curve lies under the line of slope ρ: for all neighbours $D \simeq D'$ and all orders $\alpha > 1$,

$$D_\alpha(M(D) \,\|\, M(D')) \le \rho\alpha. \tag{29.1}$$

Because RDP curves add under composition, so do the lines above them: a ρ_1-zCDP step followed by a ρ_2-zCDP step is $(\rho_1 + \rho_2)$-zCDP, even when the second step is chosen after seeing the first one's output. Post-processing cannot increase a divergence, so it cannot increase ρ either.

The Gaussian mechanism, which adds $\mathcal{N}(0, \sigma^2)$ to a query with ℓ_2 sensitivity Δ, is $\Delta^2/(2\sigma^2)$-zCDP, and the line is its exact curve. The reason also explains the name. The **privacy loss** of an output is the log of how much more likely it is under D than under D'. For two normal densities with the same variance, that log-ratio is a linear function of the output, and a linear function of a normal variable is normal: with $\Delta = 1$ the loss has mean $1/(2\sigma^2) = \rho$ and variance $1/\sigma^2 = 2\rho$. zCDP in general says the privacy loss has tails no heavier than this normal variable, "concentrated" around a small mean.

With $\sigma = 1$ one Gaussian step is 0.5-zCDP and ten are 5-zCDP; with $\sigma = 2$ a step costs 0.125. Halving the noise multiplier multiplies ρ by four, because ρ goes with the square of the shift in

units of the noise. The 2020 United States Census TopDown algorithm is built this way (Abowd et al. 2022): it measures many noisy queries with the discrete Gaussian mechanism (Discrete and floating-point-safe mechanisms), adds up the ρ's, and converts to (ε, δ) once, at the end. For Gaussian noise the line loses nothing, so the only approximation is that final conversion.

Two warnings. ρ is not ε: $\rho = 0.5$ is about $\varepsilon = 4.7$ at $\delta = 10^{-5}$ (Section 29.4). And the ledger covers only what passed through the noise: the Census Bureau released some statistics, the **invariants**, exactly; they were never charged to ρ, and they sit outside the guarantee.

29.3. Pure DP steps in a zCDP budget

A Laplace count, a randomized-response bit or an exponential-mechanism selection is pure ε-DP, and to put it in the same ledger you need its ρ. Pure DP means every output's probability changes by at most a factor e^{ε}, so its privacy loss is confined to $[-\varepsilon, \varepsilon]$. A bounded random variable has light tails, at most as heavy as a normal's, so the step should be zCDP with ρ of order ε^2. Bun & Steinke (2016) show that every ε-DP mechanism is $(\varepsilon^2/2)$-zCDP. The exponential mechanism does four times better, $\varepsilon^2/8$, because its privacy loss lies in an interval of length ε rather than 2ε, and the tail bound goes with the square of the length (Cesar & Rogers 2021).

So a Laplace count at $\varepsilon = 0.1$ costs 0.005. The constant $\varepsilon^2/2$ is the right one only as $\varepsilon \to 0$. Randomized response, which reports a bit truthfully with probability $e^{\varepsilon}/(1 + e^{\varepsilon})$, is the worst ε-DP mechanism at every order, and its smallest valid slope is $\varepsilon \tanh(\varepsilon/2)$: 0.4621 at $\varepsilon = 1$, against $\varepsilon^2/2 = 0.5000$.

The trouble is at large orders. No Rényi divergence can exceed the largest log-ratio of probabilities, so the RDP curve of a pure DP step flattens at ε, while a line keeps rising. The curve therefore lies under both,

$$\varepsilon(\alpha) \le \min\left(\varepsilon,\ \frac{\alpha\varepsilon^2}{2}\right), \tag{29.2}$$

and a zCDP accountant can keep only the line. For randomized response at $\varepsilon = 1$ the line is close near $\alpha = 1$, but at $\alpha = 50$ it says 25 where the exact curve is 0.9936; the cap is within 1% there (Figure 29.1, left).

The larger ε is, the sooner the line crosses the cap, since it does so at $\alpha = 2/\varepsilon$. At $\varepsilon = 3$ the slope $\varepsilon^2/2 = 4.5$ is already above the cap at $\alpha = 1$, so the line is worse than plain ε at every order (Figure 29.1, right). Charging a large-ε step in ρ then makes it more expensive than it is: a 5-DP step charged 12.5 converts to a worse ε than the 5 you started with. This is the usual situation in local DP, where each user randomizes their own data. Keep such steps as ε or as an RDP curve; an RDP accountant can charge them the exact curve of randomized response, below both bounds.

29.4. From zCDP to (ε, δ)

Whatever you account in, you report (ε, δ). The conversion of From Rényi DP to (ε, δ) in the full edition gives a valid ε at every order, and you take the best. Write $\ell = \ln(1/\delta)$. For a line the conversion at order α is $\rho\alpha + \ell/(\alpha - 1)$: the first term grows with the order and the second shrinks,

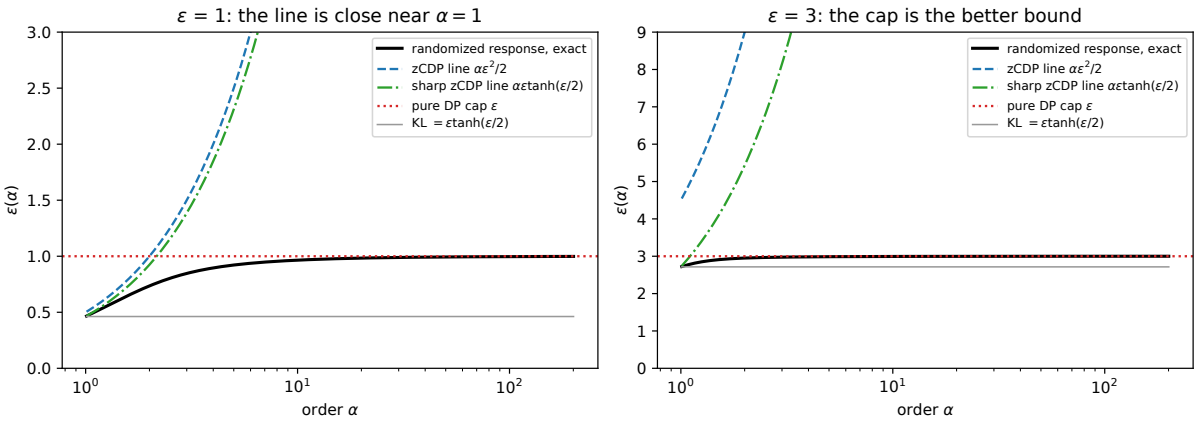


Figure 29.1.: The RDP curve of randomized response (black) against the zCDP line $\alpha\varepsilon^2/2$ (dashed), the sharp line $\alpha\varepsilon\tanh(\varepsilon/2)$ (dash-dotted) and the pure-DP cap ε (dotted), for $\varepsilon = 1$ (left) and $\varepsilon = 3$ (right). At $\varepsilon = 3$ even the sharp line crosses the cap almost at once.

so the best order balances them, at $\alpha^* = 1 + \sqrt{\ell/\rho}$. Substituting gives the **simple conversion** (Bun & Steinke 2016),

$$\varepsilon = \rho + 2\sqrt{\rho\,\ell}. \tag{29.3}$$

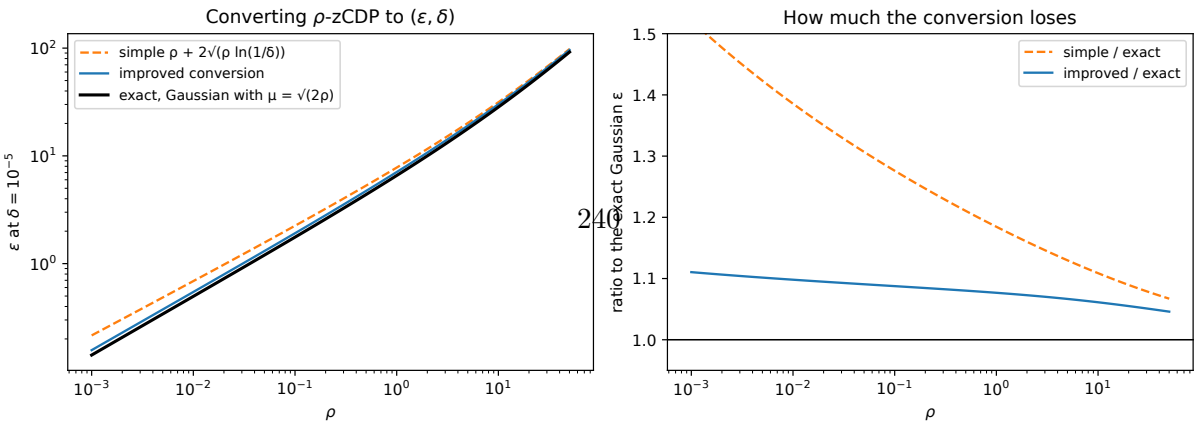
Modern accountants use the sharper **improved conversion** of Canonne, Kamath & Steinke (2020), which adds two correction terms at each order and is minimized numerically:

$$\varepsilon = \min_{\alpha>1}\left[\rho\alpha + \frac{\ell}{\alpha-1} + \ln\left(1-\frac{1}{\alpha}\right) - \frac{\ln\alpha}{\alpha-1}\right]. \tag{29.4}$$

How much do the two lose? For a Gaussian the line is the exact RDP curve, so any gap between the reported and the true ε comes from the conversion alone, and the true value is known in closed form (f-DP and Gaussian differential privacy in the full edition). One Gaussian step with $\sigma = 1$ is $\rho = 0.5$. At $\delta = 10^{-5}$, where $\ell = 11.5129$, the simple formula gives 5.299, the improved conversion 4.728, and the exact value is 4.377: the simple formula is 21% too large and the improved one 8% (Table 29.1, Figure 29.2).

Table 29.1.: ε at $\delta = 10^{-5}$ for a ρ-zCDP mechanism: simple and improved conversions, and the exact ε of the Gaussian mechanism with the same ρ.

ρ	Simple Equation 29.3	Improved Equation 29.4	Exact, Gaussian with the same ρ
0.01	0.689	0.546	0.497
0.1	2.246	1.914	1.760
0.5	5.299	4.728	4.377
1	7.786	7.077	6.573
5	20.174	19.047	17.857
50	97.985	96.035	91.817



So two systems with the same $\rho = 0.5$ can quote 5.299 and 4.728 only because they used different conversions; say which one you used. For a pipeline made only of Gaussian mechanisms on the full data, report the exact value.

To calibrate, run the simple conversion backwards. A target (ε, δ) is met by any

$$\rho \le \left(\sqrt{\varepsilon + \ell} - \sqrt{\ell}\right)^2, \tag{29.5}$$

and the noise multiplier is then $\sigma = 1/\sqrt{2\rho}$. A single Gaussian step at $(1, 10^{-5})$ needs $\rho = 0.02082$, that is $\sigma = 4.901$.

Because the map from ρ to ε is not linear, ε does not add and there is no clean "ε left": with a budget of $\rho = 1$, the whole run is 7.077-DP by the improved conversion however you split it. Add in ρ, and convert once, at the end.

29.5. Group privacy

A household, or a user with several records, differs from a neighbouring dataset in m records rather than one. For the Gaussian, the group moves the query's value m times as far, and its RDP grows with the square of the shift. zCDP inherits exactly this: a ρ-zCDP mechanism is $m^2\rho$-zCDP for datasets that differ in m records (Bun & Steinke 2016), with equality for the Gaussian.

Quadratic growth is the same price as pure DP's $m\varepsilon$, translated: with $\rho \approx \varepsilon^2/2$, $m^2\rho \approx (m\varepsilon)^2/2$. A user who contributes up to 4 records to a 0.1-zCDP release is protected at $\rho = 4^2 \times 0.1 = 1.6$. For the Gaussian with $\sigma = 1$, a group of 2 costs $\rho = 2$, or $\varepsilon = 10.725$ at $\delta = 10^{-5}$, and a group of 8 costs $\rho = 32$, or $\varepsilon = 68.616$. If users contribute many records, bound each user's contribution before adding noise and account at the user level.

29.6. Subsampling: why DP-SGD is not accounted in zCDP

DP-SGD runs each step on a Poisson sample: each record enters the batch independently with probability q. A record is usually not in the batch, so the step's RDP curve is tiny at small orders. For the part's running example, $q = 0.01$ and $\sigma = 1$, it stays around $10^{-4}\alpha$ up to $\alpha = 8$ (Figure 29.3).

High orders, however, weigh the rarest and most revealing outputs. For the subsampled step those come from the event that the record was in the batch, where it pays the full Gaussian loss. So at large orders the curve bends up, and its slope approaches that of the unsubsampled line $\alpha/(2\sigma^2)$ (Amplification by subsampling in the full edition explains the bend). A line through the origin above the whole curve must be as steep as the curve at infinity. For the Poisson-subsampled Gaussian, at any q, the smallest valid ρ is therefore $1/(2\sigma^2)$, exactly what the step would cost without subsampling. zCDP throws away all of the amplification.

With $k = 10{,}000$ steps and $\delta = 10^{-5}$, zCDP must charge $\rho = 0.5$ per step, so $\rho = 5000$ in total and $\varepsilon = 5479.9$ by the simple formula, or 5475.8 with the improved conversion. RDP accounting of the same run, on integer orders, gives 7.469 with Mironov's conversion and 6.719 with the

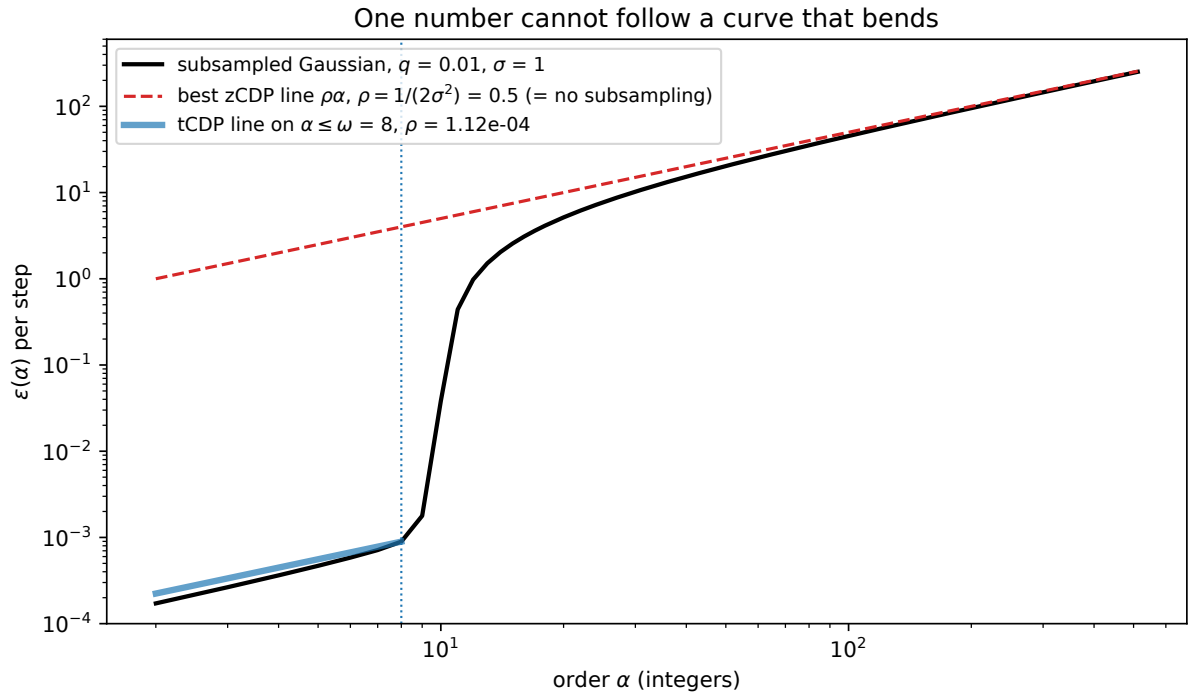


Figure 29.3.: The RDP curve of one Poisson-subsampled Gaussian step with $q = 0.01$, $\sigma = 1$, on integer orders (black), against the best zCDP line $\rho\alpha$ with $\rho = 0.5$ (red dashed), which is the unsubsampled Gaussian's line, and the tCDP line on orders up to $\omega = 8$ (blue). Both axes are logarithmic.

improved one. Both are upper bounds, but RDP kept the part of the curve the conversion uses: the small orders. The composed curve is $k\varepsilon(\alpha)$, and the conversion looks for the order where $k\varepsilon(\alpha) + \ell/(\alpha - 1)$ is smallest. Below the bend the first term is small, so the conversion pushes the order up until the bend makes the curve expensive, and stops early: here the best integer order is 4. The large orders that set the zCDP slope are never used.

Truncated concentrated differential privacy (tCDP) keeps one number per step but asks for the line only on orders up to a cut-off ω (Bun, Dwork, Rothblum & Steinke 2018). Steps compose to $(\sum_i \rho_i, \min_i \omega_i)$, and the conversion is the simple one when its best order lies below ω, otherwise order ω:

$$\varepsilon = \begin{cases} \rho + 2\sqrt{\rho\,\ell} & \text{if } \ell \le (\omega - 1)^2 \rho, \\ \rho\omega + \dfrac{\ell}{\omega - 1} & \text{otherwise.} \end{cases} \tag{29.6}$$

On orders up to $\omega = 8$, the per-step slope is 1.1171×10^{-4}, against zCDP's 0.5. With $\omega = 4$, tCDP gives 7.469, the same as RDP with Mironov's conversion, because both end up converting at order 4. But once ω passes the bend near $\alpha = 10$, the slope must cover the steep part, and tCDP collapses towards zCDP: 80.252 with $\omega = 10$, 2228.027 with $\omega = 16$.

So never account a subsampled mechanism in zCDP. If a pipeline mixes Gaussian releases with a DP-SGD run, keep it all in RDP: a ρ-zCDP step is just the RDP curve $\rho\alpha$ and joins an RDP ledger without loss. Nor is tCDP a practical substitute: RDP does at least as well as any choice of ω, and tCDP's general subsampling theorem is loose: with $\sigma = \sqrt{5}$ and a 1% sample it gives $\varepsilon = 9.037$ where RDP gives 2.046. That comparison is only indicative: the tCDP theorem assumes fixed-size batches and replace-one neighbours, while the RDP curve assumes Poisson sampling and add/remove (Amplification by subsampling in the full edition).

29.7. Approximate zCDP

Some useful mechanisms, such as stability-based histograms, are (ε, δ)-DP but cannot be zCDP at all: zCDP's strong group privacy rules them out. Approximate zCDP lets zCDP fail on an event of probability δ_a, so one ledger holds two columns. An (ε, δ)-DP step enters as $\rho = \varepsilon^2/2$ and $\delta_a = \delta$. The ρ column adds and converts as usual; the δ_a column adds up to at most $\sum_i \delta_i$, like δ in basic composition. At the end the pipeline is $(\varepsilon, \delta_a + (1 - \delta_a)\delta')$-DP, where ε is any zCDP conversion of ρ at δ' (Bun & Steinke 2016).

The gain comes from the first column. Ten stability-based steps, each $(0.5, 10^{-7})$-DP, and ten Gaussian steps with $\sigma = 1$ give $\rho = 10 \times 0.125 + 10 \times 0.5 = 6.25$ and $\delta_a = 1.0000 \times 10^{-6}$, so $\varepsilon = 22.100$ at total $\delta = 10^{-5}$. Composing the stability steps separately, by basic composition, gives $19.119 + 5 = 24.119$: in the unified ledger each $\varepsilon = 0.5$ step costs $\varepsilon^2/2$ in ρ rather than its full ε. A Gaussian step put in the δ_a column would waste δ for nothing.

29.8. Code

The whole zCDP ledger is a few lines: charge each Gaussian step its ρ, add, convert once. The improved conversion is minimized over the same grid of 20,001 orders that the full edition uses.

```
from math import log, sqrt


def zcdp_gaussian(sigma: float, sensitivity: float = 1.0) -> float:
    """rho of one Gaussian step; compose steps by adding their rho."""
    return sensitivity**2 / (2 * sigma**2)


def zcdp_to_eps_simple(rho: float, delta: float) -> float:
    return rho + 2 * sqrt(rho * log(1 / delta))


def zcdp_to_eps_improved(rho: float, delta: float) -> float:
    """Improved conversion, minimized over 20,001 log-spaced orders in (1, 1 + 1e4]."""
    best = float("inf")
    for i in range(20001):
        alpha = 1 + 10 ** (-4 + 8 * i / 20000)
        eps = rho * alpha + log(1 / delta) / (alpha - 1) + log(1 - 1 / alpha) - log(alpha) /
        best = min(best, eps)
    return best


def rho_for_target(eps: float, delta: float) -> float:
    """Largest rho whose simple conversion stays within eps at delta."""
    return (sqrt(eps + log(1 / delta)) - sqrt(log(1 / delta))) ** 2


rho = 10 * zcdp_gaussian(sigma=1.0)  # ten Gaussian steps with sigma = 1
print(f"{rho:.1f}")  # 5.0
print(f"{zcdp_to_eps_simple(rho, 1e-5):.3f}")  # 20.174
print(f"{zcdp_to_eps_improved(rho, 1e-5):.3f}")  # 19.047
r = rho_for_target(1.0, 1e-5)
print(f"{r:.5f} {1 / sqrt(2 * r):.3f}")  # 0.02082 4.901
```

The full edition's script, concentrated_dp.py, computes every number in this chapter, including the subsampled curve and the tCDP values:

```
uv run python code/part04-privacy-accounting/concentrated_dp.py
```

29.9. Cheat sheet

Here $\ell = \ln(1/\delta)$.

Table 29.2.: The results of this chapter.

Result	Formula	Notes
ρ-zCDP	$D_\alpha \le \rho\alpha$ for all $\alpha > 1$	RDP restricted to a line
Gaussian mechanism	$\rho = \Delta^2/(2\sigma^2)$	Exact: the line is the curve
Composition, post-processing	ρ's add; processing keeps ρ	Also for adaptive steps
Pure DP step	$\varepsilon^2/2$, sharply $\varepsilon\tanh(\varepsilon/2)$; exponential mechanism $\varepsilon^2/8$	Poor fit for large ε
To (ε, δ), simple	$\rho + 2\sqrt{\rho\ell}$	21% above exact at $\rho = 0.5$
To (ε, δ), improved	$\min_\alpha[\rho\alpha + \ell/(\alpha-1) + \ln(1 - 1/\alpha) - \ln\alpha/(\alpha-1)]$	8% above exact at $\rho = 0.5$
Calibration	$\rho = (\sqrt{\varepsilon+\ell} - \sqrt{\ell})^2$, $\sigma = 1/\sqrt{2\rho}$	Through the simple conversion
Group of m	$m^2\rho$	Exact for the Gaussian
Subsampled Gaussian	$\rho = 1/(2\sigma^2)$ whatever q	No amplification: use RDP
Approximate zCDP	ρ adds, δ_a adds; $(\varepsilon(\rho, \delta'), \delta_a + \delta')$-DP	For steps that are only (ε, δ)-DP

Rules of thumb

- Keep budgets in ρ, which adds, and convert to (ε, δ) once, at the end, with the improved conversion.
- One Gaussian step with $\sigma = 1$ costs $\rho = 0.5$, about $\varepsilon = 4.7$ at $\delta = 10^{-5}$.
- Charge a small-ε pure DP step $\varepsilon^2/2$; for ε of 2 or more, keep its ε or its RDP curve.
- Never account a subsampled mechanism, such as a DP-SGD step, in zCDP. Use RDP or a privacy loss distribution.

29.10. Further reading

- Bun & Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (TCC 2016): the definition, the Gaussian and pure-DP conversions, group privacy and approximate zCDP. Sections 1–3 are the core.
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020): the discrete Gaussian with its zCDP guarantee, and the improved conversion with a stable way to compute it.
- Bun, Dwork, Rothblum & Steinke, "Composable and Versatile Privacy via Truncated CDP" (STOC 2018): truncated CDP and its subsampling theorem.
- Abowd et al., "The 2020 Census Disclosure Avoidance System TopDown Algorithm" (Harvard Data Science Review, 2022): zCDP accounting at national scale, and the role of invariants.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

30. f-DP and Gaussian differential privacy

30.1. Why this matters

Every privacy guarantee answers one question: how well can an attacker who sees the output of a mechanism M tell whether it ran on a dataset D or on a neighbour D' that differs in one record? Differential privacy (DP) answers with two numbers (ε, δ), Rényi differential privacy (RDP) with a curve over orders α (Rényi DP in the full edition), and zero-concentrated differential privacy (zCDP) with one number ρ (Concentrated differential privacy in the full edition). But the question is a hypothesis test, and the complete answer to a test is its **trade-off function** $T(P, Q)$ between $P = M(D)$ and $Q = M(D')$: for each rate a at which a test may wrongly claim D' (its type I error), the smallest rate b at which it must wrongly claim D (its type II error) (Hypothesis testing). (a and b, because α is the Rényi order in this part.)

The one idea is that this curve is the guarantee, and every other notion is a summary of it that throws something away. Gaussian noise makes the curve practical: it produces a one-parameter family of curves G_μ that stays in the family under composition, so a pipeline of Gaussian steps has an exact guarantee in one number. And a central limit theorem (CLT) says that many small steps compose to a curve close to some G_μ, which gives a quick estimate, though not a guarantee, for differentially private stochastic gradient descent (SGD), DP-SGD for short.

30.2. f-DP

A privacy guarantee should say what an attacker cannot do. In testing terms, the strongest statement of that kind is a floor under the attacker's type II error at every type I error: however it tunes its test, it must miss at least this often. A floor at every a is a curve, so the definition takes a curve f as its parameter. A mechanism is **f-differentially private (f-DP)** if $T(M(D), M(D')) \geq f$ for all neighbours $D \simeq D'$ (Dong, Roth & Su 2022). Processing an output can only make two distributions harder to tell apart, because any test on the processed output is also a test on the raw one, so the curve can only rise and f-DP survives post-processing.

The familiar (ε, δ)-DP is one such floor, made of two straight lines:

$$f_{\varepsilon,\delta}(a) = \max\left(0,\ 1 - \delta - e^{\varepsilon} a,\ e^{-\varepsilon}(1 - \delta - a)\right). \tag{30.1}$$

The first line is the definition read as a test. A test that fires with probability a under D may fire with probability at most $e^{\varepsilon} a + \delta$ under D', so it misses D' with probability at least $1 - \delta - e^{\varepsilon} a$. The second line is the same statement with the roles of D and D' swapped. The floor starts at $1 - \delta$, not at 1: δ is a free shot, a chance to catch D' at no type I cost (Figure 30.1).

Two lines are a crude floor. Three mechanisms carry the label "$\varepsilon = 1$" in Table 30.1. Randomized response, which reports a bit truthfully with probability $e/(1 + e)$, sits exactly on $f_{1,0}$: it is the

worst $(1, 0)$-DP mechanism. The Laplace mechanism matches it at small a but is more private in the middle, 0.3679 against 0.3204 at $a = 0.25$. The label cannot tell them apart; the curves can. The floor is set by the worst neighbouring pair; most records are better protected.

Table 30.1.: Smallest type II error at three type I errors for three mechanisms.

Type I error a	$f_{1,0}$ (randomized response)	Laplace, $\varepsilon = 1$	Gaussian, $\sigma = 1$ (G_1)
0.01	0.9728	0.9728	0.9076
0.1	0.7282	0.7282	0.6109
0.25	0.3204	0.3679	0.3724

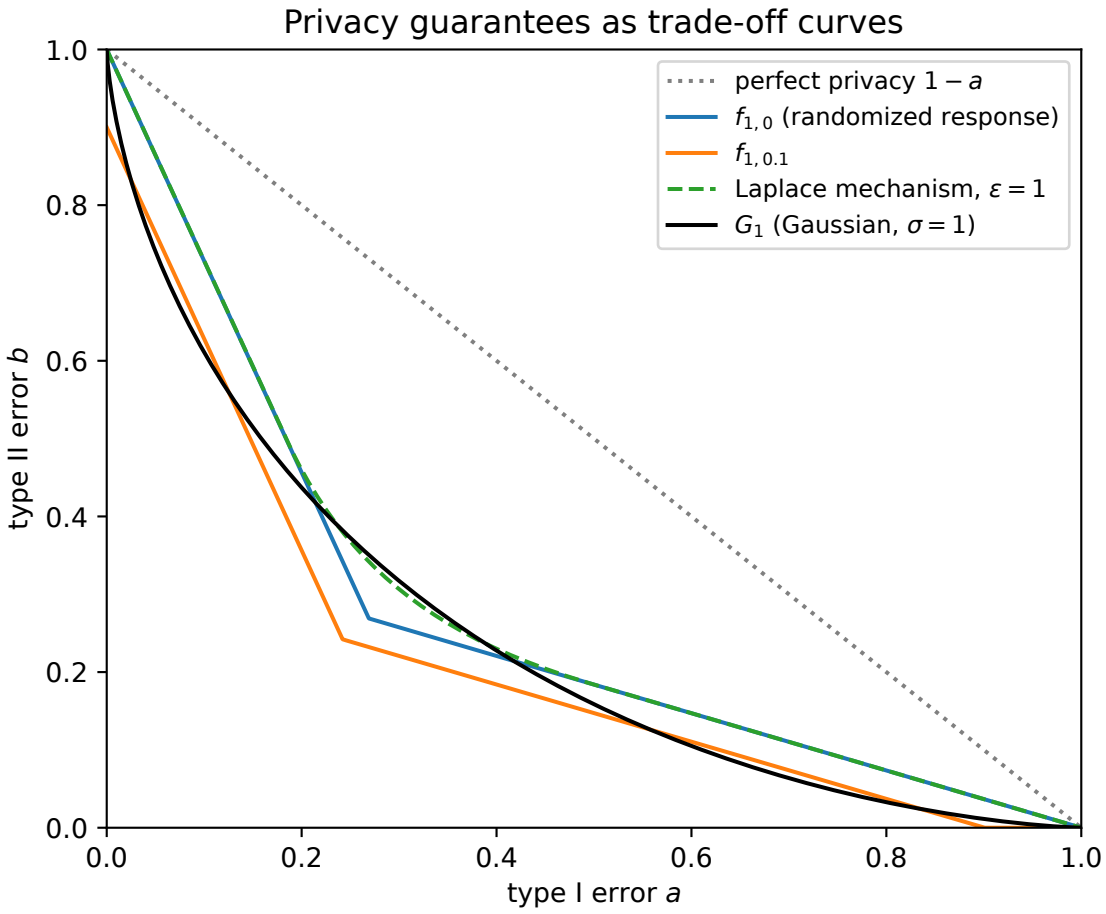


Figure 30.1.: Trade-off curves of $(1, 0)$-DP (randomized response), $(1, 0.1)$-DP, the Laplace mechanism with $\varepsilon = 1$ and the Gaussian mechanism with $\sigma = 1$. A mechanism is f-DP when its curve lies on or above f; the dotted diagonal is perfect privacy.

30.3. Gaussian differential privacy

The Gaussian mechanism's curve takes two steps to find, neither of which changes any test's errors. Divide the output by the noise standard deviation and subtract the mean under D: both maps can be undone, so a test on the new output is a test on the old one. What is left is telling $\mathcal{N}(0,1)$ from $\mathcal{N}(\mu,1)$, where μ is the distance between the two output means in units of the noise. For a vector output only the direction of the shift carries evidence, since the noise in every other direction looks the same under both hypotheses, so the problem is always one-dimensional. The best test fires when the output exceeds a threshold. At type I error a the threshold is $\Phi^{-1}(1-a)$, where Φ is the standard normal distribution function, and a draw from the shifted normal stays below it with probability

$$G_\mu(a) = \Phi(\Phi^{-1}(1-a) - \mu). \tag{30.2}$$

A mechanism is **μ-Gaussian differential privacy (GDP)** if it is G_μ-DP. The Gaussian mechanism with noise multiplier σ (the noise standard deviation in units of the ℓ_2 sensitivity) shifts the mean by at most one sensitivity, so it is $1/\sigma$-GDP, and the bound is exact for a pair that attains the sensitivity. For Gaussian noise, μ is the curve, not a bound with slack.

It reads as attack errors. A count released with $\sigma = 1$ is 1-GDP, so at a 1% false-positive rate, whatever the attacker knows about the rest of the data, it detects the presence of a record at most 9.24% of the time ($G_1(0.01) = 0.9076$); at a 10% false-positive rate its power is at most 0.3891. Dong et al. suggest that $\mu \le 0.5$ is a reasonable guarantee and $\mu \ge 6$ promises almost nothing.

To report (ε, δ) without losing anything, use the shape of the curve. A trade-off curve is convex, and a convex curve is the upper envelope of the straight lines that touch it. Each touching line is one (ε, δ) floor, so a symmetric curve is the same guarantee as being $(\varepsilon, \delta(\varepsilon))$-DP for every $\varepsilon \ge 0$ at once. That function, the **privacy profile**, has a closed form for G_μ:

$$\delta(\varepsilon) = \Phi\Big(-\frac{\varepsilon}{\mu} + \frac{\mu}{2}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\varepsilon}{\mu} - \frac{\mu}{2}\Big). \tag{30.3}$$

In words, take the test whose curve has slope $-e^{\varepsilon}$; the first term is its power and the second is e^{ε} times its type I error, so δ is how far that test beats what pure ε-DP would allow. This is the same formula as the analytic Gaussian calibration of The Laplace and Gaussian mechanisms. $\delta(\varepsilon)$ falls as ε grows, so to report ε at a target δ you solve by bisection (Table 30.2).

Table 30.2.: The ε of μ-GDP at $\delta = 10^{-5}$.

μ	0.5	1	2
ε at $\delta = 10^{-5}$	1.9931	4.3772	9.9973

The profile also explains what a small δ costs. δ is set by the first term, a normal tail, and a normal tail falls like $e^{-x^2/2}$, so reaching δ needs $x = \varepsilon/\mu - \mu/2$ of about $\sqrt{2\ln(1/\delta)}$, and ε grows roughly like $\mu\sqrt{2\ln(1/\delta)}$. Each decade of δ therefore costs extra ε in proportion to μ, a little less with each decade: at $\mu = 1$, going from $\delta = 10^{-4}$ to 10^{-5} to 10^{-6} raises ε from 3.8044 to 4.3772 to 4.8866, and at $\mu = 2$ the steps are about twice as large.

The whole profile loses nothing, but one point of it does. "1-GDP" implies "$(4.3772, 10^{-5})$-DP", but the single pair is one touching line and permits far more: at $a = 0.1$ it allows $b = 0.0113$, while $G_1(0.1) = 0.6109$ (Figure 30.2, right). Report μ, or the noise multiplier, along with any pair.

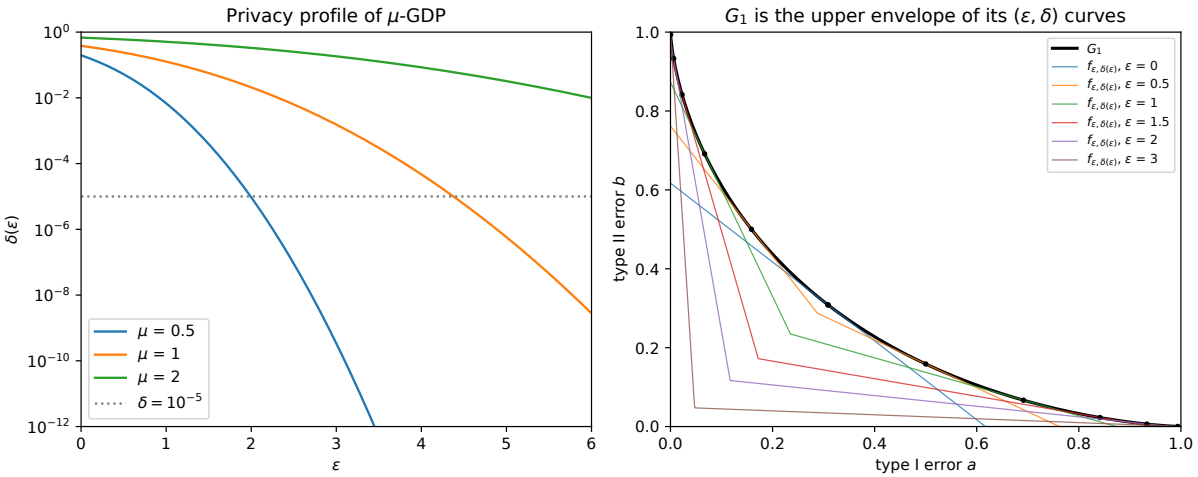


Figure 30.2.: Left: the privacy profile $\delta(\varepsilon)$ of μ-GDP for $\mu = 0.5, 1, 2$, with the level $\delta = 10^{-5}$ dotted. Right: G_1 and six of its $(\varepsilon, \delta(\varepsilon))$ curves; their upper envelope is G_1, so the whole profile loses nothing, while one pair keeps a single touching line.

30.4. Composition

An attacker who sees two outputs with independent noise faces the joint distributions, and their trade-off curve is the composed guarantee. For two Gaussian releases the joint problem is a shift in two dimensions, by μ_1 along one axis and μ_2 along the other. Only the direction of the shift carries evidence, as before, so this is a one-dimensional shift by the length of that vector, $\sqrt{\mu_1^2 + \mu_2^2}$ (Figure 30.3). Repeating the argument, k steps compose to

$$G_{\mu_1} \otimes \cdots \otimes G_{\mu_k} = G_{\mu}, \qquad \mu = \sqrt{\mu_1^2 + \cdots + \mu_k^2}, \tag{30.4}$$

even when each step is chosen after seeing the earlier outputs, and no better bound holds in general (Dong et al. 2022). μ^2 adds, just as the variances of independent noises add. Full-batch noisy gradient descent with noise multiplier σ for k steps is therefore exactly $\sqrt{k}/\sigma$-GDP. zCDP does the same arithmetic with $\rho = \mu^2/2$, but ends with an RDP bound on the curve rather than the curve.

Two 1-GDP releases compose to $G_{1.4142}$, four to G_2, and 100 releases with $\sigma = 10$ each to G_1. Four 1-GDP releases are therefore $\varepsilon = 9.9973$ at $\delta = 10^{-5}$. Suppose instead you first turn each release into one (ε, δ) point, giving each a quarter of the δ budget, $(4.6902, 2.5 \times 10^{-6})$, and then add the points by basic composition. You get $\varepsilon = 18.7607$, almost twice as large, because each point had already dropped most of its curve. Compose curves, not points.

For curves that are not Gaussian there is no closed form, and composing (ε, δ) guarantees exactly is computationally intractable when the steps differ. Numerical accountants instead compose the distribution of the privacy loss by convolution (Numerical accounting in the full edition). A subsampled step's curve is not Gaussian either, which is where the CLT comes in.

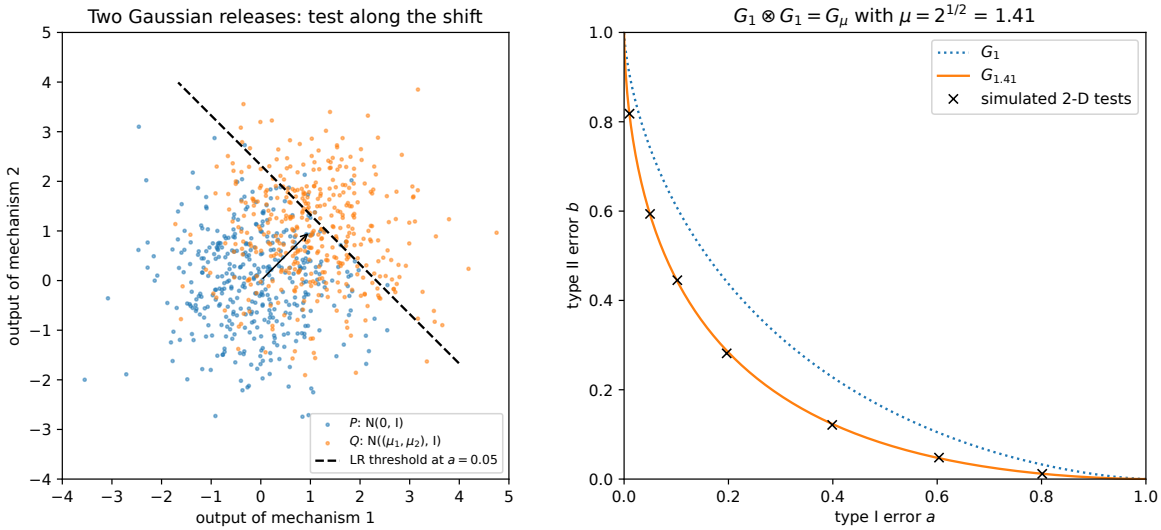


Figure 30.3.: Left: 400 draws each from $\mathcal{N}(0, I_2)$ and $\mathcal{N}((1,1), I_2)$, the outputs of two Gaussian releases on D and D'; the best test thresholds the projection on the shift. Right: G_1 for one release, $G_{\sqrt{2}}$ for two, and simulated errors of the projected test.

30.5. The central limit theorem and DP-SGD

The **privacy loss** of an output is the log of how much more likely it is under D than under D'. Independent outputs have densities that multiply, so the loss of a composition is the sum of the per-step losses. A sum of many small independent terms is nearly normal, and a normal privacy loss is exactly what a Gaussian curve has. So many small steps compose to roughly some G_μ, whatever the steps were.

DP-SGD is the case that matters. Each step runs on a Poisson sample that includes each record with probability q, clips each gradient and adds Gaussian noise with noise multiplier σ. When q is small the record is usually absent, so each step leaks little. To leading order in q, one step's privacy loss has mean $q^2(e^{1/\sigma^2} - 1)/2$ and variance $q^2(e^{1/\sigma^2} - 1)$, and a loss whose variance is twice its mean is exactly the shape of a Gaussian curve's loss, whose variance is μ^2. Adding k steps adds the variances, which gives, under add/remove neighbours, as $k \to \infty$ with $q\sqrt{k}$ held fixed,

$$\mu = q\sqrt{k(e^{1/\sigma^2} - 1)} \tag{30.5}$$

(Bu, Dong, Long & Su 2020). Compared with full-batch training, $\sqrt{k}/\sigma$, subsampling multiplies μ by about $q\sigma\sqrt{e^{1/\sigma^2} - 1}$, which is $q \times 1.3108$ at $\sigma = 1$.

This is a limit, used at finite k as an approximation. It is not a bound, and for the running example, $q = 0.01$ and $\sigma = 1$, it errs on the unsafe side. Table 30.3 turns each μ into ε at $\delta = 10^{-5}$ and compares it with a numerical accountant's bracket, whose lower end is a lower bound on the true ε, exact up to a small quadrature error. At $k = 10{,}000$ the CLT says 6.0071; the true ε is at least 6.1377 (Figure 30.4).

Table 30.3.: The running DP-SGD example: the CLT's μ and ε, and a numerical bracket for the true ε.

Steps k	CLT μ	CLT ε at 10^{-5}	Numerical accountant ε	Full batch, $\mu = \sqrt{k}$
100	0.1311	0.4575	[0.7175, 0.7185]	10.0
1,000	0.4145	1.6177	[1.8232, 1.8332]	31.6
10,000	1.3108	6.0071	[6.1377, 6.2377]	100.0

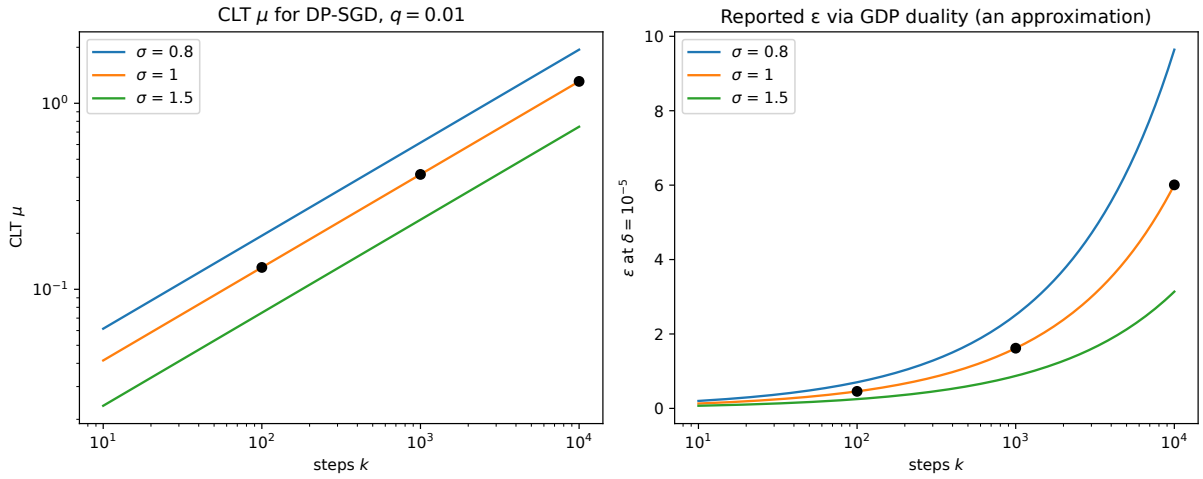


Figure 30.4.: The CLT for DP-SGD with $q = 0.01$. Left: $\mu = q\sqrt{k(e^{1/\sigma^2} - 1)}$ against the number of steps for three noise multipliers. Right: the ε at $\delta = 10^{-5}$ it implies. Dots mark the running example at $k = 100$, 1,000 and 10,000.

The reason it under-reports is where δ lives. A CLT controls the bulk of the loss distribution, but a δ of 10^{-5} is decided by rare, large values of the loss, in the tail, where the normal approximation has no control. The error is worst for short runs, furthest from the regime the limit describes: 0.4575 against at least 0.7175 at $k = 100$. Plugging exactly computed per-step moments into the CLT does not help, and gives an even smaller ε, 5.9184 at $k = 10{,}000$. A more exact μ is not a more conservative one; only a certified accountant is. Plan with the CLT; report an accountant's number.

Two more cautions. The formula is specific to Poisson sampling under add/remove neighbours: Dong et al.'s version for fixed-size batches under replace-one neighbours has a different constant, $\mu = 1.7101$ at $k = 10{,}000$ here, and the two must not be mixed (Amplification by subsampling in the full edition). And μ is very sensitive to σ through $e^{1/\sigma^2} - 1$, which is 3.7707 at $\sigma = 0.8$, 1.7183 at $\sigma = 1$ and 0.5596 at $\sigma = 1.5$.

30.6. RDP as a lossy summary of the curve

The curve also shows where RDP accounting leaves slack. An RDP curve records the exponential moments of the privacy loss, one per order. Different loss distributions can share those moments,

so the best trade-off curve that the RDP values alone can certify, f_{RDP}, has to allow the worst of them. It lies below the true curve even for the Gaussian, whose RDP curve is exact. The best (ε, δ) any RDP conversion can report is the profile of f_{RDP} (From Rényi DP to (ε, δ) in the full edition).

So the slack of RDP accounting has two parts: the conversion, which a better formula removes, and the gap between f_{RDP} and the true curve, which no conversion removes. For 1-GDP, whose RDP curve is $\varepsilon(\alpha) = \alpha/2$, the RDP summary allows an attacker an advantage of 0.4830 where the best possible is 0.3829, and the best $\delta(1)$ it can certify is 0.2149, 1.69 times the exact 0.1269. For Gaussian noise on the full data, then, account in GDP, where μ^2 adds and the profile at the end is exact.

30.7. Code

A GDP accountant needs only the normal distribution function, which the standard library provides through `math.erfc`:

```
from math import erfc, exp, sqrt


def normal_cdf(x: float) -> float:
    return 0.5 * erfc(-x / sqrt(2))


def gdp_delta(eps: float, mu: float) -> float:
    """The exact delta(eps) of mu-GDP."""
    return normal_cdf(-eps / mu + mu / 2) - exp(eps) * normal_cdf(-eps / mu - mu / 2)


def gdp_eps(mu: float, delta: float) -> float:
    """The eps at which mu-GDP reaches delta; delta(eps) falls as eps grows, so bisect."""
    lo, hi = 0.0, 500.0
    for _ in range(200):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if gdp_delta(mid, mu) > delta else (lo, mid)
    return hi


def compose_gdp(*mus: float) -> float:
    return sqrt(sum(m * m for m in mus))


def clt_mu_dpsgd(q: float, sigma: float, steps: int) -> float:
    """Approximate mu of Poisson-subsampled DP-SGD. An estimate, not a bound."""
    return q * sqrt(steps * (exp(1 / sigma**2) - 1))
```

```
print(f"{gdp_eps(1.0, 1e-5):.4f}")  # 4.3772
print(f"{gdp_eps(compose_gdp(1, 1, 1, 1), 1e-5):.4f}")  # 9.9973
mu = clt_mu_dpsgd(q=0.01, sigma=1.0, steps=10_000)
print(f"{mu:.4f} {gdp_eps(mu, 1e-5):.4f}")  # 1.3108 6.0071
```

The last line is the CLT estimate for the running example, below the numerical lower bound 6.1377. The full edition's script, gaussian_dp.py, computes every number in this chapter, including the numerical brackets:

```
uv run python code/part04-privacy-accounting/gaussian_dp.py
```

30.8. Cheat sheet

Table 30.4.: The results of this chapter.

Result	Formula	Notes
f-DP	$T(M(D), M(D')) \geq f$ for all neighbours	Type I error a, type II error b
(ε, δ)-DP as a curve	$f_{\varepsilon,\delta}(a) = \max(0, 1-\delta-e^{\varepsilon}a, e^{-\varepsilon}(1-\delta-a))$	Two straight lines
μ-GDP	$G_{\mu}(a) = \Phi(\Phi^{-1}(1-a)-\mu)$	Gaussian mechanism: $\mu = 1/\sigma$, exact
Profile	$\delta(\varepsilon) = \Phi(-\frac{\varepsilon}{\mu}+\frac{\mu}{2}) - e^{\varepsilon}\Phi(-\frac{\varepsilon}{\mu}-\frac{\mu}{2})$	Lossless; bisect for ε
Composition	$\mu = \sqrt{\sum_i \mu_i^2}$	Exact for Gaussian steps
DP-SGD, CLT	$\mu \approx q\sqrt{k(e^{1/\sigma^2}-1)}$	An approximation, not a bound; under-reports ε here
RDP from the curve	RDP implies only f_{RDP}, below the true curve	Slack no conversion removes

Rules of thumb

- The trade-off curve is the guarantee; (ε, δ), RDP and ρ summarize it. Summarize as late as possible.
- For Gaussian noise without subsampling, account in GDP: μ^2 adds, and the (ε, δ) at the end is exact.
- $\mu = 1$ means $\varepsilon \approx 4.4$ at $\delta = 10^{-5}$; $\mu = 0.5$ means about 2.
- Use the CLT for DP-SGD to plan, never as the reported guarantee.

30.9. Further reading

- Dong, Roth & Su, "Gaussian Differential Privacy" (Journal of the Royal Statistical Society Series B, 2022): f-DP, GDP, duality, composition and the CLTs. The discussion sections explain why the curve is the right object.
- Bu, Dong, Long & Su, "Deep Learning with Gaussian Differential Privacy" (Harvard Data Science Review, 2020): the CLT for Poisson-subsampled DP-SGD, with experiments.
- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021): an accountant with error bounds, and examples where the CLT under-reports ε.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy" (ICML 2018): the exact (ε, δ) profile of the Gaussian mechanism and its use for calibration.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

31. Amplification by subsampling

31.1. Why this matters

Differential privacy (DP) for deep learning would be useless without one trick. Stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD, runs ten thousand steps or more. Each step clips every example's gradient, sums them and adds Gaussian noise whose standard deviation is the **noise multiplier** σ times the clipping norm. If every record paid at every step, ten thousand steps with $\sigma = 1$ would cost $\varepsilon = 5425.5$ at $\delta = 10^{-5}$, a number nobody would report. The trick is that each step looks only at a small random batch. If a record is in the batch with probability q, an attacker must first guess whether it was used at all; with $q = 0.01$ that uncertainty turns a step that costs $\varepsilon = 1$ into one that costs about 0.017.

The one idea: **the output with the record is a mixture, "used with probability q, unused otherwise", and mixing pulls every likelihood ratio towards 1.** How much a step is worth depends on how far the mixture pulls, and because the mixture comes from how the batch was drawn, an accountant that assumes one sampler while the training code runs another reports a number that does not apply.

31.2. Amplification for (ε, δ)-DP under Poisson sampling

Under **Poisson sampling** each record enters the batch by its own independent coin flip with probability q. Take add/remove neighbours, D and $D' = D \cup \{x\}$, and fix which other records were drawn. On D the step outputs some distribution P_0. On D' it outputs P_1 if x was drawn and P_0 otherwise, so it outputs the mixture $(1-q)P_0 + qP_1$. Divide by P_0 and the likelihood ratio P_1/P_0 becomes $(1-q) + q\,P_1/P_0$, a weighted average of 1 and the old ratio. If the step was (ε, δ)-DP, the old ratio was at most e^ε outside the δ slack, so the new one is at most $1 + q(e^\varepsilon - 1)$, and the slack only matters when the record was drawn. The result (Balle, Barthe & Gaboardi 2018) is

$$\varepsilon' = \ln\left(1 + q(e^\varepsilon - 1)\right), \qquad \delta' = q\delta, \tag{31.1}$$

for add/remove neighbours, and randomized response shows it cannot be improved. Randomized response at $\varepsilon = 1$, sampled at $q = 0.1$, has its likelihood ratio pulled from $e = 2.718$ to 1.1718, so $\varepsilon' = 0.1586$.

For small ε, $e^\varepsilon - 1 \approx \varepsilon$ and the logarithm is nearly linear, so $\varepsilon' \approx q\varepsilon$: a 1% sample makes the step a hundred times cheaper. For large ε, once $qe^\varepsilon \gg 1$, the 1 inside the logarithm is negligible and $\varepsilon' \approx \varepsilon - \ln(1/q)$: sampling only subtracts. The reason is the worst case: with probability q the record was drawn, and sampling can make that event rare but not less revealing. At $q = 0.01$ a step with $\varepsilon = 1$ costs 0.0170, but one with $\varepsilon = 5$ still costs 0.9059, not 0.05, and one with $\varepsilon = 10$

costs 5.3993, close to $10 - \ln 100 = 5.395$ (Table 31.1). The switch happens near $\varepsilon \approx \ln(1/q)$, where qe^{ε} reaches 1 (Figure 31.1).

Table 31.1.: The amplified $\varepsilon' = \ln(1 + q(e^{\varepsilon} - 1))$ for three sampling rates.

q	$\varepsilon = 0.1$	$\varepsilon = 1$	$\varepsilon = 5$	$\varepsilon = 10$
0.01	0.0011	0.0170	0.9059	5.3993
0.1	0.0105	0.1586	2.7563	7.6978
0.5	0.0512	0.6201	4.3136	9.3069

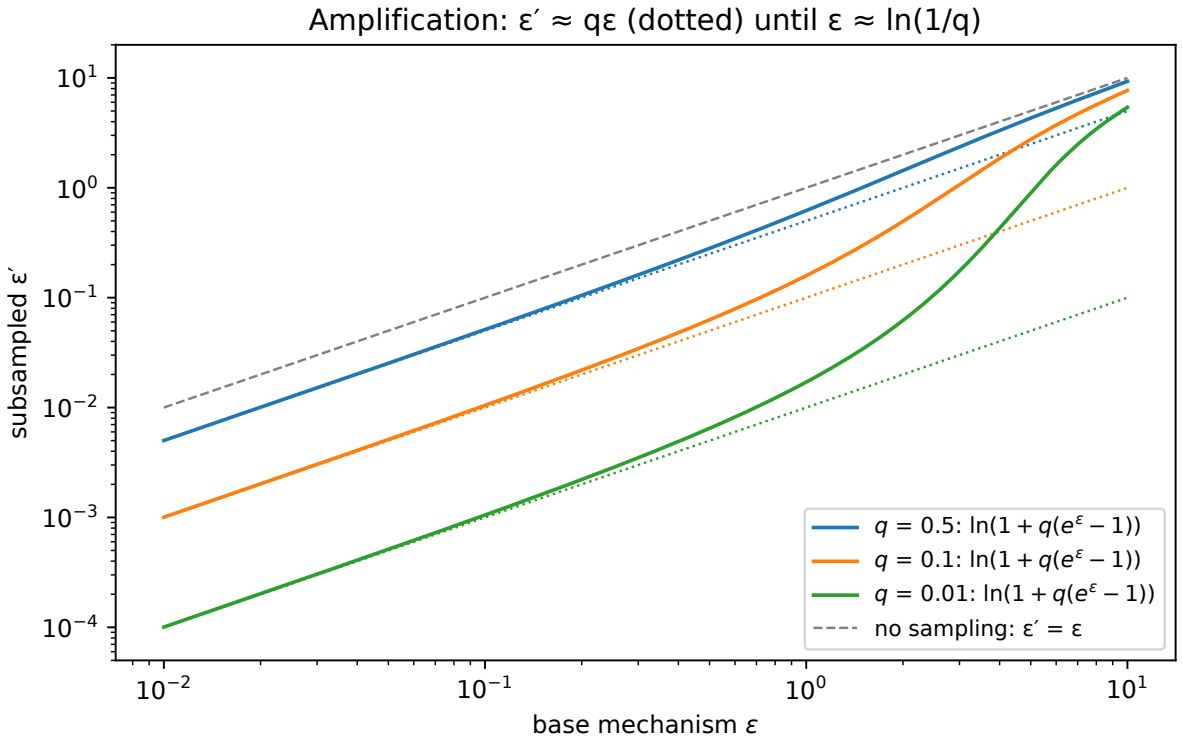


Figure 31.1.: The subsampled ε' against the base ε for three sampling rates, with the small-ε approximation $q\varepsilon$ dotted and no sampling dashed. Amplification is a factor q for small ε and only a shift by $\ln(1/q)$ for large ε.

Since the amplification is the attacker's uncertainty about whether the record was used, each record's coin must be independent of the rest of the sample, and the sample must stay secret. And Equation 31.1 is tight for one step only; composing per-step pairs over thousands of steps is hopeless (Section 31.4).

31.3. The sampling scheme must match the neighbouring relation

Poisson sampling gives batches of random size, and most training code draws fixed-size batches instead, either a fresh random batch of m out of n records at each step or a random permutation

cut into consecutive batches each epoch (**shuffle and partition**). A fixed batch size pairs naturally with **replace-one** neighbours, datasets of the same size that differ in one record's value, because adding a record would change the size. For a uniformly random batch, a record is used with probability m/n, and Equation 31.1 holds with $q = m/n$, provided the base guarantee is for replace-one neighbours (Balle, Barthe & Gaboardi 2018).

That proviso is where the cost hides. Replacing a clipped gradient can move the gradient sum twice as far as adding one: the old gradient, of norm up to C, leaves, and a new one, possibly pointing the opposite way, comes in. Twice the shift with the same noise is the same as half the noise multiplier. Table 31.2 lists what to account for under each combination, for DP-SGD's sum of gradients clipped to norm C with noise of standard deviation σC.

Table 31.2.: What to account for under each combination of sampling scheme and neighbouring relation.

Sampling	Neighbours	What to account for
Poisson, rate q	add/remove	The Poisson-subsampled Gaussian with noise multiplier σ
Fixed size m of n	replace-one	Mixtures on both sides with weight m/n; the base sums differ by $2C$, so noise multiplier $\sigma/2$
Fixed size m of n	add/remove	Dominated by the Poisson pair with noise multiplier $\sigma/2$ (Lebeda et al. 2024)
Poisson, rate q	replace-one	Both sides are mixtures and the sums can differ by $2C$; for pure DP about twice the add/remove cost at small ε
Shuffle and partition	zero-out (the target's gradient set to 0)	No simple formula; the record is in exactly one batch per epoch, never absent

Halving the noise multiplier is not a small change: for the part's running example, $q = 0.01$, $\sigma = 1$, 10,000 steps and $\delta = 10^{-5}$, fixed-size batches give $\varepsilon = 47.42$ against 6.71 for Poisson sampling, both by Rényi differential privacy (RDP) accounting, the first with noise multiplier 0.5.

Shuffle and partition is worse, because the source of amplification is missing altogether. In one epoch the record is in exactly one batch, never absent, so no step's output is a mixture with "not used". The only uncertainty is which batch held it, and the other records can give that away; Chua et al. (2024) build such a case, with a test that gives a lower bound on the true ε. For one epoch of 100 batches, Table 31.3 compares that lower bound with the Poisson accountant's upper bound and with deterministic batches, where the epoch is a plain Gaussian mechanism and its ε is exact. At $\sigma = 1$, shuffled training has $\varepsilon \geq 4.06$, while the Poisson accountant reports at most

1.21 (Figure 31.2), and shuffling buys almost nothing over a fixed batch order (4.38). At $\sigma = 3$ the order flips, which says only that this particular test is too weak there to show a gap.

Table 31.3.: The ε at $\delta = 10^{-5}$ of one epoch of 100 batches under three batch samplers. Wherever the shuffled column exceeds the Poisson column, the Poisson number is provably wrong for shuffled training.

σ	Deterministic batches (exact)	Shuffled batches (lower bound)	Poisson sampling (upper bound)
0.5	10.00	10.00	8.03
1	4.38	4.06	1.21
1.5	2.75	1.46	0.47
2	1.99	0.48	0.26
3	1.27	0.10	0.14

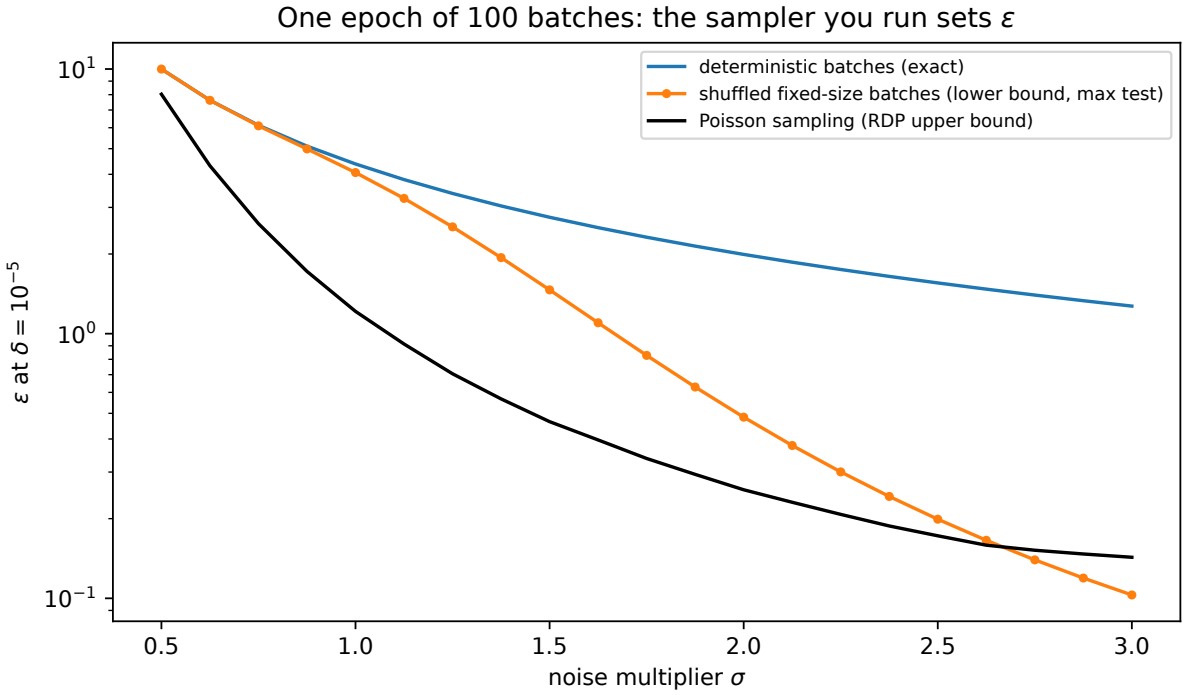


Figure 31.2.: The ε at $\delta = 10^{-5}$ of one epoch of 100 batches as a function of the noise multiplier: deterministic batches (exact), shuffled fixed-size batches (a lower bound) and Poisson sampling with $q = 0.01$ (an RDP upper bound). For σ up to about 2.5, shuffled training is provably less private than the Poisson accountant claims.

This is the largest single risk in DP-SGD accounting, larger than the choice of accountant: read the data loader before you trust a reported ε. If it draws fixed-size batches, give a Poisson accountant half the noise multiplier you actually used. If it shuffles and partitions, the Poisson number does not apply at all: switch to Poisson sampling, or say plainly that the reported ε assumes a sampler you did not run. Always state the neighbouring relation with ε.

31.4. The RDP accountant for DP-SGD

A mechanism's **RDP curve** $\varepsilon(\alpha)$ bounds the Rényi divergence of order $\alpha > 1$ between its outputs on neighbours, and under composition the curves of the steps add (Rényi DP in the full edition). So you need the curve of one subsampled Gaussian step. The other records contribute the same to both outputs and can be subtracted, and Mironov, Talwar & Zhang (2019) show that the worst case is one coordinate in which the record moves the clipped sum by the full sensitivity: a Gaussian $P_0 = \mathcal{N}(0, \sigma^2)$ against the mixture $P_q = (1-q)\mathcal{N}(0, \sigma^2) + q\,\mathcal{N}(1, \sigma^2)$. Of the two orders of the pair, the one with the mixture first is always the larger.

The Rényi divergence of order α is an average of the likelihood ratio raised to the power α. The ratio is $(1-q) + q\,r$, with r the ratio of the shifted Gaussian to P_0, and for an integer α the binomial theorem expands its power into a finite sum. With sensitivity 1 and add/remove neighbours, $\varepsilon(\alpha) = \ln A_\alpha/(\alpha-1)$, where

$$A_\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j \exp\Big(\frac{j^2 - j}{2\sigma^2}\Big). \tag{31.2}$$

Term j is the case in which j of α imagined copies of the record's contribution are present, weighted by its chance. At $\alpha = 2$, with $q = 0.01$ and $\sigma = 1$, the sum has three terms, $0.9801 + 0.0198 + 0.0001e$, so $A_2 = 1 + q^2(e-1) = 1.00017183$ and $\varepsilon(2) = 1.7181 \times 10^{-4}$, against 1 without sampling. Fractional orders need numerical integration.

After k steps the curve is $k\,\varepsilon(\alpha)$, and you convert once, at the best order:

$$\varepsilon = \min_\alpha \Big[k\,\varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}\Big] \text{ (Mironov)}, \qquad \varepsilon = \min_\alpha \Big[k\,\varepsilon(\alpha) + \ln\frac{\alpha-1}{\alpha} - \frac{\ln\delta + \ln\alpha}{\alpha - 1}\Big] \text{ (improved)}. \tag{31.3}$$

Table 31.4 compares this with the per-step route, which amplifies each step by Equation 31.1 and composes the pairs. After 10,000 steps the RDP accountant with the improved conversion gives $\varepsilon = 6.71$; per-step amplification followed by basic composition gives 11296.93 (Figure 31.3).

Table 31.4.: The ε at $\delta = 10^{-5}$ of DP-SGD with $q = 0.01$, $\sigma = 1$ under four accounting routes.

Steps k	RDP, Mironov	RDP, improved	Best order (improved)	Per step, basic	Per step, advanced
1	1.32	0.96	9.9	0.20	1.13
10	1.41	1.04	9.4	3.64	7.88
100	1.61	1.21	8.8	58.01	78.10
1,000	2.54	2.10	7.8	839.39	1258.64
10,000	7.43	6.71	4.1	11296.93	24379.55

The per-step route fails because of the shift regime of the previous section. To end at $\delta = 10^{-5}$ after 10,000 steps, basic composition can give each step only a tiny δ, and a tiny δ forces a large per-step ε, here 5.35. That is above $\ln(1/q)$, where sampling only subtracts, so the amplified step costs 1.130, only about five times less, and basic composition multiplies that by 10,000. The RDP accountant avoids the trap: each step costs almost nothing at moderate orders, and the conversion term $\ln(1/\delta)/(\alpha-1)$ is paid once.

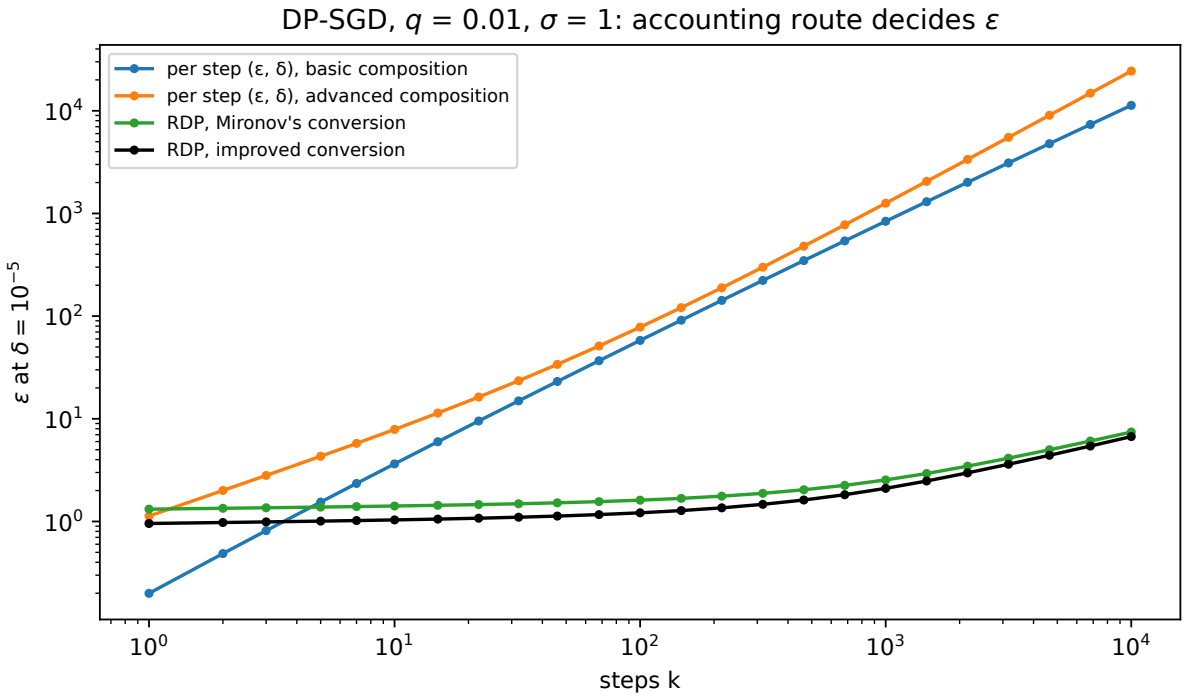


Figure 31.3.: The ε at $\delta = 10^{-5}$ of DP-SGD with $q = 0.01$, $\sigma = 1$ against the number of steps, under four accounting routes. The RDP accountant grows slowly; per-step amplification followed by basic or advanced composition is useless beyond a few steps.

That one-time term is also why RDP loses for a single step, 0.96 against 0.20. RDP wins from $k = 4$ here. The best order falls from 9.9 to 4.1 as k grows, because a larger $k\,\varepsilon(\alpha)$ pulls the minimum towards smaller orders, where the curve is lower; make sure the accountant's grid includes small and fractional orders, since removing orders can only raise ε. The improved conversion saves 9.6% at 10,000 steps (6.71 against 7.43). And 6.71 is an upper bound with slack, from the conversion and from the RDP curve itself; Numerical accounting in the full edition computes the tight value and Choosing an accountant in the full edition compares them.

31.5. Small q and the moments accountant

The sum Equation 31.2 also explains the shape of the curve. If every exponential in it were 1, the sum would be $((1-q)+q)^\alpha = 1$, and the terms $j = 0$ and 1 have exponential exactly 1, so the excess over 1 starts at $j = 2$. For small q that term dominates and gives

$$\varepsilon(\alpha) \approx \frac{\alpha q^2 (e^{1/\sigma^2} - 1)}{2} \approx \frac{\alpha q^2}{2\sigma^2} \text{ for large } \sigma. \tag{31.4}$$

The cost is quadratic in q, where the (ε, δ) amplification was linear. That gives a design rule: E epochs are $k = E/q$ steps, so the total RDP is about $E\,q\,\alpha/(2\sigma^2)$, and at a fixed number of epochs smaller batches are cheaper, as long as the orders used stay in the quadratic regime. Abadi et al. (2016), who introduced DP-SGD with the **moments accountant** (RDP in disguise), used a bound of this shape with about twice the constant, valid on a limited range of orders. Their headline example, $q = 0.01$, $\sigma = 4$, $\delta = 10^{-5}$ and 10,000 steps, reports $\varepsilon \approx 1.26$. The exact curve with Mironov's conversion gives 1.2586, at $\alpha = 20$, and the improved conversion 1.0355.

The quadratic term fails at high orders. The last term of the sum, the event that the record was in every one of the α imagined copies, has a tiny weight q^α but the full Gaussian cost, which grows like $e^{\alpha^2/(2\sigma^2)}$, and eventually it wins. That happens near $\alpha \approx 2\sigma^2 \ln(1/q)$, about 9.2 at $q = 0.01$ and $\sigma = 1$. The exact curve is 1.30 times the quadratic term at $\alpha = 8$ and 44.54 times at $\alpha = 10$, then rises towards the unsubsampled line (Figure 31.4). In the tail, as at large ε, sampling saves only about $\ln(1/q)$.

So use Equation 31.4 for estimates, never inside an accountant: beyond the breakdown it is 2246 times too small at $\alpha = 16$ for $\sigma = 1$, and an accountant using it there would under-report ε.

31.6. Other mechanisms

For (ε, δ), Equation 31.1 is one tight formula for every mechanism, because randomized response is the worst case for every ε. For RDP there is no such formula: mechanisms with the same RDP curve can subsample differently. Zhu & Wang (2019) bound the subsampled curve of any mechanism under Poisson sampling with the same binomial expansion, each term bounded by the mechanism's own curve and a factor 3 on the terms from $j = 3$ on, and Wang, Balle & Kasiviswanathan (2019) do the same for fixed-size batches. Applied to the Gaussian at $q = 0.01$, $\sigma = 1$, the general bound is exact at $\alpha = 2$, where there are no terms beyond $j = 2$, but 1.57 times the exact value at $\alpha = 8$, near the best orders for 100 to 1,000 steps. So accountants special-case the Gaussian with the exact sum. For other mechanisms, a numerical accountant must also check

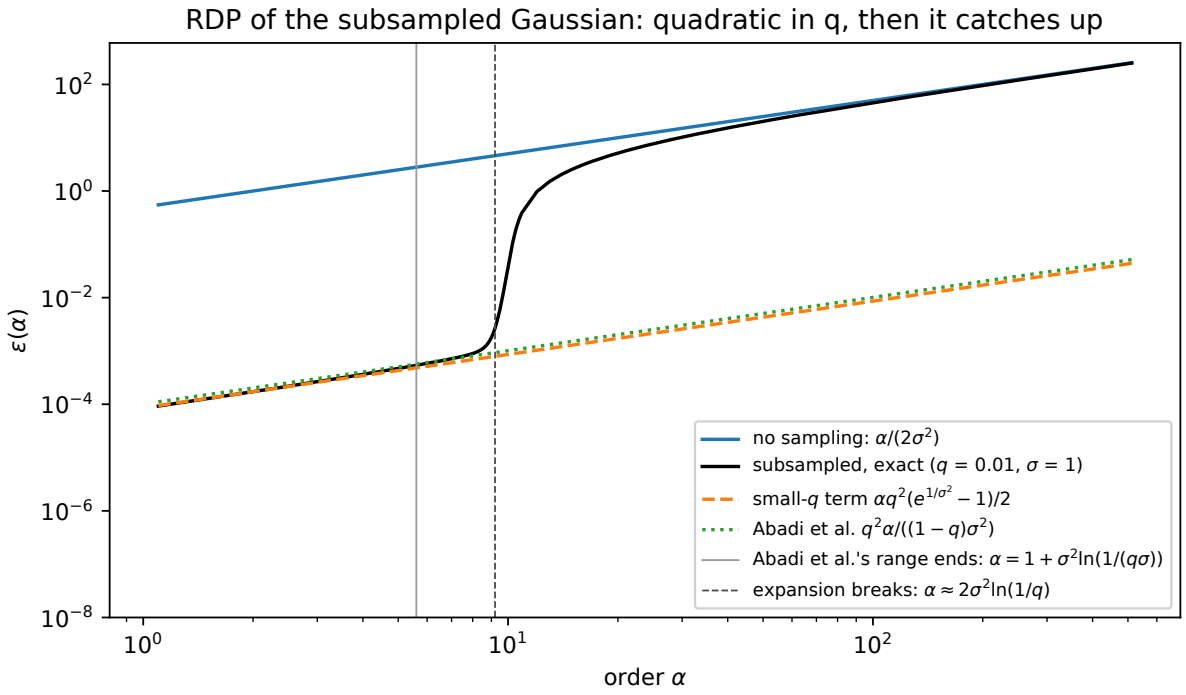


Figure 31.4.: The RDP curve of the Poisson-subsampled Gaussian with $q = 0.01$, $\sigma = 1$ (black), against the unsubsampled Gaussian $\alpha/(2\sigma^2)$, the small-q term and the leading term of Abadi et al.'s bound. The exact curve stays near the quadratic term up to about $\alpha = 8$, then rises towards the unsubsampled line.

its worst-case pair of datasets: Lebeda et al. (2024) show that the worst pair for one step need not be the worst for many.

31.7. Code

An RDP accountant for DP-SGD fits in a few lines. This one uses Equation 31.2 in log space, so large orders do not overflow, and the integer orders 2 to 63 only. With fractional orders, as in Table 31.4, the improved value drops slightly, from 6.719 to 6.71.

```
from math import exp, lgamma, log


def amplify(eps: float, delta: float, q: float) -> tuple[float, float]:
    """(eps, delta) of an add/remove-DP mechanism run on a Poisson sample with rate q."""
    return log(1 + q * (exp(eps) - 1)), q * delta


def subsampled_gaussian_rdp(alpha: int, q: float, sigma: float) -> float:
    """RDP at integer order alpha of one Poisson-subsampled Gaussian step (sensitivity 1)."""
    logs = [
        lgamma(alpha + 1) - lgamma(j + 1) - lgamma(alpha - j + 1)
        + (alpha - j) * log(1 - q) + j * log(q) + (j * j - j) / (2 * sigma**2)
        for j in range(alpha + 1)
    ]
    top = max(logs)
    log_a = top + log(sum(exp(t - top) for t in logs))  # log-sum-exp avoids overflow
    return log_a / (alpha - 1)


def dpsgd_eps(q: float, sigma: float, steps: int, delta: float, improved: bool = True) -> tu
    """Compose the RDP curve over the steps, convert to (eps, delta) at the best integer ord
    best = (float("inf"), 0)
    for alpha in range(2, 64):
        rdp = steps * subsampled_gaussian_rdp(alpha, q, sigma)
        if improved:
            eps = rdp + log((alpha - 1) / alpha) - (log(delta) + log(alpha)) / (alpha - 1)
        else:
            eps = rdp + log(1 / delta) / (alpha - 1)
        best = min(best, (eps, alpha))
    return best


print(f"{amplify(1.0, 0.0, 0.01)[0]:.4f}")  # 0.0170
print(f"{subsampled_gaussian_rdp(2, 0.01, 1.0):.4e}")  # 1.7181e-04
print("%.3f %d" % dpsgd_eps(0.01, 1.0, 10_000, 1e-5))  # 6.719 4
print(f"{dpsgd_eps(0.01, 1.0, 10_000, 1e-5, improved=False)[0]:.3f}")  # 7.469
```

```
print("%.4f %d" % dpsgd_eps(0.01, 4.0, 10_000, 1e-5, improved=False))  # 1.2586 20
print("%.4f %d" % dpsgd_eps(0.01, 4.0, 10_000, 1e-5))  # 1.0355 17
```

The last two lines reproduce Abadi et al.'s example. To account for fixed-size batches, call `dpsgd_eps` with half the noise multiplier. In production, use a maintained library and check it against a second one (Choosing an accountant in the full edition). The full edition's script, subsampling.py, computes every number in this chapter, including the fractional orders and the sampler comparisons:

```
uv run python code/part04-privacy-accounting/subsampling.py
```

31.8. Cheat sheet

Table 31.5.: The results of this chapter.

Result	Formula	Conditions
One step, (ε, δ)	$\varepsilon' = \ln(1 + q(e^{\varepsilon} - 1))$, $\delta' = q\delta$	Poisson with add/remove, or fixed size $q = m/n$ with replace-one; tight
Small and large ε	$\varepsilon' \approx q\varepsilon$; $\varepsilon' \approx \varepsilon - \ln(1/q)$	Switch near $\varepsilon \approx \ln(1/q)$
Fixed-size batches, clipped Gaussian sum, add/remove	Account Poisson with rate m/n and noise multiplier $\sigma/2$	Lebeda et al. 2024; under replace-one, the mixture pair of Table 31.2, also with $\sigma/2$
Shuffle and partition	No simple amplification; the Poisson number can be far too small	Lower bounds by attack
Subsampled Gaussian RDP	$\varepsilon(\alpha) = \ln A_\alpha/(\alpha - 1)$, $A_\alpha = \sum_j \binom{\alpha}{j}(1-q)^{\alpha-j} q^j e^{(j^2-j)/(2\sigma^2)}$	Poisson, add/remove; exact
DP-SGD	$\min_\alpha [k\,\varepsilon(\alpha) + \text{conversion}]$	Upper bound; 6.71 for the running example
Small q	$\varepsilon(\alpha) \approx \alpha q^2 (e^{1/\sigma^2} - 1)/2$	Only for $\alpha \lesssim 2\sigma^2 \ln(1/q)$

Rules of thumb

- Sampling at rate q multiplies a small ε by q, but takes at most $\ln(1/q)$ off a large one.
- Never compose per-step (ε, δ) for DP-SGD. Compose RDP curves, or privacy loss distributions, and convert once.
- Account for the sampler you run. Fixed-size batches under add/remove mean half the noise multiplier in a Poisson accountant; shuffled batches have no Poisson guarantee.
- The sampling scheme can move ε by a factor of seven; the conversion, by about a tenth.

31.9. Further reading

- Balle, Barthe & Gaboardi, "Privacy Amplification by Subsampling: Tight Analyses via Couplings and Divergences" (NeurIPS 2018): tight (ε, δ) amplification for each sampling scheme and neighbouring relation.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (2019): the subsampled Gaussian's RDP curve and its stable computation, the basis of most RDP accountants.
- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled Mechanisms under Composition" (IEEE SaTML 2025): fixed-size against Poisson sampling, and worst-case pairs that fail under composition.
- Chua, Ghazi, Kamath, Kumar, Manurangsi, Sinha & Zhang, "How Private are DP-SGD Implementations?" (ICML 2024): why shuffle-and-partition batching can be much less private than the Poisson accountant reports.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

32. Numerical accounting with privacy loss distributions

32.1. Why this matters

You trained a model with differential privacy (DP): 10,000 steps of stochastic gradient descent (SGD), each on a Poisson sample of the data with rate $q = 0.01$, with clipped gradients and Gaussian noise of multiplier $\sigma = 1$. This is DP-SGD, the running example of this part, and you need the smallest ε for which the whole run is $(\varepsilon, 10^{-5})$-DP. Rényi differential privacy (RDP), from Rényi DP in the full edition, gives an upper bound that loses an unknown amount in its conversion. The Gaussian central limit theorem (CLT) of f-DP and Gaussian DP in the full edition gives an approximation that may err in either direction.

This chapter computes the answer itself. $\delta(\varepsilon)$ depends on a mechanism only through the distribution of its privacy loss, a distribution on the real line, and composing steps adds losses, so a whole run's distribution is a convolution that a computer can evaluate on a grid. Rounding in a chosen direction gives the grid's error a known sign: you get a certified upper bound and a lower bound whose distance you know in advance.

32.2. The privacy loss distribution

Let P be the distribution of the output on your dataset D and Q on a neighbour D'. The **privacy loss** of an output y is $L(y) = \ln \frac{P(y)}{Q(y)}$ (Composition). Draw y from P and the loss is a random variable; its distribution is the **privacy loss distribution (PLD)**.

Why should this one-dimensional distribution carry everything? The definition lets each output's probability under P be up to e^{ε} times its probability under Q, and δ pays only for the excess. An output with loss ℓ has $P(y) = e^{\ell} Q(y)$, so its excess is a fraction $1 - e^{\varepsilon - \ell}$ of its probability when $\ell > \varepsilon$, and nothing otherwise. That fraction depends on the output only through its loss, so δ is an average over the PLD:

$$\delta(\varepsilon) = \mathbb{E}\big[(1 - e^{\varepsilon - L})_+\big] = P[L > \varepsilon] - e^{\varepsilon}\, Q[L > \varepsilon], \tag{32.1}$$

where $(x)_+ = \max\{x, 0\}$. The second form reads it as a gap: the chance under P that the loss exceeds ε, minus e^{ε} times that chance under Q. An output impossible under D' has infinite loss and counts fully. So the output space never enters: a run with millions of parameters and a one-dimensional mechanism with the same PLD have the same privacy curve.

The weight is 0 up to $\ell = \varepsilon$ and rises slowly towards 1, so δ is the PLD's upper tail with losses just above ε discounted (Figure 32.1). That is why the plain tail $P[L > \varepsilon]$ is a loose bound: it counts the barely-over outputs in full.

For the Gaussian mechanism, with $\mu = \Delta_2 f/\sigma$ the sensitivity in noise standard deviations, the two densities have the same shape, so their log-ratio is linear in the output and the loss is itself Gaussian, $\mathcal{N}(\mu^2/2, \mu^2)$. Its two tails give the closed form of The Laplace and Gaussian mechanisms, with Φ the standard normal distribution function:

$$\delta(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{32.2}$$

With sensitivity 1 and $\sigma = 1$, the PLD is $\mathcal{N}(0.5, 1)$; at $\varepsilon = 1$ the formula gives $\Phi(-0.5) - e\,\Phi(-1.5) = 0.3085 - 0.1816 = 0.1269$, and solving $\delta(\varepsilon) = 10^{-5}$ gives $\varepsilon = 4.3772$.

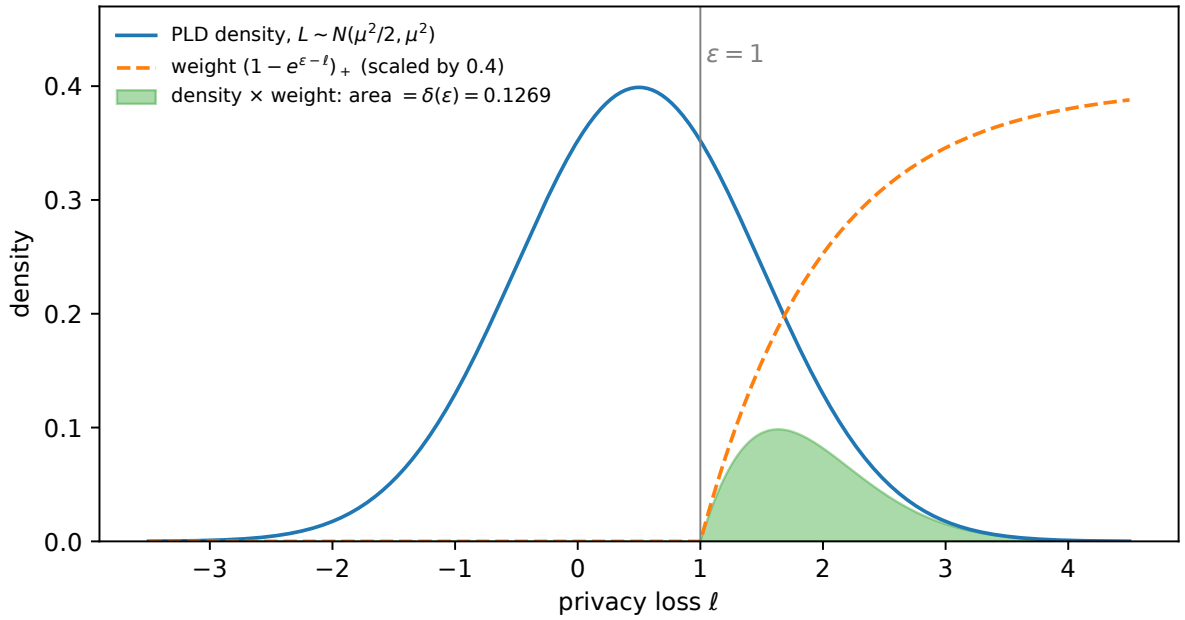


Figure 32.1.: The Gaussian PLD for $\mu = 1$ (blue), the weight $(1 - e^{\varepsilon-\ell})_+$ at $\varepsilon = 1$ (orange, scaled by 0.4 to fit), and their product (green). The green area is $\delta(1) = 0.1269$: only losses above ε count, and those just above it count little.

32.3. Composition is convolution

For independent steps the probability of the tuple of outputs is a product, its log is a sum, and so the privacy loss of the run is the sum of the step losses. The distribution of a sum of independent variables is the convolution of their distributions, so the PLD of k steps is the k-fold convolution of the per-step PLDs.

This step is exact, which sets it apart from the theorems of Composition: they see each step only through its $(\varepsilon_i, \delta_i)$ and must cover the worst step with those values, while convolution uses each step's actual shape. All the error in a PLD accountant comes from computing it. Gaussian steps give a check, since means and variances add: k steps with shift μ behave like one Gaussian mechanism with $\mu_k = \sqrt{k}\,\mu$. Ten steps with $\sigma = 1$ are $(17.8566, 10^{-5})$-DP and a hundred $(91.8173, 10^{-5})$-DP, values any accountant must reproduce at $q = 1$. The same picture

explains why the CLT is plausible, since long sums look Gaussian; the PLD accountant computes the sum instead.

DP-SGD steps are adaptive: each gradient depends on the model, so the losses are not independent. The repair is a **dominating pair**, a fixed pair at least as easy to tell apart as the step, whatever came before; convolving the dominating pairs' PLDs bounds the adaptive run (Zhu, Dong & Wang 2022). The bound is exactly as tight as the pair. For the Gaussian mechanism, $\mathcal{N}(0, \sigma^2)$ against $\mathcal{N}(1, \sigma^2)$ is produced by real datasets, so nothing is lost; for a subsampled mechanism the pair depends on how batches are drawn and must match your training code.

32.4. Pessimistic and optimistic discretization

A computer convolves vectors, so the loss goes on a grid of width h. The rounding can still give a guarantee, because the weight in Equation 32.1 never decreases as the loss grows: move every loss up and δ can only rise, move every loss down and it can only fall.

The **pessimistic** PLD rounds every loss up to the next grid point and sends any truncated upper tail to $+\infty$. The **optimistic** PLD rounds every loss down and drops the truncated lower tail. In any run, each step's rounded-up loss is at least its true loss, which is at least its rounded-down loss, and adding over k steps keeps that order for the totals. So at every ε

$$\delta_{\text{optimistic}}(\varepsilon) \;\le\; \delta(\varepsilon) \;\le\; \delta_{\text{pessimistic}}(\varepsilon). \tag{32.3}$$

The same argument gives the width: each step's two roundings are at most h apart, so the totals, and the two values of ε at your δ, are at most kh apart. For a continuous PLD the gap is almost exactly kh: the error grows linearly in k.

The two sides are not equally strong. Raising losses keeps the pessimistic distribution the PLD of a real pair that dominates the original, so the upper bound holds for adaptive composition too. Lowering them can produce a distribution no pair has, so the optimistic value bounds the mechanism from below only when real datasets produce the pair, as for the Gaussian and the subsampled Gaussian.

Randomized response with $\varepsilon_0 = \ln 3$, composed three times on the coarse grid $h = 0.5$, has exact $\delta(1) = 0.4190$ between the pessimistic 0.6758 and the optimistic 0.3648 (Figure 32.2). For Gaussian steps with $\mu = 1$ on $h = 10^{-3}$ (Table 32.1), the width is exactly kh and the exact value sits in the middle, because a loss spread evenly within a cell moves up by about $h/2$ when rounded up and down by as much when rounded down. The midpoint, like rounding to the nearest point, is a good estimate, but only the two sides are guarantees.

Table 32.1.: Composed Gaussian steps with $\mu = 1$ on the grid $h = 10^{-3}$: ε at $\delta = 10^{-5}$ from the pessimistic and optimistic PLDs against the exact value.

k	Exact ε	Pessimistic	Optimistic
10	17.8566	17.8616	17.8516
100	91.8173	91.8673	91.7673

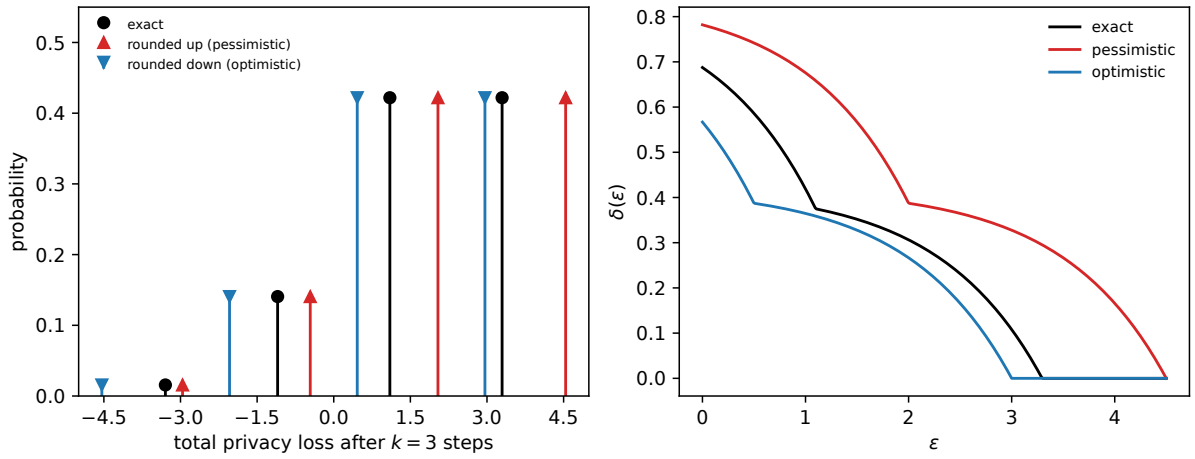


Figure 32.2.: Left: the PLD of three-fold randomized response with $\varepsilon_0 = \ln 3$ (black), rounded up (red) and rounded down (blue) on the grid $h = 0.5$. Right: the three $\delta(\varepsilon)$ curves; the pessimistic one lies above the exact one and the optimistic one below it at every ε.

32.5. Composition by FFT

On a common grid, adding two losses adds their grid indices, so composing two discretized PLDs is a discrete convolution of their mass vectors. A fast Fourier transform (FFT) turns it into a pointwise product of transforms, at a cost of order $N \log N$ for length N, and repeated squaring composes k steps in at most $2 \log_2 k$ convolutions.

The FFT has one trap, which follows from how it works: it attaches to each index a power of an N-th root of unity, so indices add modulo N, and mass that falls off one end of the window reappears at the other. Ten Gaussian steps with $\mu = 1$ on $h = 10^{-3}$ give a pessimistic $\delta(1) = 0.81891018$, against the exact 0.81851782. On a circular window from loss 0 to 10, the same computation gives 0.83478562, wrong in the second decimal: negative losses, which should count nothing, wrap to the top of the window where they count almost fully. Width 40 gets it right. The error from wraparound can go either way, so a wrapped result is no bound at all; pad every convolution to the sum of its inputs' lengths. Truncating tails at a tiny mass such as 10^{-15} is safe only if the pessimistic side moves the cut mass up, to $+\infty$.

Each tenfold refinement of h makes the vectors ten times longer. The **privacy loss random variable (PRV) accountant** (Gopi, Lee & Wutschitz 2021) rounds to the nearest point and shifts the grid to preserve the mean loss, so the per-step errors have mean zero and grow like $\sqrt{k}\,h$ instead of kh; a grid of $h \approx 1/\sqrt{k}$ then suffices, with explicit error bounds. **Connect the dots** (Doroshenko et al. 2022) builds the best pessimistic PLD on a given grid.

32.6. The subsampled Gaussian

A DP-SGD step with Poisson sampling includes each record with probability q. Without the record in question, the noisy sum is a Gaussian centred at 0; with it, the record is sampled with probability q and shifts the centre by 1, so the output is a mixture (Amplification by subsampling

in the full edition). Under add/remove neighbours the pair's two orders differ. With D' the dataset that holds the record, the **remove** direction puts it first, $P = M(D')$ and $Q = M(D)$:

$$P = (1-q)\,\mathcal{N}(0,\sigma^2) + q\,\mathcal{N}(1,\sigma^2), \qquad Q = \mathcal{N}(0,\sigma^2), \tag{32.4}$$

and the **add** direction swaps them (Zhu, Dong & Wang 2022). Dividing the mixture by the base gives the remove loss, $L_{\text{rem}}(y) = \ln\left((1-q) + q\,e^{(2y-1)/(2\sigma^2)}\right)$, and the add loss is its negative.

The formula explains the asymmetry (Figure 32.3). For an ordinary output the exponential is small, so the remove loss never falls below $\ln(1-q)$; but a large output is far likelier from the shifted component, and there the loss grows without limit: a rare large gradient betrays that the record was used. The add loss is the mirror image, bounded above by $-\ln(1-q) = 0.0101$, and since δ counts only losses above ε, one add step costs nothing beyond that. For $q = 0.01$ and $\sigma = 1$, one step is $(0.1995, 10^{-5})$-DP in the remove direction and $(0.0091, 10^{-5})$-DP in the add direction.

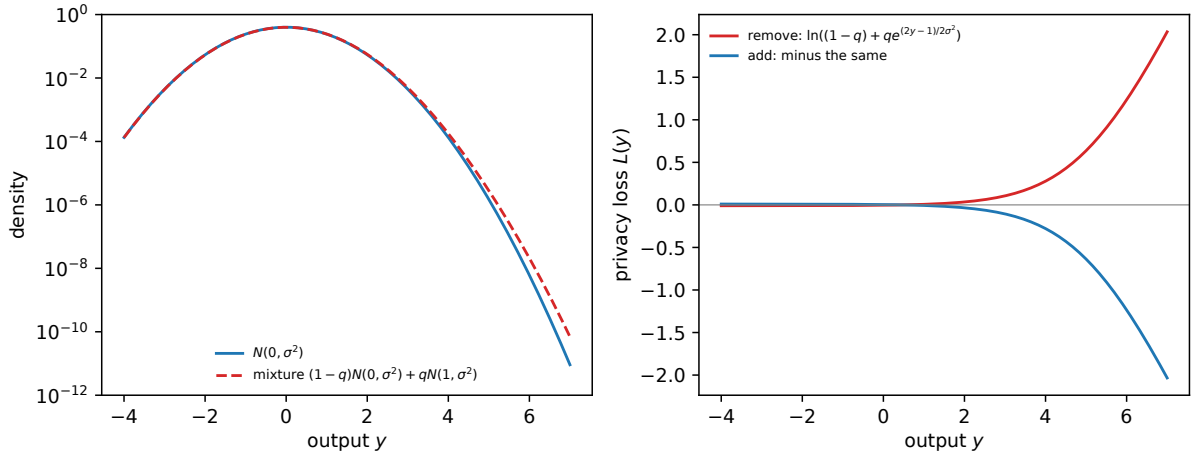


Figure 32.3.: Left: the densities of $\mathcal{N}(0,\sigma^2)$ and of the mixture $(1-q)\mathcal{N}(0,\sigma^2) + q\mathcal{N}(1,\sigma^2)$ for $q = 0.01$, $\sigma = 1$, on a log scale; they differ only in the far right tail. Right: the privacy loss as a function of the output for the remove pair (red) and the add pair (blue).

Composition narrows the gap, since many small add losses add up: after 10,000 steps the add direction's upper bound is 5.6615. The remove direction stays larger here, but that is an observation, not a theorem, so compose both and report the larger ε; an accountant using only the remove pair covers half of the add/remove relation. The pair also assumes Poisson sampling: if your loader shuffles and partitions instead, the mixture is the wrong model and these numbers do not apply.

32.7. The running example: DP-SGD

On the grid $h = 10^{-5}$, Table 32.2 and Figure 32.4 compare the PLD bounds (the larger direction) with the RDP accountant, using the improved conversion, and the Gaussian differential privacy (GDP) central-limit approximation.

Table 32.2.: DP-SGD with $q = 0.01$, $\sigma = 1$: ε at $\delta = 10^{-5}$ after k steps, from the pessimistic and optimistic PLDs on the grid $h = 10^{-5}$ (width kh), the RDP accountant, and the GDP central-limit approximation.

k	PLD upper	PLD lower	Width	RDP	GDP CLT
1	0.1995	0.1994	1.0e-05	0.9555	0.0368
10	0.3800	0.3799	1.0e-04	1.0353	0.1304
100	0.7185	0.7175	1.0e-03	1.2141	0.4575
1000	1.8332	1.8232	1.0e-02	2.1014	1.6177
10000	6.2377	6.1377	1.0e-01	6.7127	6.0071

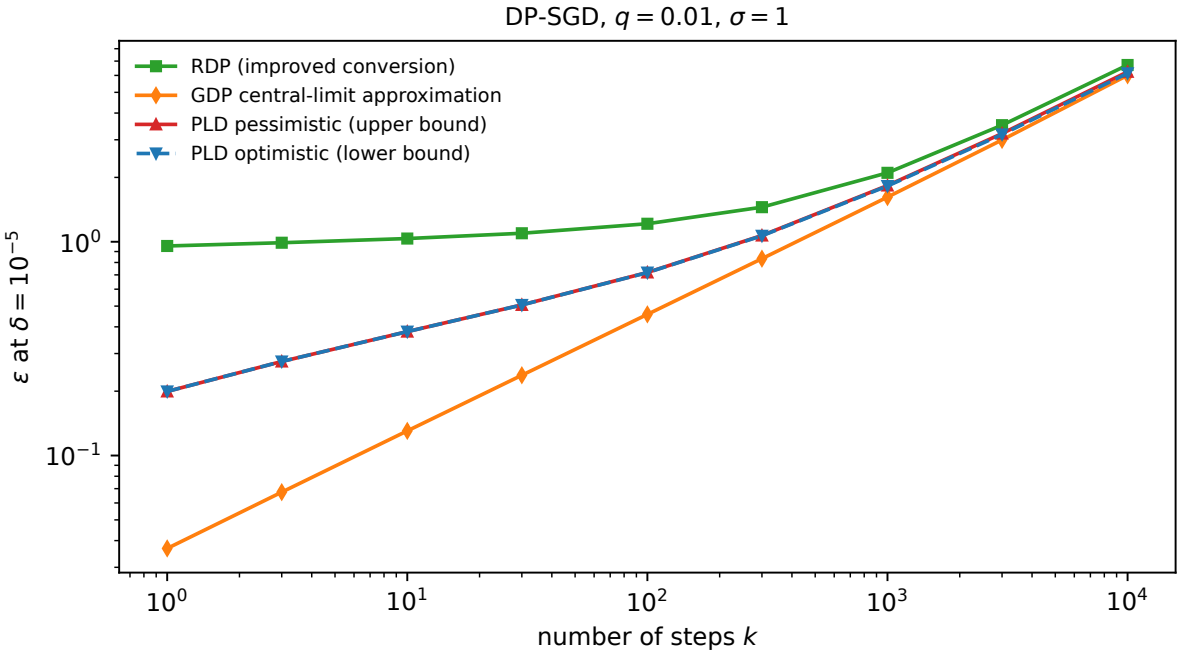


Figure 32.4.: ε at $\delta = 10^{-5}$ against the number of steps k for DP-SGD with $q = 0.01$, $\sigma = 1$: the RDP accountant (green), the GDP central-limit approximation (orange), and the pessimistic (red) and optimistic (blue) PLD bounds, which overlap at this scale. RDP lies above them and the CLT below them.

After 10,000 steps the true ε lies in $[6.1377, 6.2377]$; report 6.2377. RDP's 6.7127 is valid but over-states by 0.4750, and the single step shows why: 0.9555 against 0.1995. RDP keeps one moment of the loss per order α, and turning a moment into a tail bound costs about $\ln(1/\delta)/(\alpha-1)$ whatever the true curve looks like.

The CLT errs the other way, which is worse: it falls below the PLD lower bound at every k, so quoting its 1.62 for 1000 steps claims a guarantee the run does not have. The reason is the shape of the subsampled PLD (Figure 32.5): convolution makes its bulk bell-shaped, but its right tail stays heavy, and $\delta = 10^{-5}$ reads exactly that tail, where the Gaussian approximation is worst.

A PLD accountant is only as good as its grid. At $k = 1000$ on $h = 10^{-3}$ the width is $kh = 1$, and

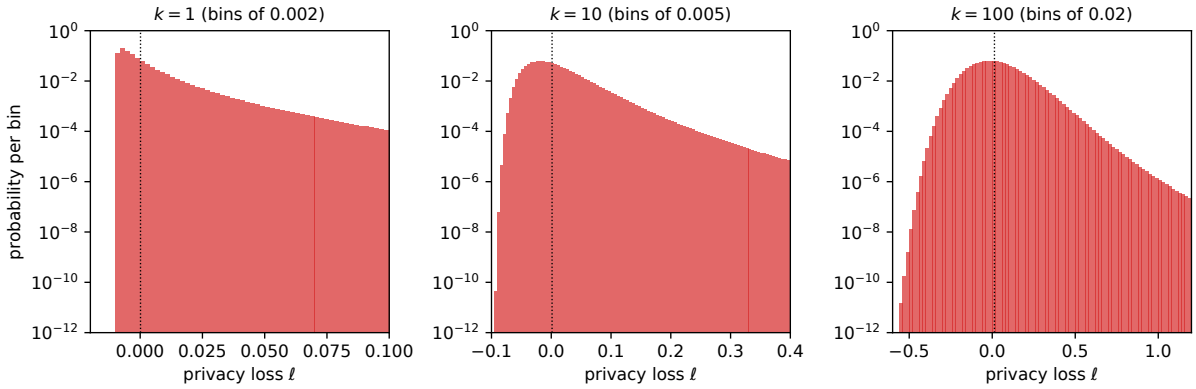


Figure 32.5.: The pessimistic remove-direction PLD of DP-SGD with $q = 0.01$, $\sigma = 1$ after 1, 10 and 100 steps (log scale; the dotted line is the mean). One step piles its mass just above $\ln(1-q)$ with a long right tail; convolution moves the bulk towards a bell shape, but the right tail stays heavier than the left.

the upper bound, 2.3279, is worse than RDP's 2.1014. Choose h for your largest k, and check the width before trusting the upper bound.

32.8. Code

The first part solves Equation 32.2 for ε: the reference values any accountant should reproduce with $q = 1$. The second is a toy PLD accountant that reproduces the sandwich of Figure 32.2; the grid $h = \ln 3$ contains every loss, so it gives the exact values.

```
from math import ceil, erfc, exp, floor, log, sqrt


def normal_cdf(x: float) -> float:
    return 0.5 * erfc(-x / sqrt(2))


def gaussian_delta(eps: float, mu: float) -> float:
    """Exact delta(eps) of the Gaussian PLD N(mu^2/2, mu^2); k steps of multiplier sigma hav
    return normal_cdf(mu / 2 - eps / mu) - exp(eps) * normal_cdf(-mu / 2 - eps / mu)


def gaussian_epsilon(delta: float, mu: float) -> float:
    lo, hi = 0.0, 1000.0  # delta(eps) falls as eps grows: bisect for delta(eps) = delta
    for _ in range(200):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if gaussian_delta(mid, mu) > delta else (lo, mid)
    return hi
```

```
PLD = dict[int, float]  # grid index i -> probability of the loss i * h


def discretize(losses: dict[float, float], h: float, up: bool) -> PLD:
    pld: PLD = {}
    for loss, p in losses.items():
        i = ceil(loss / h) if up else floor(loss / h)
        pld[i] = pld.get(i, 0.0) + p
    return pld


def compose(a: PLD, b: PLD) -> PLD:
    """The PLD of the sum of two independent losses: a convolution."""
    out: PLD = {}
    for i, p in a.items():
        for j, r in b.items():
            out[i + j] = out.get(i + j, 0.0) + p * r
    return out


def pld_delta(pld: PLD, h: float, eps: float) -> float:
    return sum(p * (1 - exp(eps - i * h)) for i, p in pld.items() if i * h > eps)


print(f"{gaussian_epsilon(1e-5, 1.0):.4f}")  # 4.3772
print(f"{gaussian_epsilon(1e-5, sqrt(10)):.4f}")  # 17.8566
print(f"{gaussian_epsilon(1e-5, 10.0):.4f}")  # 91.8173

def three_fold_rr(h: float, up: bool) -> str:
    """delta(0), delta(1), delta(2) of randomized response with eps0 = ln 3, composed three
    step = discretize({log(3): 0.75, -log(3): 0.25}, h, up)
    three = compose(compose(step, step), step)
    return " ".join(f"{pld_delta(three, h, eps):.4f}" for eps in (0, 1, 2))


print(three_fold_rr(log(3), up=True))  # 0.6875 0.4190 0.3064
print(three_fold_rr(0.5, up=True))  # 0.7820 0.6758 0.3872
print(three_fold_rr(0.5, up=False))  # 0.5669 0.3648 0.2667
```

The last three lines are the exact, pessimistic and optimistic $\delta(0)$, $\delta(1)$, $\delta(2)$. For training runs use a production accountant. The full edition's script, numerical_accounting.py, implements the FFT accountant for the subsampled Gaussian in NumPy and prints Table 32.2:

```
uv run python code/part04-privacy-accounting/numerical_accounting.py
```

32.9. Cheat sheet

Table 32.3.: The results of this chapter.

Result	Formula	Notes
δ from the PLD	$\delta(\varepsilon) = \mathbb{E}[(1 - e^{\varepsilon - L})_+] = P[L > \varepsilon] - e^{\varepsilon} Q[L > \varepsilon]$	Infinite losses count fully
Gaussian PLD	$\mathcal{N}(\mu^2/2, \mu^2)$, δ from Equation 32.2	k steps: $\mu \to \sqrt{k}\,\mu$
Composition	Convolve the per-step PLDs	Adaptive steps: convolve dominating pairs
Sandwich	Round up (pessimistic) and down (optimistic)	Width in ε at most kh
FFT	Pad each convolution to the sum of the input lengths	A short window wraps mass around silently
Subsampled Gaussian	Remove: mixture vs $\mathcal{N}(0, \sigma^2)$; add: swapped	Take the larger ε of the two
Running example, $k = 10{,}000$	PLD $[6.1377, 6.2377]$; RDP 6.7127; CLT 6.0071	$q = 0.01$, $\sigma = 1$, $\delta = 10^{-5}$

Rules of thumb

- Report the pessimistic ε, with the optimistic one next to it as the error bar.
- Choose h so that kh is an error you can accept at your largest k, and check the width rather than trusting the grid.
- Check your accountant against the Gaussian reference values (4.3772, 17.8566, 91.8173 at $\delta = 10^{-5}$) before you trust it on DP-SGD.

32.10. Further reading

- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021): the PRV accountant, with explicit error bounds and a running time of order $\sqrt{k}$; read it when you need a fast accountant with certified error bars.
- Doroshenko, Ghazi, Kamath, Kumar & Manurangsi, "Connect the Dots: Tighter Discrete Approximations of Privacy Loss Distributions" (PoPETs 2022): the cleanest account of why the pessimistic bound is valid, and the best discretization on a given grid.
- Zhu, Dong & Wang, "Optimal Accounting of Differential Privacy via Characteristic Function" (AISTATS 2022): dominating pairs, adaptive composition and the two directions of Poisson subsampling.
- Google, `dp_accounting` library documentation (the `pld` module): a production PLD accountant with pessimistic and optimistic estimates; use it to check the numbers here.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

33. Amplification by shuffling and by iteration

33.1. Why this matters

Amplification by subsampling in the full edition showed that uncertainty is worth privacy: if an adversary cannot tell whether a step used your record, the step leaks far less than its noise alone suggests. Two other kinds matter in practice. In the shuffle model, n users randomize their own data and a trusted shuffler strips identities and mixes the order, so the analyst does not know **which report is yours**. Each report alone satisfies local differential privacy (DP) at a level ε_0, usually large to keep reports useful, yet the bag of reports is central DP with an ε that falls like $1/\sqrt{n}$. And when noisy gradient descent (GD) on a convex loss releases only its final model, the adversary does not see **what happened inside training**: later steps add fresh noise and pull the parameters together, hiding early records better than composition says.

One idea gives every result. Show that what the adversary sees can be generated from a much simpler pair of distributions by a procedure that does not depend on your record. Anything the adversary learns from the real output, it could then have learned from the simple pair, so that pair's privacy carries over. For shuffling the simple pair is a pair of counts; for iteration, a pair of Gaussians. Throughout, neighbouring datasets differ by **replacing** one record. The shuffling results are about anonymous, locally randomized reports, not the shuffle-and-partition batching used to train models in the central model, where no comparable theorem applies (Amplification by subsampling in the full edition).

33.2. Shuffled randomized response, exactly

Start with the simplest randomizer, whose exact answer is the yardstick for every general bound. Each user reports a bit with binary randomized response: the true bit with probability $\theta = e^{\varepsilon_0}/(1+e^{\varepsilon_0})$, the flipped bit otherwise. Once the order is gone, a bag of bits can tell the analyst only how many of them are 1. Your report moves that count by at most one, while the other users' random flips spread it over a range of about $\sqrt{n\theta(1-\theta)}$, and that spread is what hides you.

Let your bit be 0 in D and 1 in D', and let the others all hold 0, so their number of 1-reports is binomial, $\mathrm{Bin}(n-1, 1-\theta)$, with probability mass function b. A count of y arises either when your report is 0 and the others sent y ones, or when your report is 1 and they sent $y-1$, so

$$P(y) = \theta\, b(y) + (1-\theta)\, b(y-1), \qquad Q(y) = (1-\theta)\, b(y) + \theta\, b(y-1). \tag{33.1}$$

The exact $\delta(\varepsilon)$ is the larger of the two hockey-stick divergences between P and Q, one per order, which takes milliseconds to compute (Section 33.6). Strictly, this is exact for the case where all

the others hold 0 and a lower bound on the worst case; the other configurations the full edition scans come within 10^{-4} of it, so the book treats it as exact.

Why $1/\sqrt{n}$? The count is a sum of many independent bits, so it is nearly normal. Your report shifts its mean by $2\theta - 1$, and its standard deviation is $\sqrt{n\theta(1-\theta)}$ under both datasets. A pair of normals is a Gaussian mechanism whose μ is the shift divided by the standard deviation, and simplifying gives

$$\mu \approx \frac{2\sinh(\varepsilon_0/2)}{\sqrt{n}}. \tag{33.2}$$

More users mean more noise to hide one bit in, hence $1/\sqrt{n}$; a larger ε_0 means more faithful reports and a less noisy count, hence the growth like $e^{\varepsilon_0/2}$. A company that asks 10,000 users whether they used a feature, with $\varepsilon_0 = 2$, gets reports that are each the truth with probability 0.8808, a weak local guarantee; the shuffled bag is $(0.0870, 10^{-6})$-DP.

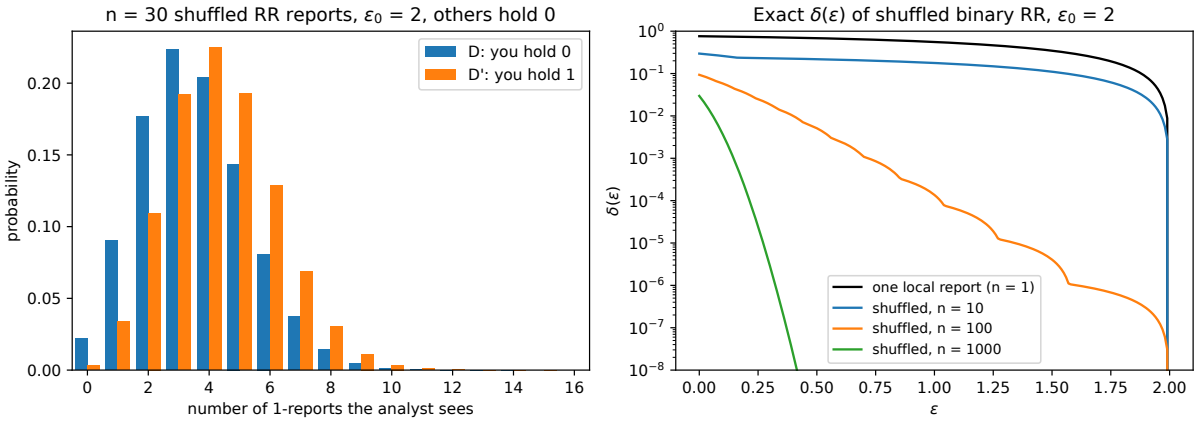


Figure 33.1.: Left: the number of 1-reports among $n = 30$ shuffled randomized-response reports with $\varepsilon_0 = 2$, when you hold 0 (blue) and 1 (orange); your report moves the count by at most one. Right: the exact $\delta(\varepsilon)$ for one unshuffled report and for $n = 10, 100, 1000$ shuffled reports. Every curve reaches 0 only at $\varepsilon = \varepsilon_0$, but for large n it is negligible far earlier.

Shuffling never improves pure DP, as the count of zero shows. If nobody reports a 1, you most likely hold 0, and the likelihood ratio of that outcome is e^{ε_0} whatever n is. What shuffling changes is how often it happens: rarely, for large n. So amplification moves the bad outcomes into δ (Figure 33.1), and a shuffled guarantee always needs $\delta > 0$. The normal approximation fails for the same reason it works: it needs many 1-reports to hide among. At $n = 1000$ and $\varepsilon_0 = 4$ the others send about 18 on average, the count is far from normal, and the approximation gives 0.9667 against the exact 1.6910.

33.3. The clone bound for any local randomizer

Users often send histograms or noisy vectors, so you need a bound for every ε_0-DP local randomizer. The argument of Feldman, McMillan & Talwar (2021) finds a hidden randomized response inside any of them. Being ε_0-DP means no output is more than e^{ε_0} times likelier for one input than for another. Read the other way, any user's report keeps at least a fraction $e^{-\varepsilon_0}$ of each output's

probability under your D-version, and likewise under your D'-version. So every other user's report can be generated by a coin: with probability $e^{-\varepsilon_0}$ it is a **clone**, a draw distributed exactly like your report could be, half the time like your D-version and half like your D'-version; otherwise it is a draw that depends only on that user's own data.

After shuffling, nobody can tell a clone from your report, so the adversary can do no better than count how many reports look like each version of you. That count is the simple pair, and its exact $\delta(\varepsilon)$ is the **numeric clone bound**, valid for every ε_0-DP randomizer, even adaptively chosen ones. The clones supply the noise: about $ne^{-\varepsilon_0}$ of them, whose split between the two versions has standard deviation $\sqrt{ne^{-\varepsilon_0}}/2$. That again gives $1/\sqrt{n}$ with a constant growing like $e^{\varepsilon_0/2}$. The analytic form (their Theorem 3.1) bounds the count pair with Chernoff bounds:

$$\varepsilon \le \ln\left(1+\frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1}\left(\frac{8\sqrt{e^{\varepsilon_0}\ln(4/\delta)}}{\sqrt{n}}+\frac{8e^{\varepsilon_0}}{n}\right)\right), \qquad \text{valid when } \varepsilon_0 \le \ln\frac{n}{16\ln(2/\delta)}. \tag{33.3}$$

The condition says there must be enough clones to hide among: it is $ne^{-\varepsilon_0} \ge 16\ln(2/\delta)$. At $\delta = 10^{-6}$ it caps ε_0 at 1.460 for $n = 1000$ and 6.066 for $n = 10^5$.

Table 33.1.: Central ε at $\delta = 10^{-6}$ after shuffling n reports that are each ε_0-DP. "n/a" marks where the condition of Equation 33.3 fails.

n	ε_0	Exact, binary randomized response	Normal approx., Equation 33.2	Clones, numeric	Clones, closed form
1,000	1	0.1266	0.1212	0.1824	0.5662
1,000	2	0.3233	0.2890	0.5455	n/a
1,000	4	1.6910	0.9667	3.9900	n/a
10,000	1	0.0357	0.0352	0.0530	0.2140
10,000	2	0.0870	0.0844	0.1550	0.5009
10,000	4	0.3146	0.2816	0.6009	n/a
100,000	1	0.0101	0.0101	0.0153	0.0726
100,000	2	0.0247	0.0244	0.0450	0.1862
100,000	4	0.0847	0.0822	0.1698	0.5346

Table 33.1 shows the cost of generality. The numeric clone bound is 1.44 to 2.36 times the exact value for randomized response, for a clear reason: in randomized response every other report is a bit that could be yours, while the clone argument counts only the $e^{-\varepsilon_0}$ fraction guaranteed to look like yours. No general bound can beat the exact column, since randomized response is one of the randomizers it must cover. The closed form loses another factor of 3.10 to 4.75 to its Chernoff constants, so use it for scaling and the numeric bound to deploy.

The choice can decide whether a deployment is feasible. To collect a bit from 10^5 users with central $\varepsilon \le 0.2$ at $\delta = 10^{-6}$, the closed form allows $\varepsilon_0 = 2$ (0.1862), the numeric clone bound allows $\varepsilon_0 = 4$ (0.1698), and the exact value at $\varepsilon_0 = 4$ is 0.0847. At $\varepsilon_0 = 4$ the noise adds only $0.0190/n$ to the variance of the estimated proportion, against up to $0.25/n$ from sampling alone. All of this rests on a uniformly random order the analyst cannot see: if the order leaks, the analyst knows which report is yours and the guarantee falls back to ε_0.

33.4. Amplification by iteration

Projected noisy stochastic gradient descent (SGD) makes one pass over the data, using record x_t at step t. If every iterate is released, x_t is protected only by the noise of its own step. If only the final model is released, the noise of every later step also covers it, and Feldman, Mironov, Talwar & Thakurta (2018) turned this into a guarantee.

The step that uses your record pushes the two runs apart. Each later noisy step can "pay off" part of that gap, and the price of paying an amount a is quadratic in a, like the Rényi divergence of two Gaussians shifted by a. Paying a gap s in m equal instalments costs $m(s/m)^2 = s^2/m$, so spreading the payment over every remaining step is cheapest. This only works if the later steps never push points apart, if they are **contractions**; a gradient step on a convex, β-smooth loss with step size $\eta \le 2/\beta$, followed by projection onto a convex set, is one. For convex, Lipschitz, smooth losses and noise multiplier $\tilde{\sigma}$ (noise per unit of the gradient's replace-one sensitivity), releasing only the last iterate gives the record used at step t of n the Rényi differential privacy (RDP) curve

$$\varepsilon_t(\alpha) = \frac{\alpha}{2\tilde{\sigma}^2} \cdot \frac{1}{n-t+1} : \tag{33.4}$$

one Gaussian step's curve divided by the number of noisy steps from t to the end.

Take one pass over $n = 10{,}000$ records with $\tilde{\sigma} = 1$, converted to $(\varepsilon, 10^{-5})$ (Rényi DP in the full edition). The first record gets $\varepsilon = 0.0481$, the one in the middle 0.0707, and the one used 100 steps before the end 0.4849. The last record gets 5.2985, exactly what it would get if every iterate were released: no later step pays anything for it.

So a report must state the worst record's value, which gets no amplification at all; randomizing the stopping time spreads the benefit across records. The bound is exact in the worst case, when the step that uses your record wipes out all earlier noise, and loose for typical runs, where earlier noise and strict contraction also help (Figure 33.2). It covers one pass in a fixed order; several passes or subsampling need other theorems (Altschuler & Talwar 2022). And it needs convexity, because a non-contractive step can stretch a small gap faster than the noise pays it off.

33.5. Hidden state: privacy that stops growing

Full-batch noisy GD uses every record at every step, and composition charges every step, so the privacy loss grows linearly in the number of steps T, and amplification by iteration alone does not help: each step opens a fresh gap, and paying each one off as it comes is composition again. Two properties can change that when only the final model is released. If the parameters live in a **bounded convex set**, the two runs can never be further apart than its diameter, however long they run, so only the last stretch of steps has to hide that gap and the cost stops depending on T. If the loss is **strongly convex**, each step is a strict contraction that shrinks the gap for free.

Altschuler & Talwar (2022) prove the bounded-domain version: the last iterate's RDP grows like composition up to a number of steps proportional to the diameter, then stays flat, and their lower bound shows the shape is right. For a quadratic loss the answer is exact. With contraction factor $c < 1$ per step, your record's per-step shift s reaches the end damped, so the total signal saturates

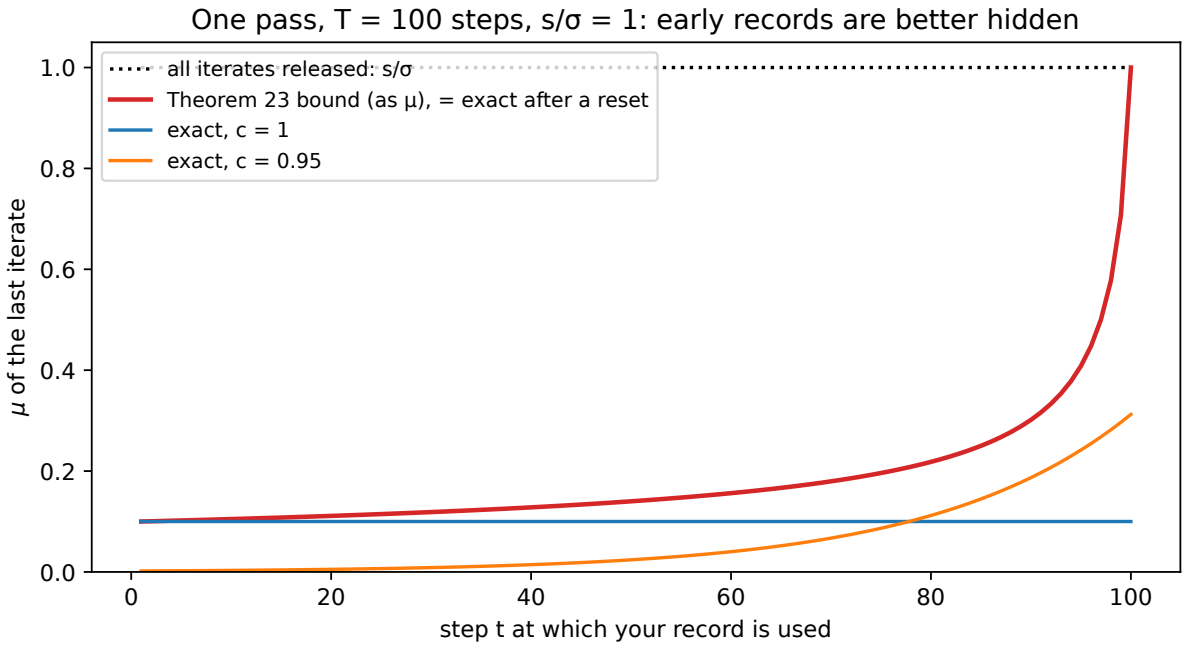


Figure 33.2.: The μ of the last iterate against the step t at which your record is used, for a one-pass run of $T = 100$ steps with per-step shift equal to the noise standard deviation. Releasing all iterates gives $\mu = 1$ for every record (dotted). The bound Equation 33.4 (red) is exact in the worst case; for typical contractions (blue, orange) the true value is smaller.

at $s/(1-c)$; the noise saturates the same way, at a standard deviation of $\sigma/\sqrt{1-c^2}$. Their ratio is the limiting Gaussian mechanism,

$$\mu_\infty = \frac{s}{\sigma}\sqrt{\frac{1+c}{1-c}}, \tag{33.5}$$

reached after about $1/(1-c)$ steps, the time it takes c^T to become small; for a gradient step of size $1/\beta$ on a λ-strongly convex, β-smooth loss, that is the condition number β/λ. With $c = 1$ the signal grows like T and the noise only like $\sqrt{T}$, so there is no plateau: without contraction, hiding the state buys nothing.

Run 10,000 steps of full-batch noisy GD with $\sigma/s = 10$ on a quadratic with $c = 0.99$. Releasing every iterate is a Gaussian mechanism with $\mu = 10$, which is $\varepsilon = 91.8173$ at $\delta = 10^{-5}$: no guarantee at all. Releasing only the final model is at the plateau, $\mu = 1.4107$, which is $\varepsilon = 6.5534$, with the same noise. The bounded-domain bound is looser: for a linear loss on the unit interval (Figure 33.3, right), after 2000 steps composition charges an RDP of 20.0000 at $\alpha = 2$ and the bound 4.0000, while a hard instance reaches 0.3786. The shape is right; the constant is loose.

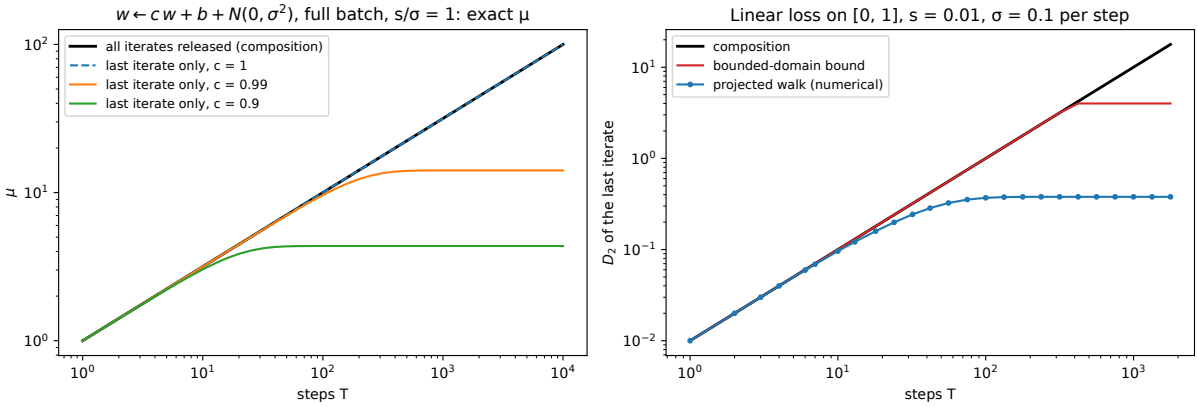


Figure 33.3.: Left: the exact μ of the last iterate of one-dimensional full-batch noisy GD against the number of steps T, for $c = 1$ (on top of composition), $c = 0.99$ and $c = 0.9$; with contraction it plateaus. Right: the Rényi divergence of order 2 of the last iterate for a linear loss on $[0, 1]$: composition, the bounded-domain bound, and the exact value for two projected random walks.

Both conditions are load-bearing. The argument needs **only the last iterate released**, because the adversary must not see the run before it has forgotten: checkpoints, logged intermediate models, a model published after each epoch, or the best of several checkpoints put you back to composition. It also needs **contractive steps**, which require convexity. For non-convex losses, Annamalai (2024) built a loss for which the last iterate of DP-SGD leaks as much as all iterates together. That attacks seeing only the final model have fallen short of the DP-SGD bound (Nasr et al. 2021) is an observation about attacks, not a guarantee. Do not report a hidden-state ε for deep learning.

33.6. Code

This standard-library snippet builds Equation 33.1 in log space, takes the larger hockey-stick divergence over both directions, and bisects for ε:

```
from math import exp, lgamma, log


def log_binom_pmf(k: int, n: int, p: float) -> float:
    return lgamma(n + 1) - lgamma(k + 1) - lgamma(n - k + 1) + k * log(p) + (n - k) * log(1


def shuffled_rr_pair(n: int, eps0: float) -> tuple[list[float], list[float]]:
    """Distribution of the number of 1-reports when you hold 0 (P) or 1 (Q); the n - 1 other
    theta = exp(eps0) / (1 + exp(eps0))
    b = [exp(log_binom_pmf(y, n - 1, 1 - theta)) for y in range(n)] + [0.0]
    P = [theta * b[y] + (1 - theta) * (b[y - 1] if y else 0.0) for y in range(n + 1)]
    Q = [(1 - theta) * b[y] + theta * (b[y - 1] if y else 0.0) for y in range(n + 1)]
    return P, Q


def hockey_stick(P: list[float], Q: list[float], eps: float) -> float:
    return sum(max(p - exp(eps) * q, 0.0) for p, q in zip(P, Q))


def shuffled_rr_epsilon(n: int, eps0: float, delta: float) -> float:
    P, Q = shuffled_rr_pair(n, eps0)
    lo, hi = 0.0, eps0  # shuffling never does worse than the local eps0
    for _ in range(50):
        mid = (lo + hi) / 2
        worst = max(hockey_stick(P, Q, mid), hockey_stick(Q, P, mid))
        lo, hi = (mid, hi) if worst > delta else (lo, mid)
    return hi


print(f"{shuffled_rr_epsilon(10_000, 2.0, 1e-6):.4f}")  # 0.0870
print(f"{shuffled_rr_epsilon(100_000, 4.0, 1e-6):.4f}")  # 0.0847
print(f"{shuffled_rr_epsilon(1_000, 4.0, 1e-6):.4f}")  # 1.6910
```

It reproduces the exact column of Table 33.1. The full edition's script, shuffling_iteration.py, also computes the numeric clone bound and every iteration result:

```
uv run python code/part04-privacy-accounting/shuffling_iteration.py
```

33.7. Cheat sheet

Table 33.2.: The results of this chapter. All use replace-one neighbours.

Result	Statement	Use
Shuffled binary randomized response	Count pair Equation 33.1; $\mu \approx 2\sinh(\varepsilon_0/2)/\sqrt{n}$	Compute exactly; the approximation fails with few 1-reports
Numeric clone bound	Exact $\delta(\varepsilon)$ of the clone count pair	Deploy with it for any ε_0-DP randomizer; 1.44–2.36 times the exact value for randomized response
Closed-form clone bound	Equation 33.3, needs $\varepsilon_0 \le \ln \frac{n}{16\ln(2/\delta)}$	Scaling only; 3.10–4.75 times the numeric bound
One-pass noisy SGD, last iterate	$\varepsilon_t(\alpha) = \frac{\alpha}{2\tilde{\sigma}^2(n-t+1)}$	Convex, smooth losses; report the last record's value
Bounded domain or strong convexity	RDP of the last iterate plateaus; quadratic limit $\frac{s}{\sigma}\sqrt{\frac{1+c}{1-c}}$	Convex training, last iterate only

Rules of thumb

- Shuffling turns ε_0-local reports into a central ε that falls like $1/\sqrt{n}$, but never lowers the pure-DP level.
- Amplification by iteration needs convex, smooth losses and a hidden state; the record used last gets none.
- Never claim hidden-state privacy for non-convex training or when checkpoints are released.

33.8. Further reading

- Feldman, McMillan & Talwar, "Hiding Among the Clones: A Simple and Nearly Optimal Analysis of Privacy Amplification by Shuffling" (FOCS 2021): the clone reduction and bound, and the numerical method to compute it.
- Feldman, Mironov, Talwar & Thakurta, "Privacy Amplification by Iteration" (FOCS 2018): the shifted Rényi divergence and the one-pass noisy SGD guarantee.
- Altschuler & Talwar, "Privacy of Noisy Stochastic Gradient Descent: More Iterations without More Privacy Loss" (NeurIPS 2022): the bounded-domain plateau, with a matching lower bound.
- Annamalai, "It's Our Loss: No Privacy Amplification for Hidden State DP-SGD With Non-Convex Loss" (AISec 2024): why the hidden-state results do not extend to deep learning.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

34. Adaptive composition: filters, odometers and individual accounting

34.1. Why this matters

You train a model with differential privacy (DP) using stochastic gradient descent (SGD) with clipped per-example gradients and Gaussian noise, called DP-SGD. You may want to stop when the validation loss stalls, lower the noise mid-run, or let well-fitted records keep training. Every composition theorem so far assumes the opposite: the per-step budgets $\varepsilon_1, \dots, \varepsilon_k$ and the number of steps k are fixed before the run (Advanced and optimal composition). Once they depend on what the run released, they are random, and plugging the values you happened to use into a fixed-parameter theorem afterwards is not valid.

So the question splits in two. A **privacy filter** answers "may I run one more step without exceeding a budget fixed in advance?" A **privacy odometer** answers "how much have I spent so far?", and since you may stop at any moment, its answer must hold at every step at once. Both rest on one fact about Rényi differential privacy (RDP): at an order fixed before the run, adding up the spends as you go is as safe as if they had been fixed. That makes a filter free, an odometer cheap, and per-record accounting possible.

34.2. Why plugging in the realized parameters fails

Advanced composition is a high-probability statement: at a time t fixed in advance, the total privacy loss exceeds a bound U_t, its mean plus a spread growing like $\sqrt{t}$, only with probability δ. It says nothing about a time chosen by watching the outputs. An analyst who stops when the loss happens to be high picks exactly the paths on which the bound fails, and so does stopping on anything correlated with it, such as "these answers look unusual".

Ask randomized response with $\varepsilon = 0.1$ about one person's bit at every step. Each report moves the privacy loss up or down by 0.1, so the loss is a random walk with a slight upward drift. At any single step up to 1,000, the plug-in bound at $\delta = 0.01$ fails with probability at most 0.0016. Now stop at the first crossing and report U_T. The bound is narrow early, when $\sqrt{t}$ is small, so a few walks cross it early and are frozen there (Figure 34.1). By 10^4 steps a fraction 0.0318 of runs have crossed (Table 34.1): "$\varepsilon = U_T$ with $\delta = 0.01$" is wrong 3.18 times as often as it claims. A smaller δ does not save it (Table 34.1, last row).

Table 34.1.: Exact probability that randomized response with $\varepsilon = 0.1$ crosses the plug-in advanced-composition bound by step n. It exceeds δ and keeps growing with the horizon.

Horizon n	100	1000	10,000
$\delta = 0.05$	0.0575	0.1004	0.1424
$\delta = 0.01$	0.0090	0.0202	0.0318
$\delta = 10^{-5}$	3.12×10^{-6}	1.80×10^{-5}	3.69×10^{-5}

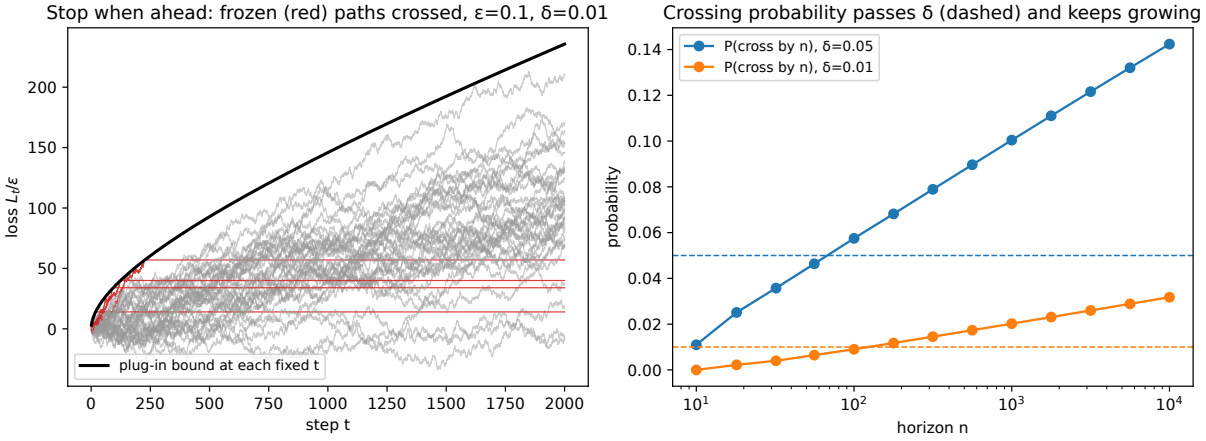


Figure 34.1.: Left: forty runs of randomized response with $\varepsilon = 0.1$, the loss in units of ε, and the plug-in bound at $\delta = 0.01$; an analyst who stops at the first crossing freezes the red paths there. Right: the probability that the bound is crossed by step n, for $\delta = 0.05$ and 0.01 (dashed lines); it passes δ and keeps growing.

The probability keeps growing because a random walk eventually wanders further than any fixed multiple of $\sqrt{t}$ (its reach grows like $\sqrt{t \ln \ln t}$), so given time it crosses with certainty. A valid running bound must be wider, and Rogers et al. (2016) showed that every odometer pays at least an extra $\ln \ln n$ inside the logarithm for n steps.

Basic composition escapes the problem: the sum of the realized ε_t bounds the loss of every output, so there is no bad event for a stopping rule to select. A filter escapes it too, because it makes no running claim to catch at a bad moment; it only asks whether the next step fits a budget fixed in advance.

34.3. The Rényi filter

RDP describes each step by a curve $\varepsilon_t(\alpha)$ over orders $\alpha > 1$, and with fixed parameters the curves of a run add (Rényi DP in the full edition). This survives adaptivity at one fixed order. Fix an order α and an RDP budget B before the run. Let step t be $(\alpha, \varepsilon_t(\alpha))$-RDP whatever happened before it, with $\varepsilon_t(\alpha)$ known before the step runs. Run step t only if the spends so far, its own included, fit:

$$\sum_{j \le t} \varepsilon_j(\alpha) \le B. \tag{34.1}$$

Then the stopped run is (α, B)-RDP, as if the spends had been fixed.

The reason is a betting argument. Treat $e^{(\alpha-1)(\text{loss}-\text{spend})}$ as your wealth in a game. RDP at order α bounds the average of $e^{(\alpha-1)\times\text{loss}}$, so a step charged its RDP spend cannot make this wealth grow on average: the game is fair or tilted against you. In such a game no quitting rule raises your expected final wealth, as long as you decide whether to play each round before seeing its result. If the total spend is at most B on every path, the average of $e^{(\alpha-1)\times\text{loss}}$ at the end is at most $e^{(\alpha-1)B}$, which is exactly what (α, B)-RDP says.

Each condition is a step of that argument. A step's price must be known before it runs: a mechanism allowed to charge little on exactly the outputs that reveal most would make the game a winning one. The budget must hold on every path, not on average, because the paths that spend most may be those where the loss is high. And the order must be fixed in advance. You may track several orders and halt when any one is exhausted, since then each held on every path; continuing while *some* order still fits, chosen after seeing the spends, is not free (Section 34.5 prices it).

You train with full-batch noisy gradient descent towards $(5.299, 10^{-5})$ at order $\alpha = 5.8$. Mironov's conversion turns (α, B)-RDP into $(B + \ln(1/\delta)/(\alpha - 1), \delta)$-DP, so run backwards it sets $B = 5.299 - \ln(10^5)/4.8 = 2.900$. A full-batch Gaussian step with noise multiplier σ_t spends $\alpha/(2\sigma_t^2)$. With $\sigma_t = 10$ each step spends $5.8/200 = 0.0290$ and you get 100 steps. If training plateaus at step 60 and you drop to $\sigma_t = 5$, each later step costs $5.8/50 = 0.1160$, and the filter allows $(2.900 - 60 \times 0.0290)/0.1160 = 10$ more. Either way the run is $(5.8, 2.900)$-RDP, whatever rule decided the switch.

At a fixed order the filter costs nothing. It can even beat fixed-curve composition, which charges every step its worst branch, while the filter charges what each path spent.

34.4. Filters for pure differential privacy

An interactive query system has a global budget (ε_g, δ), and each query brings a pure-DP ε_t chosen after earlier answers. Summing the ε_t is valid but grows like the number of queries k; you want advanced composition's $\sqrt{k}$ without the failure of Section 34.2.

The Rényi filter delivers it. An ε_t-DP step spends at most $\alpha\varepsilon_t^2/2$ at every order α (Concentrated differential privacy in the full edition). So a rule that keeps $\frac{1}{2}\sum_t \varepsilon_t^2$ below a fixed level is a Rényi filter at every order at once, and because the rule does not mention the order, the conversion may pick the best one afterwards. Doing that conversion gives the **Whitehouse filter** (Whitehouse et al. 2023): continue while

$$\sqrt{2\ln(1/\delta)\sum_t \varepsilon_t^2} + \tfrac{1}{2}\sum_t \varepsilon_t^2 \le \varepsilon_g, \tag{34.2}$$

with sums over every step so far. It reads like advanced composition, the first term the spread of the total loss and the second its mean, and the stopped run is (ε_g, δ)-DP.

At $\delta = 10^{-6}$ and $\varepsilon_g = 1$, the filter allows 349 steps of 0.01, slightly more than the 337 of advanced composition with k fixed in advance (Table 34.2). It wins, despite allowing adaptivity, because its mean term, $\varepsilon_t^2/2$ per step, is tighter than the $\varepsilon_t(e^{\varepsilon_t} - 1)$ of the fixed-k form. From the budget side, 1,000 steps of 0.01 need 1.712 under the Whitehouse filter, 1.763 under fixed-k advanced composition and 10.000 under the basic filter $\sum_t \varepsilon_t \le \varepsilon_g$.

Table 34.2.: Steps of ε_0 allowed by a global budget $\varepsilon_g = 1$ at $\delta = 10^{-6}$.

ε_0	Basic	Advanced (fixed k)	Rogers filter	Whitehouse filter
0.01	100	337	154	349
0.001	1000	33,785	15,481	34,937

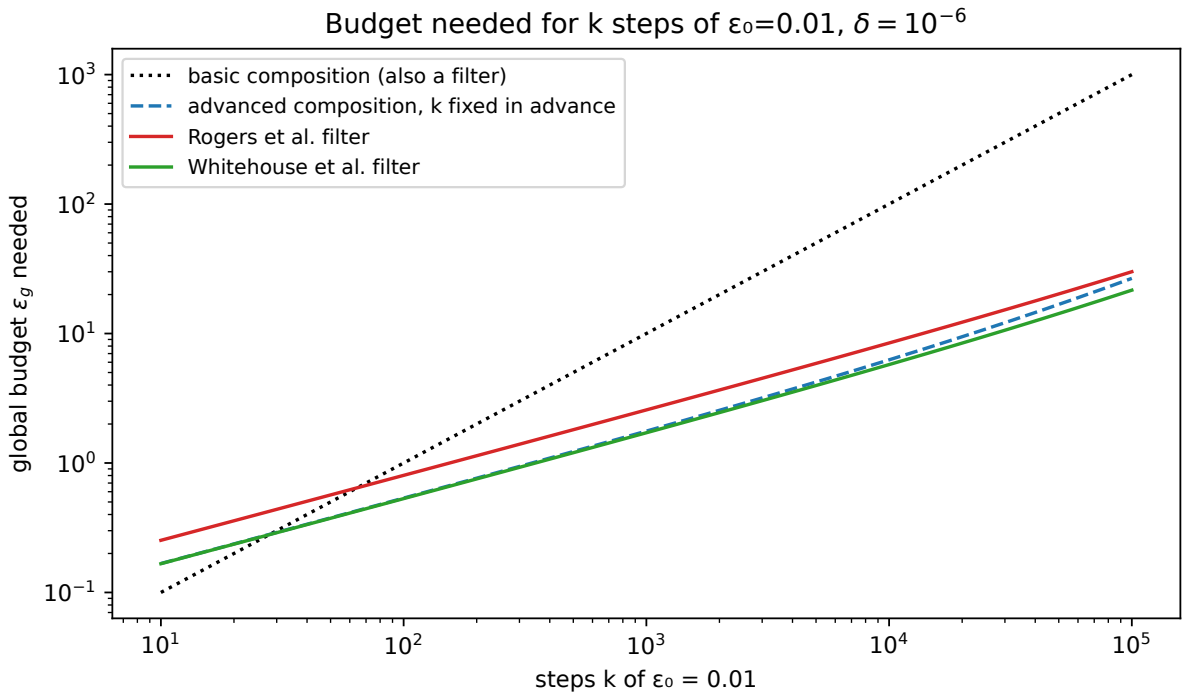


Figure 34.2.: Global budget needed to run k steps of $\varepsilon_0 = 0.01$ at $\delta = 10^{-6}$: basic composition, advanced composition with k fixed in advance, and the Rogers and Whitehouse filters. The Whitehouse filter lies on or below fixed-k advanced composition; the Rogers filter sits a constant factor above.

The older Rogers et al. (2016) filter has the same shape, but its inequality must hold for every value of $\sum_t \varepsilon_t^2$ at once, and that costs 1.40 to 1.51 times the budget and fewer than half the steps; use the Whitehouse form. Check it *before* each step, with that step's ε_t included, since the guarantee is about the decision to run it. For steps with $\delta_t > 0$, add a second rule, $\sum_t \delta_t \le \delta''$, and the guarantee becomes $(\varepsilon_g, \delta + \delta'')$.

34.5. Odometers

Early stopping and open-ended analyses need a running "how much have I spent?" that is true whenever the run stops. Return to the betting game of Section 34.3. Markov's inequality says that the wealth, which starts at 1, reaches c at one given time with probability at most $1/c$. Ville's inequality extends this to reaching c at *any* time, because "stop the first time you reach

c" is itself a quitting rule and cannot raise the mean. Unpacked, at one order fixed in advance, Mironov's conversion of the running spend holds at every step at once, for free.

The cost comes from the order: the best order for 10 steps is not the best for 1,000. To choose it late, fix a grid Λ of $|\Lambda|$ orders, track the spend $S_t(\alpha) = \sum_{j \le t} \varepsilon_j(\alpha)$ at each, and give each order a share $\delta/|\Lambda|$ of the failure probability. The **Ville odometer** reports the best order's conversion,

$$U_t = \min_{\alpha \in \Lambda} \Big[S_t(\alpha) + \frac{\ln(|\Lambda|/\delta)}{\alpha - 1} \Big], \tag{34.3}$$

and with probability at least $1 - \delta$ the loss stays below U_t at every step. It is Mironov's conversion with δ replaced by $\delta/|\Lambda|$; that union bound is the whole price.

For full-batch Gaussian steps with $\sigma = 10$ at $\delta = 10^{-5}$, the part's 154 orders raise $\ln(1/\delta) = 11.513$ to 16.550, and the odometer costs 15–19% over the fixed-time value: 6.253 against 5.299 at $t = 100$ (Table 34.3). A single order chosen in advance, 5.8, is free at $t = 100$, its design point, and poor elsewhere: 31.399 at $t = 1000$. On the part's running example (DP-SGD with $q = 0.01$, $\sigma = 1$, 10^4 steps, $\delta = 10^{-5}$), the fixed-time RDP value is 7.43, and an odometer valid whenever the run stops costs 8.82 (Ville) or 9.08 (the slightly looser odometer of Lécuyer 2021).

Table 34.3.: Running ε at $\delta = 10^{-5}$ for t full-batch Gaussian steps with $\sigma = 10$. Only the last three columns are valid when t is chosen from the outputs.

Steps t	Fixed-time conversion	Ville odometer (154 orders)	Lécuyer odometer	Ville at $\alpha = 5.8$ only
10	1.568	1.869	1.916	2.689
100	5.299	6.253	6.507	5.299
1000	20.175	23.194	24.633	31.399

Keep the grid small. Read an odometer for what it is: "$\varepsilon = U_T$" says the loss exceeds the random number U_T with probability at most δ, a tail bound rather than an (ε, δ)-DP guarantee. To get a guarantee, stop by a rule that keeps $U_T \le \varepsilon_g$, which turns the odometer into a filter; if you have a budget anyway, a Rényi filter at one order is cheaper.

34.6. Individual privacy accounting in DP-SGD

DP-SGD's usual analysis charges every record at every step as if its clipped gradient had norm C, the clipping norm. But once the model fits a record, its gradient shrinks and it moves the noisy sum less, so the same noise hides it better.

That cost is exact. With full-batch steps and noise of standard deviation σC on the summed clipped gradients, removing record i shifts the output by its clipped gradient $\bar{g}_{i,t}$, so its spend is the Gaussian RDP for that shift:

$$\varepsilon_{i,t}(\alpha) = \frac{\alpha \|\bar{g}_{i,t}\|^2}{2\sigma^2 C^2} \le \frac{\alpha}{2\sigma^2}. \tag{34.4}$$

It depends only on the record and the current model, which depends only on earlier releases, so it is known before the step: what the Rényi filter needs. The **individual filter** (Feldman & Zrnic

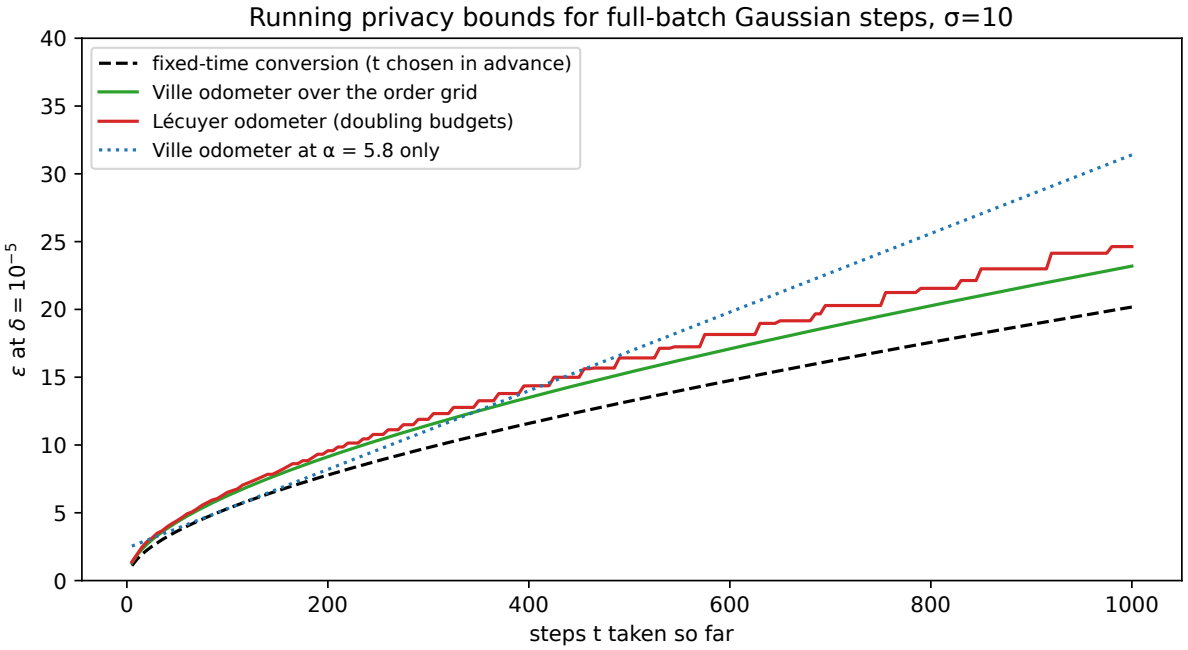


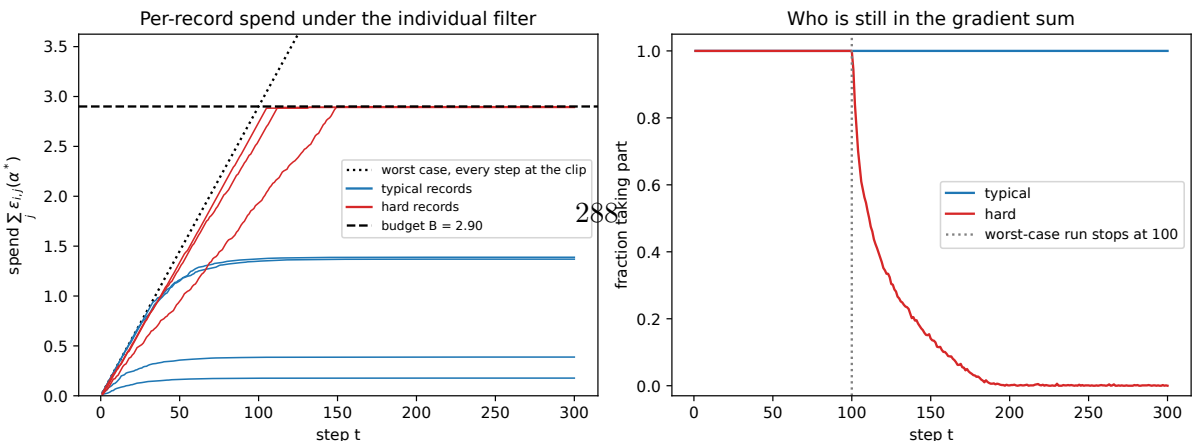
Figure 34.3.: Running privacy bounds at $\delta = 10^{-5}$ for full-batch Gaussian steps with $\sigma = 10$: the fixed-time conversion (dashed, not a valid odometer), the Ville odometer over 154 orders, Lécuyer's odometer, and a Ville odometer at the single order 5.8.

2021) runs one Rényi filter per record. Record i joins the sum only while its own spends total at most B; once dropped, it no longer changes the output and spends nothing. Neighbouring datasets differ in one record, whose filter held, so the run is (α, B)-RDP. No step costs more than the worst case, so every record trains at least as long as in the worst-case run.

The full edition simulates 10,000 records ($C = 1$, $\sigma = 10$, full batch): 90% typical records whose gradients shrink as the model fits them, 10% hard records it never fits. The worst-case run has 100 steps and $\varepsilon = 5.299$ at $\delta = 10^{-5}$, best at order 5.8, where $B = 2.900$. Converting each record's own spend after 100 steps (Table 34.4) gives a typical median of 3.17 and a hard median of 5.02, near the worst case. Run the individual filter for 300 steps and hard records reach the budget and are dropped from step 101 (the median one at step 111), while no typical record ever is: their median spend at step 300 is 0.778, a quarter of the budget (Figure 34.4).

Table 34.4.: Individual ε at $\delta = 10^{-5}$ after 100 full-batch steps with $\sigma = 10$, at order 5.8. The worst case is 5.299.

Records	10th percentile	Median	90th percentile
All	2.56	3.25	3.81
Typical	2.55	3.17	3.67
Hard	4.23	5.02	5.26



using the subsampled Gaussian at noise multiplier $\sigma C/\|\bar{g}_{i,t}\|$ (Amplification by subsampling in the full edition), but you need every record's gradient norm at every step, sampled or not.

34.7. Code

Filters and odometers are a sum of spends and a comparison. This snippet implements the Whitehouse filter, the Rényi filter for a noise schedule and the Ville odometer for full-batch Gaussian steps, and reproduces the numbers above.

```
from math import log, sqrt


def whitehouse_statistic(eps_steps: list[float], delta: float) -> float:
    """Advanced composition of pure-DP budgets with mean term eps^2 / 2: valid under adaptiv
    v = sum(e * e for e in eps_steps)
    return sqrt(2 * log(1 / delta) * v) + v / 2


def steps_allowed(eps_step: float, eps_global: float, delta: float) -> int:
    """Run steps of eps_step while the Whitehouse filter, checked before each step, says con
    spent: list[float] = []
    while whitehouse_statistic([*spent, eps_step], delta) <= eps_global:
        spent.append(eps_step)
    return len(spent)


def gaussian_rdp(alpha: float, sigma: float) -> float:
    """RDP at order alpha of one full-batch Gaussian step with noise multiplier sigma."""
    return alpha / (2 * sigma**2)


def renyi_filter(sigmas: list[float], alpha: float, budget: float) -> int:
    """Number of steps of a noise schedule that a Renyi filter at order alpha lets through."
    spent, steps = 0.0, 0
    for sigma in sigmas:
        cost = gaussian_rdp(alpha, sigma)
        if spent + cost > budget + 1e-12:  # tolerance for floating-point sums
            break
        spent, steps = spent + cost, steps + 1
    return steps


ORDERS = [1 + x / 10 for x in range(1, 100)] + [float(x) for x in range(12, 64)] + [128.0, 2


def ville_odometer(t: int, sigma: float, delta: float, orders: list[float]) -> float:
```

```
    """Running epsilon after t full-batch Gaussian steps, valid whenever the run stops."""
    return min(t * gaussian_rdp(a, sigma) + log(len(orders) / delta) / (a - 1) for a in orde


print(f"{whitehouse_statistic([0.01] * 1000, 1e-6):.3f}")  # 1.712
print(steps_allowed(0.01, 1.0, 1e-6))  # 349
alpha, budget = 5.8, 2.9
print(renyi_filter([10.0] * 1000, alpha, budget))  # 100
print(renyi_filter([10.0] * 60 + [5.0] * 1000, alpha, budget) - 60)  # 10
print(f"{budget + log(1 / 1e-5) / (alpha - 1):.3f}")  # 5.299
print(len(ORDERS))  # 154
print(f"{ville_odometer(100, 10.0, 1e-5, ORDERS):.3f}")  # 6.253
print(f"{ville_odometer(100, 10.0, 1e-5, [alpha]):.3f}")  # 5.299
print(f"{ville_odometer(1000, 10.0, 1e-5, [alpha]):.3f}")  # 31.399
```

`ORDERS` is the part's grid of orders. The full edition's script, adaptive_composition.py, also computes the plug-in failure exactly, checks the Rényi filter by simulation and runs the individual-accounting experiment:

```
uv run python code/part04-privacy-accounting/adaptive_composition.py
```

34.8. Cheat sheet

Table 34.5.: Filters, odometers and individual accounting.

Object	Rule or bound	Guarantee
Basic filter and odometer	Continue while $\sum_t \varepsilon_t \le \varepsilon_g$; report $\sum_{j\le t} \varepsilon_j$	$(\varepsilon_g, 0)$-DP; holds on every path
Whitehouse filter (pure DP)	$\sqrt{2\ln(1/\delta)\sum_t \varepsilon_t^2} + \frac{1}{2}\sum_t \varepsilon_t^2 \le \varepsilon_g$	(ε_g, δ)-DP
Rényi filter	$\sum_{j\le t} \varepsilon_j(\alpha) \le B$ on every path, α fixed in advance	(α, B)-RDP; DP target: $B = \varepsilon_g - \ln(1/\delta)/(\alpha-1)$
Ville odometer	$U_t = \min_{\alpha\in\Lambda}[S_t(\alpha) + \ln(\lvert\Lambda\rvert/\delta)/(\alpha-1)]$	Loss above U_t at some t with probability at most δ
Individual spend, Gaussian sum	$\varepsilon_{i,t}(\alpha) = \alpha\lVert\bar g_{i,t}\rVert^2/(2\sigma^2C^2)$, at most $\alpha/(2\sigma^2)$	Per-record filter gives (α, B)-RDP for every record

Rules of thumb

- Never report a fixed-time bound at a time chosen from the outputs. With a budget, use a Rényi filter at an order fixed before the run: it costs nothing. Without one, use an odometer over a small grid of orders.
- Per-record ε values are sensitive estimates, not guarantees.

34.9. Further reading

- Rogers, Roth, Ullman & Vadhan, "Privacy Odometers and Filters: Pay-as-you-Go Composition" (NeurIPS 2016): the definitions, and why odometers cost more.
- Whitehouse, Ramdas, Rogers & Wu, "Fully-Adaptive Composition in Differential Privacy" (ICML 2023): the filter that matches advanced composition.
- Feldman & Zrnic, "Individual Privacy Accounting via a Rényi Filter" (NeurIPS 2021): the Rényi filter and per-record accounting.
- Koskela, Tobaben & Honkela, "Individual Privacy Accounting with Gaussian Differential Privacy" (ICLR 2023): tighter individual accounting, including subsampled DP-SGD.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

35. Choosing an accountant

35.1. Why this matters

You trained a model with differential privacy (DP), using stochastic gradient descent (SGD) with clipping and Gaussian noise, called DP-SGD: each record joined each batch independently with probability $q = 0.01$ (Poisson sampling), gradients were clipped to norm C, and noise of standard deviation σC was added to their sum, with **noise multiplier** $\sigma = 1$, for $k = 10{,}000$ steps. Now you must write "the model is $(\varepsilon, 10^{-5})$-DP" in a model card. The procedure that bounds or estimates ε is an **accountant**. On this run, composing per-step guarantees gives 11296.93, Rényi differential privacy (RDP) gives 7.4287 or 6.7127 depending on the conversion, the Gaussian differential privacy (GDP) central limit theorem (CLT) gives 6.0071, and a privacy loss distribution (PLD) accountant gives the interval $[6.1377, 6.2377]$. Which do you report, and what else must you say?

The numbers differ because they are different kinds of statement. An **upper bound** comes with a proof that the run is (ε, δ)-DP; it may be loose, but it is true. An **approximation** has no proof, and if it lands below the truth, the claim it supports is false. A certified **lower bound** says the true ε is at least this large: not a guarantee, but a referee for every other number. Hence the rule this chapter unpacks: **report the tightest upper bound whose assumptions hold for the code you ran, with enough detail that someone else can recompute it.**

35.2. Every accountant on one run

Every accountant starts from what one step reveals about one record. Along that record's clipped gradient, in units of C, the output is $\mathcal{N}(0, \sigma^2)$ without the record. With it, the record is sampled with probability q and then shifts the output by 1, giving the mixture $(1-q)\mathcal{N}(0, \sigma^2) + q\mathcal{N}(1, \sigma^2)$. Accountants differ in how much of this pair they keep before combining k copies, and whatever they throw away must be replaced by a worst case. So the more they keep, the tighter they are.

The crudest keeps two numbers per step, an amplified (ε, δ) pair (Amplification by subsampling in the full edition), and composes them. That is a valid bound, but composition must assume every step leaks its worst case, while in reality the losses of different steps mostly cancel. It is right at $k = 1$ and over a thousand times too large at $k = 10{,}000$ (Table 35.1); advanced composition is even worse here.

RDP keeps a whole curve, the Rényi divergence of the pair at every order α (Rényi DP in the full edition). Curves add exactly under composition, and a conversion then turns k times the curve into one (ε, δ) pair (From Rényi DP to (ε, δ) and to tests in the full edition). Mironov's, the improved and the optimal conversion are each proved for every pair of distributions with

that curve, so all three are upper bounds, ordered Mironov ≥ improved ≥ optimal ≥ true ε. The improved conversion beats Mironov's at every order, so never use the older one.

RDP's excess has two sources, and the run's length decides which dominates. One is the conversion's toll, $\ln(1/\delta)/(\alpha-1)$, about $11.5/(\alpha-1)$ at $\delta = 10^{-5}$, paid however small the curve is. After one step the curve is tiny, the toll is most of the answer, and RDP with the improved conversion overstates the PLD by 379.1%. In a long run the toll is negligible, but the curve itself keeps only moments of the privacy loss, not its distribution: that costs the last 7.6% at $k = 10{,}000$, and no conversion can remove it.

The GDP CLT treats the run as μ-GDP with $\mu = q\sqrt{k(e^{1/\sigma^2}-1)}$ (f-DP and Gaussian differential privacy in the full edition), since a sum of many small losses should be nearly Gaussian. But that is a large-k limit with no error bound, and $\delta = 10^{-5}$ is read far in the tail, where a Gaussian approximation is least reliable. On this run it falls below the certified lower bound at every k.

The PLD accountant keeps everything: the whole distribution of the privacy loss on a grid of width h, composed numerically (Numerical accounting with privacy loss distributions in the full edition). Rounding every loss up describes a pair that leaks at least as much as the real one, which stays true through composition, so the result is an upper bound; rounding down gives a lower bound. Each step's rounding moves the loss by less than h, so after k steps the two are at most kh apart: 0.1 here, with $h = 10^{-5}$.

Table 35.1.: DP-SGD with $q = 0.01$, $\sigma = 1$: ε at $\delta = 10^{-5}$ after k steps under every accountant. The PLD columns use the grid $h = 10^{-5}$.

k	Basic	Advanced	RDP, Mironov	RDP, improved	RDP, optimal	GDP CLT	PLD upper	PLD lower
1	0.20	1.13	1.3175	0.9555	0.4899	0.0368	0.1995	0.1994
10	3.64	7.88	1.4145	1.0353	0.7951	0.1304	0.3800	0.3799
100	58.01	78.10	1.6118	1.2141	1.1538	0.4575	0.7185	0.7175
1,000	839.39	1258.64	2.5380	2.1014	2.1009	1.6177	1.8332	1.8232
10,000	11296.93	24379.55	7.4287	6.7127	6.7127	6.0071	6.2377	6.1377

Suppose you ran 1,000 steps. Your paper might say $\varepsilon = 2.54$ (RDP with Mironov's conversion, as in older code), 2.10 (RDP, improved), 1.83 (PLD) or 1.62 (CLT). The first three are true, and 1.83 says the most. The fourth is false: the optimistic PLD proves $\varepsilon \geq 1.8232$, so the run is not $(1.62, 10^{-5})$-DP.

35.3. Calibrating the noise for a target ε

In practice you fix the budget, the steps and the batch size, and ask how much noise to add. Then a loose accountant costs utility: it overstates ε at every σ, so it asks for more noise to meet the same target. The averaged gradient's noise has standard deviation $\sigma C/(qn)$ for n records, so every percent of σ is a percent of gradient noise.

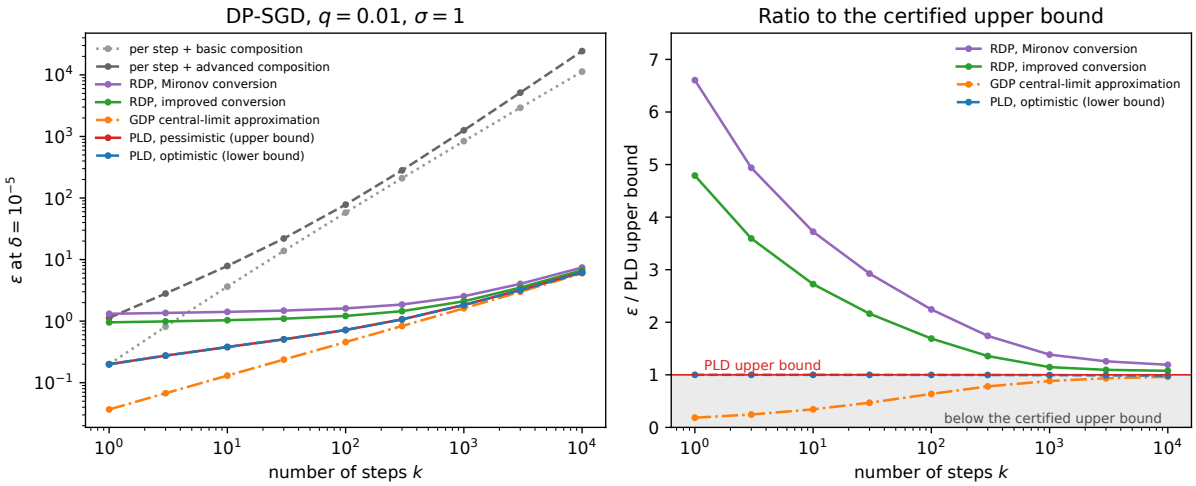


Figure 35.1.: Left: ε at $\delta = 10^{-5}$ against the number of steps for DP-SGD with $q = 0.01$, $\sigma = 1$, under every accountant (the two PLD curves overlap). Right: each divided by the PLD upper bound. Ratios above 1 are valid but loose; the shaded region below 1 is where only the PLD lower bound may lie, and the CLT lies there at every k.

The noise accountant A requires for a target ε^* is the smallest σ at which its reported $\varepsilon_A(\sigma)$ meets the target:

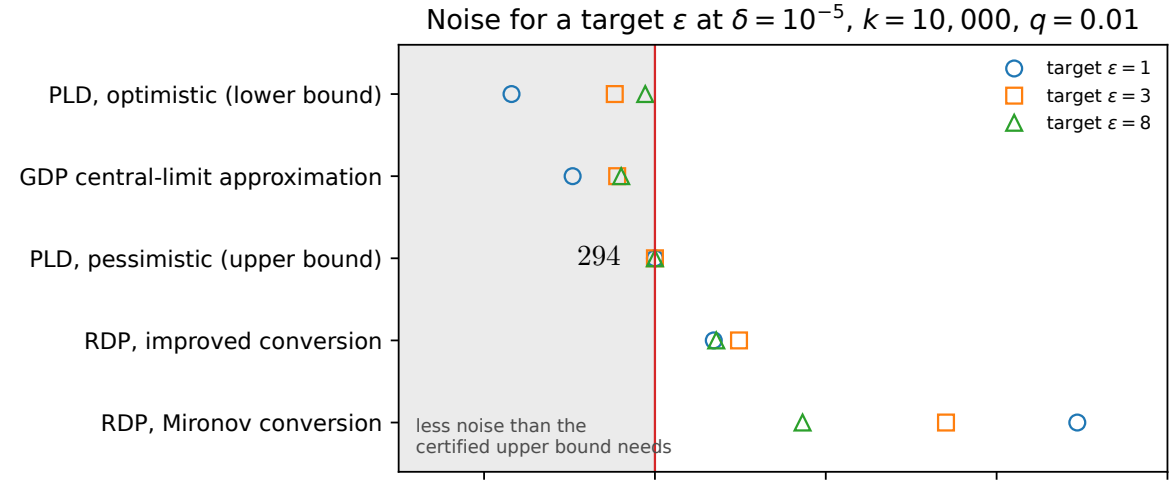
$$\sigma_A(\varepsilon^*) = \min\{\sigma > 0 : \varepsilon_A(\sigma) \le \varepsilon^*\}. \tag{35.1}$$

More noise never raises a correct accountant's ε: a noisier run can be made from a less noisy one by adding independent noise, without looking at the data, and that cannot reveal more. So this is where a decreasing function crosses a level, and bisection finds it. Return the end of the final bracket where $\varepsilon_A \le \varepsilon^*$ was actually checked, never the midpoint, which may miss the target.

Table 35.2.: The noise multiplier each accountant requires for a target ε at $\delta = 10^{-5}$, with $k = 10{,}000$ steps at $q = 0.01$.

Target ε	Advanced	RDP, Mironov	RDP, improved	GDP CLT	PLD upper	PLD lower
1	24.682	4.974	4.126	3.796	3.988	3.654
3	10.505	1.854	1.662	1.549	1.584	1.547
8	5.711	0.962	0.917	0.868	0.885	0.880

With $n = 50{,}000$ records the expected batch holds 500. At $\varepsilon = 8$, RDP with the improved conversion sets $\sigma = 0.917$ and the PLD $\sigma = 0.885$: 3.6% more noise for the same certified budget (Table 35.2). Across the targets the improved conversion costs 3.4 to 4.9% more noise, less than its 7.6% excess in ε, because in this range a small rise in σ buys a larger fall in ε. Mironov's conversion costs 8.6 to 24.7% more, most at the strictest target, where its toll weighs most.



the optimistic PLD already gives 8.2594. At small targets the PLD's fixed width kh is a large fraction of ε: at $\varepsilon = 1$ it brackets σ only to $[3.654, 3.988]$, so refine h.

35.4. Where each accountant is tight

The ranking changes with the mechanism, by the principle above: an accountant is tight when its summary describes the mechanism's privacy loss exactly.

For the plain Gaussian mechanism, with no subsampling, the composed pair is exactly Gaussian, and GDP describes it with one number, μ, whose squares add. GDP is then exact. A team that releases 100 noisy counts at $\sigma = 10$ has exactly $\mu = 1$ and $\varepsilon = 4.3772$; an old RDP accountant with Mironov's conversion reports 5.2985, 21.0% too high, and the improved conversion 4.7285, because RDP keeps this curve exactly but still pays in the conversion.

For pure-DP steps such as randomized response, the privacy loss takes only a few values, so the PLD needs no rounding and is exact. For 100 steps of randomized response at $\varepsilon_0 = 0.1$, the exact PLD gives 4.3068, RDP 4.6154, zero-concentrated differential privacy (zCDP) 4.7263, advanced composition 5.8502 and basic composition 10.0000: each summary that keeps less pays more.

For DP-SGD, subsampling makes each step a mixture that no closed form describes exactly, and only the PLD is both certified and tight. zCDP is the wrong tool here: its single line $\rho\alpha$ must lie above the RDP curve at every order, and at high orders the subsampled curve's slope approaches the unsubsampled one, so the line cannot express the gain from subsampling. When the noise, the stopping time or per-record budgets are chosen during training, the per-step pair is no longer fixed in advance as the PLD accountant assumes; an RDP filter at a fixed order loses nothing to that adaptivity (Adaptive composition in the full edition).

Table 35.3.: Which accountant to use for which mechanism.

Mechanism	Use	Avoid
Gaussian, no subsampling	GDP, or zCDP converted exactly	Per-step (ε, δ) composition
Poisson-subsampled Gaussian (DP-SGD)	Pessimistic PLD, optimistic value as error bar	zCDP; the CLT as a guarantee
Pure-DP steps (randomized response, Laplace, exponential mechanism)	PLD (exact for finite outputs), or RDP with the mechanism's own curve	Advanced composition when a tighter accountant is available
Adaptive noise, stopping or per-record budgets	RDP filter at a fixed order	Plugging realized ε_t into a fixed-parameter theorem
Local randomizers behind a shuffler	Exact computation or a numeric bound (Amplification by shuffling and by iteration in the full edition)	The Poisson-subsampling accountant

Mechanism	Use	Avoid
Only the last iterate released	Amplification by iteration, for convex smooth losses with no released checkpoints	Any hidden-state claim for non-convex training

35.5. Matching the accountant to the code

An accountant takes q, σ and k and trusts that the code does what its theorem assumes. A mismatch makes the number a statement about a different algorithm.

The sampling scheme is the most expensive mismatch, for a mechanical reason. With fixed-size random batches, adding a record must push another one out, so the batch sum can move by up to $2C$ instead of C, and such a run is covered by the Poisson accountant only at noise multiplier $\sigma/2$ (Lebeda et al. 2024). With shuffle and partition, each record is used exactly once per epoch, in a batch fixed by the shuffle, and no amplification theorem holds for every dataset. What remains certain is one unamplified Gaussian step per epoch, for zero-out neighbours (one record's gradient replaced by zero).

In a code review: a script uses a data loader with `shuffle=True` and `batch_size=500` on 50,000 records for 100 epochs, and calls an RDP accountant with `sample_rate=0.01`, which reports 6.71. That sampler is shuffle and partition, so what holds is 100 unamplified steps at $\sigma = 1$: $\mu = 10$ GDP and $\varepsilon = 91.8173$, or 284.39 under replace-one, where the sum moves by $2C$. Fixed-size random batches, accounted correctly, would give 47.42 (RDP) and 43.42 (PLD). Switching to Poisson sampling makes the PLD's 6.24 honest. The factor of 6.96 from fixed-size batches dwarfs the factor of at most 1.19 between RDP and PLD at 10,000 steps: check the sampler before arguing about accountants.

Other assumptions fail quietly. If you tune 10 configurations on the private data and release the best, which one is best depends on all 10 runs, so the release costs all 10 composed: $\varepsilon = 27.23$ with RDP. The guarantee protects whatever differs between neighbours, so example-level DP protects each example, not a person who contributed many. And the theorems assume exact real-valued Gaussian noise; floating-point sampling can break a mechanism that is correct on paper (Discrete and floating-point-safe mechanisms).

Table 35.4.: A checklist for matching the accountant to the training code.

Item	The accountant assumes	Check in the code	If it does not match
Sampling scheme	Poisson sampling at rate q	Each record included independently, variable batch sizes?	Fixed-size random batches: account with noise multiplier $\sigma/2$. Shuffle and partition: report the guarantee without amplification, for zero-out neighbours
Neighbouring relation	Add/remove one record	Add/remove, replace-one or zero-out?	Under Poisson sampling, replace-one costs more per step: about twice the add/remove cost for a pure-DP step with small ε
Privacy unit	One example	Can one person contribute several examples?	Clip and sample per user, or pay group privacy
Hyperparameter tuning	Not counted	Was anything chosen from results on the private data?	Account for the tuning runs, or tune on public data
Adaptive parameters	σ, q, k fixed in advance	Early stopping, noise schedules, per-record budgets?	Use a filter
Implementation	Exact Gaussian noise, exact clipping	Floating-point noise, accumulation across devices	Outside the accountant: verify the implementation

35.6. What to report

An ε without its assumptions cannot be checked or compared. NIST SP 800-226 (2025) and Ponomareva et al. (2023) ask for the setting (who is trusted), what is covered (one run, or tuning too), the unit of privacy, the neighbouring relation, the accountant and whether its assumptions hold, and, for DP-SGD, everything needed to recompute ε: sampling scheme, q, k, σ, C, and the accountant's version and discretization.

Choose δ with its meaning in mind: it is the probability with which the promise may fail. A mechanism that publishes one randomly chosen record in full meets a δ of about $1/n$ with $\varepsilon = 0$, while exposing someone completely. So δ must be much smaller than $1/n$ for n units of privacy (The definition of differential privacy): Ponomareva et al. suggest $n^{-1.1}$, and NIST

gives $1/n^2$ as a common recommendation and calls values above 10^{-5} suspicious. For $n = 50{,}000$, $1/n = 2.0 \times 10^{-5}$, so the customary $\delta = 10^{-5}$ is only half of it.

Report ε at several δ, not one pair. Many different mechanisms share a single (ε, δ) pair and behave differently at smaller δ, so a reader cannot convert your pair to the δ they care about. One composed PLD gives the whole profile cheaply (Table 35.5). For the same reason, give the parameters behind any conversion: the RDP accountant's is its curve, $\varepsilon(4.1) = 3.7336$ at the best order 4.1, which the improved conversion turns into 6.7127.

Table 35.5.: The running example's privacy profile: ε at several δ for $k = 10{,}000$, $q = 0.01$, $\sigma = 1$, $n = 50{,}000$.

δ	PLD upper	PLD lower	RDP, improved
2.0×10^{-5} $(1/n)$	6.0068	5.9068	6.4884
1.0×10^{-5}	6.2377	6.1377	6.7127
6.8×10^{-6} $(n^{-1.1})$	6.3641	6.2641	6.8353
1.0×10^{-6}	6.9574	6.8574	7.4140
4.0×10^{-10} $(1/n^2)$	9.0502	8.9378	9.4540

A filled-in statement:

> **Privacy guarantee.** The model was trained with DP-SGD in the central model: the training server is trusted to run the code below, and the guarantee covers every intermediate noisy gradient, hence every checkpoint and the final model. **Unit of privacy:** one training example; each person contributes at most one example. **Neighbouring relation:** add or remove one example. **Training:** Poisson sampling with rate $q = 0.01$ ($n = 50{,}000$ examples, expected batch size 500), $k = 10{,}000$ steps (100 epochs), per-example clipping norm $C = 1$, Gaussian noise multiplier $\sigma = 1$. **Accountant:** privacy loss distributions, pessimistic discretization on the grid $h = 10^{-5}$, both add and remove directions. **Guarantee:** $(6.24, 10^{-5})$-DP and $(9.06, 4.0 \times 10^{-10})$-DP. The optimistic discretization gives 6.1377 and 8.9378, so the accountant's own error is at most 0.1000 and 0.1124. For comparison, the RDP accountant with the improved conversion gives 6.7127 at $\delta = 10^{-5}$. **Not covered:** hyperparameters were tuned on a public proxy dataset; no tuning on the private data was performed. **If the sampling assumption failed** (for example, shuffled fixed-size batches), the guarantee without amplification is $(91.82, 10^{-5})$-DP for zero-out neighbours.

Three habits in it follow from what an upper bound is. It is rounded up: 9.0502 becomes 9.06, because 9.05 would be a claim the accountant has not certified. The optimistic value is its error bar, which an RDP number cannot have, since RDP has no matching lower bound. And where an assumption is in doubt, the statement gives the fallback guarantee that does not rely on it.

35.7. Checking an accountant

Test an accountant on cases whose answers you know. With $q = 1$ there is no subsampling, so k steps are exactly $\sqrt{k}/\sigma$-GDP, and every upper bound must lie above that closed form. For $\sigma = 1$

and $k = 10$ the exact ε is 17.8566, and the RDP accountant gives 19.0536, above it as it must be. The CLT formula at $q = 1$ gives $\mu = 4.1452$ instead of 3.1623 and $\varepsilon = 25.5853$: it fails, because its derivation assumes small q. The PLD bracket for the plain Gaussian must contain the same closed form, and it does: $[17.8516, 17.8616]$ at $h = 10^{-3}$.

Two properties hold for every correct accountant, so a violation is a bug, not slack. The orderings of Section 35.2 must hold at every k, with no reported upper bound below the optimistic PLD. And ε must never decrease with k, because what the first k steps release is part of what $k + 1$ steps release.

Then compare against an independent library: Google's `dp_accounting` (RDP and PLD), Opacus (RDP, GDP, and a privacy loss random variable (PRV) accountant, its default), or Microsoft's `prv_accountant`. When two RDP accountants disagree, the size of the gap says why: the improved conversion beats Mironov's by $\frac{\ln \alpha}{\alpha - 1} + \ln \frac{\alpha}{\alpha - 1}$ at order α (0.3612 at order 10); on the running example it is what separates Mironov's 7.43 from the improved 6.71. Certified PLD and PRV brackets from different libraries each contain the truth, so they must overlap; if they do not, one has a bug.

35.8. Code

The snippet is a small RDP accountant for DP-SGD (exact subsampled-Gaussian RDP at integer orders, improved conversion), the GDP closed form used by the checks, and the noise calibration of Equation 35.1.

```
from math import e, erfc, exp, lgamma, log, sqrt


def normal_cdf(x: float) -> float:
    return 0.5 * erfc(-x / sqrt(2))


def gdp_epsilon(mu: float, delta: float) -> float:
    """Exact epsilon at delta of a mu-GDP mechanism, by bisection on its delta(eps) curve."""

    def gdp_delta(eps: float) -> float:
        return normal_cdf(mu / 2 - eps / mu) - exp(eps) * normal_cdf(-mu / 2 - eps / mu)

    lo, hi = 0.0, 1000.0
    for _ in range(200):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if gdp_delta(mid) > delta else (lo, mid)
    return hi


def subsampled_gaussian_rdp(alpha: int, q: float, sigma: float) -> float:
    """Exact RDP at an integer order of one Poisson-subsampled Gaussian step."""
    if q == 1.0:
```

```
        return alpha / (2 * sigma**2)
    terms = [  # log of each term of the binomial sum, combined with log-sum-exp
        lgamma(alpha + 1) - lgamma(j + 1) - lgamma(alpha - j + 1)
        + (alpha - j) * log(1 - q) + j * log(q) + (j * j - j) / (2 * sigma**2)
        for j in range(alpha + 1)
    ]
    top = max(terms)
    return (top + log(sum(exp(t - top) for t in terms))) / (alpha - 1)


def rdp_epsilon(q: float, sigma: float, steps: int, delta: float) -> float:
    """Upper bound on DP-SGD's epsilon: RDP on integer orders 2..64, improved conversion."""
    return min(
        steps * subsampled_gaussian_rdp(a, q, sigma) + log((a - 1) / a) - (log(delta) + log(
        for a in range(2, 65)
    )


def sigma_for_target(eps_target: float, q: float, steps: int, delta: float) -> float:
    """Smallest noise multiplier meeting the target, by bisection; returns the end that was
    lo, hi = 0.3, 50.0
    while hi - lo > 1e-4:
        mid = (lo + hi) / 2
        lo, hi = (lo, mid) if rdp_epsilon(q, mid, steps, delta) <= eps_target else (mid, hi)
    return hi


q, sigma, steps, delta = 0.01, 1.0, 10_000, 1e-5
print(f"{rdp_epsilon(q, sigma, steps, delta):.4f}")  # 6.7194
print(f"{gdp_epsilon(q * sqrt(steps * (e - 1)), delta):.4f}")  # 6.0071
print(f"{sigma_for_target(3.0, q, steps, delta):.3f}")  # 1.665
print(f"{gdp_epsilon(sqrt(10) / sigma, delta):.4f}")  # 17.8566
print(f"{rdp_epsilon(1.0, sigma, 10, delta):.4f}")  # 19.8017
print(f"{gdp_epsilon(sqrt(10 * (e - 1)), delta):.4f}")  # 25.5853
print(f"{gdp_epsilon(sqrt(100) / sigma, delta):.4f}")  # 91.8173
```

The first three lines are the running example: the RDP bound, the CLT value (below the PLD lower bound, so not a guarantee) and the noise for $\varepsilon = 3$. The rest are checks: the exact $q = 1$ value, the RDP bound above it, the CLT failing it, and the shuffle-and-partition fallback. Integer orders give 6.7194, 1.665 and 19.8017 where the full edition's fractional orders give 6.7127, 1.662 and 19.0536: a coarser grid costs tightness, never validity. The full edition's script, choosing_an_accountant.py, runs every accountant, PLD included, in about two and a half minutes:

```
uv run python code/part04-privacy-accounting/choosing_an_accountant.py
```

35.9. Cheat sheet

Table 35.6.: The accountants on the running example ($q = 0.01$, $\sigma = 1$, $\delta = 10^{-5}$), what kind of statement each makes, and when to use it.

Accountant	Kind	ε at $k = 10{,}000$	σ for $\varepsilon = 3$	When to use it
Per step + advanced composition	Upper bound	24379.55	10.505	Never for DP-SGD
RDP, Mironov conversion	Upper bound	7.4287	1.854	Never: the improved conversion is free
RDP, improved conversion	Upper bound	6.7127	1.662	Quick bounds, adaptive filters, many heterogeneous mechanisms
PLD, pessimistic	Upper bound, within kh of the truth	6.2377	1.584	The number to report
PLD, optimistic	Lower bound	6.1377	1.547	The error bar, and the referee for other accountants
GDP CLT	Approximation	6.0071	1.549	Intuition only; under-reports here

Rules of thumb

- Report the pessimistic PLD with the optimistic value beside it, and calibrate σ with the accountant you will report. Never report a CLT number as a guarantee.
- Check the sampling scheme first: fixed-size or shuffled batches accounted as Poisson is the most expensive error in practice.
- Report ε at several δ, including one well below $1/n$, with everything needed to recompute it.

35.10. Further reading

- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): matching accounting to the data pipeline, setting δ, and what to report.
- Near, Darais, Lefkovitz & Howarth, "Guidelines for Evaluating Differential Privacy Guarantees" (NIST SP 800-226, 2025): what an evaluator will ask of your claim.

- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled Mechanisms under Composition" (IEEE SaTML 2025): fixed-size batches versus Poisson sampling.
- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021): the PRV accountant with certified error bounds, and where the CLT under-reports.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

Part V.

Differential privacy in machine learning

The previous parts built the guarantee and the tools to account for it. This part trains models with it, and asks the questions that decide whether a private model is worth deploying: how do you train with differential privacy (DP), how much accuracy does it cost, which knobs move that cost, and how do you check that the trained model is as private as the accountant claims? The answers are mostly about one algorithm, stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, called DP-SGD, and about everything around it.

Read this part if you train, review or sign off on a model that claims an ε. It assumes the Gaussian mechanism and clipping from Differential privacy foundations, and amplification by subsampling and the accountant for DP-SGD from Privacy accounting. Each chapter restates what it uses.

The common thread

A private training algorithm has two halves. The **privacy half** says which quantity has bounded sensitivity, meaning how far one record can move it, and how much noise covers that move; it is a theorem, and the accountants of Part 4 turn it into a number. The **utility half** says how much the noise and the bounding cost in accuracy; it is mostly measured, rarely proved. Every chapter moves one of the two, and keeping them apart is the most useful habit here: a choice that only affects accuracy can be tuned freely, one that touches the privacy half must be accounted for.

The first two chapters build the algorithm. Private empirical risk minimization in the full edition compares three places to put the noise, on the final weights, inside the objective and on every gradient, and shows that only noise on the gradients survives non-convex models, because it needs nothing but a bound on each record's gradient. DP-SGD in the full edition supplies that bound by clipping, adds random minibatches and an accountant, and works out what each knob does. Every later chapter uses it, so these two come first.

The next three ask what else spends the budget, or could spend it better. Hyperparameter tuning in the full edition shows that picking the best of several private runs is itself a computation on the data, and how to pay for it honestly. Mechanisms built on the follow the regularized leader (FTRL) algorithm, DP-FTRL and matrix factorization in the full edition, replace the random batches that DP-SGD's amplification relies on with correlated noise, for settings where sampling cannot be trusted or controlled. Private prediction in the full edition privatizes answers instead of weights, which can be cheaper when you only need a limited number of predictions.

The next three change the setting rather than the algorithm. Public data and private fine-tuning in the full edition explains why a public pretrained model helps so much: the noise grows with the number of parameters you train, so fine-tuning fewer of them can be more accurate. User-level differential privacy in the full edition changes the unit of protection from one record to one person, which is usually what you meant to promise. Disparate impact in the full edition asks who pays for the noise and the clipping, and finds that small groups and hard examples pay most.

The last two close the loop between theory and practice. Choosing ε in the full edition turns a budget into what it means for an attacker and asks how to justify one. Auditing private training in the full edition runs attacks with planted records against a trained model and turns their success into an empirical lower bound on ε, which catches implementations that do not deliver what the accountant says. It comes last because it checks what the others claim.

The running examples

Two models recur. The first is ridge-regularized logistic regression in ten dimensions on synthetic data, where the exact minimizer can be computed, so every private method can be compared with the non-private optimum. The second is a small neural network trained with DP-SGD on synthetic data, small enough that every experiment runs on a laptop in seconds, yet non-convex, so clipping and noise behave as they do in deep learning. It runs exactly the configuration whose ε Part 4 computed, so you can see what that number costs in accuracy.

How to read this part

Read the first two chapters in order. Hyperparameter tuning and DP-FTRL change how the algorithm is run and can be read in either order. The chapters on private prediction, public data, user-level privacy and disparate impact are independent of each other. Read choosing ε after them, and auditing last. The derivations are in the full edition.

What you will be able to do

By the end of this part you will be able to train a convex model privately and predict what it costs, and run DP-SGD knowing which knobs set ε and which only set accuracy. You will be able to check an implementation for the details that silently break its guarantee, account for tuning, decide when correlated noise, private prediction or a public pretrained model is the better route, and state the privacy unit you protect. And you will be able to say who pays for the noise, choose an ε you can defend, and audit a trained model to test whether it is as private as claimed.

36. Private empirical risk minimization

36.1. Why this matters

Most of machine learning is **empirical risk minimization (ERM)**: choose the weights w that minimize the average loss on the training data, usually plus a ridge penalty $\frac{\lambda}{2}\|w\|^2$. The trained weights are a function of every record, and Membership inference showed that they give some of those records away. Differential privacy (DP) promises that the distribution of whatever you release changes by at most a factor e^{ε}, plus a slack δ, when one record changes. To keep that promise for a trained model, randomness has to enter somewhere between the data and the released weights, and there are three natural places: on the final weights (**output perturbation**), inside the objective (**objective perturbation**), or on every gradient during training (**gradient perturbation**).

The one idea that organizes all three is that **privacy needs a bound on how far one record can move what you release**. That bound is the **sensitivity**, and the noise is sized to hide a move of that length. The three methods differ only in which quantity they bound, and each step back toward the data needs fewer assumptions. Output perturbation needs an exact, unique minimizer of a strongly convex objective. Objective perturbation needs that and a loss with bounded curvature. Gradient perturbation needs only that each record's gradient is bounded, with no convexity and no exact solution, which is why it is the one that survives in deep learning, as stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD in the full edition.

Throughout, neighbouring datasets differ by **replacing** one record, so both have n records; L bounds the length of every record's loss gradient. The running example is ridge-regularized logistic regression in $d = 10$ dimensions with $\lambda = 0.01$, features on the unit sphere (so $L = 1$), and a target of $\varepsilon = 1$, with $\delta = 10^{-5}$ for the methods that need one. Quality is measured by the **excess risk**, how much worse the private weights fit the training objective than the exact minimizer.

36.2. Output perturbation

The simplest private trainer treats training as a black box: compute the exact minimizer w^*, then add noise. For that you need the minimizer's sensitivity. Replacing one record changes the average loss by one term in and one out, so the objective's gradient changes by at most $2L/n$ anywhere. The ridge makes the objective **λ-strongly convex**: its gradient grows by at least λ per unit of distance, so a point where the gradient is g long lies within g/λ of the minimizer. At the new minimizer, the old objective's gradient is at most $2L/n$, hence the minimizer moves by at most

$$\|w^*(D) - w^*(D')\| \le \frac{2L}{\lambda n}. \tag{36.1}$$

Add Gaussian noise to that, and you have the Gaussian mechanism (The Laplace and Gaussian mechanisms). Its privacy depends only on the ratio μ of the sensitivity to the noise standard deviation, and the exact calibration (Balle & Wang 2018) gives the largest admissible ratio $\mu(\varepsilon, \delta)$. At $\varepsilon = 1$ and $\delta = 10^{-5}$, $\mu = 0.2681$, so the noise must be $\sigma = 3.7306$ times the sensitivity. The classic formula $\sqrt{2\ln(1.25/\delta)}/\varepsilon$ asks for 4.8448 times, 30% more noise for the same guarantee, so always calibrate exactly.

The noise must hide the worst case, not the actual move. In one dimension with $n = 20$ records and $\lambda = 0.5$, replacing one record moves the minimizer by 0.0832, but the bound is $2/(0.5 \times 20) = 0.2$, and the noise is sized for 0.2 (Figure 36.1).

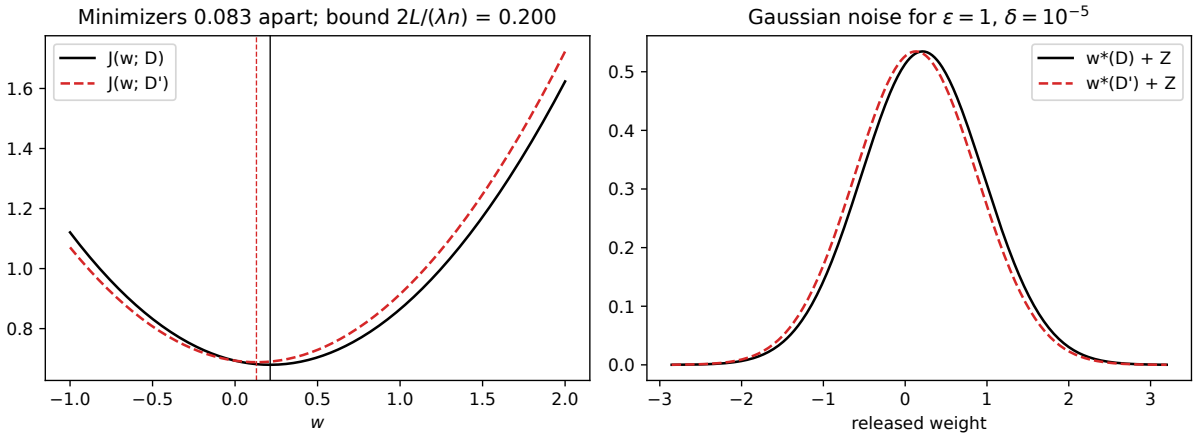


Figure 36.1.: Left: two neighbouring regularized logistic objectives in one dimension ($n = 20$, $\lambda = 0.5$), their minimizers (vertical lines) 0.083 apart, below the bound $2L/(\lambda n) = 0.2$. Right: the released weight's density under each dataset with Gaussian noise for $\varepsilon = 1$, $\delta = 10^{-5}$; the noise must hide a shift as large as the bound, not the actual shift.

What does the noise cost? Near the minimizer the objective is a bowl whose curvature is the Hessian H, so a random displacement Z raises it by about $\frac{1}{2}Z^\top HZ$, and for Gaussian noise that averages to

$$\mathbb{E}[J(\tilde{w}) - J(w^*)] \approx \frac{\sigma^2}{2} \operatorname{tr} H. \tag{36.2}$$

Notice the mismatch: the noise is calibrated to the curvature the ridge guarantees, λ, but the damage is weighed by the actual curvature H, data included. A small ridge buys a better non-private fit and costs a lot of noise, since $\sigma \propto 1/\lambda$. In the running example at $n = 10{,}000$, the sensitivity is $2/(0.01 \times 10{,}000) = 0.02$, so $\sigma = 0.07461$ per coordinate, next to a minimizer of length 2.886. The Hessian has trace 0.3129, and Equation 36.2 predicts an excess risk of 8.711×10^{-4}; 20 simulated releases average 9.274×10^{-4}. Because the noise falls like $1/n$, the excess risk falls like $1/n^2$, faster than the $1/n$ statistical error of the non-private model, so in this convex, low-dimensional setting privacy eventually costs less than sampling does.

The weak point is that the sensitivity bound holds for the *exact* minimizer. If the optimizer stops at a point whose gradient has length g, each output may sit g/λ from its exact minimizer, and the sensitivity becomes $2L/(\lambda n) + 2g/\lambda$. In the running example a tolerance of $g = 10^{-3}$, which looks tight, gives a sensitivity of 0.22 instead of 0.02: eleven times the noise and roughly 121

times the excess risk. To keep the extra term small you need g well below L/n, which shrinks with the dataset. And the stopping rule must be a gradient check, not a step count: the privacy proof needs the bound on every pair of datasets, and "50 Newton steps" only guarantees it on datasets where 50 steps happen to be enough.

36.3. Objective perturbation

Output perturbation wastes the data's curvature: it pays for the worst case λ and is judged on the actual H. Chaudhuri, Monteleoni & Sarwate (2011) put the noise inside the optimization instead. They add a random linear term $\frac{1}{n}\langle b, w\rangle$ to the objective and minimize exactly. A linear term acts like a constant force pushing the minimizer, and all of the objective's curvature resists it, so where the data add curvature the minimizer moves less.

The privacy argument is subtler, because the output is no longer "something fixed plus noise", and its result says where the budget goes. Each noise vector b produces exactly one output, so the output's density is the noise density at the matching b times a stretching factor from the change of variables. Both factors differ between neighbours, and each consumes part of ε. The stretching factor is governed by how much one record can change the objective's curvature, which needs a loss of the form $g(y\langle w, x\rangle)$ with $|g'| \le 1$, curvature $0 \le g'' \le c$ and $\|x\| \le 1$; logistic regression qualifies with $c = \frac{1}{4}$. What is left for the noise is

$$\varepsilon' = \varepsilon - 2\ln\Big(1 + \frac{c}{n\lambda}\Big), \tag{36.3}$$

and b is drawn with density proportional to $e^{-\varepsilon'\|b\|/2}$. The result is pure ε-DP. The stretching share shrinks as $n\lambda$ grows: at $n = 10{,}000$ in the running example it is 0.0050, so $\varepsilon' = 0.9950$, while at $n = 100$ the noise would get only $\varepsilon' = 0.5537$. If ε' comes out non-positive, the method raises the ridge until the stretching takes only half the budget.

The benefit shows in the excess risk. To first order the minimizer moves by $-H^{-1}b/n$, so the damage is weighed by H^{-1} rather than H. If every eigenvalue of H is close to some h, objective perturbation beats output perturbation by about $(h/\lambda)^2$. When the ridge supplies all the curvature the two are the same; when the data add curvature, objective perturbation wins quadratically. In the running example the curvature lies between 0.0263 and 0.0330, about three times the ridge, and the measured excess risk at $n = 10{,}000$ is 8.156×10^{-5}, against 8.321×10^{-4} for pure-DP output perturbation at the same ε: ten times better, close to the $3^2 = 9$ the rule of thumb predicts.

The analysis is safe, not tight. In one dimension the output densities can be computed exactly, and over 500 random datasets of 5 records with $\lambda = 0.1$ and $\varepsilon = 1$, the largest log ratio is 0.3779 (Figure 36.2). The noise part never exceeds 0.0992 of its budget of 0.1891, and the stretching part never exceeds 0.3395 of its budget of 0.8109, because the worst cases each factor assumes cannot happen together for logistic loss. The bound has to cover every dataset, so you cannot spend that slack, but for logistic loss the guarantee is conservative.

The price is in the requirements: exact minimization (the change of variables needs a unique minimizer), a twice-differentiable loss (a support vector machine's hinge loss must be smoothed first) and a known curvature bound. Kifer, Smith & Thakurta (2012) extend it to Gaussian noise and constraints, and Iyengar et al. (2019) give a version that tolerates approximate solvers by adding a little output noise on top.

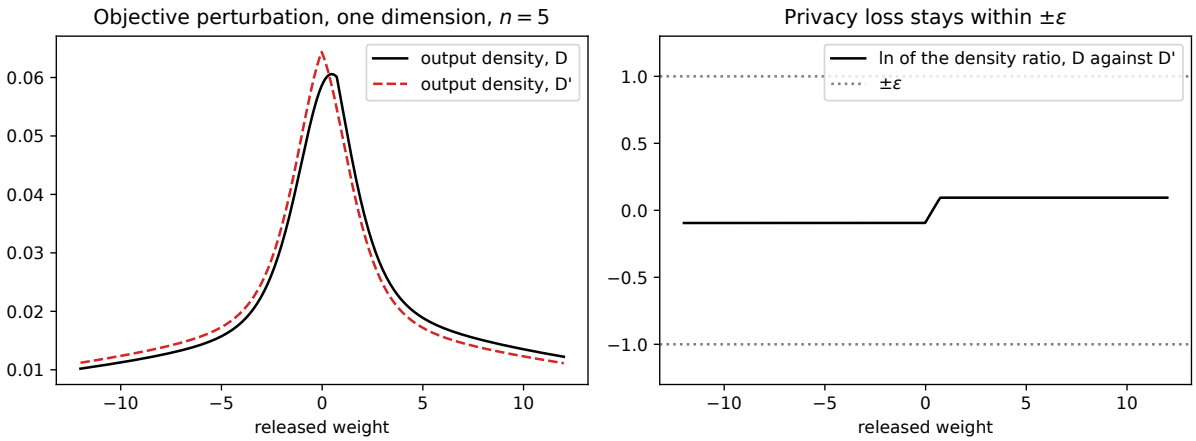


Figure 36.2.: Objective perturbation in one dimension, $n = 5$, $\lambda = 0.1$, $\varepsilon = 1$. Left: the exact densities of the released weight under a dataset and its neighbour, computed by change of variables. Right: their log ratio, the privacy loss, which stays well inside $\pm\varepsilon$.

36.4. Gradient perturbation: noisy gradient descent

Real training is iterative and stops early. Gradient perturbation, analysed by Song, Chaudhuri & Sarwate (2013) and Bassily, Smith & Thakurta (2014), makes each step of the optimizer private and composes the steps. Its simplest form is full-batch gradient descent with Gaussian noise added to the mean gradient at each of T steps.

Each step is a Gaussian mechanism. Given the current weights, which are public at that moment because they were computed from earlier noisy outputs, replacing one record moves the mean gradient by at most $2L/n$, whatever the weights are. Gaussian steps compose exactly: T steps, each with ratio μ_{step}, are as private as one Gaussian mechanism with ratio $\sqrt{T}\,\mu_{\text{step}}$ (f-DP and Gaussian differential privacy). Setting that equal to $\mu(\varepsilon, \delta)$ gives the noise on the mean gradient,

$$\sigma = \frac{2L}{n} \cdot \frac{\sqrt{T}}{\mu(\varepsilon, \delta)}. \tag{36.4}$$

Releasing every iterate is covered. Nothing in this argument used convexity, a minimizer or smoothness: only the gradient bound at the points the algorithm visits.

The consequence for practice is that **the step count is a privacy parameter**. Every step spends budget, so the per-step noise grows like $\sqrt{T}$. For a strongly convex objective, the excess risk is an optimization error that shrinks geometrically with T plus a noise floor, roughly $d\sigma^2/(4\beta)$ for a step size $1/\beta$ set by the largest curvature β, that grows in proportion to T. Too few steps leave optimization error; too many drown the solution in noise (Figure 36.3).

You cannot pick T by trying several and keeping the best validation score for free: that choice is itself a computation on the data (Hyperparameter tuning in the full edition). One honest option is a rule that uses only public quantities, the ridge, a curvature bound, the dimension, n and the budget, and minimizes the predicted excess risk. For $n = 10{,}000$ it picks $T = 218$, with per-step noise $\sigma = 0.01102$ and a predicted excess risk of 1.301×10^{-3}; simulated runs average 1.384×10^{-3}.

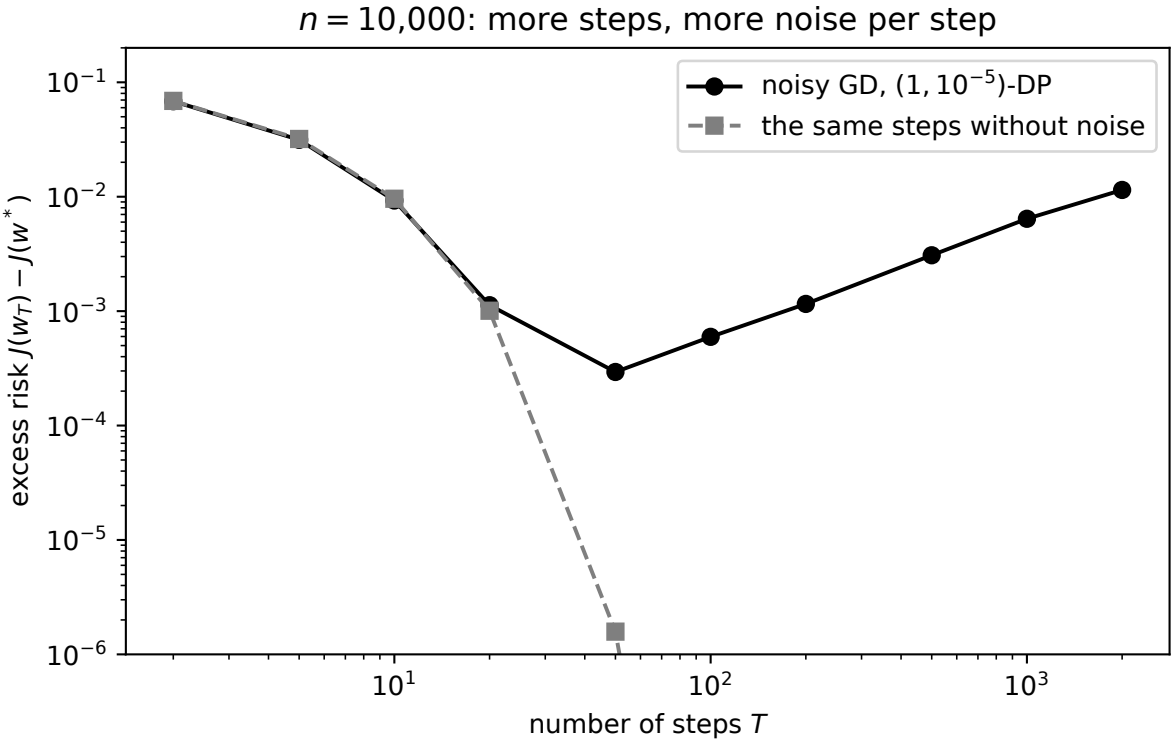


Figure 36.3.: Excess risk of noisy gradient descent at $n = 10{,}000$, $(\varepsilon, \delta) = (1, 10^{-5})$, against the number of steps T, averaged over datasets, with the same steps without noise (dashed). The noiseless error falls geometrically and reaches machine precision; the noisy run is limited by noise that grows with T, so its error is smallest near $T = 50$.

The rule is pessimistic, because it only knows that the curvature is at least λ. With the data's actual curvature the best T is near 50, with 2.875×10^{-4}, five times less, but you may only use that knowledge if you obtained it privately or from similar public data.

36.5. Which to use

Each method is private by its own argument, so the choice is about accuracy. Theory says that for strongly convex problems the best achievable excess risk under (ε, δ)-DP falls like

$$\frac{L^2\, d\, \ln(1/\delta)}{\lambda\, (n\varepsilon)^2}, \tag{36.5}$$

up to logarithmic factors in n (Bassily, Smith & Thakurta 2014), and that a version of noisy gradient descent attains it; no method can do better, by the fingerprinting lower bounds of Lower bounds. Two practical readings follow. The cost grows with the dimension d, so more parameters cost more privacy. And n and ε appear only as the product $n\varepsilon$, because every sensitivity is proportional to $1/n$ and every noise to sensitivity over ε: to keep the same accuracy at half the budget, you need twice the data.

All methods share the rate; the constants differ, and the constant is what you choose by. Table 36.1 measures them on the running example.

Table 36.1.: Mean excess risk over 20 datasets of the running example ($d = 10$, $\lambda = 0.01$, $\varepsilon = 1$, $\delta = 10^{-5}$ for the Gaussian methods), and the slope of a straight-line fit of log excess risk against $\log n$.

n	Output, Gaussian	Output, pure	Objective, pure	Noisy gradient descent (public T)
1,000	8.170×10^{-2}	7.060×10^{-2}	1.009×10^{-2}	6.558×10^{-2}
10,000	9.274×10^{-4}	8.321×10^{-4}	8.156×10^{-5}	1.384×10^{-3}
100,000	8.543×10^{-6}	6.379×10^{-6}	8.798×10^{-7}	1.621×10^{-5}
Slope in $\log n$	-2.01	-2.00	-2.01	-1.79

The first three columns fall like $1/n^2$; noisy gradient descent falls slightly slower, because its step count grows with $\ln n$. Objective perturbation is about ten times better than output perturbation throughout (Figure 36.4), and noisy gradient descent with the public step rule is the worst for $n \geq 10{,}000$; tuned privately to $T \approx 50$ it would sit between the two perturbation methods.

The pure-DP column is slightly better than the Gaussian one here only because $d = 10$ is small. The pure-DP noise has a length that grows faster with the dimension than the Gaussian's: at $d = 10$ their mean squared lengths are $110\,\Delta^2$ against $139.2\,\Delta^2$, but at $d = 100$ they are $10{,}100\,\Delta^2$ against $1{,}392\,\Delta^2$. Pure DP costs a factor that grows like d, the price of $\delta = 0$, which Lower bounds shows is unavoidable; in high dimension a small δ is much cheaper.

Excess risk is a fine measure, but you care about predictions, and those are forgiving. On held-out data the non-private model has accuracy 0.747 at $n = 1{,}000$; output perturbation drops it to

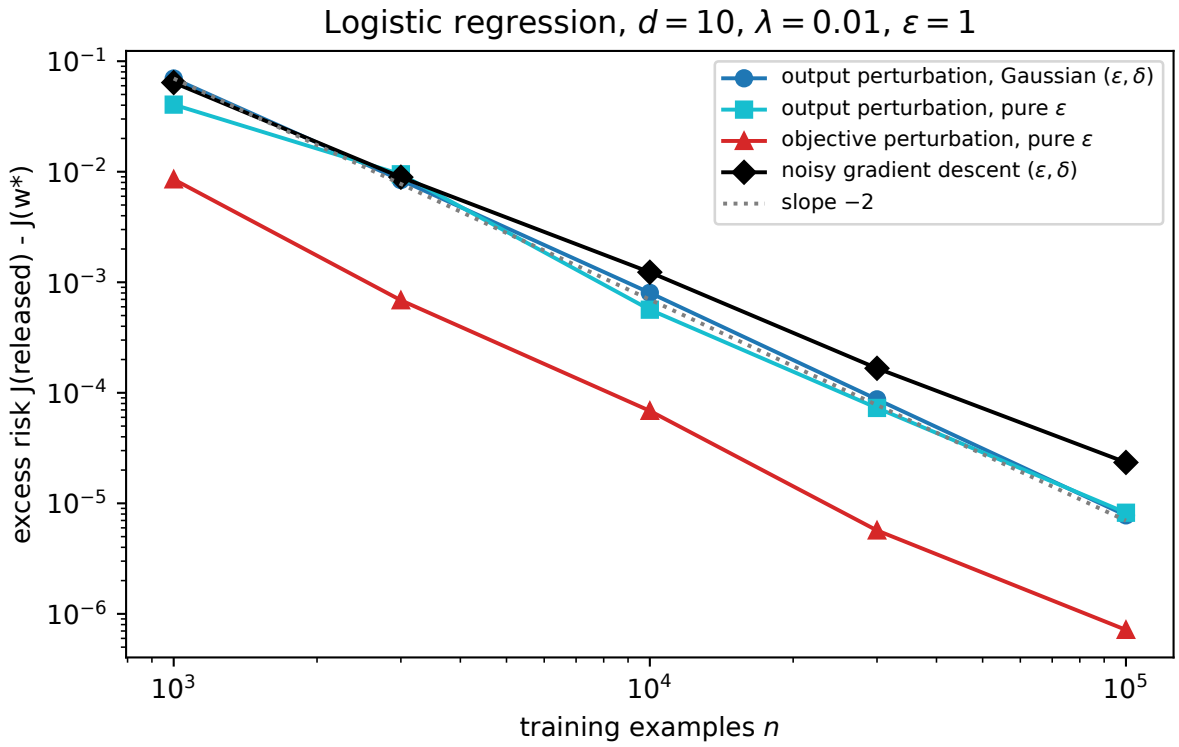


Figure 36.4.: Excess risk of the four private releases against the dataset size on the running example, log-log, with a line of slope -2. All fall like $1/n^2$; objective perturbation is about ten times better than output perturbation throughout.

0.681, noisy gradient descent to 0.695 and objective perturbation only to 0.736. By $n = 10{,}000$ all four are within 0.002 of the non-private 0.753. A low-dimensional convex model on ten thousand records can be made private almost for free.

36.6. When the model is not convex

A neural network's loss is not convex, and that breaks the first two methods outright. Without strong convexity the minimizer need not be unique or move continuously with the data. Take a one-dimensional loss with two wells near ± 1, where the empirical minimizer sits in the well favoured by the majority label: with 501 labels of one kind and 500 of the other, changing one label flips the majority and moves the minimizer by 2.0010. No $1/n$ appears, so output perturbation would need $\sigma = 7.465$ at $\varepsilon = 1$, more than three times the distance between the wells, whatever the dataset size. Objective perturbation fails at its first step, since the map from noise to output is no longer invertible.

Gradient perturbation keeps its guarantee, because its argument only used the gradient bound at the visited points. What it loses is the accuracy guarantee: whether it learns anything becomes an empirical question. For a neural network there is no bound L either, since gradients can be arbitrarily long, but you can force one by **clipping**: rescale each record's gradient to length at most C before averaging, as Beyond global sensitivity clips a value before averaging it. The mean of clipped gradients moves by at most $2C/n$, and the privacy argument goes through with $L = C$. Clipping has a cost of its own: the mean of clipped gradients is no longer the gradient of the objective, so the algorithm optimizes something slightly different.

Full-batch steps are also too slow for large datasets. Using a random minibatch at each step makes steps cheap and, by amplification by subsampling, more private, since each record is used only with a small probability. Clipping, noise, minibatches and a subsampled-Gaussian accountant together are DP-SGD in the full edition.

36.7. Code

The calibrations of this chapter take a few lines: find the largest admissible Gaussian ratio by bisection on the exact δ, then size the noise for each method.

```
from math import erfc, exp, log, sqrt


def phi(x: float) -> float:
    """Standard normal cumulative distribution function."""
    return 0.5 * erfc(-x / sqrt(2))


def gaussian_delta(eps: float, mu: float) -> float:
    """Exact delta of a Gaussian mechanism with sensitivity-to-noise ratio mu."""
    return phi(mu / 2 - eps / mu) - exp(eps) * phi(-mu / 2 - eps / mu)
```

```
def gaussian_mu(eps: float, delta: float) -> float:
    """Largest ratio mu for which the Gaussian mechanism is (eps, delta)-DP."""
    lo, hi = 1e-6, 50.0
    for _ in range(200):
        mid = (lo + hi) / 2
        lo, hi = (lo, mid) if gaussian_delta(eps, mid) > delta else (mid, hi)
    return lo


mu = gaussian_mu(1.0, 1e-5)
print(f"{mu:.4f} {1 / mu:.4f}")  # 0.2681 3.7306
print(f"{sqrt(2 * log(1.25 / 1e-5)):.4f}")  # 4.8448
# Output perturbation: sensitivity 2L/(lambda n), L = 1, lambda = 0.01, n = 10,000.
print(f"{2 / (0.01 * 10_000) / mu:.5f}")  # 0.07461
# Objective perturbation: the share of eps left for the noise, c = 1/4.
print(f"{1 - 2 * log(1 + 0.25 / (10_000 * 0.01)):.4f}")  # 0.9950
# Noisy gradient descent: noise on the mean gradient for T = 218 steps.
print(f"{2 / 10_000 * sqrt(218) / mu:.5f}")  # 0.01102
```

The full edition's script implements the three methods for logistic regression with exact Newton solves, computes the exact one-dimensional densities of objective perturbation, and runs the comparison:

```
uv run python code/part05-dp-ml/dp_erm.py
```

36.8. Cheat sheet

Table 36.2.: The methods of this chapter, for replace-one neighbours, with $\mu = \mu(\varepsilon, \delta)$ the largest admissible Gaussian ratio.

Method	Noise	Needs	Guarantee
Output perturbation	$w^* + \mathcal{N}(0, \sigma^2 I)$, $\sigma = 2L/(\lambda n \mu)$	Strong convexity; exact minimizer, or sensitivity $2L/(\lambda n) + 2g/\lambda$ at gradient tolerance g	(ε, δ)-DP
Objective perturbation	$\arg\min J + \langle b, w\rangle/n$, density $\propto e^{-\varepsilon' \|b\|/2}$, $\varepsilon' = \varepsilon - 2\ln(1 + c/(n\lambda))$	Loss $g(y\langle w, x\rangle)$ with $\|g'\| \le 1$, $0 \le g'' \le c$, $\|x\| \le 1$; exact minimizer	ε-DP
Noisy gradient descent	$\sigma = (2L/n)\sqrt{T}/\mu$ on the mean gradient	Per-record gradients of length at most L, or clipped	(ε, δ)-DP, all iterates

Method	Noise	Needs	Guarantee
Exact Gaussian ratio	$\mu = 0.2681$, $\sigma = 3.7306\,\Delta$ at $(1, 10^{-5})$	—	Classic formula gives $4.8448\,\Delta$
Excess risk	Output $\approx \frac{\sigma^2}{2}\operatorname{tr} H$; objective better by $\approx (h/\lambda)^2$	Strong convexity	All fall like $1/(n\varepsilon)^2$

Halving ε costs as much as halving the data. Stop output and objective perturbation on a gradient tolerance well below L/n, never on a step count. Count the steps of noisy gradient descent as part of the budget, and choose them from public quantities or pay for choosing them. Without convexity only gradient perturbation is private, and clipping supplies the bound it needs.

36.9. Further reading

- Chaudhuri, Monteleoni & Sarwate, "Differentially Private Empirical Risk Minimization" (JMLR 2011): output and objective perturbation with their excess-risk bounds.
- Bassily, Smith & Thakurta, "Private Empirical Risk Minimization: Efficient Algorithms and Tight Error Bounds" (FOCS 2014): noisy gradient methods that attain the optimal rates, and the matching lower bounds.
- Iyengar, Near, Song, Thakkar, Thakurta & Wang, "Towards Practical Differentially Private Convex Optimization" (IEEE S&P 2019): objective perturbation that tolerates inexact solvers, and a benchmark of the three methods.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

37. DP-SGD

37.1. Why this matters

Private empirical risk minimization in the full edition ended with one method left standing for non-convex models: gradient descent with Gaussian noise on every step, made safe for any model by clipping each record's gradient. Two more changes make it practical for differential privacy (DP) at scale. Each step uses a random minibatch, which makes steps cheap and, by amplification, more private. And an accountant composes thousands of such steps into one (ε, δ). The result is stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, **DP-SGD** (Abadi et al. 2016), which nearly every private deep-learning system runs. Privacy accounting computed its ε without running it; this chapter runs it, and asks what the noise and the clipping cost and how to set the knobs that control them.

The one idea is that **the privacy analysis never looks inside the model**. Each step releases a noisy sum of clipped per-example gradients. Clipping caps each record's contribution at length C, so adding or removing one record moves the sum by at most C, whatever the network, the loss or the current weights, and the proof stops there. That splits the knobs in two. The noise multiplier σ, the sampling rate q and the number of steps T set ε, and nothing else does. The clipping norm C and the step size η do not change ε at all; they decide how much of the gradient survives and how far the noise pushes the weights. The cost of privacy then comes in two forms: a **bias** from clipping, which changes the direction of the average gradient, and a **variance** from the noise.

The running example is a network with one hidden layer of 64 units, $p = 3{,}329$ weights, trained on $n = 10{,}000$ synthetic records and tested on 20,000 more, with $q = 0.01$ (expected batch $B = 100$), $C = 0.5$, $\eta = 0.5$, $\sigma = 1$ and $T = 10{,}000$ steps: Part 4's running example, now actually trained.

37.2. The algorithm and what its guarantee assumes

Each step of DP-SGD does five things, and each is an assumption of the proof:

1. **Sample**: put each record in the batch independently with probability q (Poisson sampling).
2. **Per-example gradients**: compute each sampled record's gradient g_i separately.
3. **Clip**: rescale each to length at most C, $\bar{g}_i = g_i \min(1, C/\|g_i\|)$.
4. **Noisy sum**: add Gaussian noise with standard deviation σC to each coordinate of $\sum_i \bar{g}_i$.
5. **Step**: divide by the expected batch size $B = qn$, a public constant, and take a gradient step.

Why is this private? With the weights and the other records fixed, the record in question adds at most one term of length C to the sum, so a step on which it is drawn is a Gaussian mechanism whose shift is one noise multiplier's worth of noise, ratio $1/\sigma$; the clipping norm cancels. Under

Poisson sampling it is drawn with probability q independently of everyone else, so its effect on a step is a mixture of "shifted" and "not there", which is the subsampled Gaussian of Amplification by subsampling. Dividing by a public B is post-processing. The steps depend on earlier outputs only through the weights, and Rényi differential privacy (RDP) curves add even for such adaptive sequences (Rényi DP), so the accountant adds the T per-step curves and converts once (From Rényi DP to (ε, δ)). Neighbours differ by adding or removing a record, which is the relation Poisson sampling pairs with. For the running example the accountant gives $\varepsilon = 6.71$ at $\delta = 10^{-5}$, attained at order $\alpha = 4.1$, and Table 37.1 gives the noise multiplier other budgets need with the same q and T.

Table 37.1.: The noise multiplier that DP-SGD with $q = 0.01$ and $T = 10{,}000$ steps needs for each budget, from the RDP accountant.

ε at $\delta = 10^{-5}$	0.5	1	2	4	6.71	8
Noise multiplier σ	7.72	4.13	2.28	1.36	1	0.917

Because the proof rests on these steps, an implementation can break the guarantee by changing a detail, with no visible symptom. Batch normalization computes statistics over the batch, so one record changes every other record's gradient and the sum can move by much more than C; private models use normalizations that act on one example at a time, such as group or layer normalization. Choosing C from the current batch, say as its median gradient norm, makes the noise scale depend on the data; fix C in advance, from public data or a private estimate. Shuffling the data and cutting it into fixed batches is a different mechanism, which can be much less private than the Poisson accountant reports. Dividing by the actual batch size $|S_t|$ reveals how many records were drawn, which changes when one is added, so it is not post-processing; divide by B. The accountant must see every step, including the steps of runs discarded during tuning (Hyperparameter tuning in the full edition). And the guarantee is per record: a person with many records needs user-level privacy in the full edition, and preprocessing that uses the data, such as normalizing features by their mean, must itself be private or use public statistics.

How large is the noise in practice? The averaged update carries noise with standard deviation $\sigma C/B = 1 \times 0.5/100 = 0.005$ per coordinate. A Gaussian vector of p such coordinates has length close to $0.005\sqrt{p}$, which is about 0.288 for 3,329 weights. The averaged clipped gradient is 0.070 long at the first step and between 0.015 and 0.027 from step 1,000 on, so every step's noise is about ten times longer than its signal. Training works anyway because the signal points in roughly the same direction from step to step while the noise is independent: over k steps the signals add to about k times their length and the noise to about $\sqrt{k}$ times, since independent vectors add in squares. Signal overtakes noise when $k \cdot 0.026 > \sqrt{k} \cdot 0.288$, after about 123 steps. The final model reaches test accuracy 0.9589, against 0.9627 for non-private SGD.

37.3. Clipping: bias, noise and the choice of C

The clipping norm does not affect privacy, but it controls both forms of cost. Averaged over the sampling and the noise, the update direction is

$$\mathbb{E}\Big[\frac{G_t}{B}\Big] = \frac{1}{n}\sum_{i=1}^{n} \min\Big(1, \frac{C}{\|g_i\|}\Big) g_i, \tag{37.1}$$

the average gradient with each record weighted by $\min(1, C/\|g_i\|)$. Each record is in the batch with probability q and the noise has mean zero, so dividing the noisy sum by $B = qn$ cancels the q exactly; that is why the denominator is the expected batch size. The weights are where the bias lives. Records the model fits badly, such as rare examples, members of small groups or mislabelled points, have large gradients and are down-weighted, while well-fitted records keep their full weight. So DP-SGD follows the gradient of a reweighted loss whose weights change during training, and Disparate impact in the full edition measures who pays for this.

Two regimes follow from the formula. When C is below every per-example gradient norm, every gradient is clipped to exactly length C, so the clipped sum is C times a sum of unit vectors, and the noise is C times σ times a standard normal vector. The whole update is then $\eta C/B$ times something that does not depend on C: DP-SGD becomes normalized SGD plus noise, and depends on η and C only through their product ηC. Lowering C further changes nothing once η is scaled up to match, which is why practitioners often clip aggressively and tune only the step size (De et al. 2022 report that small clipping norms work well). When C is above most gradient norms, the clipped average stops growing with C but the noise $\sigma C/B$ keeps growing, so the signal-to-noise ratio falls like $1/C$. A large C is never better than the smallest C that clips almost nothing.

Table 37.2 shows both regimes on the running example at $\varepsilon = 6.71$, once with η fixed and once with ηC fixed at 0.25.

Table 37.2.: Test accuracy of the running example at $\sigma = 1$ for seven clipping norms, one run each. With ηC fixed, small clipping norms all give the same accuracy.

C	0.01	0.03	0.1	0.3	1	3	10
Fixed $\eta = 0.5$	0.9244	0.9510	0.9567	0.9607	0.9583	0.9523	0.8347
Fixed $\eta C = 0.25$	0.9627	0.9623	0.9605	0.9599	0.9590	0.9558	0.9494

With the step size fixed, a small C is too slow, because every step has length proportional to ηC, and a large one is too noisy. With ηC fixed, accuracy is flat for $C \le 0.3$, as scale invariance predicts, and falls slowly for large C. The best run, 0.9627 at $C = 0.01$, equals the non-private model.

Gradient norms also move during training, so a fixed C changes regime as you go. In the running example the median per-example gradient norm is 3.592 at the first step and 0.006 at step 1,000, because the logistic loss flattens on records classified with a wide margin; non-private training behaves the same way (Figure 37.1, right). With $C = 0.5$ every gradient is clipped at the start, and later only between 9% and 25% of the batch, the records still near the decision boundary.

37.4. Batch size at a fixed budget

Suppose the budget (ε, δ) and the number of epochs E are fixed. Each epoch is $1/q$ steps, so should you take many small steps or few large ones? Recall from Amplification by subsampling

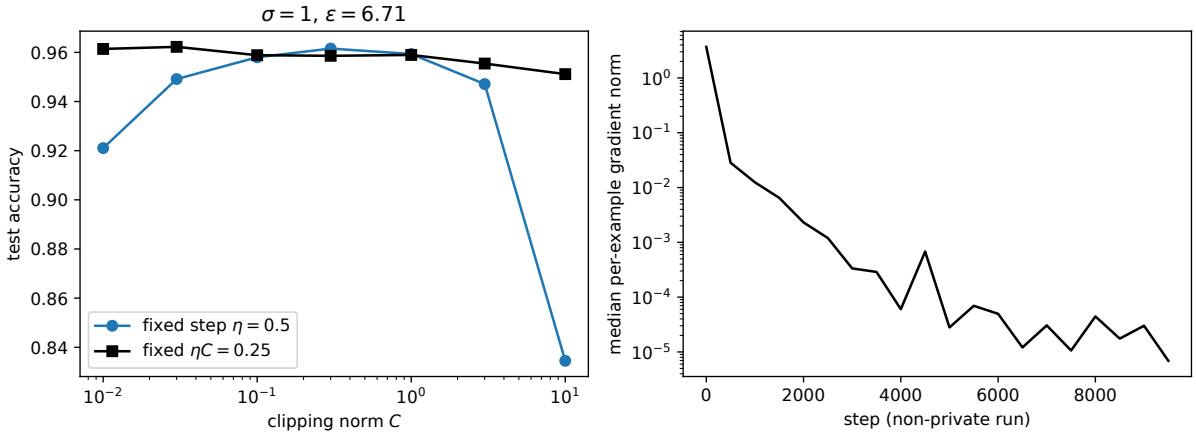


Figure 37.1.: Left: test accuracy of the running example at $\sigma = 1$ ($\varepsilon = 6.71$) against the clipping norm, with the step size fixed at $\eta = 0.5$ (circles) and with ηC fixed at 0.25 (squares). Right: the median per-example gradient norm during non-private training, on a log scale; it falls by about five orders of magnitude.

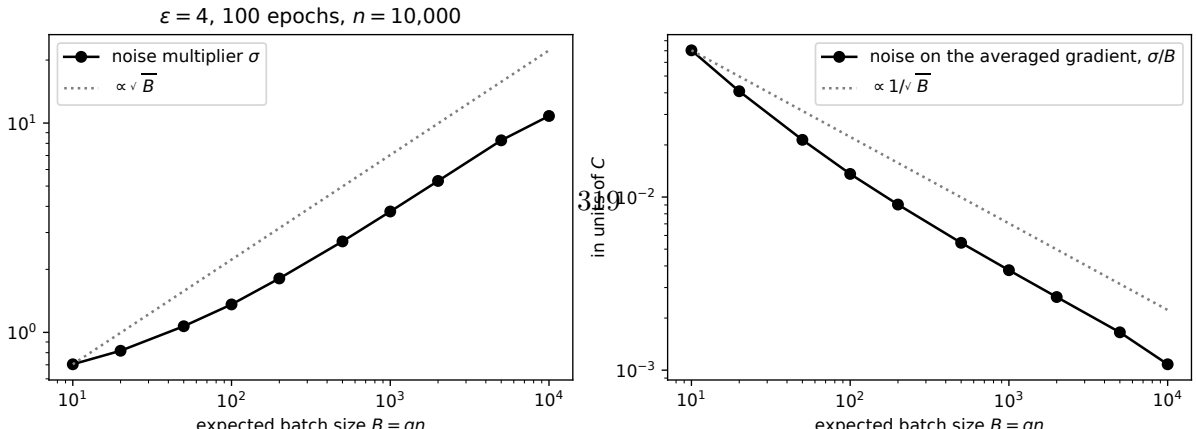
that for small q a step's RDP is about $\frac{\alpha}{2}q^2(e^{1/\sigma^2} - 1)$, close to $\alpha q^2/(2\sigma^2)$ when σ is well above 1. Over E/q steps that adds up to about $\alpha E q/(2\sigma^2)$, so ε stays fixed when q/σ^2 does: the noise multiplier must grow like $\sqrt{q}$. The noise on the averaged gradient, $\sigma C/B$ with $B = qn$, then falls like $1/\sqrt{B}$. Larger batches give cleaner gradients.

Small batches are worse than that law says. They need small σ, and when σ is near or below 1, $e^{1/\sigma^2} - 1$ is much larger than $1/\sigma^2$, so each step costs more than the approximation assumes, by the factor $\sigma^2(e^{1/\sigma^2} - 1)$. Steps with little noise are expensive per unit of signal because sampling buys little in the tail of the privacy loss. Table 37.3 fixes $\varepsilon = 4$, $\delta = 10^{-5}$ and 100 epochs and finds the noise each batch size needs.

Table 37.3.: Noise needed for $\varepsilon = 4$ at $\delta = 10^{-5}$ over 100 epochs, for four of the full edition's seven sampling rates. The noise on the averaged gradient, in units of C, falls faster than $1/\sqrt{B}$.

q	B	Steps T	σ	Noise per averaged gradient σ/B	$\sigma^2(e^{1/\sigma^2} - 1)$
0.001	10	100,000	0.704	0.0704	3.23
0.01	100	10,000	1.36	0.0136	1.33
0.1	1,000	1,000	3.78	0.00378	1.04
1	10,000	100	10.8	0.00108	1

From $B = 10$ to $B = 1{,}000$, a hundredfold increase, the noise per averaged gradient falls by a factor of 18.6, against the 10 that $1/\sqrt{B}$ predicts (Figure 37.2). The full batch needs no subsampling accountant: every record is in every step, and the 100 plain Gaussian steps compose exactly.



three step sizes goes from 0.25 at $B = 30$ to between 1 and 2 from $B = 100$ to $B = 1{,}000$, and at their best step sizes the batch sizes tried reach between 0.9544 and 0.9591, with the two largest doing best. (Picking the best step size by test accuracy is itself a leak; it is done here only to compare batch sizes.) De et al. (2022) and Anil et al. (2022) train with batches of thousands to hundreds of thousands and report that large batches are a main ingredient of accurate private models. They also parallelize well.

37.5. Per-example gradient norms without per-example gradients

Ordinary backpropagation computes the gradient of the batch's summed loss, never each record's. Forming all B per-example gradients costs B times the model's gradient memory, which is prohibitive for large models. **Ghost clipping** avoids it for dense layers. A dense layer's gradient for one record is an outer product $\delta_i a_i^\top$ of the error flowing back into the layer and the layer's input, and the length of an outer product is the product of the lengths, $\|\delta_i\| \, \|a_i\|$. So one backward pass gives every record's gradient norm, layer by layer, without forming any gradient; the clipped sum then comes from rescaling each record's error by its clipping factor and doing the usual batch product. For a sequence of T tokens, the squared norm is a sum over pairs of tokens of input dot products times error dot products, which costs two $T \times T$ Gram matrices per record. That beats forming the gradient only when $2T^2 < p_{\text{in}} p_{\text{out}}$, so long sequences in narrow layers favour explicit gradients, and Bu, Mao & Xu (2022) choose per layer.

Table 37.4.: Numbers stored to clip one dense layer, explicitly and with ghost norms.

Layer	Explicit $Bp_{\text{in}}p_{\text{out}}$	Ghost	Ratio
Running example, first layer: $B = 100$, $p = 51 \to 64$	326,400	11,500	28.4
$B = 32$, one token, $768 \to 768$	18,874,368	49,152	384.0
$B = 32$, 512 tokens, $768 \to 768$	18,874,368	16,777,216	1.1
$B = 8$, 4,096 tokens, $4{,}096 \to 4{,}096$	134,217,728	268,435,456	0.5

Privacy is untouched either way: these are exact computations of the same quantities. In the running example the ghost norms agree with explicitly formed gradients to a relative difference of 2.2×10^{-16}, rounding error.

37.6. Privacy against accuracy

Table 37.5 puts the pieces together: the running example trained at five budgets, each with the noise multiplier of Table 37.1, three runs each.

Table 37.5.: Test accuracy of the running example under DP-SGD at five budgets, with $\delta = 10^{-5}$.

	Noise multiplier	Test accuracy (mean ± standard deviation over 3 runs)
$\varepsilon = 0.5$	7.72	0.8209 ± 0.0180
$\varepsilon = 1$	4.13	0.9175 ± 0.0161
$\varepsilon = 2$	2.28	0.9441 ± 0.0051
$\varepsilon = 4$	1.36	0.9548 ± 0.0019
$\varepsilon = 8$	0.917	0.9604 ± 0.0019
Clipped, no noise	0	0.9649 ± 0.0009
Non-private SGD, 1,000 steps	—	0.9610 ± 0.0005

On this problem clipping costs nothing: without noise it reaches 0.9649, slightly above non-private SGD, since it only limits the few records with large gradients. The whole loss comes from the noise, and it is steeply non-linear in ε: from 8 to 4 the accuracy drops by half a point, from 2 to 1 by 2.7 points, and from 1 to 0.5 by almost 10 (Figure 37.3).

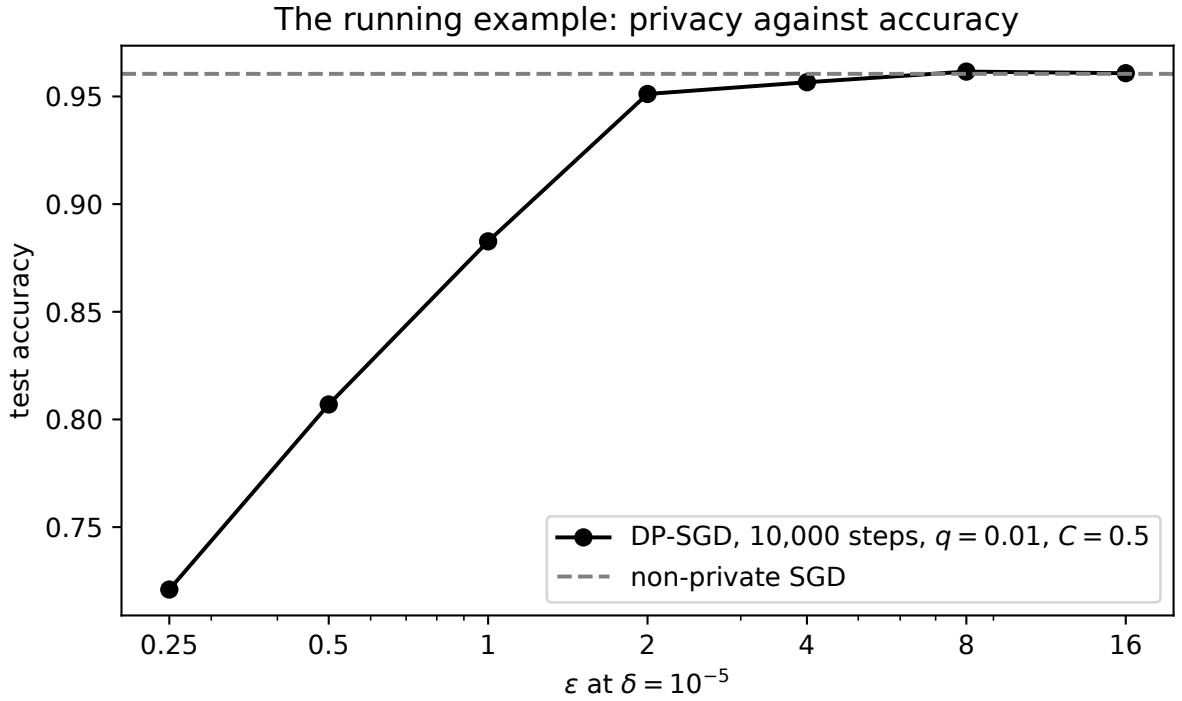


Figure 37.3.: Test accuracy of DP-SGD on the running example at $\delta = 10^{-5}$ against ε (log scale), with the non-private model dashed. Accuracy is close to non-private above $\varepsilon = 4$ and falls steeply below $\varepsilon = 1$.

The signal-against-noise argument of Section 37.2 explains the collapse. At $\varepsilon = 1$ the noise on the averaged gradient is about 1.19 long against a signal of about 0.026, so signal overtakes noise only after about 2,100 steps, and the 10,000 steps hold only a few independent useful moves. With

$\sigma = 4$ the model reaches its accuracy early and then wanders (Figure 37.4). Before promising a budget, locate where that collapse happens for your task.

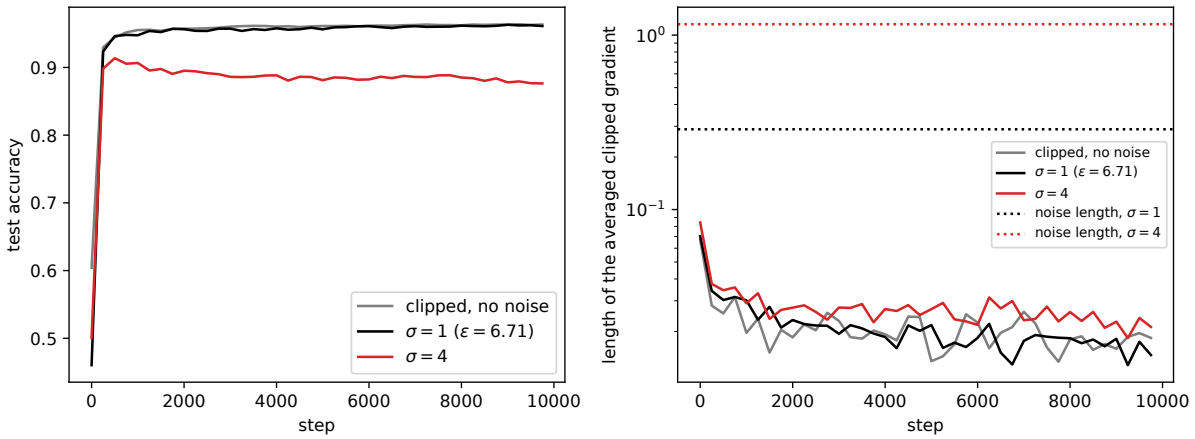


Figure 37.4.: Left: test accuracy during training with clipping alone, with $\sigma = 1$ ($\varepsilon = 6.71$) and with $\sigma = 4$. Right: the length of the averaged clipped gradient at each step (solid, log scale) against the length of the noise on it (dotted). The noise is about ten times longer than the signal at $\sigma = 1$, and about forty times at $\sigma = 4$.

Run-to-run variation grows with the noise too: the standard deviation over runs is 0.0019 at $\varepsilon = 8$ and 0.018 at $\varepsilon = 0.5$. A single private run is one sample from a wide distribution, so comparing two configurations by one run each is reliable only for differences well above that spread. For Part 4's configuration, $\varepsilon = 6.71$ costs 0.4 points of accuracy here; reaching $\varepsilon = 1$ would cost about four. Choosing ε in the full edition asks which trade is worth making, and Auditing private training in the full edition checks whether the trained model is as private as the accountant says.

37.7. Code

Before a long run, check how the noise compares with the signal. This snippet computes the noise on the averaged gradient and the number of steps after which a steady signal overtakes it, for the running example at $\sigma = 1$ and at $\sigma = 4.13$ ($\varepsilon = 1$), using the signal length of about 0.026 measured in the run:

```
from math import sqrt


def noise_length(sigma: float, clip: float, batch: float, params: int) -> float:
    """Approximate length of the noise on the averaged clipped gradient."""
    return sigma * clip / batch * sqrt(params)


def break_even_steps(noise: float, signal: float) -> float:
    """Steps after which a steady signal (adds like k) overtakes independent noise (adds lik
    return (noise / signal) ** 2
```

```
print(f"{1.0 * 0.5 / 100:.3f}")  # 0.005
print(f"{noise_length(1.0, 0.5, 100, 3329):.3f}")  # 0.288
print(f"{break_even_steps(0.288, 0.026):.0f}")  # 123
print(f"{noise_length(4.13, 0.5, 100, 3329):.2f}")  # 1.19
print(f"{break_even_steps(1.19, 0.026):.0f}")  # 2095
```

The accountant itself is in the code section of Amplification by subsampling. The full edition's script implements the network, ghost norms, DP-SGD with Poisson sampling and the accountant in NumPy, and reproduces every table of this chapter:

```
uv run python code/part05-dp-ml/dp_sgd.py
```

37.8. Cheat sheet

Table 37.6.: The results of this chapter.

Result	Formula	Conditions
Update	$w_{t+1} = w_t - \frac{\eta}{B}\big(\sum_{i\in S_t} g_i \min(1, \frac{C}{\|g_i\|}) + \mathcal{N}(0, \sigma^2 C^2 I)\big)$	Poisson batches, per-example clipping, $B = qn$
Privacy	RDP of T subsampled Gaussian steps, converted once; set by σ, q, T only	Fixed C, public B, no batch statistics, add/remove neighbours
Running example	$\varepsilon = 6.71$ at $\delta = 10^{-5}$	$q = 0.01$, $\sigma = 1$, $T = 10{,}000$
Expected update	$\frac{1}{n}\sum_i \min(1, C/\|g_i\|) g_i$	Divide by B, not $\|S_t\|$
Noise	$\sigma C/B$ per coordinate, length about $\sigma C\sqrt{p}/B$	—
Scale invariance	Depends on ηC only	Every gradient clipped
Batch size	$\sigma \propto \sqrt{q}$, noise $\propto 1/\sqrt{B}$; small batches worse	Fixed ε and epochs, small q
Ghost norms	$\|\delta_i a_i^\top\| = \|\delta_i\| \, \|a_i\|$	Dense layers
Signal against noise	Overtakes after about (noise length/signal length)^2 steps	Signal roughly constant in direction

Only σ, q and T set ε; C and η are free for accuracy. Clip small and tune ηC. Prefer large batches at a fixed number of epochs, and scale the step size up with them. Check the implementation for Poisson sampling, per-example gradients, no batch statistics, a public denominator and a fixed C.

37.9. Further reading

- Abadi, Chu, Goodfellow, McMahan, Mironov, Talwar & Zhang, "Deep Learning with Differential Privacy" (CCS 2016): DP-SGD and the moments accountant.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): the practitioner's reference for what to clip, how to tune and what to report.
- De, Berrada, Hayes, Smith & Balle, "Unlocking High-Accuracy Differentially Private Image Classification through Scale" (arXiv 2022): large batches, small clipping norms and other practices that close much of the gap.
- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022): ghost clipping for sequence models and private fine-tuning of language models.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

38. Hyperparameter tuning and private selection

38.1. Why this matters

Stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, run once with a fixed step size, clipping norm, batch size and number of steps, satisfies differential privacy (DP) with the (ε, δ) its accountant reports; this is DP-SGD in the full edition, and its guarantee covers one training run. Nobody trains once. You try a grid of step sizes, score each model on a validation set and ship the best, and every run and every score touched private data. How much does the search leak, and how do you pay for it honestly?

Releasing the best of k private runs can cost as much as releasing all k, and composition, which charges for every run, turns a grid of 10 DP-SGD runs of this part's running example from $\varepsilon = 6.71$ into 27.23. But one change to the procedure changes the answer: draw the *number* of runs at random, from a suitable distribution, before the search starts. Then the best-of release costs a constant multiple of one run plus a term that grows only with the logarithm of the expected number of runs. For 10 runs in expectation, the cost falls from 27.23 to between 10.08 and 13.67, depending on the distribution, and for 1,000 runs from 1,035.93 to 13.46.

The one idea: **the model you ship depends on every run, and a fixed number of runs lets an attacker test for the event "every run came out badly", whose likelihood ratio is the product of the per-run ratios. A random number of runs explains that event mostly by "there happened to be few runs", which does not depend on the data.**

38.2. The best of k runs costs k runs

A choice among eight step sizes carries at most three bits, so why charge for it? Because the released model is not just the choice: it is a whole private run, picked *because* of how it came out. The worst case is easy to build. Let one run be randomized response on two outcomes, "good" and "bad": on a dataset D it comes out good with probability $1/(1+e^{\varepsilon})$, on its neighbour D' with probability $e^{\varepsilon}/(1+e^{\varepsilon})$, so one run is exactly ε-DP. The best of k runs is bad only if every run is bad. The runs are independent, so the probability of that event is a product of k per-run probabilities on each dataset, and its likelihood ratio is the product of k ratios of e^{ε} each:

$$\frac{P_D[\text{best of } k \text{ is bad}]}{P_{D'}[\text{best of } k \text{ is bad}]} = e^{k\varepsilon}. \tag{38.1}$$

Here the best of k is exactly $k\varepsilon$-DP, no better than releasing all k runs. With $\varepsilon = 1$ and $k = 10$, the whole loss sits on one rare event: the privacy loss is -0.0446 with probability 0.9564 and 10 with probability 0.0436 (Figure 38.1, right). An attacker who checks whether the released model is bad gets a likelihood ratio of e^{10} whenever the test fires.

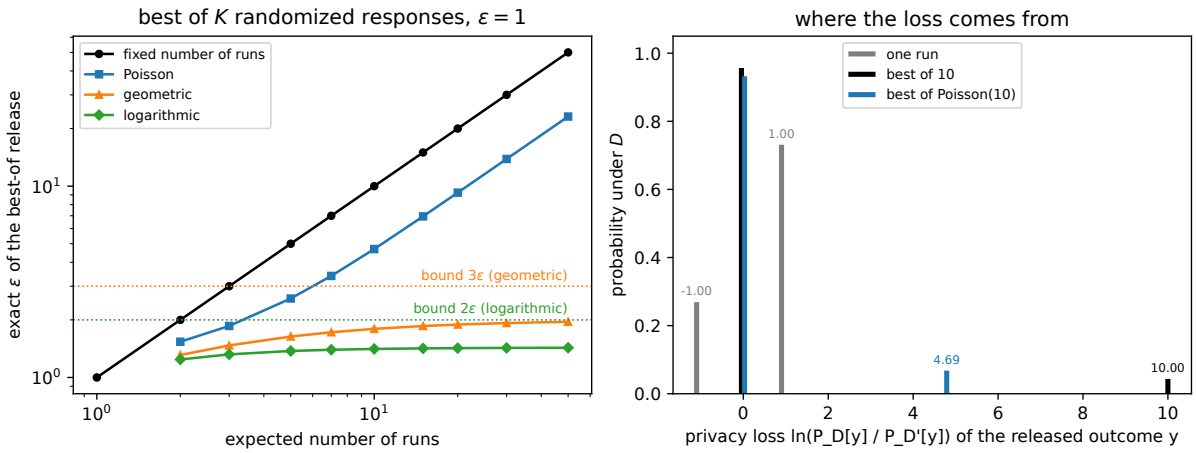


Figure 38.1.: Left: the exact ε of releasing the best of K randomized-response runs, each 1-DP, against the expected number of runs, for a fixed number of runs, a Poisson, a geometric and a logarithmic number. A fixed number grows linearly; the geometric and logarithmic numbers stay below their bounds 3ε and 2ε. Right: the privacy loss of the released outcome under D for one run, the best of 10 runs and the best of a Poisson number with mean 10; the best of 10 puts a loss of 10 on an event of probability 0.0436.

This is not a contrived corner: a DP-SGD run that sometimes diverges is a run with a "bad" outcome whose probability depends a little on the data, and best-of-k multiplies those small dependences. A particular search may leak much less, but no analysis that knows only each run's guarantee can promise it, so a cheaper guarantee needs a different procedure.

Tuning without noise and then training once privately does not help: the winning hyperparameters are a non-private function of the data, which the final run's ε does not cover. Papernot & Steinke (2022) build a model whose best regularization weight alone reveals whether a few outliers were in the training set.

The safe baseline is composition: charge every run. For DP-SGD this is simple, because the accountant charges each step the same whatever weights it is applied to, so k runs of T steps cost exactly one run of kT steps. Table 38.1 applies it to the running example ($q = 0.01$, $\sigma = 1$, 10,000 steps, one run at $\varepsilon = 6.71$, $\delta = 10^{-5}$).

Table 38.1.: Composition of k DP-SGD runs of the running example, at $\delta = 10^{-5}$.

Runs k	1	3	10	30	100
ε of k runs at $\sigma = 1$	6.71	12.73	27.23	58.25	145.17
Ratio to one run	1.00	1.90	4.06	8.68	21.63
σ per run for a total of 6.71	1	1.46	2.45	4.12	7.44

Ten runs cost 4.06 times one run, not 10, because the conversion to (ε, δ) pays its $\ln(1/\delta)$ term

once, but the cost still grows roughly linearly: from 30 to 100 runs, by a factor 2.5. Keeping the total fixed instead means $\sigma = 2.45$ per run for 10 runs, and accuracy falls steeply as noise rises. For two or three candidates composition is fine; beyond a few runs, the methods below pull away (Figure 38.2).

38.3. Random stopping

Liu & Talwar (2019) found the change that breaks the $k\varepsilon$ barrier. After each run, flip a coin and stop with probability γ; then release the best run seen. The number of runs is geometric with mean $1/\gamma$, and if each run is ε-DP, the release is 3ε-DP whatever γ is.

Why a constant, and why 3? Write p for the probability that one run produces outcome y, $p_>$ for the probability that it produces something strictly better, and $p_\geq = p_> + p$. Averaging the best-of-k probabilities over the geometric number of runs gives

$$P[\text{release} = y] = \frac{\gamma\, p}{(\gamma + (1-\gamma)p_>)(\gamma + (1-\gamma)p_\geq)}. \tag{38.2}$$

Each of p, $p_>$ and $p_\geq$ is the probability of an event about one run, so each moves by at most a factor e^ε between neighbours. The constant γ, which does not depend on the data, cannot amplify those moves. Three factors, each moving by at most e^ε: the ratio is at most $e^{3\varepsilon}$. Compare the fixed-k case, where the ratio was a k-th power. The event "every run was bad" is now mostly the event "the coin stopped early".

The 3 can be approached: on a three-outcome mechanism that is exactly 1-DP, the release reaches 2.9987. With two outcomes, as in randomized response, nothing is better than the best outcome, so only two factors move: at $\varepsilon = 1$ the exact value rises from 1.7957 at $\gamma = 0.1$ to 1.9977 at $\gamma = 0.001$ (Figure 38.1, left).

The price is paid in runs. The release misses the top fraction π of a single run's outcomes with probability at most γ/π. With a grid of 8 and $\gamma = 1/8$, a mean of 8 runs, the chance of never trying the best eighth is 0.4667; it takes $\gamma = 1/80$, a mean of 80 runs, to bring it to 0.0805. The geometric distribution is spread out, stopping after one run with probability 1/8 at $\gamma = 1/8$, and that spread is what buys the privacy.

38.4. Rényi bounds for DP-SGD

DP-SGD is not usefully ε-DP for any small ε. Its guarantee is a curve in Rényi differential privacy (RDP): a bound $\varepsilon(\lambda)$ on the Rényi divergence of order λ between its outputs on neighbours, where curves add under composition and are converted to (ε, δ) once, at the best order (Rényi differential privacy). Papernot & Steinke (2022) extended random stopping to RDP and to a family of distributions for the number of runs K, the truncated negative binomial with a shape parameter $\eta > -1$. The geometric distribution is $\eta = 1$; $\eta = 0$ is the **logarithmic distribution**. For a run with RDP curve $\varepsilon(\lambda)$, the best of K runs has curve at most

$$\varepsilon(\lambda) + (1+\eta)\Big(1 - \frac{1}{\hat\lambda}\Big)\varepsilon(\hat\lambda) + \frac{(1+\eta)\ln(1/\gamma)}{\hat\lambda} + \frac{\ln \mathbb{E}[K]}{\lambda - 1}, \tag{38.3}$$

for any second order $\hat{\lambda} \geq 1$ you choose, where γ is the distribution's other parameter, set by the mean.

Each term pays for something. The first is one run, the floor, since the release is a run. The last is the number of runs, and it is logarithmic: ten times more runs adds $\ln 10/(\lambda - 1)$. The middle terms pay for the tail of K, how well an attacker can tell "many runs, all bad" from "few runs", as a constant multiple of the run's own privacy at another order. For a pure ε-DP run the bound becomes $(2 + \eta)\varepsilon$: 3ε for the geometric, as before, and 2ε for the logarithmic.

For a Poisson number of runs with mean μ, the bound is

$$\varepsilon(\lambda) + \mu\hat{\delta} + \frac{\ln \mu}{\lambda - 1}, \tag{38.4}$$

where the run must be $(\hat{\varepsilon}, \hat{\delta})$-DP with $e^{\hat{\varepsilon}} \leq 1 + 1/(\lambda - 1)$. The tail term $\mu\hat{\delta}$ is linear in μ, but it is multiplied by the run's $\hat{\delta}$ at a small $\hat{\varepsilon}$, so it is small only when one run is very private.

On the running example (Table 38.2, Figure 38.2), the logarithmic distribution costs 10.08 for 10 runs in expectation, 1.5 times one run, and going to 1,000 runs adds only 3.38. The Poisson bound starts well but grows nearly linearly, because one run at $\varepsilon = 6.71$ is far from private enough at the small $\hat{\varepsilon}$ the condition demands.

Table 38.2.: The ε at $\delta = 10^{-5}$ of tuning with DP-SGD runs of the running example (one run: $\varepsilon = 6.71$), by the number of runs and how it is chosen.

Expected runs	3	10	30	100	1,000
Composition	12.73	27.23	58.25	145.17	1035.93
Poisson	8.93	13.67	26.43	67.77	302.57
Geometric ($\eta = 1$)	9.22	11.31	12.85	14.32	16.76
$\eta = 0.5$	9.03	10.75	11.98	13.15	15.09
Logarithmic ($\eta = 0$)	8.75	10.08	11.04	11.95	13.46

Smaller η gives a cheaper bound but a more skewed K, which often runs only a few candidates. With mean 10, the logarithmic distribution stops after one run with probability 0.2692, against 0.1000 for the geometric; the Poisson is concentrated near its mean, with $P[K \leq 1] = 4.99 \times 10^{-4}$ (Figure 38.3). Section 38.6 shows what that costs.

These are upper bounds for any run with the given curve, so they can be loose for a particular one: on randomized response with $\varepsilon = 0.5$ and a logarithmic number of runs with mean 10, at the orders where conversions usually land, Equation 38.3 was two to three times above the exact divergence. They also have conditions. Each run must draw its candidate independently from a distribution fixed in advance, typically uniform on the grid: this is random search. Adaptive methods that choose the next candidate from the scores so far, such as Bayesian optimization, are not covered and need adaptive composition. And the score that ranks the runs must itself be part of each run's private output.

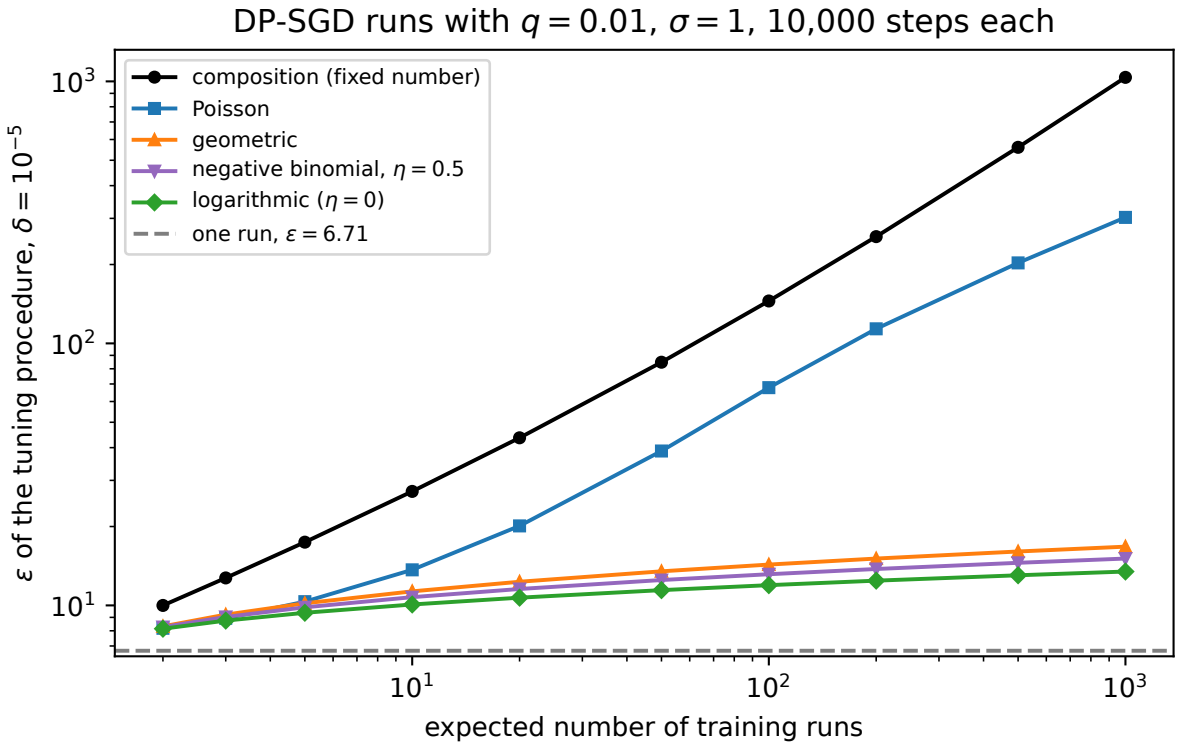


Figure 38.2.: The ε at $\delta = 10^{-5}$ of releasing the best of several DP-SGD runs of the running example ($q = 0.01$, $\sigma = 1$, 10,000 steps, one run $\varepsilon = 6.71$), against the expected number of runs, by composition, by the Poisson bound and by the truncated negative binomial bound with $\eta = 1$ (geometric), 0.5 and 0 (logarithmic). Both axes are logarithmic.

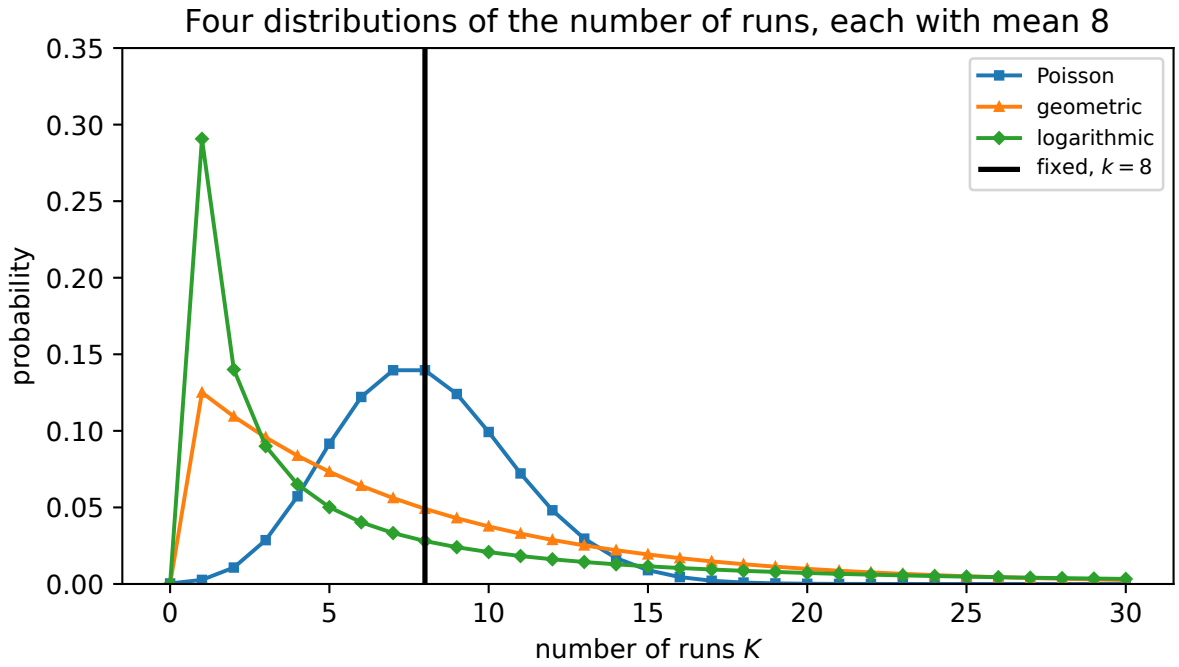


Figure 38.3.: Four distributions of the number of runs, each with mean 8: Poisson, geometric, logarithmic and a fixed 8. The logarithmic distribution stops after one run with probability 0.29.

38.5. The validation score, and selecting among fixed candidates

That last condition is easy to miss: ranked by an exact validation accuracy, the validation records are not protected at all. Score each run instead by its count of correct predictions on a held-out validation set, plus Gaussian noise of standard deviation σ_{val}. The count is the right score because adding or removing one validation record changes it by at most 1, while an accuracy divides by the number of records, itself private. The noisy count is a Gaussian mechanism with RDP curve $\lambda/(2\sigma_{\text{val}}^2)$.

It is also usually free. Each record sits in exactly one of the training and validation sets, and a pair of neighbours differs in one record, so the procedure's cost is the larger of the cost in training records and the cost in validation records, not their sum. With $\sigma_{\text{val}} = 20$ correct predictions, one noisy count alone has $\varepsilon = 0.182$ and eight composed 0.546. In the experiment below, eight DP-SGD runs at $\sigma = 2.07$ cost 1.994 together, and adding the eight noisy counts leaves the total at 1.994, because the counts' curve stays below the training curve at the orders that matter.

When the candidates do not depend on the private training data, such as public pretrained checkpoints, only the choice is private. That is a pure selection problem, and the **exponential mechanism** of Selection solves it: pick candidate r with probability proportional to $e^{\varepsilon u(r)/2}$, where $u(r)$ is its count of correct predictions. It is ε-DP, and its expected shortfall from the best count among k candidates is at most $2(\ln k + 1)/\varepsilon$ correct predictions, logarithmic in k with no linear privacy cost. Table 38.3 runs it on eight trained models whose counts out of 5,000 range from 4,123 to 4,694, the best beating the second by 13.

Table 38.3.: The exponential mechanism selecting among eight trained models by their correct predictions on 5,000 private validation records.

ε_{sel}	P[best]	Expected gap (correct predictions)	Accuracy gap
0.01	0.2478	87.62	1.752%
0.1	0.6484	5.61	0.112%
1	0.9985	0.02	0.000%

At $\varepsilon_{\text{sel}} = 0.1$ the choice costs about a tenth of a point of accuracy. Selection is cheap; training is not, and for candidates trained on private data you still pay for every run by one of the methods above.

38.6. Tuning the running example

To see the trade-offs together, tune the step size of the running example (the network of DP-SGD in the full edition, $n = 10{,}000$ training records, $q = 0.01$, $C = 0.5$) over eight values from 0.0625 to 8. To keep the script fast, each run is 1,000 steps rather than 10,000. Scores are noisy counts on 5,000 validation records with $\sigma_{\text{val}} = 20$. The whole procedure gets $\varepsilon = 2$ at $\delta = 10^{-5}$, and each strategy picks the noise multiplier σ that keeps it within that budget (Table 38.4).

Table 38.4.: Tuning the step size of the running example (1,000 steps per run) at a total budget of $\varepsilon = 2$, $\delta = 10^{-5}$. Test accuracy of the released model, mean and standard deviation over repetitions of the whole procedure, estimated by resampling from three runs per step size; runs on private data is the average number of training runs.

Strategy	σ	ε of one run	Total ε	Test accuracy	Runs on private data
No tuning: step size 0.5 from DP-SGD in the full edition	1.03	1.97	1.97	0.9509 ± 0.0015	1
Grid of 8, single-run ε reported	1.03	1.97	5.60	0.9479 ± 0.0030	8
Tune on proxy data, then one run	1.03	1.97	1.97	0.9487 ± 0.0025	1
Grid of 8, composition	2.07	0.66	1.99	0.9430 ± 0.0022	8

Strategy	σ	ε of one run	Total ε	Test accuracy	Runs on private data
Random search, Poisson, mean 8	1.53	0.98	1.99	0.9433 ± 0.0060	8.01
Random search, logarithmic, mean 8	1.36	1.17	2.00	0.9372 ± 0.0200	7.95

The honest searches cost about a point: 0.9430 for composition and 0.9433 for Poisson random search, against 0.9509 without tuning. The second row is the common reporting bug: run the grid, report the single run's 1.97, spend up to 5.60. A paper reporting "0.948 at $\varepsilon = 1.97$" looks better than one reporting "0.943 at $\varepsilon = 1.99$", yet in Table 38.4 they are the same algorithm, accounted differently.

Even misreported, the grid search does worse than not tuning. Step size 0.5 is the best of the grid, and the noisy count sometimes picks a neighbour; it is the best because DP-SGD in the full edition chose its settings with knowledge of this very task, a leak of exactly this chapter's kind. On a new task you would have no such guess.

The random-K strategies show the trade-off of Section 38.4. Poisson random search allows $\sigma = 1.53$ instead of 2.07, but misses any given step size with probability 0.3679 and varies more, so at this small grid the effects roughly cancel. The logarithmic distribution allows the least noise, $\sigma = 1.36$, but with mean 8 it stops after one run with probability 0.2907 and ships a random step size, so it is the least accurate and most variable. Papernot & Steinke also found the Poisson distribution best in their small search. Tuning on a proxy task, built the same way with a different teacher, costs one private run and reaches 0.9487.

Figure 38.4 repeats the comparison at $\varepsilon = 1$ and 4, with noisier points.

38.7. What to do and what to report

The cheapest fix is upstream. In Table 38.4, the two strategies that spend the whole budget on one private run beat every honest search. That will not always hold, since a poor guess can be far worse than a noisy search, but it says where to start: get hyperparameters from public data, proxy data or similar tasks, and shrink what is left. Below the typical gradient norm only the product of step size and clipping norm matters (DP-SGD in the full edition), so one of the two need not be searched.

Search the rest privately, and fix the search before touching the data: candidates, distribution of the number of runs, score, validation set. The random-K theorems are about a procedure, and an engineer who looks at the first results and decides to "try a few more" has changed it. Then account for the whole search: compose for a few candidates; for more, use random search with Equation 38.4 when each run is already very private and the search is moderate, or Equation 38.3

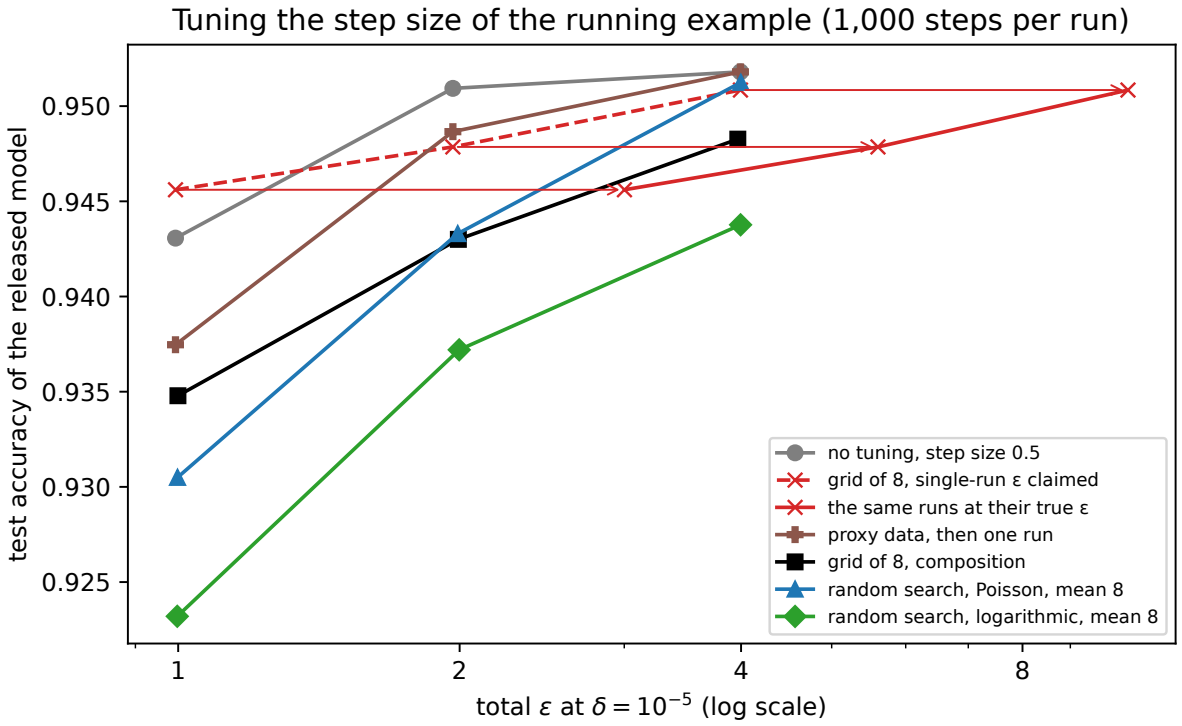


Figure 38.4.: Test accuracy of the model released by each tuning strategy against the total ε at $\delta = 10^{-5}$ it costs (1,000 DP-SGD steps per run). The red dashed line places the grid search at the single-run ε that the reporting bug claims, and the arrows move each point to its honest cost by composition. Points at $\varepsilon = 2$ use three runs per step size; those at 1 and 4 one run.

for many runs. Every run on the private data counts, including crashed runs, debugging runs and evaluations used for early stopping.

Finally, report two numbers: the ε of the released run, and the ε of the whole procedure with how it was computed. For the Poisson row of Table 38.4 a complete report reads: "DP-SGD with Poisson sampling rate 0.01, clipping norm 0.5, noise multiplier 1.53, 1,000 steps; one run satisfies $(0.98, 10^{-5})$-DP. The step size was selected by random search over $\{0.0625, 0.125, \dots, 8\}$ with a Poisson number of runs with mean 8, scored by correct predictions on a disjoint validation set of 5,000 records with Gaussian noise of standard deviation 20; the whole procedure satisfies $(1.99, 10^{-5})$-DP by the Rényi bound of Papernot & Steinke (2022, Theorem 6), add/remove neighbours, record level."

An audit in the full edition will not catch a missing tuning charge: it tests the released model, not how it was chosen.

38.8. Code

The best-of-K formula is one line once you have the **generating function** $f(x) = \mathbb{E}[x^K]$ of the number of runs: the best of K runs is bad exactly when all runs are bad, with probability $f(P[\text{bad}])$. This snippet computes the exact ε for randomized response with a fixed and a geometric number of runs:

```
from collections.abc import Callable
from math import exp, log


def best_of_rr_epsilon(eps: float, f: Callable[[float], float]) -> float:
    """Exact epsilon of releasing the best of K randomized-response runs, f(x) = E[x^K]."""
    bad_d = exp(eps) / (1 + exp(eps))  # one run comes out bad on D
    bad_d2 = 1 / (1 + exp(eps))  # ... and on its neighbour D'
    # The best is bad only if every run is bad: probability f(P[bad]).
    out_d = [f(bad_d), 1 - f(bad_d)]
    out_d2 = [f(bad_d2), 1 - f(bad_d2)]
    return max(abs(log(a / b)) for a, b in zip(out_d, out_d2))


def fixed(k: int) -> Callable[[float], float]:
    return lambda x: x**k


def geometric(gamma: float) -> Callable[[float], float]:
    """Stop after each run with probability gamma; the mean number of runs is 1/gamma."""
    return lambda x: gamma * x / (1 - (1 - gamma) * x)


print(f"{best_of_rr_epsilon(1.0, fixed(10)):.4f}")  # 10.0000
```

```
print(f"{best_of_rr_epsilon(1.0, geometric(0.1)):.4f}")  # 1.7957
print(f"{best_of_rr_epsilon(1.0, geometric(0.001)):.4f}")  # 1.9977
```

For DP-SGD, the full edition's script, hyperparameter_tuning.py, has the bounds: `tuned_epsilon(q, sigma, steps, strategy)` gives the total ε of a search, with `strategy` one of `("fixed", k)`, `("poisson", mean)` or `("tnb<η>", mean)`, and `sigma_for_tuning(q, steps, eps, strategy)` the noise multiplier for a budget. Running it reproduces every number in this chapter (two to four minutes):

```
uv run python code/part05-dp-ml/hyperparameter_tuning.py
```

38.9. Cheat sheet

Table 38.5.: The results of this chapter.

Result	Formula or value	Conditions
Best of a fixed k	Up to k times one run; composition's bound is attained in the worst case	Any runs; for DP-SGD, k runs of T steps cost one run of kT
Random stopping	3ε-DP; misses the top fraction π with probability $\leq \gamma/\pi$	ε-DP runs, geometric K with mean $1/\gamma$
Truncated negative binomial	One run + constant × one run at order $\hat{\lambda} + \ln \mathbb{E}[K]/(\lambda - 1)$; pure DP: $(2+\eta)\varepsilon$	Random search, K fixed in advance
Poisson	$\varepsilon(\lambda) + \mu\hat{\delta} + \ln \mu/(\lambda - 1)$	Good when each run is very private
Running example, 10 runs	27.23 (composition), 13.67 (Poisson), 10.08 (logarithmic)	One run 6.71
Validation score	Noisy count of correct predictions; cost is the larger of training and validation cost	Disjoint training and validation sets
Fixed candidates	Exponential mechanism, gap $\leq 2(\ln k + 1)/\varepsilon$ correct predictions	Candidates independent of the private training data

Rules of thumb

- The best of k runs is not cheaper than k runs unless k is random. Report the cost of the search, not of the released run.
- Small search at a small per-run budget: Poisson random search. Large search: the logarithmic or $\eta = 0.5$ distribution, at the price of often trying few candidates.

- Validation scores on private data must be noisy; a few thousand validation records make the noise cheap.
- The best tuning is the tuning you do not do on private data.

38.10. Further reading

- Papernot & Steinke, "Hyperparameter Tuning with Renyi Differential Privacy" (ICLR 2022): the Poisson and truncated negative binomial bounds for DP-SGD, the lower bound for a fixed number of runs, and an experiment that recommends the Poisson distribution for small searches.
- Liu & Talwar, "Private Selection from Private Candidates" (STOC 2019): random stopping and the 3ε bound, with threshold and (ε, δ) variants.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): what to tune, how, and what to report.
- Mohapatra, Sasy, He, Kamath & Thakkar, "The Role of Adaptive Optimizers for Honest Private Hyperparameter Selection" (2021): honest tuning by composition, and optimizers that need less of it.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

39. DP-FTRL and matrix factorization

39.1. Why this matters

Stochastic gradient descent (SGD) with differential privacy (DP), the DP-SGD in the full edition of this part, gets much of its privacy from **Poisson sampling**: each record enters a batch only with probability q, and the accountant credits the run for the attacker's uncertainty about whether the record was used at all (Amplification by subsampling). Many pipelines cannot honestly claim it: data loaders shuffle and cut fixed batches, streams arrive in order, and a federated server cannot choose which phones check in. The Poisson accountant can then understate ε considerably, and giving up amplification is expensive: on this part's running example, for $\varepsilon = 2$ over five passes through the data, the noise multiplier has to grow from 0.928 to 4.458, and test accuracy falls from 0.9460 to 0.8941.

How do you train privately without random batches, and can correlated noise beat independent noise? It can, by a lot. The first algorithm to show it ran the follow the regularized leader (FTRL) optimizer on noisy gradient sums released through a binary tree, and is called DP-FTRL. Its modern form, the **matrix factorization mechanism**, contains the tree as a special case, and on the running example it closes nearly all of the gap to amplified DP-SGD without any sampling.

The one idea: **training releases running sums of gradients, so put the noise where the running sums need it.** Noise added at one step may partly cancel the noise added before it; the price is that each record's gradient then has to be hidden in several noisy quantities at once.

39.2. Training is a stream of prefix sums

Plain gradient descent, without momentum, adds up its steps. If x_t is the sum of the clipped gradients of the batch at step t, each of length at most the clipping norm C, and $\hat{x}_t$ is its noisy version, then after t steps with step size η and batch size B

$$w_{t+1} = w_0 - \frac{\eta}{B}\hat{S}_t, \qquad \hat{S}_t = \hat{x}_1 + \cdots + \hat{x}_t. \tag{39.1}$$

Every model along the way is a fixed function of the noisy **prefix sums** $\hat{S}_t$, so releasing those privately makes every model private, and the noise in a model is the noise in its prefix sum. FTRL with a quadratic regularizer produces the same weights. All T prefix sums must be released, not just the last, because training is adaptive: the gradient at step t is computed at w_t, which depends on the earlier sums.

Two mechanisms suggest themselves. **Independent noise on each step**, which is DP-SGD without amplification, hides a record's single gradient with noise of the size of one gradient, but the noise in $\hat{S}_t$ is a sum of t independent terms, a random walk whose standard deviation grows

like $\sqrt{t}$. **Independent noise on each prefix sum** keeps every error the same size, but a record at step 1 appears in all T sums, so the noise must be $\sqrt{T}$ times larger to hide it, and every sum is as bad as the worst sum of the first mechanism. Neither uses the structure of the problem.

Take one epoch of the running example, $T = 100$ steps, at $\varepsilon = 2$ and $\delta = 10^{-5}$. Independent noise needs a noise scale of 1.994 per step, in units of C, so the final prefix sum carries noise of standard deviation 19.94. DP-SGD with Poisson sampling at $q = 0.01$ and the same ε needs only $\sigma = 0.827$, and its final prefix sum carries 8.27: amplification buys a factor of 2.4. The rest of the chapter shows that correlated noise can beat even that without sampling.

39.3. The matrix mechanism

Releasing all prefix sums is a linear map, $S = Ax$, where A is the $T \times T$ lower-triangular matrix of ones and x stacks the x_t as rows. The **matrix mechanism** splits A into two factors, $A = LR$, adds Gaussian noise z in the middle, and releases

$$\hat{S} = L(Rx + z) = Ax + Lz, \tag{39.2}$$

where z has independent entries with standard deviation sC, and s is the **noise scale**, the analogue of DP-SGD's noise multiplier. The **encoder** R decides what gets noised; the **decoder** L recovers the prefix sums from it.

Privacy lives in the columns of R. A record that takes part in step τ changes row τ of x by a gradient g of length at most C, which moves Rx by column τ of R times g. Recall that the privacy of a Gaussian mechanism depends only on one ratio, the sensitivity divided by the noise standard deviation, called μ (The Laplace and Gaussian mechanisms). So the mechanism has

$$\mu = \frac{\operatorname{sens}(R)}{s}, \qquad \operatorname{sens}(R) = \max_j \|r_j\|, \tag{39.3}$$

the length of the longest column r_j. To meet an (ε, δ) target, find the largest admissible ratio $\mu(\varepsilon, \delta)$ from the exact Gaussian calibration, for example $\mu(2, 10^{-5}) = 0.502$ and $\mu(8, 10^{-5}) = 1.666$, and set $s = \operatorname{sens}(R)/\mu$.

Error lives in the rows of L. The error in $\hat{S}_t$ is row t of L, written ℓ_t, times the noise, a weighted sum of independent normals, so its standard deviation in each coordinate is $sC\|\ell_t\|$. At a fixed privacy level that is $(C/\mu)\, e_t$, with

$$e_t = \operatorname{sens}(R)\, \|\ell_t\| \tag{39.4}$$

the **error per unit of privacy**. Multiplying R by 2 and dividing L by 2 changes nothing, so only the product matters, and the best factorization balances short columns against short rows. Independent noise on each step is $R = I$, $L = A$: columns of length 1, rows of length $\sqrt{t}$. Noise on each prefix sum is $R = A$, $L = I$, the opposite extreme.

Does this survive adaptive training, where later gradients depend on earlier outputs? Yes, if both factors are lower-triangular (Denisov et al. 2022, Theorem 2.1). Then row j of $Rx + z$ depends only on steps up to j and can be released at step j. Before the record's step the outputs have the same distribution on both datasets, and from then on the record shifts the remaining rows by column τ times g, so the accounting is the same as for inputs fixed in advance. A factor with entries above the diagonal would need noise or data from future steps.

When R is square and invertible, the mechanism is noisy SGD with correlated noise: it adds $\zeta = R^{-1}z$ to the gradients, each step's noise a fixed combination of the noise drawn so far. Neighbours here leave the record's slot in the public data order empty; if they replace the record instead, every sensitivity doubles.

How good can a factorization be? No factorization can push the root-mean-square of e_t over a run below the trace norm of A (the sum of its singular values) divided by T: 2.18 at $T = 100$ and 3.63 at $T = 10{,}000$. Table 39.1 shows how far each mechanism is from that bound; the next two sections explain the rows.

Table 39.1.: Error of each released prefix sum per unit of privacy ($\mu = 1$, $C = 1$), for one participation per record. The optimal factorization is normalized to columns of length 1 and was computed for $T \le 256$ only.

Mechanism	$T = 100$: sens(R)	root-mean-square e_t	max e_t	$T = 10{,}000$: sens(R)	root-mean-square e_t	max e_t
Independent noise ($R = I$)	1	7.11	10.00	1	70.71	100.00
Noise on prefix sums ($R = A$)	10	10.00	10.00	100	100.00	100.00
Binary tree	2.828	5.05	6.93	3.873	9.84	13.96
Square root	1.591	2.37	2.53	2.000	3.84	4.00
Optimal (Frobenius)	1	2.24	2.45	—	—	—
Lower bound	—	2.18	—	—	3.63	—

From $T = 100$ to $T = 10{,}000$, a hundredfold longer run, the independent-noise error grows tenfold, the tree's by a factor of 2 and the square root's by a factor of 1.6.

39.4. Tree aggregation and DP-FTRL

The first mechanism with error far below $\sqrt{T}$ was the **binary tree**, from the theory of private running counts (Dwork, Naor, Pitassi & Rothblum 2010; Chan, Shi & Song 2010). Build a tree over the steps: each leaf is one step, each node above covers a block of $2, 4, 8, \ldots$ consecutive steps. Release every node's gradient sum with independent noise. Any prefix $1, \ldots, t$ is the union of one node for each 1 in the binary expansion of t: $7 = 4 + 2 + 1$ uses the nodes covering steps 1 to 4, 5 and 6, and 7. So a prefix sum collects at most $\log_2 t + 1$ noise terms instead of t. The price is that each step sits in one node per level, so its column in R has length $\sqrt{L_T}$, with $L_T = \lceil \log_2 T \rceil + 1$ levels, and the noise must be that much larger.

That trade is a large win for long runs: at $T = 10{,}000$ the root-mean-square error is 9.84 against 70.71 for independent noise. But the error jumps around: the prefix $127 = 1111111_2$ uses seven

nodes and 128 uses one (Figure 39.2, left). And the encoder is wasteful: few prefixes use the top nodes, yet every step pays for being in them.

Kairouz et al. (2021) ran FTRL on prefix sums released by the tree, with momentum and periodic restarts, and called it DP-FTRL. They report that it compares favourably with amplified DP-SGD in their experiments while allowing much more flexible data access, and Xu et al. (2023) used it to train production keyboard language models in federated learning.

39.5. The square-root factorization

A natural symmetric choice is $L = R$, a lower-triangular matrix whose square is A. It exists and has a closed form, because lower-triangular matrices that are constant along each diagonal multiply like power series in a variable x, coefficient by coefficient. A, all ones, is the series $1 + x + x^2 + \cdots = 1/(1-x)$, so its square root is the series $(1-x)^{-1/2}$, whose coefficients are

$$r_k = \binom{2k}{k} 4^{-k} : \qquad r = 1, \tfrac{1}{2}, \tfrac{3}{8}, \tfrac{5}{16}, \tfrac{35}{128}, \ldots , \tag{39.5}$$

computed by $r_0 = 1$, $r_k = r_{k-1}(2k-1)/(2k)$. The first column of R is the longest and the last row of $L = R$ the longest, both with squared length $\sum_{k<T} r_k^2$, so that sum is both $\mathrm{sens}(R)^2$ and the largest error per unit of privacy. It grows like $\ln(T)/\pi$: logarithmic, like the tree's error, but much smaller than the tree's at both horizons of Table 39.1, and far below $\sqrt{T}$. At $T = 100$, $\sum r_k^2 = 2.531$, so $\mathrm{sens}(R) = 1.591$ and the final prefix sum's error is 2.53, a quarter of independent noise's 10.

Seen as noisy SGD, the noise added to step t's gradient comes from R^{-1}, the series $(1-x)^{1/2} = 1 - \frac{1}{2}x - \frac{1}{8}x^2 - \frac{1}{16}x^3 - \ldots$:

$$\zeta_t = z_t - \tfrac{1}{2} z_{t-1} - \tfrac{1}{8} z_{t-2} - \tfrac{1}{16} z_{t-3} - \cdots , \tag{39.6}$$

fresh noise minus a decaying share of the noise already injected. Each step's noise is slightly larger than independent noise, with standard deviation tending to $\sqrt{4/\pi} = 1.128$ times the noise scale, but the negative coefficients pull the accumulated noise back towards zero, so the running sums no longer drift like a random walk (Figure 39.1).

The square root is close to optimal: its root-mean-square error, 2.37 at $T = 100$ and 3.84 at $T = 10{,}000$, is within 9% and 6% of the lower bound. The best factorization, computed by a fixed-point iteration (Denisov et al. 2022), reaches 2.24 at $T = 100$, but each iteration costs cubic time in T, so the square root, free and nearly as good, is the usual default for long runs (Figure 39.2). The two even look alike (Figure 39.3).

39.6. Several participations

Training for E epochs puts a record's gradient into E steps, each through a different column of R, and the shifts add before the norm is taken. When all inner products between columns are nonnegative (as for any R with nonnegative entries: the identity, the tree, the square root), the

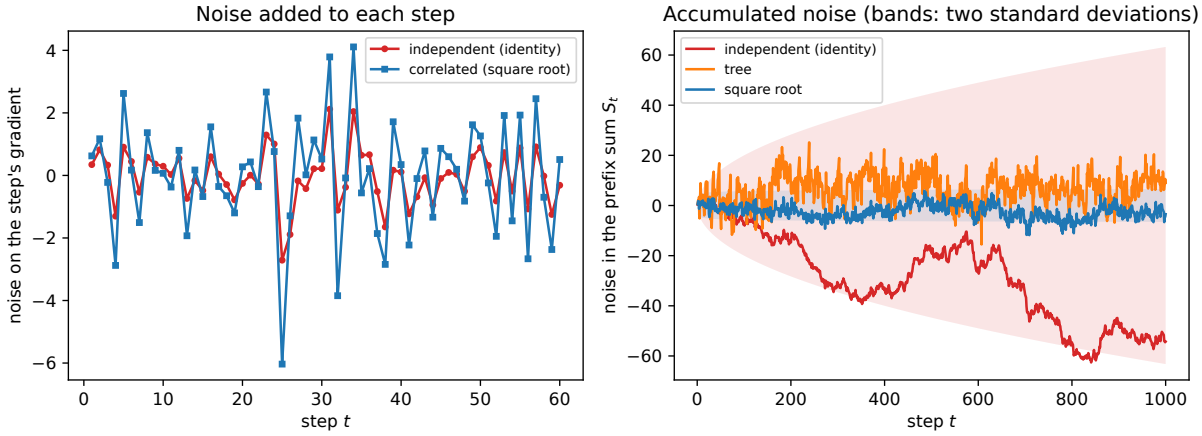


Figure 39.1.: Left: one coordinate of the noise added to each of the first 60 steps' gradients, independent (identity factorization) and square-root correlated, both calibrated to $\mu = 1$ for one participation. Right: the noise in the prefix sums over 1,000 steps, for independent noise, the binary tree and the square root, with bands of two standard deviations. Independent noise drifts like a random walk; the square root's stays within a narrow band.

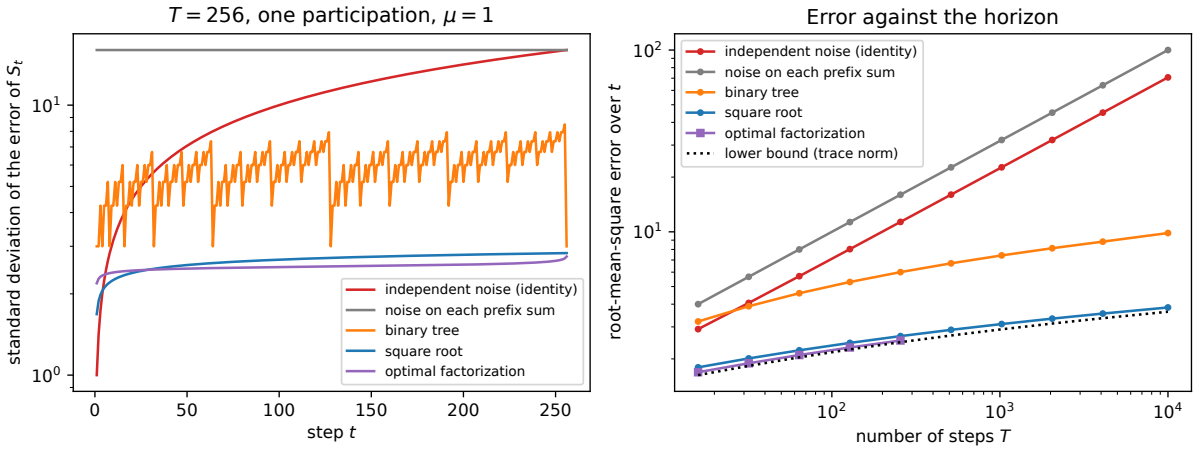


Figure 39.2.: Left: standard deviation of the error of each released prefix sum S_t for $T = 256$, one participation, at $\mu = 1$ and $C = 1$, for five factorizations. Right: the root-mean-square error over a run against its length T, with the trace-norm lower bound dotted; the optimal factorization is computed up to $T = 256$.

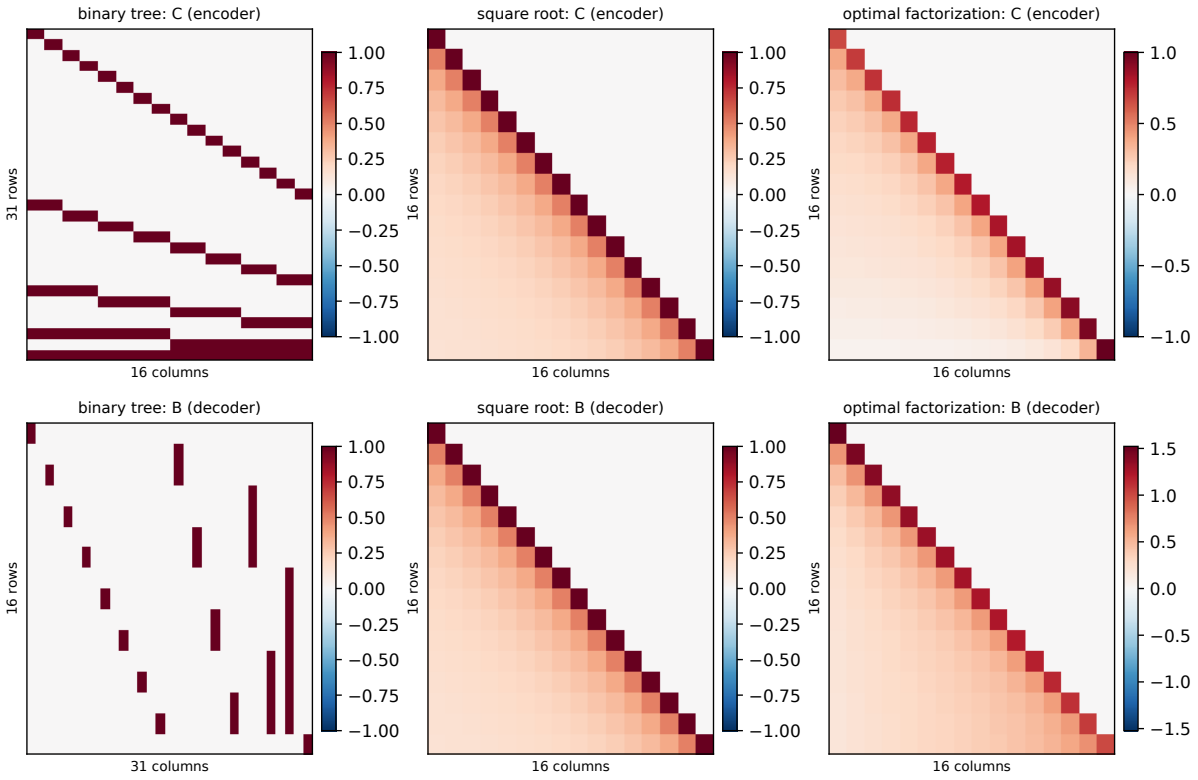


Figure 39.3.: The encoder R (top) and decoder L (bottom) of three factorizations of the 16×16 prefix-sum matrix. The tree's encoder has one row per node (31 rows), and its decoder picks the nodes of each prefix. The square root uses the same matrix for both; the optimal factorization's encoder has columns of length exactly 1.

worst case is the same gradient every time, and the sensitivity is the length of the *sum* of the record's columns, maximized over the possible participation patterns.

For independent noise the columns are orthogonal, so E participations cost $\sqrt{E}$, the usual composition of Gaussian mechanisms. For the square root, columns of nearby steps overlap and point in similar directions, so a record that appears again lands on top of its earlier contribution and the sensitivity grows faster than $\sqrt{E}$. The tree is worse still: a node covering many steps holds several participations of the same record, and their contributions add inside it.

Banding fixes this. Keep only the first b diagonals of the square-root encoder, and make sure any two participations of a record are at least b steps apart (b**-min-separation**). Then the columns of different participations do not overlap at all, the cross terms vanish, and the sensitivity is back to $\sqrt{E}$ times one column. The banded decoder, $L = AR^{-1}$, is less accurate than the square root's for a single participation, because the band no longer spreads the noise across the whole run, but with many participations the sensitivity saving dominates. Table 39.2 uses the running example's order: 100 steps per epoch, each record exactly 100 steps apart, band $b = 100$.

Table 39.2.: Sensitivity (in units of C) and error of the final prefix sum per unit of privacy, for E participations 100 steps apart, $T = 100E$.

Epochs E	Independent: sens / final e_T	Tree	Square root	Banded square root
1	1.000 / 10.00	2.828 / 4.90	1.591 / 2.53	1.591 / 2.53
2	1.414 / 20.00	4.690 / 8.12	2.531 / 4.20	2.250 / 4.16
5	2.236 / 50.00	8.944 / 21.91	5.080 / 8.86	3.558 / 8.56
10	3.162 / 100.00	16.186 / 39.65	9.154 / 16.54	5.031 / 15.69
20	4.472 / 200.00	30.430 / 74.54	17.191 / 32.09	7.115 / 29.85

At 20 epochs the banded sensitivity is $\sqrt{20} \times 1.591 = 7.115$, while the full square root's is 17.191, more than twice as large. The final errors are closer, 29.85 against 32.09, because the banded decoder is less accurate; both are between a sixth and a seventh of independent noise's 200.

So the data order is part of the mechanism. A fixed permutation, drawn independently of the data and repeated every epoch, guarantees a separation of exactly n/B steps; reshuffling every epoch can put a record at the end of one epoch and the start of the next, one step apart, which breaks the banded bound. The order must be fixed in the analysis, but need not be random. Banded factorizations can also use amplification by sampling when it is available (Choquette-Choo et al. 2023).

39.7. The running example: matrix mechanisms against DP-SGD

Does less noise in the prefix sums mean better models? Train the running example (the network of DP-SGD in the full edition, $n = 10{,}000$ records, batch 100, $C = 0.5$) in a fixed order with each mechanism, and compare with DP-SGD with Poisson sampling at $q = 0.01$ and the Rényi differential privacy (RDP) accountant, at the same ε (Table 39.3, Table 39.4).

Table 39.3.: Noise needed by each method at equal ε ($\delta = 10^{-5}$, 100 steps per epoch) and the resulting noise in the final prefix sum, all in units of C. DP-SGD uses Poisson sampling with $q = 0.01$ and the RDP accountant; the others use a fixed order and exact Gaussian accounting.

ε, epochs	DP-SGD σ	Independent s	Tree s	Square root s	Banded s	Final prefix noise: DP-SGD / independent / tree / square root / banded
2, 1	0.827	1.994	5.639	3.172	3.172	8.27 / 19.94 / 9.77 / 5.05 / 5.05
2, 5	0.928	4.458	17.833	10.129	7.093	20.75 / 99.69 / 43.68 / 17.67 / 17.06
2, 10	1.03	6.305	32.273	18.251	10.031	32.57 / 199.38 / 79.05 / 32.98 / 31.28
8, 1	0.501	0.600	1.698	0.955	0.955	5.01 / 6.00 / 2.94 / 1.52 / 1.52
8, 5	0.573	1.342	5.369	3.049	2.135	12.81 / 30.01 / 13.15 / 5.32 / 5.14
8, 10	0.616	1.898	9.716	5.495	3.020	19.48 / 60.02 / 23.80 / 9.93 / 9.42

Table 39.4.: Test accuracy (mean ± standard deviation over three runs) at equal ε, $\delta = 10^{-5}$, batch 100, clipping norm 0.5, best of step sizes 0.5 and 2. Without noise, fixed-order training with clipping reaches 0.9420, 0.9580 and 0.9606 after 1, 5 and 10 epochs.

ε, epochs	DP-SGD (Poisson)	Independent	Tree	Square root	Banded square root
2, 1	0.9343 ± 0.0044	0.9066 ± 0.0031	0.9134 ± 0.0038	0.9361 ± 0.0046	0.9358 ± 0.0019

ε, epochs	DP-SGD (Poisson)	Independent	Tree	Square root	Banded square root
2, 5	0.9460 ± 0.0021	0.8941 ± 0.0071	0.9183 ± 0.0084	0.9430 ± 0.0011	0.9456 ± 0.0019
2, 10	0.9493 ± 0.0012	0.8641 ± 0.0130	0.9096 ± 0.0074	0.9398 ± 0.0029	0.9491 ± 0.0024
8, 1	0.9371 ± 0.0016	0.9395 ± 0.0018	0.9412 ± 0.0006	0.9461 ± 0.0022	0.9434 ± 0.0017
8, 5	0.9524 ± 0.0026	0.9420 ± 0.0029	0.9461 ± 0.0012	0.9497 ± 0.0025	0.9542 ± 0.0014
8, 10	0.9520 ± 0.0006	0.9410 ± 0.0032	0.9484 ± 0.0004	0.9530 ± 0.0015	0.9549 ± 0.0024

Without amplification, correlated noise is essential. At $\varepsilon = 2$, independent noise loses 2.8 points to DP-SGD after one epoch and 8.5 points after ten, and gets worse with more epochs, because its per-step noise grows like $\sqrt{E}$ and then accumulates like a random walk. The tree recovers between a quarter and a half of the gap. The square root and the banded square root recover nearly all of it: at ten epochs the banded encoder reaches 0.9491, within 0.0002 of DP-SGD.

For short runs and large ε, matrix mechanisms can beat amplified DP-SGD outright, because amplification gains little there. At $\varepsilon = 8$ and one epoch, DP-SGD needs $\sigma = 0.501$ and independent noise without sampling 0.600, barely more, while the square root's final prefix sum carries 1.52 against DP-SGD's 5.01, and it reaches 0.9461 against 0.9371. DP-SGD keeps its lead at small ε over many epochs, where amplification is strongest: at $\varepsilon = 2$ its per-step noise grows only from 0.827 to 1.03 between one and ten epochs, while every fixed-order mechanism pays for ten participations. There the full square root falls behind, 0.9398 against 0.9493, because its overlapping columns make repeated participation expensive, and only banding keeps up. Across the table, the method with the least final-prefix noise is the most accurate, or within the run-to-run spread of the most accurate, in every row: that number is a good guide when choosing.

Three caveats. The RDP accountant overstates DP-SGD's ε somewhat compared with a numerical accountant (Choosing an accountant), so a tighter one would lower its noise somewhat. The step size was picked per method by test accuracy, a leak that Hyperparameter tuning in the full edition charges for; here it only compares methods at their best. And the runs are short, at most 1,000 steps; the noise gaps of Table 39.2 keep widening with epochs, so longer runs should separate the methods more, not less. Figure 39.4 shows the same pattern across budgets at five epochs.

This is what makes matrix mechanisms the tool for federated learning. The server cannot sample devices independently with a known rate, so the Poisson accountant does not apply, but it can enforce that a device contributes at most once every b rounds. With that min-separation, a banded mechanism has a provable guarantee, and on the running example's scale it is as accurate as DP-SGD with sampling would be. That is why production federated training of language models uses DP-FTRL (Xu et al. 2023); the privacy unit there is a user, not an example, which User-level differential privacy in the full edition treats.

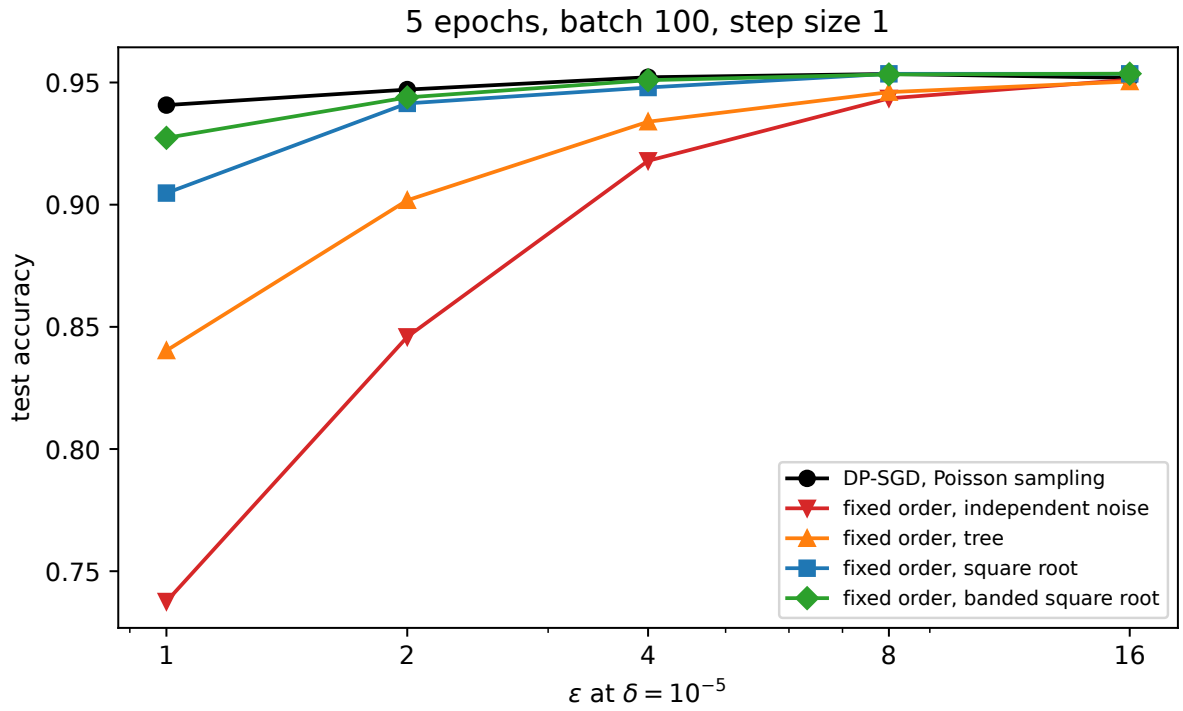


Figure 39.4.: Test accuracy of the running example after 5 epochs (500 steps of batch 100) at step size 1, against ε at $\delta = 10^{-5}$, one run per point. DP-SGD uses Poisson sampling; the other four use a fixed data order and differ only in how their noise is correlated.

39.8. Code

A correlated-noise mechanism is a few lines on top of noisy SGD; what takes care is the sensitivity, which depends on the data order and must be computed for the order you actually use. This snippet computes the square root's coefficients, its single-participation errors, the multi-epoch sensitivities of Table 39.2 and the resulting noise scale:

```
from math import ceil, erfc, exp, log2, sqrt


def sqrt_coeffs(t: int) -> list[float]:
    """r_k = C(2k, k)/4^k: the first column of the lower-triangular square root of A."""
    r = [1.0]
    for k in range(1, t):
        r.append(r[-1] * (2 * k - 1) / (2 * k))
    return r


def toeplitz_sensitivity(c: list[float], t: int, sep: int, epochs: int) -> float:
    """Length of the sum of the encoder columns a record touches, worst slot (entries of c >
    best = 0.0
    for offset in range(sep):
        steps = range(offset, t, sep)[:epochs]
        col = [sum(c[i - p] for p in steps if 0 <= i - p < len(c)) for i in range(t)]
        best = max(best, sqrt(sum(v * v for v in col)))
    return best


def tree_sensitivity(t: int, sep: int, epochs: int) -> float:
    """A record's contributions to one tree node add up before the norm is taken."""
    best = 0.0
    for offset in range(sep):
        steps = range(offset, t, sep)[:epochs]
        total = 0
        for level in range(ceil(log2(t)) + 1):
            counts: dict[int, int] = {}
            for p in steps:
                counts[p >> level] = counts.get(p >> level, 0) + 1
            total += sum(n * n for n in counts.values())
        best = max(best, sqrt(total))
    return best


def phi(x: float) -> float:
    return 0.5 * erfc(-x / sqrt(2))
```

```
def gaussian_mu(eps: float, delta: float) -> float:
    """Largest sensitivity-to-noise ratio for which a Gaussian mechanism is (eps, delta)-DP.
    lo, hi = 1e-9, 100.0
    for _ in range(200):
        mu = (lo + hi) / 2
        if phi(mu / 2 - eps / mu) - exp(eps) * phi(-mu / 2 - eps / mu) > delta:
            hi = mu
        else:
            lo = mu
    return lo


T = 100
r = sqrt_coeffs(T)
sens = sqrt(sum(v * v for v in r))  # the first column is the longest
rows = [sqrt(sum(v * v for v in r[:t])) for t in range(1, T + 1)]  # row t of L = R
print(f"{sens:.3f} {sens * rows[-1]:.2f}")  # 1.591 2.53
print(f"{sens * sqrt(sum(x * x for x in rows) / T):.2f}")  # 2.37
print(f"{sqrt(sum(range(1, T + 1)) / T):.2f}")  # 7.11

E = 20  # epochs of 100 steps, each record exactly 100 steps apart
print(f"{toeplitz_sensitivity(sqrt_coeffs(T * E), T * E, T, E):.3f}")  # 17.191
print(f"{toeplitz_sensitivity(r, T * E, T, E):.3f}")  # 7.115
print(f"{tree_sensitivity(T * E, T, E):.3f}")  # 30.430
print(f"{toeplitz_sensitivity(r, T * 10, T, 10) / gaussian_mu(2.0, 1e-5):.3f}")  # 10.031
```

The banded encoder is the square root cut to its first 100 coefficients, which is why it reuses `r`; the last line is its noise scale for $\varepsilon = 2$ over ten epochs, as in Table 39.3. The full edition's script, matrix_mechanisms.py, builds every factorization, including the optimal one, and trains the running example with each:

```
uv run python code/part05-dp-ml/matrix_mechanisms.py
```

39.9. Cheat sheet

Table 39.5.: The results of this chapter.

Result	Formula	Conditions
Training releases prefix sums	$w_{t+1} = w_0 - \frac{\eta}{B}\hat{S}_t$	Gradient descent or FTRL, no momentum
Matrix mechanism	$\hat{S} = L(Rx + z) = Ax + Lz$, $A = LR$	L, R lower-triangular for adaptive training
Privacy	$\mu = \max_j \lVert r_j \rVert / s$; set $s = \mathrm{sens}(R)/\mu(\varepsilon, \delta)$	One participation

Result	Formula	Conditions
Error	$\mathrm{sd}(\hat{S}_t - S_t) = sC\|\ell_t\|$	Per coordinate
Independent noise	Sensitivity 1, error $\sqrt{t}$	DP-SGD without amplification
Binary tree	Sensitivity $\sqrt{\lceil \log_2 T \rceil + 1}$, error grows like $\log T$	DP-FTRL
Square root	$r_k = \binom{2k}{k} 4^{-k}$; $\mathrm{sens}^2 = \max$ error $= \sum_{k<T} r_k^2$, grows like $\ln(T)/\pi$	Root-mean-square error within 9% of the bound at $T = 100$
Several participations	Sensitivity = length of the sum of the participating columns	Nonnegative inner products between columns
Banded encoder	$\sqrt{E}$ times one column	Band $\leq$ separation of participations

Rules of thumb

- If you cannot implement Poisson sampling, do not account as if you did. Fix the data order in advance and use a matrix mechanism.
- One pass: the square-root factorization. Several passes: a fixed order with evenly separated participations and a banded encoder whose band is at most the separation.
- Compute the sensitivity for the participation pattern you actually have; reshuffling every epoch changes it.
- Compare methods by the noise in the final prefix sum at equal ε.

39.10. Further reading

- Kairouz, McMahan, Song, Thakkar, Thakurta & Xu, "Practical and Private (Deep) Learning without Sampling or Shuffling" (ICML 2021): DP-FTRL with tree aggregation, and its comparison with amplified DP-SGD.
- Denisov, McMahan, Rush, Smith & Thakurta, "Improved Differential Privacy for SGD via Optimal Private Linear Operators on Adaptive Streams" (NeurIPS 2022): the matrix mechanism for adaptive streams, and the optimal factorization.
- Choquette-Choo, Ganesh, McKenna, McMahan, Rush, Thakurta & Xu, "(Amplified) Banded Matrix Factorizations: A Unified Approach to Private Training" (NeurIPS 2023): banded encoders, min-separation, and amplification for matrix mechanisms.
- Xu, Zhang, Andrew, Choquette-Choo, Kairouz, McMahan, Rosenstock & Zhang, "Federated Learning of Gboard Language Models with Differential Privacy" (ACL 2023, industry track): DP-FTRL in a production federated system.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

40. Private prediction: PATE and in-context learning

40.1. Why this matters

Training with differential privacy (DP) by stochastic gradient descent (SGD) with clipped, noisy gradients, called DP-SGD (DP-SGD in the full edition), makes the weights private, and after that every use of the model is free: answering a query is post-processing of the weights. But you pay during training, and you must own the training loop. Sometimes you cannot. The model may be a large language model (LLM) you can prompt but not train, a model without gradients such as a decision forest, or a tool that will answer a few dozen questions in its whole life, for which a private training run is overkill.

The alternative is to train ordinary models, **never release them**, and make each *answer* private instead. The idea that makes this work is one record, one vote. Split the private records into k disjoint shards and train one model, a **teacher**, on each. To answer a query, let every teacher vote for a label, add noise to the vote counts, and release only the winning label. A record lives in one shard, so it can change one teacher and so one vote, however that teacher was trained and however badly it overfits. That is the whole privacy argument. The accuracy argument is about the **gap** between the top two vote counts against the noise, and the noise must grow with the number of answers. Private prediction is therefore cheap for a few queries on which the teachers agree, and expensive otherwise. The method that turns those answers into a releasable model is private aggregation of teacher ensembles (PATE), due to Papernot et al. (2017).

40.2. The noisy vote and its cost

Fix the partition before looking at the data, by a hash of a record identifier rather than by position. Then adding or removing a record changes exactly one shard and one teacher, and that teacher's vote can move from one label to another: one count goes down by 1 and another goes up by 1. The vote histogram moves by at most $\sqrt{2}$ in Euclidean length. If shards were cut by position, inserting one record at the front would shift every later record into the next shard and could change every teacher; the guarantee would be gone.

Two mechanisms release the label. **LNMax** adds Laplace noise of scale b to each count and releases the index of the largest noisy count. It is report-noisy-max (Selection) on scores that one record moves by at most 1 each, in opposite directions, so each answer is $(2/b)$-DP. **GNMax** adds Gaussian noise of standard deviation σ to each count instead. Adding the noise is the Gaussian mechanism on a vector of sensitivity $\sqrt{2}$, and taking the argmax is post-processing, so in Rényi differential privacy (RDP), the accounting whose curves simply add under composition (Rényi DP), each answer costs α/σ^2 at order α (Papernot et al. 2018). Q answers cost $Q\alpha/\sigma^2$, even when each query is chosen after seeing the earlier answers. That curve is a line whose slope is

Q/σ^2, and a line converts to an (ε, δ) that depends only on its slope. Call $\rho(\varepsilon, \delta)$ the largest slope that still converts to your budget; then the noise you need is

$$\sigma = \sqrt{\frac{Q}{\rho(\varepsilon, \delta)}}. \tag{40.1}$$

At $\delta = 10^{-5}$, $\rho = 0.0306$ for $\varepsilon = 1$ and 0.3731 for $\varepsilon = 4$.

Three consequences follow from this formula. First, the noise grows like $\sqrt{Q}$: a hundred times more answers need ten times more noise. Second, neither k nor anything about the teachers appears in it. A teacher can be a network trained without noise, a decision forest or a human annotator who saw one shard; only the noise and the number of answers set the cost. The noise can be large in absolute terms because the counts are large: with 250 teachers, noise of standard deviation 40 is noticeable but not overwhelming. Third, release only the label. Publishing the noisy counts too would release a whole vector, and the data-dependent analysis below needs the label alone.

Table 40.1.: Noise per count for Q answers at $(\varepsilon, 10^{-5})$, from the RDP accountant; Laplace answers are charged the RDP curve of randomized response, the worst pure-DP mechanism.

Budget	Answers Q	GNMax σ	LNMax standard deviation $\sqrt{2}b$
$\varepsilon = 1$	1	5.72	2.85
	100	57.2	113
	1,000	181	361
	10,000	572	1,140
$\varepsilon = 4$	1	1.64	0.709
	100	16.4	32.1
	1,000	51.8	103
	10,000	164	327

Table 40.1 shows why GNMax is the default. LNMax is analysed as a pure-DP step, which is cheap for one answer, and indeed needs less noise there. But the Gaussian's privacy loss is itself Gaussian and composes without slack, so from 100 answers on GNMax needs about half the noise. The RDP accountant is not exact here: at the σ it picks for $\varepsilon = 1$, the exact ε of a Gaussian histogram is 0.915, so the accountant overstates the cost by about 9%.

40.3. When a noisy vote is accurate

On two labels, GNMax returns the plurality label when the noise difference between the two counts does not overturn the gap g. The difference of two independent Gaussians of standard deviation σ is a Gaussian of standard deviation $\sqrt{2}\sigma$, so the plurality wins with probability $\Phi(g/(\sqrt{2}\sigma))$, where Φ is the standard normal distribution function. With more labels, the probability of losing to any competitor is at most the sum of the probabilities of losing to each, a union bound that is exact on two labels and loose when several labels are nearly tied with the winner.

The part's running example makes this concrete: 50-dimensional Gaussian inputs labelled ± 1 by a fixed random network, 10,000 private records, teachers that are regularized logistic regressions on one shard each. With $k = 100$ teachers and the noise for 1,000 answers at $\varepsilon = 4$, $\sigma = 51.8$, a query on which 98 teachers vote one way, a gap of 96, gets the plurality label with probability 0.9051. One split 70 to 30 gets it with probability 0.7076, and one split 52 to 48 with probability 0.5218, barely better than a coin (Figure 40.1).

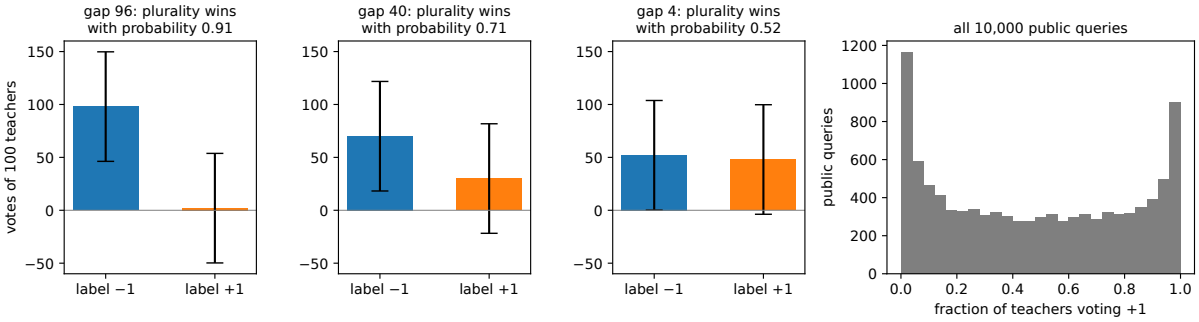


Figure 40.1.: Left three panels: the votes of 100 teachers on three public queries, with bars of one noise standard deviation ($\sigma = 51.8$, the GNMax noise for 1,000 answers at $\varepsilon = 4$); the title gives the probability that the noisy vote returns the plurality label. Right: across all 10,000 public queries, the fraction of the 100 teachers voting +1.

How many teachers? More teachers multiply the gap by k while the noise stays fixed, but each teacher sees fewer records and agrees less with the others. On the running example the second effect is weaker than you might fear. Going from 25 teachers of 400 records to 500 teachers of 20, each teacher's accuracy falls from 0.8954 to 0.6459, but the noiseless plurality vote hardly moves, staying between 0.9336 and 0.9403: a majority of many weak, nearly independent voters is right far more often than any one of them. The median gap grows in absolute terms, from 25 to 138, and since the noise does not depend on k, more teachers win. With 100 answers at $\varepsilon = 4$, 500 teachers answer with accuracy 0.9237, about a point below their noiseless vote. With 1,000 answers at $\varepsilon = 1$, even 500 teachers fall to 0.6987. Note the ceiling: noise can only pull answers towards a coin flip, so no private answer beats the noiseless vote, and on queries the teachers find hard any noise makes the answer close to random.

40.4. Private answers against a private model

You have one budget. Spend it on DP-SGD, and the model's accuracy is fixed however many queries it answers. Spend it on answers, and accuracy starts near the noiseless ensemble and falls towards a coin flip as Q grows, because σ grows like $\sqrt{Q}$. A falling curve crosses a constant at most once, so there is a break-even number of queries Q^* below which private answers win.

Table 40.2.: DP-SGD on the running example against private answers by GNMax: the largest number of queries for which the answers' expected accuracy is at least the private model's.

ε	DP-SGD accuracy (2 runs)	Break-even Q^*	Best k
0.5	0.8164 ± 0.0060	58	500
1	0.9128 ± 0.0207	18	500
2	0.9438 ± 0.0039	0	—
4	0.9573 ± 0.0024	0	—
8	0.9605 ± 0.0004	0	—

On the running example (Table 40.2), at $\varepsilon = 0.5$, where DP-SGD reaches only 0.8164, 500 teachers answer up to 58 queries more accurately than the private model, and at $\varepsilon = 1$ up to 18. From $\varepsilon = 2$ on, DP-SGD beats even a single noiseless vote, because the ensemble's ceiling is below DP-SGD's 0.9438 (Figure 40.2). The pattern is general: private prediction wins at small budgets, where DP-SGD loses much, and for few queries. Two things the comparison leaves out cut the other way. If you cannot run DP-SGD at all, because the model is behind an interface or the data cannot be pooled, the comparison is with nothing. And private prediction runs k models on every query, while DP-SGD trains once.

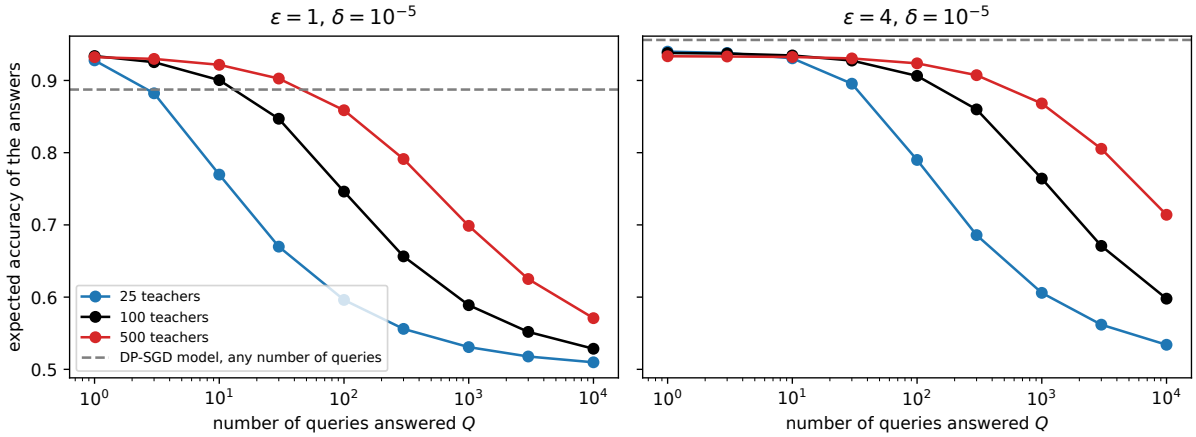


Figure 40.2.: Expected accuracy of Q private answers by GNMax against Q, for 25, 100 and 500 teachers, at $\varepsilon = 1$ (left) and $\varepsilon = 4$ (right), $\delta = 10^{-5}$. The dashed line is a DP-SGD model trained at the same budget, whose accuracy does not depend on the number of queries.

40.5. PATE: from answers to a model

Answers cost budget forever; a model costs it once. PATE spends the budget on labelling Q *public, unlabelled* inputs with GNMax, then trains a **student** on them and releases only the student. The student's training reads public inputs and the Q noisy labels and nothing else, so

it is post-processing: its weights and every prediction it ever makes cost exactly the Q answers, whatever its size, training algorithm or number of predictions. The student may choose which inputs to have labelled after seeing earlier labels, since the composition allows adaptive queries.

Three things limit the student: it learns from Q examples rather than n, some of its labels are wrong, and the plurality vote has its ceiling. More labels mean more noise per label, so for each ε there is a best Q. On the running example, with 250 teachers and the part's network as student, the best student reaches 0.8238 at $\varepsilon = 1$ and 0.8898 at $\varepsilon = 4$, against 0.9128 and 0.9573 for DP-SGD, which wins at every budget. The best Q was chosen by looking at test accuracy, itself a leak that Hyperparameter tuning in the full edition shows how to pay for. The reasons PATE loses here are the conditions under which it wins elsewhere: the task is binary, so a wrong label is as damaging as it can be; the teachers are weak and often disagree; the student needs thousands of labels; and DP-SGD does well even at $\varepsilon = 1$. On a handwritten-digit benchmark (MNIST), where teachers agree on most inputs, Papernot et al. (2018) report a student with 98.5% accuracy against a non-private 99.2%, at $\varepsilon = 1.97$ with 286 answered queries.

PATE needs public inputs from the same distribution as the private data, which are plentiful for images or text and absent for medical records, and they must really be public: an input that is someone else's private record is protected by nothing. And the teachers are non-private models that must stay secret. An attacker who steals one can attack its shard directly, so PATE moves the trust from the training algorithm to the server that holds the teachers.

40.6. Paying less when the teachers agree

The cost α/σ^2 is a worst case over vote histograms, including ties that one teacher decides. When 249 of 250 teachers agree, one record cannot realistically change the answer, and the **data-dependent analysis** of Papernot et al. charges such a query less: its cost is roughly proportional to the probability that the noise overturns the plurality. Figure 40.3 shows it for one answer with $\sigma = 40$ at order 8. Up to a gap of about 100 the bound is no better than the data-independent 0.005; then it falls steeply, to 1.23×10^{-3} at a gap of 150. The bound is valid but not tight: the exact cost on two labels is five times lower at that gap.

The total saving depends on how many queries have large gaps. On the running example only 0.2879 of the public queries have a gap of at least 150 with 250 teachers, so 1,000 answers with $\sigma = 40$ cost $\varepsilon = 4.37$ instead of 5.38 at $\delta = 10^{-5}$, about 20% less. On tasks where nearly every teacher agrees on nearly every input, almost every answer is nearly free. **Confident-GNMax** exploits this by design: it first checks, with a noisy maximum count of large noise σ_1, whether the consensus clears a threshold, and answers with GNMax only if it does. The check costs a little on every query, the answer only on the confident ones, which are also the ones more likely to be right. The price is that the student never sees labels near the decision boundary. On the running example it answered 459 of 1,000 queries and trained a student to 0.8894 at a data-independent $\varepsilon = 3.59$, about what plain GNMax reached at $\varepsilon = 4$.

There is a trap. The data-dependent ε is computed from the votes, so it is itself private information, and its sensitivity is large: one record changes one teacher, whose votes may change on every query. Papernot et al. publish it only after adding noise calibrated to its smooth sensitivity, and pay for that release; on MNIST their data-dependent $\varepsilon = 2.00$ becomes 2.52 ± 0.385 once

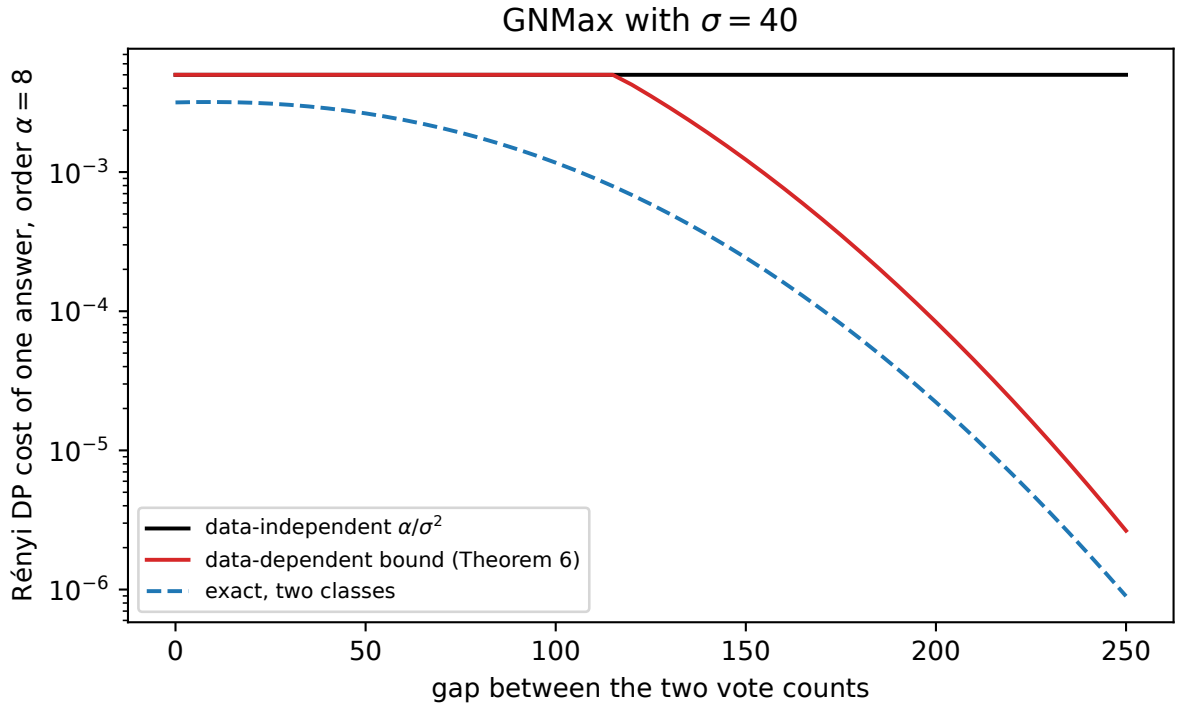


Figure 40.3.: The RDP cost at order 8 of one GNMax answer with $\sigma = 40$, against the gap between the two vote counts, on a log scale: the data-independent α/σ^2, the data-dependent bound of Papernot et al. (2018), and the exact data-dependent cost on two labels.

sanitized. Until it is sanitized, a data-dependent ε is a measurement, not a guarantee. Report the data-independent value, or pay for the release.

40.7. Private in-context learning

An LLM learns a task from a few labelled examples, the **exemplars**, placed in its prompt. If the exemplars are private, every answer is computed from them. Duan et al. (2023) showed a membership inference attack (MIA) on GPT-3 prompted with one private example: the model's probability for the correct label is markedly higher on the example in the prompt. The script's stand-in learner shows the same: the attack's area under the curve (AUC), the chance a member scores above a non-member, is 1.000 against a 1-shot learner and 0.889 against a 16-shot one. Putting private exemplars in a prompt that answers users directly protects nothing.

The fix is PATE with prompts as teachers (Wu et al. 2024; Duan et al. 2023). Split the exemplars into disjoint groups by a fixed rule, prompt the model once per group with the query, and release the GNMax label. One exemplar changes one prompt and so one vote, whatever the LLM computes, and the cost is exactly that of GNMax. Wu et al. also draw a fresh Poisson sample of the exemplars for each query, each record kept with probability r; a record is then absent from most queries, and the cost is that of the subsampled Gaussian of Amplification by subsampling with rate r and noise multiplier $\sigma/\sqrt{2}$, the same accountant DP-SGD uses. The $\sqrt{2}$ is the vote histogram's sensitivity. Within one query the groups must still be disjoint, or a record in two groups moves two votes. **PromptPATE** (Duan et al.) instead labels a few public inputs this way and deploys them as the exemplars of one public prompt, the student in prompt form, free to use forever.

This is where private prediction is strongest. A prompted LLM knows most of the task from pretraining, so its votes with different exemplars agree far more than independently trained small models do, and high agreement is exactly what makes answers accurate and data-dependent costs small. The papers report, on a movie-review sentiment task with GPT-3 models, 92.7% accuracy at $(\varepsilon, \delta) = (0.147, 10^{-6})$ with PromptPATE against 95.2% without privacy (Duan et al.), and 91.97% at $\varepsilon = 1$ for 10,000 queries each answered by ten 4-shot prompts, against 91.89% for one non-private 4-shot prompt (Wu et al.). The script's stand-in learner has no pretraining at all, so it shows the accounting, not the accuracy. Subsampling makes its noise small: 1,000 answers at $\varepsilon = 1$ need $\sigma = 7.39$ with $r = 0.04$, against 181 without, yet with only 25 voters its accuracy still falls from 0.8090 to 0.6800 (Figure 40.4).

40.8. Code

This snippet answers the two questions to ask before deploying a noisy vote: how much noise Q answers need (Equation 40.1, with ρ found by searching the RDP conversion over orders), and how often a vote with a given gap survives it.

```
from math import erfc, log, sqrt


def eps_of_line(rho: float, delta: float) -> float:
```

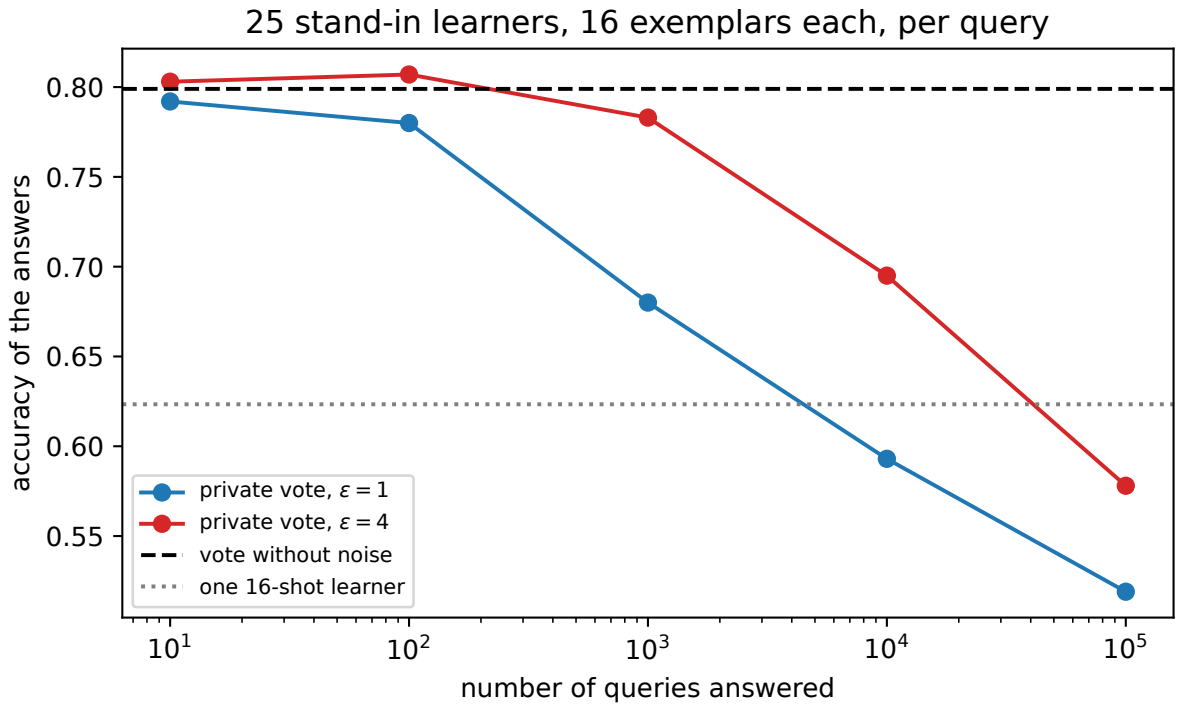


Figure 40.4.: Accuracy of private answers from the stand-in in-context learner against the number of queries answered at $\varepsilon = 1$ and $\varepsilon = 4$ ($\delta = 10^{-5}$), with 25 groups of about 16 exemplars drawn per query at rate 0.04. The dashed line is the vote without noise and the dotted line a single 16-shot learner.

```
    """(eps, delta) of the RDP line rho * alpha, by the improved conversion at the best orde
    orders = [1 + j / 100 for j in range(1, 6_000)]
    return min(rho * a + log((a - 1) / a) - (log(delta) + log(a)) / (a - 1) for a in orders)


def largest_slope(eps: float, delta: float) -> float:
    """The largest rho whose RDP line rho * alpha converts to (eps, delta), by bisection."""
    lo, hi = 1e-6, 10.0
    for _ in range(60):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if eps_of_line(mid, delta) <= eps else (lo, mid)
    return lo


def gnmax_sigma(answers: int, eps: float, delta: float) -> float:
    """Noise per vote count so that `answers` GNMax answers cost (eps, delta) in total."""
    return sqrt(answers / largest_slope(eps, delta))


def p_plurality(gap: float, sigma: float) -> float:
    """Probability that a two-label GNMax vote returns the plurality label."""
    return 1 - 0.5 * erfc(gap / (2 * sigma))


print(f"{largest_slope(1.0, 1e-5):.4f}")  # 0.0306
print(f"{gnmax_sigma(100, 1.0, 1e-5):.1f}")  # 57.2
print(f"{gnmax_sigma(10_000, 1.0, 1e-5):.0f}")  # 572
sigma = gnmax_sigma(1000, 4.0, 1e-5)
print(f"{sigma:.1f}")  # 51.8
print(f"{p_plurality(96, sigma):.4f}")  # 0.9051
print(f"{p_plurality(4, sigma):.4f}")  # 0.5218
```

The full edition's script trains the teachers, students and DP-SGD baselines, implements the data-dependent accountants and Confident-GNMax, and simulates in-context learning:

```
uv run python code/part05-dp-ml/private_prediction.py
```

40.9. Cheat sheet

Table 40.3.: The results of this chapter.

Result	Formula	Notes
One record, one vote	Histogram moves by $\sqrt{2}$ in ℓ_2	Shards fixed by a hash, not by position

Result	Formula	Notes
LNMax	$(2/b)$-DP per answer	Best for one or two answers only
GNMax	α/σ^2 RDP per answer	Release the label only; adaptive queries allowed
Noise for Q answers	$\sigma = \sqrt{Q/\rho(\varepsilon,\delta)}$; $\rho = 0.0306$ at $\varepsilon = 1$, $\delta = 10^{-5}$	Independent of k
Accuracy, two labels	$\Phi(g/(\sqrt{2}\sigma))$	Capped by the noiseless vote
Answers against a model	Answers win for $Q < Q^*(\varepsilon)$	58 at $\varepsilon = 0.5$, none from $\varepsilon = 2$, running example
PATE student	Costs its Q labels; predictions free	Needs public inputs; teachers stay secret
Data-dependent ε	Small when gaps are large	Not a guarantee until sanitized
Private in-context learning	GNMax over disjoint exemplar groups; subsampled Gaussian with $\sigma/\sqrt{2}$ under Poisson sampling	Never put private exemplars in a public-facing prompt

40.10. Further reading

- Papernot, Song, Mironov, Raghunathan, Talwar & Erlingsson, "Scalable Private Learning with PATE" (ICLR 2018): GNMax, the data-dependent analysis, Confident-GNMax and how to sanitize the data-dependent ε.
- Papernot, Abadi, Erlingsson, Goodfellow & Talwar, "Semi-supervised Knowledge Transfer for Deep Learning from Private Training Data" (ICLR 2017): the original PATE, with LNMax and semi-supervised students.
- Duan, Dziedzic, Papernot & Boenisch, "Flocks of Stochastic Parrots" (NeurIPS 2023), and Wu, Panda, Wang & Mittal, "Privacy-Preserving In-Context Learning for Large Language Models" (ICLR 2024): the attack on prompts, PromptPATE, and subsampled voting over exemplar groups, including text generation.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

41. Public data and private fine-tuning

41.1. Why this matters

Training with differential privacy (DP) from random weights, by stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD (DP-SGD in the full edition), costs little on the part's running example at $\varepsilon = 8$ and a great deal at $\varepsilon = 1$ and below. The most accurate private models are not trained that way. They start from a model **pretrained** non-privately on a large **public** dataset and only **fine-tune** it privately on the sensitive data, and at the same ε they are far more accurate than the same models trained privately from scratch (Tramèr & Boneh 2021; Yu et al. 2022; Li et al. 2022; De et al. 2022). A second practice helps on top of that: fine-tuning only a small part of the model often beats fine-tuning all of it under DP, although without privacy it would usually be the other way round.

Both follow from one idea: **the noise is paid per trainable direction, the signal is not.** DP-SGD adds noise of the same size to every coordinate it updates, while the useful part of the gradient is bounded in length however many coordinates there are, and usually concentrated in a few directions. A coordinate the task does not need only collects noise. Public data helps twice: it moves the starting point close to a good model, so little remains to be learned privately, and it tells you, for free, which few directions that remainder lies in. What it does not do is protect the people in the public data.

41.2. Why dimension costs, and when it does not

Each step of DP-SGD adds Gaussian noise of standard deviation σC to a sum of per-example gradients clipped to length C, where σ is the noise multiplier, and divides by the expected batch size B. Nothing in the privacy proof depends on which weights the gradient is taken with respect to, so whatever you train, the accountant gives the same ε. What changes is the noise. Over k trainable coordinates, a Gaussian vector with standard deviation $s = \sigma C/B$ in each coordinate has length about $s\sqrt{k}$, because its squared length is a sum of k terms of mean s^2:

$$\text{noise length} \approx \frac{\sigma C\sqrt{k}}{B}. \tag{41.1}$$

The signal, the average clipped gradient, is at most C long whatever k is, and in practice it is often *longer* when k is smaller, because each record's gradient restricted to fewer coordinates is shorter and less often clipped. The part's running example shows both effects: 50-dimensional Gaussian inputs labelled by a fixed random network, $n = 10{,}000$ private records, a student network with 3,329 weights pretrained on a related public task (Section 41.6), and DP-SGD with sampling rate $q = 0.01$, $C = 0.5$ and 10,000 steps, which needs $\sigma = 4.13$ for $\varepsilon = 1$ at $\delta = 10^{-5}$. From the pretrained weights, the signal is 0.0130 long when all weights are trained, 0.0207 for

a rank-4 adapter with the last layer, and 0.0250 for the last layer alone. At $\varepsilon = 1$ the noise is 92.0 times the signal for all weights, 22.9 times for the adapter and 6.7 times for the last layer (Table 41.1, Figure 41.1). A signal of constant direction outgrows a random walk of noise after about (noise/signal)^2 steps: 8,455, 523 and 44 steps.

Table 41.1.: Length of the noise on the averaged gradient, $\sigma C\sqrt{k}/B$ with $B = 100$ and $C = 0.5$, for three numbers of trainable weights.

ε	σ	All weights, $k = 3{,}329$	Rank-4 adapter and last layer, $k = 525$	Last layer, $k = 65$
0.5	7.72	2.227	0.884	0.311
1	4.13	1.191	0.473	0.166
2	2.28	0.658	0.261	0.092
8	0.917	0.265	0.105	0.037

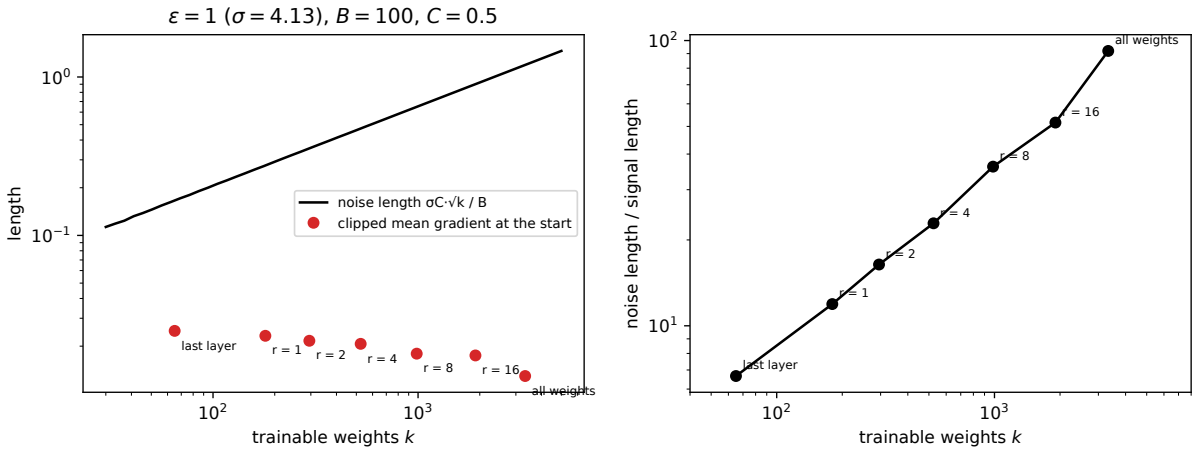


Figure 41.1.: Left: at $\varepsilon = 1$ ($\sigma = 4.13$, $B = 100$, $C = 0.5$), the noise length $\sigma C\sqrt{k}/B$ (line) against the number of trainable weights k, and the length of the average clipped gradient at the pretrained model (points) for the last layer, low-rank adapters of ranks 1 to 16, and all weights. Right: their ratio. The noise grows like $\sqrt{k}$ while the signal shrinks, because longer gradient vectors are clipped more.

Yet large language models with hundreds of millions of weights are fine-tuned privately with good results, so the count k cannot be the whole price. It is not, because noise only hurts in directions the loss cares about. Model the loss near the starting point as a quadratic. In a direction with curvature h, noise kicks the weights around and the curvature pulls them back. A curved direction reaches a steady state and charges a fixed amount for it; a nearly flat direction keeps accumulating kicks but charges almost nothing for them, because moving along it barely changes the loss. Adding this up over T steps of step size η gives a loss increase of about

$$\Delta \approx \frac{\eta s^2}{4}\, d_{\text{eff}}, \qquad d_{\text{eff}} = \sum_{j=1}^{k} \Big(1 - (1 - \eta h_j)^{2T}\Big), \tag{41.2}$$

where h_j are the curvatures of the trainable directions. Each term is close to 1 for a direction whose curvature is above about $1/(2\eta T)$ and close to 0 below it. So the **effective dimension** d_{eff} roughly counts the directions the noise has time to fill, and it is at most k. Li, Liu et al. (2022) give this kind of explanation for why private fine-tuning of large models does not suffer from their size.

On the running example, at the pretrained weights with $\eta = 0.1$ and $T = 10{,}000$, only 406 of the 3,329 directions have curvature above $1/(2\eta T) = 5 \times 10^{-4}$, and $d_{\text{eff}} = 451$; for the last layer, $d_{\text{eff}} = 60.2$ of 65. Full fine-tuning's noise is 7.2 times longer than the last layer's, and by Equation 41.2 it costs only about 7.5 times more, not the 51 times that the ratio of weight counts suggests. Freezing weights can only lower d_{eff}, but it adds **bias**: a frozen direction the task needs can never be learned. And d_{eff} grows with ηT, so a larger step size fills more flat directions with noise, one reason private fine-tuning wants smaller step sizes than training from scratch. The quadratic is a local model and loss is not accuracy, so use Equation 41.2 to compare strategies, not to predict accuracy.

41.3. The last layer: linear probing

The simplest strategy freezes everything except the last layer and trains a linear classifier on the frozen features, **linear probing**. Since the features come from public data, they are a fixed transformation of each record, and the problem becomes private convex empirical risk minimization (ERM) in k dimensions, where the methods and rates are known (Private ERM in the full edition). Tramèr & Boneh (2021) showed that this beats end-to-end private training on standard image benchmarks at moderate budgets, even with handcrafted features.

A probe's error splits in two. The **feature gap** is the distance from the best linear model on these features to the best model of any kind; it is paid even with infinite data and no privacy. The **privacy cost** is private convex ERM's, which falls like $\sqrt{k}/(n\varepsilon)$ up to logarithmic factors (Bassily, Smith & Thakurta 2014). So features cap the accuracy, and privacy only decides how close you get to the cap. On the running example, the pretrained network labels 0.9035 of the private test inputs correctly without any private data. Fitting its last layer exactly and non-privately on the 10,000 private records raises this to 0.9536, the probe's ceiling, against 0.9606 for a network trained non-privately from scratch, a feature gap of 0.007. The private probe reaches 0.9478 at $\varepsilon = 0.5$ and 0.9514 at $\varepsilon = 8$: within 0.006 of its ceiling at every budget, and unable to pass it at any.

That makes the probe the safe first baseline: convex, with few hyperparameters, and if it meets your accuracy target there is little reason to train more.

41.4. Low-rank adapters

When the features are not quite right for the new task, the probe stops at its ceiling, and training all weights pays for every weight. **Low-rank adaptation (LoRA)** (Hu et al. 2022) sits in between. It freezes each pretrained matrix W_0, of size $d_{\text{out}} \times d_{\text{in}}$, and learns a correction $W = W_0 + UV^\top$

with U of size $d_{\text{out}} \times r$ and V of size $d_{\text{in}} \times r$, starting from $U = 0$ so that training begins exactly at the pretrained model. It trains

$$k = r(d_{\text{in}} + d_{\text{out}}) \tag{41.3}$$

weights instead of $d_{\text{in}} d_{\text{out}}$. A rank-8 adapter on a 768×768 projection of a transformer trains 12,288 weights instead of 589,824, 48 times fewer, so its noise is 6.9 times shorter. Its per-example gradients are outer products of short vectors, so their norms can be computed without forming them, the ghost clipping of DP-SGD in the full edition; on the running example the ghost norms agree with explicitly formed gradients to a relative difference of 3.3×10^{-16}. The privacy guarantee is DP-SGD's with the same q, σ and number of steps.

What the restriction costs is direction. At the start, with $U = 0$ and V random, the update projects the true gradient onto a random r-dimensional subspace of the layer's inputs, which captures little of it. As training proceeds V moves and the subspace aligns with the useful directions. Adapters work when the change the new task needs is itself close to low rank, and the rank trades bias against noise. Yu et al. (2022) found that private fine-tuning of language models with LoRA matches or beats private full fine-tuning; Li, Tramèr, Liang & Hashimoto (2022) found the reverse once full fine-tuning had large batches and a tuned step size, which Equation 41.2 allows, since full fine-tuning pays d_{eff}, not k. The published evidence goes both ways. The adapter's other advantages are certain: it stores only k numbers per task and clips much smaller gradients.

41.5. Gradients near a public subspace

If noise costs little in directions the loss does not use, you could add noise only in the directions it does use, if you knew them. Public data gives an estimate: compute per-example gradients on public records, find their top principal directions, and project the private clipped gradient sum and its noise onto those m directions. Projection never lengthens a vector, so the sensitivity is still C, and because the subspace is fixed before any private data are read, the guarantee is unchanged. This is projected DP-SGD (Zhou, Wu & Banerjee 2021).

The trade is between noise and bias. The noise falls by $\sqrt{p/m}$, where p is the number of weights. The signal keeps only the fraction f_m of its energy that lies in the subspace, and the lost part is a bias that no number of steps recovers. If the public data resemble the private task, f_m is close to 1 for small m and the trade is excellent. If they do not, the subspace is no better than a random one, $f_m \approx m/p$, and the projection removes as much signal as noise. On the running example, a single direction of 500 public per-example gradients holds 60% of the private gradient's energy, and 64 directions, 2% of the dimensions, hold 73%, against 2.2% for 64 random directions. At $\varepsilon = 1$ the kept signal outgrows the noise after 222 steps in the 64-dimensional subspace, against 8,455 for all weights; the price is the 27% outside it, which a fixed projection never learns, so methods recompute the subspace as training proceeds. The same concentration is what makes d_{eff} small for full fine-tuning (Figure 41.2).

41.6. Pretraining and fine-tuning the running example

The public task labels 50,000 inputs from the private input distribution with a **related teacher**: the private teacher's hidden units, each slightly perturbed, read out with weights turned partly

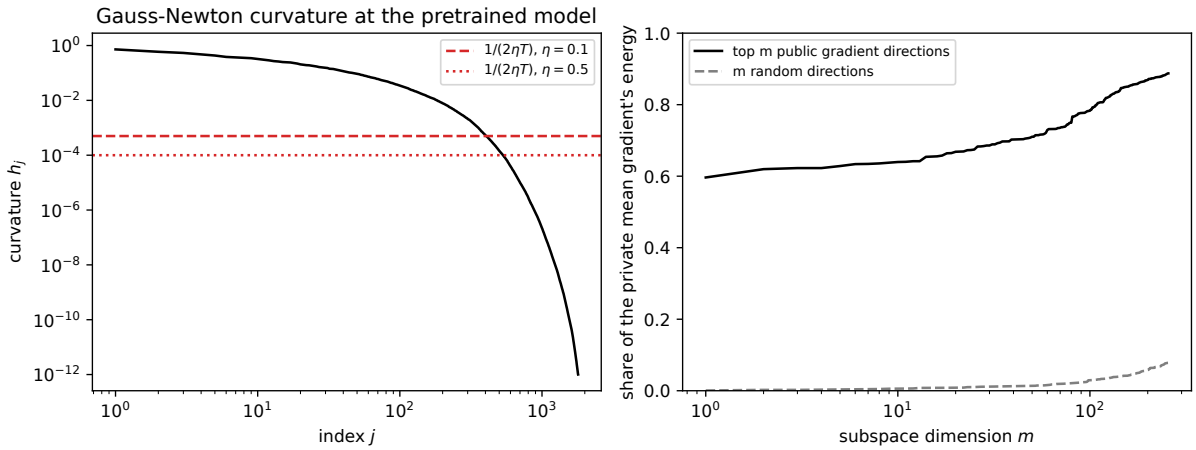


Figure 41.2.: Left: eigenvalues of the Gauss–Newton curvature of the private loss at the pretrained weights, estimated from 2,000 private records, on log scales, with the thresholds $1/(2\eta T)$ for $\eta = 0.1$ and 0.5 and $T = 10{,}000$; directions below a threshold contribute little noise cost. Right: the fraction of the energy of the private average clipped gradient that lies in the top m principal directions of 500 public per-example gradients (solid), and in m random directions (dashed).

towards a random vector. It agrees with the private teacher on 0.9004 of the private test inputs, so pretraining helps without solving the private task. The student is pretrained on it non-privately and then fine-tuned with DP-SGD in four ways: all weights from a random start (*scratch*), all weights from the pretrained ones (*full*), the last layer only, and a rank-4 adapter on the first layer plus the last layer. Each method's step size was chosen at $\varepsilon = 1$ on half of the private test set, without paying for that choice (Hyperparameter tuning in the full edition shows how to), and accuracy is reported on the other half.

Table 41.2.: Test accuracy on the report half of the private test set, mean and standard deviation over two runs, $\delta = 10^{-5}$. The pretrained model without fine-tuning has 0.9035.

	Scratch	Full	Last layer	Adapter, rank 4
Trainable weights k	3,329	3,329	65	525
$\varepsilon = 0.5$	0.8589 ± 0.0075	0.9395 ± 0.0006	0.9478 ± 0.0010	0.9464 ± 0.0005
$\varepsilon = 1$	0.9246 ± 0.0050	0.9599 ± 0.0013	0.9497 ± 0.0000	0.9616 ± 0.0001
$\varepsilon = 2$	0.9518 ± 0.0010	0.9663 ± 0.0001	0.9519 ± 0.0005	0.9653 ± 0.0002
$\varepsilon = 8$	0.9611 ± 0.0012	0.9677 ± 0.0001	0.9514 ± 0.0005	0.9655 ± 0.0001
Non-private	0.9606	0.9704	0.9536	—

On this task pretraining is worth about a factor of eight in ε (Table 41.2). Every fine-tuning method at $\varepsilon = 0.5$ beats training from scratch at $\varepsilon = 1$, and the adapter at $\varepsilon = 1$, 0.9616, matches training from scratch at $\varepsilon = 8$ and the non-private model trained from scratch.

Which weights to train depends on the budget, as the noise lengths and the ceiling explain together. At $\varepsilon = 0.5$ the last layer is best: its noise is 7.2 times shorter than full fine-tuning's,

and its ceiling, 0.9536, is not yet binding. From $\varepsilon = 1$ on it is stuck near that ceiling, while the adapter and full fine-tuning keep improving, because they can correct the features and the noise is now small enough to let them. The adapter is never more than 0.0022 below the best method at any budget, so it is the robust middle choice. Do not over-read the small differences: two runs estimate the spread only roughly, a repeat of the adapter at $\varepsilon = 1$ gave a standard deviation of 0.0023, and differences between the pretrained methods below about 0.005 are not rankings (Figure 41.3, a separate set of single runs, orders them differently at $\varepsilon = 0.5$ and 1). Step sizes matter as much as the method: full fine-tuning lost 1.3 points at a step size three times too large, and the best step size for full fine-tuning and the adapter, 0.1, was 2.5 times smaller than from scratch, as the effective-dimension argument predicts.

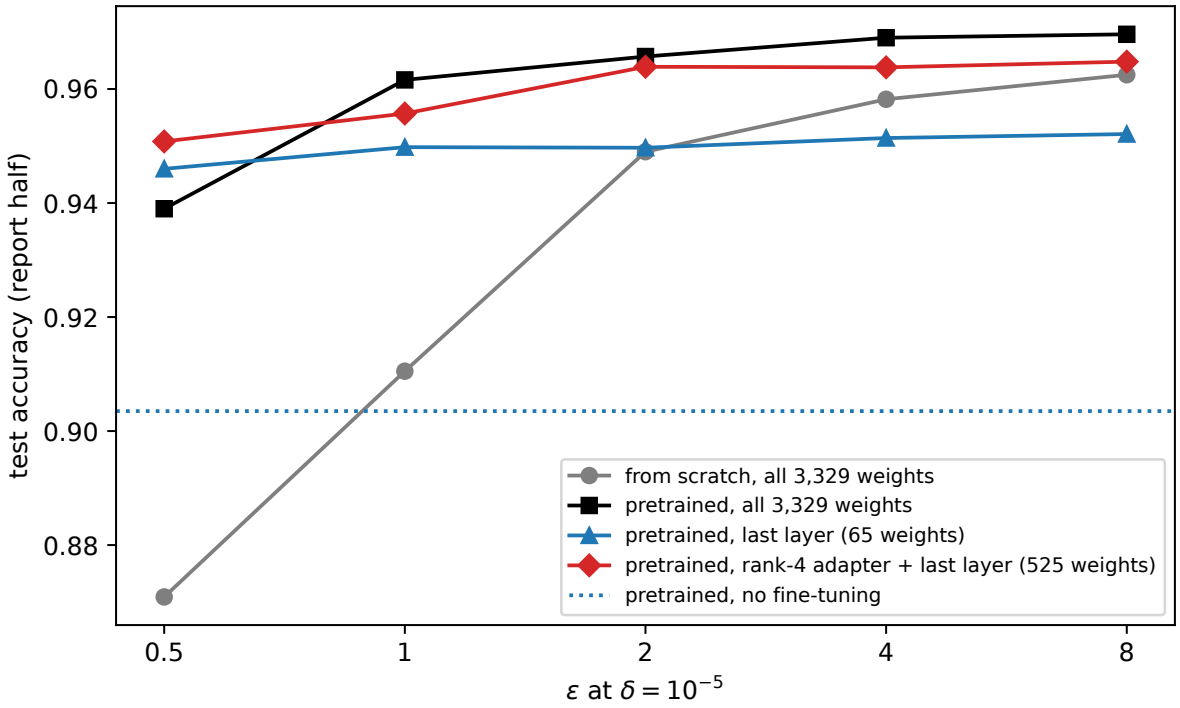


Figure 41.3.: Test accuracy of the four strategies against ε at $\delta = 10^{-5}$, one run each with the tuned step sizes and a different seed from Table 41.2, with the pretrained model's accuracy before fine-tuning (dotted). The pretrained methods are far ahead of training from scratch at small budgets; the last layer stays near its ceiling from $\varepsilon = 1$ on, and training from scratch passes it at larger budgets.

41.7. What public data does not buy

The guarantee treats the public data as a fixed input. For every fixed public dataset, the map from the private dataset to the fine-tuned model is (ε, δ)-DP, because the DP-SGD proof holds for any starting weights chosen independently of the private data. With respect to the public data there is no guarantee at all: non-private pretraining can memorize individual records (Memorization and extraction), and the fine-tuned model inherits whatever the pretrained weights encode.

This matters most for a person whose record is in both datasets. At $\varepsilon = 1$ and $\delta = 10^{-5}$, any test of whether a record was in the private set that has a false positive rate of 0.01 has a true positive rate of at most $e \times 0.01 + 10^{-5} = 0.0272$. If the same record is also in the public set, a membership test against the pretrained model, such as the likelihood-ratio attack of Membership inference, is bounded by nothing; it may fail in practice, but no theorem says it will. Compare a record that appears twice in the private set: group privacy gives the two copies $(2\varepsilon, 2e^{\varepsilon}\delta) = (2, 5.44 \times 10^{-5})$, weaker but real. A copy in the public set is worse than a duplicate in the private set. Web-scale "public" data contain posts, profiles and leaked documents, so overlap with private medical notes or emails is a real risk, and Tramèr, Kamath & Carlini (2024) argue that "differentially private" should be used with care for such models. Deduplication against the private set is the minimum check, and exact matching protects copies, not people: paraphrases and other records about the same person survive it.

Two further caveats. Benchmarks whose "private" set resembles curated public data can overstate what the method does on genuinely sensitive data, which is often unlike anything public. And any choice made by looking at private data, such as which public model to start from or the step size, is part of the private computation and must be paid for; choosing on public validation data is free. When you report ε, say what the public data are, how they were obtained, whether overlap was checked, and that the guarantee covers the private data given the pretrained model.

41.8. Code

Before choosing what to fine-tune, count: the noise length for each candidate set of trainable weights, at the σ your accountant gives, is the first number to compare with the signal.

```
from math import sqrt


def noise_length(sigma: float, clip: float, batch: float, trainable: int) -> float:
    """Length of DP-SGD's noise on the averaged gradient over `trainable` coordinates."""
    return sigma * clip * sqrt(trainable) / batch


def lora_weights(rank: int, d_in: int, d_out: int) -> int:
    """Trainable weights of a rank-r adapter W0 + U V^T on a d_out x d_in matrix."""
    return rank * (d_in + d_out)


adapter = lora_weights(4, 51, 64) + 65  # first-layer adapter plus the 65-weight last layer
print(adapter)  # 525
print(f"{noise_length(4.13, 0.5, 100, 3329):.3f}")  # 1.191
print(f"{noise_length(4.13, 0.5, 100, adapter):.3f}")  # 0.473
print(f"{noise_length(4.13, 0.5, 100, 65):.3f}")  # 0.166
big = lora_weights(8, 768, 768)
print(big, 768 * 768 // big)  # 12288 48
print(f"{sqrt(768 * 768 / big):.1f}")  # 6.9
```

The full edition's script builds the public task, pretrains, runs every fine-tuning strategy and estimates the curvature and the public subspace; it takes about four minutes:

```
uv run python code/part05-dp-ml/public_data.py
```

41.9. Cheat sheet

Table 41.3.: The results of this chapter.

Result	Formula	Notes
Noise length	$\sigma C\sqrt{k}/B$ over k trainable weights	Same ε whatever is trained
Noise cost	$\approx \frac{\eta s^2}{4} d_{\text{eff}}$, $d_{\text{eff}} = \sum_j (1-(1-\eta h_j)^{2T}) \le k$	Counts directions with curvature above about $1/(2\eta T)$
Linear probe	Private convex ERM in k dimensions	Error = feature gap + privacy cost; cannot pass its ceiling
LoRA	$r(d_{\text{in}} + d_{\text{out}})$ weights per matrix	Ghost clipping applies; robust middle choice
Public subspace	Noise down by $\sqrt{p/m}$, signal kept f_m	Subspace from public data only; $1 - f_m$ is bias
Pretraining	About a factor of eight in ε on the running example	Depends on how close the public task is
Guarantee	(ε, δ) for the private data given the public data	Nothing for records in the public data

41.10. Further reading

- Tramèr & Boneh, "Differentially Private Learning Needs Better Features (or Much More Data)" (ICLR 2021): why linear probing on good features beats end-to-end private training at moderate budgets.
- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022), and Yu et al., "Differentially Private Fine-tuning of Language Models" (ICLR 2022): full and parameter-efficient private fine-tuning, and the hyperparameters that make each work.
- Li, Liu, Hashimoto, Inan, Kulkarni, Lee & Thakurta, "When Does Differentially Private Learning Not Suffer in High Dimensions?" (NeurIPS 2022): the effective-dimension view behind Section 41.2.
- Tramèr, Kamath & Carlini, "Position: Considerations for Differentially Private Learning with Large-Scale Public Pretraining" (ICML 2024): what public pretraining does and does not protect, and how benchmarks mislead.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

42. User-level differential privacy

42.1. Why this matters

Every differential privacy (DP) guarantee in this part so far protects one record. Stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD in the full edition, reports $\varepsilon = 6.71$ at $\delta = 10^{-5}$ for the part's running example, and that number says an adversary who sees the model learns little about whether any single training example was used. But people do not contribute one record each. A user of a messaging app writes hundreds of messages, a patient has dozens of visits, a photographer uploads thousands of photos. Each record is protected; the person behind all of them is not, because what an adversary learns from all of a person's records together can be far more than what they learn from one.

The one idea: **the neighbouring relation is the privacy unit.** DP compares outputs on two datasets that differ in one unit, and whatever differs between them is what is protected. If neighbours differ in one record, records are protected; if they differ in all of one person's records, people are. Nothing else in the definition changes. So there are two ways to protect a person. You can keep the record as the unit and pay for the person afterwards, which is cheap to build and expensive in privacy. Or you can make the person the unit from the start: sample people, bound each person's total contribution, and run the usual accountant with people in place of records. On the running example the first route gives a useless model at any reasonable budget, and the second a model within a few points of the non-private one.

42.2. The privacy unit

A mechanism is **user-level** (ε, δ)-DP if the definition of DP holds for every pair of datasets that differ by adding or removing all the records of one user; it is **example-level** DP if it holds for datasets that differ in one record. User-level DP is not a new privacy notion, only a coarser neighbouring relation, so post-processing, composition and every accountant of Part 4 hold for it unchanged. What changes is the sensitivity: how far one unit can move the computation. Records, users, households, devices and organizations are all possible units, and the right one answers the question "whose participation do I want to hide?" A guarantee that does not name its unit leaves that question open.

What does an example-level guarantee promise a person with k records? Adding that person's records one at a time walks from one dataset to the other in k example-level steps. **Group privacy** applies the one-record guarantee at each step (Post-processing, group privacy and composition): each step multiplies the probability bound by e^{ε}, so the person gets $k\varepsilon$, and each step's slack δ is multiplied by the e^{ε} of every later step,

$$\delta_k = \delta \, \frac{e^{k\varepsilon} - 1}{e^{\varepsilon} - 1}. \tag{42.1}$$

Some mechanisms attain this bound, so nothing better holds for a mechanism known only to be (ε, δ)-DP. The ε part is linear in k, already weak: at $\varepsilon = 1$ per record, a person with 10 records gets $\varepsilon = 10$. The δ part is worse, because it grows exponentially in $k\varepsilon$.

Read naively, the running example's $(6.71, 10^{-5})$ gives a user with two records $\varepsilon = 13.42$ and $\delta_2 = 10^{-5}(1 + e^{6.71}) = 0.0082$: a chance of almost one in a hundred that the guarantee fails outright, which is no guarantee. You can do better for DP-SGD, because its accountant converts to (ε, δ) at any δ: pick the per-record δ that makes $\delta_2 = 10^{-5}$ exactly. That gives a per-record ε of 9.30 and a user-level $(18.61, 10^{-5})$. For five records the same procedure gives a user-level ε of 118.12. For ten it gives nothing at all. Shrinking the per-record δ raises the per-record ε, and δ_k is a race between a falling δ and a rising $e^{(k-1)\varepsilon}$; beyond a few records the rising term wins at every δ, and no choice brings δ_{10} down to 10^{-5}. In the running example's population of 1,000 users, 244 have at least ten records.

42.3. What a record-level model gives a heavy user

Group privacy treats DP-SGD as a black box, and it is not one: you know exactly how a user's records enter each step. Recall that DP-SGD draws each record into the batch independently with probability q (Poisson sampling), clips each record's gradient to norm C, sums them and adds Gaussian noise of standard deviation σC, where σ is the noise multiplier. A user with k records therefore has a random number of them in each batch, between 0 and k, binomially distributed, and the worst case is that all the drawn ones push the sum by the full C in the same direction. So, in units of C, one step compares the noise alone, $\mathcal{N}(0, \sigma^2)$, with a mixture of Gaussians shifted by $m = 0, 1, \ldots, k$, each weighted by the probability that m of the user's records were drawn. Charles et al. (2024, Theorem 3) prove that this pair, composed over the steps, bounds the user's privacy loss, and that some loss function, one that aligns the user's gradients, attains it. An accountant built on the pair computes the user's guarantee directly, the same way the subsampling accountant does for one record.

The result has a simple reading while kq is small: **a user with k records looks like one record sampled at rate kq.** Amplification by subsampling works because the unit is usually absent from the batch, and what matters is the probability that some record of the user is present, about kq. Halving the sampling rate exactly compensates doubling the user's records. So the quick estimate is to rerun the record-level accountant with q replaced by kq. It errs on the low side, because it drops the steps in which several of the user's records are drawn together, and those steps shift the sum by several clipping norms at once, a mechanism with noise multiplier σ/m whose tail no amount of rarity can hide.

Table 42.1 applies the accountant to the running example's DP-SGD run, $q = 0.01$, $\sigma = 1$, 10,000 steps, $\delta = 10^{-5}$.

Table 42.1.: The user-level ε at $\delta = 10^{-5}$ of the running example's example-level DP-SGD, for a user with k records. "None" means group privacy gives no $(\varepsilon, 10^{-5})$ guarantee.

Records of the user k	1	2	5	10	50
User accountant	6.72	15.54	52.36	146.20	2161.43
Group privacy	6.71	18.61	118.12	none	none

Records of the user k	1	2	5	10	50
One record at rate kq	6.71	15.32	50.11	134.50	1673.08

Already at two records the person is protected at $\varepsilon = 15.54$, more than twice the record's 6.71; at ten, 146.20 means no protection at all. The direct accountant beats group privacy for every user with more than one record and stays finite where group privacy gives up (Figure 42.1). The rate-kq estimate is within 1.5% at two records and 8% at ten, and below the accountant in every row. One more point matters for an auditor: the accountant is a worst case over gradients, not a pessimistic estimate. Real users' gradients are partly aligned, which is exactly why their records are correlated, and the theorem says some loss attains the bound, so treat it as the guarantee.

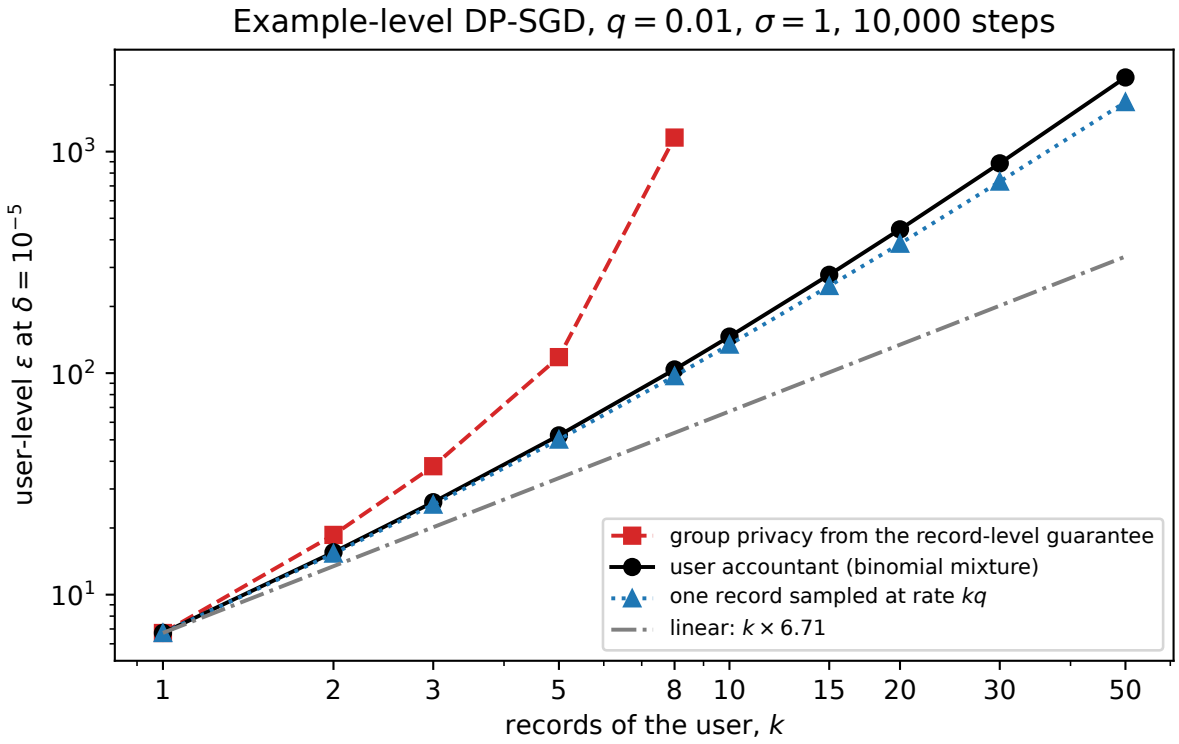


Figure 42.1.: The user-level ε at $\delta = 10^{-5}$ of the running example's DP-SGD ($q = 0.01$, $\sigma = 1$, 10,000 steps; $\varepsilon = 6.71$ per record) for a user with k records. The direct accountant (black) is close to a single record sampled at rate kq (blue, dotted) and grows faster than linearly (grey). Group privacy from the record-level guarantee (red, dashed) is larger, and gives no guarantee at all for $k = 10$ and above.

42.4. User-level DP-SGD

Accounting for users after the fact charges a heavy user for every one of their records. The alternative, due to McMahan, Ramage, Talwar & Zhang (2018), who used it for federated learning of a language model, changes what DP-SGD samples and clips. At each step, put each *user* in

the batch independently with probability q. For each sampled user, compute one vector from that user's records alone, for example their average gradient, or the change in weights after a few local SGD steps on their data. Clip that vector to norm C, sum over the sampled users, add Gaussian noise of standard deviation σC, and divide by the expected number of sampled users, qU for U users.

The guarantee follows because each property the DP-SGD proof used still holds with users in place of records. Adding a user adds one clipped vector to the sum and changes no other, so one user moves the sum by at most C however many records they have. The user is in the batch with probability q, independently of the others, so each step is the same subsampled Gaussian mechanism as before. The result is user-level DP with exactly the accountant of example-level DP-SGD, with q now the user sampling rate.

Two consequences follow, one bad and one good. The bad one: **you pay for the number of units.** There are fewer people than records, so to keep the same expected batch you must sample at a higher rate, and amplification is weaker. On the running example, with 1,000 steps and an expected batch of 100 units, a record-level $\varepsilon = 4$ needs $\sigma = 0.778$ at $q = 0.01$, while a user-level $\varepsilon = 4$ needs $\sigma = 3.78$ at $q = 0.1$. The noise on the averaged update is about five times larger, 0.0189 against 0.00389. That is the honest price of protecting people: there are ten times fewer of them than records, and DP's cost per unit is fixed. The good one: **records inside a user are free.** The guarantee does not depend on how many records a user has, so they can be averaged, used for several local steps or subsampled without changing ε. More records make each user's vector a better estimate of their gradient, so the signal improves while the noise stays put.

A common variant keeps per-record clipping: keep at most k records per user, clip each to C, and sum them, with users still sampled whole. One user then moves the sum by up to kC, and the guarantee is that of noise multiplier σ/k, a factor k in noise that averaging before clipping avoids. Summing all of a user's records and then clipping the sum is also valid, with sensitivity C.

The proof assumes two things that implementations can break. Each user's vector must be computed from that user's data alone, which rules out normalizing across users and requires that one person is not spread over several "users"; if one person has several accounts, the unit is the account and the person is again a group. And users must be Poisson-sampled; iterating over users in a fixed shuffled order is a different mechanism, exactly as for records.

42.5. Contribution bounding

User-level DP-SGD needs a pipeline that groups records by user and computes one vector per user. Many pipelines are built around records instead: shuffled files of examples, per-example gradient code, fixed-length batches. You can keep such a pipeline and still protect users with **contribution bounding**: keep at most k records of each user, chosen without looking at other users' data (for example at random), discard the rest, and run example-level DP-SGD on what remains. Adding a user then adds at most k records, all theirs, so the user accountant of Section 42.3 with k records gives the user-level guarantee. The selection must look only at the user's own records; a rule that kept, say, the records most similar to the rest of the dataset could change which records other users keep when one user is added.

The cap is a bias–variance trade-off (Amin, Kulesza, Muñoz Medina & Vassilvitskii 2019). A larger cap keeps more data but, since the user looks like a record sampled at rate kq, it needs

noise growing roughly in proportion to k. A smaller cap needs less noise but throws away data, all of it from heavy users, so what remains is both smaller and reweighted toward light users. Table 42.2 shows the trade-off at a user-level $\varepsilon = 4$ on the running example, with 1,000 steps and an expected batch of 100 records.

Table 42.2.: Contribution bounding at user-level $\varepsilon = 4$: example-level DP-SGD on at most k records per user, accounted for k records, one run each.

Cap k	Records kept	Noise multiplier σ	Test accuracy
1	1,000 (10.0%)	3.78	0.8917
2	1,807 (18.1%)	4.18	0.8920
5	3,506 (35.1%)	5.34	0.8790
10	5,063 (50.6%)	7.34	0.8276
20	6,631 (66.3%)	11.2	0.7562
50	8,437 (84.4%)	21.8	0.6401

Raising the cap from 1 to 50 multiplies the data by 8.4 and the noise multiplier by 5.8, and accuracy falls from 0.8917 to 0.6401. On this task the noise dominates: one record per user already holds most of what the network can learn privately at this budget, and every extra record costs more in noise than it adds in signal. Caps 1 and 2 are tied. Two cautions follow. The best cap depends on the distribution of user sizes, so it should come from public knowledge of the population or be chosen privately; trying several caps and keeping the best test accuracy, as this table does to show the trade-off, spends privacy that the accountant does not see (Hyperparameter tuning in the full edition). And the two techniques combine: cap records to bound compute, then compute one clipped vector per user, so the cap costs data but no longer noise.

42.6. Heavy users and the objective

User sizes are rarely equal. In the running example's population the mean user has 10 records, the median 4, and the largest 348; the 30 users with at least 50 records hold 30.6% of all records (Figure 42.2).

Example-level training gives each user a weight proportional to their records, so those 30 users carry 30.6% of the expected update. User-level DP-SGD with averaged, clipped vectors gives each user at most $1/U$, because each clipped vector is at most C long and the sum is divided by the number of users; the same 30 users carry at most 3.0%. So user-level training changes the objective, not only the noise: it minimizes roughly the average over users of each user's average loss, not the average over records. Whether that is what you want depends on the application. For a keyboard model, weighting people equally keeps a few very active users from defining everyone's suggestions. For a model meant to fit the overall distribution of records, it under-weights heavy users' content, which matters when heavy users differ from light ones in language, topic or device. Summing a user's gradients before clipping sits in between: it weights records until a user's sum reaches C, and users beyond that.

On the running example the reweighting costs nothing measurable. At user-level $\varepsilon = 4$, user-level DP-SGD reaches 0.9361 on test points whose nearest training record belongs to a heavy user and

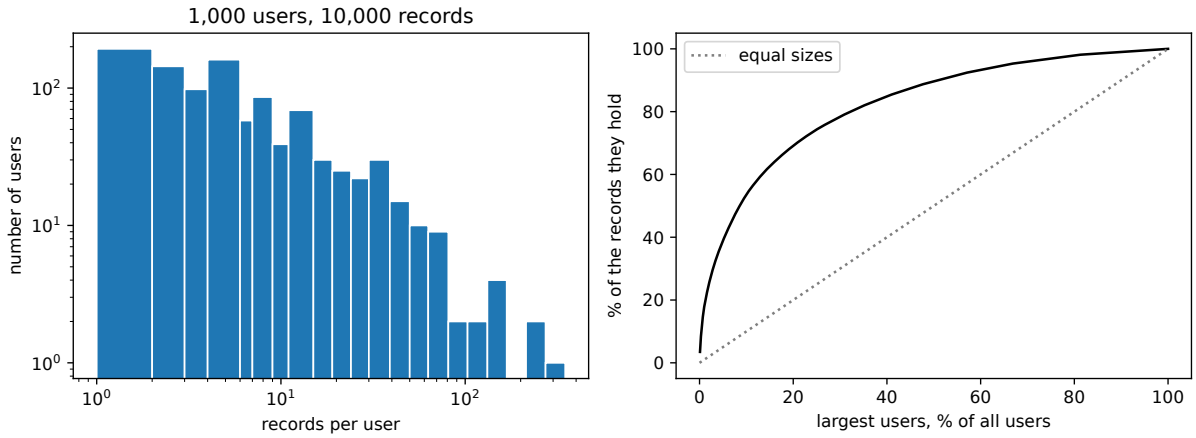


Figure 42.2.: Left: the number of records per user in the running example's population of 1,000 users, on log–log axes; 193 users have one record and the largest has 348. Right: the share of all records held by the largest users. The largest 10 users hold 17.9% of the records and the smallest half of the users 10.3%.

0.9319 on the others, against 0.9607 and 0.9622 for the non-private model. That is because one teacher network labels every record, so light users' records teach the same function. On data where heavy users follow their own patterns, the same reweighting would cost accuracy on those patterns, and clipping adds to it: a heavy user whose data are unusual has a long average gradient, which is clipped to the same C as everyone else's, the user-level version of what clipping does to small groups in Disparate impact in the full edition.

42.7. Choosing a strategy

Three ways to train the running example with a user-level guarantee are now on the table: (a) example-level DP-SGD on all records, accounted for the heaviest user, $k = 348$; (b) user-level DP-SGD; (c) a cap of two records per user, then example-level DP-SGD. Table 42.3 gives each one's noise multiplier and test accuracy at the same user-level budget, with 1,000 steps and an expected batch of 100 units, one run each.

Table 42.3.: Noise multiplier and test accuracy of the three strategies at the same user-level budget, $\delta = 10^{-5}$, one run each. The non-private model reaches 0.9612.

User-level ε	(a) All records: σ	accuracy	(b) User-level DP-SGD: σ	accuracy	(c) Cap of 2: σ	accuracy
1	449	0.5318	12.9	0.8065	14.4	0.7084
2	237	0.5582	6.89	0.9028	7.62	0.8374
4	128	0.5713	3.78	0.9347	4.18	0.8810
8	70.3	0.5478	2.18	0.9442	2.4	0.9333

Keeping the record as the unit fails: strategy (a) stays near chance at every budget up to $\varepsilon = 8$.

The heaviest user has on average $348 \times 0.01 = 3.48$ records in every batch, so subsampling hardly hides them, and the noise must cover a shift of several clipping norms in nearly every step, which takes σ in the hundreds and drowns every gradient. Removing that user would not help much either, since the next-largest has 258 records.

Changing the unit wins. User-level DP-SGD is the best strategy at every budget, 0.9347 at $\varepsilon = 4$ against 0.9612 non-private, with its largest advantage at small budgets: 9.8 points ahead of the cap at $\varepsilon = 1$, 1.1 points at $\varepsilon = 8$. It needs a little less noise than the cap and, unlike the cap, uses every record through the per-user average. Its cost is compute: each step processes all the records of about 100 users, about 1,000 records, ten times what the record-based strategies process. Bounding is the fallback when the pipeline must stay record-based; it comes close at $\varepsilon = 8$, where the discarded data, not the noise, limits accuracy. Single runs vary by about a point at these budgets, so read the large gaps, not the small ones (Figure 42.3).

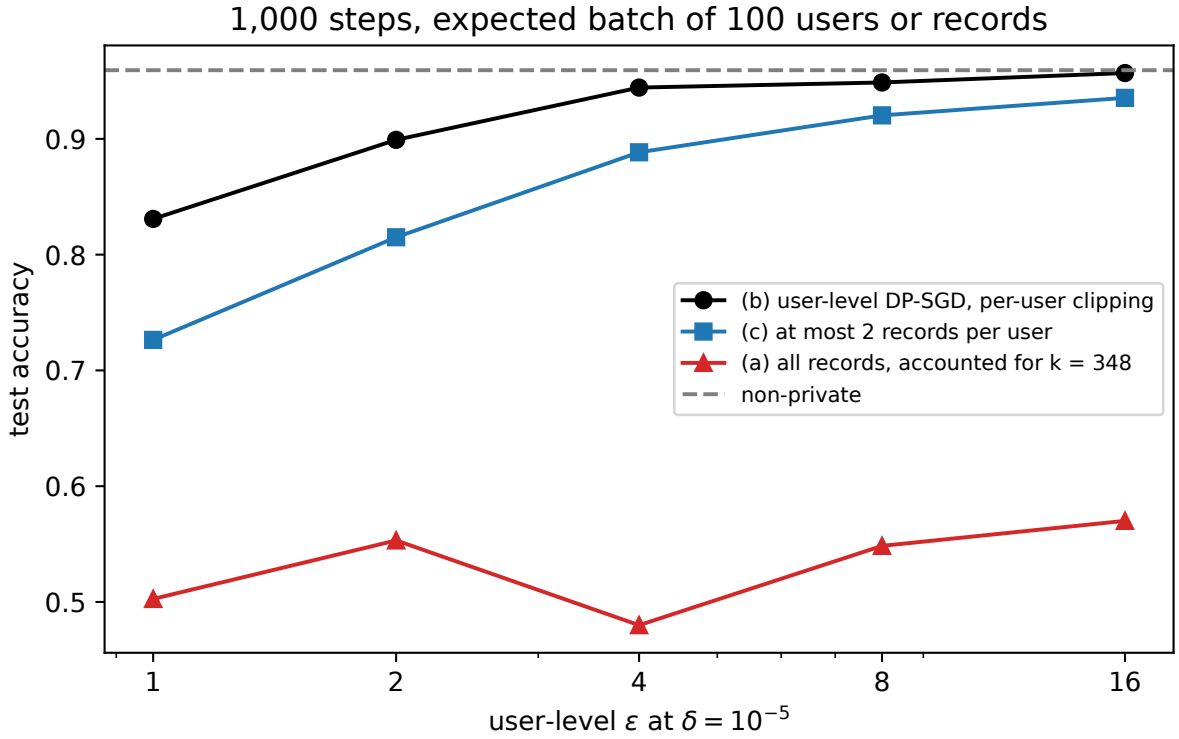


Figure 42.3.: Test accuracy of the running example against the user-level ε at $\delta = 10^{-5}$, for the three strategies (1,000 steps, expected batch 100, one run each; the runs differ from Table 42.3 in their random seeds). User-level DP-SGD is best at every budget; example-level DP-SGD on all records, accounted for its heaviest user, stays near chance.

Published comparisons on language models, with different data, models and accountants, point the same way. Chua et al. (2024) find that user-level DP-SGD with per-user averaging generally beats contribution bounding, especially at small budgets, though they account for the cap with generic group privacy, which costs them more noise than the direct accountant would. Charles et al. (2024) report that sampling users usually does better when each user's data are varied and when strong privacy is required, and that sampling examples sometimes wins.

The price of the unit, on this example, is 2.7 points at $\varepsilon = 4$. The record-level model of DP-SGD in the full edition at the same ε reached 0.9548; the user-level one protects 1,000 people instead of 10,000 records, including the person with 348 records as well as the one with a single record.

42.8. Code

The full edition's script builds the user population, implements the user accountant by numerical integration and checks it against exact formulas, compares it with group privacy, and trains the three strategies with the part's network. It takes about a minute and a half:

```
uv run python code/part05-dp-ml/user_level.py
```

42.9. Cheat sheet

Table 42.4.: The results of this chapter.

Result	Formula or rule	Conditions
Group privacy	$(k\varepsilon,\ \delta(e^{k\varepsilon}-1)/(e^{\varepsilon}-1))$	Any example-level (ε, δ) mechanism; can give no guarantee for DP-SGD beyond a few records
User with k records, example-level DP-SGD	Noise alone against a mixture of shifts $0, \dots, k$ with binomial weights	Tight (Charles et al. 2024, Theorem 3)
Quick estimate	Record-level accountant at rate kq	Small kq; slightly optimistic
User-level DP-SGD	DP-SGD's accountant with users as units, q the user sampling rate	One clipped vector per user, from that user's data alone; users Poisson-sampled
Capped sum per user	Noise multiplier σ/k	Per-record clipping, at most k records, users sampled whole
Contribution bounding	User accountant for k records	Records kept per user without looking at other users; cap chosen from public knowledge
Weight of a user	At most $1/U$ (user-level), n_u/n (example-level)	Averaged, clipped per-user vectors
Running example	6.71 per record; 15.54 for a user with 2 records, 146.20 with 10	$q = 0.01$, $\sigma = 1$, 10,000 steps

Name the privacy unit with every ε; a record-level number says little about a person with many records. Prefer user-level DP-SGD with per-user averaging, which pays for the number of people rather than their records and uses all their data, and decide deliberately how heavy users should count.

42.10. Further reading

- McMahan, Ramage, Talwar & Zhang, "Learning Differentially Private Recurrent Language Models" (ICLR 2018): user-level DP-SGD in federated learning, the algorithm of Section 42.4.
- Charles, Ganesh, McKenna, McMahan, Mitchell, Pillutla & Rush, "Fine-Tuning Large Language Models with User-Level Differential Privacy" (arXiv 2407.07737, 2024): the tight accountant for a user under example-level sampling, and an empirical comparison of sampling examples against sampling users.
- Chua, Ghazi, Huang, Kamath, Kumar, Liu, Manurangsi, Sinha & Zhang, "Mind the Privacy Unit! User-Level Differential Privacy for Language Model Fine-Tuning" (Conference on Language Modeling 2024): contribution bounding against user-wise DP-SGD, with rules for which records to keep.
- Amin, Kulesza, Muñoz Medina & Vassilvitskii, "Bounding User Contributions: A Bias-Variance Trade-off in Differential Privacy" (ICML 2019): how to think about the cap, and how the best one depends on the distribution of user sizes.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

43. Disparate impact

43.1. Why this matters

Training the part's running example with differential privacy (DP) at $\varepsilon = 4$ costs less than one point of test accuracy: 0.9548 against 0.9610 without privacy (private gradient descent in the full edition). That number is an average over the test set, and it says nothing about who loses the point. Bagdasaryan, Poursaeed & Shmatikov (2019) asked, and found that when a network is trained with stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD, the lost accuracy is concentrated on the classes and subgroups that are under-represented in the training data, and existing gaps between groups grow. They called this the **disparate impact** of DP on model accuracy. A model that is "less than a point worse" can be ten points worse for a group of people, and the groups that pay are often the ones privacy is meant to protect: small populations, rare conditions, unusual records.

The one idea: **every record gets the same budget of influence, and every group gets the same noise.** DP-SGD caps each record's pull on the update at the clipping norm C, then adds noise whose size depends only on C and the noise multiplier σ, not on the data. Both rules are blind to groups, and that is exactly why they are not neutral. Clipping cuts back hardest the groups whose gradients are long, which are the groups the model fits badly. And the noise is the same for everyone while a group's signal is proportional to its size, so a small group's signal-to-noise ratio is smaller by its share of the data. Small groups that the model finds hard suffer from both, and the two compound.

43.2. Disparate impact, measured

An accuracy gap between groups in a private model is not enough to blame DP, because the non-private model may have the gap already. What privacy adds is measured by each group's **privacy cost**: the accuracy the group loses when the same training recipe is made private,

$$\pi_g = \mathrm{acc}_g(\text{non-private}) - \mathbb{E}[\mathrm{acc}_g(\text{private})], \tag{43.1}$$

averaged over the randomness of private training (Esipova et al. 2023). DP has disparate impact when groups' privacy costs differ.

The running example shows it. Of its 10,000 training records, 9,500 are the usual ones and 500, a 5% minority, come from a different region of input space, have larger inputs, and follow their own labelling rule, which the majority does not teach. The non-private model learns both: 0.9529 on the majority and 0.9411 on the minority, a gap of 1.2 points, so any large gap in a private model is caused by privacy. Each group is tested on 20,000 fresh records, so test error is small next to the effects, and each row of Table 43.1 averages three training runs.

Table 43.1.: Test accuracy of each group (mean ± standard deviation over 3 runs) of the running example with a 5% minority, trained with DP-SGD at four budgets ($\delta = 10^{-5}$), with clipping alone, and without privacy.

Training	Majority accuracy	Minority accuracy	Privacy cost: majority	minority
Non-private SGD	0.9529 ± 0.0018	0.9411 ± 0.0052	—	—
Clipping only, $\sigma = 0$	0.9581 ± 0.0015	0.9007 ± 0.0048	-0.0051	0.0404
$\varepsilon = 8$	0.9550 ± 0.0006	0.8553 ± 0.0110	-0.0020	0.0858
$\varepsilon = 4$	0.9470 ± 0.0037	0.8477 ± 0.0211	0.0059	0.0934
$\varepsilon = 2$	0.9366 ± 0.0082	0.7574 ± 0.0379	0.0163	0.1836
$\varepsilon = 1$	0.8822 ± 0.0032	0.6133 ± 0.0173	0.0707	0.3278

The minority pays almost all of it. At $\varepsilon = 4$ the majority loses 0.59 points and the minority 9.34, 15.7 times as much. At $\varepsilon = 8$, a budget at which the running example is nearly as accurate as without privacy, the majority loses nothing and the minority 8.58 points. Yet a test set drawn like the training set, 95% majority, would report a loss of about one point at $\varepsilon = 4$; a test set with equal numbers of both groups would report about five. The overall number depends on the test set's mix as much as on the model, which is why it hides the effect.

The table also separates the two causes. The clipping-only row has no noise at all, yet the minority loses 4.04 points while the majority gains 0.51: clipping alone is disparate. Adding noise then widens the gap as ε falls (Figure 43.1). Published experiments show the same shape: Esipova et al. (2023) report, for MNIST digits with class 8 undersampled to about 1% of the training set, a privacy cost of 8.9 points for a typical class and 57.9 points for class 8 at $\varepsilon = 5.90$.

43.3. Clipping reweights the groups

How does a rule that treats every record the same end up treating groups differently? Recall what DP-SGD's expected update is: each record's gradient, multiplied by its **clipping weight** $\min(1, C/\|g_i\|)$, averaged over all records. Group those records. Each group then pulls with its share of the records, s_g, times its average clipped gradient, and since every clipped gradient is at most C long, a group's pull is never longer than $s_g C$, however badly the model fits it. A group of large-gradient records is cut back much more than a group whose gradients are already short.

When a group's gradients point roughly the same way, the effect is a reweighting. Clipping keeps a fraction c_g of the group's average gradient (its average clipping weight, when the group's norms are equal), and the update becomes the gradient of a mixture in which group g has **effective share**

$$\tilde{s}_g = \frac{s_g\, c_g}{\sum_h s_h\, c_h} \tag{43.2}$$

instead of its true share s_g. This splits clipping's damage in two. The denominator shortens the whole update, which a larger step size undoes. The effective shares change *which* mixture of

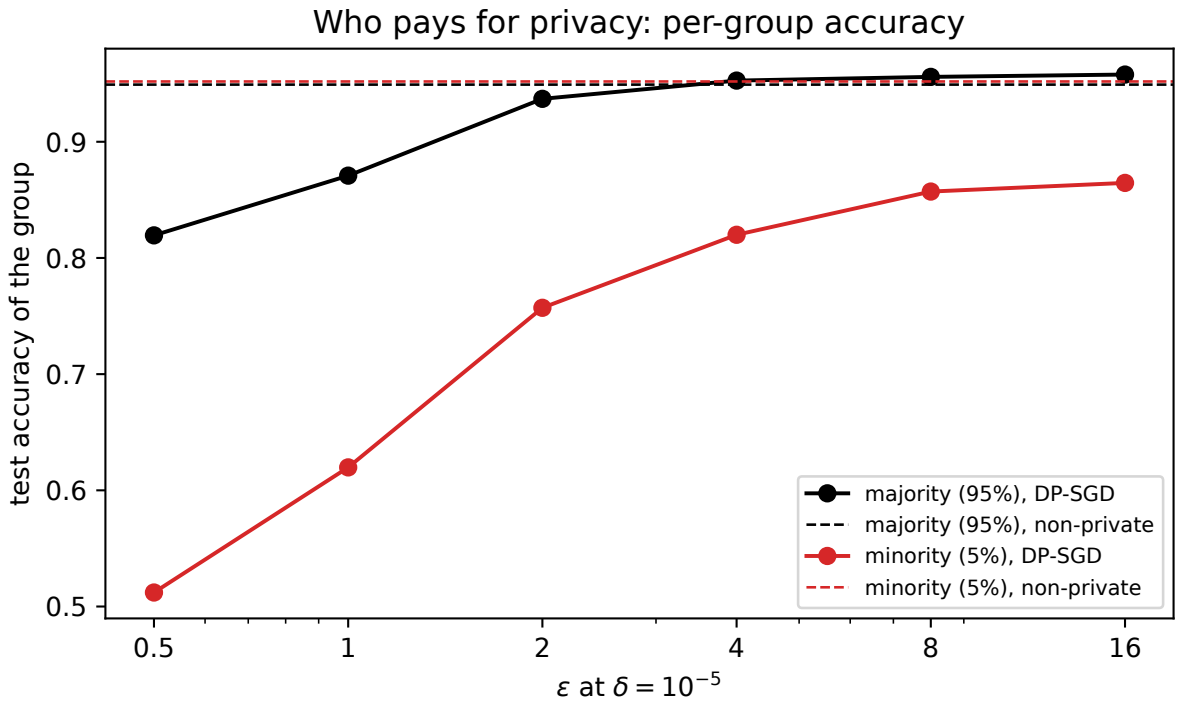


Figure 43.1.: Test accuracy of each group of the running example against ε (log scale, one run per point, $\delta = 10^{-5}$), with each group's non-private accuracy dashed. The majority is close to its non-private accuracy down to $\varepsilon = 2$; the minority falls far below it at every budget.

groups is being descended, and no step size undoes that. Esipova et al. (2023) find this direction error far more harmful than the length error, especially to the under-represented class.

A two-dimensional example makes it concrete. Take 95 majority records with gradient $(0.2, 0)$, already short, and 5 minority records with gradient $(0, 4)$, long because the model fits them badly. Without clipping the average is $(0.19, 0.2)$: the five minority records pull as hard as the ninety-five others. Clip at $C = 0.5$. The majority's gradients are untouched, the minority's shrink eightfold, and the average becomes $(0.19, 0.025)$. The minority's part of the update falls from 0.513 to 0.116, the update turns by 39.0 degrees toward the majority, and the minority's effective share falls from 0.05 to 0.0065. Scaling every gradient by the same factor instead would shorten the update just as much but keep its direction (Figure 43.2).

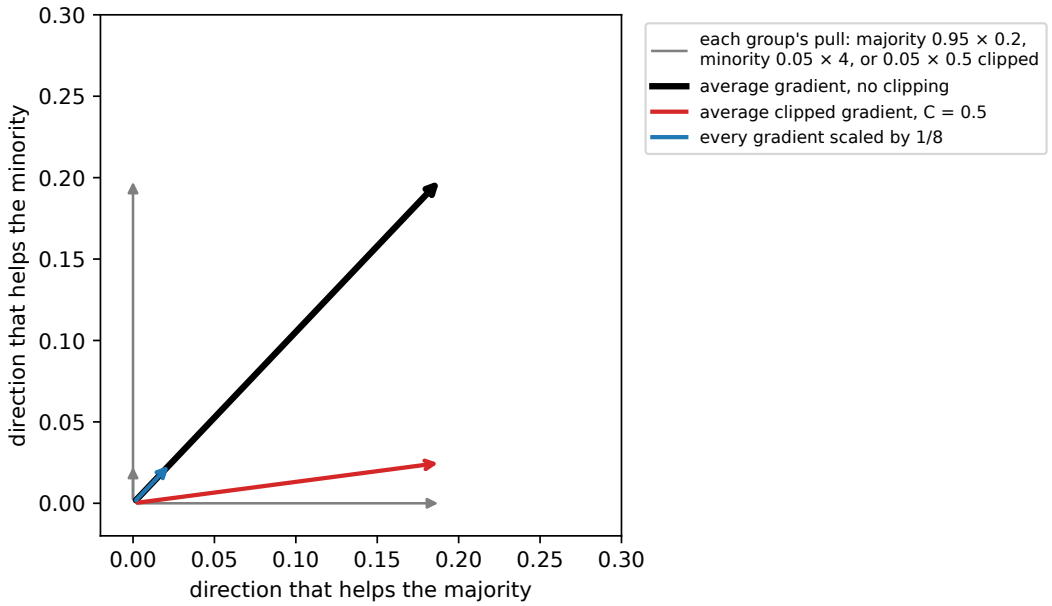


Figure 43.2.: The clipping example in two dimensions: 95 majority records with gradient (0.2, 0) and 5 minority records with gradient (0, 4). Without clipping the two groups pull about equally hard (black); clipping at C = 0.5 cuts the minority's pull eightfold and turns the update toward the majority (red). Scaling every gradient by the same factor 1/8 instead keeps the direction (blue).

Gradients are long for two reasons, and a small group the model has not learned has both. The model fits the group badly, so the loss's derivative is large; and the group's inputs are large, since a dense layer's per-example gradient grows with the length of its input (Tran, Dinh & Fioretto 2021). That sets up a feedback loop during training. At the start every record is clipped and every group has a similar weight. As the majority is learned, its gradients shrink below C and its weight rises toward 1, while the minority, not yet learned, keeps long gradients and a low weight. Its effective share falls below its true share, so it is learned more slowly, so its gradients stay long. In the running example with clipping and no noise, the minority's effective share starts at its true 0.0495, falls to 0.0251 at step 2,500, half its share exactly when it most needs to be learned, and recovers only to 0.0387 by the end. With the noise of $\varepsilon = 4$ it is 0.0261 at step 2,500 and 0.0358 at the end: the noise keeps the minority's gradients long, so clipping keeps cutting them back.

43.4. The noise floor of a small group

The noise is added to the sum, the same for everyone, so why should it hurt a small group more? Because what the noise competes with is each group's signal, and a group's signal in the sum is proportional to its size.

Two arguments make this quantitative. The first is about learning at all. A signal that keeps pointing the same way grows linearly with the number of steps, while independent noise grows only like its square root, so a signal beats the noise after about $(\ell_{\text{noise}}/\ell_{\text{sig}})^2$ steps, the squared ratio of the noise's length to the signal's per step. If the part of the update that serves a group is a fraction s_g of the population's signal, that count is multiplied by $1/s_g^2$. In the running example at $\varepsilon = 4$, the averaged clipped gradient has length about 0.026 and the noise 0.392, so a signal shared by the whole population emerges after 228 steps. A signal that is 20% of it needs 5,693 steps; one that is 5% needs 91,085, nine times the entire run. A group whose rule the rest of the data does not teach is in this position. Running longer does not help, because at a fixed budget more steps mean more noise per step, and the break-even count grows as fast as the run.

The second argument is about where training settles. In a simple model where each group's loss is a quadratic bowl in its own weights, the training pull on a group's weights is proportional to its share, while the noise is not. Noisy gradient descent then settles at an excess loss for the group of roughly

$$\mathbb{E}[L_g] \approx \frac{k\,\eta\,\tau^2}{4\,s_g}, \tag{43.3}$$

where k is the number of the group's weights, η the step size and $\tau = \sigma C/B$ the noise per coordinate on the averaged update, with B the expected batch size. **The floor scales like one over the share**: a group with a twentieth of the data sits at about twenty times the excess loss. The bowl's curvature cancels, because a sharper loss pulls back harder but also charges more for each displacement. In a simulation with the noise of $\varepsilon = 4$, the formula and the simulation agree within 1%, and the floor grows from 4.07×10^{-4} for a group with 95% of the data to 2.96×10^{-2} for a group with 1%, a factor of 73. With clipping, the share is replaced by the clipped share $s_g c_g$, so the two mechanisms compound: a 5% group whose gradients clipping halves pays the floor of a 2.5% group. A network is not a set of bowls, and where a minority's task overlaps the majority's, the majority's gradient does part of its work and the penalty is smaller; the model's value is that it makes the role of the share explicit.

The network bears this out. Table 43.2 varies the minority's share with everything else fixed.

Table 43.2.: The minority's test accuracy for three sizes of the minority, non-private and at $\varepsilon = 4$, and each group's privacy cost (mean ± standard deviation over 2 runs; the 5% row over 3 runs).

Minority share	Records	Minority, non-private	Minority, $\varepsilon = 4$	Privacy cost: majority	minority
1%	100	0.8498 ± 0.0021	0.5009 ± 0.0102	−0.0002	0.3488
5%	500	0.9411 ± 0.0052	0.8477 ± 0.0211	0.0059	0.0934

Minority share	Records	Minority, non-private	Minority, $\varepsilon = 4$	Privacy cost: majority	minority
20%	2,000	0.9787 ± 0.0004	0.9507 ± 0.0038	0.0041	0.0280

The minority's privacy cost falls steeply with its share: 34.88 points at 1%, 9.34 at 5%, 2.80 at 20%, a little less than in proportion to the share, and at 1% the minority is at chance and cannot lose more. The majority's cost stays below one point throughout. At 20% the minority is no longer a minority in any sense that matters to the noise (Figure 43.3).

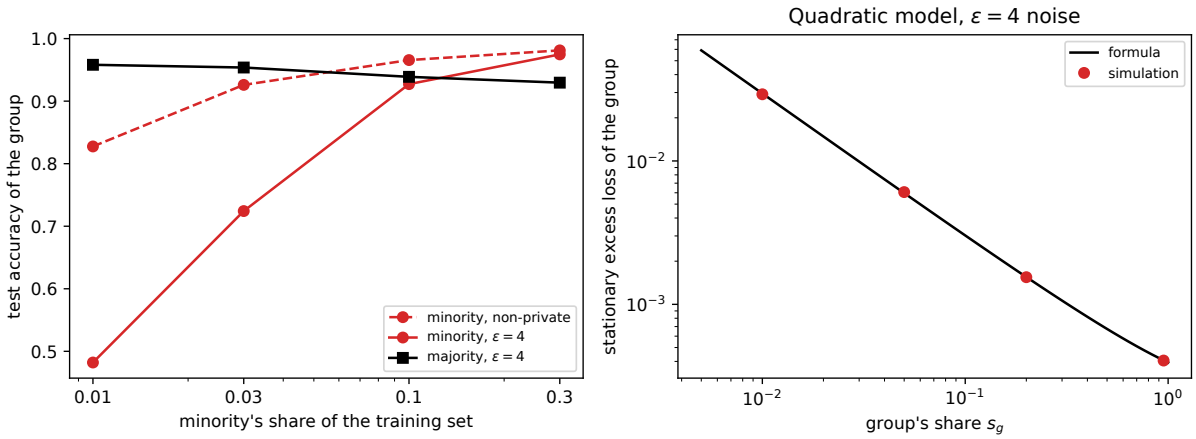


Figure 43.3.: Left: each group's test accuracy of the running example against the minority's share of the 10,000 training records (one run per point), non-private and at $\varepsilon = 4$. Right: the stationary excess loss of a group in the quadratic model against its share, with the noise of $\varepsilon = 4$, compared with simulation.

43.5. Measuring per-group accuracy

Every number so far used 20,000 test records per group. Real evaluations are not so generous: a 2,000-record test split with a 5% minority holds 100 minority records. Each test record is classified right or wrong, so a group's measured accuracy is a binomial proportion, and the **Clopper–Pearson interval** inverts the binomial exactly, covering the true accuracy with probability at least 95% (Hypothesis testing). Its width falls only like one over the square root of the number of records. For the gap between two groups measured on disjoint test records, the two errors are independent and their variances add, so an approximate 95% interval is

$$(\hat{p}_a - \hat{p}_b) \pm 1.96\sqrt{\frac{\hat{p}_a(1-\hat{p}_a)}{m_a} + \frac{\hat{p}_b(1-\hat{p}_b)}{m_b}}, \tag{43.4}$$

with $\hat{p}_g$ the measured accuracies and m_g the numbers of test records.

Take one model trained at $\varepsilon = 4$, whose accuracy on all 20,000 minority test records is 0.8641. On 100 of them it gets 88 right, an estimate of 0.880 with interval $[0.800, 0.936]$, 13.6 points wide. The

non-private model's 0.9411 lies just above that interval, so on this test set alone the minority's privacy cost could be anywhere from about half a point to fourteen. On 1,900 majority records the model gets 0.954, and the gap of 0.074 has the interval 0.010 to 0.138: real, but anywhere from one point to fourteen (Figure 43.4).

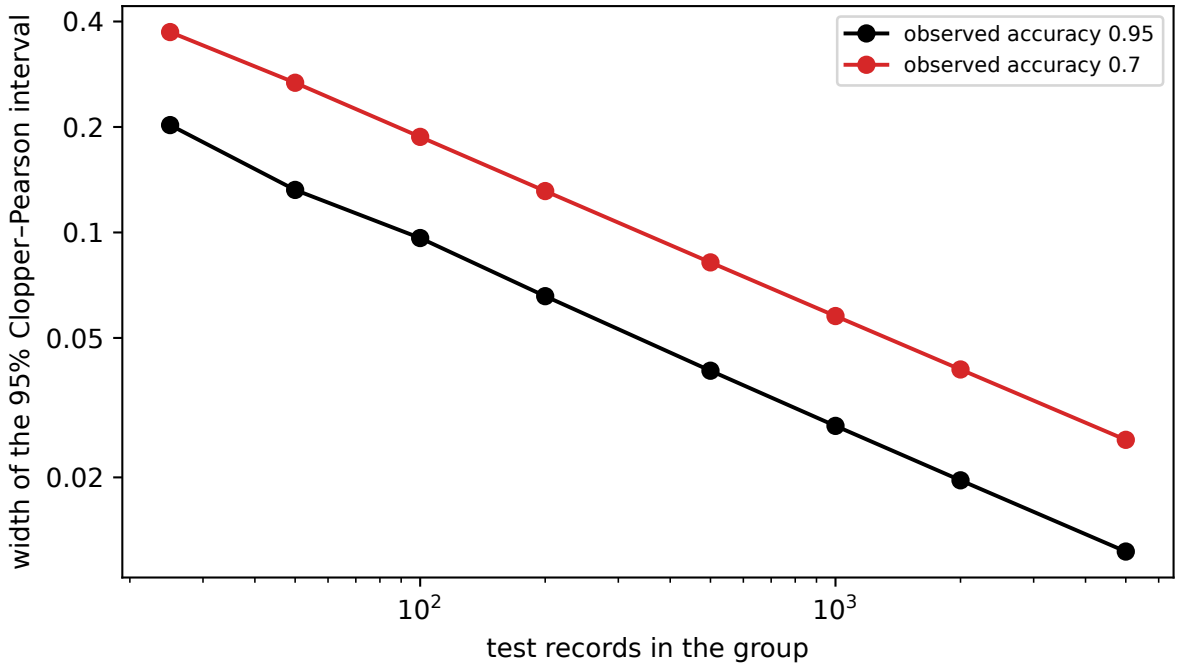


Figure 43.4.: Width of the 95% Clopper–Pearson interval for a group's accuracy against the number of test records in the group, for observed accuracies 0.95 and 0.7, on log scales. The width falls like one over the square root of the number of records.

Training is a second, separate source of variation, and a larger test set does nothing about it: it pins down the model you have, but another run gives another model. In Table 43.1 the run-to-run standard deviation at $\varepsilon = 4$ is 0.0211 for the minority against 0.0037 for the majority, almost six times as large, so a comparison of two recipes on one run each can easily be reversed for the small group. Report each group's interval and the spread over several runs. And per-group accuracies computed on private test data are statistics of that data: if you publish them, they need their own privacy analysis (Choosing ε in the full edition).

43.6. Mitigations and what they cost

Both mechanisms come from DP-SGD's two knobs, clipping and noise. The tempting fixes move the cost around rather than remove it, and it pays to know where to.

Changing the clipping norm trades one mechanism for the other. A smaller C clips more, so the minority's effective share falls. A larger C clips less, so the update points in the right direction, but the noise σC grows with C while short gradients do not, so it is larger relative to their signal. In the limit, scaling every gradient by the same factor C/Z, with no gradient longer than Z, is

exactly DP-SGD with clipping norm Z and a smaller step: the right direction, the most noise. Table 43.3 shows both failures on the running example, keeping ηC fixed.

Table 43.3.: Mitigations at $\varepsilon = 4$ for every record, on the running example with a 5% minority (mean ± standard deviation over 2 or 3 runs).

Change	σ	Majority accuracy	Minority accuracy
None: $C = 0.5$, $\eta = 0.5$	1.36	0.9470 ± 0.0037	0.8477 ± 0.0211
Smaller clipping norm: $C = 0.05$, $\eta = 5$	1.36	0.9541 ± 0.0008	0.8313 ± 0.0157
Larger clipping norm: $C = 5$, $\eta = 0.05$	1.36	0.9463 ± 0.0027	0.8133 ± 0.0166
Oversample the minority four times	4.71	0.8669 ± 0.0079	0.8270 ± 0.0111

Neither clipping norm helps the minority: the smaller one costs it 1.64 points through clipping, the larger one 3.44 points through noise. These differences are of the size of the run-to-run spread, so only their direction is clear. Esipova et al. (2023) propose a variant of global scaling that clips the rare gradients above Z and adapts Z from a privately estimated count, and report that it reduces the privacy cost gap on their datasets; it needs no group labels. Group-aware methods, such as giving more-clipped groups a larger clipping bound (Xu, Du & Wu 2021), need each training record's group, which is itself sensitive, and their per-group statistics must be computed privately. Choosing C from the largest gradient observed in the data is not allowed at all: the clipping norm must be fixed in advance or chosen privately.

Upweighting a group's loss does not work either, because clipping undoes it: the gradients grow and are then rescaled back to C. Oversampling the group does raise its pull, but each record's privacy depends on its own sampling rate, so the minority's records sampled four times as often need $\sigma = 4.71$ instead of 1.36 to stay at $\varepsilon = 4$. The minority's pull grows by 4 and the noise by 3.46, so its signal-to-noise ratio improves by only 1.16, while the majority's falls by 3.46 and its records end up at $\varepsilon = 0.86$, budget wasted. The gap narrows from 9.93 to 3.99 points, but by **levelling down**: the minority's accuracy is 2.07 points lower than without oversampling and the majority's 8.01 lower. A smaller gap is not by itself a fairer model, so report both groups' accuracy and privacy cost, never the gap alone. And oversampling without raising σ would silently give the minority $\varepsilon = 20.96$: the group already disadvantaged in accuracy would get the weakest privacy too.

Some of the cost is unavoidable. Learning a group's own rule privately is partly a private estimation problem on that group's records, and Lower bounds shows that private estimation has a price no algorithm avoids: the error that n_g records allow shrinks only like $1/(n_g\varepsilon)$. Feldman (2020) argues more broadly that fitting a long tail of small subpopulations well requires memorizing their few examples, which is what DP forbids. What helps is more data from the group, a larger ε with its own costs, or a starting point that already serves the group, such as a public pretrained model (Public data and private fine-tuning in the full edition).

43.7. Code

This snippet reproduces the two-dimensional clipping example and the interval for the accuracy gap measured on 1,900 majority and 100 minority test records:

```
from math import acos, degrees, hypot, sqrt


def clipped_mean(groups: list[tuple[float, tuple[float, float]]], c: float) -> tuple[float,
    """Expected DP-SGD update: each group's share times its gradient, clipped to norm c."""
    x = y = 0.0
    for share, (gx, gy) in groups:
        scale = min(1.0, c / hypot(gx, gy))
        x, y = x + share * scale * gx, y + share * scale * gy
    return x, y


def angle(u: tuple[float, float], v: tuple[float, float]) -> float:
    """Angle between two vectors, in degrees."""
    return degrees(acos((u[0] * v[0] + u[1] * v[1]) / (hypot(*u) * hypot(*v))))


def effective_share(share: float, weight: float, other_weight: float) -> float:
    """A group's share of the update after clipping, against one other group."""
    return share * weight / (share * weight + (1 - share) * other_weight)


def gap_interval(pa: float, ma: int, pb: float, mb: int) -> tuple[float, float]:
    """Approximate 95% interval for the accuracy gap pa - pb, measured on disjoint test sets
    se = sqrt(pa * (1 - pa) / ma + pb * (1 - pb) / mb)
    return pa - pb - 1.96 * se, pa - pb + 1.96 * se


groups = [(0.95, (0.2, 0.0)), (0.05, (0.0, 4.0))]
true, clipped = clipped_mean(groups, float("inf")), clipped_mean(groups, 0.5)
print(f"{true[1] / (true[0] + true[1]):.3f} {clipped[1] / (clipped[0] + clipped[1]):.3f}")
print(f"{angle(true, clipped):.1f}")  # 39.0
print(f"{effective_share(0.05, 0.125, 1.0):.4f}")  # 0.0065
print("%.3f %.3f" % gap_interval(0.954, 1_900, 0.880, 100))  # 0.010 0.138
```

The full edition's script trains the running example with a minority group, logs per-group gradient statistics, and checks every number in this chapter:

```
uv run python code/part05-dp-ml/disparate_impact.py
```

43.8. Cheat sheet

Table 43.4.: The results of this chapter.

Result	Formula or rule	Conditions
Privacy cost of a group	$\text{acc}_g(\text{non-private}) - \mathbb{E}[\text{acc}_g(\text{private})]$	Same data and recipe; average over runs
A group's pull	At most $s_g C$	Any gradients
Effective share	$s_g c_g / \sum_h s_h c_h$	Gradients roughly parallel within each group
Break-even steps	$(\ell_{\text{noise}}/\ell_{\text{sig}})^2/s_g^2$	Group's signal a fraction s_g of the population's
Noise floor	$\approx k\eta\tau^2/(4s_g)$, $\tau = \sigma C/B$	Quadratic model, the group's own k weights
Accuracy gap interval	$\hat{p}_a - \hat{p}_b \pm 1.96\sqrt{\sum_g \hat{p}_g(1-\hat{p}_g)/m_g}$	Disjoint test records, large counts
Global scaling	Same as clipping norm Z with a smaller step	No gradient above Z
Oversampling at rate vq	Those records' ε is the accountant's at rate vq	Rate fixed in advance per group

Report accuracy per group, with intervals and the spread over several runs, next to the non-private model's. Expect a group's privacy cost to grow like one over its share, and faster for groups with long gradients. Don't upweight a group's loss, and don't oversample it without accounting at its higher rate. When a group is too small to learn privately, get more data from it or start from a public model that already serves it.

43.9. Further reading

- Bagdasaryan, Poursaeed & Shmatikov, "Differential Privacy Has Disparate Impact on Model Accuracy" (NeurIPS 2019): the observation this chapter explains.
- Esipova, Ghomi, Luo & Cresswell, "Disparate Impact in Differential Privacy from Gradient Misalignment" (ICLR 2023): separates clipping's direction error from its length error, and proposes global scaling with an adaptive threshold.
- Tran, Dinh & Fioretto, "Differentially Private Empirical Risk Minimization under the Fairness Lens" (NeurIPS 2021): a group's excess risk split into clipping and noise terms, and the role of gradient and input norms.
- Feldman, "Does Learning Require Memorization? A Short Tale about a Long Tail" (STOC 2020): why fitting small subpopulations well conflicts with privacy.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

44. Choosing ε

44.1. Why this matters

Every model trained with differential privacy (DP) ships with a budget ε, and someone has to pick it. Stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD, makes the accuracy side of that choice easy to measure. On this part's running example, a small neural network trained on 10,000 records at $\delta = 10^{-5}$, test accuracy is 0.9175 at $\varepsilon = 1$, 0.9441 at $\varepsilon = 2$, 0.9548 at $\varepsilon = 4$ and 0.9604 at $\varepsilon = 8$, against 0.9610 without privacy (DP-SGD in the full edition). That is half of the decision. The other half is what each budget allows an attacker to learn about one person in the data. Without it, ε gets chosen by habit ("everyone uses 8") or by the accuracy curve alone, and neither answer holds up when a reviewer asks what the number promises.

The idea that organizes the chapter is that ε **is one line under a curve.** An attacker deciding whether a record was used runs a test, and the test has a false positive rate and a true positive rate. A pair (ε, δ) draws one straight line that caps those rates. A training algorithm actually has a whole curve, the best true positive rate at every false positive rate, and that curve can lie far below the line. At $\varepsilon = 8$ the line allows a test that catches nearly every member while almost never accusing a non-member. The curve of DP-SGD at the same ε caps the true positive rate at 0.0589 when the false positive rate is 10^{-3}. Choosing ε well means computing the curve, reading it at the error rates your threat cares about, and reporting it.

44.2. What one (ε, δ) pair promises

Start with what "trained with $(8, 10^{-5})$" forbids on its own, which is all you can say about a model whose training algorithm you have not seen. A membership attacker sees the model and says "member" on some set of outputs. Its **true positive rate (TPR)** is how often it says "member" when the target was in the training set, and its **false positive rate (FPR)** is how often it says so when the target was not. Both are probabilities of the same event in the two worlds, and DP bounds exactly that (The definition of differential privacy). Applied to the "member" event it gives the first line below; applied the other way round to the "non-member" event, it gives the second:

$$\mathrm{TPR} \le \min\left\{1,\ e^{\varepsilon}\,\mathrm{FPR} + \delta,\ 1 - e^{-\varepsilon}(1 - \delta - \mathrm{FPR})\right\}. \tag{44.1}$$

Attacks are judged by their TPR at an FPR of 10^{-3} or below (Membership inference), because an attack that flags members while almost never flagging non-members is one that identifies people with confidence. There the first line binds. At $\varepsilon = 1$ it caps the TPR at $e \times 10^{-3} + 10^{-5} = 0.002728$, a real constraint. At $\varepsilon = 4$ the cap is 0.05461. At $\varepsilon = 8$ the line reaches 1 from an FPR of

3.35×10^{-4} on, so the pair says nothing at all at the false positive rates where attacks are usually evaluated.

The **advantage**, TPR minus FPR, is often quoted instead. The pair bounds it by $(e^{\varepsilon} - 1 + 2\delta)/(e^{\varepsilon} + 1)$: 0.4621 at $\varepsilon = 1$ and 0.9640 at $\varepsilon = 4$, numbers that make $\varepsilon = 4$ sound hopeless. But the bound is tight only for the worst mechanism with this ε, randomized response, whose curve is exactly the two lines, and DP-SGD is not that mechanism. Read through the pair alone, $\varepsilon \leq 2$ is strong everywhere, $\varepsilon = 4$ is meaningful only at very low FPR, and $\varepsilon \geq 8$ is no guarantee against membership inference. For DP-SGD, the second and third of these conclusions are far too pessimistic.

44.3. The whole curve of DP-SGD

The accountant that reported ε at $\delta = 10^{-5}$ could just as well have reported it at $\delta = 10^{-6}$ or 10^{-3}. The mechanism has a whole **privacy profile** $\delta(\varepsilon)$, and every point of it is another line that every test must satisfy. The best TPR an attacker can reach at each FPR is the lower envelope of all those lines, the **trade-off curve**. Each line touches the curve at one point, and the line the accountant reports at $\delta = 10^{-5}$ touches it only at a false positive rate below 10^{-4} for every run here. Everywhere else the reported line is a loose summary.

For DP-SGD the curve can be computed. The best test thresholds the **privacy loss**, the log of how much more likely the run's output is with the record than without (the Neyman–Pearson lemma). Each step adds fresh noise, so the losses of the steps add up. The distribution of the total, the **privacy loss distribution (PLD)**, is what a numerical accountant already computes, rounded up and down on a grid so that it brackets the truth (Numerical accounting). Two cheaper answers sit around it. The **Rényi differential privacy (RDP)** curve that set the noise gives a valid but looser bound for free. The **Gaussian differential privacy (GDP)** approximation replaces the run by two unit normals at distance $\mu = q\sqrt{T(e^{1/\sigma^2} - 1)}$, for sampling rate q, noise multiplier σ and T steps (f-DP and Gaussian DP); it is an approximation, not a bound.

Table 44.1 puts the four answers side by side at FPR 10^{-3} for the running example, with $q = 0.01$, $T = 10{,}000$ and the noise multiplier the RDP accountant requires for each target.

Table 44.1.: The largest TPR of a membership test against the running example's DP-SGD at FPR 10^{-3}, $\delta = 10^{-5}$: from the pair, from the RDP curve, exactly from the PLD (optimistic and pessimistic), and from the GDP approximation.

ε	σ	Pair	RDP bound	Exact (PLD bracket)	GDP
0.5	7.72	0.001659	0.00161	[0.001524, 0.001554]	0.001537
1	4.13	0.002728	0.002427	[0.002209, 0.00225]	0.002224
2	2.28	0.007399	0.005033	[0.004267, 0.004342]	0.004273
4	1.36	0.05461	0.01668	[0.01215, 0.01312]	0.01244

ε	σ	Pair	RDP bound	Exact (PLD bracket)	GDP
8	0.917	0.9997	0.09016	[0.05555, 0.0589]	0.05719
16	0.695	1	0.4971	[0.2714, 0.2808]	0.3234

The pair overstates the attacker, and more so the larger ε (Figure 44.1): by a factor of about 1.2 at $\varepsilon = 1$, about 4 at $\varepsilon = 4$ and about 17 at $\varepsilon = 8$. The advantage tells the same story, since the pair allows 0.9640 at $\varepsilon = 4$ and DP-SGD at most 0.3429. At this FPR, GDP lands inside the exact bracket up to $\varepsilon = 8$, but at $\varepsilon = 16$ it reports 0.3234 against an exact value of at most 0.2808. There $\mu = 2.632$, outside the regime where the central limit theorem behind it holds. Use GDP to explain a curve, and the PLD or the RDP bound to certify one.

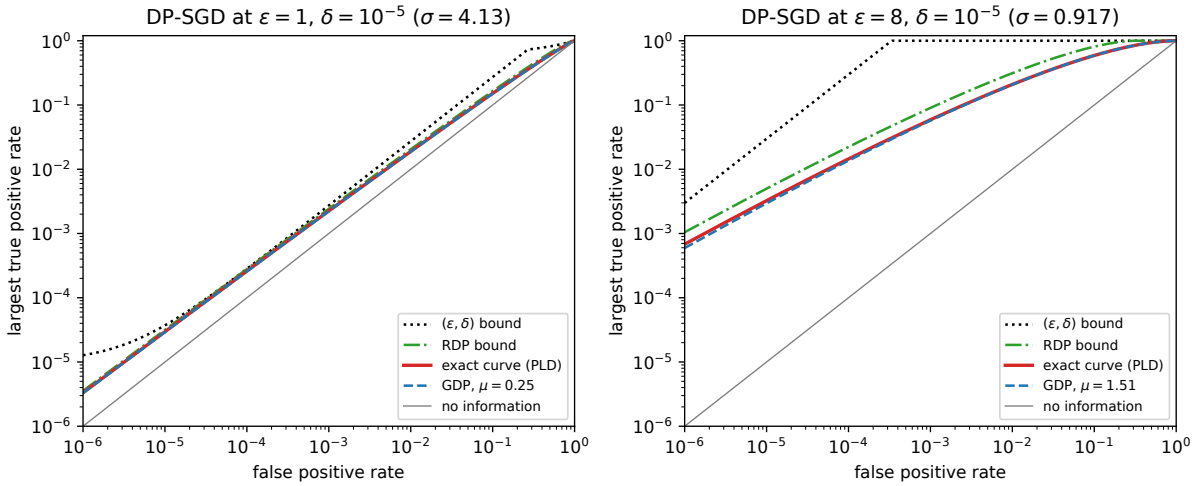


Figure 44.1.: The largest TPR of a membership test against DP-SGD on the running example, on log axes, at $\varepsilon = 1$ (left) and $\varepsilon = 8$ (right), $\delta = 10^{-5}$: the bound from the single pair (dotted), the bound from the whole RDP curve (dash-dotted), the exact curve from the pessimistic PLD (solid red), and the GDP approximation (dashed). The grey diagonal is an attacker with no information.

The curve is the best test of a strong attacker: one who knows every other record and every hyperparameter, sees every intermediate model, and targets a record whose clipped gradient is as large as clipping allows at every step. Nasr et al. (2021) showed that such an attacker, choosing the target's gradient, does reach the accountant's guarantee. A typical record attacked through the final model is much safer. On the running example, a loss-threshold attack has its TPR within 0.004 of its FPR at every budget, nearly at chance even against the non-private model. That describes this attack on this task, not a guarantee. Auditing private training in the full edition builds the attacks that come close to the curve.

44.4. What an attacker can believe

People who are not statisticians ask a different question: if someone sees the model, how sure can they be that I was in the data? Bayes' rule answers it. The attacker's odds that you are a member, after seeing an output, are its odds before times e^L, where L is that output's privacy loss. If no output has loss above ε, a prior belief p can rise to at most

$$\frac{p\,e^{\varepsilon}}{p\,e^{\varepsilon}+1-p}. \tag{44.2}$$

DP bounds the change of belief, not the belief. At $\varepsilon = 1$ an attacker who starts at 1% can reach at most 2.67%. At $\varepsilon = 8$ it can reach 96.79%, so read through this bound, $\varepsilon = 8$ permits near certainty.

That bound treats every output as the worst one. For DP-SGD the loss is a random variable, and the PLD says how it is spread (Figure 44.2). For a worst-case member of the $\varepsilon = 8$ run, the median total loss is 1.136, which moves a 1% prior to 0.0305, and the loss exceeds 5.971, which moves it to 0.7984, only with probability 10^{-3}. At $\varepsilon = 4$ the same two points give 0.0149 and 0.1771. The probability that the loss exceeds the reported ε at all is of the order of δ, between 3.78×10^{-6} and 1.10×10^{-4} across the runs.

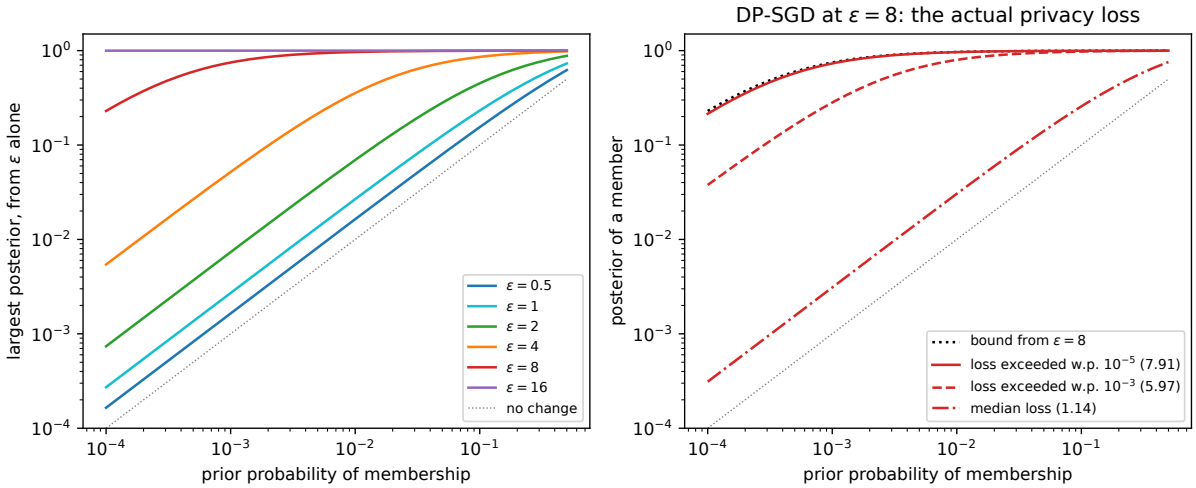


Figure 44.2.: Left: the largest posterior probability of membership against the prior, from Equation 44.2, for the six budgets (log axes). Right: for the $\varepsilon = 8$ run, the bound (dotted) and the posterior of a member whose total privacy loss is the median of the PLD, or is exceeded with probability 10^{-3} or 10^{-5}.

Base rates matter just as much. An attacker who acts on a flag is right with probability $p\,\mathrm{TPR}/(p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR})$, and at a small prior even a large ratio TPR/FPR leaves most flags wrong. At $\varepsilon = 8$, prior 1% and FPR 10^{-2}, the pair allows a precision of 0.5024, while the exact curve allows 0.1744: five flags out of six are false. So say who the attacker is. Against the $\varepsilon = 2$ run, an insider who knows you are one of two candidates (prior 0.5) can reach 0.8808 on the worst output, while a stranger with prior 0.01 reaches at most 0.0695. The guarantee is the same; what it protects against depends on what the attacker knew before.

44.5. Reconstruction

Many harms need more than membership: recovering a diagnosis or a face the attacker does not have (Reconstruction attacks). Give the attacker every other training record and a prior over the target, and let κ be its best success without the model, $1/m$ for one of m equally likely candidates. Any reconstruction attack is also a membership test for its own guess, and averaged over the prior that test's false positive rate is at most κ, because a model trained without the target cannot help guess it. So reconstruction succeeds with probability at most the trade-off curve read at FPR κ (Balle, Cherubin & Hayes 2022; Hayes, Mahloujifar & Balle 2023), or $e^{\varepsilon}\kappa + \delta$ from the pair alone.

A large prior uncertainty is a small FPR, exactly where DP-SGD's curve lies far below the line. At $\varepsilon = 8$ the pair gives no bound for any m up to 1,000, but the curve caps the success of choosing among 1,000 equally likely candidates at 0.0589, against 0.001 by guessing. Even $\varepsilon = 16$ keeps it at most 0.2808. The catch is the prior. If the attacker can narrow the target to two values, as with one unknown binary field in an otherwise known record, $\kappa = 0.5$ and reconstruction is as easy as membership at FPR 0.5. A defensible reconstruction claim names its prior: "an attacker who knows the target is one of m records".

44.6. How small δ must be

A deliberately bad mechanism shows what δ permits: publish each record in full, independently with probability δ. It is $(0, \delta)$-DP, yet it exposes at least one of n records with probability $1 - (1 - \delta)^n$, which is 0.6321 at $\delta = 1/n$ for $n = 10{,}000$. So a meaningful guarantee needs $n\delta \ll 1$, where n counts the privacy units, usually people (User-level differential privacy in the full edition). NIST's guidelines (SP 800-226) write that a common recommendation is $\delta \leq 1/n^2$, that $1/(n \log n)$ is another sensible choice, and that values above 10^{-5} are suspicious and should be justified. The running example's 10^{-5} with 10,000 records has $n\delta = 0.1$.

DP-SGD spends its δ very differently from that mechanism: on outputs whose loss is just above ε, not on outputs that identify someone with certainty, because its loss has a light, nearly Gaussian tail. That is why shrinking δ is cheap (Table 44.2). For the $\varepsilon = 1$ and $\varepsilon = 4$ runs, every factor of 100 in δ costs between about 0.2 and 0.9 in ε. Meeting the $1/n^2 = 10^{-8}$ rule makes the $\varepsilon = 4$ run $(5.209, 10^{-8})$-DP. When in doubt, use a smaller δ.

Table 44.2.: The ε of three DP-SGD runs of the running example at five values of δ, from the RDP accountant.

Run (σ)	$\delta = 10^{-4}$	10^{-5}	10^{-6}	10^{-8}	10^{-10}
$\varepsilon = 1$ (4.13)	0.852	0.999	1.129	1.354	1.548
$\varepsilon = 4$ (1.36)	3.501	3.993	4.432	5.209	5.890
$\varepsilon = 8$ (0.917)	7.088	7.997	8.821	10.301	11.627

44.7. Deployed values and their units

"Industry uses ε between 1 and 10" is a common justification. It hides two things every ε needs: the **privacy unit**, meaning what one neighbouring change is (one record, one user-day, one user forever), and the period over which the budget is spent. Table 44.3 lists documented deployments, and the units differ from row to row.

Table 44.3.: Published differential-privacy parameters of some real deployments, with their privacy units, as stated in the primary sources.

Deployment	Accounting	Privacy unit and period	Published parameters
US Census Bureau, 2020 redistricting data	Zero-concentrated DP	One person's record, replace-one; one release	$\varepsilon = 17.14$ for persons plus 2.47 for housing, at $\delta = 10^{-10}$
Google Gboard language models	Zero-concentrated DP	One device's data, whole training run	ε from 4.49 to 13.69 at $\delta = 10^{-10}$; later models down to 0.994
Apple, local DP on iOS and macOS	Pure ε per donation	One donation; limited donations per day	Emoji 4, one per day; QuickType 8, two per day
LinkedIn Audience Engagements API	Gaussian and Laplace noise on counts	One member, per analyst, per month	$(0.15, 10^{-10})$ per result; $(34.9, 7 \times 10^{-9})$ per month

These numbers are not comparable until they are converted. The Census and Gboard figures come from **zero-concentrated differential privacy (zCDP)**, whose parameter ρ converts to ε by $\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}$ or, a little more tightly, numerically (Concentrated differential privacy). A replace-one guarantee is two add/remove steps. A per-day budget composes over days: QuickType's two donations spend 16 a day. Try calling the running example's $\varepsilon = 8$ run "less than half the Census budget". The Census figure is replace-one at $\delta = 10^{-10}$, while the run is add/remove at 10^{-5}. At $\delta = 10^{-10}$ the run is at 11.627, and replace-one can double an add/remove guarantee. After converting, the two are of the same order, a weaker claim than the slogan. NIST notes that many deployments have used $1 < \varepsilon \leq 20$, which is a fact about practice, not evidence that a value is safe.

44.8. Justifying a choice

A defensible ε fixes the claim before the number. State the unit, the neighbouring relation and the period, counting every training run and hyperparameter search that touched the data (Hyperparameter tuning in the full edition). Name the threat, because an insider reads the curve near its middle and an attacker choosing among a thousand candidates reads it at 10^{-3}. Compute the whole guarantee with a tight accountant and δ well below $1/n$. Add evidence that the code runs the algorithm the proof is about, such as an audit (Auditing private training in the full edition). Only then compare with reference points. Ponomareva et al. (2023) call $\varepsilon \leq 1$ strong

formal guarantees and $\varepsilon \leq 10$ a reasonable level of anonymization for many applications; above 10, the guarantee alone cannot be taken as sufficient evidence and measures such as auditing may be necessary. NIST gives no rule for choosing ε, warns that releases with ε of 10 or more can leave outliers vulnerable, and recommends publishing the parameters.

The last step is utility, read against risk. The accuracy curve is steeply non-linear, so the cost of halving ε varies by an order of magnitude along it (Figure 44.3). Going from $\varepsilon = 1$ to 2 gains 2.66 points of accuracy and moves the exact TPR at FPR 1% from at most 0.01893 to at most 0.03141. From 4 to 8 gains 0.56 points and nearly triples that TPR, to at most 0.2092. Past $\varepsilon = 4$, each point of accuracy is bought with a much larger increase in worst-case risk, and the smallest budget past the bend in accuracy is the natural candidate.

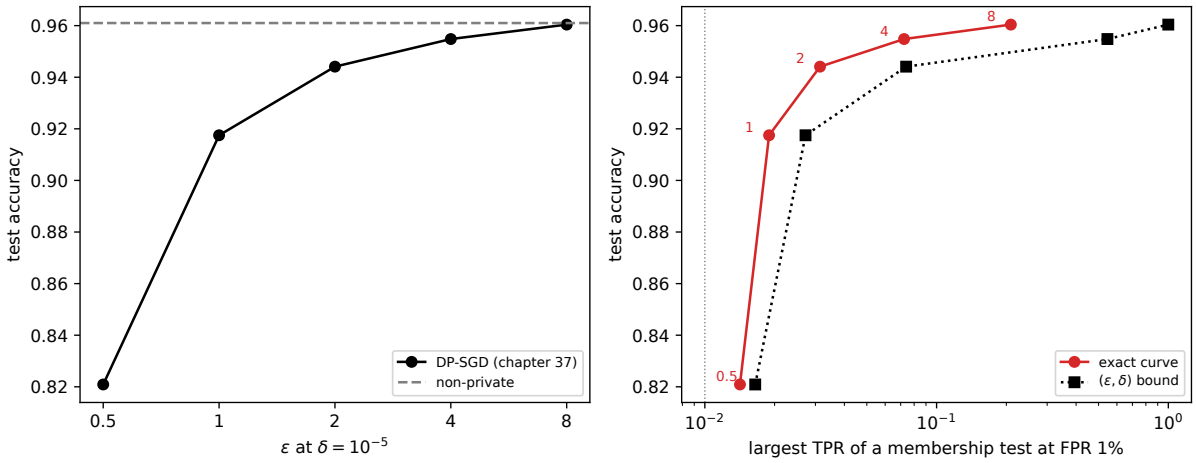


Figure 44.3.: Left: test accuracy of the running example against ε at $\delta = 10^{-5}$, from the DP-SGD chapter's table (three runs each), with the non-private model dashed. Right: the same accuracies against the largest TPR of a membership test at FPR 1%, from the exact curve (circles, labelled by ε) and from the pair alone (squares). The dotted vertical line is an attacker with no information.

The usual tiers make sense in this light. Below 1 the pair's line is already tight enough for any mechanism. Between 1 and 10, DP-SGD's actual curve is far below the line. Above 10 even the actual curve allows substantial attacks on worst-case records, with a TPR above 0.27 at FPR 10^{-3} at $\varepsilon = 16$. Following the procedure, a team that picks the $\varepsilon = 4$ run might write:

> The model was trained with DP-SGD (Poisson sampling rate 0.01, noise multiplier 1.36, clipping norm 0.5, 10,000 steps) on 10,000 records, one record per person. The privacy unit is one record under add/remove neighbours; no other run touched the training data. The RDP accountant gives $(3.993, 10^{-5})$-DP and $(5.209, 10^{-8})$-DP; a PLD accountant gives $\varepsilon \leq 3.729$ at $\delta = 10^{-5}$. Against an attacker who knows every other record and sees every iterate, any membership test has TPR at most 0.01312 at FPR 10^{-3} and at most 0.0724 at FPR 10^{-2}, and any attack that picks a record out of 100 equally likely candidates succeeds with probability at most 0.0724. Test accuracy is 0.9548, against 0.9610 without privacy.

A reviewer who thinks $\varepsilon = 4$ is too large can now argue about the TPR at a specific FPR, which is where the substance is.

44.9. Code

This snippet turns a claimed pair into what it allows, and checks the conversions of this chapter:

```
from math import exp, log, sqrt


def pair_tpr_cap(eps: float, delta: float, fpr: float) -> float:
    """The largest TPR any test can reach at a given FPR, from the pair (eps, delta) alone."
    return min(1.0, exp(eps) * fpr + delta, 1 - exp(-eps) * (1 - delta - fpr))


def posterior_bound(eps: float, prior: float) -> float:
    """The largest posterior probability of membership when the privacy loss is at most eps.
    return prior * exp(eps) / (prior * exp(eps) + 1 - prior)


def flag_precision(prior: float, tpr: float, fpr: float) -> float:
    """How often an attacker acting on a 'member' flag is right."""
    return prior * tpr / (prior * tpr + (1 - prior) * fpr)


def exposure(delta: float, n: int) -> float:
    """Probability that 'publish each record with probability delta' exposes someone."""
    return 1 - (1 - delta) ** n


def zcdp_to_eps(rho: float, delta: float) -> float:
    """The simple zCDP to (eps, delta) conversion of Bun & Steinke."""
    return rho + 2 * sqrt(rho * log(1 / delta))


print(f"{pair_tpr_cap(4, 1e-5, 1e-3):.5f}")  # 0.05461
print(f"{posterior_bound(8, 0.01):.4f}")  # 0.9679
print(f"{flag_precision(0.01, 0.2092, 0.01):.4f}")  # 0.1744
print(f"{exposure(1e-4, 10_000):.4f}")  # 0.6321
print(f"{zcdp_to_eps(2.56, 1e-10):.3f}")  # 17.915
```

The last line converts the Census Bureau's $\rho = 2.56$ for persons; the Bureau's 17.14 comes from a tighter conversion. The exact curve needs a PLD accountant; the full edition's script computes it and every other number in this chapter, in two to three minutes:

```
uv run python code/part05-dp-ml/choosing_epsilon.py
```

44.10. Cheat sheet

Table 44.4.: The results of this chapter.

Result	Formula or value	Notes
One pair	$\text{TPR} \le e^{\varepsilon}\text{FPR} + \delta$; advantage $\le \frac{e^{\varepsilon}-1+2\delta}{e^{\varepsilon}+1}$	Any (ε, δ)-DP mechanism; no constraint once $e^{\varepsilon}\text{FPR} + \delta \ge 1$
Whole curve	Lower envelope of the lines of the privacy profile; traced by thresholding the PLD	Exact for DP-SGD from a numerical accountant
RDP and GDP curves	RDP: valid bound, looser; GDP: $\mu = q\sqrt{T(e^{1/\sigma^2}-1)}$	GDP is an approximation, not a bound
Posterior	$\le pe^{\varepsilon}/(pe^{\varepsilon}+1-p)$	Bounds the change in belief; the typical loss is far below ε
Precision of a flag	$p\,\text{TPR}/(p\,\text{TPR} + (1-p)\text{FPR})$	Base rates dominate at small p
Reconstruction	Success $\le$ the curve at FPR κ; pair: $e^{\varepsilon}\kappa + \delta$	State the attacker's prior
δ	$\delta \ll 1/n$, NIST: $\le 1/n^2$	n counts privacy units; cheap to shrink for DP-SGD

Rules of thumb

- Report the curve, not just the pair: the TPR at FPR 10^{-3} and 10^{-2}, from a PLD accountant or bounded with RDP.
- State the unit, the neighbouring relation, the period and δ with every ε, and convert before comparing with anyone else's.
- $\varepsilon \le 1$ is strong for any mechanism. Between 1 and 10, DP-SGD's curve is your justification. Above 10, show the curve and an audit, and expect worst-case records to be exposed.
- Put the accuracy curve next to the risk curve and pick the smallest budget past the bend.

44.11. Further reading

- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015), and Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS B, 2022): the testing view of DP and trade-off curves.
- Hayes, Mahloujifar & Balle, "Bounding Training Data Reconstruction in DP-SGD" (NeurIPS 2023): the reconstruction bound from DP-SGD's curve, and experiments showing it is nearly tight.
- Near, Darais, Lefkovitz & Howarth, *Guidelines for Evaluating Differential Privacy Guarantees* (NIST SP 800-226, 2025): privacy units, how small δ should be, and the hazards of large ε.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): the tiers of ε for machine learning and what to report.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

45. Auditing private training

45.1. Why this matters

Every chapter of this part ends with a number from an accountant. Take the running example: stochastic gradient descent (SGD) under differential privacy (DP), with per-example clipping and Gaussian noise (DP-SGD), at sampling rate $q = 0.01$, noise multiplier $\sigma = 1$ and 10,000 steps on 10,000 records. The Rényi differential privacy (RDP) accountant reports $\varepsilon = 6.71$ at $\delta = 10^{-5}$. That number is the conclusion of a proof about an algorithm on paper, and what ships is code. An implementation can quietly differ from the algorithm in many places: noise scaled for the averaged gradient but added to the sum, the wrong dataset size handed to the accountant, batches sampled one way and accounted another, a random seed reused. None of these changes the accuracy much, so the usual tests don't catch them. Is the trained model as private as the accountant says?

An **audit** answers by attacking. It plants records of its own, **canaries**, trains with and without them, runs a membership test, and counts how often the test is right. The one idea is that **privacy caps every test, so a measured test bounds privacy from below.** If the test is right too often for any ε below some value, the algorithm cannot be private at that level. A **lower bound** like this can never prove privacy. It can refute a claim when it lands above it, and it measures how much of the accountant's worst case a given attacker actually realizes.

45.2. From an attack to a lower bound

Any membership test says "canary in" on some set of outputs. Its **true positive rate (TPR)** is the probability of saying so when the canary was trained on, and its **false positive rate (FPR)** the probability when it was not. Both are probabilities of one event in two neighbouring worlds, and (ε, δ)-DP says the first is at most e^{ε} times the second plus δ (The definition of differential privacy). Applied to the complementary event it also says that a test cannot be almost always right about absent canaries while rarely missing present ones. Solve both for ε and any test forces

$$\varepsilon \geq \ln \frac{\mathrm{TPR} - \delta}{\mathrm{FPR}} \quad \text{and} \quad \varepsilon \geq \ln \frac{1 - \mathrm{FPR} - \delta}{1 - \mathrm{TPR}}. \tag{45.1}$$

The rates are not known, only estimated from n trials with the canary and n without, each trial being a fresh training run. So the bound must use the pessimistic end of each estimate: an upper bound $\bar{\alpha}$ on the FPR and a lower bound $\underline{\tau}$ on the TPR. The standard choice is the one-sided **Clopper–Pearson** bound, the rate at which seeing as few (or as many) events as you saw would have probability only 2.5%. A true rate beyond it would make your data implausible. With both

bounds at level $\beta/2$ they hold together with probability $1-\beta$, and then the algorithm is not (ε, δ)-DP for any ε below

$$\varepsilon_{\text{lo}} = \max\left\{ \ln \frac{\underline{\tau} - \delta}{\bar{\alpha}},\ \ln \frac{1 - \bar{\alpha} - \delta}{1 - \underline{\tau}} \right\}. \tag{45.2}$$

Confidence is expensive. Suppose 1,000 runs with the canary and 1,000 without give 500 true positives and 1 false positive. The point estimate is $\ln(0.5/0.001) = 6.21$. But one false positive in 1,000 is consistent with a true FPR of 0.00556, five times larger, and the TPR could be as low as 0.4685, so the certified bound is only 4.43. With ten times the trials and the same rates the bound rises to 5.59. The point estimate does not change; the price of confidence falls, slowly.

That slowness has a hard limit. Even a perfect attack, with no false positives and no misses, cannot show an FPR below about $\ln(2/\beta)/n$, because zero events in n trials is consistent with any rate up to roughly that (the **rule of three**, $3/n$ at 95%). Since ε is about $\ln(1/\text{FPR})$ when the TPR is near 1, n trials per hypothesis can certify at most about

$$\varepsilon_{\max}(n) \approx \ln \frac{n}{\ln(2/\beta)}, \tag{45.3}$$

and each extra unit of ε costs a factor of e in trials (Table 45.1). Certifying the running example's 6.71 needs at least 3,038 trials per hypothesis, so 6,076 training runs, even for an attack that never errs. For a real network, each run is a full training job.

Table 45.1.: The most a perfect attack can certify with n trials per hypothesis, at 95% confidence and $\delta = 10^{-5}$.

Trials per hypothesis n	100	1,000	10,000	100,000	1,000,000
Largest certifiable ε	3.28	5.60	7.90	10.21	12.51

Two pitfalls follow from the reasoning. The confidence level assumes the threshold was fixed before counting. Sweeping the threshold over the same trials and reporting the best bound, which is common in parts of the literature, makes the confidence level wrong, because among many thresholds one is lucky. Choose the threshold on half the trials and count on the other half. And try both forms of Equation 45.2: a test that says "in" almost always, with FPR 0.5 and TPR 0.999, gets 0.69 from the first form and 6.21 from the second.

45.3. Canaries and threat models

DP is a promise about the worst pair of datasets and the worst test, so an audit has to build both. A typical record and a loss threshold give a valid but nearly useless bound, because most records are barely exposed (Membership inference). The auditor chooses a canary and a view of training. An **input-space canary** is an ordinary record designed to be exposed, such as a mislabeled example or one far from the data. A **gradient-space canary** lets the auditor write the canary's gradient directly, as if it controlled the training code. The strongest is a **Dirac canary**: a gradient of norm exactly C, the clipping norm, on one coordinate that no other record's gradient touches

(Nasr et al. 2023). In the **black-box** threat model the attacker sees only the final model. In the **white-box** model it sees every iterate, and therefore every noisy gradient sum.

A Dirac canary seen white-box makes the audit face exactly the accountant's pair. On the canary's coordinate, every other record contributes nothing, so each step's noisy sum, divided by C, is pure noise $\mathcal{N}(0, \sigma^2)$ without the canary. With it, the step adds 1 whenever the canary is sampled, with probability q, so the value comes from the mixture $(1-q)\mathcal{N}(0, \sigma^2) + q\,\mathcal{N}(1, \sigma^2)$. The sampling and the noise are fresh at every step, so the T steps are independent. That is the pair the accountant of Amplification by subsampling analyses. The best test, by the Neyman–Pearson lemma, adds up each step's log likelihood ratio,

$$\Lambda = \sum_{t=1}^{T} \ln\left(1 - q + q\, e^{(x_t - 1/2)/\sigma^2}\right), \tag{45.4}$$

where x_t is the step's value on the canary coordinate. No attacker can do better against this canary, since the rest of training carries no evidence beyond the x_t.

The simpler **sum of projections** $\sum_t x_t$, which an attacker who sees only the first and last models can compute, treats every step alike. The likelihood ratio does not: a step at $x_t = 3$ is three noise standard deviations out, which is very unlikely without the canary, so the exponential gives it far more weight than three steps at 1. In Gaussian differential privacy (GDP) terms, where a run behaves like distinguishing two unit normals at distance μ (f-DP and Gaussian DP), Λ has $\mu = q\sqrt{T(e^{1/\sigma^2} - 1)} = 1.3108$ on the running example, and the sum only 0.9951. Read through the Gaussian privacy profile at $\delta = 10^{-5}$, that is the difference between $\varepsilon = 6.01$ and 4.35, settled before a single trial is run.

Black-box audits with input canaries are weaker. The final model is one function of all the noisy sums, so it carries at most as much information as all of them, and usually much less. An input canary's gradient shrinks or turns once the model fits it, and it shares directions with the other records' gradients, which act as extra noise. Nasr et al. (2021) found the lower bound tight for an adversary who sees every iterate and chooses the dataset and canary, and much weaker as the adversary was restricted towards a realistic one.

So the two threat models answer different questions. A white-box gradient audit tests the mechanism, whether the code clips and adds the noise the proof assumes, and its bound should approach the accountant's. A black-box input audit tests the deployment, what an attacker with the released model can do, and its gap to the accountant measures how far this attacker is from the proof's worst case, not a bug.

45.4. Why even the best audit stops short

The white-box Dirac audit with enough trials converges to the true ε of the pair, not to the accountant's number. The RDP accountant's 6.71 is an upper bound. The numerical accountant puts the true value of the running example between 6.14 and 6.24, and the Gaussian approximation gives 6.01. So even an audit with infinitely many trials stops about half a unit below 6.71, and that gap belongs to the accountant.

The main reason real audits fall much further short is where ε lives. The pair (ε, δ) is a line touching the trade-off curve, the best TPR at each FPR, and at $\delta = 10^{-5}$ it touches the Gaussian

approximation of the running example's curve at an FPR of 8.1×10^{-8}, with a TPR of 4.3×10^{-5}. At the rates you can measure, the ratio TPR/FPR is far smaller. For $\mu = 1.3108$, $\ln(\text{TPR}/\text{FPR})$ is 2.74 at FPR 10^{-2}, 3.63 at 10^{-3} and 4.38 at 10^{-4}. Reaching 6 needs FPR near 10^{-7}, and about 1.2×10^{8} runs without the canary to see ten false positives there (Figure 45.1).

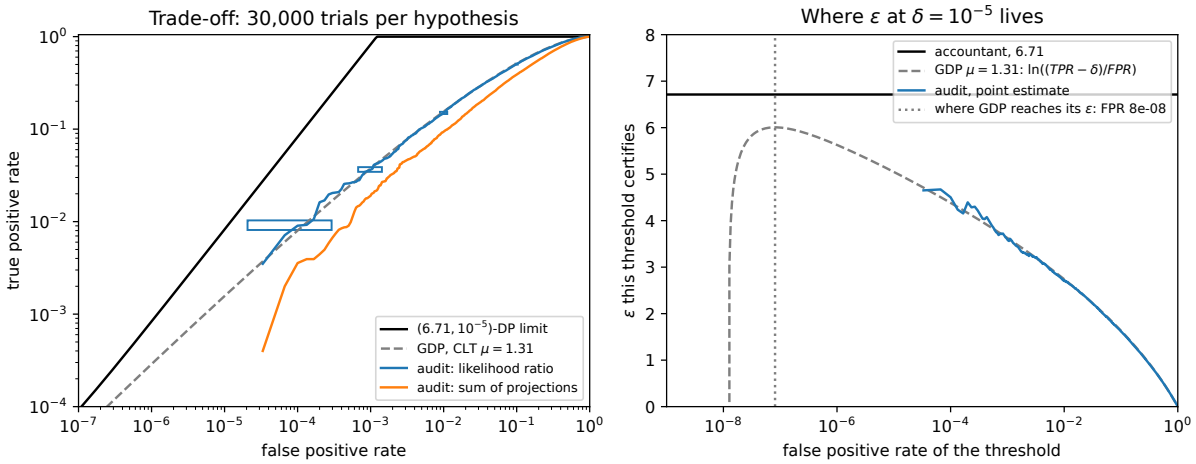


Figure 45.1.: Left: the trade-off of the white-box audit of the running example on log axes, from 30,000 trials per hypothesis. The black line is the most any test can achieve under $(6.71, 10^{-5})$-DP; the dashed grey curve is μ-GDP with the central limit theorem's $\mu = 1.31$. The blue curve is the likelihood-ratio statistic, the orange one the sum of projections; boxes are 95% Clopper–Pearson regions at four thresholds. Right: the ε a threshold would certify if its rates were known exactly, $\ln((\text{TPR} - \delta)/\text{FPR})$, against its false positive rate. The Gaussian curve reaches its ε of 6.01 only at FPR 8×10^{-8} (dotted line), far below the rates a feasible audit can measure.

Nasr et al. (2023) work around the tail by assuming the shape of the curve. If the mechanism is μ-GDP, a single test at moderate FPR bounds μ from below,

$$\mu \geq \Phi^{-1}(\text{TPR}) - \Phi^{-1}(\text{FPR}), \tag{45.5}$$

with Φ the standard normal distribution function, and the Gaussian shape carries the bound into the tail. The bound on μ is rigorous. The conversion to ε is valid only if the curve really is Gaussian. For the subsampled Gaussian of DP-SGD it is not exactly: its tail is heavier, and the μ that Equation 45.5 gives depends on where you read it, from 1.300 at FPR 10^{-1} to 1.492 at 2×10^{-4} in the simulation below.

The full edition's script simulates the white-box Dirac audit from the per-step pair. That is statistically identical to training the network with a Dirac canary, and thousands of times cheaper. Table 45.2 gives the certified bounds, each with the threshold chosen on the first half of the trials.

Table 45.2.: The white-box Dirac audit of the running example with the likelihood-ratio statistic, against the accountant's $\varepsilon = 6.71$. All bounds hold with probability 95%.

Trials per hypothesis	1,000	3,000	10,000	50,000
ε_{lo}, Equation 45.2	1.64	1.88	2.84	3.63
ε from the μ bound (assumes the Gaussian shape)	3.36	4.70	5.13	5.33
Ceiling at half the trials	4.91	6.01	7.21	8.82

The rigorous bound reaches 3.63 with 100,000 simulated training runs, about half the claim. The Gaussian-shape estimate climbs faster, to 5.33, approaching 6.01 from below. Bounds read in the far tail are also noisy. The sum of projections certifies 3.26 with 100,000 trials per hypothesis but 2.27 with 1,000,000 in this run, because a threshold chosen to let through a handful of false positives let through more on the other half. Every number is below the claim, as it must be for a correct implementation.

45.5. One-run auditing

Thousands of training runs are out of the question for a model that takes a day to train. Steinke, Nasr & Jagielski (2023) get many trials from one run. Insert m canaries, each included by an independent fair coin, train once, and score them all. Guess "in" for the $r/2$ highest scores and "out" for the $r/2$ lowest, and abstain on the rest. Under ε-DP, each guess is right with probability at most

$$p_\varepsilon = \frac{e^\varepsilon}{1 + e^\varepsilon}, \tag{45.6}$$

whatever happened to the guesses before it. The reason is Bayes' rule: the canary's inclusion had prior one half, and seeing the trained model can multiply the odds by at most e^ε, since flipping that one coin adds or removes one record. So the number of correct guesses is dominated by a binomial with r trials and success probability p_ε, and observing too many correct guesses rejects ε. The paper's example: 75 correct out of 100, which is what $\varepsilon = \ln 3$ produces on average, certifies 0.702 at 95% confidence.

Under (ε, δ)-DP a correction of order $2m\delta$ is added to the binomial tail, which is small next to $\beta = 0.05$ for 1,000 canaries at $\delta = 10^{-5}$ but can swallow the whole significance budget as m grows.

The coin flips must be independent and fair, because the argument uses the prior of one half. With a deterministic split, guesses could be right for reasons unrelated to training, such as the order of the canaries. And the ceiling is again logarithmic. All r guesses right certifies about $\ln(r/\ln(1/\beta))$: 3.49 for 100 canaries, 5.81 for 1,000 and 8.11 for 10,000. Certifying 6.71 needs

about 2,500 canaries guessed without an error. Each canary is a trial, so one run saves training runs, not trials.

One run is also not tight for DP-SGD. The binomial is calibrated to randomized response, whose trade-off curve has a sharp corner, while DP-SGD's is smooth, and each canary is guessed once, so the audit reads the curve near FPR $1/m$ at best. Simulating 1,000 white-box Dirac canaries in one run of the running example, and paying for trying six list lengths with a Bonferroni correction, the 50 most confident guesses are all right and certify 2.22. That compares with 2.84 from 10,000 runs per hypothesis, at a twenty-thousandth of the cost.

45.6. What a black-box attacker learns

The white-box audit tests the mechanism. To see what the released network itself gives away, take 1,000 test records and flip their labels, so that the model can fit them only by memorizing. Include each with probability one half, train once, and score each canary by minus its loss on the final model. Run the same audit on a model trained with clipping but no noise, and on one without clipping or noise, as controls (Table 45.3).

Table 45.3.: Black-box one-run audit of the running example's network with 1,000 mislabeled canaries, one training run each, at 95% overall confidence and $\delta = 10^{-5}$.

Training	Test accuracy	Mean loss, in	Mean loss, out	Correct of $r = 100$	Best ε_{lo}
DP-SGD, $\sigma = 1$, claim $\varepsilon = 6.71$	0.9645	5.565	6.013	59	0.13
Clipping only, $\sigma = 0$, $\varepsilon = \infty$	0.9638	5.511	6.155	67	0.24
Non-private, no clipping	0.9155	0.023	9.041	91	2.96

The non-private model memorizes: the canaries it trained on have mean loss 0.023 against 9.041 for those it did not see, and the audit certifies 2.96. DP-SGD does not, and its bound is 0.13. The clipping-only model, which has no finite ε at all, is barely easier at 0.24. That control is what makes the result readable. Clipping alone limits each canary's pull on the weights to C per step, against the combined pull of 10,000 correctly labelled records, and that, more than the noise, is what defeats this attacker. So a small black-box bound is a statement about the attacker, not about the model's privacy. Nothing here contradicts 6.71, and nothing confirms it (Figure 45.2).

45.7. Finding bugs

Because an audit bound above the claim refutes it with confidence $1 - \beta$, audits work as a test suite for private training code: silent on a correct implementation, loud on a broken one. The

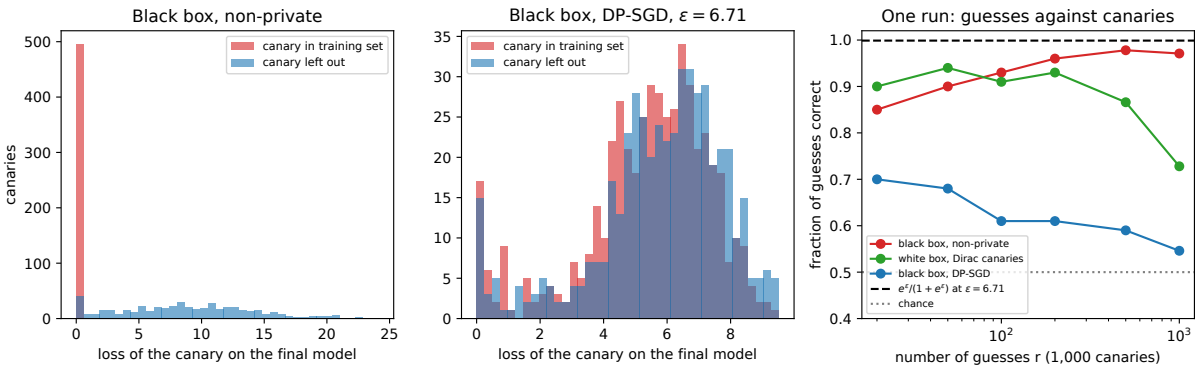


Figure 45.2.: One-run audits with 1,000 canaries. Left and middle: the loss of each mislabeled canary on the final model, for canaries in the training set (red) and left out (blue), without privacy and with DP-SGD at $\varepsilon = 6.71$. Right: the fraction of the r guesses that are correct, against r, for the non-private network (black box), the DP-SGD network (black box) and the mechanism with white-box Dirac canaries; the dashed line is the most a 6.71-DP algorithm allows on average, $e^{\varepsilon}/(1+e^{\varepsilon})$.

cheapest is a **per-step audit**. Put a Dirac canary into every step of one run and leave it out of another, and treat each step's projection as a trial. With the canary always present there is no sampling coin, so each step of a correct implementation is a plain Gaussian mechanism: $\mathcal{N}(0,\sigma^2)$ against $\mathcal{N}(1,\sigma^2)$, exactly μ-GDP with $\mu = 1/\sigma$. Two runs of 10,000 steps give 10,000 trials per hypothesis, and Equation 45.5 with Clopper–Pearson rates gives a rigorous μ_{lo}. If $\mu_{\text{lo}} > 1/\sigma$, the code does not add the noise it claims (Table 45.4, Figure 45.3).

Table 45.4.: Per-step audits of three implementations of the running example's DP-SGD step, from 10,000 steps with and 10,000 without a Dirac canary, at 95% confidence. $B = 100$ is the expected batch size.

Implementation	Noise on the sum	Claimed μ	Audited μ_{lo}	Verdict
Correct	σC	1	0.886	consistent
Variance passed as a standard deviation	$\sigma^2 C^2 = 0.5\,C$	1	1.745	refuted
Noise scaled for the mean, added to the sum	$\sigma C/B = 0.01\,C$	1	6.359	refuted

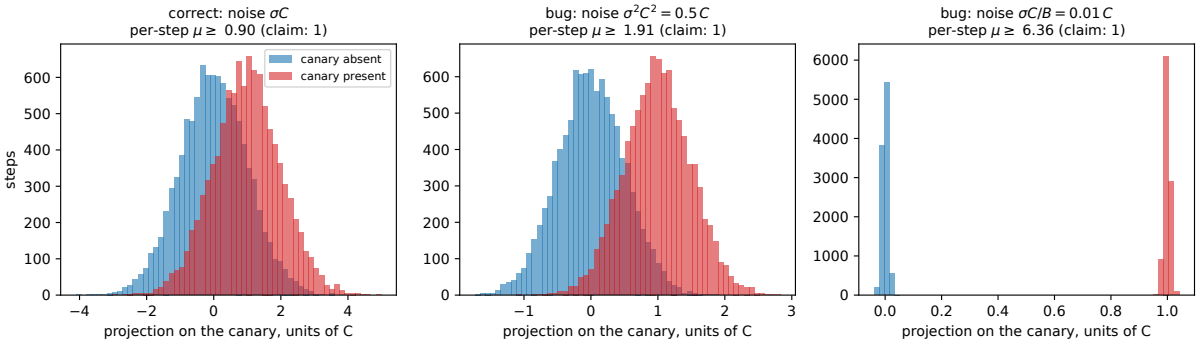


Figure 45.3.: Per-step audits of three implementations: the projections of 10,000 steps on a Dirac canary, without it (blue) and with it at every step (red), in units of C. A correct implementation has noise of standard deviation σC; the two bugs shrink it by a factor of 2 and of 100. The titles give the certified lower bound on the per-step μ; the claim is 1.

The correct implementation passes with room to spare. The two noise bugs fail by margins no bad

dataset has 20,000 records instead of 10,000, as can happen when augmented copies are counted. It computes $q = 0.005$ and reports $\varepsilon = 3.06$. The per-step audit measures the right noise, and composed at the claimed q its bound gives 2.47, no contradiction. The end-to-end audit of Table 45.2 catches it: its 3.63 exceeds 3.06, so the claim is false with 95% confidence.

An end-to-end audit only catches bugs that raise the true ε by more than its shortfall, about three units at 100,000 runs here, on the datasets and records it tried. Batches drawn by shuffling but accounted as Poisson can have an ε several times the reported one, yet a per-step audit never exercises the sampling, and an end-to-end audit sees the gap only where it exceeds the shortfall. And audits catch errors only in the unsafe direction: noise σC added to the mean instead of the sum is B times more noise than claimed, wastes accuracy, and passes every audit. Passing an audit is evidence, never a certificate.

45.8. Code

This snippet computes the bounds of this chapter from counts alone, with Clopper–Pearson intervals from exact binomial sums:

```
from collections.abc import Callable
from math import exp, lgamma, log
from statistics import NormalDist

N = NormalDist()


def binom_cdf(k: int, n: int, p: float) -> float:
    """Pr[Bin(n, p) <= k], summed in logs so that large n is safe."""
    if p <= 0.0:
        return 1.0
    if p >= 1.0:
        return 1.0 if k >= n else 0.0
    log_terms = (lgamma(n + 1) - lgamma(i + 1) - lgamma(n - i + 1) + i * log(p) + (n - i) *
    return min(1.0, sum(exp(t) for t in log_terms))


def bisect(f: Callable[[float], float], target: float, lo: float, hi: float) -> float:
    """The x in [lo, hi] with f(x) = target, for f decreasing in x."""
    for _ in range(100):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if f(mid) > target else (lo, mid)
    return lo


def cp_upper(k: int, n: int, a: float) -> float:
    """One-sided Clopper-Pearson upper bound: the rate at which k or fewer successes has pro
    return bisect(lambda p: binom_cdf(k, n, p), a, 0.0, 1.0)
```

```
def eps_lower_bound(fp: int, tp: int, n: int, delta: float, beta: float = 0.05) -> float:
    """The eps certified with confidence 1 - beta from fp false and tp true positives in n t
    fpr_hi = cp_upper(fp, n, beta / 2)
    tpr_lo = 1 - cp_upper(n - tp, n, beta / 2)
    return max(log((tpr_lo - delta) / fpr_hi), log((1 - fpr_hi - delta) / (1 - tpr_lo)))


def ceiling(n: int, delta: float, beta: float = 0.05) -> float:
    """The most a perfect attack can certify with n trials per hypothesis."""
    r = (beta / 2) ** (1 / n)
    return log((r - delta) / (1 - r))


def one_run_bound(correct: int, guesses: int, beta: float = 0.05) -> float:
    """The eps a one-run audit certifies under pure DP: the largest eps the binomial tail re

    def tail(eps: float) -> float:
        return -(1 - binom_cdf(correct - 1, guesses, exp(eps) / (1 + exp(eps))))

    return bisect(tail, -beta, 0.0, 20.0)


def gdp_eps(mu: float, delta: float) -> float:
    """The eps of mu-GDP at a given delta, from the Gaussian privacy profile."""

    def profile(eps: float) -> float:
        return N.cdf(-eps / mu + mu / 2) - exp(eps) * N.cdf(-eps / mu - mu / 2)

    return bisect(profile, delta, 0.0, 50.0)


print(f"{eps_lower_bound(1, 500, 1_000, 1e-5):.2f}")  # 4.43
print(f"{eps_lower_bound(10, 5_000, 10_000, 1e-5):.2f}")  # 5.59
print(f"{ceiling(1_000, 1e-5):.2f}")  # 5.60
print(f"{ceiling(10_000, 1e-5):.2f}")  # 7.90
print(f"{one_run_bound(75, 100):.3f}")  # 0.702
print(f"{one_run_bound(100, 100):.2f}")  # 3.49
print(f"{gdp_eps(1.3108, 1e-5):.2f}")  # 6.01
print(f"{gdp_eps(0.9951, 1e-5):.2f}")  # 4.35
```

The full edition's script runs every audit in this chapter, including 100,000 simulated white-box trials and three trained networks, in a few minutes:

```
uv run python code/part05-dp-ml/auditing.py
```

45.9. Cheat sheet

Table 45.5.: The results of this chapter.

Result	Formula or value	Conditions
DP caps every test	$\mathrm{TPR} \le e^{\varepsilon}\mathrm{FPR} + \delta$, $1 - \mathrm{FPR} \le e^{\varepsilon}(1 - \mathrm{TPR}) + \delta$	Any test, any neighbours
Certified bound	$\max\{\ln \frac{\tau-\delta}{\bar{\alpha}}, \ln \frac{1-\bar{\alpha}-\delta}{1-\tau}\}$ with Clopper–Pearson bounds	Independent trials; threshold chosen on other trials
Ceiling	$\approx \ln(n/\ln(2/\beta))$: 5.60 at $n = 1{,}000$, 7.90 at 10,000	Perfect attack
Best statistic, Dirac canary	$\Lambda = \sum_t \ln(1-q+qe^{(x_t-1/2)/\sigma^2})$	White box, untouched coordinate
GDP bound from one test	$\mu \ge \Phi^{-1}(\mathrm{TPR}) - \Phi^{-1}(\mathrm{FPR})$	Rigorous for μ; for ε only if the curve is Gaussian
One run	Correct guesses $\preceq \mathrm{Bin}(r, e^{\varepsilon}/(1+e^{\varepsilon}))$, plus a $2m\delta$-type correction	Canaries included by independent fair coins
Per-step audit	$\mathcal{N}(0,\sigma^2)$ against $\mathcal{N}(1,\sigma^2)$; claim $\mu = 1/\sigma$	Tests noise and clipping only

Rules of thumb

- An audit can refute a claim, never prove one. Report its threat model with its number.
- Budget trials before you start: each unit of ε costs a factor of e in trials, and ε at small δ lives at false positive rates you cannot measure.
- To test an implementation, use the strongest attack: white box, Dirac gradient canary, likelihood ratio. Audit the noise per step and the claim end to end.
- Read a small black-box bound as a statement about the attacker, not as the model's privacy.

45.10. Further reading

- Jagielski, Ullman & Oprea, "Auditing Differentially Private Machine Learning: How Private is Private SGD?" (NeurIPS 2020): an early audit of DP-SGD with poisoning canaries and Clopper–Pearson intervals.
- Nasr, Hayes, Steinke, Balle, Tramèr, Jagielski, Carlini & Terzis, "Tight Auditing of Differentially Private Machine Learning" (USENIX Security 2023): Dirac canaries, per-step audits, Gaussian-shape bounds, and real bugs found.
- Steinke, Nasr & Jagielski, "Privacy Auditing with One (1) Training Run" (NeurIPS 2023): the one-run audit, with code for the bound.
- Tramèr, Terzis, Steinke, Song, Jagielski & Carlini, "Debugging Differential Privacy: A Case Study for Privacy Auditing" (arXiv 2022): an audit that refuted the guarantee of a published implementation.

Derivations. Every result in this chapter is derived step by step in the full edition's chapter in the full edition.

A. Glossary

Acronyms used in the book and the pages that use them.

API Application programming interface. Used in Property inference, Choosing ε.

AUC Area under the curve. Used in Hypothesis testing, Membership inference, Private prediction: PATE and in-context learning.

CDF Cumulative distribution function. Used in Probability distributions, Divergences, Hypothesis testing, Membership inference, Discrete and floating-point-safe mechanisms.

CDP Concentrated differential privacy. Used in Concentrated differential privacy.

CLT Central limit theorem. Used in Probability distributions, Concentration inequalities, f-DP and Gaussian differential privacy, Numerical accounting with privacy loss distributions, Choosing an accountant.

CPA Chosen-plaintext attack. Used in Cryptography basics.

DLG Deep Leakage from Gradients. Used in Gradient inversion.

DNA Deoxyribonucleic acid. Used in Lower bounds: what privacy costs.

DP Differential privacy. Used in Threat models and attacks, Differential privacy foundations, The definition of differential privacy, Post-processing, group privacy and composition, The Laplace and Gaussian mechanisms, Advanced and optimal composition, Selection: exponential mechanism, report-noisy-max, sparse vector, Beyond global sensitivity, Discrete and floating-point-safe mechanisms, Local, shuffle and distributed trust models, Lower bounds: what privacy costs, Privacy accounting, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy, f-DP and Gaussian differential privacy, Amplification by subsampling, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Adaptive composition: filters, odometers and individual accounting, Choosing an accountant, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Hyperparameter tuning and private selection, DP-FTRL and matrix factorization, Private prediction: PATE and in-context learning, Public data and private fine-tuning, User-level differential privacy, Disparate impact, Choosing ε, Auditing private training.

DPI Data processing inequality. Used in Divergences.

EMD Earth mover's distance. Used in Syntactic anonymity and how it breaks.

ERM Empirical risk minimization. Used in Optimization, Algorithmic stability and generalization, Private empirical risk minimization, Public data and private fine-tuning.

FFT Fast Fourier transform. Used in Numerical accounting with privacy loss distributions.

FL Federated learning. Used in Gradient inversion.

FPR False positive rate. Used in Hypothesis testing, Threat models and attacks, Threat models and privacy games, Membership inference, Attribute inference and model inversion, Property inference, Memorization and training-data extraction, The definition of differential privacy, Lower bounds: what privacy costs, Choosing ε, Auditing private training.

FTRL Follow the regularized leader. Used in Differential privacy in machine learning, DP-FTRL and matrix factorization.

GD Gradient descent. Used in Amplification by shuffling and by iteration.

GDP Gaussian differential privacy. Used in f-DP and Gaussian differential privacy, Numerical accounting with privacy loss distributions, Choosing an accountant, Choosing ε, Auditing private training.

HIV Human immunodeficiency virus. Used in Syntactic anonymity and how it breaks.

KL Kullback–Leibler. Used in Prerequisites, Divergences, Information theory, Rényi differential privacy.

LLM Large language model. Used in Private prediction: PATE and in-context learning.

LWE Learning with errors. Used in Cryptography basics.

MAP Maximum a posteriori. Used in Information theory, Attribute inference and model inversion.

MGF Moment generating function. Used in Rényi differential privacy.

MIA Membership inference attack. Used in Private prediction: PATE and in-context learning.

ML Machine learning. Used in Choosing an accountant, DP-SGD, Hyperparameter tuning and private selection, Choosing ε.

PATE Private aggregation of teacher ensembles. Used in Beyond global sensitivity, Private prediction: PATE and in-context learning.

PIN Personal identification number. Used in Information theory.

PLD Privacy loss distribution. Used in Numerical accounting with privacy loss distributions, Choosing an accountant, Choosing ε.

PMF Probability mass function. Used in Discrete and floating-point-safe mechanisms.

PRF Pseudorandom function. Used in Cryptography basics.

PRG Pseudorandom generator. Used in Cryptography basics.

PRV Privacy loss random variable. Used in Numerical accounting with privacy loss distributions, Choosing an accountant.

RDP Rényi differential privacy. Used in Privacy accounting, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy, f-DP and Gaussian differential privacy, Amplification by subsampling, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Adaptive composition: filters, odometers and individual accounting, Choosing an accountant, DP-SGD, Hyperparameter tuning and private selection, DP-FTRL and matrix factorization, Private prediction: PATE and in-context learning, Choosing ε, Auditing private training.

ROC Receiver operating characteristic. Used in Hypothesis testing, Threat models and privacy games, Membership inference.

SGD Stochastic gradient descent. Used in Prerequisites, Optimization, Algorithmic stability and generalization, Privacy accounting, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy, f-DP and Gaussian differential privacy, Amplification by subsampling, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Adaptive composition: filters, odometers and individual accounting, Choosing an accountant, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Hyperparameter tuning and private selection, DP-FTRL and matrix factorization, Private prediction: PATE and in-context learning, Public data and private fine-tuning, User-level differential privacy, Disparate impact, Choosing ε, Auditing private training.

TPR True positive rate. Used in Hypothesis testing, Threat models and attacks, Threat models and privacy games, Membership inference, Attribute inference and model inversion, Property inference, The definition of differential privacy, Lower bounds: what privacy costs, Choosing ε, Auditing private training.

TV Total variation. Used in Divergences, Hypothesis testing, Cryptography basics, Threat models

and privacy games, Property inference, Syntactic anonymity and how it breaks.

XOR Exclusive or. Used in Cryptography basics.

ZIP Zone Improvement Plan code, the US postal code. Used in Linkage and re-identification, Syntactic anonymity and how it breaks, Lower bounds: what privacy costs.

B. Index

AboveThreshold, 178, 179, 181
adaptive composition, 222, 274, 291
advanced composition, 166, 170, 172, 173, 228, 260, 284, 286, 291, 295, 301
amplification by iteration, 282, 296
amplification by shuffling, 282, 295
amplification by subsampling, 221, 237, 265, 304
analytic Gaussian mechanism, 161, 162, 164
approximate differential privacy, 212, 213, 218
approximate zCDP, 245
attribute inference, 60, 88
auxiliary knowledge, 62

basic composition, 155, 156, 166, 168, 170, 172, 181, 286
Bernoulli distribution, 17
binomial distribution, 6, 10, 33, 329, 335, 336, 376
black-box attack, 62–64, 95, 189, 402, 406
blatantly non-private, 138, 139

canary, 100, 104, 402, 403, 406
central limit theorem, 251, 253, 254, 271, 274, 277, 293–295, 301, 302, 400
Chernoff bound, 10, 17, 19
clipping, 187, 189, 190, 297, 298, 304, 305, 318, 319, 322–324, 344, 367, 376, 379, 381, 385, 387, 394, 402, 406
Clopper–Pearson interval, 32, 33, 384, 400, 406
collision probability, 54
composition, 128, 130, 131, 141, 149, 155, 156, 166, 168–170, 172, 173, 181, 221, 222, 228, 230, 237, 245, 253, 254, 260, 265, 274, 280, 284, 286, 291, 295, 301, 302, 326, 328, 329, 331, 333, 335, 336, 396
composition attack, 128, 131
conditional entropy, 41

Deep Leakage from Gradients, 109, 112, 113
differential privacy, 1, 60, 113, 141, 149, 156, 164, 173, 181, 189, 190, 198, 208, 218, 221, 230, 237, 245, 254, 265, 274, 291, 301, 302, 304, 315, 324, 336, 349, 367, 377, 387, 393, 396, 406
discrete Gaussian distribution, 195, 197, 198, 209, 237, 245
discrete Laplace distribution, 197
distribution inference, 95
dominating pair, 274
DP-SGD, 221, 222, 230, 236, 245, 251, 253, 254, 259, 260, 264, 265, 271, 272, 274, 282, 291, 293–295, 298, 301, 304, 305, 317, 319, 321, 324, 326, 328, 329, 331, 333, 335, 336, 344–346, 349, 353, 370, 371, 373–377, 379, 389, 390, 392, 394, 396, 402, 403, 406

earth mover's distance, 130
eccentricity, 119, 121, 122
empirical risk minimization, 46, 51, 304, 315, 367, 387
entropy, 35, 38, 39, 41, 112, 116, 117, 130
exponential mechanism, 175, 176, 181, 245, 295, 331, 335
extractable memorization, 104

f-divergence, 26
f-DP, 221, 222, 247, 253, 254
frequency oracle, 208
fully adaptive composition, 291

Gaussian differential privacy, 173, 248, 249, 253, 254, 271, 291, 293–295, 301,

389, 390, 396, 400, 406
Gaussian mechanism, 141, 162–164, 169, 198, 209, 215, 216, 221, 222, 225, 230, 240, 245, 247, 253, 254, 265, 304
generalization gap, 79
global sensitivity, 185
gradient inversion, 1, 60, 113
group privacy, 141, 155, 156, 230, 245, 297, 370, 371, 376

hidden state, 282, 296
hockey-stick divergence, 4, 24
honest-but-curious, 62

imputation baseline, 88
individual privacy accounting, 288, 291

k-anonymity, 130, 131
Kullback–Leibler divergence, 26

l-diversity, 126, 130, 131
label-only attack, 62–64
Laplace distribution, 4
Laplace mechanism, 162–164, 170, 201, 211, 225, 247
learning with errors, 57, 58
likelihood ratio attack, 60, 61, 73–79
Lipschitz, 43, 46, 51
local differential privacy, 208, 393
local randomizer, 208, 295
local sensitivity, 185, 189
loss attack, 73, 78, 79

max-divergence, 22, 24, 26
McDiarmid's inequality, 19
membership advantage, 63, 66, 91, 147
membership inference, 1, 60, 68, 79, 95
meta-classifier, 92, 93, 95
min-entropy, 38, 39, 41
model inversion, 60, 88
moments accountant, 230, 324
Mondrian, 124, 126
mutual information, 41

neighbouring datasets, 142, 149, 158, 159, 163, 164, 183, 257, 265, 297, 298, 396
Neyman–Pearson lemma, 33
normal distribution, 141

objective perturbation, 309, 312, 314, 315
onion effect, 79
optimal composition, 141, 166, 168–170, 172
optimistic discretization, 298
output perturbation, 312, 314

packing lower bound, 211, 213
parallel composition, 155, 156
permutation invariance, 95
pessimistic discretization, 298
Pinsker's inequality, 24, 26
Poisson distribution, 6, 10, 222, 242, 254, 257, 258, 262, 264, 265, 274, 295, 297, 298, 301, 302, 316, 323, 326, 328–330, 332, 335, 336, 344–346, 349, 359, 376, 394
Poisson sampling, 222, 258, 265, 274, 295, 297, 298, 302, 316, 344, 346, 349, 359, 394
post-processing, 141, 155, 156, 245
privacy game, 60, 68
privacy loss, 141, 153, 159, 166, 172, 179, 183, 195, 216, 221, 236, 237, 245, 264, 270, 274, 282, 298, 309, 326, 391
privacy loss distribution, 236, 245, 264, 267–269, 271, 272, 274, 293–295, 298, 301, 389–391, 394, 396
privacy odometer, 156, 173, 291
privacy profile, 151, 162, 164, 170, 173, 249, 298, 396
property inference, 1, 60, 95
propose-test-release, 185, 189
PRV accountant, 274, 302
pure differential privacy, 145, 162, 172, 189, 197, 212, 213, 218, 225, 229, 240, 245, 257, 282, 290, 295, 297, 335, 351

quantitative information flow, 41
quasi-identifier, 115, 124

randomized response, 146, 149, 151, 153, 166, 170, 172, 201–203, 208, 224, 225, 230, 240, 247, 269, 276, 277, 282, 284, 295, 326, 351
RDP curve, 222, 224, 225, 228, 234, 237, 240, 242, 245, 262, 264, 265, 351, 389, 390

re-identification, 1, 61, 118, 122
reconstruction attack, 61, 139
report-noisy-max, 181, 218
ROC curve, 30, 33, 66, 68, 75, 77, 79
Rényi differential privacy, 221, 222, 224, 225, 228–230, 234, 236, 237, 240, 242, 245, 253, 258–260, 262, 264, 265, 271, 274, 282, 290, 293–295, 298, 301, 317, 323, 344, 351, 355, 359, 389, 390, 392, 394, 396
Rényi divergence, 22, 26, 280, 282
Rényi filter, 290, 291

sampling without replacement, 258, 264, 298, 302
secure aggregation, 113, 198, 209
sensitivity, 141, 142, 161, 163, 164, 169, 185, 189, 195, 197, 224, 225, 229, 230, 304, 314, 343, 349
shadow model, 61, 72–74, 76, 77, 79, 95
shifted Rényi divergence, 282
shuffle model, 142, 208, 295
simulation paradigm, 58
skewness attack, 128
smooth sensitivity, 185, 189
snapping mechanism, 198
sparse vector technique, 178, 181
stochastic gradient descent, 3, 43, 44, 46, 49, 51, 221, 222, 230, 236, 245, 251, 253, 254, 259, 260, 264, 265, 271, 272, 274, 282, 291, 293–295, 298, 301, 304, 305, 317, 319, 321, 324, 326, 328, 329, 331, 333, 335, 336, 344–346, 349, 353, 370, 371, 373–377, 379, 389, 390, 392, 394, 396, 402, 403, 406
strong convexity, 46, 282, 314, 315
sub-Gaussian, 19
subsample-and-aggregate, 186, 189
subsampled Gaussian mechanism, 242, 245, 257, 262, 264, 265, 274, 295, 323, 359
subset-sum query, 138

t-closeness, 126, 130, 131
threat model, 1, 60, 61, 68, 141, 406
total variation distance, 4, 24, 126, 128
tracing attack, 139, 215, 218
trade-off function, 4, 247, 253, 396
training-data extraction, 60
truncated concentrated differential privacy, 242, 245
type I error, 247, 253
type II error, 247, 253

unary encoding, 202, 208
uniform stability, 51
union bound, 54, 211
unit of privacy, 142, 149, 298, 339, 343, 370, 373–377

variance, 3, 7, 10, 18, 19, 22, 24, 46, 73, 74, 79, 190, 197, 202, 208, 218, 377, 403
vulnerability, 41

white-box attack, 62–64, 400, 401, 403

zero-concentrated differential privacy, 240, 242, 245, 295